

Projeto de Software Orientado a Objetos

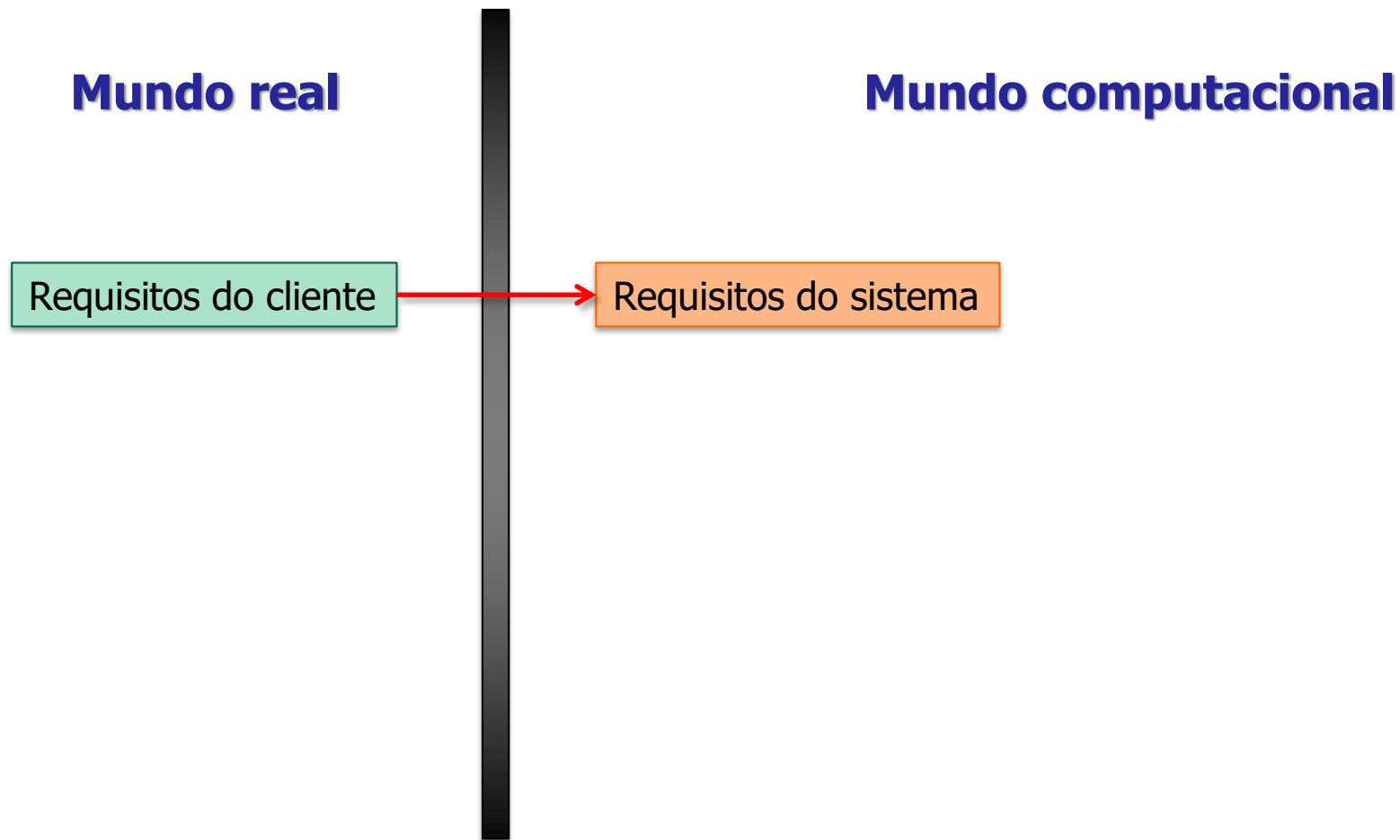
1ª parte

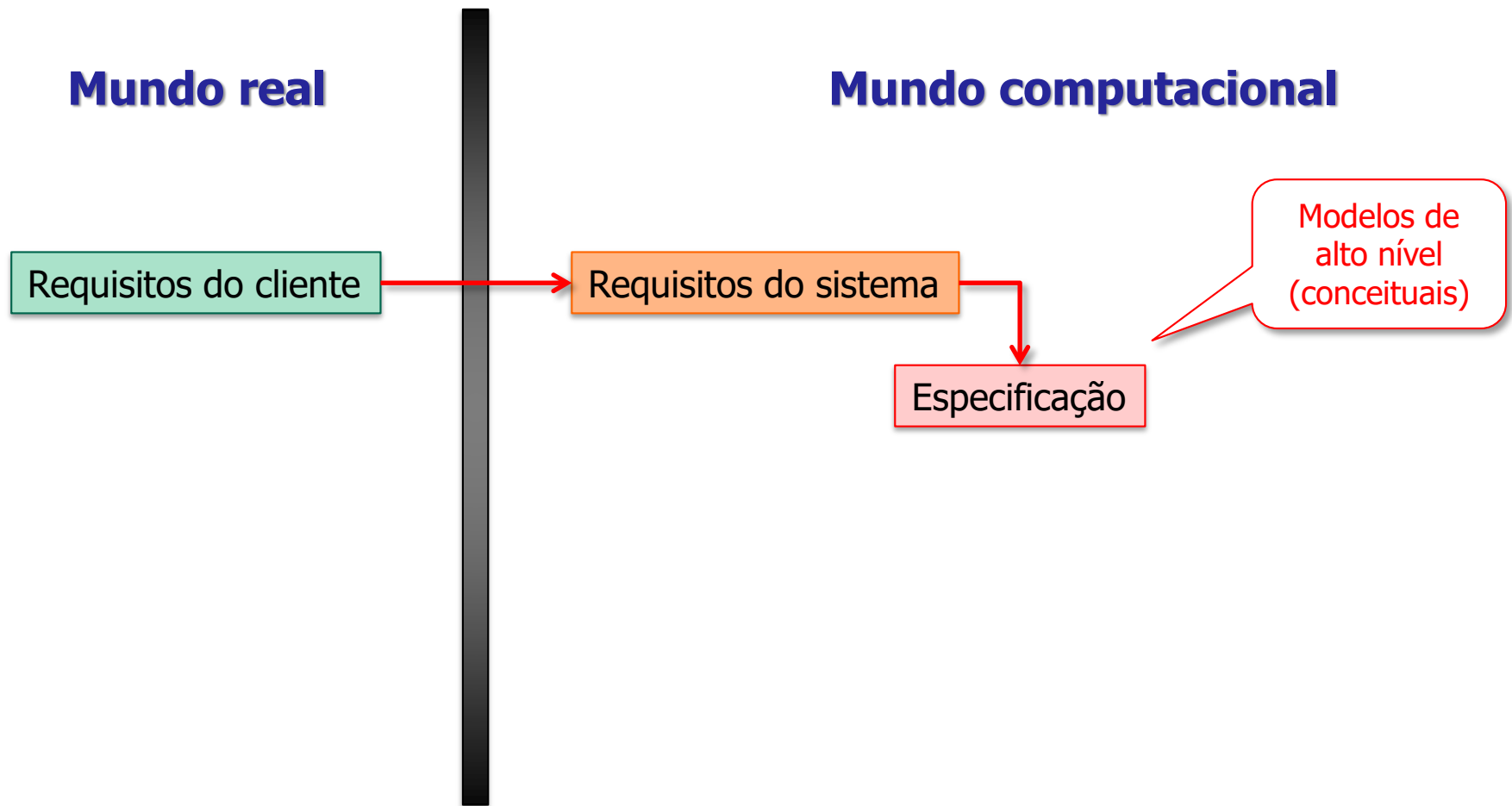
Prof. Jobson Massollar
jobson.silva@uva.br

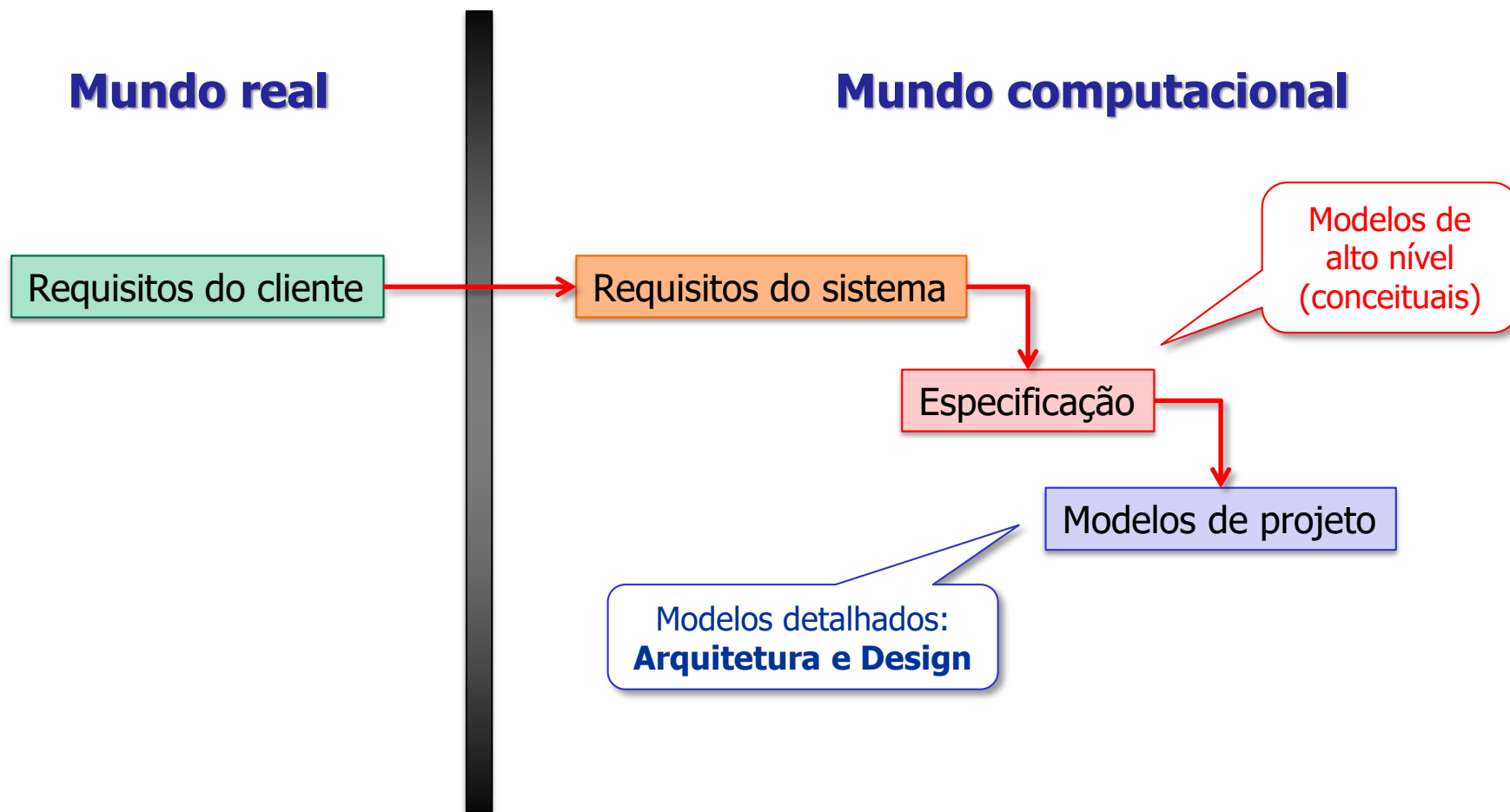
Mundo real

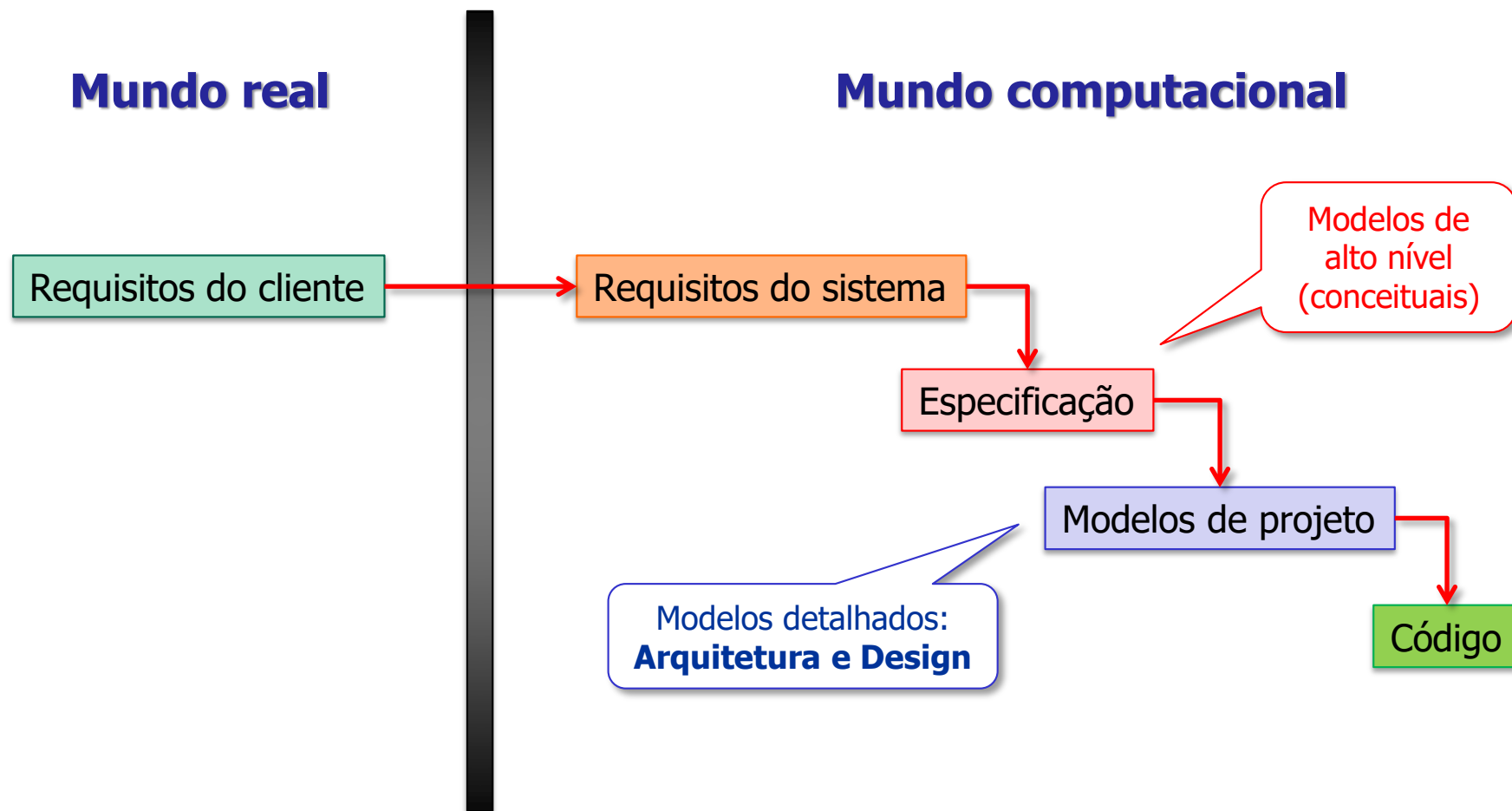
Requisitos do cliente

Mundo computacional









- Por que essas fases são importantes?
 - ✓ A construção de sistemas computacionais complexos exige a implementação de dezenas, centenas ou milhares de componentes.

- Por que essas fases são importantes?
 - ✓ A construção de sistemas computacionais complexos exige a implementação de dezenas, centenas ou milhares de componentes.
- Como organizar esses componentes de forma que possamos:
 - ✓ Atender os requisitos funcionais e não funcionais definidos na especificação?
 - ✓ Reusar componentes de forma a não implementar o mesmo serviço mais de uma vez?
 - ✓ Facilitar a manutenção do sistema (troca de componentes, mudança de regras, etc.)?
 - ✓ Trazer portabilidade para o sistema (execução em plataformas de hardware/software distintas)?
 - ✓ ...

- Nesse contexto, a elaboração de uma solução computacional passa pela compreensão e aplicação de conhecimentos relacionados a:
 - ✓ Estilos Arquiteturais
 - ✓ Princípios de Projeto
 - ✓ Padrões de Projeto
- O domínio desses conceitos e sua aplicação desempenha um papel fundamental no gerenciamento da complexidade inerente ao software a ser desenvolvido.

- Antes de mais nada é preciso entender:

Qual a diferença entre *design* e arquitetura ?

- Existe uma grande diferença entre esses dois conceitos, embora alguns autores chamem de Arquitetura aquilo que é realizado durante o Design.

➤ Arquitetura de Software

- ✓ A arquitetura de um software define, em termos computacionais, quais são seus **elementos arquiteturais** e como ocorre a **interação** entre eles.

(FIELDING, 2000)

- ✓ Uma arquitetura de software envolve a descrição de **elementos arquiteturais** a partir dos quais os sistemas serão construídos, **interações** entre esses elementos, **padrões** que guiam suas composições e **restrições** sobre estes padrões.

(PFLEEGER, 1998)

➤ Design de Software

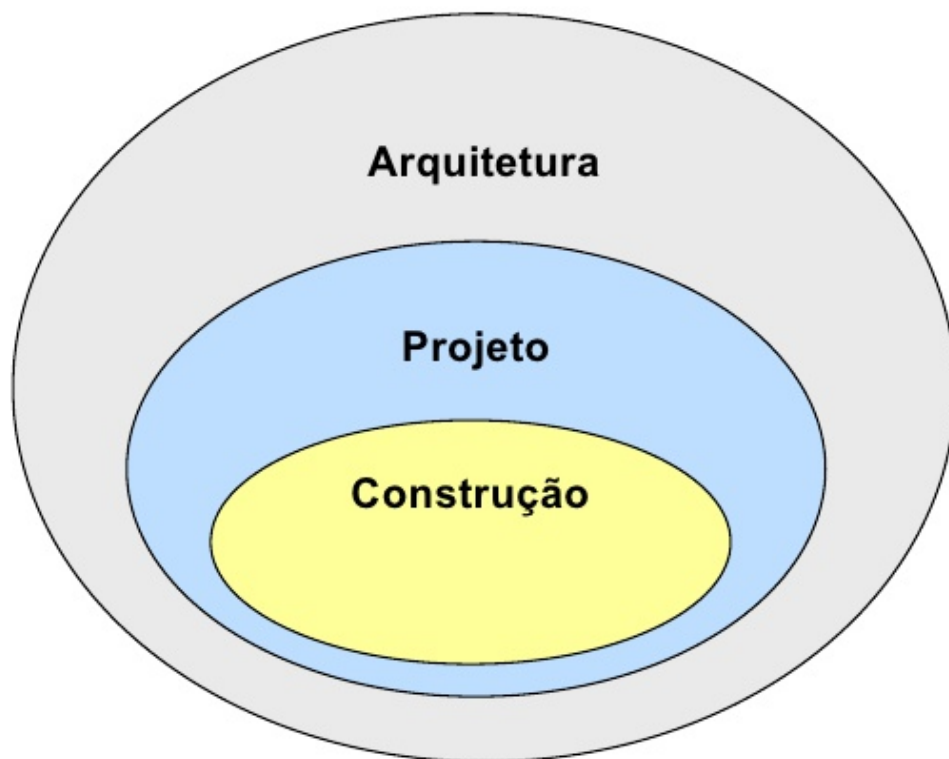
- ✓ Após a definição do propósito e da especificação do software os desenvolvedores devem criar um plano detalhado de solução.
- ✓ Assim, o design é um processo de resolução de problemas e planejamento **detalhado** de uma solução de software.
- ✓ Esse plano inclui a definição de componentes de **mais baixo nível**, incluindo questões relacionadas à implementação dos algoritmos
- ✓ Esse plano deve estar **aderente** ao estilo arquitetural.

➤ Arquitetura de Software

- ✓ Fornece uma visão mais alto nível do sistema.
- ✓ Estabelece uma metáfora para organização dos diversos componentes do software, ou seja, para o design.
- ✓ Decisões arquiteturais são, via de regra, mais difíceis de serem alteradas depois que o software é construído.

➤ Design de Software

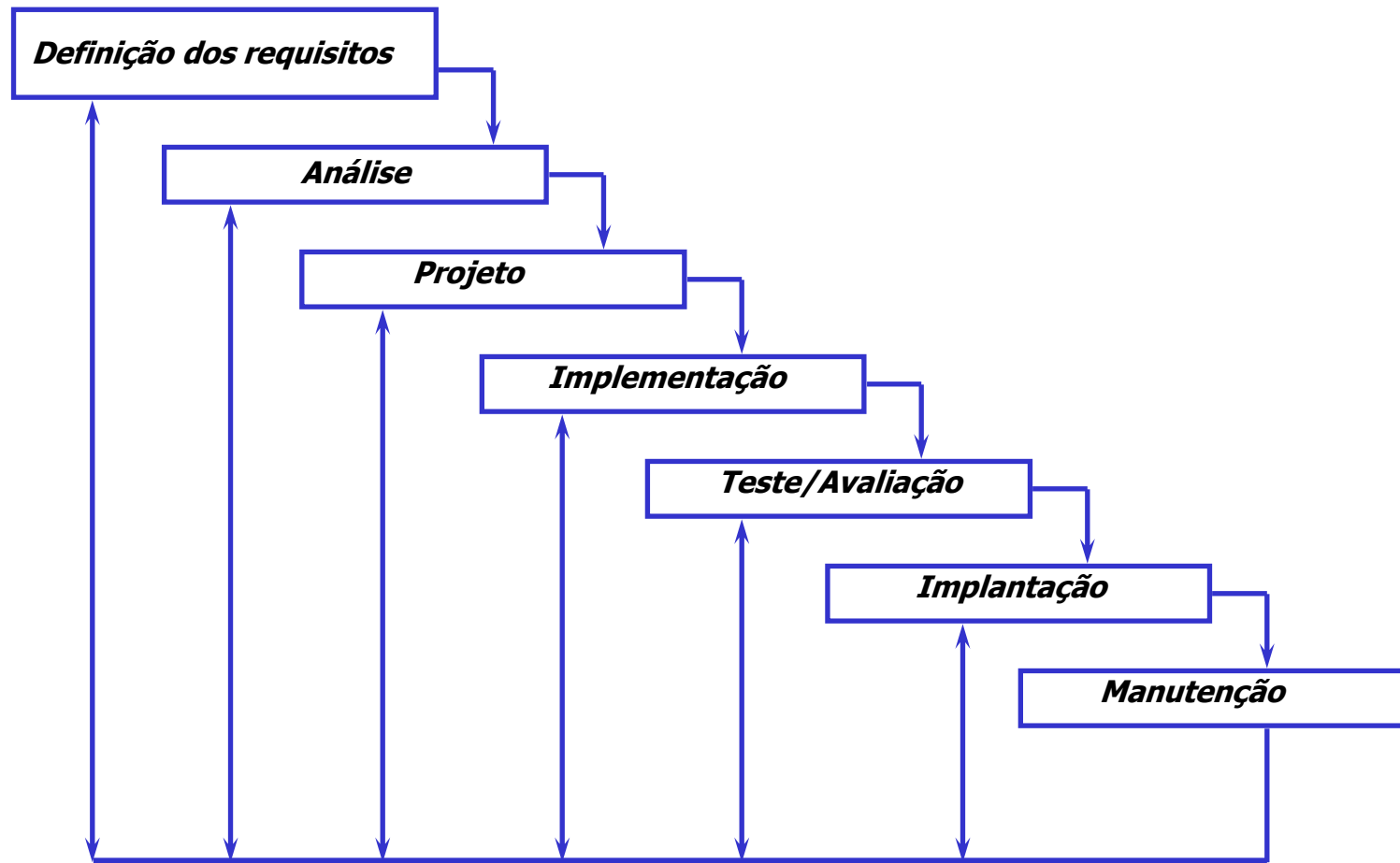
- ✓ É um processo que envolve refinamento e detalhamento de como os componentes do sistema serão construídos.
- ✓ Deve estar aderente ao estilo arquitetural definido.
- ✓ É formado por várias visões, por vezes distintas (projeto do BD, projeto da interface com o usuário, projeto da interface com outros sistemas, etc.).
- ✓ É fortemente afetado por alterações nos requisitos.



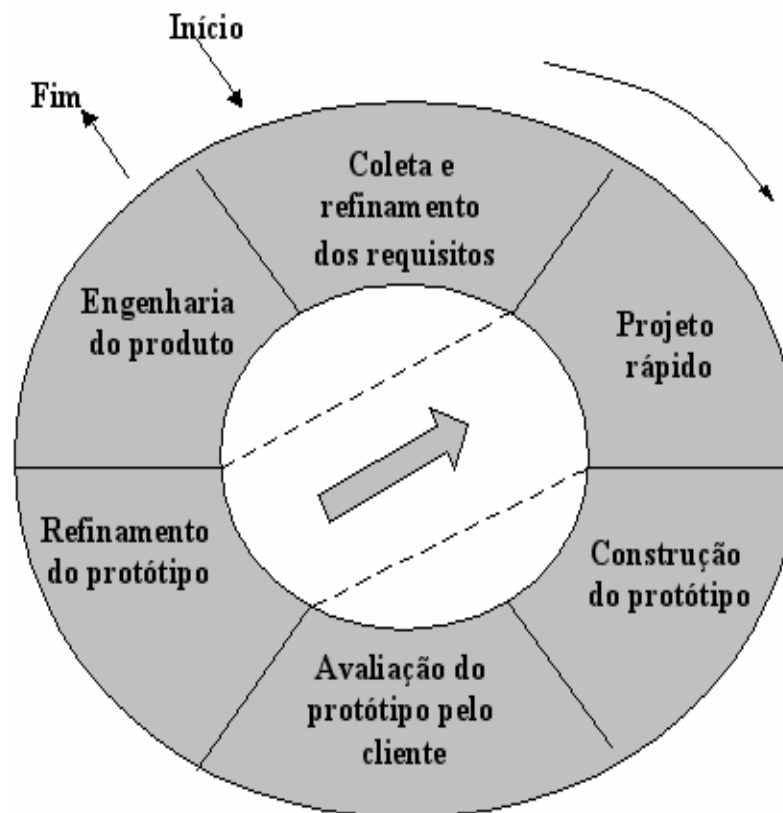
Mudanças nas definições arquiteturais tem um impacto bastante significativo em um software.

Quanto maior e mais complexo for o software, mais relevante se torna a definição de sua arquitetura.

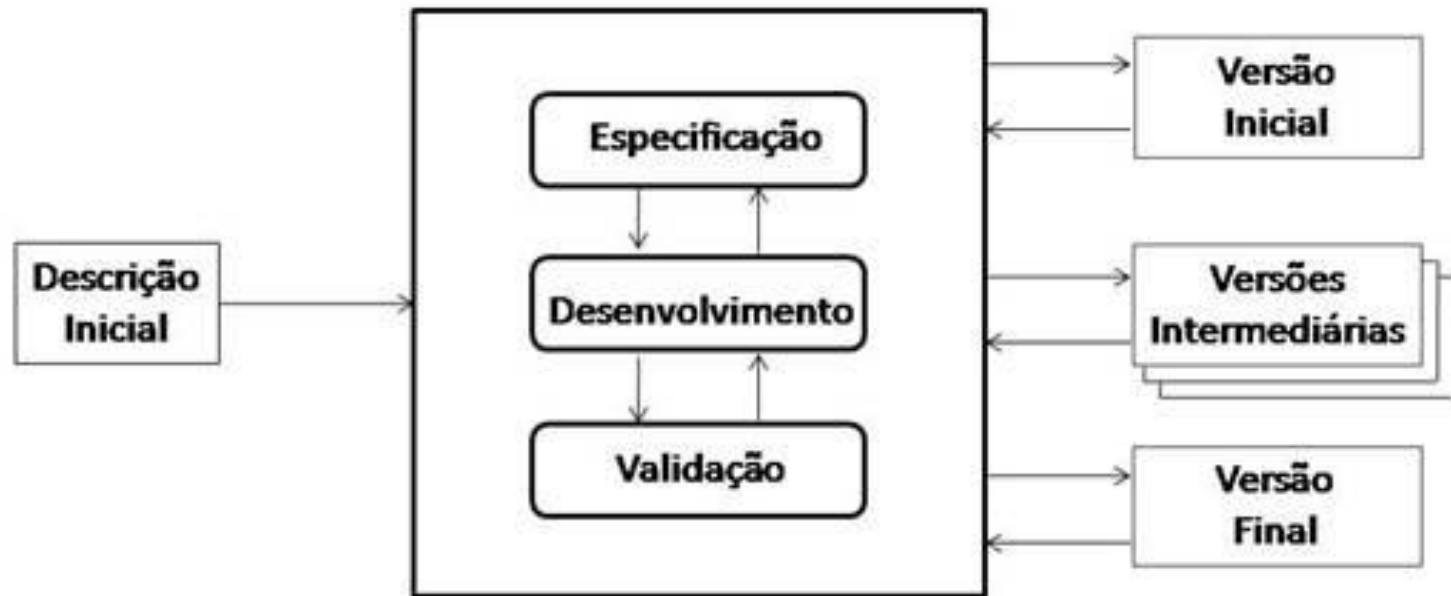
- Modelo Cascata (Royce, 1970): uma fase só inicia quando a anterior termina.



- **Prototipagem:** tentar entender os requisitos mais obscuros ou mal entendidos.



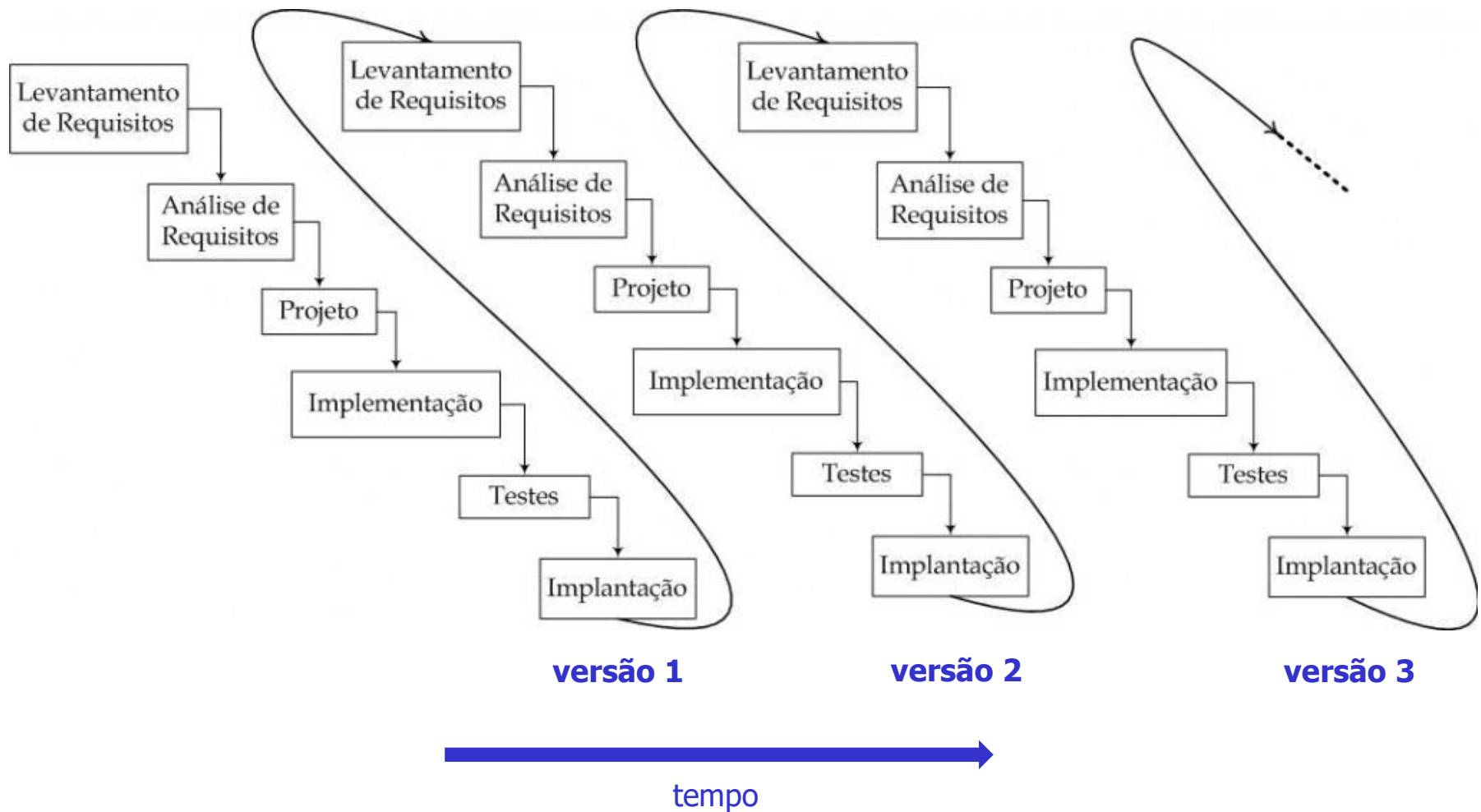
- **Desenvolvimento Evolucionário:** inicia com as partes do software que são mais bem compreendidas.



- Para o desenvolvimento de **grandes sistemas**:
 - ✓ Pode haver a necessidade de diferentes abordagens para diferentes partes do sistema, de maneira que um **modelo híbrido** deve ser utilizado.
 - ✓ Há também a necessidade de **iteração**, pois o tamanho e a complexidade do sistema nos levam a dividir o desenvolvimento do sistema em várias partes (**produtos** dividir-para-conquistar).
- Existem dois modelos híbridos, que apoiam diferentes abordagens do desenvolvimento e que foram explicitamente projetados para apoiarem a iteração do processo:
 - ✓ Modelo Incremental
 - ✓ Modelo em Espiral

- Combina os elementos do modelo cascata com a prototipagem.
- Divide o desenvolvimento de um produto de software em ciclos, com o software sendo desenvolvido em partes (incrementos).
- Pode haver **paralelismo** entre as atividades realizadas em cada incremento, desde que haja recursos humanos para isso.
- Diferentemente da Prototipagem, cada incremento produz uma **versão operacional do software**.
- Dessa forma as funcionalidades do software são entregues **por estágios**, onde as funcionalidades prioritárias são entregues nos estágios iniciais (primeiras versões).

- Cada versão pode ser desenvolvida com o ciclo cascata.



- Entrega por Estágio

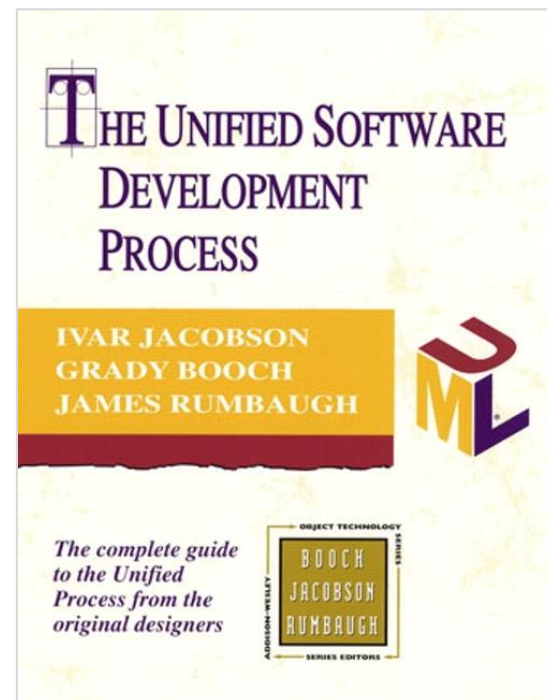


- No início, existiam vários métodos propostos para a Análise e Projeto OO:
 - ✓ Coad & Yourdon (*Object Oriented Analysis* e *Object Oriented Design*)
 - ✓ Shlaer & Mellor (*Object Oriented System Analysis*)
 - ✓ Wirfs-Brock (*Responsability Driven Design*)
 - ✓ Booch (*Booch Method*)
 - ✓ Jacobson (*Object Oriented Software Engineering*)
 - ✓ Rumbaugh (*Object Modeling Technique*)

- No início, existiam vários métodos propostos para a Análise e Projeto OO:
 - ✓ Coad & Yourdon (*Object Oriented Analysis* e *Object Oriented Design*)
 - ✓ Shlaer & Mellor (*Object Oriented System Analysis*)
 - ✓ Wirfs-Brock (*Responsability Driven Design*)
- ✓ Booch (*Booch Method*)
 - ✓ Jacobson (*Object Oriented Software Engineering*)
 - ✓ Rumbaugh (*Object Modeling Technique*)

Booch, Jacobson e Rumbaugh
juntaram seus métodos e criaram a
UML (Unified Modeling Language)

- Em 1995, Booch, Rumbaugh e Jacobson criaram o **Processo Unificado** (*Unified Process – UP*) como um *framework* para desenvolvimento OO.
- O **Processo Unificado** define atividades específicas voltadas para análise, arquitetura e design de sistemas.



➤ Principais características do Processo Unificado:

1. Iterativo e incremental

- ✓ Sistema é dividido em partes e o resultado de cada iteração é um incremento (versão) do sistema.
- ✓ A cada iteração mais detalhes são adicionados.

2. Dirigido por Casos de Uso

- ✓ São usados para especificar os requisitos.
- ✓ Também são usados no planejamento e no acompanhamento das iterações.
- ✓ Os testes do sistema também são baseados nos casos de uso.

- Principais características do Processo Unificado:

3. Centrado em arquitetura

- ✓ A arquitetura do sistema é definida nas primeiras iterações e detalhada nas demais.
- ✓ A arquitetura serve para organizar o desenvolvimento, estruturar a solução e identificar oportunidades de reuso.
- ✓ Os casos de uso dizem **o que** deve ser feito e a arquitetura descreve **como** deve ser feito.

4. Orientado a Objetos

- ✓ Utiliza a UML como linguagem de modelagem para elaboração dos artefatos ao longo do ciclo de vida.

- Posteriormente, com base no Processo Unificado, a empresa IBM/Rational criou um produto comercial chamado *Rational Unified Process (RUP)*.
- O RUP é:
 - ✓ Um produto comercial.
 - ✓ Desenvolvido e mantido pela IBM/Rational.
 - ✓ Mais complexo e completo que o Processo Unificado.
- Existe também uma versão *open source* baseada no Processo Unificado chamado **OpenUP** (<http://epf.eclipse.org/wikis/openup/>). O OpenUP é diferente do RUP.

- O ciclo de vida do RUP é composto de 4 Fases:

CONCEPÇÃO

ELABORAÇÃO

CONSTRUÇÃO

TRANSIÇÃO

- Em linhas gerais, as fases se destinam a:



➤ Concepção (ênfase na definição do escopo)

✓ Consiste de:

- Identificação do escopo do projeto.
- Identificação inicial dos requisitos.
- Esboço dos casos de uso (10% a 20%).
- Identificação de arquiteturas candidatas.
- Estimativas iniciais de cronograma e custo (plano geral).

✓ Principais características:

- Menor fase do projeto.
- Escopo ainda vago.
- Estimativas ainda vagas.

➤ Elaboração (ênfase na arquitetura e design)

✓ Consiste de:

- Detalhamento da maioria dos requisitos e casos de uso (cerca de 80%).
- Definição e validação da arquitetura do software.
- Detalhamento das estimativas de cronograma e custo (plano detalhado).

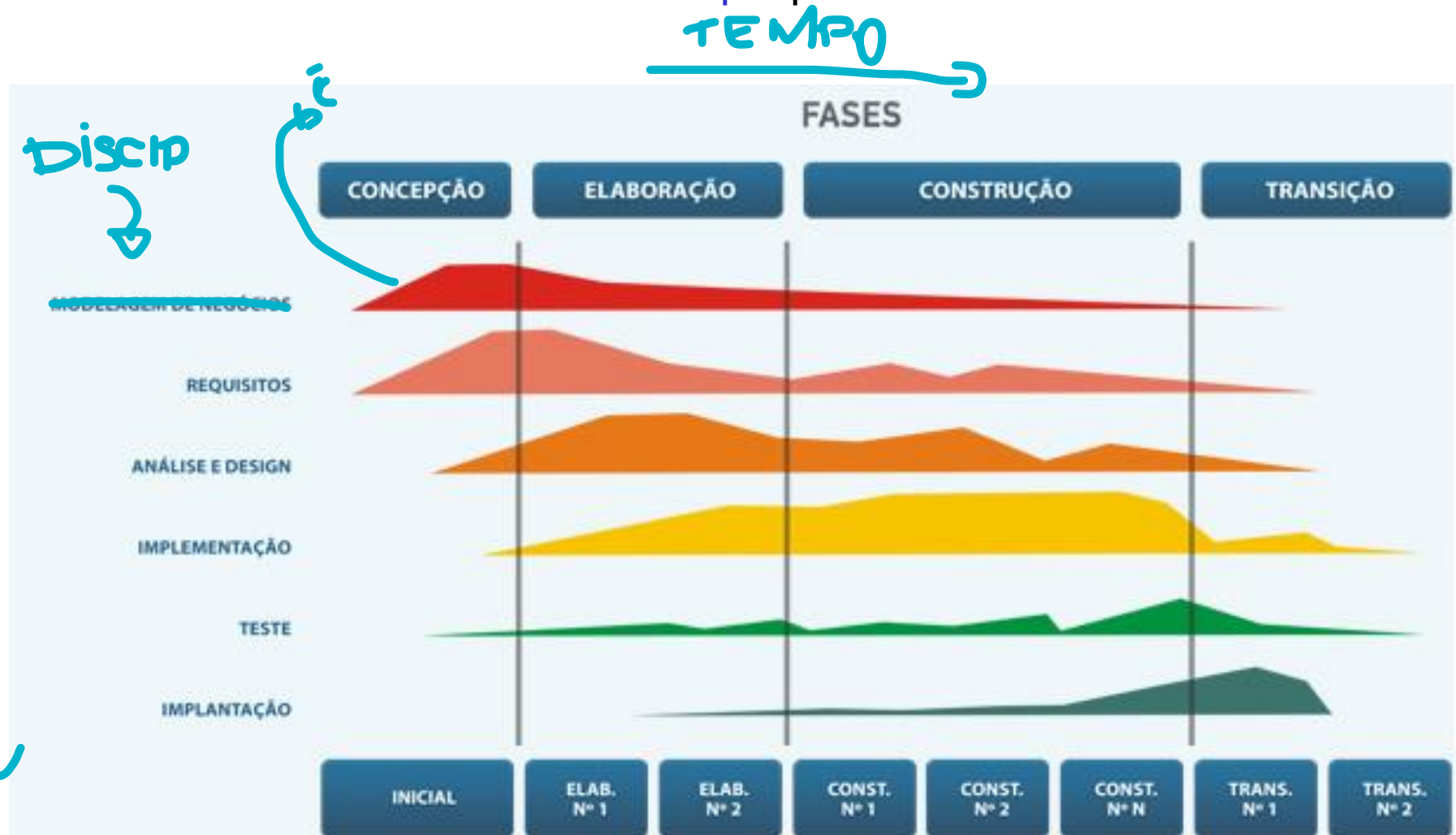
✓ Principais características:

- Grande parte das atividades de análise e design já concluída.
- Diminuição significativa das incertezas.
- Definição da arquitetura é estabelecida.

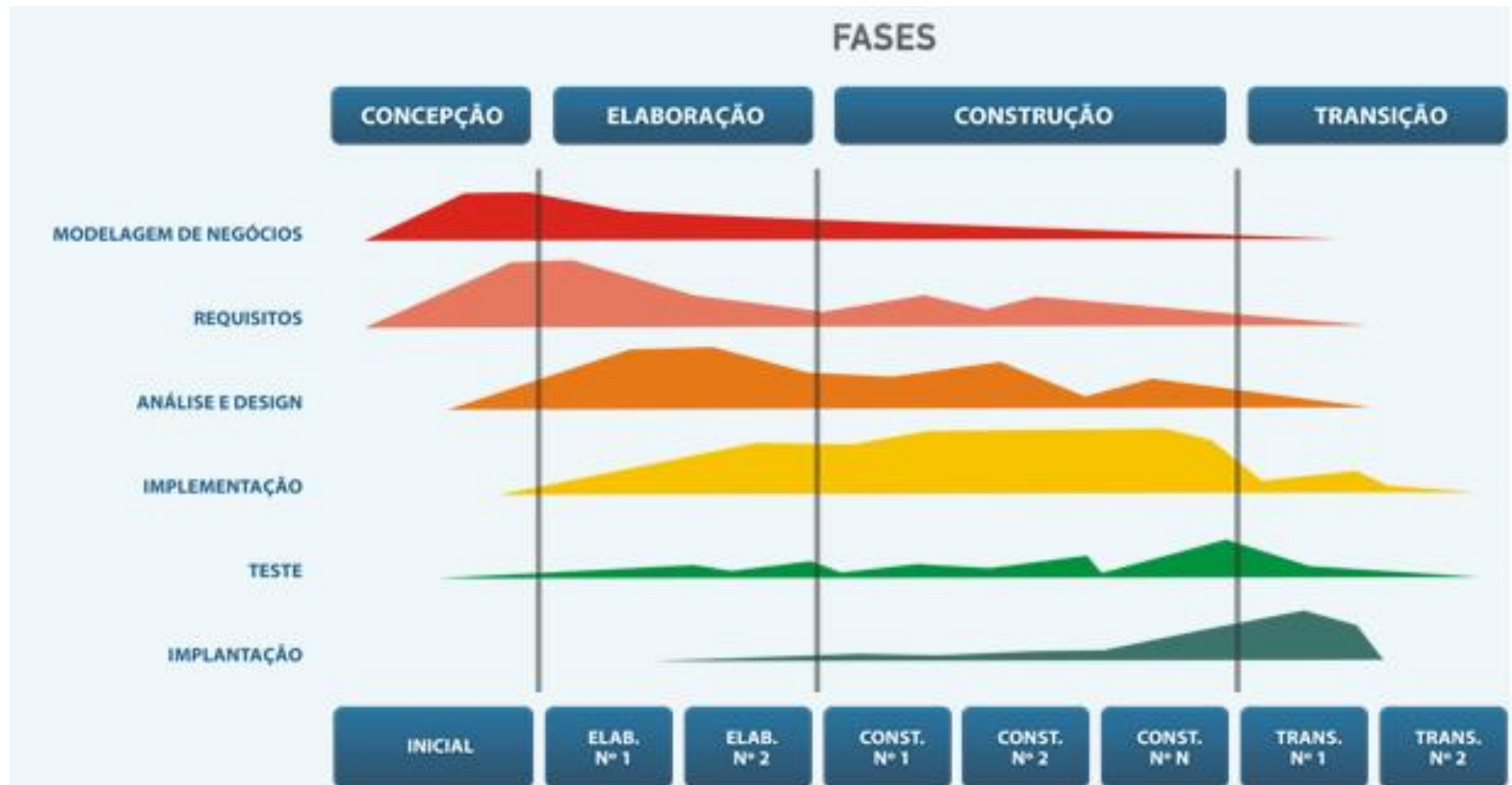
- Construção (ênfase na codificação e testes)
 - ✓ Consiste de:
 - Completar modelos de análise e design.
 - Desenvolver ou integrar componentes de software.
 - Tornar cada caso de uso operacional.
 - Elaborar e executar testes.
 - ✓ Principais características:
 - Maior fase do projeto.
 - Definição dos testes é estabelecida.

- Transição (ênfase nos testes de aceitação e implantação)
 - ✓ Consiste de:
 - Execução de testes finais no ambiente dos usuários.
 - Implantação do produto.
 - Treinamento dos usuários.
 - Conversão de dados.
 - ✓ Principais características:
 - Executado no ambiente do usuário.
 - Realiza a entrega de uma versão utilizável do software.

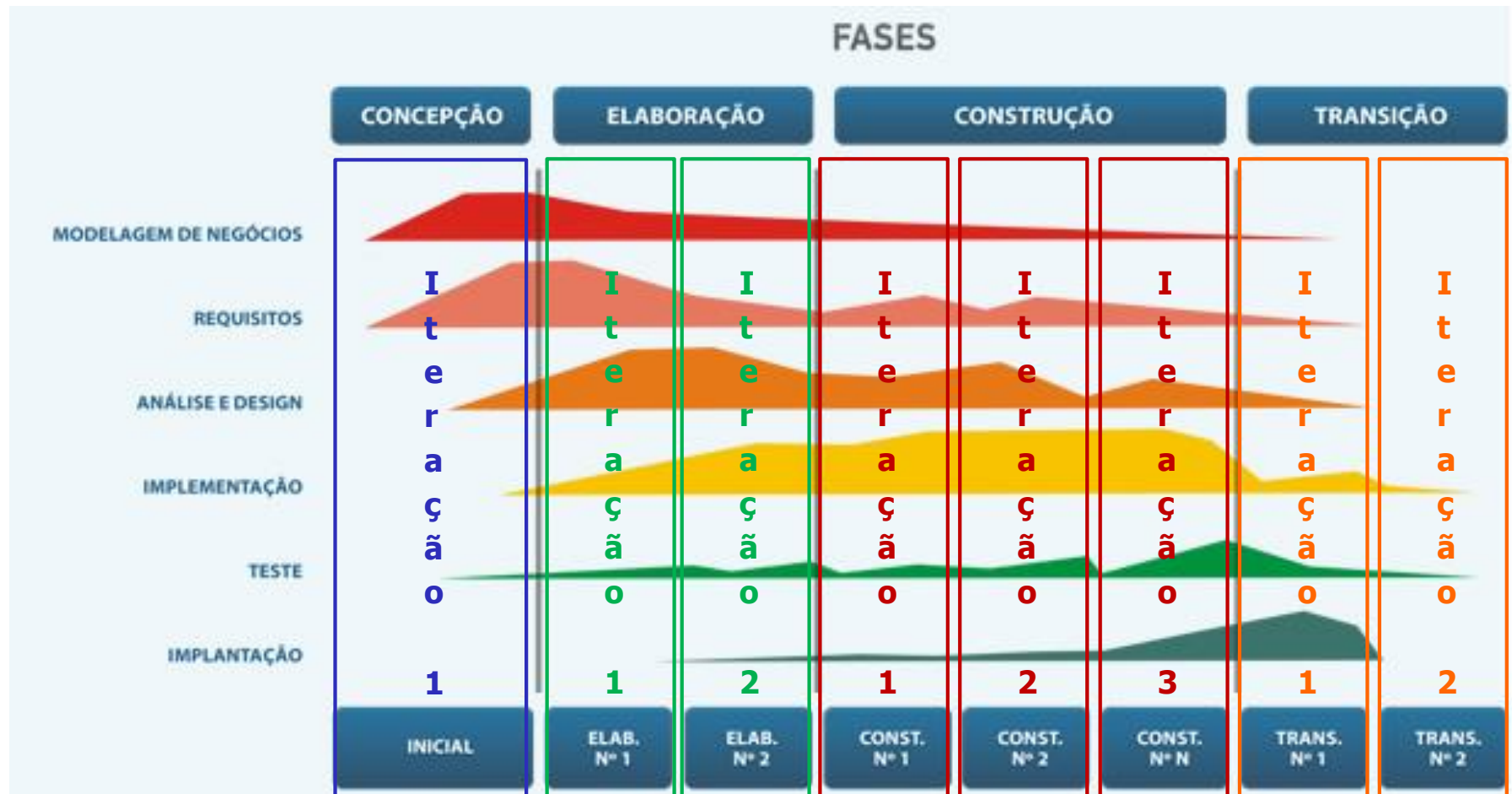
- Existem 9 Fluxos de Trabalho que permeiam as 4 Fases do RUP:



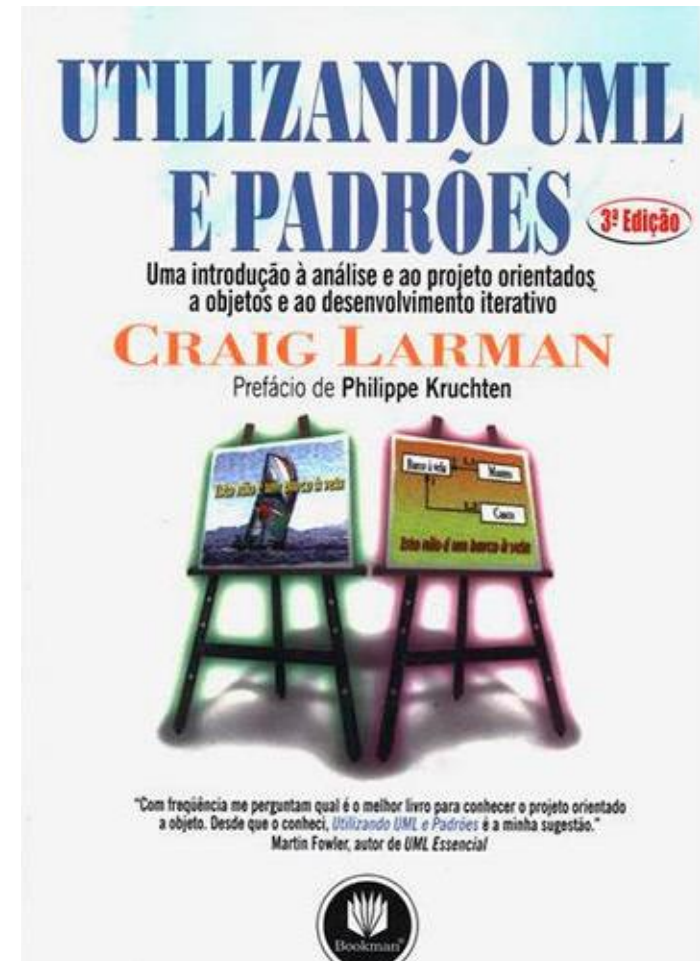
- Note que a **quantidade de esforço** em cada Fluxo de Trabalho varia conforme a Fase.



- Cada fase do RUP pode ser executada em **N** iterações.

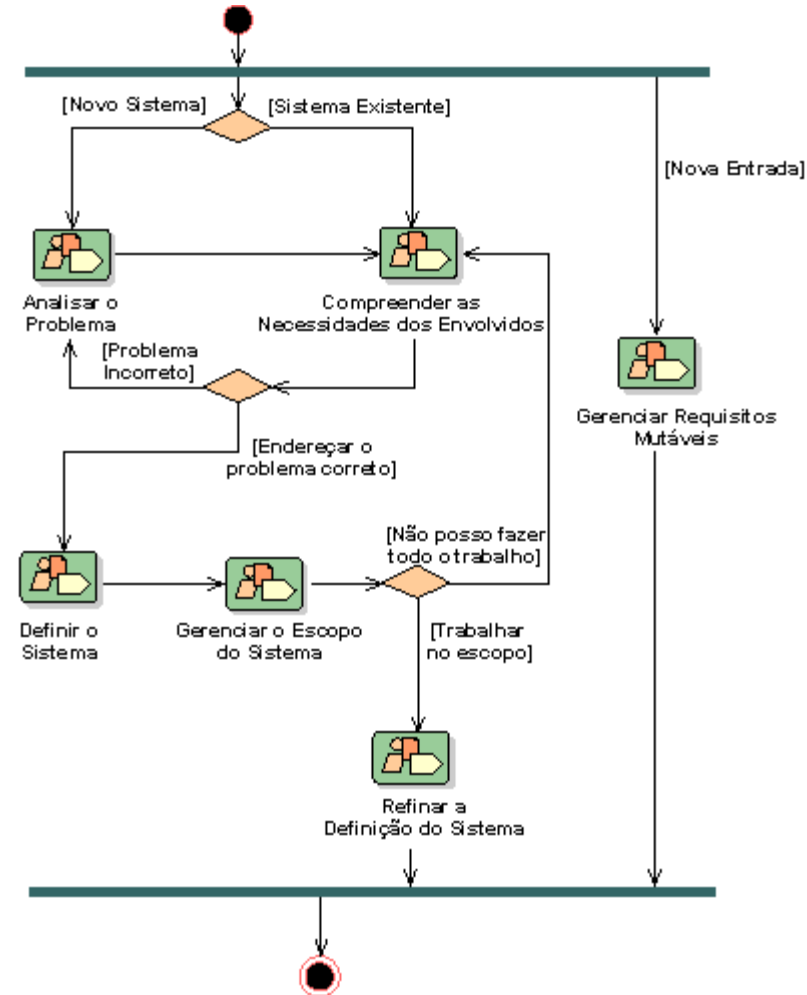


- Para entender melhor como o Processo Unificado combina o conceito de iteração com as fases e fluxos de trabalho, leia a seção 2.4 (pp. 53-55) do livro Utilizando UML e Padrões, disponível na biblioteca virtual.



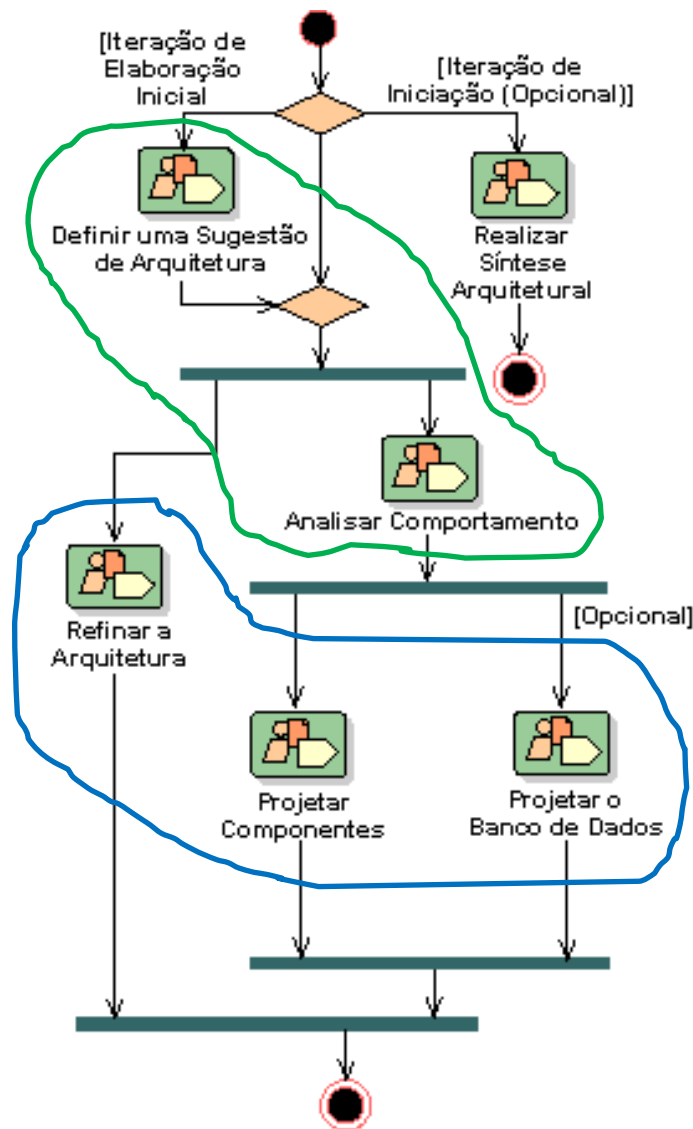
➤ Fluxo de Requisitos do RUP

- ✓ Estabelecer um acordo com os clientes sobre o que o sistema deve fazer, ou seja, definir as **fronteiras do sistema**.
- ✓ Identificar os envolvidos e suas demandas.
- ✓ Elaborar o Documento de Visão, os Casos de Uso e a Especificação Suplementar que descrevem **o que** o sistema deve fazer.



➤ Fluxo de Análise e Design do RUP

- ✓ Desenvolver uma arquitetura robusta para o sistema.
- ✓ Transformar os requisitos em um projeto detalhado do sistema.



➤ A disciplina de **Análise e Design** é composta dos seguintes fluxos de trabalho:

1. Definir uma Sugestão de Arquitetura

1. Análise Arquitetural
2. Análise de Caso de Uso

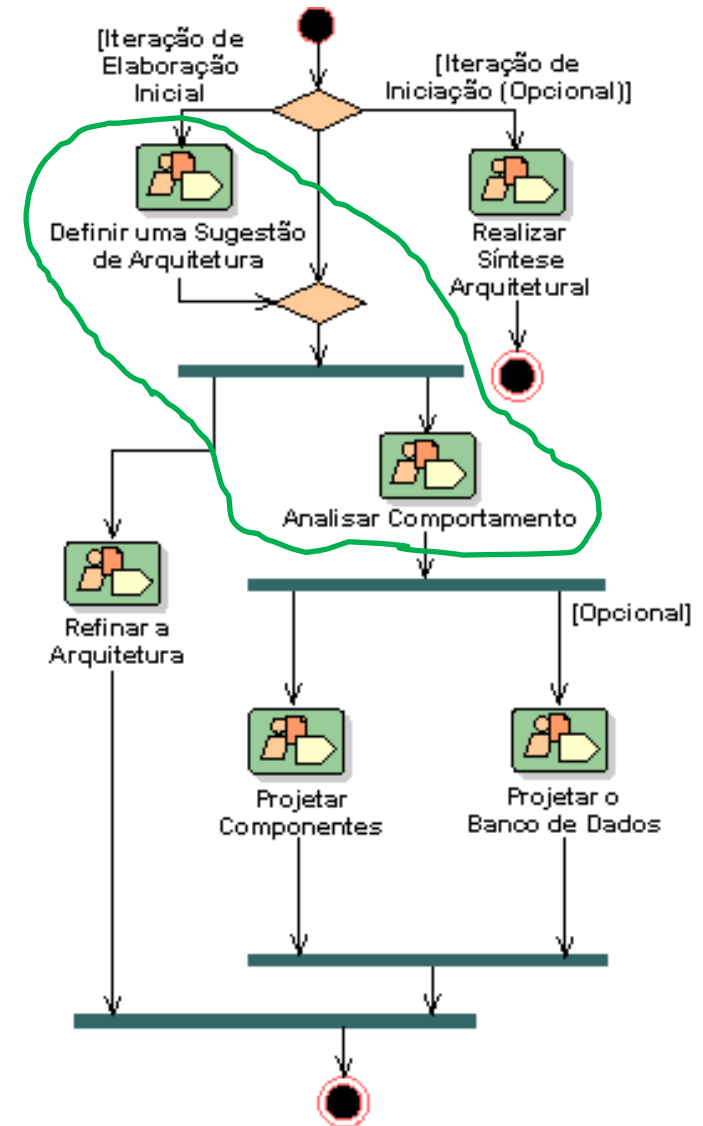
2. Analisar Comportamento

1. Análise de Caso de Uso
2. Identificar Elementos de Design
3. Revisar o Design

3. Projetar Componentes

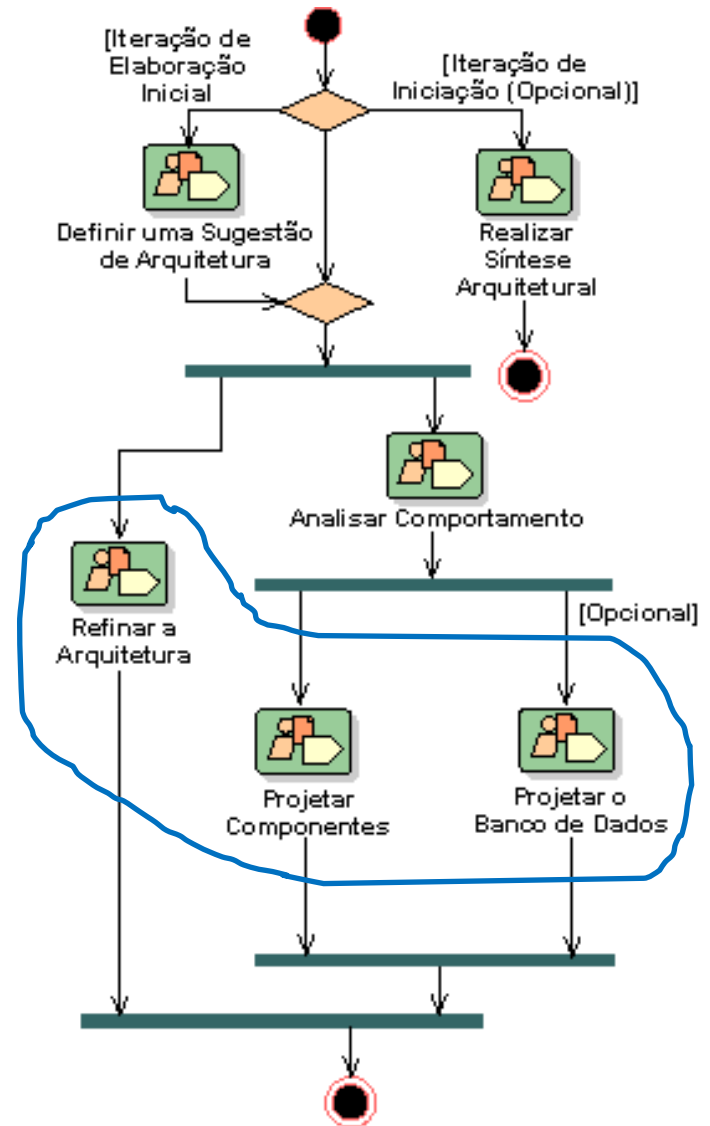
4. Projetar BD

5. Refinar a Arquitetura



➤ A disciplina de **Análise e Design** é composta dos seguintes fluxos de trabalho:

1. Definir uma Sugestão de Arquitetura
2. Analisar Comportamento
3. **Projetar Componentes**
 1. Design de Caso de Uso
 2. Design de Classe
 3. Design de Subsistema
4. **Projetar BD**
 1. Elaborar Design do BD
5. Refinar a Arquitetura



Projeto de Software Orientado a Objetos

2ª parte

Prof. Jobson Massollar
jobson.silva@uva.br

- Arquitetura de Software (PFLEEGER, 1998):
 - Uma arquitetura de software envolve a **descrição de elementos arquiteturais** a partir dos quais os sistemas serão construídos, **interações** entre esses elementos, **padrões** que guiam suas **composições** e **restrições** sobre estes padrões.

- Por que se preocupar com a arquitetura?
 - ✓ Apoia o planejamento do desenvolvimento.
 - ✓ Permite tratar os atributos de qualidade de software (requisitos não funcionais).
 - ✓ Guia as atividades de projeto e implementação.
 - ✓ Ajuda a gerenciar os riscos técnicos e a complexidade da solução.
 - ✓ Apoia a análise de impacto de mudanças.

- Elementos ou Componentes Arquiteturais:
 - ✓ Executam ou apoiam a execução das funcionalidades do sistema, ou seja, são responsáveis pela execução dos serviços computacionais propriamente ditos.
 - ✓ Componentes são abstrações que representam o hardware e o software (desenvolvido pela equipe ou por terceiros) que fazem parte de um sistema.

➤ Exemplos de Componentes:

✓ Softwares:

- Desenvolvidos pela equipe (aplicativos, pacotes, módulos, bibliotecas, web services, etc.)
- SGBDs (SQL Server, MySQL, Oracle, etc.)
- Servidores de aplicação (Apache Tomcat, Zend Server, IIS, etc.)
- Containers (Docker, Kubernetes, Google Cloud, Microsoft Azure, etc.)
- Sistemas operacionais (Windows, Linux, Android, iOS, etc.)
- Bibliotecas de terceiros (jQuery, Apache Commons, OpenGL, etc.)

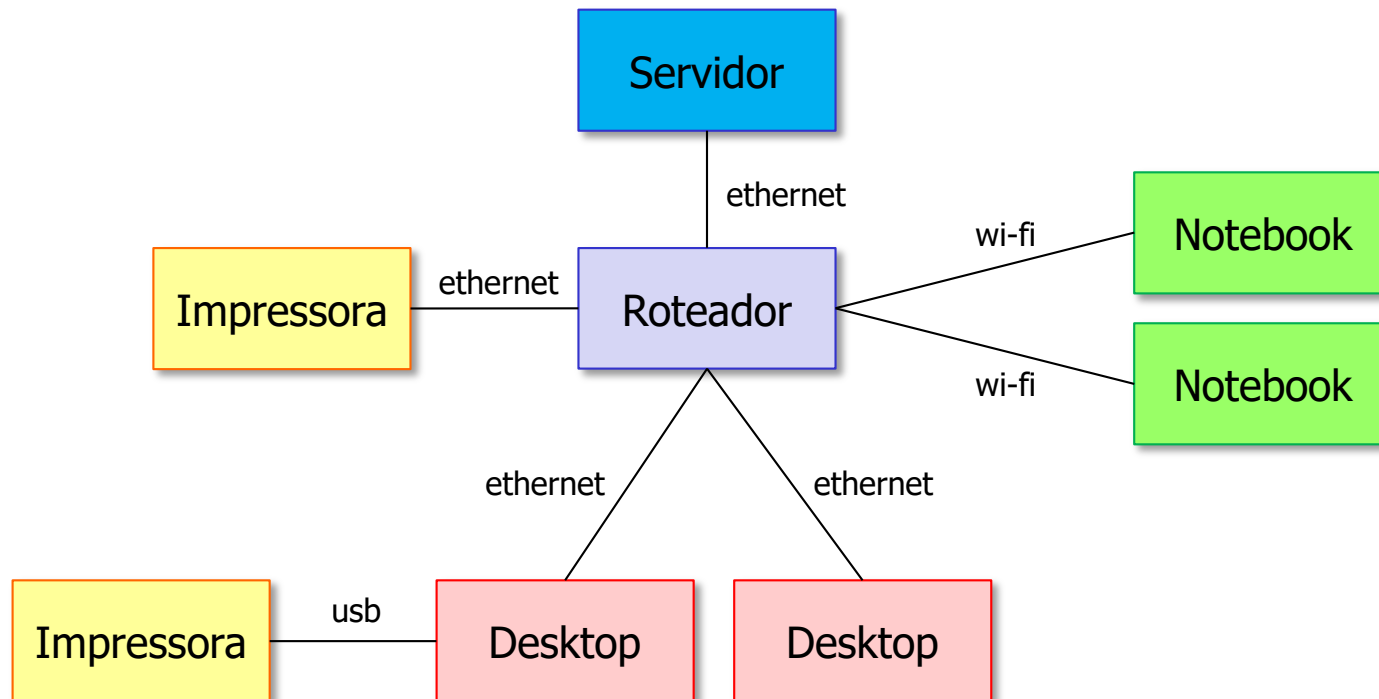
✓ Hardwares:

- Servidores
- Computadores pessoais (desktop, notebook, etc.)
- Dispositivos móveis (smartphone, tablet, smartwatch, etc.)
- Sensores/Leitores
- Atuadores (equipamentos diversos controlados por software)

➤ Conectores Arquiteturais:

- ✓ Definem a **interação** entre os componentes da arquitetura, permitindo representar a **comunicação**, **coordenação** e **cooperação** entre eles.
- ✓ Conectores podem representar a conexão entre componentes de **hardware** (**equipamentos**) ou entre componentes de **software**.

- Exemplo de Componentes e Conectores arquiteturais (visão da infraestrutura de hardware):



- Os Conectores de software podem ser usados para representar:
 - ✓ Comunicação de dados (HTTP, HTTPS, TCP/IP, SSL, etc.)
 - ✓ Chamada de funções, procedimentos ou métodos
 - ✓ Invocação de serviços
 - ✓ Acesso a dados:
 - Conexão com banco de dados (ODBC, JDBC, ADO.NET, etc.)
 - Conexão com arquivos (file input/output)
 - ✓ Streams
 - ✓ Eventos
 - ✓ etc.

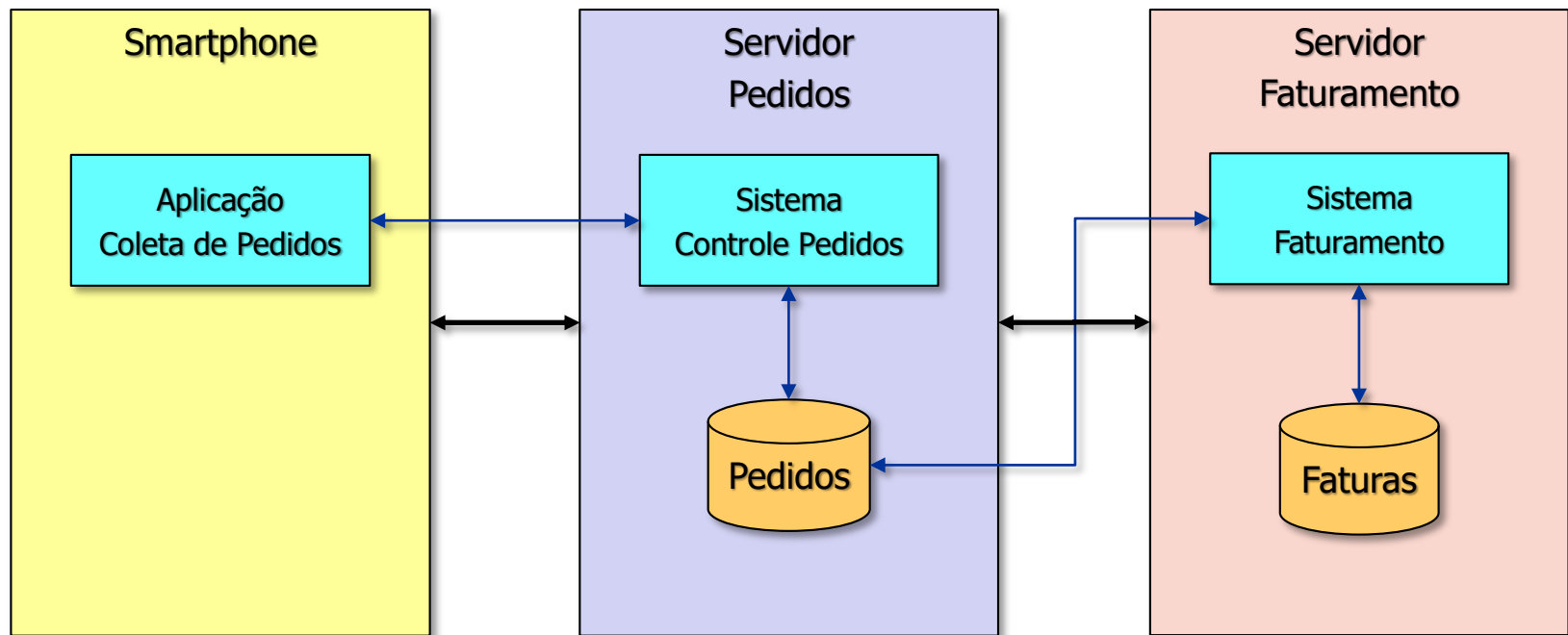
- Como já visto anteriormente, a Arquitetura de Software:
 - ✓ Fornece uma visão de mais alto nível do sistema.
 - ✓ Estabelece uma metáfora para organização dos diversos componentes do software, ou seja, para o design.

- Importante:
 - ✓ Detalhes **internos** dos componentes ou elementos arquiteturais **não são relevantes**.
 - ✓ Captura apenas as informações sobre os componentes e as interações entre eles.

- Qual seria a arquitetura candidata de um sistema para atender ao seguinte cenário?

Uma empresa de venda de bebidas (refrigerantes e cervejas) tem um catálogo com cerca de 20 produtos e pouco mais de 100 vendedores. Cada vendedor visita, por dia, aproximadamente 50 clientes, que estão distribuídos por toda a cidade do Rio de Janeiro, Baixada Fluminense e Niterói. O roteiro de visita de cada vendedor é fixo por dia da semana. A empresa gostaria de desenvolver um sistema que permitisse aos vendedores coletar eletronicamente os pedidos dos clientes para que, ao final do dia, esses pedidos fossem faturados e entregues no dia seguinte. Toda a transmissão de dados deve ser feita de forma segura, ou seja, criptografada. Os vendedores trabalham distribuídos por toda a cidade, ou seja, não existe um local único de onde os vendedores partam ou cheguem ao final do dia.

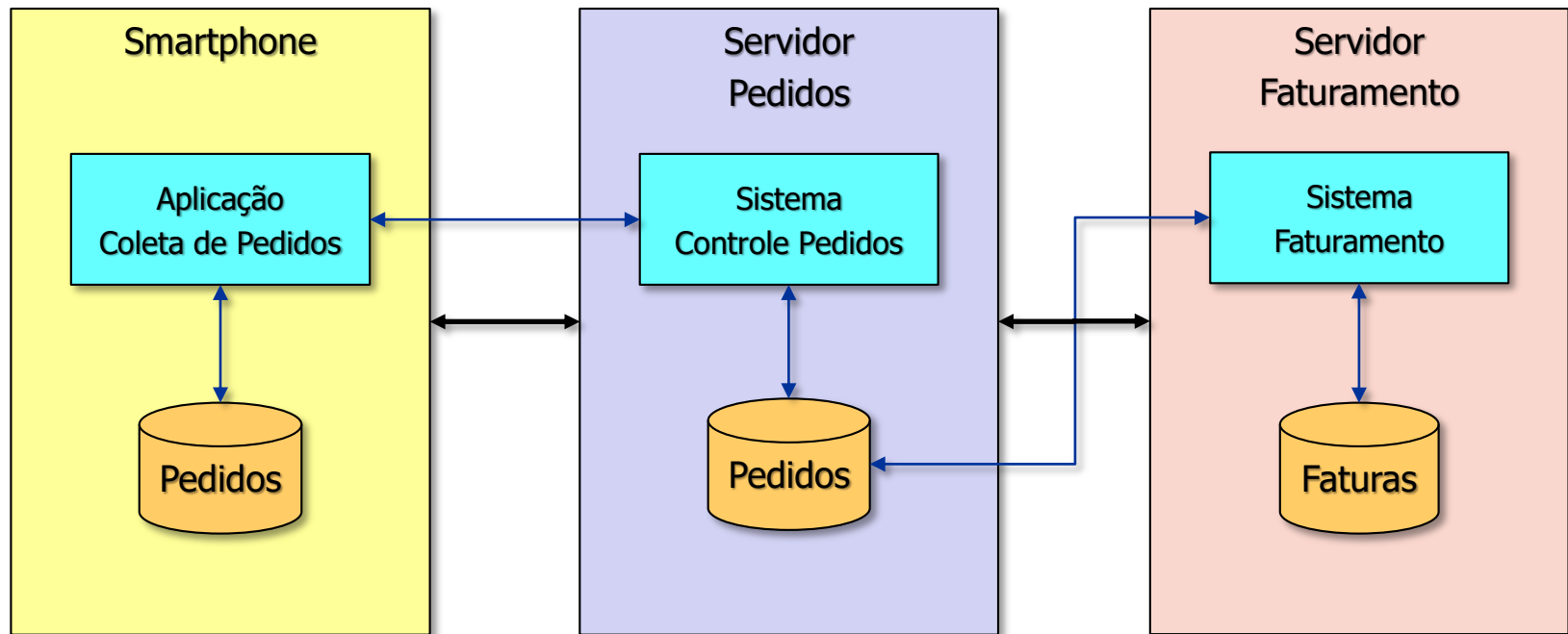
- Arquitetura candidata 1: pedidos transmitidos *online*.



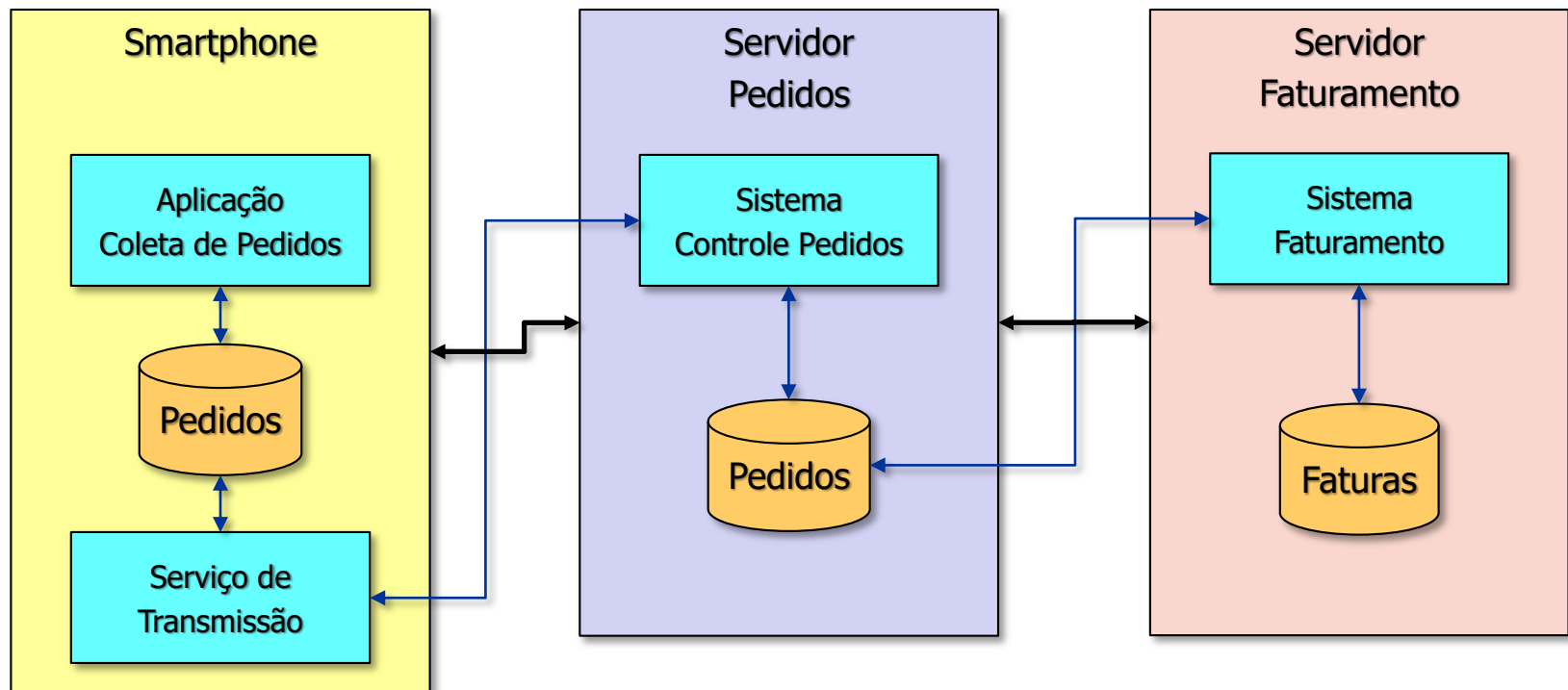
- Qual seria a arquitetura candidata de um sistema para atender ao seguinte cenário?

Uma empresa de venda de bebidas (refrigerantes e cervejas) tem um catálogo com cerca de 20 produtos e pouco mais de 100 vendedores. Cada vendedor visita, por dia, aproximadamente 50 clientes, que estão distribuídos por toda a cidade do Rio de Janeiro, Baixada Fluminense e Niterói. O roteiro de visita de cada vendedor é fixo por dia da semana. A empresa gostaria de desenvolver um sistema que permitisse aos vendedores coletar eletronicamente os pedidos dos clientes para que, ao final do dia, esses pedidos fossem faturados e entregues no dia seguinte. Toda a transmissão de dados deve ser feita de forma segura, ou seja, criptografada. Os vendedores trabalham distribuídos por toda a cidade, ou seja, não existe um local único de onde os vendedores partam ou cheguem ao final do dia. Em muitos clientes, principalmente os mais afastados, não há sinal de telefonia celular disponível ou o sinal é muito fraco. O software de vendas deve ser atualizado automaticamente nos dispositivos dos vendedores, caso haja uma nova versão disponível.

- Arquitetura candidata 2: pedidos transmitidos em lote ao final do dia.



- Arquitetura candidata 3: pedidos transmitidos por demanda.



➤ Estilos Arquiteturais

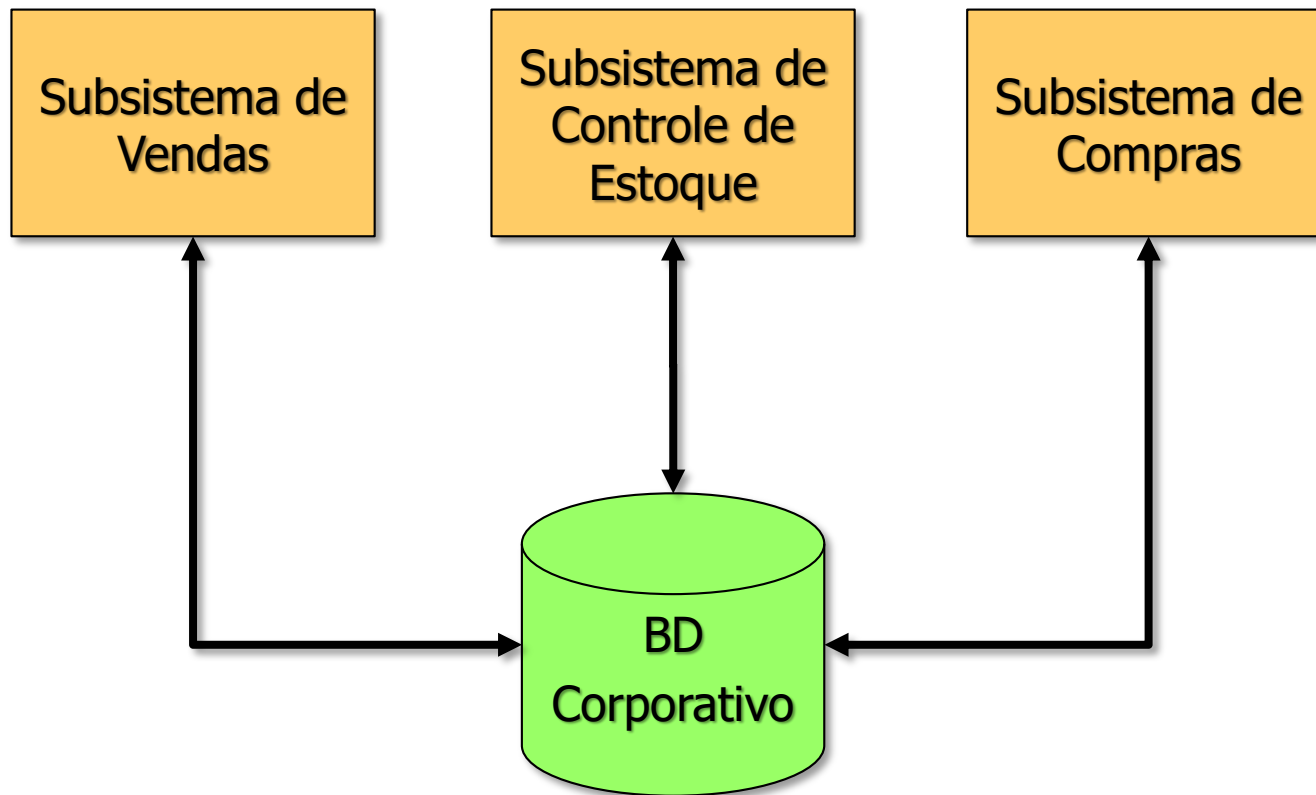
Um estilo arquitetural define um vocabulário para os elementos da arquitetura, que tipos de elementos podem aparecer, além das regras e restrições que regem a combinação entre eles.

- Os estilos arquiteturais podem simplificar o problema de definição de arquiteturas de sistema.
- A maioria dos sistemas de grande porte adere a **vários** estilos.
- Estilos arquiteturais = "modelos arquiteturais".

- Existem diversos estilos arquiteturais propostos na literatura:
 - ✓ Repositório
 - ✓ Camadas (Layers)
 - ✓ Cliente-Servidor (2-Tier)
 - ✓ N-Tier
 - ✓ Model-View-Controller (MVC)
 - ✓ Microservices
 - ✓ Publish-Subscribe
 - ✓ Dutos e Filtros (Pipes & Filters)
 - ✓ Peer-to-peer (P2P)
 - ✓ Microkernel
 - ✓ Blackboard
 - ✓ Broker
 - ✓ Model-View-Presenter (MVP)
 - ✓ etc.

- Características:
 - ✓ Permite que vários subsistemas manipulem uma mesma base de dados, ou seja, os dados são compartilhados entre os subsistemas.
 - ✓ Um ou mais subsistemas podem gerar ou consumir dados.
- Quando adotá-lo?
 - ✓ Quando temos uma família de sistemas (subsistemas) que necessitam compartilhar um grande conjunto de dados.

➤ Exemplo:



➤ Vantagens:

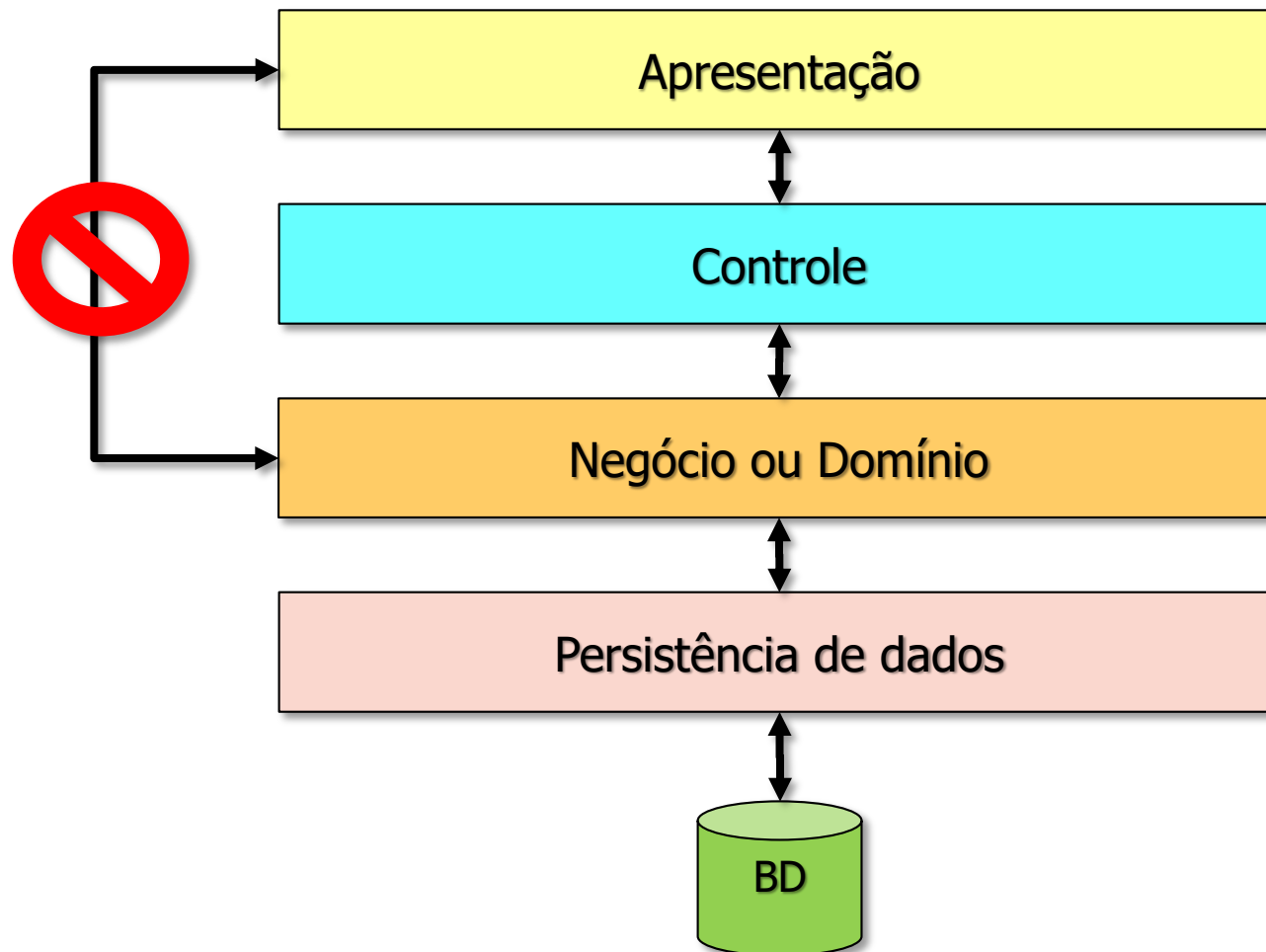
- ✓ Maneira eficiente de compartilhar dados.
- ✓ Backup é centralizado (mais fácil).
- ✓ Formas de proteção dos dados podem ser implementadas.
- ✓ Os subsistemas que geram dados não necessitam saber quem os consome.
- ✓ Fácil integrar novos subsistemas.

➤ Desvantagens:

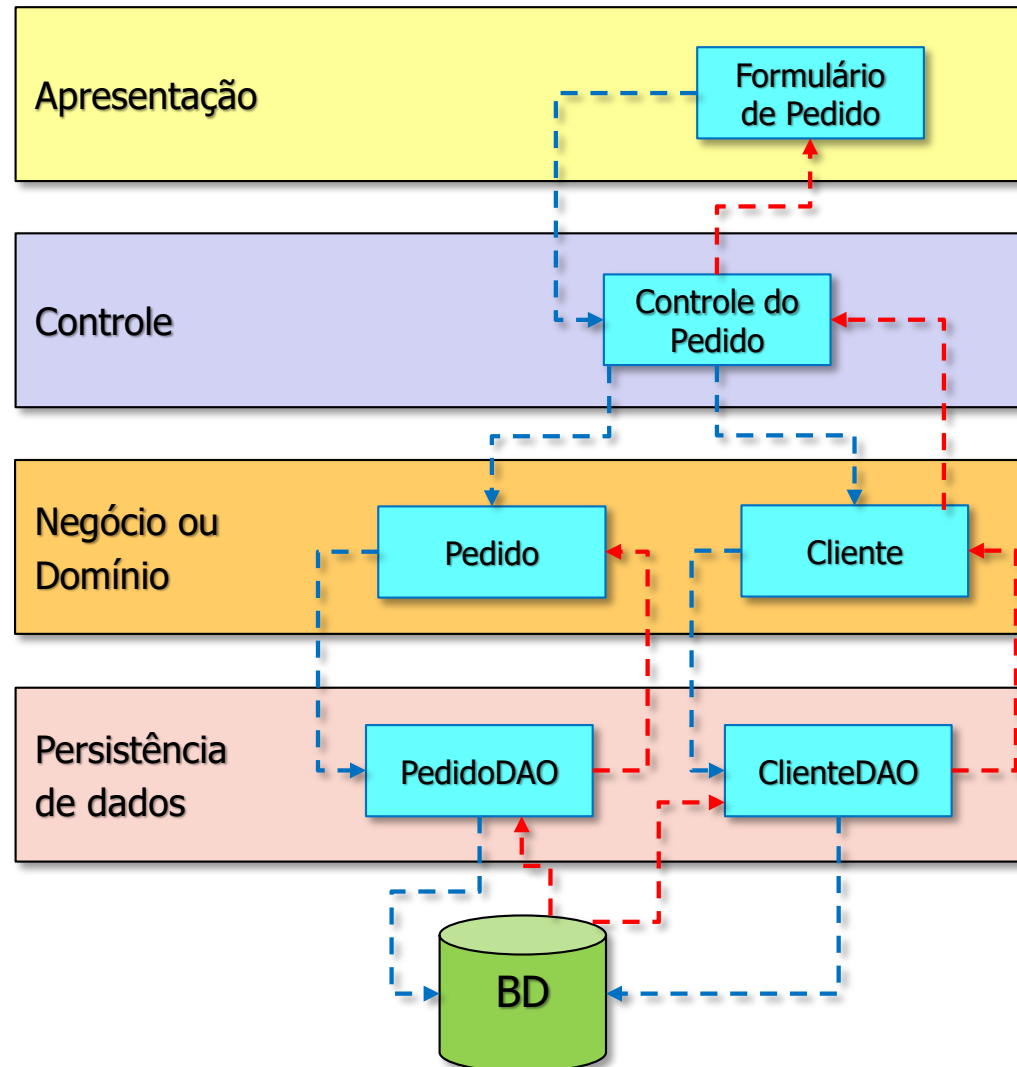
- ✓ É preciso definir um padrão de acesso e manipulação dos dados e os subsistemas precisam se adaptar a este padrão.
- ✓ Necessário tratar requisitos não-funcionais divergentes entre os subsistemas: alta disponibilidade x alta segurança.

- Características:
 - ✓ Organiza o sistema em um conjunto de camadas.
 - ✓ Cada camada oferece um conjunto de serviços.
 - ✓ Cada camada tem uma função bem definida no sistema.
 - ✓ Uma camada somente:
 - Solicita serviços da camada inferior, e;
 - Fornece serviços para a camada superior.
 - ✓ É importante destacar que *layers* se referem a camadas lógicas.

➤ Exemplo:



➤ Exemplo:



➤ Exemplo: modelo OSI



➤ Exemplo: Android



➤ Vantagens:

- ✓ É possível desenvolver cada camada de forma independente (paralelismo).
- ✓ As camadas podem ser facilmente substituídas por equivalentes.
- ✓ Mudanças em uma camada só impactam a camada imediatamente superior.

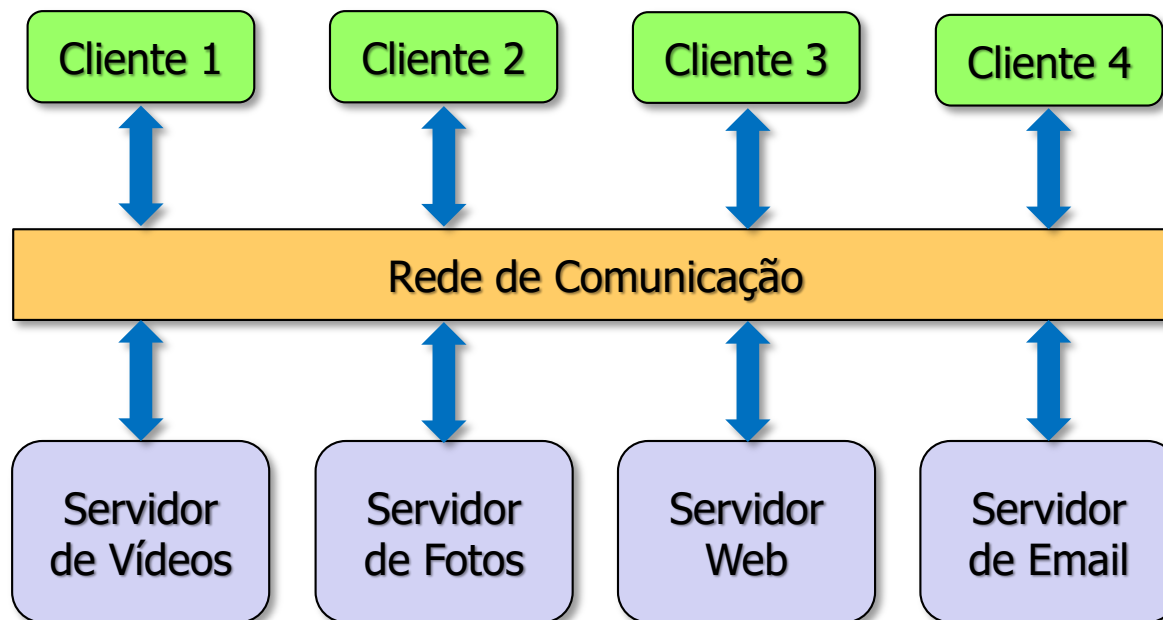
➤ Desvantagens:

- ✓ Cuidados extras devem ser tomados para garantir a integridade entre as camadas, por exemplo, o estado de um objeto na camada de persistência deve ser refletivo na camada de domínio e também na camada de apresentação.
- ✓ Muitas camadas podem comprometer o desempenho do sistema, pois a requisição precisa trafegar pelas várias camadas até ser atendida.

➤ Características:

- ✓ Organizada como um conjunto de serviços (servidores e clientes dos serviços).
- ✓ Cliente e servidor rodam em máquinas **distintas**.
- ✓ Requer uma estrutura de rede para clientes acessarem os servidores remotamente.
- ✓ Clientes devem saber quais serviços e servidores estão disponíveis.
- ✓ Servidores não conhecem seus clientes.
- ✓ Baseado no protocolo *request-reply*:
 - Um cliente faz um pedido ao servidor e espera pela resposta.
 - O servidor executa o serviço e responde ao cliente.
- ✓ É importante destacar que **tiers** se referem a camadas **físicas**.

➤ Exemplo:



➤ Vantagens:

- ✓ A distribuição de dados é fácil e direta.
- ✓ Possibilita hardware mais barato (clientes leves).
- ✓ Facilidade em adicionar novos servidores ou atualizar servidores existentes.

➤ Desvantagens:

- ✓ Pode haver redundância de serviços em diferentes servidores.
- ✓ Não prevê um registro central de serviços, ou seja, os clientes devem saber onde estão os servidores e quais serviços eles disponibilizam.

➤ Existem dois tipos possíveis de cliente:

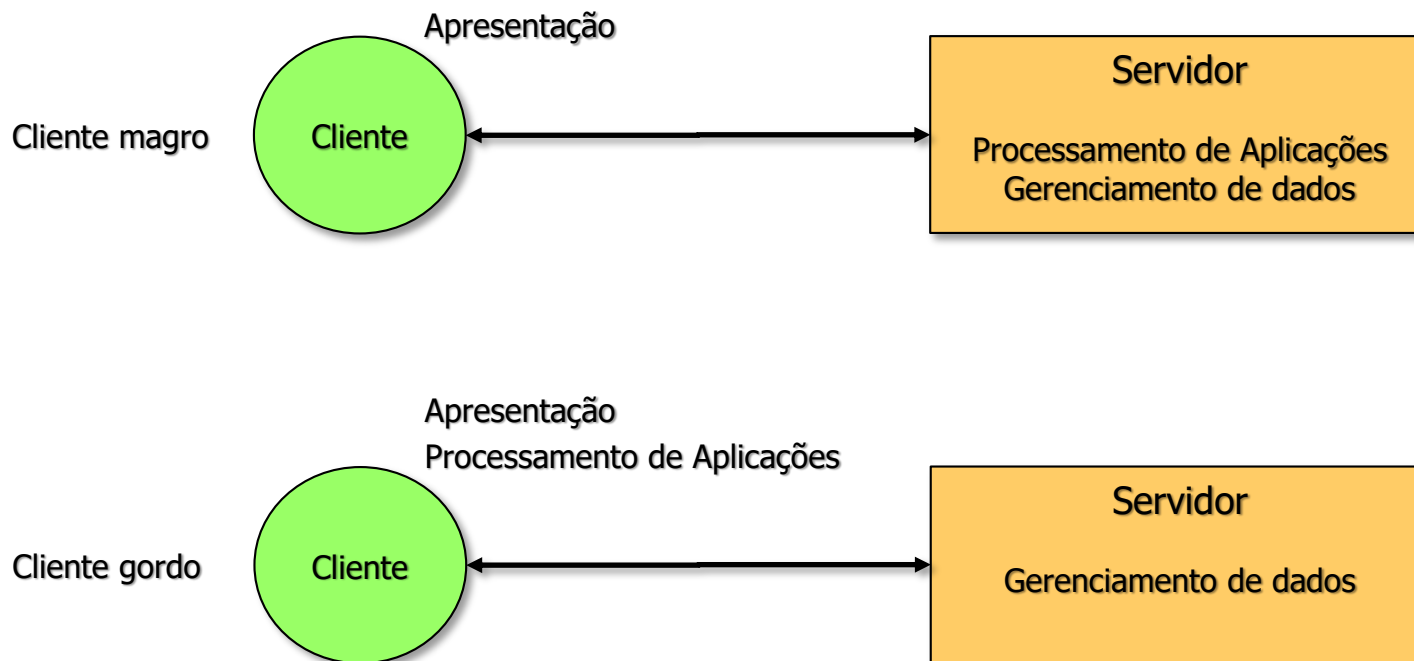
1. Thin Client (cliente magro)

- ✓ O cliente é responsável apenas pela apresentação dos dados.
- ✓ Todo o processamento da aplicação (lógica do negócio) e gerenciamento de dados são realizados no servidor.

2. Fat Client (cliente gordo)

- ✓ O cliente implementa a lógica do negócio e a apresentação.
- ✓ O servidor é responsável apenas pelo gerenciamento de dados.

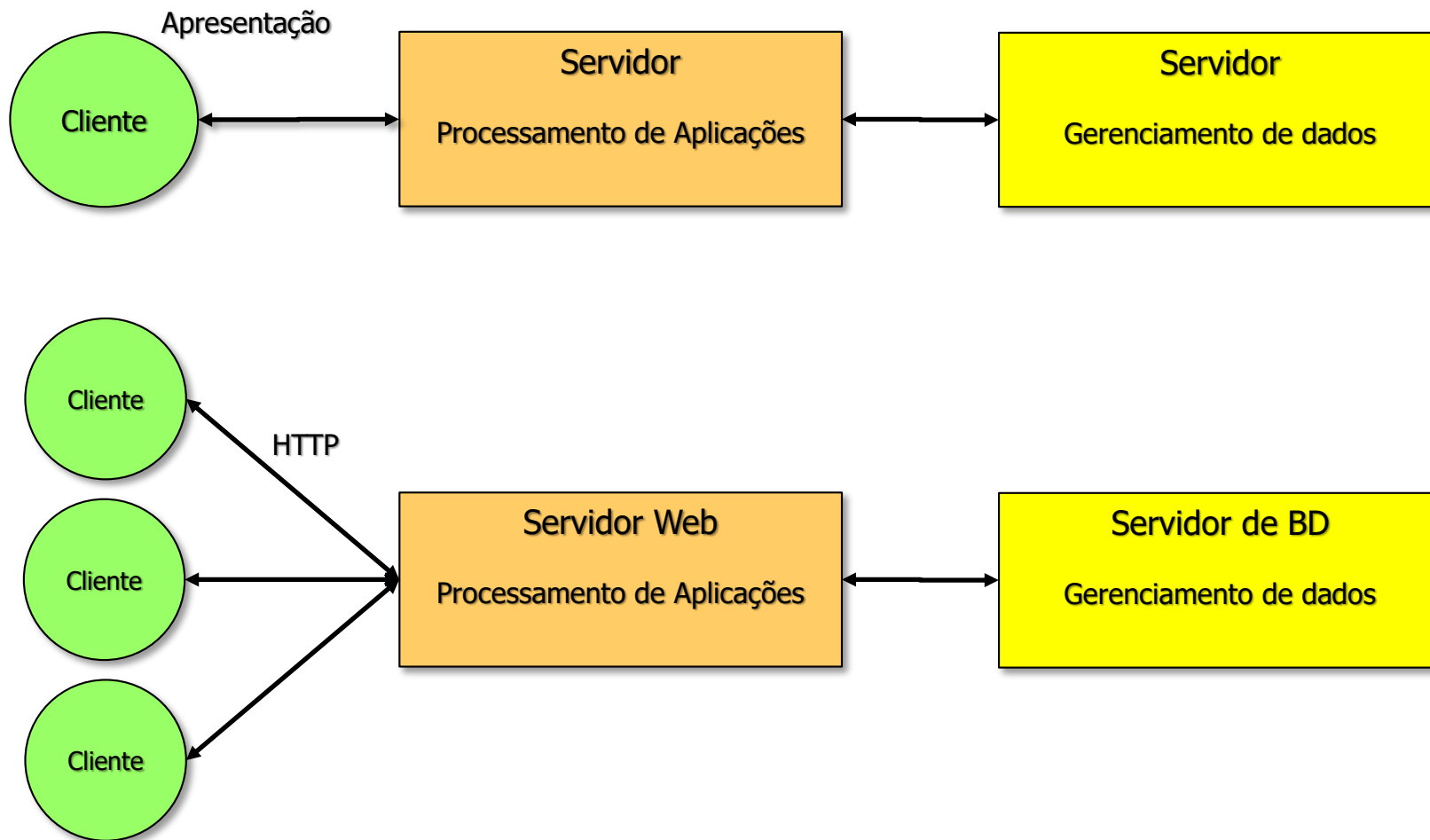
- Existem dois tipos possíveis de cliente:



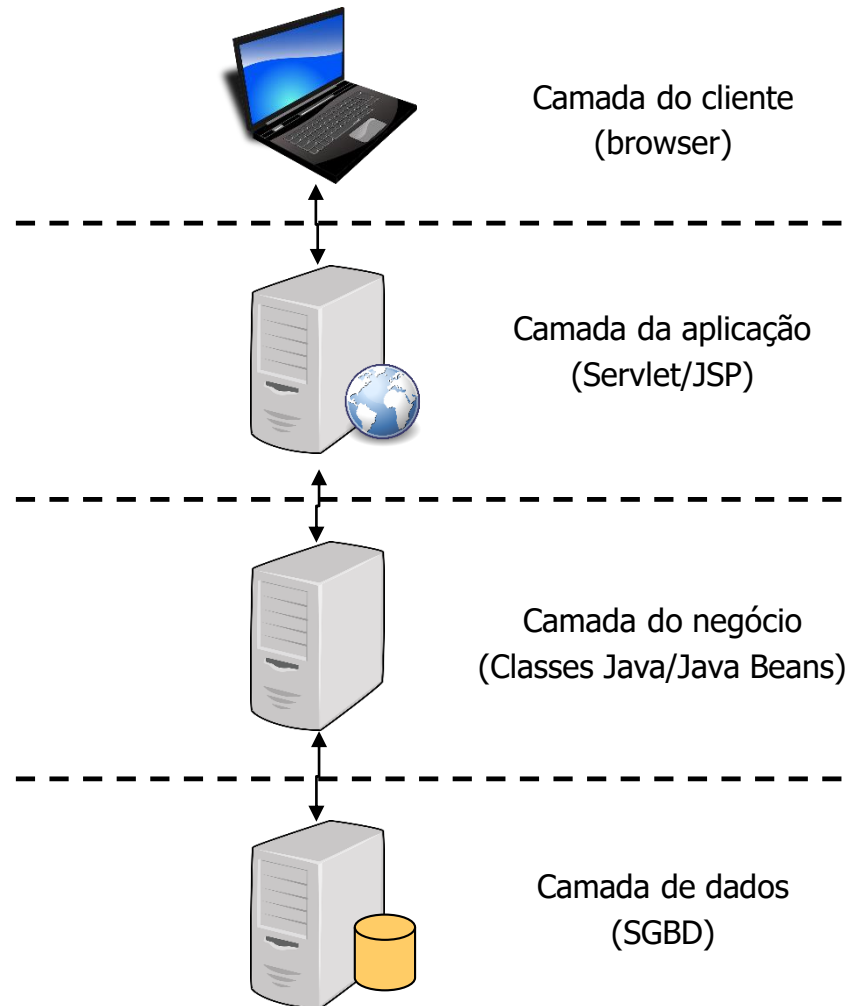
- Existem dois tipos possíveis de cliente:
- Desvantagens do *Thin Client*:
 - ✓ Impõe maior carga de processamento no servidor.
 - ✓ Tende a ter alto tráfego na rede entre o cliente e o servidor.
- Desvantagens do *Fat Client*:
 - ✓ Gerenciamento mais complexo da aplicação.
 - ✓ Uma alteração nas regras de negócio requer atualizações em todos os clientes.

- A arquitetura Cliente-Servidor tem a desvantagem de organizar **três camadas lógicas** (apresentação, negócio e dados) em **duas camadas físicas** (cliente e servidor).
- Assim, ela evoluiu para uma arquitetura multicamadas (3 ou mais):
 - ✓ Separa as lógicas de apresentação, negócio e dados em máquinas distintas.

➤ Exemplo de arquitetura 3-Tier:



➤ Exemplo de arquitetura 4-Tier:



- Vantagens:
 - ✓ Melhor balanceamento de carga entre as diversas camadas.
 - ✓ Aumenta a escalabilidade: fácil adicionar novos servidores de aplicação quando o número de usuários aumenta.
- Desvantagens:
 - ✓ Maior complexidade no desenvolvimento para um número maior de camadas (4 ou mais).
 - ✓ Quanto maior o número de camadas, maior o *overhead* de comunicação entre elas.

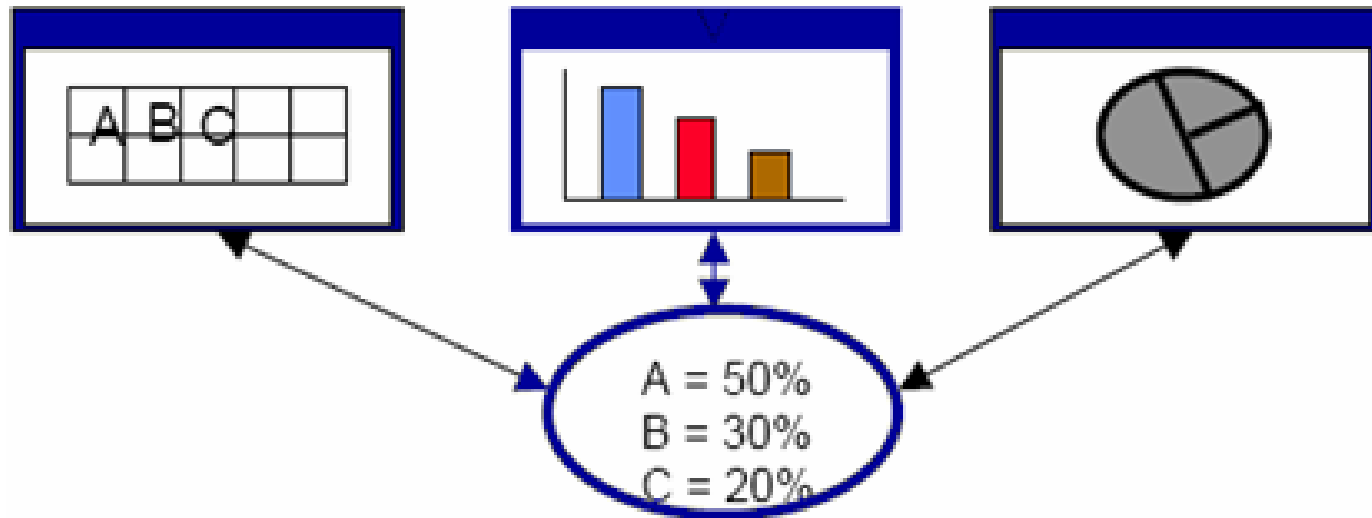
➤ Características:

- ✓ Divide uma aplicação interativa em 3 partes:
 - **Model**: encapsula os dados e as funcionalidades do negócio e é independente de uma apresentação específica.
 - **View**: apresenta informações do Model ao usuário.
 - **Controller**: trata a entrada do usuário.
- ✓ Podem existir múltiplas Views para um mesmo Model.
- ✓ Cada View está associada a um único Controller.
- ✓ Juntos, Views e Controllers formam a interface com o usuário.

➤ Onde usá-lo?

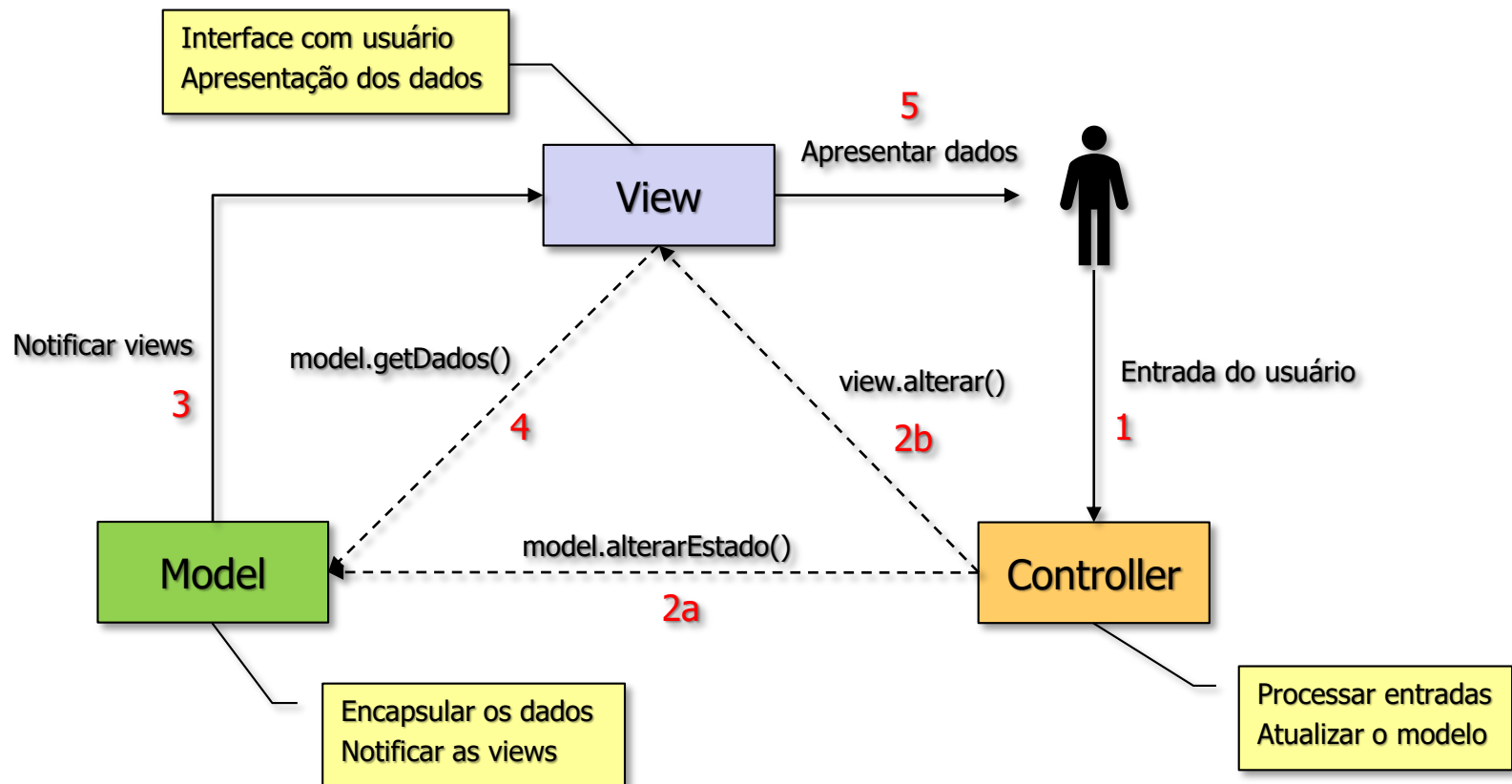
- ✓ Em sistemas interativos, quando existem várias maneiras de visualizar e interagir com os dados.
- ✓ Quando os requisitos de interação com os usuários são voláteis.
- ✓ Aplicações que requerem interfaces em diversas plataformas distintas de hardware/software.

- Exemplo: um Model com 3 Views distintas. Note que a mesma informação (Model) é apresentada de 3 formas diferentes (Views).



- O MVC estabelece um mecanismo para propagação das mudanças de forma a manter a consistência entre Model e View:
 1. Os Controllers recebem a entrada do usuário (geralmente eventos).
 2. Eventos são traduzidos em requisições de serviço que são enviadas para o Model ou para a View.
 3. Após receber a requisição e alterar seu estado interno o Model notifica todas as Views a ele conectadas.
 4. As Views notificadas recuperam os dados do Model e atualizam as informações para o usuário.

- O MVC estabelece um mecanismo para propagação das mudanças de forma a manter a consistência entre Model e View:



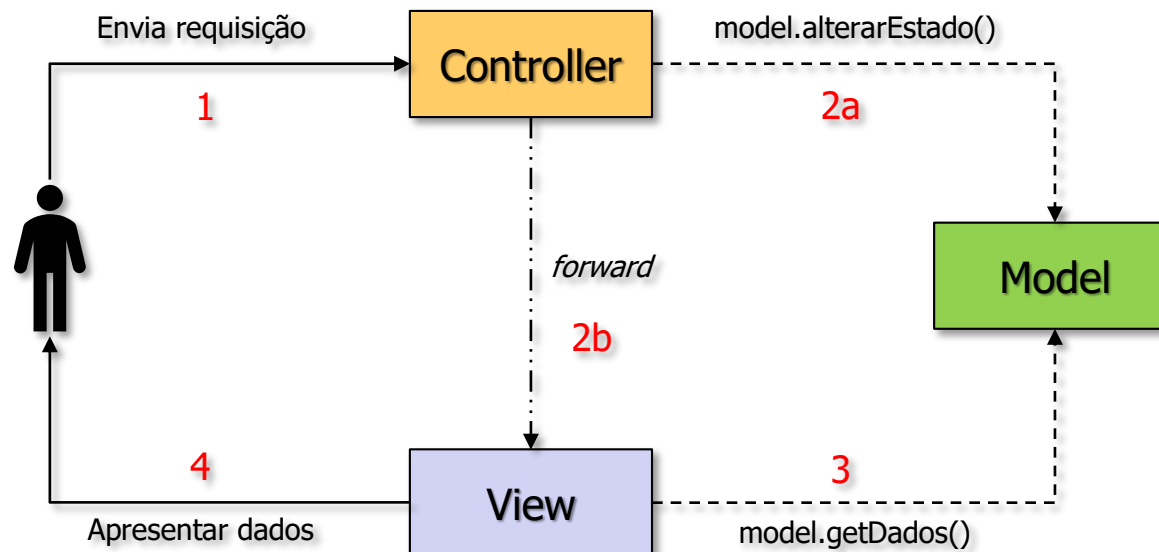
➤ Vantagens:

- ✓ Permite diferentes padrões/estilos de interação sem alterar o núcleo da aplicação.
- ✓ Permite a visualização da mesma informação em formatos distintos, porém sincronizados.
- ✓ Permite a alteração de Views/Controllers em "tempo de execução".

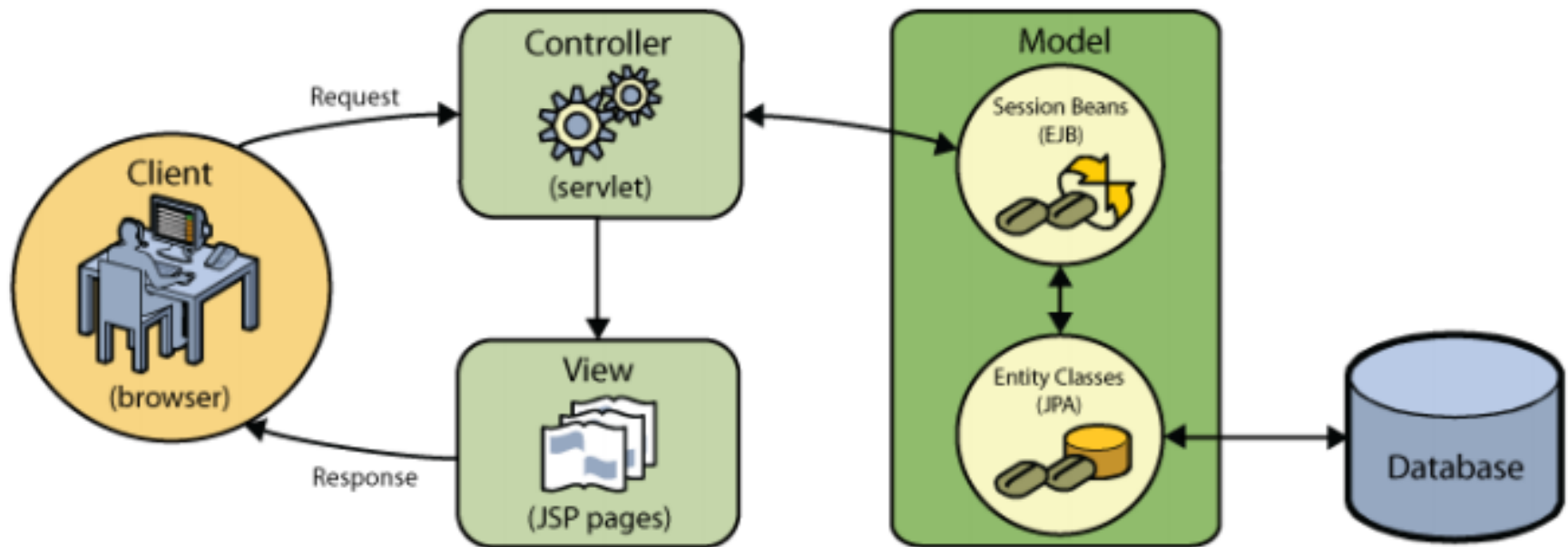
➤ Desvantagens:

- ✓ Maior complexidade quando o modelo de dados e de interações é muito simples.
- ✓ Views e Controllers fortemente interconectados.

- O MVC apresentado anteriormente é o modelo **clássico**.
- Entretanto, algumas modificações nesse modelo foram feitas para adequá-lo ao desenvolvimento de **aplicações Web**.



- MVC usado no desenvolvimento de aplicações Web com JSP/Servlet/EJB/JPA.



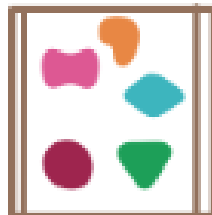
- O padrão MVC é adotado por vários *frameworks* de desenvolvimento:
 - ✓ Struts (Java)
 - ✓ JSF (Java)
 - ✓ Spring MVC (Java)
 - ✓ ASP.NET MVC (C#)
 - ✓ Yii (PHP)
 - ✓ Laravel (PHP)
 - ✓ Zend (PHP)
 - ✓ Django (Python)
 - ✓ Ruby on Rails

- Características:
 - ✓ Vem ganhando popularidade nos últimos anos.
 - ✓ Consiste em dividir a aplicação em um conjunto de pequenos serviços independentes que se comunicam usando mecanismos "leves", ou seja, com pouco *overhead* de comunicação.
 - ✓ Normalmente o mecanismo de comunicação adotado é o HTTP/REST com JSON.

- Onde aplicá-lo?
 - ✓ Sistemas onde a escalabilidade seja uma característica importante.
 - Essa arquitetura permite quebrar o aspecto monolítico do sistema e distribuir os serviços em diversos servidores.
 - ✓ Família de sistemas onde é necessário compartilhar serviços entre eles, ou seja, é necessário implementar os mesmos serviços em uma família de sistemas.

➤ Exemplo:

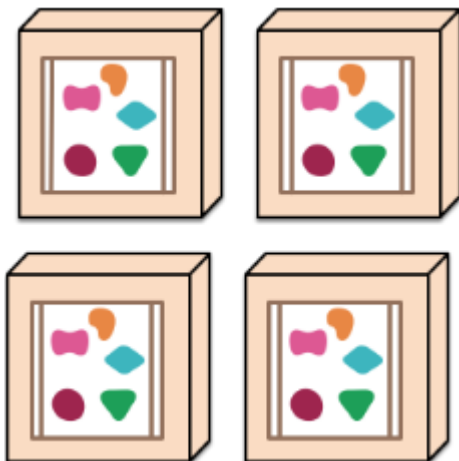
Um sistema monolítico implementa todas as funcionalidades em um único processo...



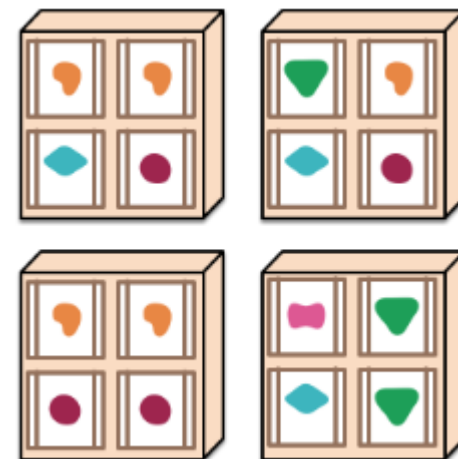
Uma arquitetura de microserviços cada funcionalidade é implementada em um serviço distinto...



...e escala esse sistema replicando o monolito em diversos servidores.



...e escala esse sistema distribuindo os serviços em diversos servidores, replicando onde necessário.



➤ Vantagens:

- ✓ Desenvolvimento independente dos serviços.
- ✓ Uso de diferentes linguagens de programação, apropriadas para o tipo do serviço a ser implementado.
- ✓ Implementação do serviço pode ser mudada sem impacto no restante da aplicação.
- ✓ Maior escalabilidade.

➤ Desvantagens:

- ✓ Desenvolvedores tem que lidar com tolerância a falhas e latência da rede.
- ✓ Esforço extra no desenvolvimento para implementar a comunicação entre os serviços.
- ✓ Não há uma regra de como dividir o sistema em serviços.
- ✓ Dependência entre serviços pode aumentar a complexidade.

➤ Características:

- ✓ Adequado para sistemas que devem tratar eventos gerados externamente.
- ✓ O momento em que os eventos ocorrem é totalmente assíncrono, ou seja, está fora do controle dos componentes que os processam.
- ✓ Eventos são transmitidos a todos os componentes interessados em processá-los.

➤ Componentes:

- ✓ **Publisher (Publicador)**: serviço que publica (disponibiliza) mensagens aos assinantes pelo canal de comunicação, por intermédio de uma interface. O envio de mensagens ocorre somente aos assinantes previamente registrados.
- ✓ **Subscriber (Assinante)**: serviços que assinam (recebem) mensagens do canal de comunicação, após o registro nos publicadores.

➤ Componentes:



Eventos são gerados assincronamente por componentes diversos.

Assinantes **se inscrevem** no Publicador e passam a ser notificados dos eventos que ocorrem.

➤ Onde usá-lo?

- ✓ Sistemas heterogêneos com redes dinâmicas (clientes se conectam/desconectam da rede a todo o instante).
- ✓ Sistemas em rede reativos, ou seja, devem reagir a eventos gerados remotamente.

➤ Vantagens:

- ✓ É efetivo na integração de sistemas que executam em computadores diferentes em uma rede (interoperabilidade).
- ✓ Permite tratar eventos assíncronos, ou seja, eventos para os quais não há uma ordem ou momento definidos.

➤ Desvantagens:

- ✓ O Publicador deve ser projetado cuidadosamente, caso contrário se a quantidade de Assinantes aumentar consideravelmente podem ocorrer problemas de escalabilidade.



RSS



Twitter



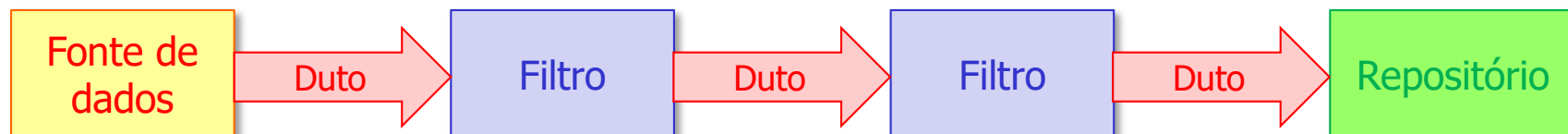
Whatsapp

➤ Características:

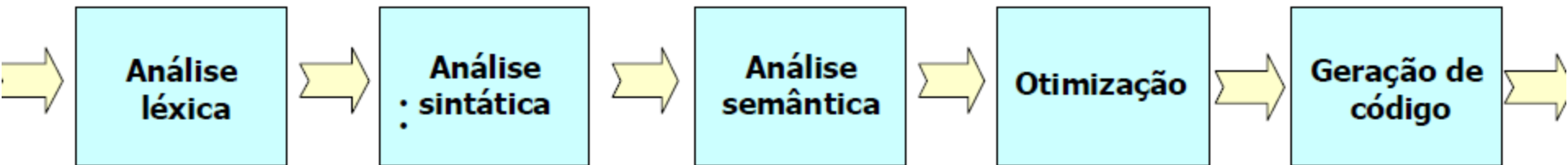
- ✓ Tem como origem o sistema UNIX e o projeto de compiladores.
- ✓ Provê uma estrutura para sistemas que processam fluxos de dados.
- ✓ Cada passo do processamento é encapsulado em um componente chamado Filtro.
- ✓ Dados são passados entre Filtros por conectores chamados de Dutos.
- ✓ Recombinando-se Filtros é possível construir-se famílias de sistemas relacionados.



- Quando usá-lo?
 - ✓ Contexto: sistemas que processam cadeias de dados.
 - ✓ Problema: processar uma cadeia de dados, mas implementar esse processamento de uma só vez é inviável.
 - ✓ Solução: dividir a tarefa em uma sequência de etapas.



- Exemplo: compilador



Projeto de Software Orientado a Objetos

3ª parte

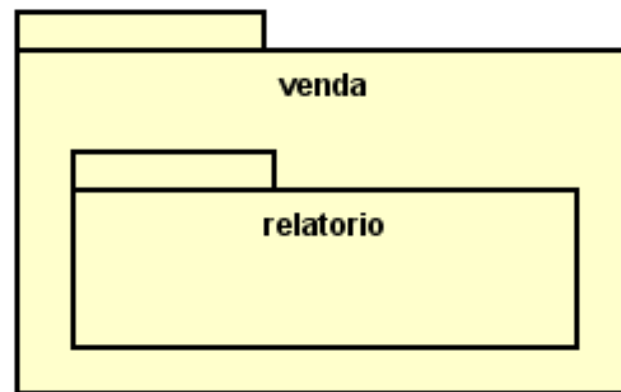
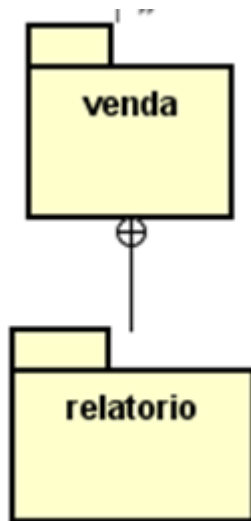
Prof. Jobson Massollar
jobson.silva@uva.br

- Um dos modelos mais usados para documentar arquiteturas é o **modelo 4+1** (Kruchten, 1995).

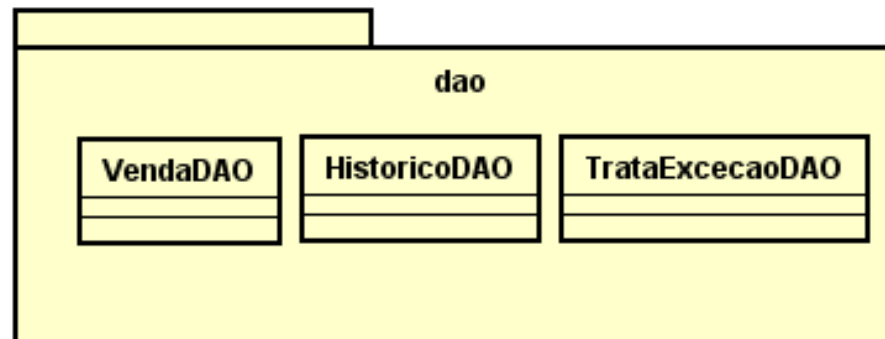


- Visão de Desenvolvimento ou Implementação:
 - ✓ Usada para descrever subsistemas, módulos, bibliotecas, pacotes ou componentes que compõem o sistema.
 - ✓ É importante destacar que subsistemas, módulos, bibliotecas, pacotes ou componentes são elementos **mais abstratos** do que classes, funções ou rotinas, normalmente encontrados nas linguagens de programação.

- Um **pacote** (*package*) em UML é um mecanismo de agrupamento genérico (lógico ou físico).
- Em um pacote podemos agrupar classes, componentes, nós de uma infraestrutura de hardware, dentre outros.
- É comum usar o diagrama de pacotes em conjunto com outros diagramas (classes, componentes, implantação, etc.) para mostrar como esses elementos são agrupados.
- Além disso, um pacote pode conter outros pacotes.

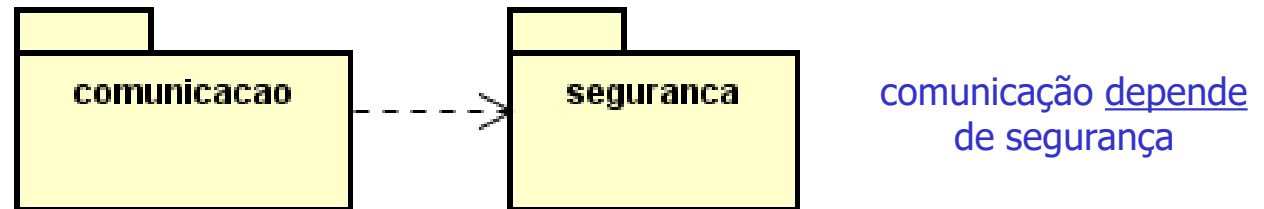


- Um **pacote lógico** é um agrupamento lógico de classes e relações entre essas classes.
- Nesse caso, o pacote representa um conjunto de classes que possui uma responsabilidade em comum de interesse para o sistema.
- Corresponde ao conceito de *package* no Java ou *namespace* no C++ e C#.
- Um pacote forma um **espaço de nomes**: `dao::VendaDAO`

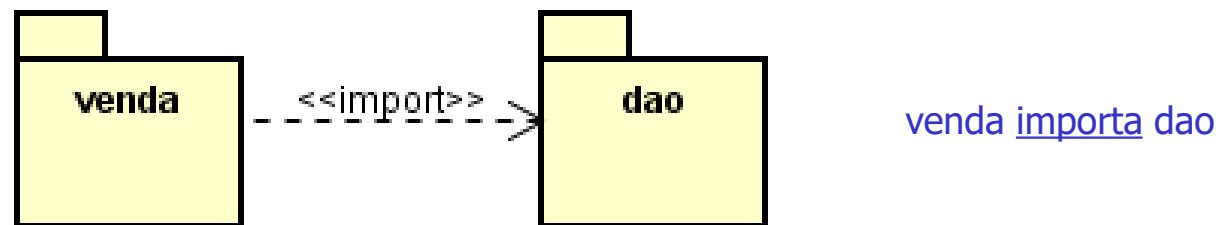


- Um pacote pode se relacionar com outros pacotes. Os relacionamentos mais comuns são:

- ✓ **Dependência simples:** o pacote A depende, de alguma forma, do pacote B.

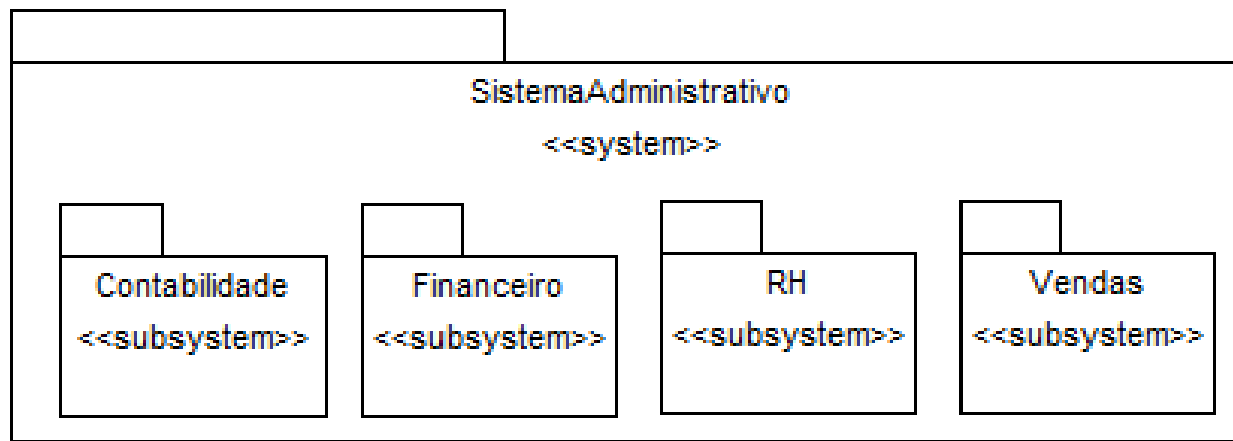


- ✓ **Dependência de «import»:** o pacote A importa os elementos públicos do pacote B.

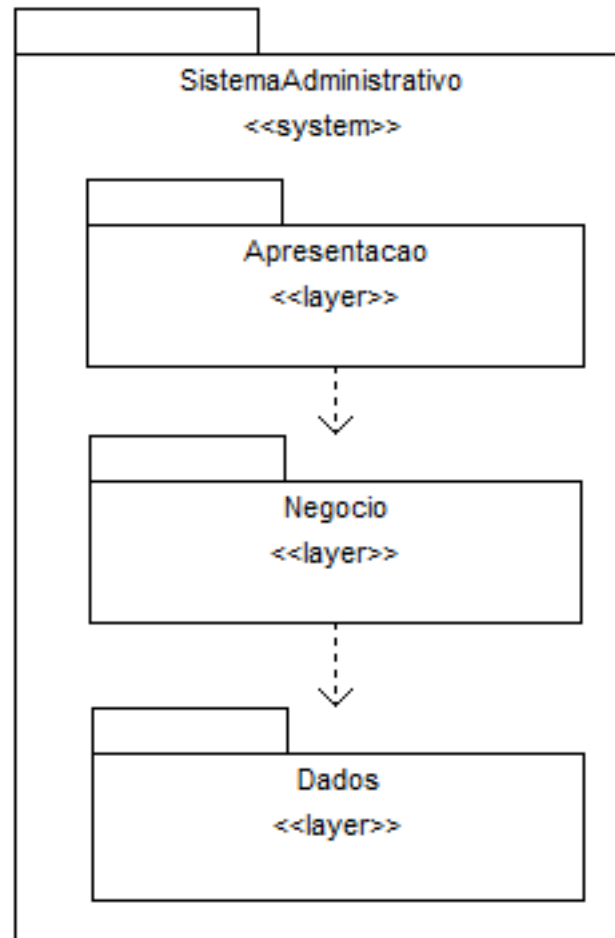


- Alguns estereótipos para pacotes:
 - ✓ «system» - pacote que representa o sistema completo que está sendo modelado.
 - ✓ «subsystem» - pacote que representa uma parte independente do sistema que está sendo modelado. Corresponde normalmente a um corte "vertical".
 - ✓ «layer» - pacote que representa uma camada lógica de um sistema.
 - ✓ «framework» - pacote que representa um conjunto de classes abstratas e concretas concebido para ser estendido, implementando uma funcionalidade específica.
 - ✓ «facade» (fachada) - pacote sem nenhum elemento (vazio) que representa uma visão sobre outro pacote (não acrescenta funcionalidades, apenas as apresenta de forma diferente).

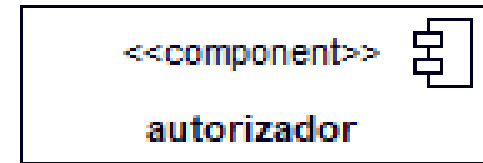
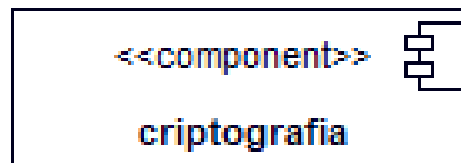
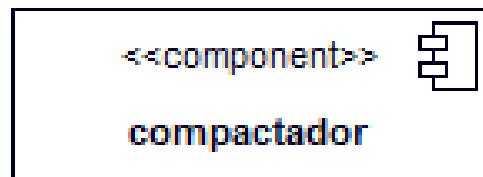
➤ Exemplo 1: Sistema e subsistemas



➤ Exemplo 2: Camadas



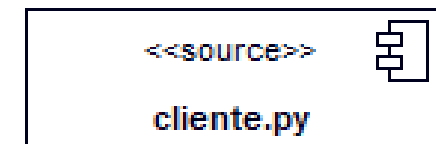
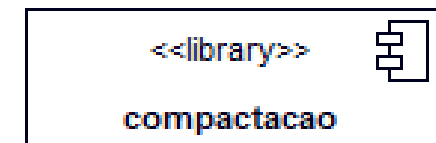
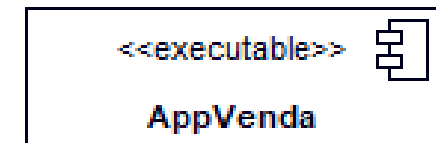
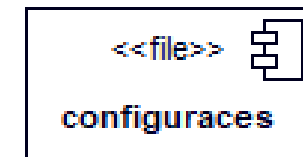
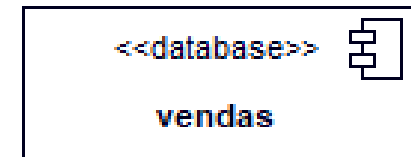
- Um componente é uma parte **reusável** e **substituível** do software.
- Um componente é um **elemento abstrato** e não necessariamente uma única classe.



- **Importante:** existe uma diferença na definição de componente da UML 1.x para a UML 2.x.

- Na UML 1.x, um componente representa uma **estrutura física** (arquivo) que define:

- ✓ Um arquivo executável;
- ✓ Uma biblioteca de classes ou funções;
- ✓ Um arquivo de dados;
- ✓ Um banco de dados;
- ✓ Uma tabela do banco de dados;
- ✓ Um arquivo de configuração;
- ✓ Uma imagem;
- ✓ Um documento;
- ✓ Um arquivo-fonte;
- ✓ etc.

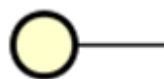


- Na UML 2.x esses elementos são representados como **artefatos**.

- Para que sejam componentes reusáveis e substituíveis eles devem estar **fracamente acoplados**, ou seja, a dependência entre eles deve ser **fraca**.
- Isso vai permitir que um componente seja substituído por outro **sem afetar** os demais componentes.
- Para isso, componentes devem interagir uns com os outros sempre através de **interfaces**.

➤ Existem dois tipos de interface:

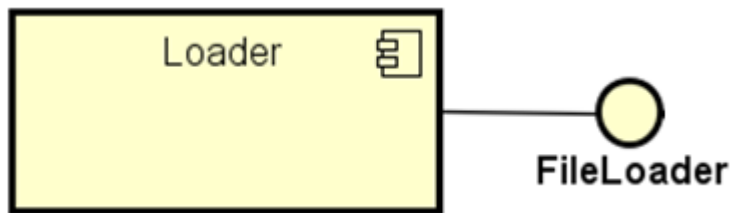
- ✓ **Interface provida**: é o serviço ou informação provida ou oferecida por um componente. Pode ser representado por uma "bola".



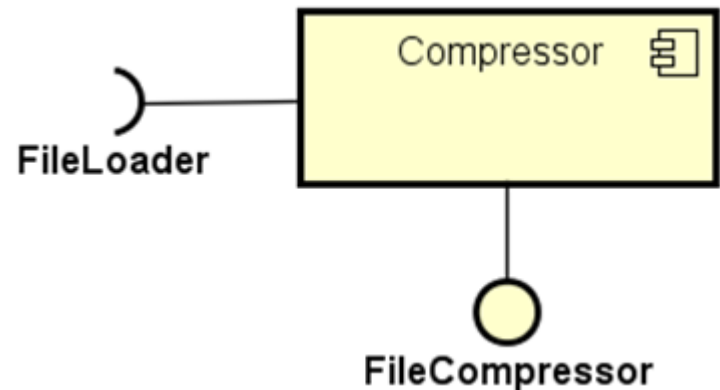
- ✓ **Interface requerida**: é o serviço ou informação que o componente necessita para realizar o seu trabalho. Pode ser representado por um "soquete".



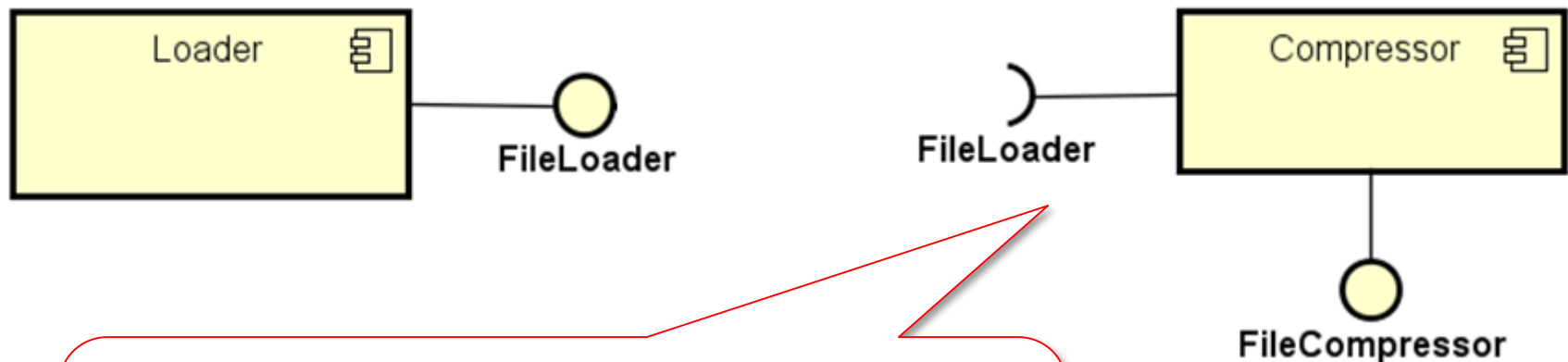
- As interfaces **providas** e **requeridas** ("bola e soquete") devem estar associadas aos componentes.
- Exemplos:



O componente **Loader** é responsável por carregar os dados de um arquivo para a memória. Ele disponibiliza o serviço de carga de um arquivo na interface **FileLoader**.

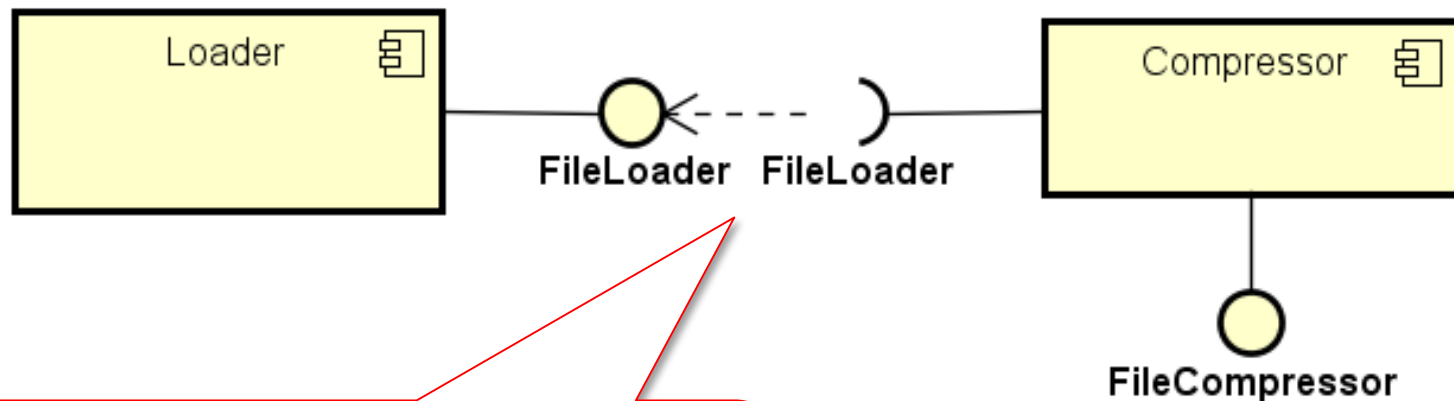


- As interfaces **providas** e **requeridas** ("bola" e "soquete") devem estar associadas aos componentes.
- Exemplos:



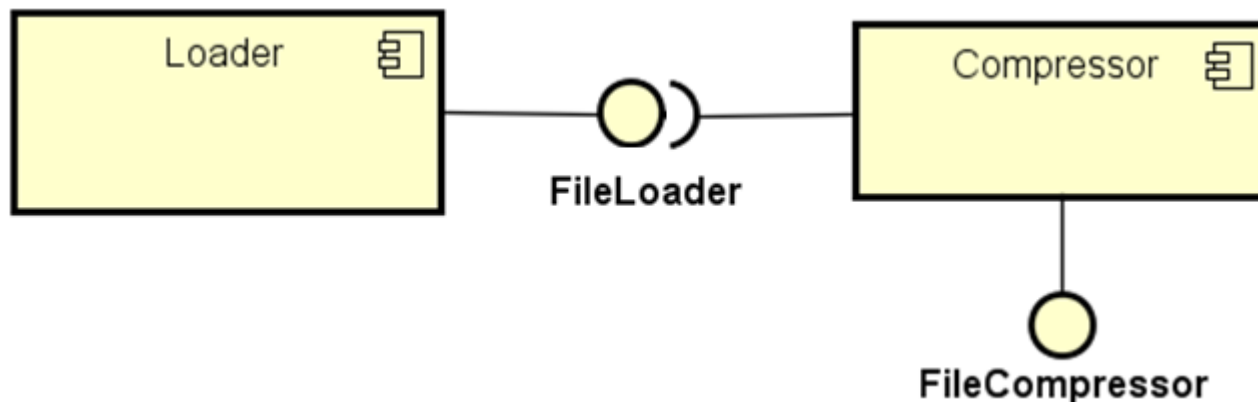
O componente **Compressor** é responsável por compactar dados. Ele disponibiliza o serviço de compactação de **arquivos** na interface **FileCompressor**. Para implementar a compressão, o **Compressor** requer que **outro** componente faça a carga dos dados do arquivo para a memória.

- A ligação entre a interface **provida** e a **requerida** pode ser definida com o relacionamento de **dependência** (seta pontilhada).
- Exemplo:

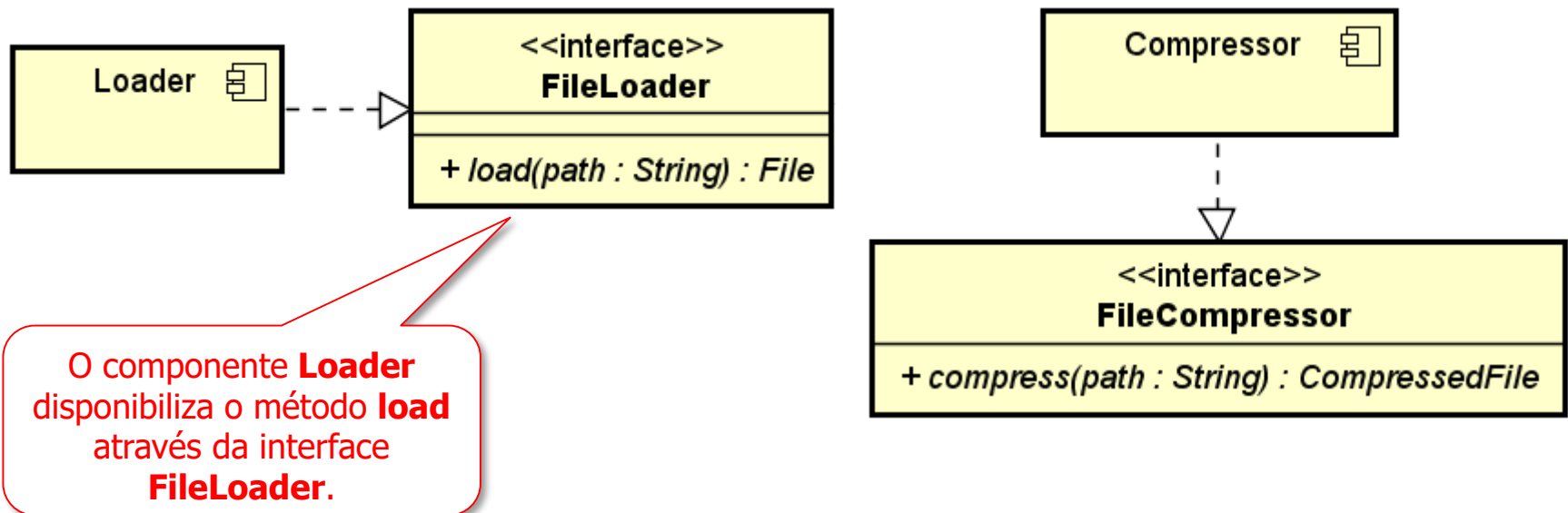


O componente **Compressor** usa o serviço provido pelo componente **Loader**, ou seja, o componente **Compressor** depende do componente **Loader** para carregar o arquivo a ser comprimido.

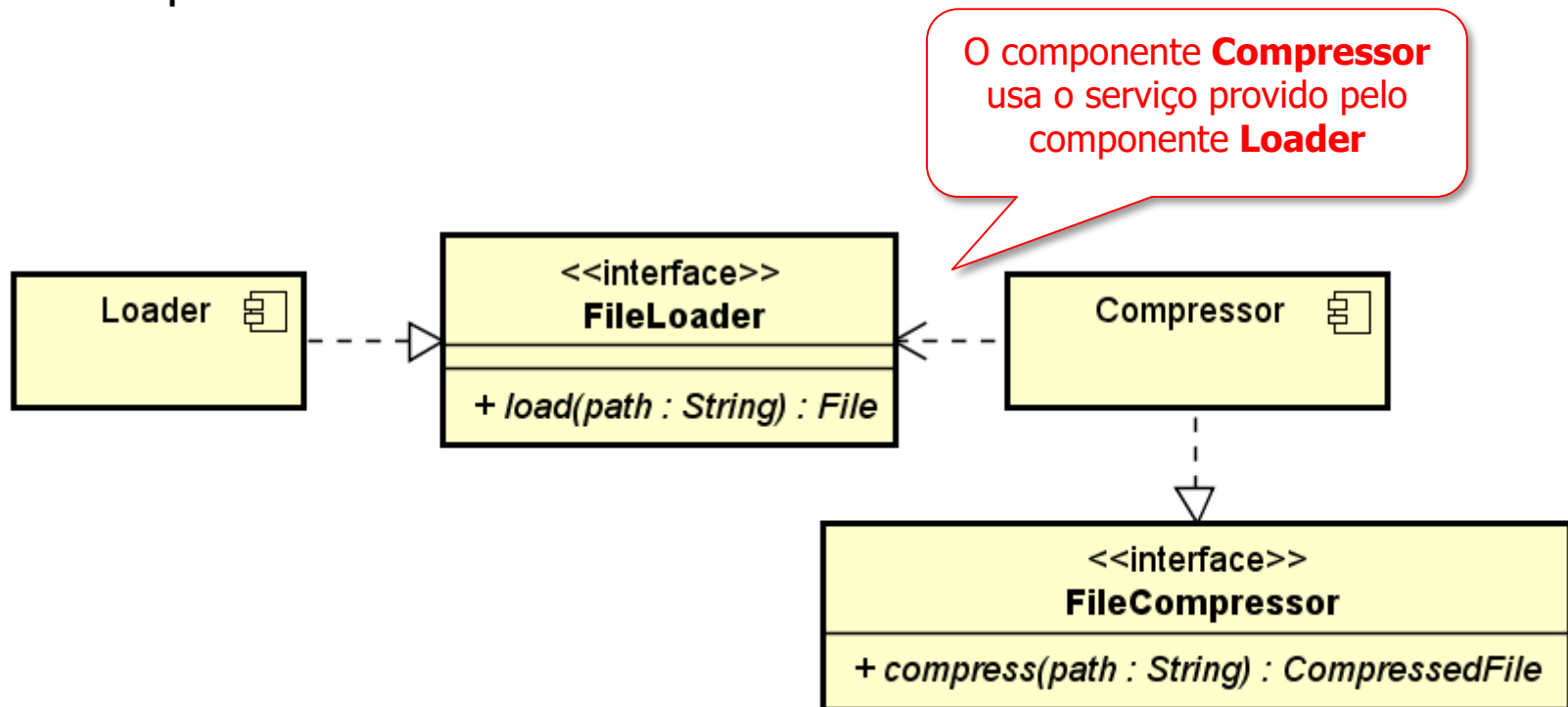
- **Observação:** algumas ferramentas UML permitem a ligação direta da bola com o soquete, sem a necessidade da dependência.
- Exemplo:



- Para **explicitar** exatamente quais serviços estão disponíveis em uma **interface** podemos adotar outra notação.
- Nesse caso, é possível visualizar os **métodos** disponíveis, seus parâmetros e tipo de retorno.
- Exemplo:



- Nessa segunda notação, a ligação entre a interface **provida** e a **requerida** também é definida com o relacionamento de **dependência** (seta pontilhada).
- Exemplo:

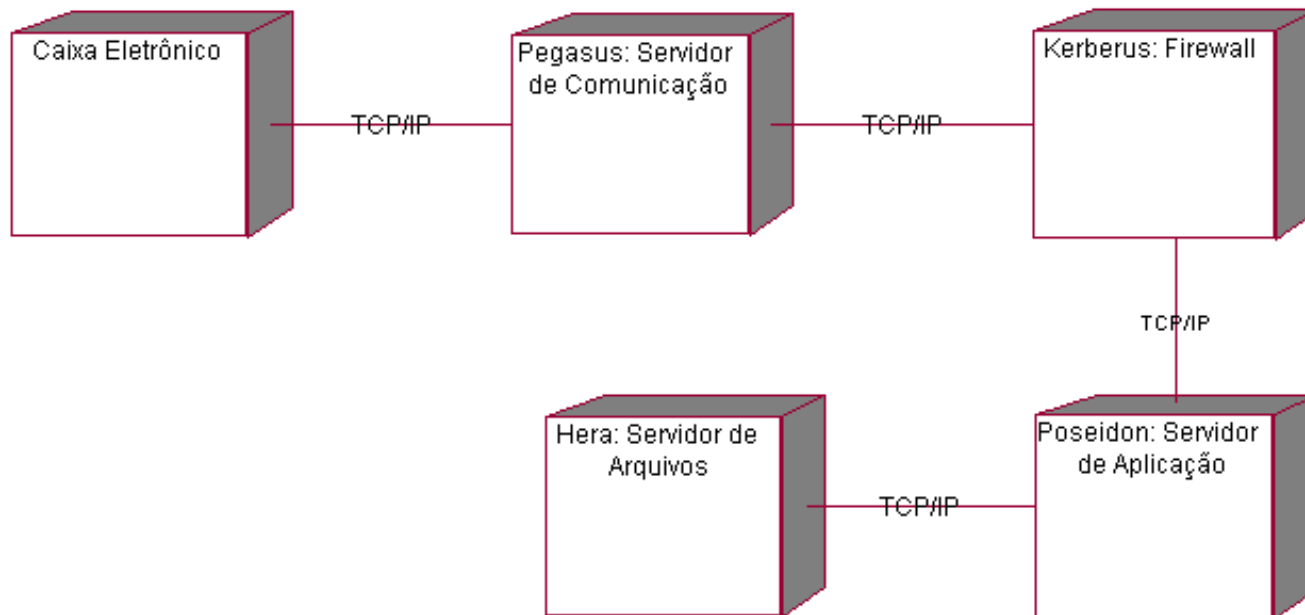


- Visão Física:
 - ✓ Usada para descrever a topologia dos componentes de hardware e software que fazem parte do sistema.

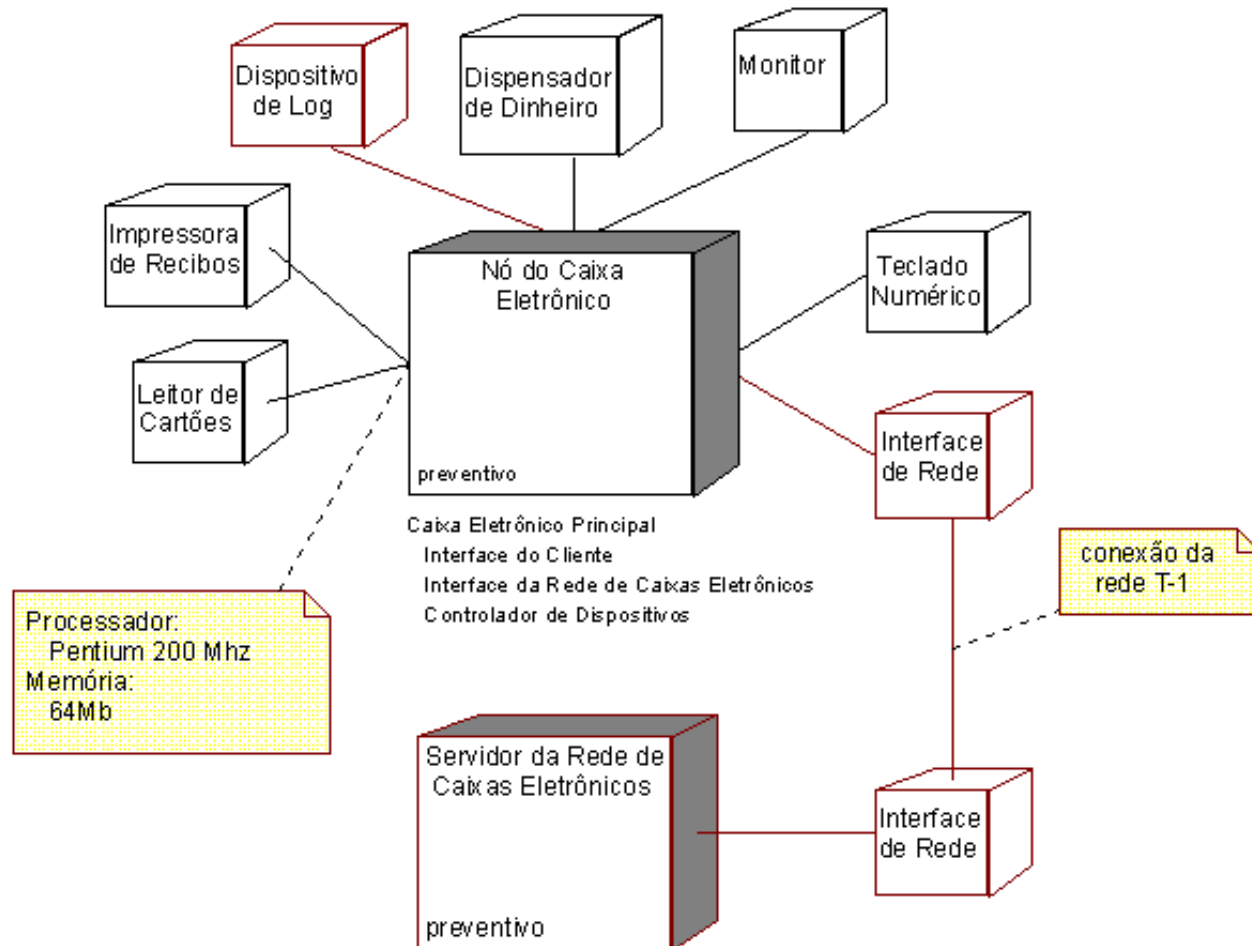
- Mostra o **layout de implantação de um sistema**, revelando quais partes do software são executadas em quais partes do hardware.
- Enfoca a estrutura sobre a qual o software será implantado e executado em termos de **hardware** ou combinando **hardware e software**.
- Define como os **dispositivos de hardware** estarão **conectados** e através de quais **protocolos** elas se comunicarão.
- É muito útil quando o sistema é projetado para ser executado em múltiplos nós.

- Os elementos principais são:
 - ✓ **Nós:** podem ser de dois tipos.
 - **Dispositivos de hardware:** representam recursos computacionais físicos (servidores, dispositivos móveis, notebooks, sensores, impressoras, roteadores, câmeras, leitores biométricos, etc.)
 - **Ambientes de execução de software:** representam recursos computacionais lógicos que oferecem um ambiente de execução para componentes específicos (sistemas operacionais, servidores web, servidores de aplicação, SGBD's, etc.)
 - ✓ **Artefatos:** representam elementos físicos concretos (arquivos executáveis, bibliotecas, arquivos de configuração, etc.)
 - ✓ **Caminhos de comunicação:** associações entre nós que permitem a comunicação, ou seja, a troca de dados, sinais e mensagens entre esses nós.

- Exemplo: dispositivos (nós) interligados em uma rede de comunicação



- Exemplo: dispositivo e seus subcomponentes representados como nós.



- É possível, também, estereotipar os nós e associações do diagrama para definir que **tipo de elemento** ele representa:

1. Estereótipos para dispositivos de **hardware**:

- ✓ «computer»
- ✓ «notebook»
- ✓ «smartphone»
- ✓ «mobile device»
- ✓ «sensor»
- ✓ «pc»
- ✓ «pc client»
- ✓ «pc server»
- ✓ «server»
- ✓ «storage»
- ✓ «disk array»
- ✓ «unix server»
- ✓ «database server»

2. Estereótipos para **ambientes de execução**:

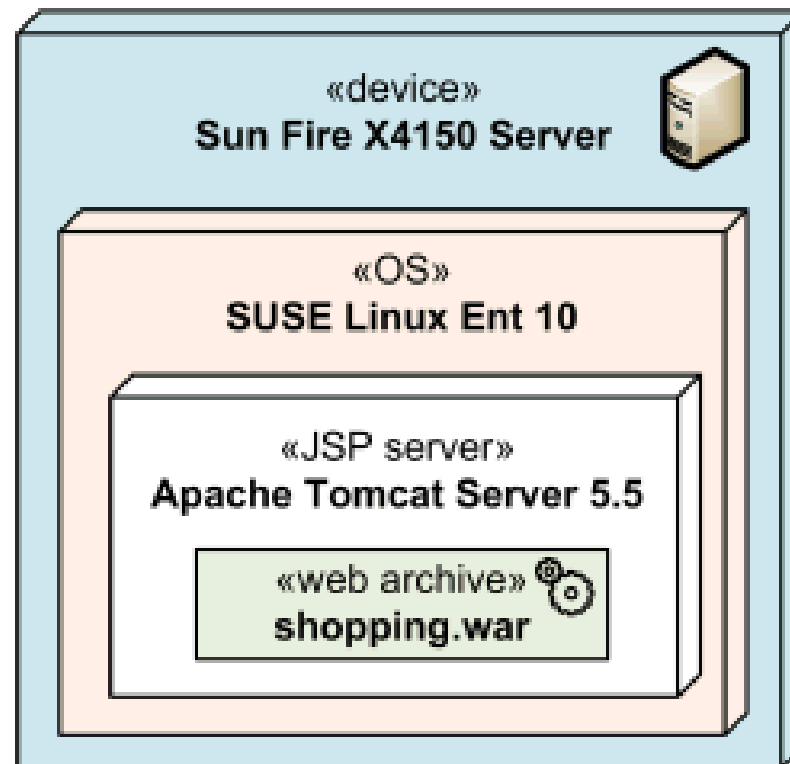
- ✓ «application server»
- ✓ «database server»
- ✓ «schema»
- ✓ «database»
- ✓ «sgbd»
- ✓ «OS»

3. Estereótipos para **associações**:

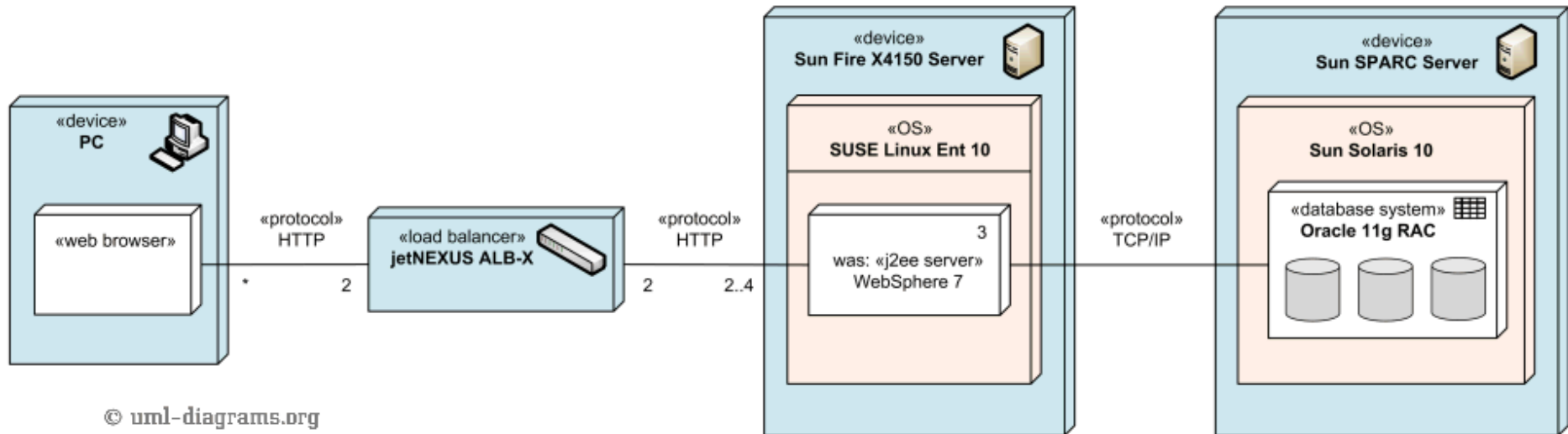
- ✓ «protocol»
- ✓ «http»
- ✓ «tcp/ip»
- ✓ «3G»

- Assim como os demais diagramas da UML, o diagrama de implantação pode representar a arquitetura do sistema sob diversas perspectivas.
- Três perspectivas comuns são:
 - ✓ **Diagrama de arquitetura de rede:** apresenta os dispositivos de hardware e os links de comunicação entre eles.
 - ✓ **Diagrama de implantação no nível de especificação:** apresenta a combinação entre os dispositivos de hardware e os componentes de software, sem especificar as instâncias dos nós.
 - ✓ **Diagrama de implantação no nível de instância:** apresenta a combinação entre os dispositivos de hardware e os componentes de software, especificando as instâncias nos nós.

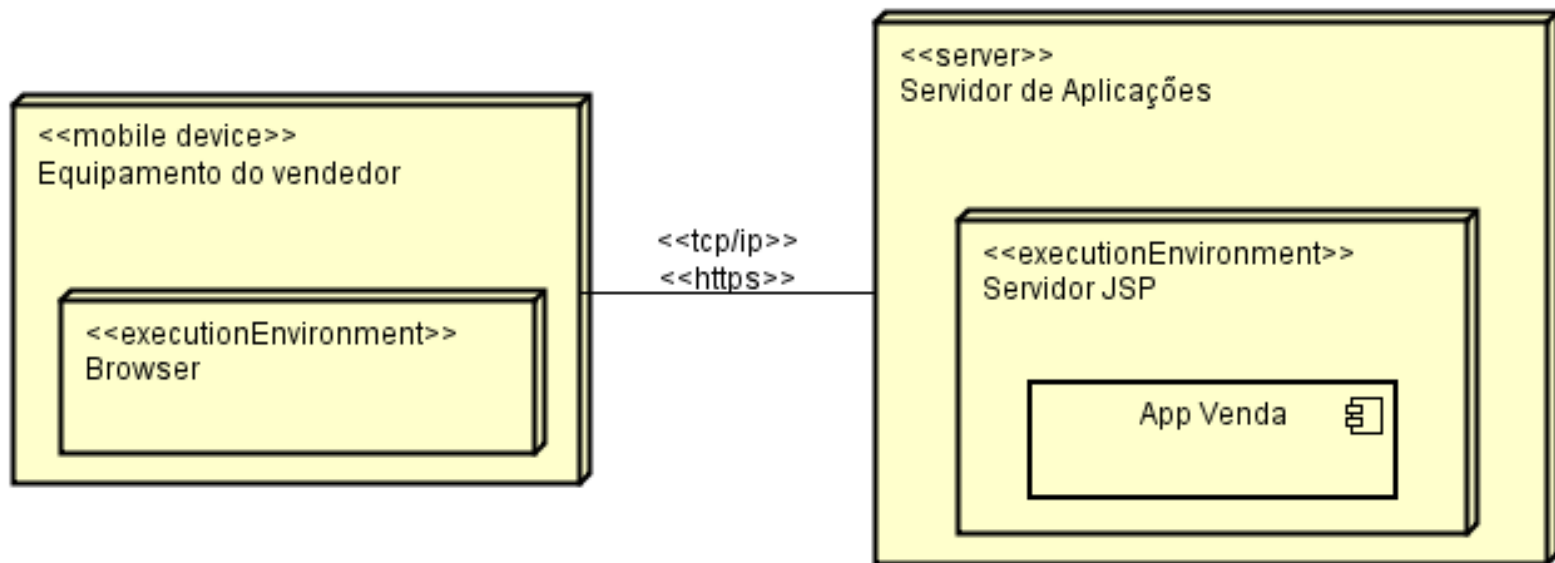
- Exemplo: dispositivo de hardware e seus ambientes de execução.



- Exemplo: nós físicos + rede de comunicação + composição lógica dos nós.



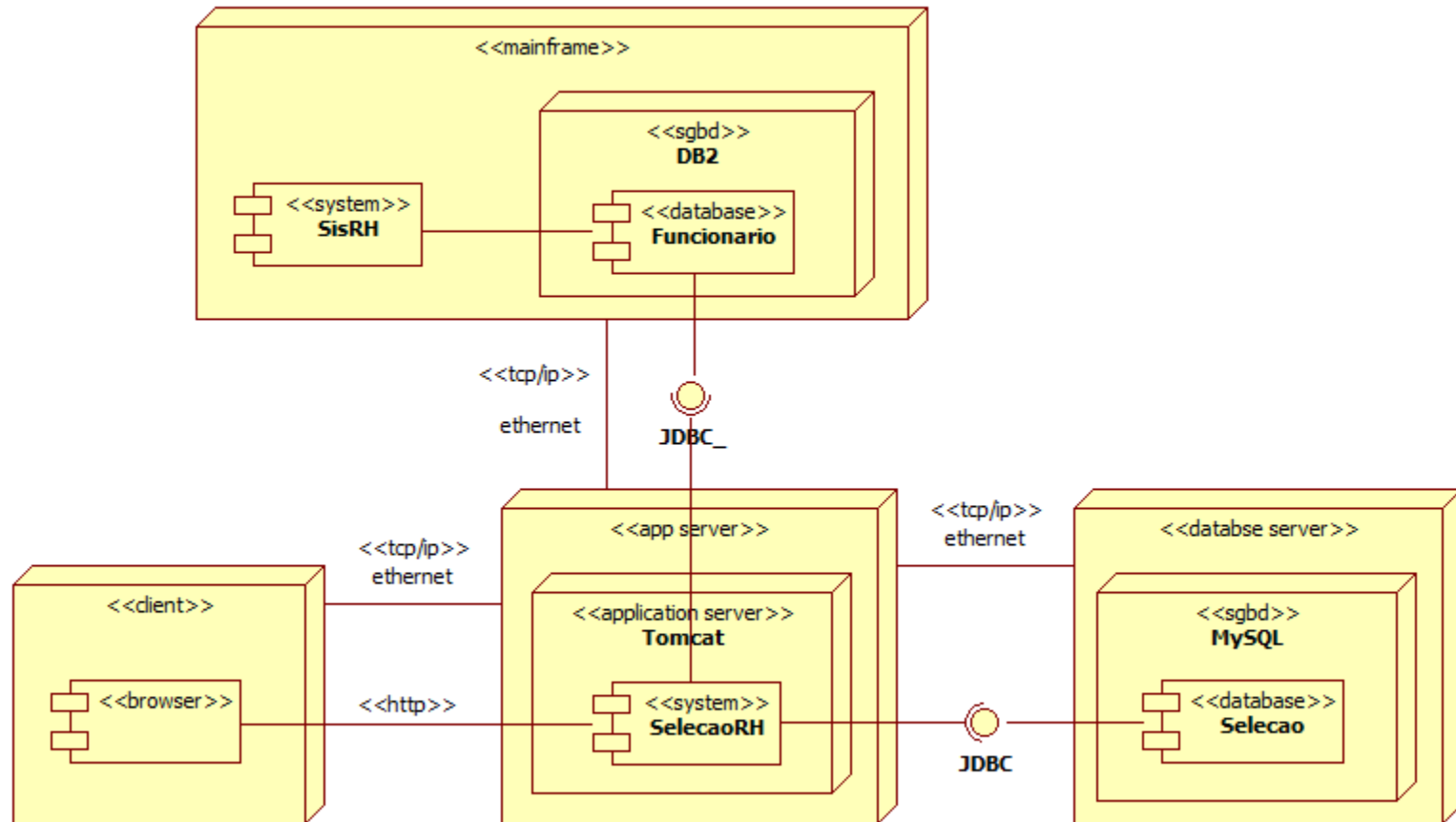
➤ Exemplo:



➤ Solução arquitetural para o seguinte cenário:

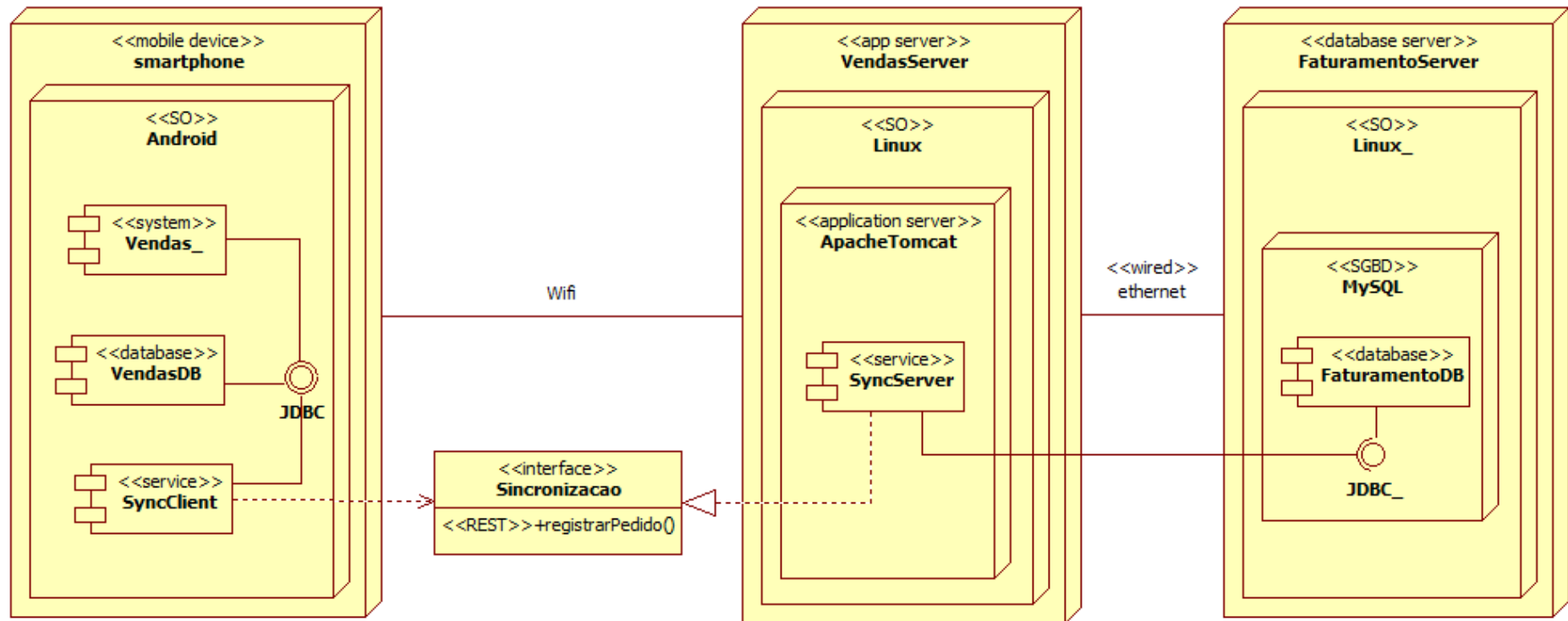
Uma grande empresa de consultoria tributária contratou o desenvolvimento de um sistema de informação para gerenciamento do processo de seleção de novos funcionários chamado SelecaoRH. Este sistema tem como requisito fundamental a plataforma Web e, por isso, será utilizada a linguagem Java e o banco de dados MySQL. Este novo sistema necessita acessar informações de funcionários mantidas por um sistema legado de controle de RH, denominado SisRH, implementado em COBOL com banco de dados DB2.

➤ Diagrama de implantação:



➤ Solução arquitetural para o seguinte cenário:

Uma empresa de venda de bebidas (refrigerantes e cervejas) tem um catálogo com cerca de 20 produtos e pouco mais de 100 vendedores. Cada vendedor visita, por dia, aproximadamente 50 clientes distribuídos por toda a cidade do Rio de Janeiro, Baixada Fluminense e Niterói. O roteiro de visita de cada vendedor é fixo por dia da semana. A empresa gostaria de desenvolver um sistema que permitisse aos vendedores coletar eletronicamente os pedidos dos clientes para que, ao final do dia, esses pedidos fossem faturados e entregues no dia seguinte. Toda a transmissão de dados deve ser feita de forma segura, ou seja, criptografada. Os vendedores trabalham distribuídos por toda a cidade, ou seja, não existe um local único de onde os vendedores partam ou cheguem ao final do dia. Em muitos clientes, principalmente os mais afastados, não há sinal de telefonia celular disponível ou o sinal é muito fraco.



Projeto de Software Orientado a Objetos

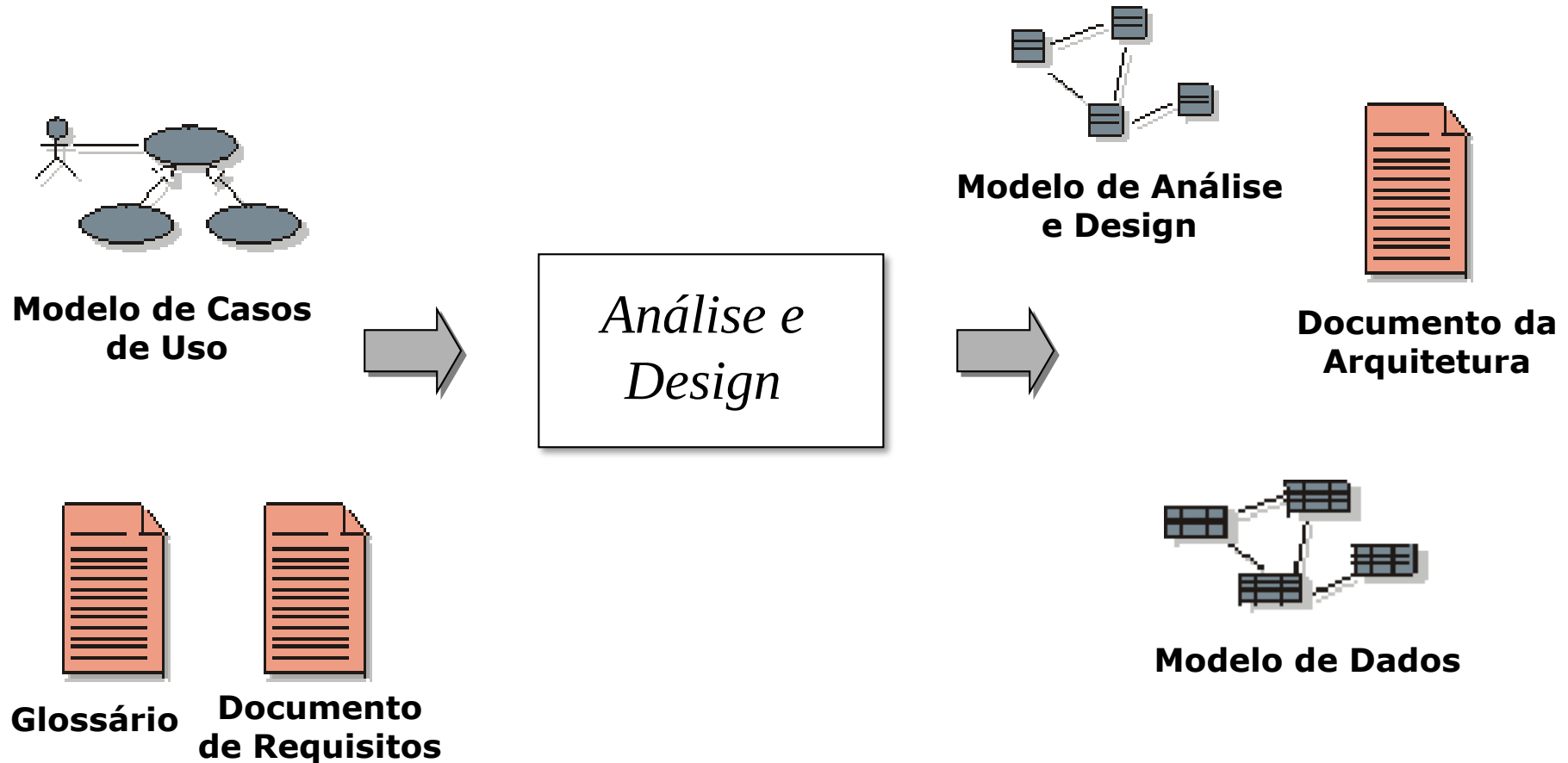
4ª parte

Prof. Jobson Massollar
jobson.silva@uva.br

- Durante a fase de Análise Orientada a Objetos, podemos elaborar diversos modelos de **alto nível** (**modelos abstratos**):
 - ✓ **Modelo de Casos de Uso** (Diagrama de Casos de Uso + Descrição dos Casos de Uso): descrever as funcionalidades do sistema do ponto de vista dos atores.
 - ✓ **Modelo de Domínio** (Diagrama de Classes): representar os conceitos do domínio do problema, suas características e seus relacionamentos.
 - ✓ **Diagrama de Transição de Estados (DTE)**: descrever o comportamento de um único objeto, permitindo a descrição dos eventos e transições de estado aos quais ele está sujeito.
 - ✓ **Diagrama de Atividades**: descrever casos de uso, especificar processos, regras de negócio ou cálculos complexos que o sistema deve implementar.

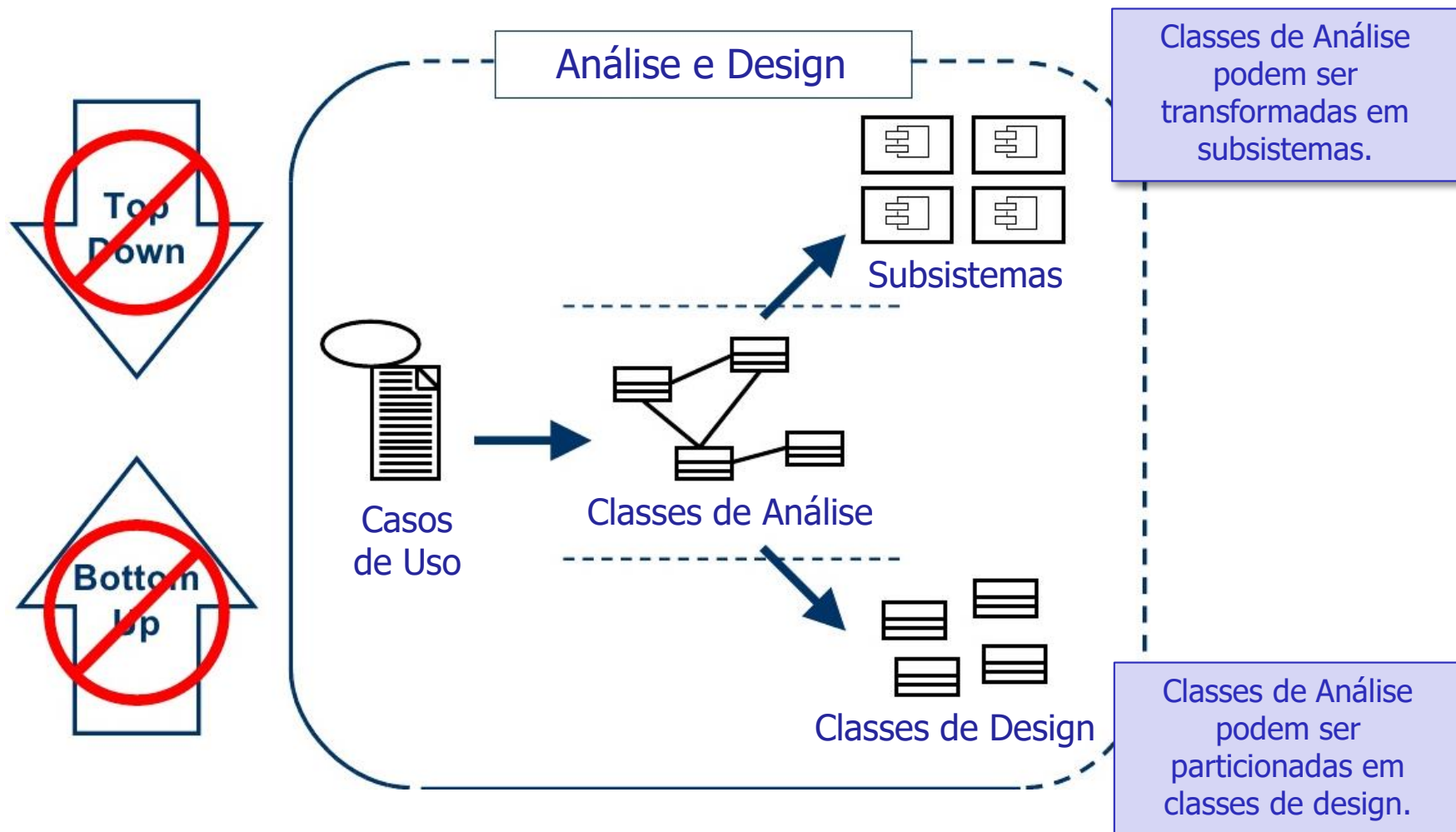
- A disciplina de **Análise e Design do RUP** tem o objetivo de:
 - ✓ Definir uma **arquitetura** robusta para o sistema.
 - ✓ Transformar os requisitos em um **design** do sistema.
 - ✓ Adaptar o **design** para o ambiente de implementação (hardware, sistema operacional, frameworks, SGBD, softwares de apoio, etc.).

- Visão geral os insumos e produtos da *Análise e Design*:



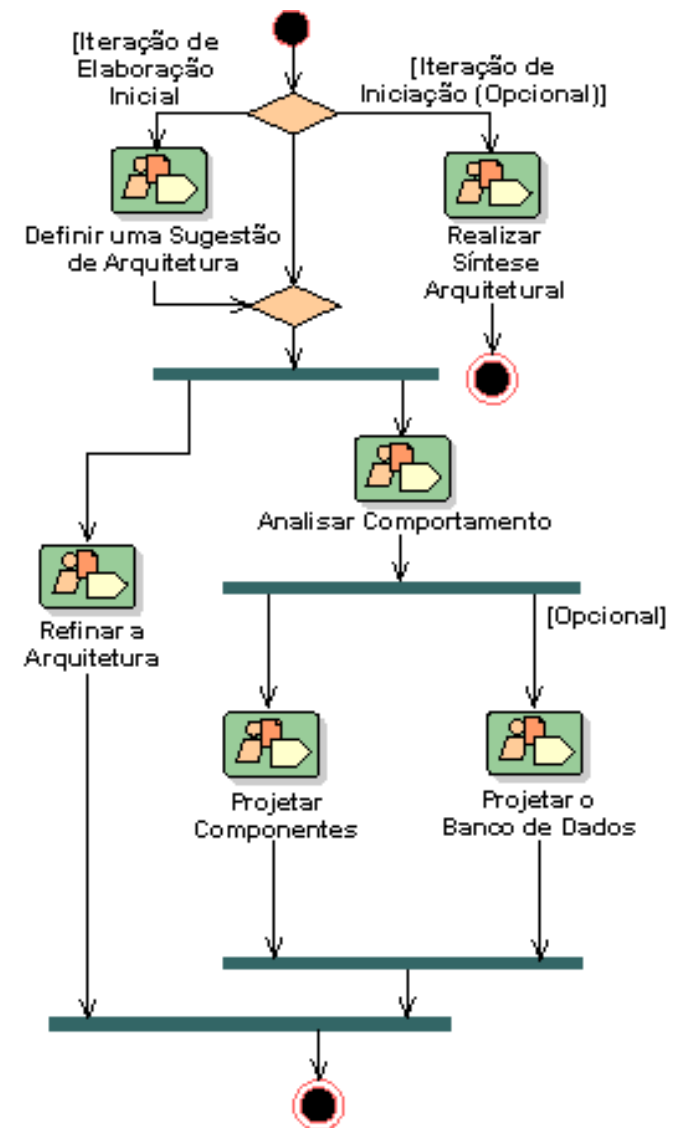
- A construção do **Modelo de Análise e Design** é o principal objetivo desta disciplina no RUP.
- Esse modelo contém as **realizações dos Casos de Uso** e pode ser particionado em dois modelos:
 - ✓ **Modelo de Análise**: modelo de classes em alto nível (chamadas classes de análise).
 - ✓ **Modelo de Design**: modelos de classes implementáveis.
- O **Documento da Arquitetura** é usado para descrever em detalhes uma determinada arquitetura.
- O **Modelo de Dados** contém o mapeamento do modelo OO para o modelo relacional.

- Análise e Design não é nem *top-down* nem *bottom-up*:



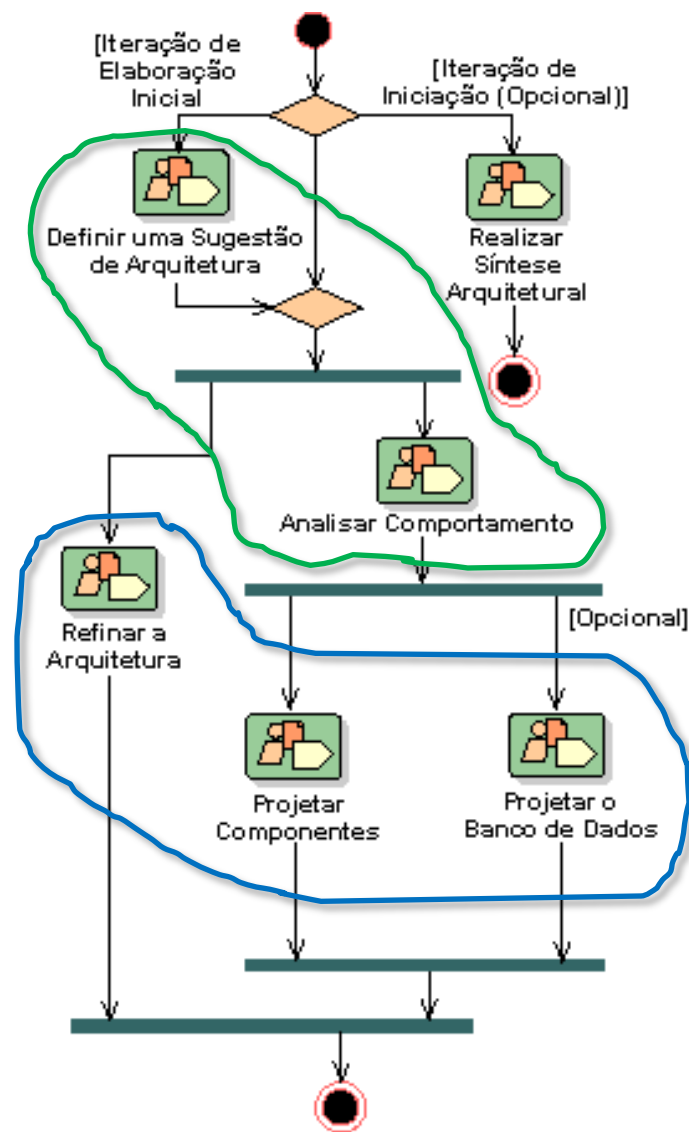
➤ A disciplina de **Análise e Design** é composta dos seguintes fluxos de trabalho:

1. Definir uma Sugestão de Arquitetura
2. Analisar Comportamento
3. Projetar Componentes
4. Projetar BD
5. Refinar a Arquitetura



- # Design

1. Definir uma Sugestão de Arquitetura
2. Analisar Comportamento
3. Projetar Componentes
4. Projetar BD
5. Refinar a Arquitetura



➤ A disciplina de **Análise e Design** é composta dos seguintes fluxos de trabalho:

1. Definir uma Sugestão de Arquitetura

1. Análise Arquitetural

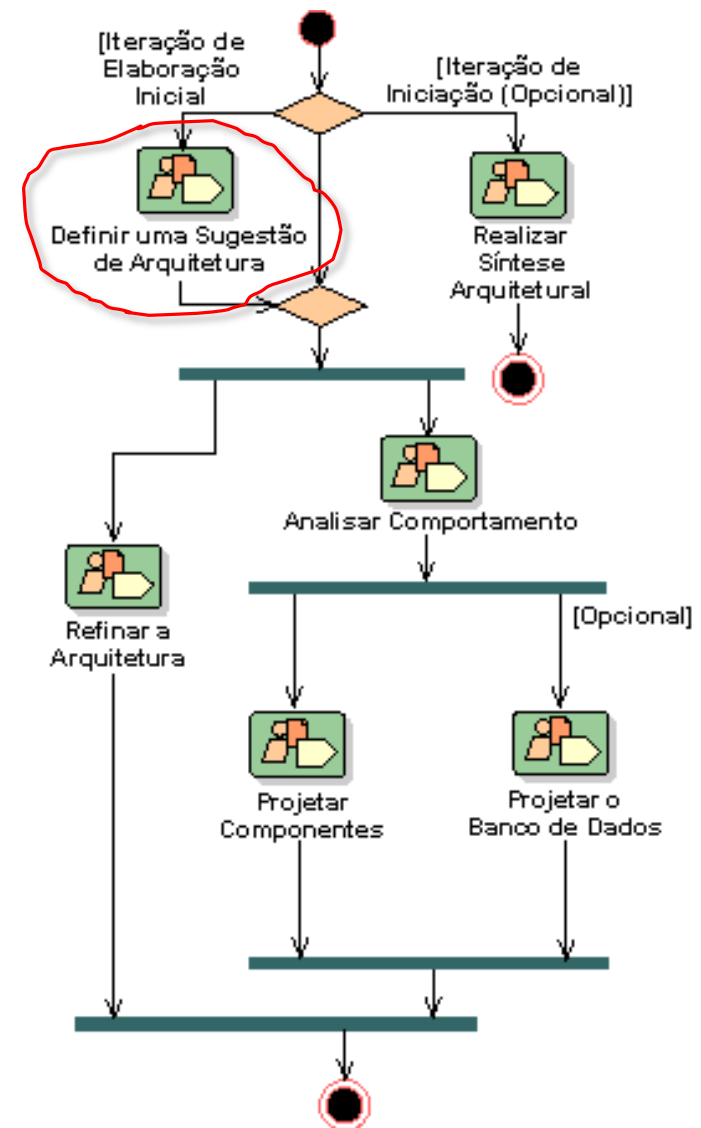
2. Análise de Caso de Uso

2. Analisar Comportamento

3. Projetar Componentes

4. Projetar BD

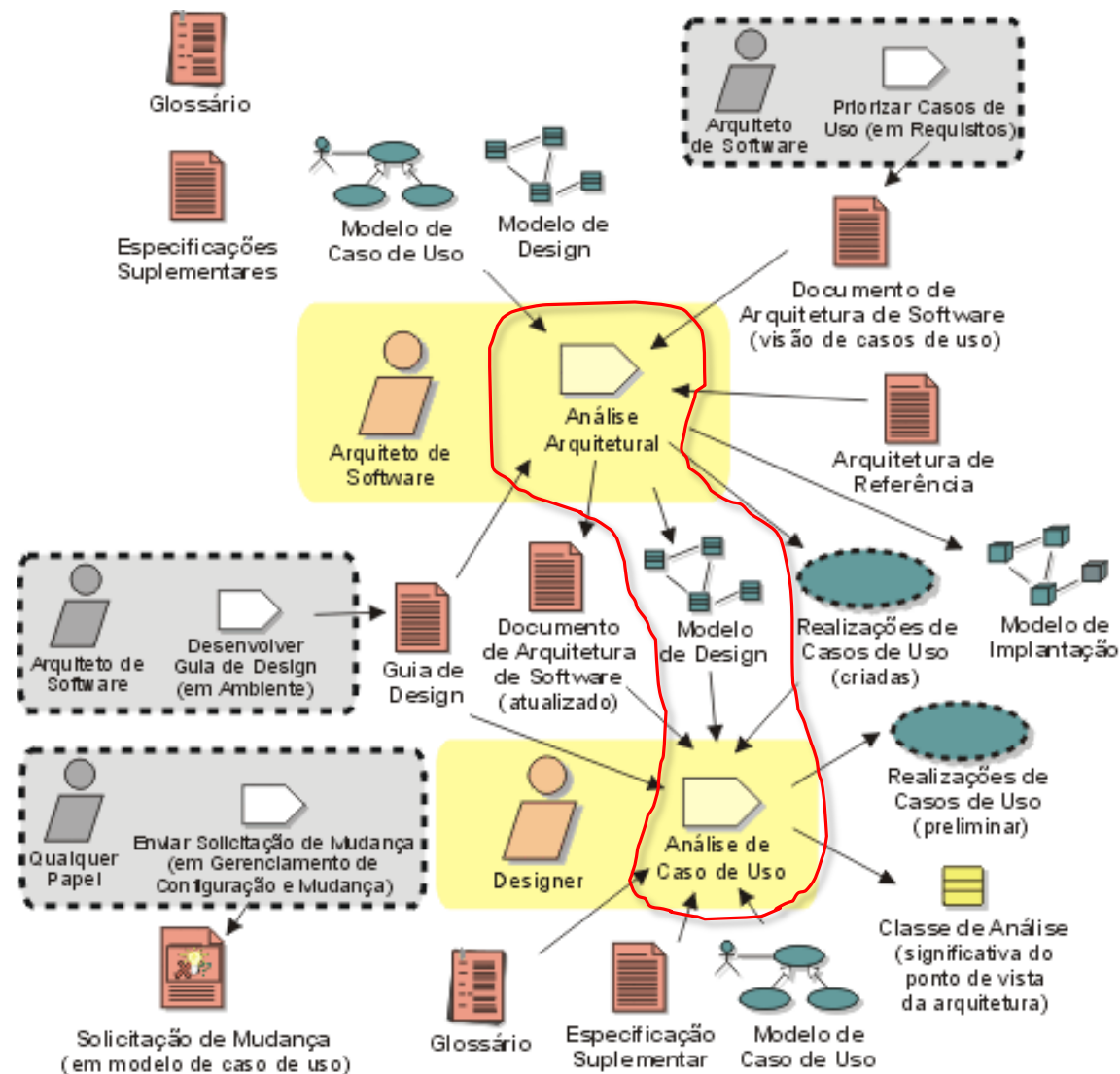
5. Refinar a Arquitetura



- A finalidade desse fluxo de trabalho é:
 1. Criar o **esboço inicial** da arquitetura do sistema.
 2. Identificar **classes de análise** dos casos de uso **significativos** em termos de arquitetura.
 3. Criar as **realizações de casos de uso** com as interações entre as classes de análise.

➤ Esse fluxo de trabalho é melhor realizado em **várias sessões**, com **iteração** entre essas duas atividades:

1. Realize inicialmente a Análise Arquitetural.
2. Escolha os casos de uso **mais significativo em termos de arquitetura** e realize a Análise de Casos de Uso para eles.
3. Realize novamente a Análise Arquitetural para atualizar a arquitetura de forma que ela acomode as alterações no comportamento do sistema.
4. Selecione um novo conjunto de casos de uso para Análise e repita o processo até finalizar os casos de uso relevantes.



- Essa atividade tem como objetivo:
 - ✓ Definir o primeiro esboço de arquitetura para o sistema com base na experiência obtida com sistemas ou domínios de problema semelhantes.
 - ✓ Definir os padrões de arquitetura e os principais mecanismos.
- Envolve os seguintes passos:
 1. Desenvolver a visão geral da arquitetura
 2. Avaliar os recursos disponíveis
 3. Definir a organização de alto nível do sistema
 4. Identificar as abstrações-chave
 5. Desenvolver a visão geral da implantação
 6. Identificar os mecanismos de análise
 7. Criar Realizações de Casos de Uso
 8. Revisar os Resultados

3. Definir a Organização de Alto Nível do Sistema

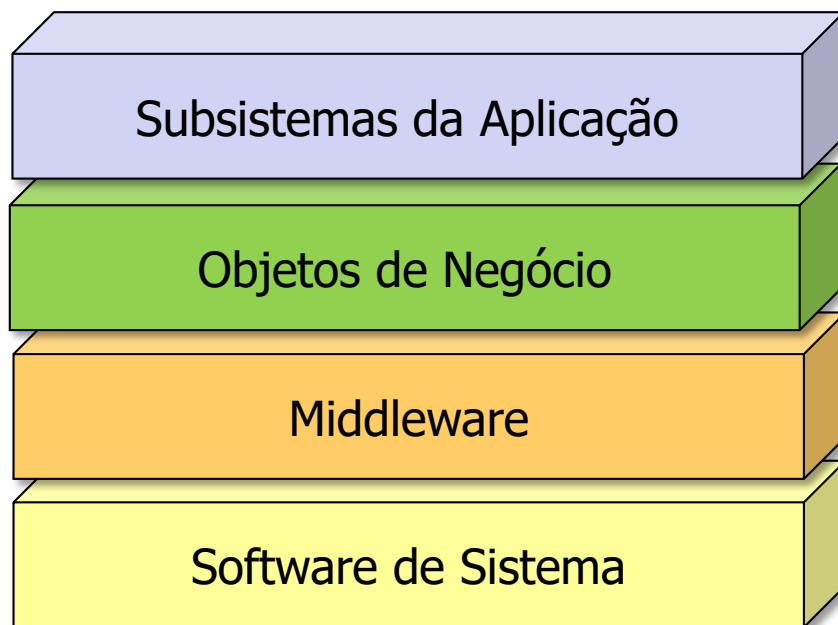
- Nesse ponto o arquiteto deve definir a estrutura inicial para o **Modelo de Design**, ou seja, como as classes serão **agrupadas**.
- A abordagem mais comumente usada em sistema de médio e grande porte é a **Divisão em Camadas** (*Layering*).

Camada: um conjunto de serviços ou funcionalidades agrupadas segundo algum critério.

- A quantidade de camadas não é fixa, pois diversos fatores influenciam essa decisão.

- A forma mais comum de dividir um sistema em camadas segue a seguinte lógica:
 1. As funcionalidades específicas da **aplicação** são alocadas nas camadas **superiores**.
 2. As funcionalidades associadas ao **domínio** do problema ficam nas camadas **intermediárias**.
 3. As funcionalidades específicas relacionadas às **tecnologias** adotadas e ao **ambiente de implementação** ficam nas camadas **inferiores**.
- É importante notar que isso **não** significa que temos somente 3 camadas. Cada uma dessas pode ser subdividida em outras camadas.
- Nessa fase, o foco deve estar nas **duas primeiras camadas** (aplicação e domínio). As demais camadas serão tratadas conforme o design da aplicação for evoluindo.

- Abordagem **típica** da divisão em camadas:
- ✓ Camadas superiores: específicas da aplicação.
 - ✓ Camadas intermediárias: não associadas nem a uma aplicação nem a um conjunto de tecnologias específicas.
 - ✓ Camadas inferiores: mais generalistas e associadas às tecnologias adotadas na solução.



Subsistemas ou pacotes que compõem a aplicação. Contém o software desenvolvido que agrega valor para o negócio.

Subsistemas ou pacotes reusáveis associados ao negócio e não a uma aplicação específica.

Subsistemas ou pacotes utilitários que oferecem serviços técnicos independentes da plataforma.

Softwares básicos associados à infraestrutura real, como sistemas operacionais, drivers, etc.

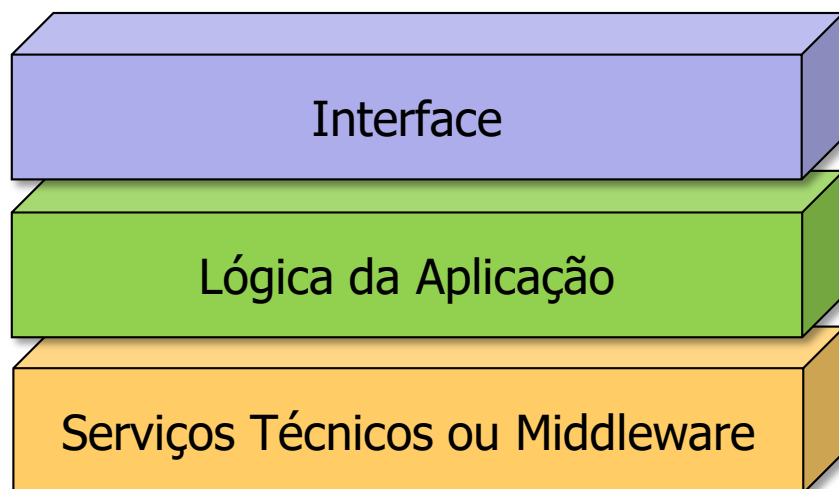
Atenção: esse esquema representa uma visão geral e conceitual.

➤ Algumas diretrizes para divisão em camadas:

1. **Visibilidade:** os elementos de uma camada só podem depender de outros na mesma camada ou na camada inferior.
2. **Volatilidade:**
 - ✓ Nas camadas **superiores**, insira elementos que variam quando os **requisitos dos usuários** são alterados.
 - ✓ Nas camadas **inferiores**, insira elementos que variam quando o **ambiente de implementação** é alterado (hardware, sistema operacional, frameworks, SGBD, softwares de apoio, etc.).
 - ✓ Entre essas camadas, insira elementos que geralmente se aplicam a diversos tipos de sistemas e ambientes de implementação.
 - ✓ Acrescente camadas quando quebras adicionais nessas categorias ajudarem a organizar o modelo.
3. **Generalidade:** elementos abstratos do modelo costumam ser inseridos nas camadas inferiores no modelo. Se não forem específicos da implementação, ficarão geralmente nas camadas intermediárias.

- Algumas diretrizes para divisão em camadas:
 - 4. **Número de camadas:** para sistemas pequenos, 3 camadas são suficientes. Para sistemas mais complexos, 5 a 7 camadas são normalmente suficientes. Independente da complexidade, a existência de mais de 10 camadas deve ser tratada com reservas.

➤ Exemplo 1: divisão em 3 camadas

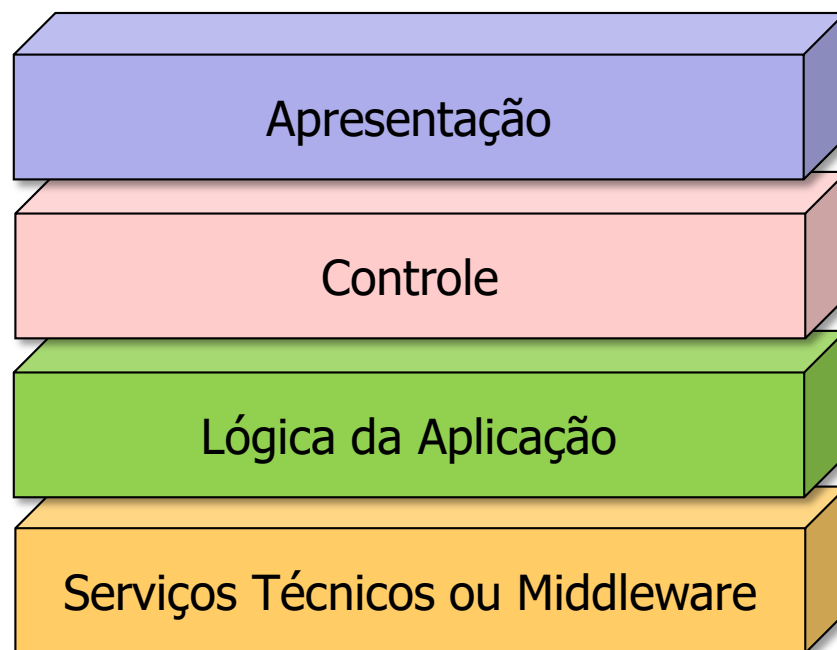


Camada da interface com o meio externo.

Camada da lógica que controla o funcionamento da aplicação.

Serviços técnicos de alto nível como persistência de dados, transmissão de dados, criptografia, autenticação, log, etc.

➤ Exemplo 2: divisão em 4 camadas



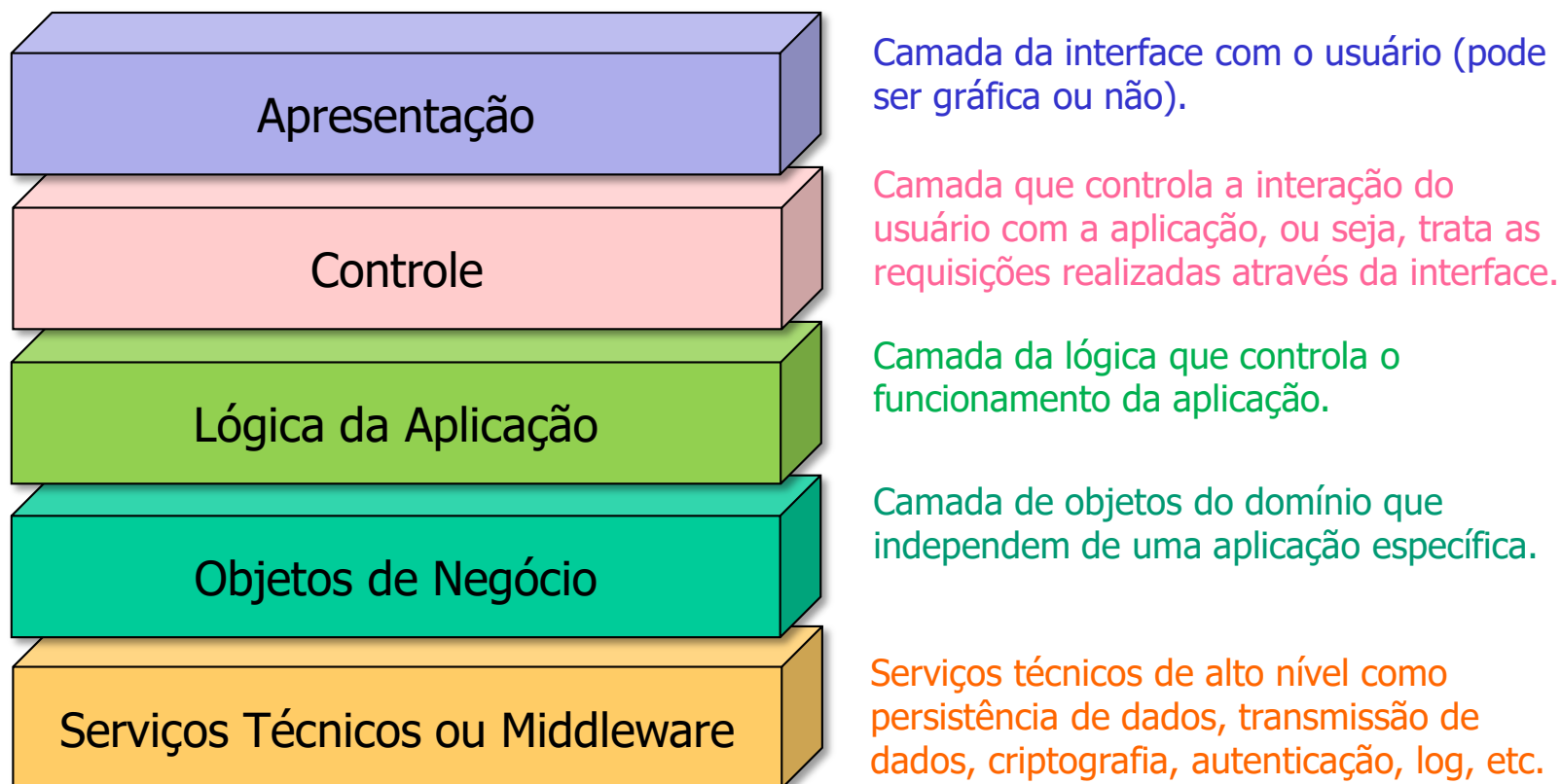
Camada da interface com o usuário (pode ser gráfica ou não).

Camada que controla a interação do usuário com a aplicação, ou seja, trata as requisições realizadas através da interface.

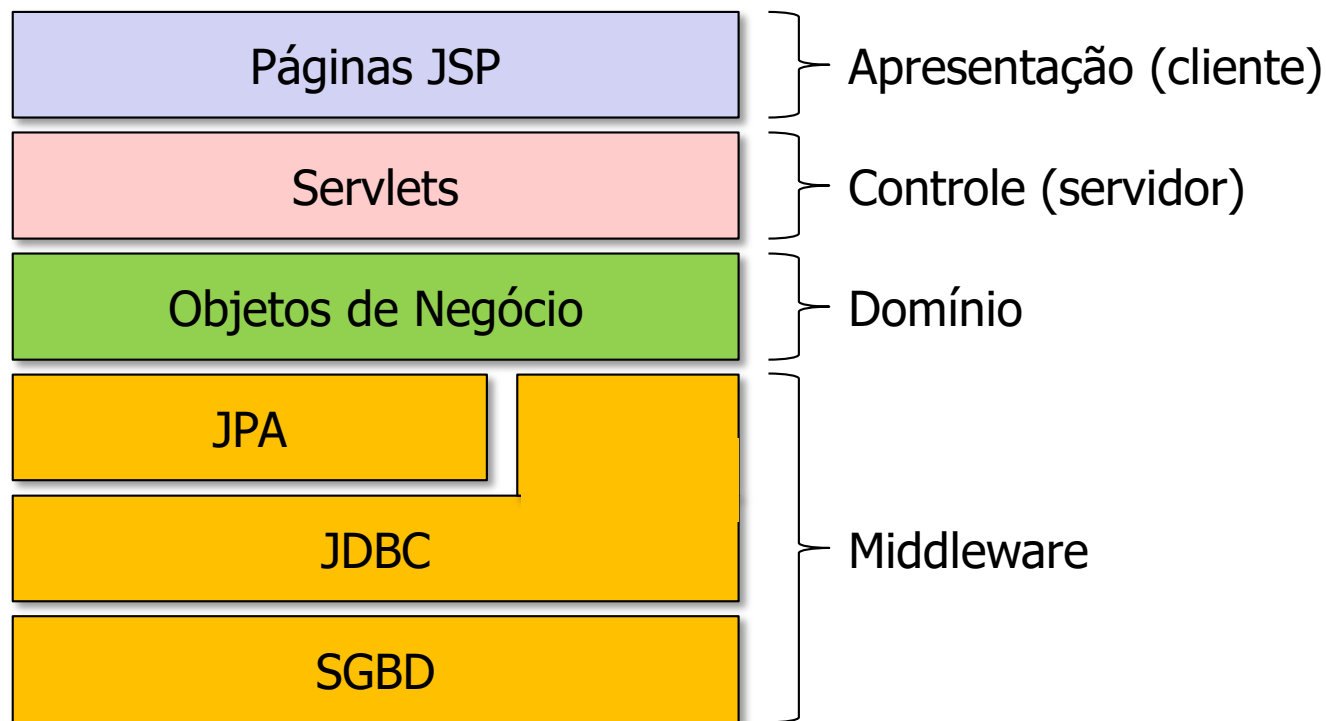
Camada da lógica que controla o funcionamento da aplicação.

Serviços técnicos de alto nível como persistência de dados, transmissão de dados, criptografia, autenticação, log, etc.

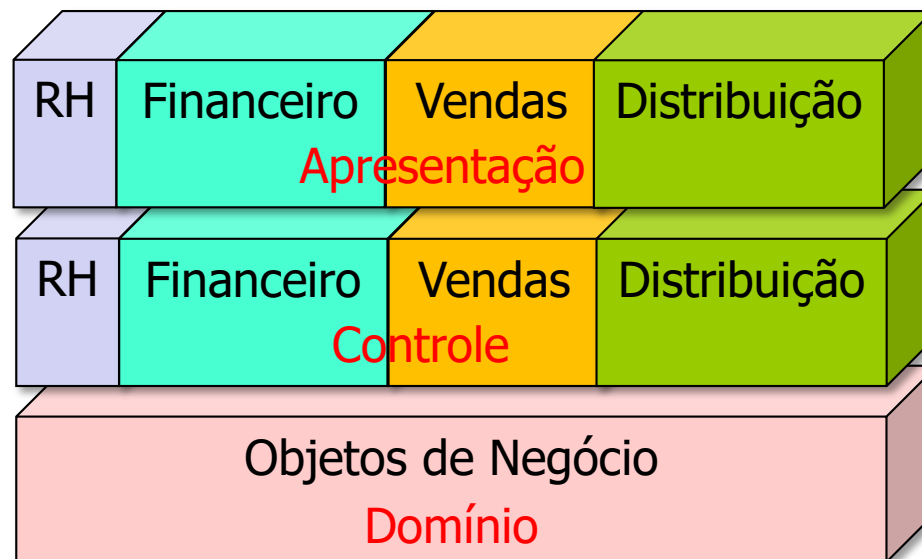
➤ Exemplo 3: divisão em 5 camadas



- Exemplo 4: divisão em 4 camadas particionadas



- Outras diretrizes para esse particionamento também podem ser consideradas:
1. **Organização do usuário:** os subsistemas podem ser definidos de forma a refletir a **estrutura da organização**. Exemplo: particionamento feito por departamento (subsistema de RH, subsistema Financeiro, subsistema de Vendas, etc.).



➤ Outras diretrizes para esse particionamento também podem ser consideradas:

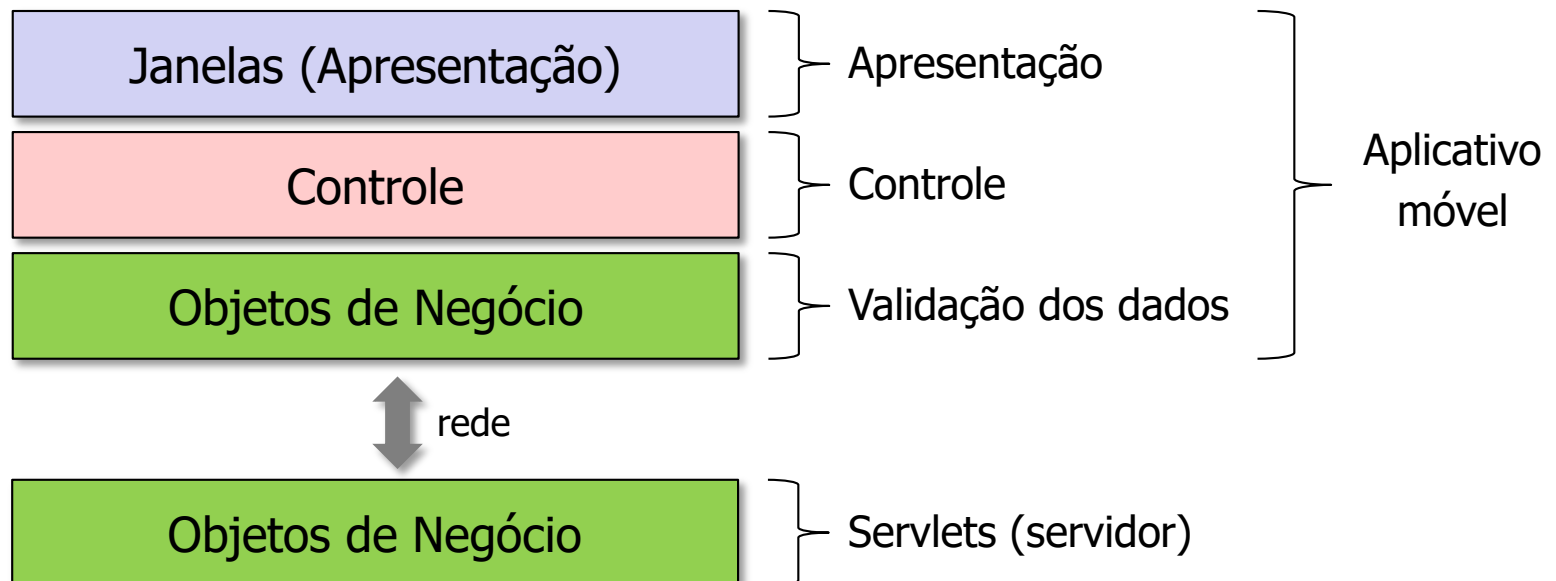
2. **Áreas de competência e/ou habilidades:** é possível organizar subsistemas/pacotes de modo que o modelo seja particionado entre grupos distintos da equipe de desenvolvimento. Em geral, essa divisão reflete a necessidade de especialização de determinadas partes do software. Esse critério pode ser usados para particionar tanto as camadas mais altas quanto as mais baixas.

Exemplos:

- ✓ Pacote de cálculos financeiros (camada mais alta)
- ✓ Pacote de persistência de dados (camada mais baixa)
- ✓ Pacote de criptografia (camada mais baixa)

➤ Outras diretrizes para esse particionamento também podem ser consideradas:

3. **Distribuição do sistema:** em qualquer uma das camadas do sistema, é possível particioná-las "horizontalmente" para refletir a distribuição física da funcionalidade. Esse particionamento permite visualizar a comunicação de rede que ocorrerá quando o sistema for executado.



- Outras diretrizes para esse particionamento também podem ser consideradas:
 4. **Áreas de sigilo**: alguns aplicativos, principalmente aqueles que exigem autorização de segurança especial para serem desenvolvidos e/ou suportados, requerem partições adicionais com acesso privilegiado. Exemplo: urna eletrônica.
 5. **Áreas de variabilidade**: a funcionalidade que tende a ser opcional e, portanto, liberada apenas em algumas versões do sistema, deve ser organizada em subsistemas independentes que são desenvolvidos e apresentados sem depender das funcionalidades obrigatórias do sistema. Exemplo: gerenciamento de NFC no Android.

4. Identificar as Abstrações-Chave

- Nesse passo os casos de uso e demais requisitos funcionais são analisados para elaboração do primeiro diagrama de classes do sistema.
- Esse diagrama de classes ainda é conceitual e está fortemente associado ao domínio no qual o sistema está inserido. Por isso, é chamado de **Modelo de Domínio**.
- Ele só possui as **classes do domínio** e seus **principais atributos e relacionamentos**, ou seja, as classes identificadas nesse modelo são apenas as principais classes do domínio do problema.
- Exemplo: em um sistema de controle de leilão teríamos cliente, leilão, lance, lote, etc.

- Uma outra abordagem possível para criação do modelo de domínio é deixá-lo a cargo do **próprio analista que cria os casos de uso**.
- Nesse caso, o modelo do domínio é criado juntamente com os casos de uso.
- Essa abordagem faz bastante sentido, uma vez que esse modelo descreve os conceitos do domínio do problema e seus relacionamentos e complementa a visão dos casos de uso:
 - ✓ **Casos de uso**: visão das funcionalidades que o sistema deve implementar.
 - ✓ **Modelo do domínio**: visão dos conceitos do domínio.

5. Desenvolver a Visão Geral da Implantação

- Nesse passo o arquiteto poderá criar um **Diagrama de Implantação** de alto nível e descrever os recursos de hardware e software necessários para o funcionamento do sistema.
- É importante que esse diagrama reflita a **distribuição geográfica** e a **complexidade do sistema** em termos dos seus principais componentes de hardware e software.

6. Identificar os Mecanismos de Análise

- Primeiramente, o que é um mecanismo?
- ✓ Um mecanismo representa uma solução para um problema recorrente no sistema.
- ✓ Exemplos:
 - Se as funcionalidades de um sistema só podem ser acessadas por pessoal autorizado, precisamos adotar um mecanismo de autenticação e/ou autorização.
 - Quando um sistema necessita que alguns objetos mantenham seus estados por um longo período, precisamos adotar um mecanismo de persistência dos dados.
 - Se um sistema necessita que as atividades dos usuários sejam registradas para fins de auditoria, precisamos adotar um mecanismo de registro ou log de atividades.

- Mecanismos são classificados como:
 - ✓ **Mecanismo de Análise**: uma descrição abstrata de uma solução para um problema recorrente identificado no sistema.

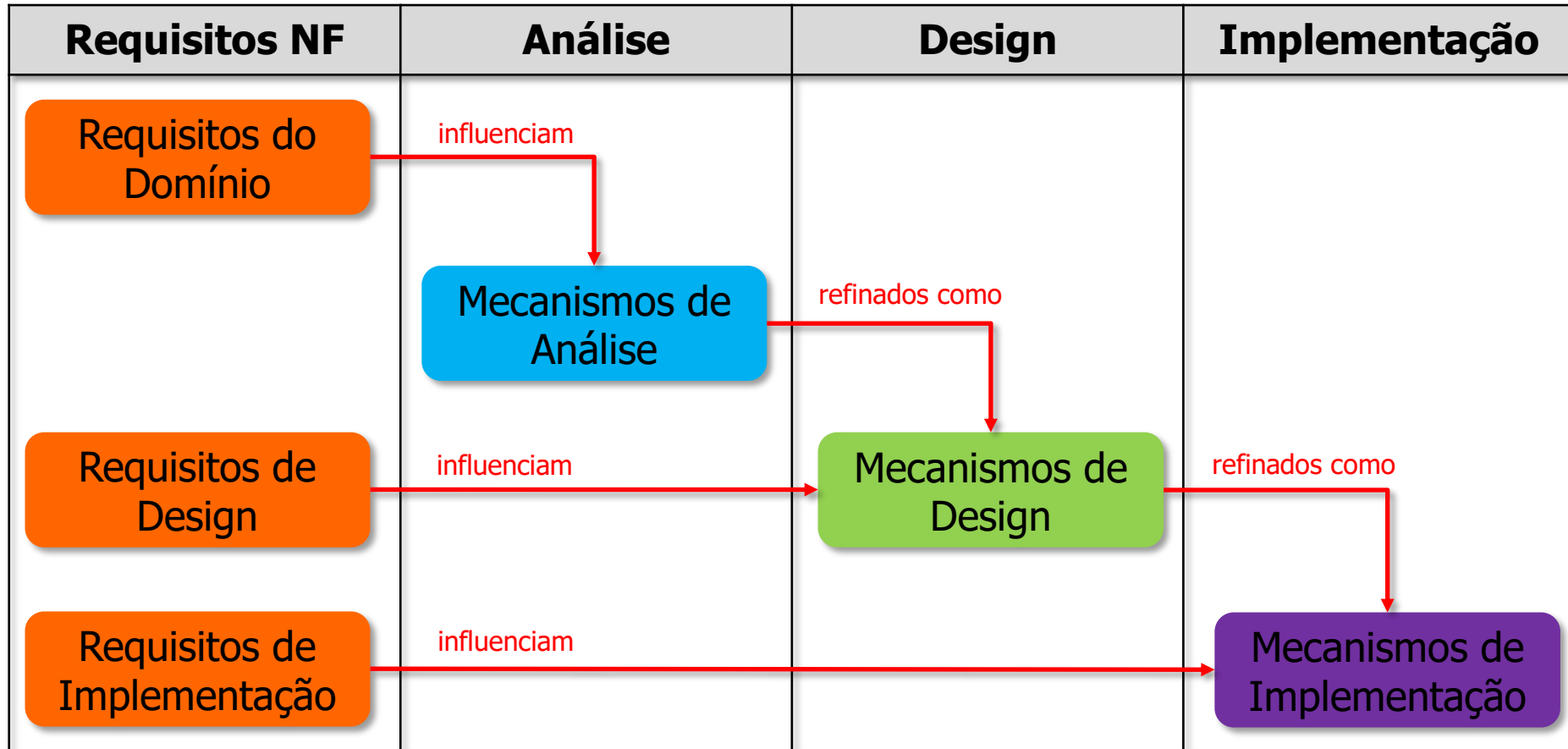


- ✓ **Mecanismo de Design**: é um refinamento do Mecanismo de Análise, mas sem estar associado a nenhuma implementação específica, ou seja, é o direcionamento do Mecanismo de Análise para alguma tecnologia geral.



- ✓ **Mecanismo de Implementação**: é a instanciação do Mecanismo de Design, ou seja, é a adoção de uma implementação específica relacionada ao Mecanismo de Design.

- Mecanismos são influenciados pelos **Requisitos Não Funcionais**:



➤ Exemplos de mecanismos:

Análise	Design	Implementação
Persistência	Banco relacional	SQL Server 2000 MySQL 5.0 Oracle 12
	Banco NoSQL	MongoDB Redis
	Arquivos binários	Java IO API
Transferência de dados entre dispositivos	Arquivo XML + FTP	JAXP + Apache Commons Net
	Arquivo texto + Socket TCP	Java IO API + Java NET API
Log de atividades	Arquivo texto	Log4J Java IO API
	Banco relacional	SQL Server 2000 MySQL 5.0 Oracle 12
Criptografia	Algoritmo AES	Pacote javax.crypto
	Algoritmo RCA	
	Protocolo HTTPS	-

➤ Exemplos de mecanismos:

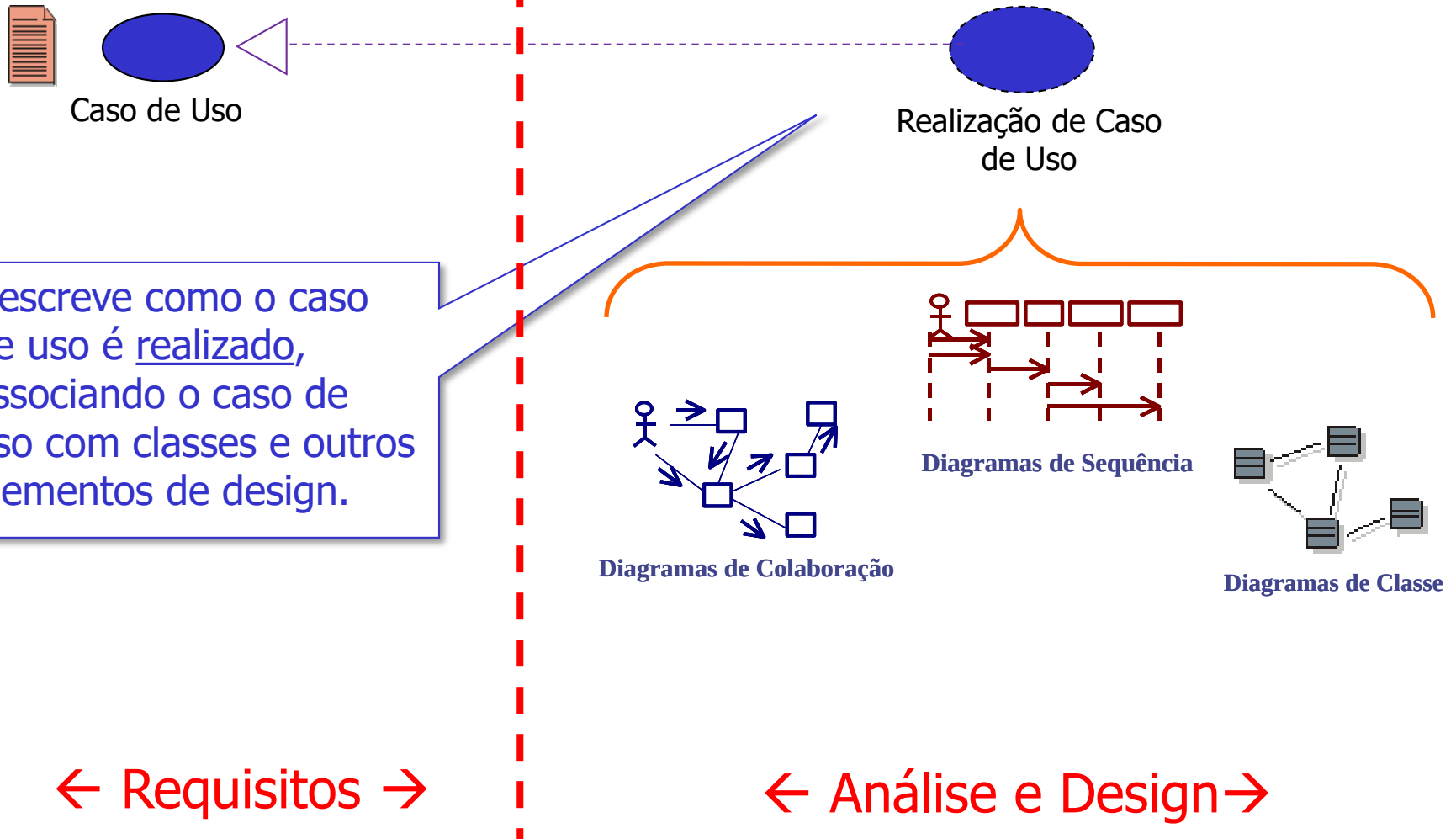
Análise	Design	Implementação
Autenticação	Login e senha	Verificar se existe algum usuário cadastrado com o login e senha fornecidos
	Dado biométrico	Verificar se existe algum usuário cadastrado com a digital do polegar direito ou esquerdo idêntica à fornecida.
Autorização	Baseada no caso de uso	Verificar se o perfil do usuário tem acesso ao caso de uso
	Baseada em papéis	J2EE Security Model
Envio de mensagens	Email	Java Mail API Apache Commons Net
	SMS	Pacote android.telephony
Comunicação entre processos	Arquivo texto	Java IO API
	Socket TCP	Java NET API
	RPC (Remote Procedure Call)	Java RMI (Remote Method Invocation) SOAP REST
	CORBA	Orbix Visibroker

7. Criar Realizações de Casos de Uso*

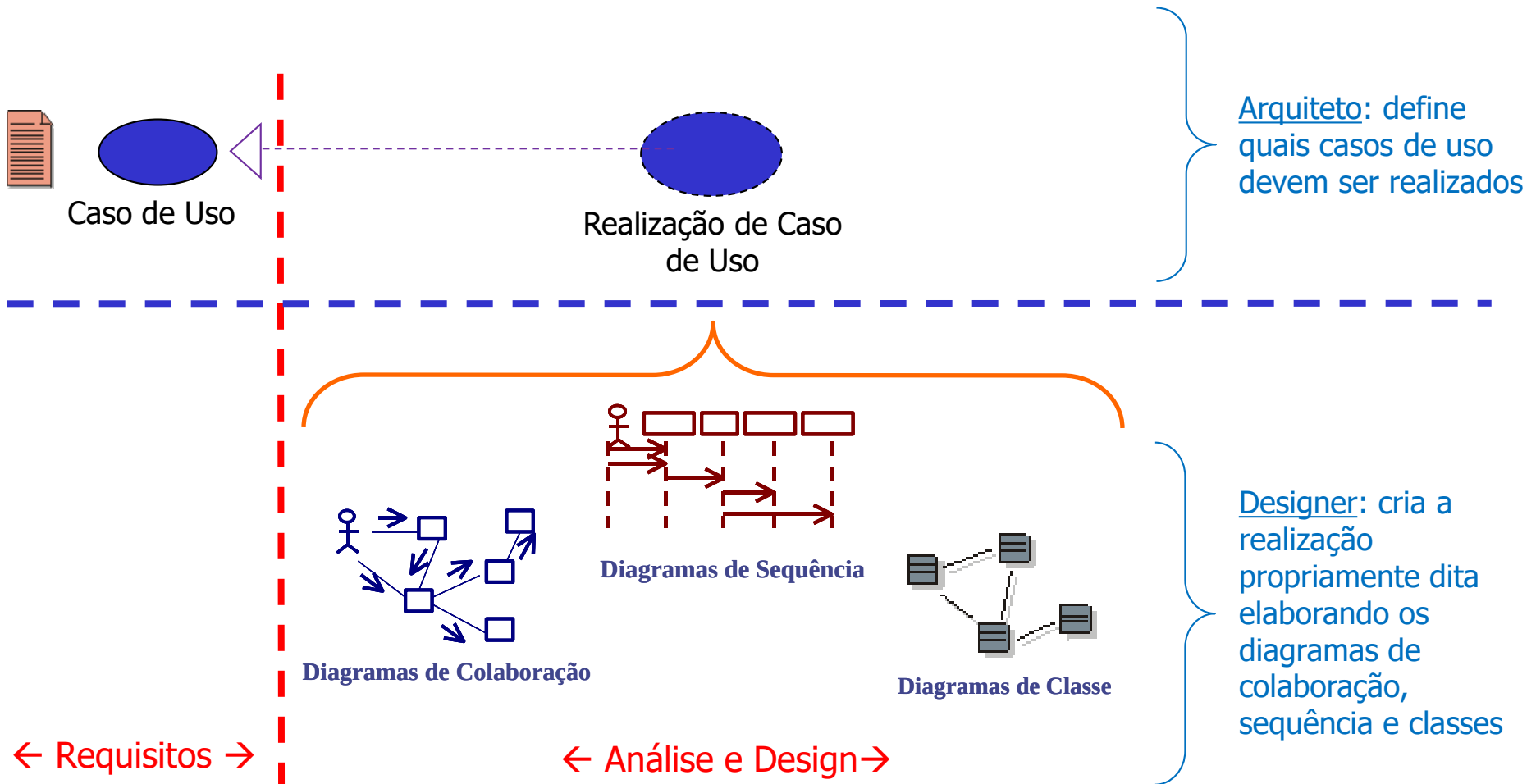
- Nesse passo será criada uma **Realização de Caso de Uso** no Modelo de Design para cada caso de uso **significativo** do ponto de vista da arquitetura.
- Um caso de uso **significativo** do ponto de vista da arquitetura é aquele que:
 - ✓ Representa uma funcionalidade central e significativa do sistema.
 - ✓ Usa vários elementos arquiteturais na sua execução.
 - ✓ Ilustra um determinado ponto complexo da arquitetura.
- Exemplo: em um sistema de controle de caixa eletrônico, o caso de uso "Sacar dinheiro" é muito mais significativo do que o caso de uso "Consultar saldo da conta".
- Mas, o que é uma "Realização de Caso de Uso"?

* A palavra "realização" vem da tradução de "realize", que significa, dentre outras coisas, concretizar algo, tornar real, implementar.

- O que é "Realização do Caso de Uso" ?



- O que é "Realização do Caso de Uso" ?

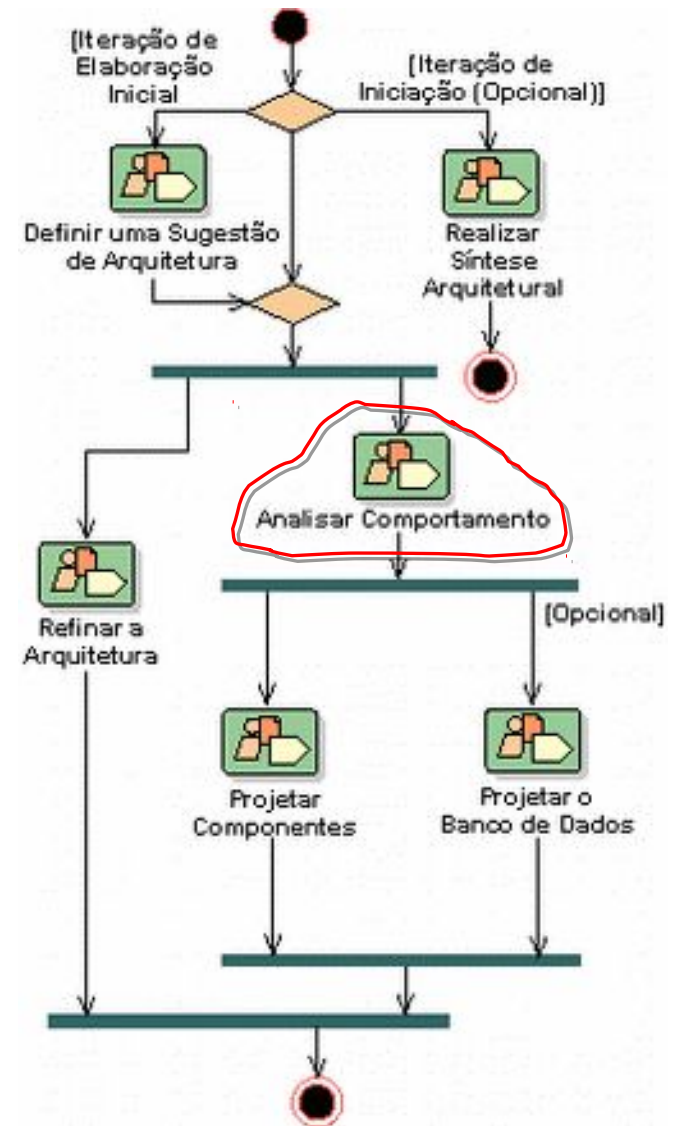


Projeto de Software Orientado a Objetos

5ª parte

Prof. Jobson Massollar

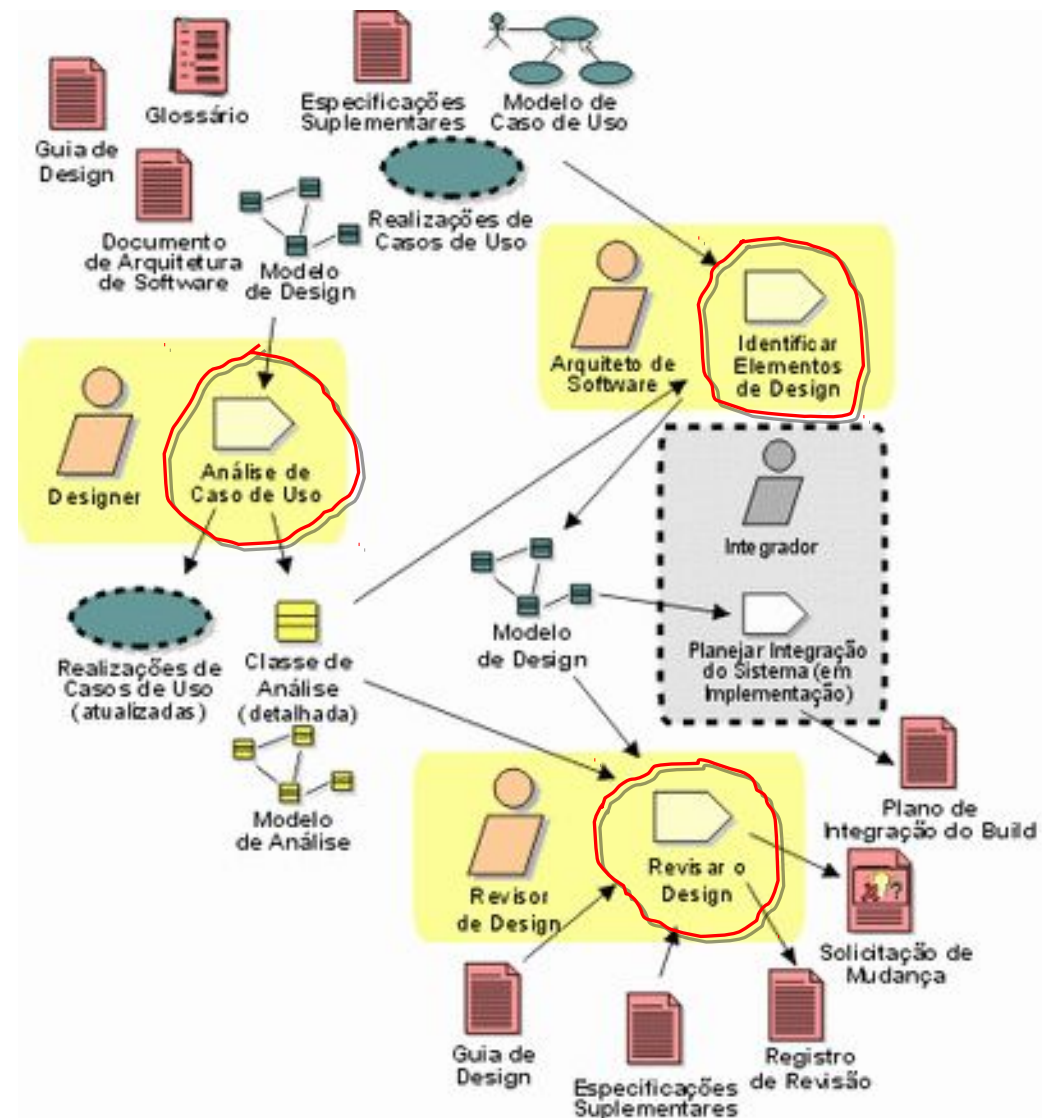
- A disciplina de **Análise e Design** é composta dos seguintes fluxos de trabalho:
1. Definir uma Sugestão de Arquitetura
 2. **Analisar Comportamento**
 1. **Análise de Caso de Uso**
 2. **Identificar Elementos de Design**
 3. **Revisar o Design**
 3. Projetar Componentes
 4. Projetar BD
 5. Refinar a Arquitetura



- A finalidade desse fluxo de trabalho é:
 - ✓ Transformar as descrições dos comportamentos do sistema, fornecidas pelos caso de uso, em um conjunto de elementos que servirão de base para o design.

➤ Esse fluxo de trabalho é composto de atividades que são executadas da seguinte forma:

1. Realize workshops para Analisar os Casos de Uso e identificar as classes de análise.
2. A partir das classes de análise, realize a Identificação dos Elementos de Design para identificar classes de design e subsistemas.
3. Quando o trabalho de design for concluído e estabilizado, faça a Revisão do Design.



- Essa atividade tem como objetivo:
 - ✓ Identificar as classes de análise que realizam os fluxos de eventos dos casos de uso.
 - ✓ Distribuir o comportamento do caso de uso entre essas classes de análise utilizando as Realizações de Casos de Uso.
 - ✓ Identificar as responsabilidades, os atributos e as associações das classes de análise.

- Essa atividade representa um ponto de transição importante, já que ela estabelece uma ponte entre a visão do usuário do sistema (representada pelos casos de uso) e a visão do designer do sistema (representada, nesse momento, por classes de análise).

- De forma resumida, envolve os seguintes passos:
 1. Suplementar as descrições dos casos de uso
 2. Para cada Realização de Caso de Uso
 1. Localizar Classes de Análise
 2. Distribuir os comportamentos do Caso de Uso para as Classes de Análise
 3. Para cada Classe de Análise
 1. Descrever responsabilidades
 2. Descrever atributos e associações
 3. Qualificar mecanismos de análise
 4. Avaliar resultado da análise

- De forma resumida, envolve os seguintes passos:
 1. **Suplementar as descrições dos casos de uso** 
 2. Para cada Realização de Caso de Uso
 1. Localizar Classes de Análise
 2. Distribuir os comportamentos do Caso de Uso para as Classes de Análise
 3. Para cada Classe de Análise
 1. Descrever responsabilidades
 2. Descrever atributos e associações
 3. Qualificar mecanismos de análise
 4. Avaliar resultado da análise

1. Suplementar as Descrições dos Casos de Uso

- Capturar informações adicionais necessárias para entender o comportamento **interno** do sistema que pode estar faltando na descrição do Caso de Uso.
- Esse passo é importante porque a descrição de cada Caso de Uso **nem sempre tem os detalhes suficientes** para localizar as classes de análise.

➤ Exemplo:

- ✓ No caso de uso foi feita a seguinte descrição:
"O Sistema de Caixa Eletrônico valida o cartão de Cliente do Banco"
- ✓ Embora possa ser suficiente para o cliente entender o comportamento do sistema, essa frase não nos oferece nenhuma ideia do que realmente ocorre no **interior** do Sistema de Caixa Eletrônico para validar o cartão.
- ✓ Para obter uma imagem interna de como o sistema funciona, em um nível suficiente de detalhe que identifique os objetos, podem ser necessárias informações adicionais.

- Uma descrição ampliada poderia ser:

"O Sistema de Caixa Eletrônico valida o cartão de Cliente do Banco"

"O Sistema de Caixa Eletrônico lê os dados do cartão e envia o número da conta e a senha para o Servidor para serem validados. O Servidor retorna um indicador de sucesso se o número da conta do cliente existir e a senha for válida. Caso contrário, o Servidor enviará um indicador de insucesso."

- Pode ser que o designer tenha que consultar o analista ou o especialista no domínio para entender em detalhes o que o sistema tem que fazer.

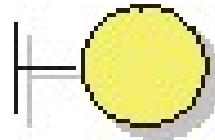
- De forma resumida, envolve os seguintes passos:
 1. Suplementar as descrições dos casos de uso
 2. Para cada Realização de Caso de Uso
 1. Localizar Classes de Análise
 2. Distribuir os comportamentos do Caso de Uso para as Classes de Análise
 3. Para cada Classe de Análise 
 1. Descrever responsabilidades
 2. Descrever atributos e associações
 3. Qualificar mecanismos de análise
 4. Avaliar resultado da análise

2.1 Localizar Classes de Análise

- Identificar um conjunto de elementos (chamados de Classes de Análise) que poderão executar o comportamento descrito nos Casos de Uso.
- Ou seja, a ideia é identificar classes que possam **realizar os comportamentos** definidos no Caso de Uso.
- Uma técnica bastante difundida para realizar esse passo é trabalhar com os conceitos de classes de **fronteira (boundary)**, classes de **controle (control)** e classes de **entidade (entity)**.
- **Atenção:** apesar dos nomes, Boundary, Control e Entity **não** representam um modelo MVC!

➤ Classes de Fronteira (Boundary)

- ✓ Classe usada para modelar a **interação** entre o sistema e o seu **ambiente**, ou seja, essas classes fazem a **intermediação** entre o sistema e algo que está **fora dele**.
- ✓ Algumas classes de fronteira comuns são:
 - Janelas, formulários e relatórios
 - Outros sistemas
 - Sensores
 - Dispositivos diversos controlados pelo sistema.
- ✓ Classes de fronteira são representadas com o seguinte ícone:



➤ Classes de Fronteira

✓ Podemos classificar classes de fronteira em:

- a) **Classes de interface com o usuário**: classes que realizam a interação com usuários do sistema.
- b) **Classes de interface com outro sistema**: classes que intermediam a comunicação com outros sistemas.
- c) **Classes de interface com dispositivo**: classes que fornecem a interface para dispositivos (por exemplo, sensores).

a) Classes de Fronteira para Interface com o Usuário

- ✓ Existe pelo menos **uma** classe de fronteira quando o ator do caso de uso é um **usuário**.
- ✓ Para interfaces baseadas em janelas ou formulários, cada janela ou formulário não representa, necessariamente, uma classe de fronteira.
- ✓ A classe de fronteira deve representar a abstração usada na interação com o usuário. Exemplo:
 - Se um usuário preenche um formulário de cadastro, então esse formulário deve ser representado como **uma única** classe de fronteira, independente de quantas telas ou janelas ele ocupe.

a) Classes de Fronteira para Interface com o Usuário

- ✓ Caso a mesma janela ou formulário seja usada em mais de um caso de uso, **reuse** a classe de fronteira definida anteriormente.
- ✓ Caso existam protótipos da interface associados ao caso de uso, use esses protótipos para identificar as classes de fronteira.
- ✓ Não defina detalhes como botões, listas, campos ou demais elementos da interface, concentre-se em **qual informação** será apresentada.

b) Classes de Fronteira para Interface com Sistemas Externos

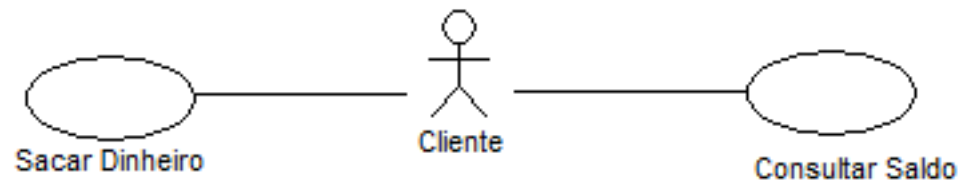
- ✓ Existe **uma** classe de fronteira quando o ator do caso de uso é **outro sistema** (sistema externo).
- ✓ Essa classe vai encapsular **toda** a comunicação com o sistema externo.
- ✓ Caso a interface com determinado sistema externo apareça em mais de um caso de uso, **reuse** a classe de fronteira definida anteriormente.

c) Classes de Fronteira para Interface com Dispositivos

- ✓ Existe **uma** classe de fronteira quando o ator do caso de uso é um **dispositivo**.
- ✓ Mesmo nos casos onde o dispositivo não é representado explicitamente como um ator no caso de uso, ele deve ser representado como uma classe de fronteira quando o sistema demanda algum tipo de **interação** com esse dispositivo. Por exemplo: um leitor biométrico em um caixa eletrônico.
- ✓ Caso a interface com determinado dispositivo apareça em mais de um caso de uso, **reuse** a classe de fronteira definida anteriormente.

➤ Classes de Fronteira

- ✓ Quais as classes de fronteira para os casos de uso "Sacar dinheiro" e "Consultar Saldo" em um caixa eletrônico?

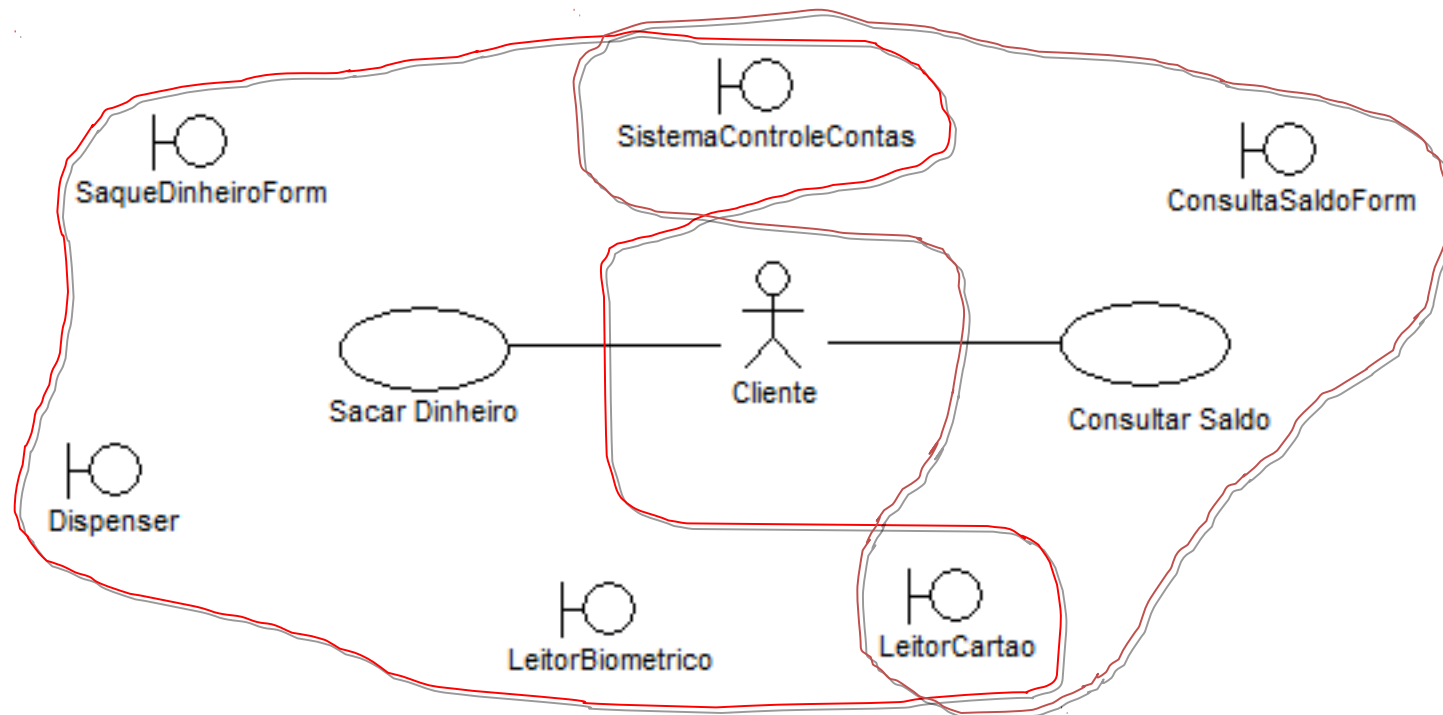


➤ Classes de Fronteira

- ✓ Quais as classes de fronteira para os casos de uso "Sacar dinheiro" e "Consultar Saldo" em um caixa eletrônico?
 - Formulários com os quais o cliente interage.
 - Leitor biométrico.
 - Leitor de cartão.
 - Dispenser (responsável por separar e entregar as notas ao cliente).
 - Sistema de *backoffice* do banco (responsável por autorizar as transações).
 - Impressora (se for necessário imprimir um recibo de saque ou o saldo).

➤ Classes de Fronteira

- ✓ Quais as classes de fronteira para os casos de uso "Sacar dinheiro" e "Consultar Saldo" em um caixa eletrônico?



➤ Classes de Fronteira

- ✓ É importante notar que, caso algum elemento de fronteira seja substituído (por exemplo, troca do leitor biométrico) **somente a classe de fronteira** precisa ser alterada/redefinida.
- ✓ Além disso, a partir das classes de fronteira será possível perceber que outras classes serão necessárias. Exemplos:
 - Se existe uma classe de fronteira para uma impressora, provavelmente haverá uma classe para formatar os dados que serão impressos.
 - Se existe uma classe de fronteira para um formulário, provavelmente haverá uma classe para validar os dados informados pelo usuário.
 - Se existe uma classe de fronteira para um sistema externo, provavelmente haverá uma classe para converter/formatar os dados que serão enviados ou recebidos desse sistema.

➤ Classes de Controle (Control)

- ✓ Classe usada para modelar o comportamento relacionado ao controle de um ou alguns casos de uso. Assim, classes de controle estão estreitamente relacionadas com a **realização do caso de uso**.
- ✓ Objetos de controle operam como um **coordenador** ou **orquestrador**, ou seja, coordenam os objetos que participam da realização do caso de uso.
- ✓ Observe que um objeto de controle não realiza tudo o que é necessário em um caso de uso. Em vez disso, ela **coordena** as atividades de outros objetos que implementam as funcionalidades necessárias.
- ✓ Classes de controle são representadas com o seguinte ícone:

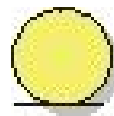


➤ Classes de Controle

- ✓ É possível ter:
 - Uma classe de controle associada à realização de um único caso de uso;
 - Uma mesma classe de controle associada à realização de vários casos de uso;
 - Várias classes de controle associadas à realização de um único caso de uso.
- ✓ Entretanto, nesse momento, podemos associar **uma classe de controle para cada caso de uso** sem prejuízo algum.
- ✓ Essa discussão será retomada posteriormente na atividade **Design de Caso de Uso**.

➤ Classes de Entidade (Entity)

- ✓ Classe usada para modelar **conceitos relacionados ao domínio** do problema e suas informações, como um conceito, um evento, uma pessoa ou algum objeto real.
- ✓ Exemplos:
 - Sistema bancário: Cliente, Conta, Saque e Depósito
 - Sistema de automação residencial: Morador, Porta, Ar condicionado.
 - Sistema acadêmico: Aluno, Curso, Turma e Disciplina
- ✓ Normalmente são ortogonais aos casos de uso, ou seja, podem estar relacionadas com vários casos de uso.
- ✓ Classes de entidade são representadas com o seguinte ícone:



➤ Classes de Entidade

- ✓ As classes de entidade **podem ser extraídas do modelo de domínio** (gerado anteriormente), que representa justamente os conceitos-chave e seus relacionamentos.
- ✓ Eventualmente, ao analisar a descrição detalhada do caso de uso, o designer pode identificar uma classe de entidade que ainda não consta do modelo de domínio.
- ✓ As técnicas para localização de classes de entidade a partir das descrições dos casos de uso são **as mesmas** usadas para criar o **modelo de domínio** a partir dos casos de uso.

➤ Classes de Entidade

- ✓ Para identificar classes e seus atributos a partir dos casos de uso podemos adotar a seguinte estratégia:
 1. Extraia os substantivos (simples ou compostos) dos requisitos, observando os grupos de candidatos a objetos;
 2. Descarte os objetos que não estão associadas ao domínio do problema;
 3. Verifique se o substantivo representa um objeto ou atributo ou um valor do atributo;
 4. Verifique se existem sinônimos.

➤ **Classes de Entidade:** o que procurar? (essa lista não é definitiva!)

Candidatos a Classes de Entidade	Exemplos
Coisas concretas	Avião, Trem, Fita, PDV
Descritores ou especificações	Modelo do Avião, Categoria do produto
Lugares	Aeroporto, Vídeo Locadora, Loja
Transações	Empréstimo, Reserva, Venda, Pedido
Itens de transação	Item de Empréstimo, Item de Venda
Papéis ou funções	Piloto, Atendente, Sócio, Passageiro, Aluno
Unidades organizacionais	Departamento de Vendas, Gerência de Suporte
Conceitos do negócio	Frete, Disciplina, Turma
Eventos ou ocorrências	Voo, Acidente, Ligação, Sinistro
Registro fiscal/contratual	Cupom Fiscal, Contrato de Trabalho, Comprovante de inscrição
Coleções de outras coisas	Acervo, Prateleira, Catálogo
Serviços	Consulta Médica, Atendimento
Documentos	Currículo, Certidão, Especificação

➤ Classes de Entidade

- ✓ Uma regra útil que podemos usar para identificar se um conceito **X** é uma **classe** ou um **atributo** é a seguinte:
 - Se conseguimos descrever/representar o conceito **X** como um **número** ou **texto** no mundo real, ele provavelmente é um **atributo**; caso contrário, provavelmente é uma **classe**.
 - Se existe a necessidade de ter informações sobre o conceito **X**, então provavelmente **X** é uma **classe**. Identifique os atributos de **X**.

- Exemplo 1: **Voo** é uma entidade ou atributo?
- Cenário 1:
 - ✓ É possível associar um **número** ou **texto** ao Voo?
 - Sim, voo 3523 ou voo 241.
 - ✓ Existe a necessidade de ter **informações** sobre Voo?
 - Não.
- Cenário 2:
 - ✓ É possível associar um **número** ou **texto** ao Voo?
 - Acho que não.
 - ✓ Existe a necessidade de ter **informações** sobre Voo?
 - Sim, é preciso saber o número, a data e a hora do voo.

- Exemplo 2: **Endereço** é uma entidade ou atributo?
- Cenário 1:
 - ✓ É possível associar um **número** ou **texto** ao Endereço?
 - Sim, "Av. Presidente Vargas, 248, sala 810, Centro".
 - ✓ Existe a necessidade de ter **informações** sobre Endereço?
 - Não.
- Cenário 2:
 - ✓ É possível associar um **número** ou **texto** ao Endereço?
 - Sim, "Av. Presidente Vargas, 248, sala 810, Centro".
 - ✓ Existe a necessidade de ter **informações** sobre Endereço?
 - Sim, é preciso saber o nome da rua, o número e o bairro.

Objetivo: permitir ao cliente sacar uma quantia em dinheiro da sua CC.

Ator: cliente

Pré-condições: não há

Trigger: cliente aciona a opção de sacar dinheiro

Fluxo Principal:

1. Sistema solicita que o cliente insira o cartão magnético.
2. Cliente insere o cartão magnético.
3. Sistema valida o cartão magnético.
4. Sistema solicita que o cartão magnético seja retirado.
5. Cliente retira o cartão magnético.
6. Sistema solicita a senha.
7. Cliente informa a senha.
8. Sistema valida a senha.
9. Sistema solicita o valor desejado.
10. Cliente informa o valor.
11. Sistema verifica situação do caixa.
12. Sistema verifica situação da conta.
13. Sistema entrega o dinheiro ao cliente.
14. Cliente retira o dinheiro.
15. Sistema debita o valor sacado da conta.

Pós-condições: cliente obtém o valor desejado e o valor sacado é debitado da conta.

Objetivo: permitir ao cliente sacar uma quantia em dinheiro da sua CC.

Ator: cliente

Pré-condições: não há

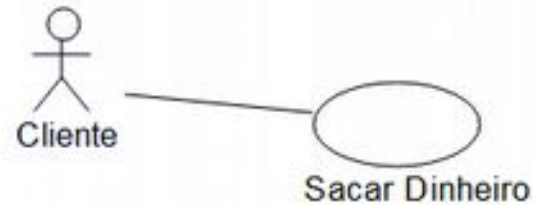
Trigger: cliente aciona a opção de sacar dinheiro

Fluxo Principal:

1. Sistema solicita que o **cliente** insira o **cartão** magnético.
2. Cliente insere o cartão magnético.
3. Sistema valida o cartão magnético.
4. Sistema solicita que o cartão magnético seja retirado.
5. Cliente retira o cartão magnético.
6. Sistema solicita a **senha**.
7. Cliente informa a senha.
8. Sistema valida a senha.
9. Sistema solicita o **valor** desejado.
10. Cliente informa o valor.
11. Sistema verifica **situação** do **caixa**.
12. Sistema verifica situação da **conta**.
13. Sistema entrega o **dinheiro** ao cliente.
14. Cliente retira o dinheiro.
15. Sistema debita o valor sacado da conta.

Pós-condições: cliente obtém o valor desejado e o valor sacado é debitado da conta.

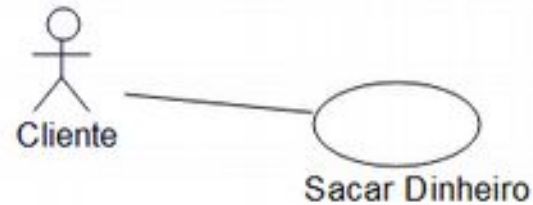
➤ Exemplo 1:



Modelo de Casos de Uso

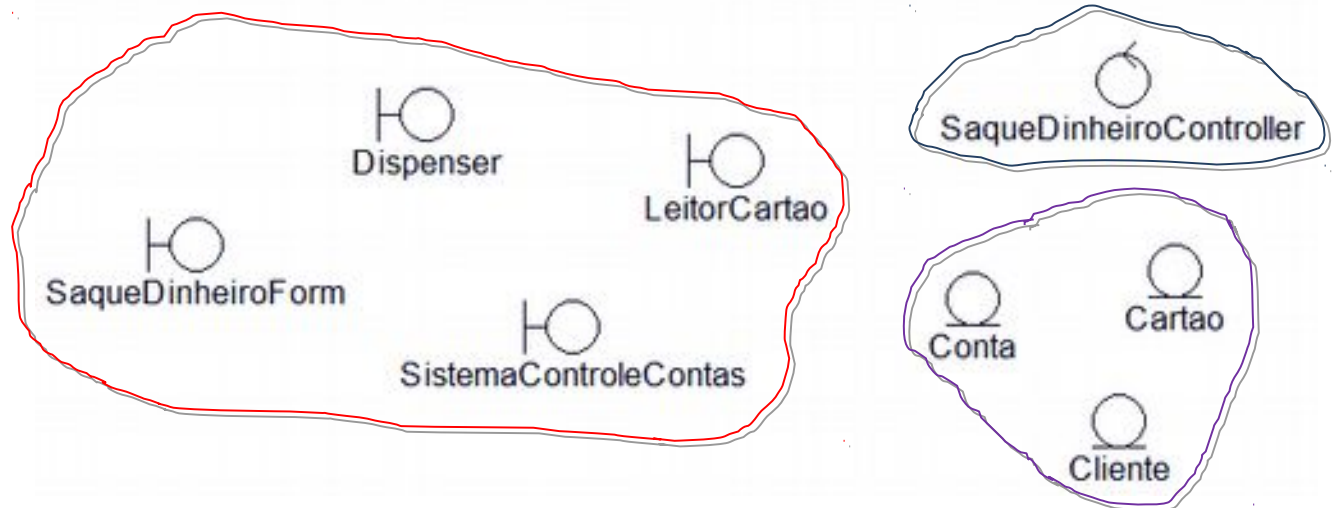
Classes de Análise

➤ Exemplo 1:

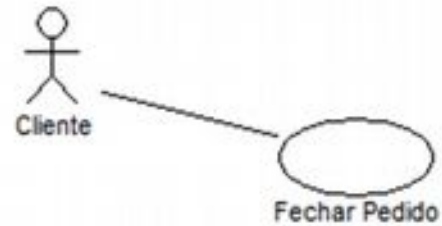


Modelo de Casos de Uso

Classes de Análise



➤ Exemplo 2:

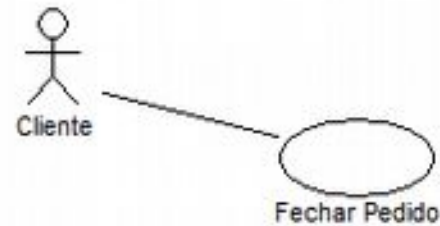


Modelo de Casos de Uso

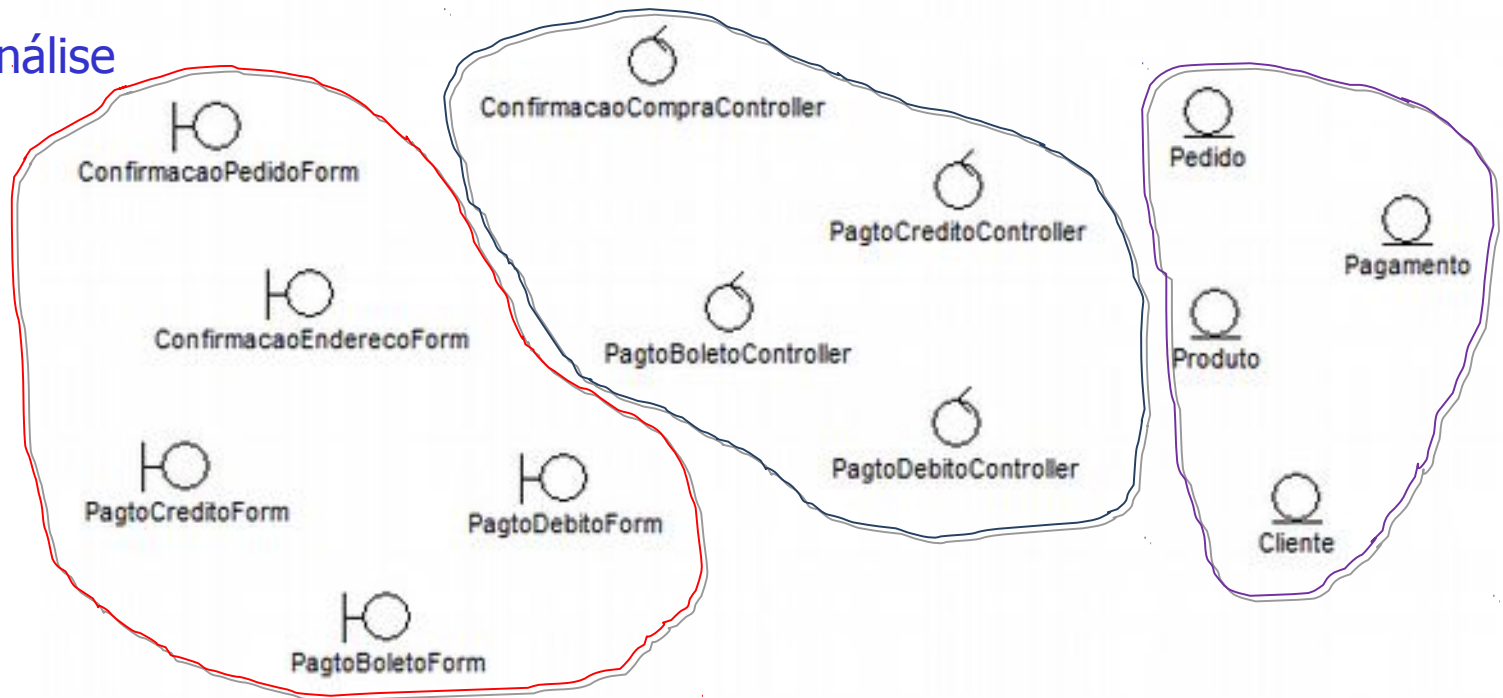
Classes de Análise

➤ Exemplo 2:

Modelo de Casos de Uso



Classes de Análise



- De forma resumida, envolve os seguintes passos:
 1. Suplementar as descrições dos casos de uso
 2. Para cada Realização de Caso de Uso
 1. Localizar Classes de Análise
 2. Distribuir os comportamentos do Caso de Uso para as Classes de Análise
 3. Para cada Classe de Análise
 1. Descrever responsabilidades
 2. Descrever atributos e associações
 3. Qualificar mecanismos de análise
 4. Avaliar resultado da análise



2.2 Distribuir os Comportamentos dos Casos de Uso para as Classes de Análise

- ✓ Definir como os comportamentos descritos nos Casos de Uso serão realizados pelas Classes de Análise descobertas anteriormente, ou seja, definir as **responsabilidades** das classes de análise.
- ✓ Durante essa tarefa novas Classes de Análise podem ser descobertas e a descrição do próprio Caso de Uso pode necessitar de maior detalhamento.
- ✓ Para realizar essa distribuição serão usados os **diagramas de interação**.

- No paradigma OO, objetos interagem uns com os outros para resolver um problema ou realizar determinada tarefa.
- A interação entre objetos acontece por meio da **troca de mensagens**.
- Uma mensagem define como um objeto solicita a outro que este execute determinada tarefa.

- O que esse modelo representa?

- O **ControladorAcesso** solicita ao **Usuario** da classe **validar(login, senha)** o login e a senha.

- Para representar as interações entre objetos, a UML define dois diagramas:
 - ✓ Diagrama de Comunicação (antigo Diagrama de Colaboração)
 - ✓ Diagrama de Sequência
- Principais características:

Diagrama de Comunicação	Diagrama de Sequência
A ordem das mensagens é definida somente no rótulo da mensagem (menos explícito).	Mostra a sequência explícita de mensagens.
Visualização do fluxo de execução é prejudicada.	Melhor para visualizar o fluxo de execução.
Mais simples. Melhor para ser usado em reuniões.	Melhor para cenários complexos.

- Para representar as interações entre objetos, a UML define dois diagramas:
 - ✓ Diagrama de Comunicação (antigo Diagrama de Colaboração)
 - ✓ Diagrama de Sequência
- Principais características:

Antes de seguir adiante vamos ver/rever os conceitos do Diagrama de Sequência na parte 6 do material

A d
sor
explícito).

S.

Visualização do fluxo de execução é prejudicada.

Melhor para visualizar o fluxo de execução.

Mais simples. Melhor para ser usado em reuniões.

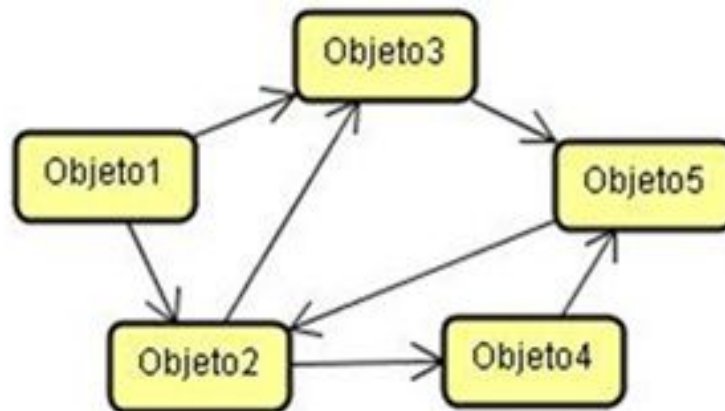
Melhor para cenários complexos.

Projeto de Software Orientado a Objetos

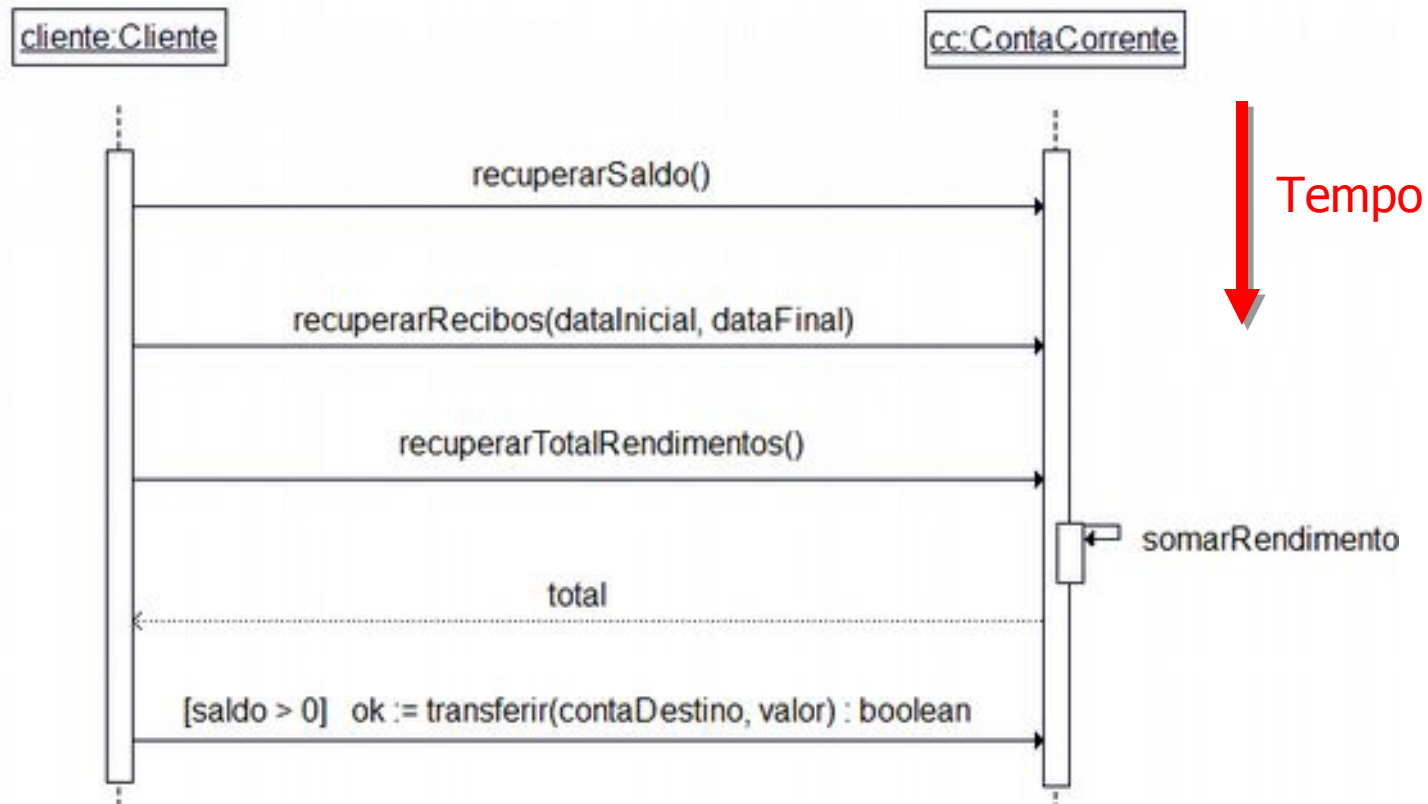
6ª parte

Prof. Jobson Massollar

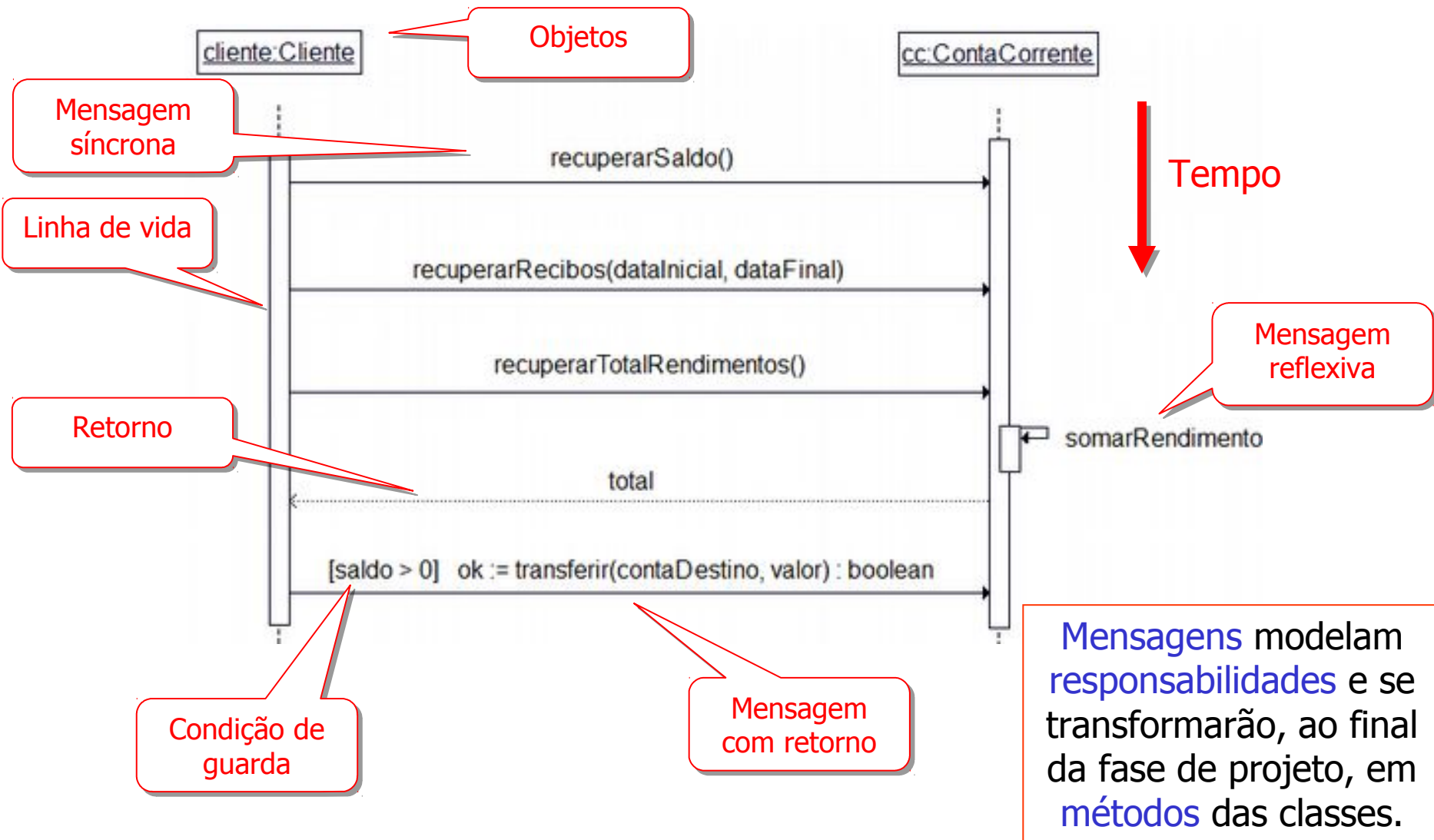
- Os Diagramas de Sequência são usados para modelar a **interação** entre os objetos.
- Em outras palavras, esses diagramas modelam a **troca de mensagens entre os objetos**, ou seja, ajudam a definir os comportamentos (métodos) das classes.



- Os Diagramas de Sequência podem ser usados para apoiar a **identificação** de classes, atributos e métodos necessários para implementar os casos de uso em um sistema orientado a objetos.
- Assim, com o apoio dos Diagramas de Sequência o modelo de classes elaborado inicialmente vai **evoluindo** até um modelo de classes de implementação.
- Os Diagramas de Sequência são, normalmente, usados durante a **transição** da Análise OO para o Projeto OO e na própria fase de Projeto OO.



O Diagrama de Sequência modela a **troca de mensagens** entre **objetos** tendo por base uma **linha de tempo**, ou seja, a **ordem** em que as mensagens são enviadas é capturada nesse diagrama.



➤ Sintaxe geral das mensagens:

```
*[condição] valor := mensagem(parâmetro:tipo) : tipo
```

onde:

mensagem	É o nome da mensagem enviada para o outro objeto (único elemento obrigatório).
valor	É uma "variável" que vai receber o valor retornado pela mensagem.
parâmetro	É um valor enviado na mensagem (podem existir zero ou vários parâmetros).
tipo	É o tipo do parâmetro ou do retorno da mensagem.
condição	É uma expressão booleana que define se a mensagem vai ou não ser enviada.
*	É um marcador que indica que a mensagem será enviada enquanto a condição for verdadeira (loop).

➤ Exemplos:

```
calcularSalario()
```

```
aumentarSalario(percentual)
```

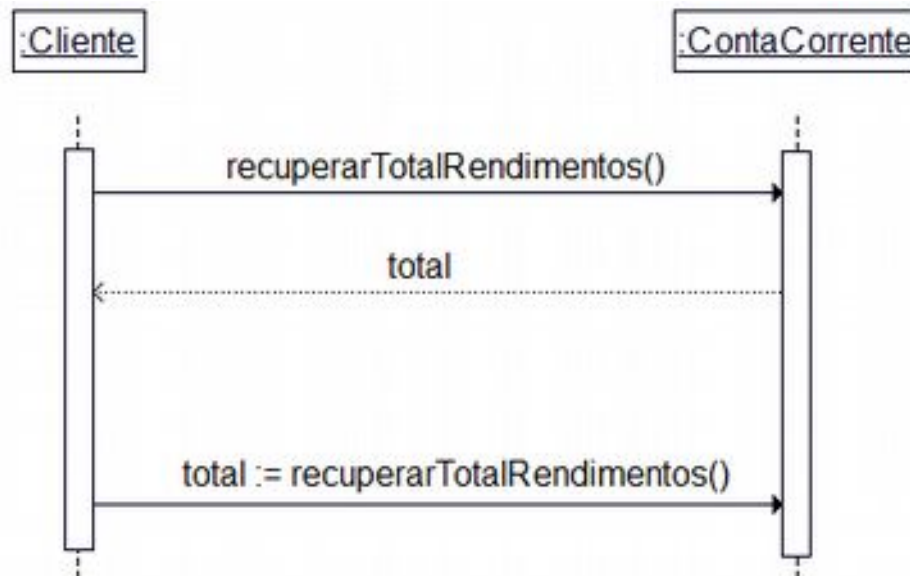
```
[percentual > 0] aumentarSalario(percentual)
```

```
novoSalario := aumentarSalario(percentual) : float
```

```
[percentual > 0] novoSalario := aumentarSalario(percentual)
```

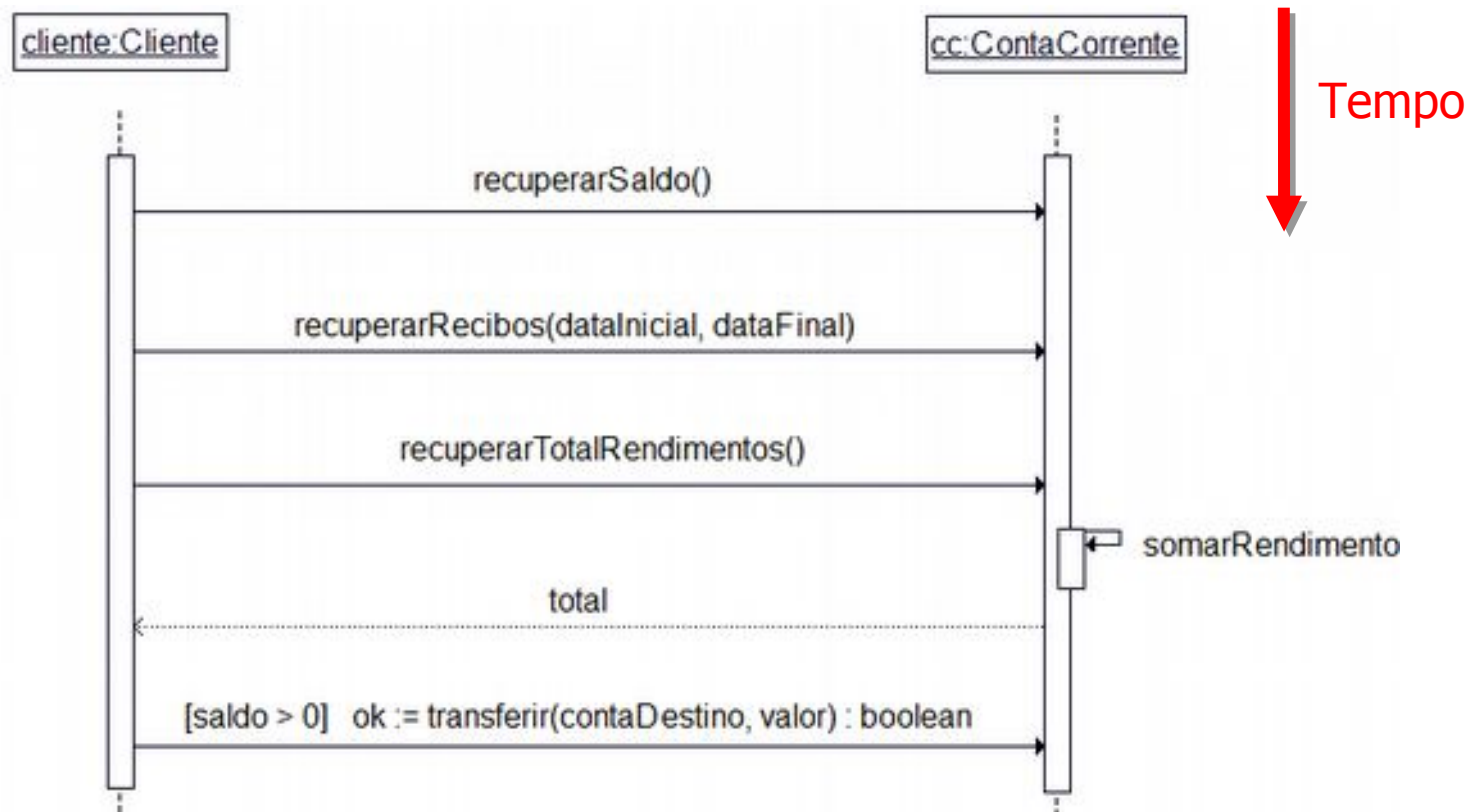
```
*[existe item] registrarItem(código, quantidade)
```

- Existem duas formas de modelar o **valor retornado** pela mensagem:
- a) **Retorno** (seta pontilhada)
 - b) **Atribuição** ($:=$)

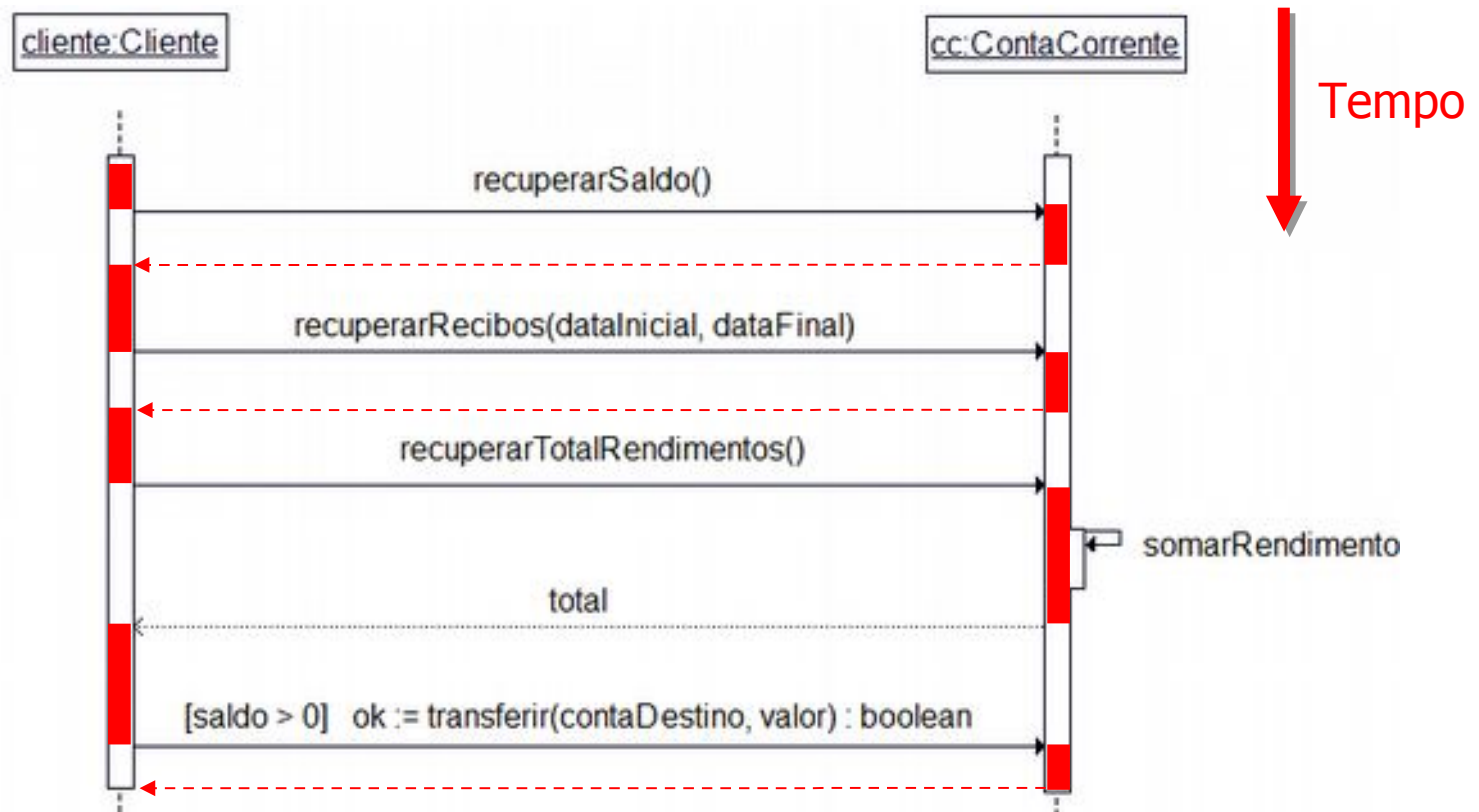


1. Mensagem **síncrona** (seta preenchida) é aquela em que o objeto chamador envia a mensagem para outro objeto e **fica aguardando o retorno** do objeto chamado antes de prosseguir.
 - ✓ Processamento *mono thread*: apenas **um** objeto está em execução em um dado momento.
2. Mensagem **assíncrona** (seta sem preenchimento) é aquela em que o objeto chamador envia a mensagem para outro objeto e **continua o processamento sem aguardar o retorno** do objeto chamado.
 - ✓ Processamento *multi thread*: **vários** objetos podem estar em execução em um dado momento.

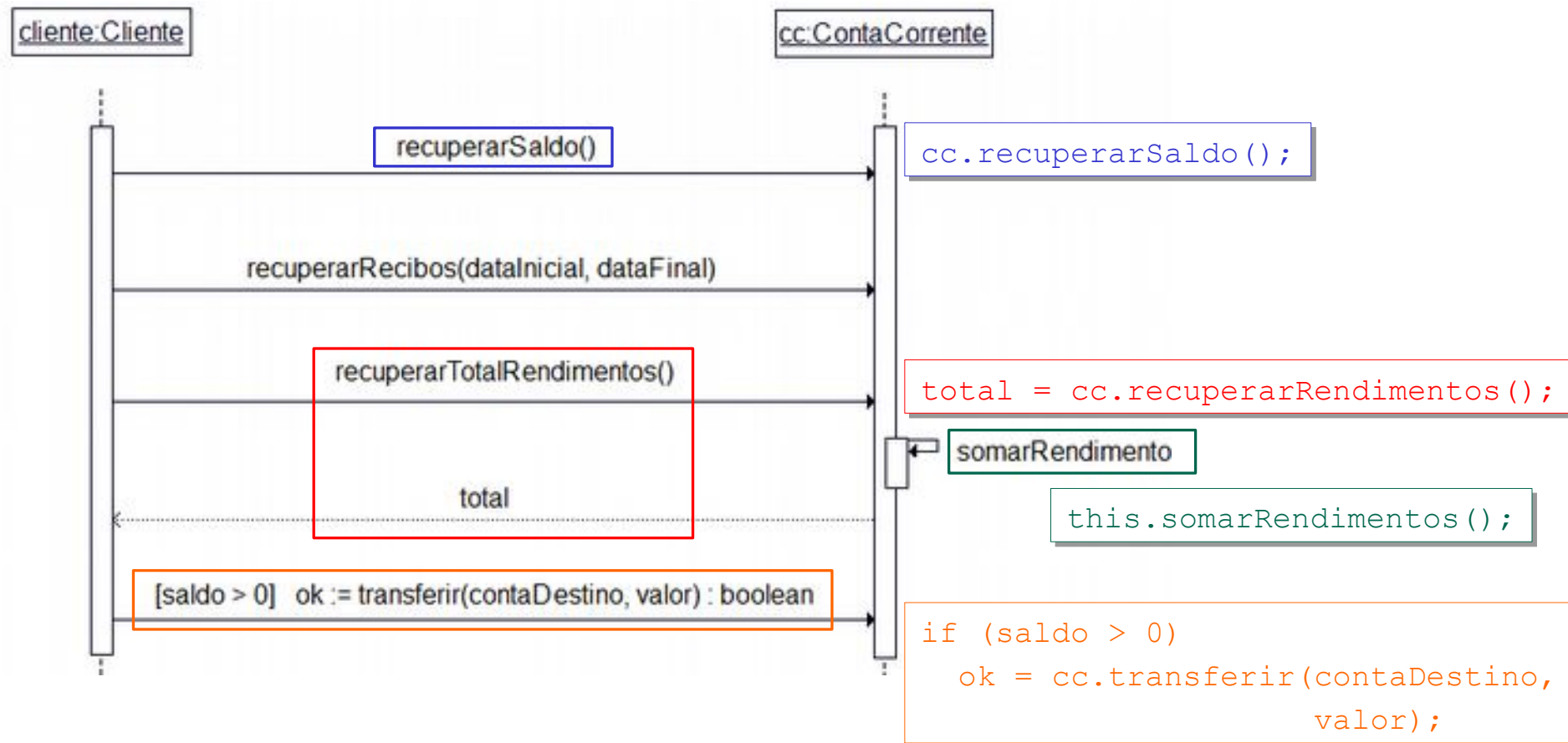
- Quando as mensagens são síncronas, apenas **um** objeto está em execução em determinado momento no tempo:



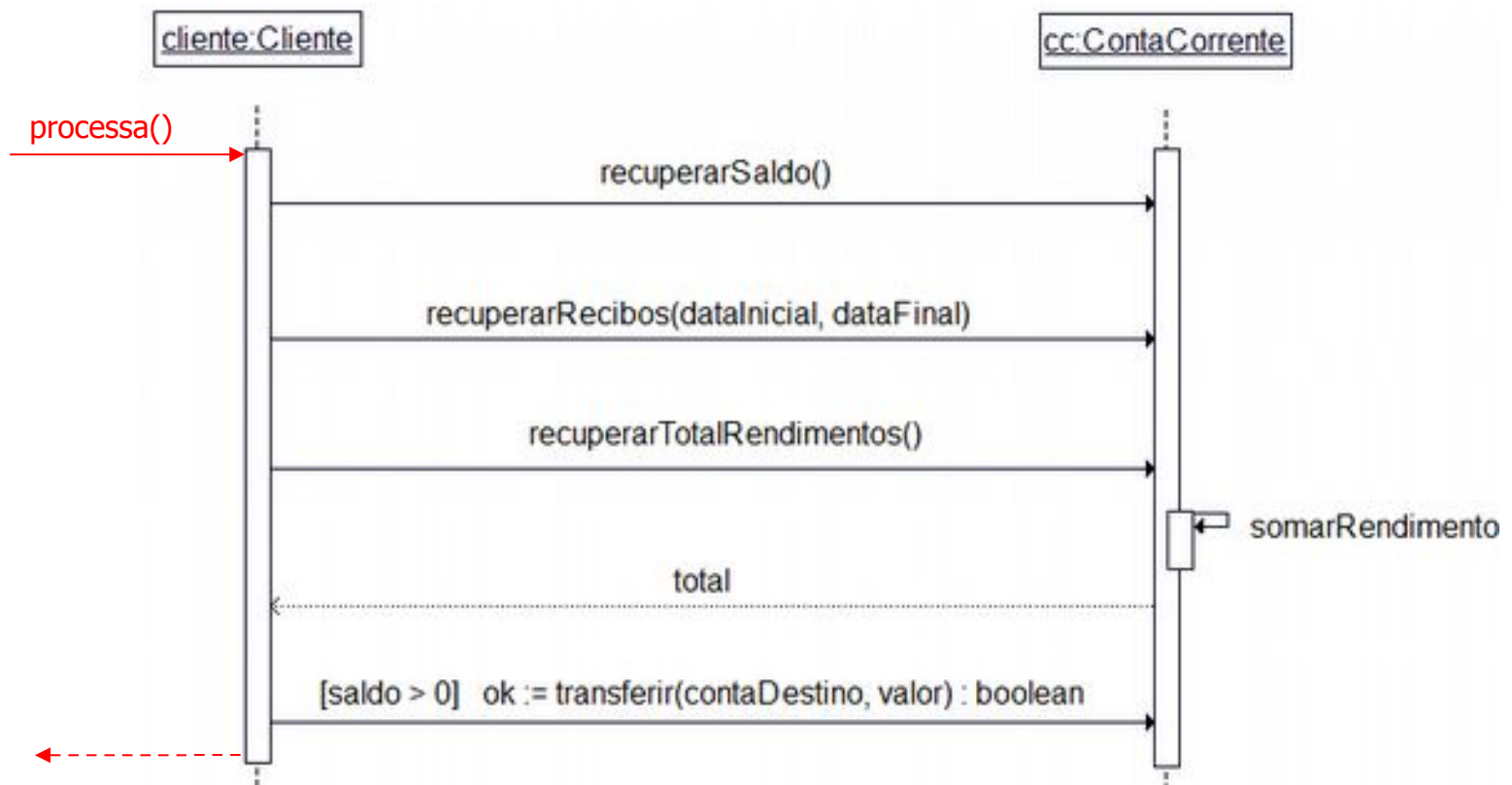
- Quando as mensagens são síncronas, apenas **um** objeto está em execução em determinado momento no tempo:



- Traduzindo essas mensagens para chamadas de **métodos** na linguagem Java:



- Para entender o cenário completo, imagine que o **cliente** recebeu a mensagem síncrona **processa()**:



- Nesse caso, as classes **Cliente** e **ContaCorrente** teriam os seguintes métodos:

```
public class Cliente
{
    public void processa()
    {
        cc.recuperarSaldo();

        cc.recuperarRecibos(dataInicial,
                           dataFinal);

        total = cc.recuperarRendimentos();

        if (saldo > 0)
            ok = cc.transfere(contaDestino,
                             valor);
    }
}
```

```
public class ContaCorrente
{
    public float recuperarSaldo() {
    }

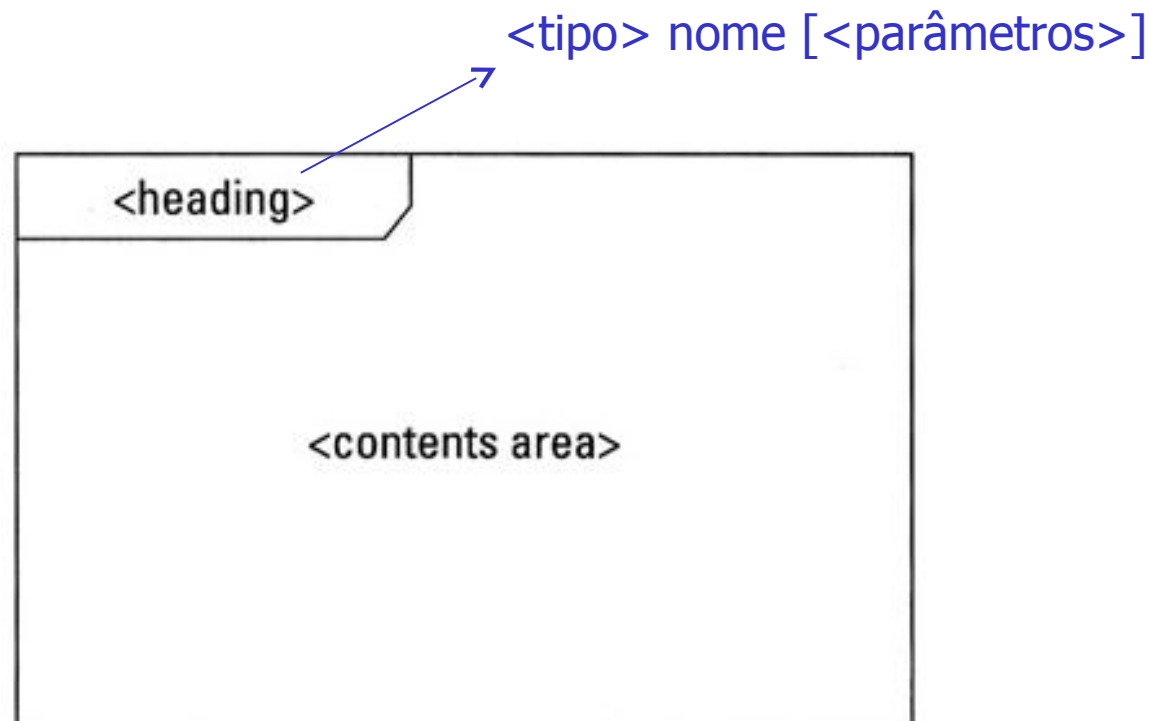
    public void recuperarRecibos(Date dataIni,
                                Date dataFim) {
    }

    public float recuperarRendimentos() {
        total = this.somarRendimentos();
        return total;
    }

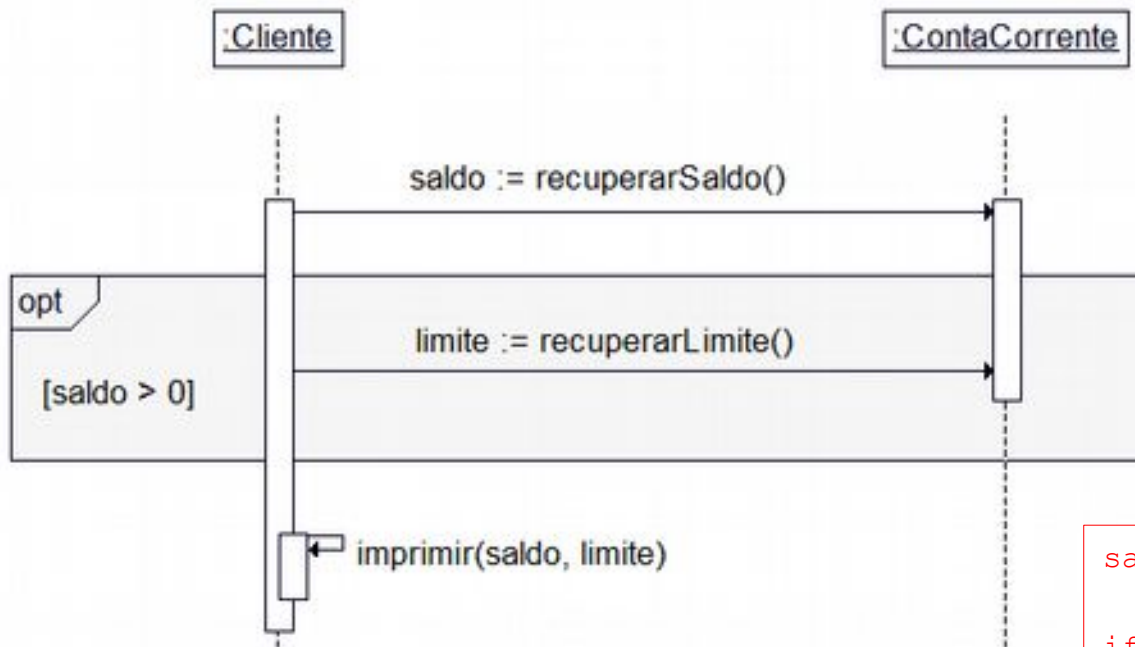
    public void somarRendimentos() {
    }

    public boolean transfere(ContaCorrente destino,
                             float valor) {
    }
}
```

- Uma alteração importante na UML 2.0 foi a introdução de **frames** e seus **tipos**.
- Frame: define um **contexto parametrizável** em um diagrama de seqüência, possibilitando desvios de fluxo e também o seu **reuso**.



- **optional (opt)**: define um bloco de mensagens que são enviadas somente se uma determinada condição for satisfeita (condição de guarda). Equivalente a um *if-then*.

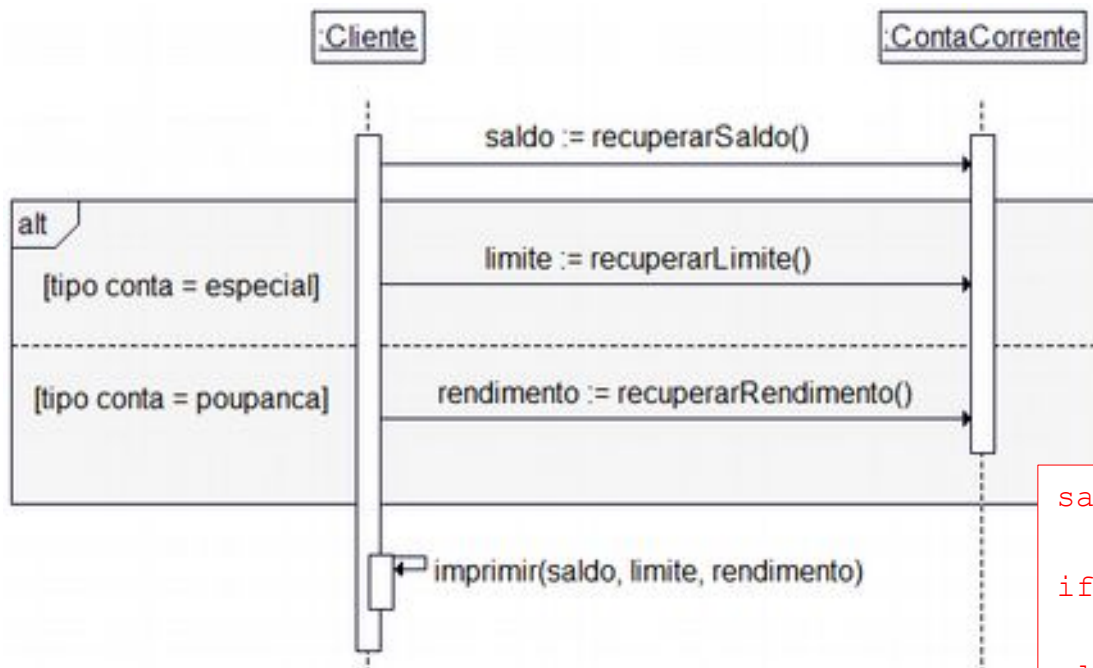


```
saldo = cc.recuperarSaldo();

if (saldo > 0)
    limite = cc.recuperarLimite();
end

imprimir(saldo, limite);
```

- **alternative (alt):** define blocos de mensagens que são enviadas se as condições associadas a cada um deles for satisfeita. Apenas um conjunto é executado. Equivalente a um *if-then-elseif-then-elseif-then-elseif...*

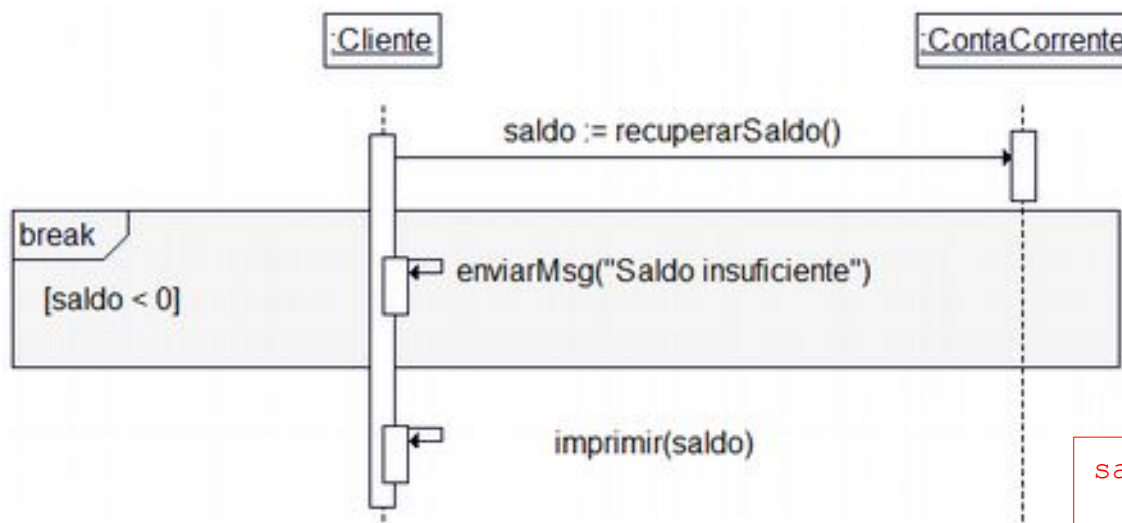


```
saldo = cc.recuperarSaldo();

if (tipoConta == 'E')
    limite = cc.recuperarLimite();
else if (tipoConta == 'P')
    rendimento = recuperarRendimento();
end

imprimir(saldo, limite, rendimento);
```

- **break**: equivalente ao **opt**, só que após a execução do bloco a execução de todo o diagrama é **interrompida** (*return*).

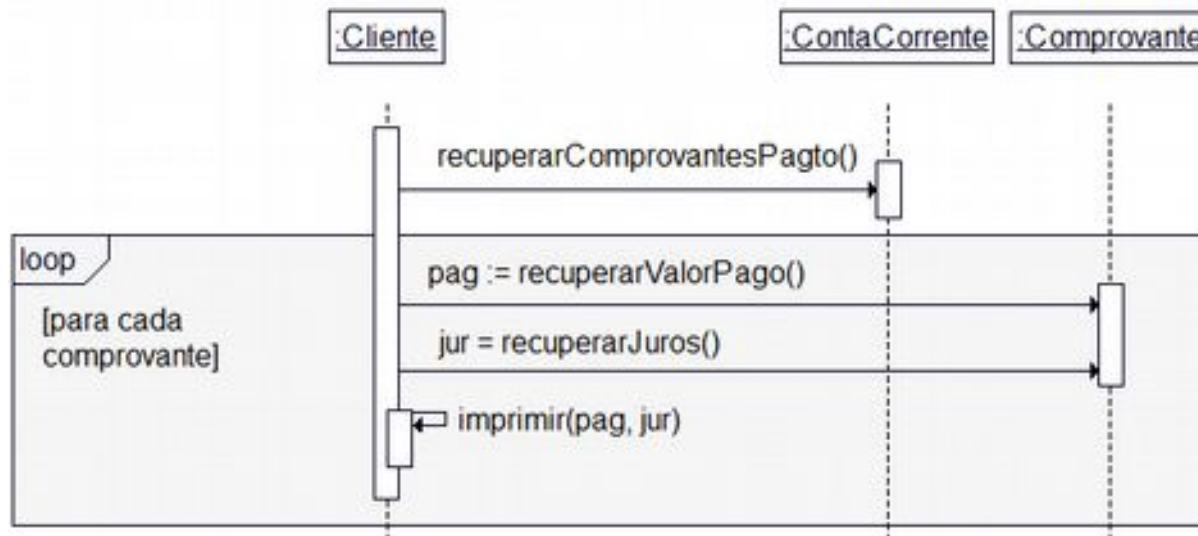


```
saldo = cc.recuperarSaldo();

if (saldo < 0) {
    enviarMsg("Saldo insuficiente");
    return;
}

imprimir(saldo);
```

- **loop**: define um bloco de mensagens que são enviadas em *loop*.



```
comprovantes = cc.recuperarComprovantesPagto();

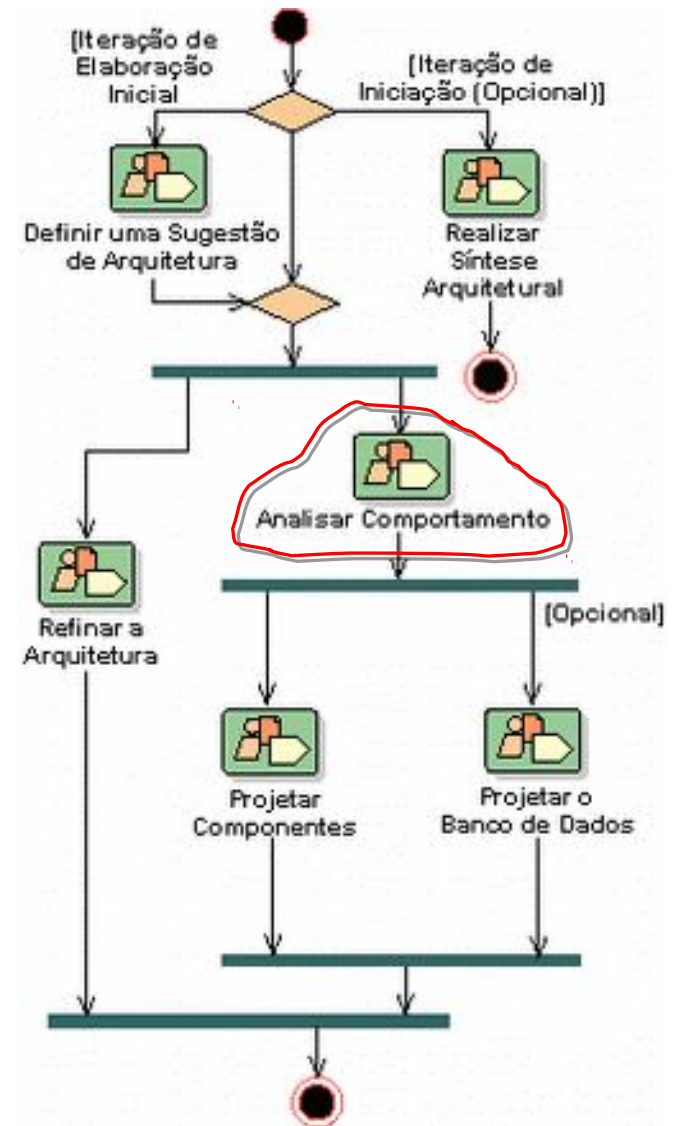
for (Comprovante c in comprovantes) {
    pag = c.recuperarValorPago();
    jur = c.recuperarJuros();
    imprimir(pag, jur);
}
```

Projeto de Software Orientado a Objetos

7ª parte

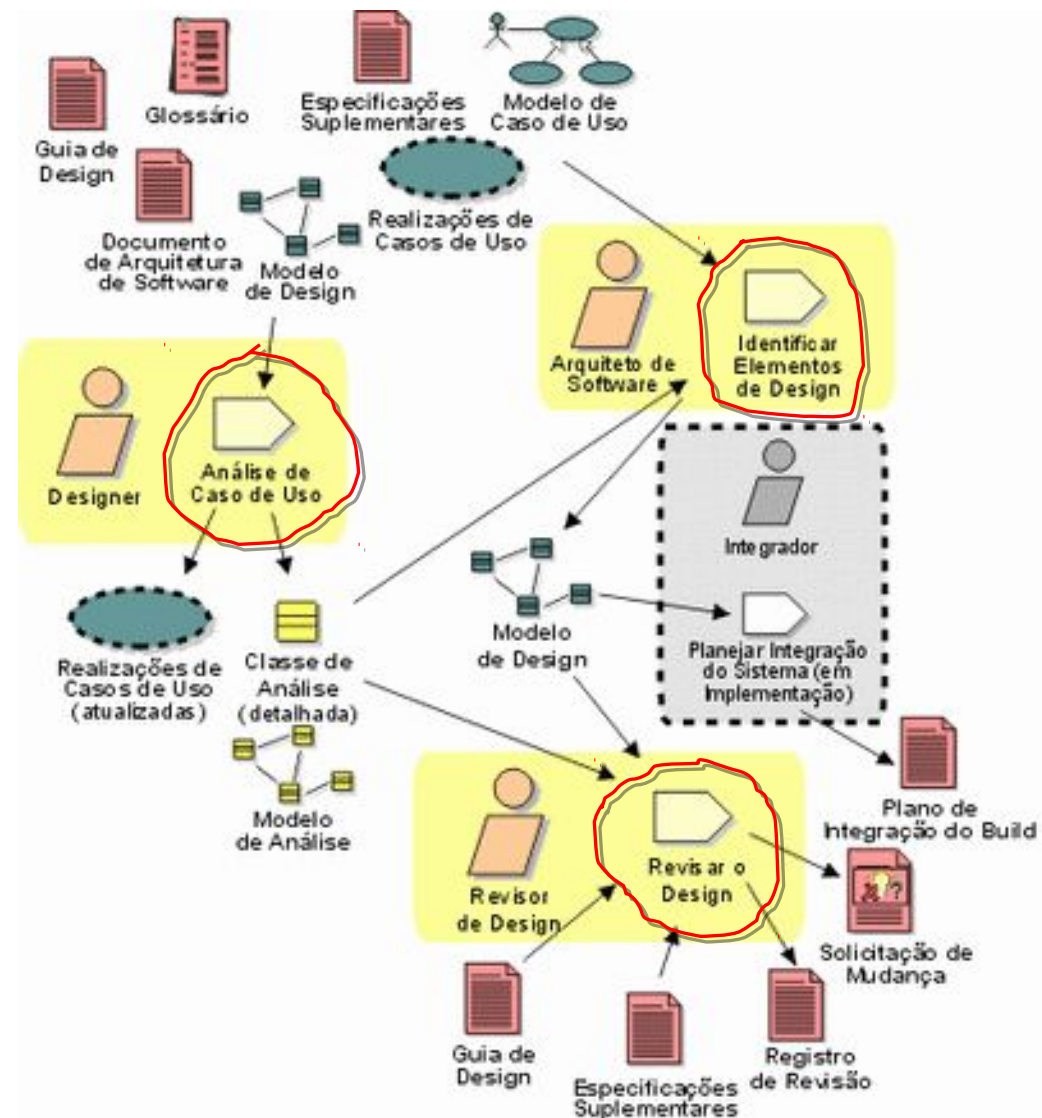
Prof. Jobson Massollar

- A disciplina de **Análise e Design** é composta dos seguintes fluxos de trabalho:
1. Definir uma Sugestão de Arquitetura
 2. **Analisar Comportamento**
 1. **Análise de Caso de Uso**
 2. **Identificar Elementos de Design**
 3. **Revisar o Design**
 3. Projetar Componentes
 4. Projetar BD
 5. Refinar a Arquitetura



➤ Esse fluxo de trabalho é composto de atividades que são executadas da seguinte forma:

1. Realize workshops para Analisar os Casos de Uso e identificar as classes de análise.
2. A partir das classes de análise, realize a Identificação dos Elementos de Design para identificar classes de design e subsistemas.
3. Quando o trabalho de design for concluído e estabilizado, faça a Revisão do Design.



- Essa atividade tem como objetivo:
 - ✓ Identificar as classes de análise que realizam os fluxos de eventos dos casos de uso.
 - ✓ Distribuir o comportamento do caso de uso entre essas classes de análise utilizando as Realizações de Casos de Uso.
 - ✓ Identificar as responsabilidades, os atributos e as associações das classes de análise.

- Essa atividade representa um ponto de transição importante, já que ela estabelece uma ponte entre a visão do usuário do sistema (representada pelos casos de uso) e a visão do designer do sistema (representada, nesse momento, por classes de análise).

- De forma resumida, envolve os seguintes passos:
 1. **Suplementar as descrições dos casos de uso** 
 2. Para cada Realização de Caso de Uso
 1. Localizar Classes de Análise
 2. Distribuir os comportamentos do Caso de Uso para as Classes de Análise
 3. Para cada Classe de Análise
 1. Descrever responsabilidades
 2. Descrever atributos e associações
 3. Qualificar mecanismos de análise
 4. Avaliar resultado da análise

- De forma resumida, envolve os seguintes passos:
 1. Suplementar as descrições dos casos de uso
 2. Para cada Realização de Caso de Uso
 1. Localizar Classes de Análise 
 2. Distribuir os comportamentos do Caso de Uso para as Classes de Análise
 3. Para cada Classe de Análise
 1. Descrever responsabilidades
 2. Descrever atributos e associações
 3. Qualificar mecanismos de análise
 4. Avaliar resultado da análise

2.1 Localizar Classes de Análise

- Identificar um conjunto de elementos (chamados de Classes de Análise) que poderão executar o comportamento descrito nos Casos de Uso.
- Ou seja, a ideia é identificar classes que possam **realizar os comportamentos** definidos no Caso de Uso.
- Uma técnica bastante difundida para realizar esse passo é trabalhar com os conceitos de classes de **fronteira (boundary)**, classes de **controle (control)** e classes de **entidade (entity)**.
- **Atenção:** apesar dos nomes, Boundary, Control e Entity **não** representam um modelo MVC!

Objetivo: permitir ao cliente sacar uma quantia em dinheiro da sua CC.

Ator: cliente

Pré-condições: não há

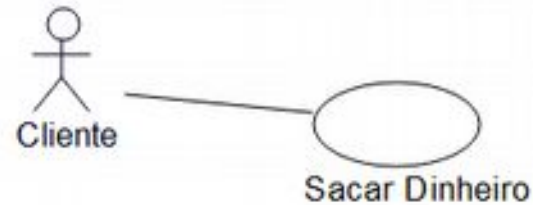
Trigger: cliente aciona a opção de sacar dinheiro

Fluxo Principal:

1. Sistema solicita que o **cliente** insira o **cartão** magnético.
2. Cliente insere o cartão magnético.
3. Sistema valida o cartão magnético.
4. Sistema solicita que o cartão magnético seja retirado.
5. Cliente retira o cartão magnético.
6. Sistema solicita a **senha**.
7. Cliente informa a senha.
8. Sistema valida a senha.
9. Sistema solicita o **valor** desejado.
10. Cliente informa o valor.
11. Sistema verifica **situação** do **caixa**.
12. Sistema verifica situação da **conta**.
13. Sistema entrega o **dinheiro** ao cliente.
14. Cliente retira o dinheiro.
15. Sistema debita o valor sacado da conta.

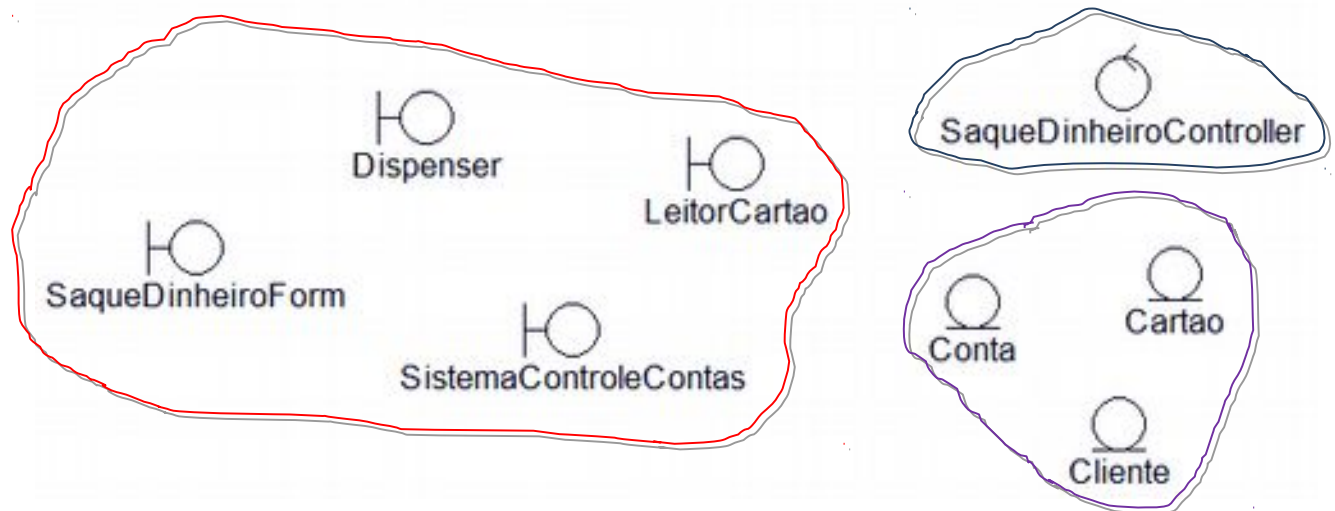
Pós-condições: cliente obtém o valor desejado e o valor sacado é debitado da conta.

➤ Exemplo 1:



Modelo de Casos de Uso

Classes de Análise



- De forma resumida, envolve os seguintes passos:
 1. Suplementar as descrições dos casos de uso
 2. Para cada Realização de Caso de Uso
 1. Localizar Classes de Análise
 2. Distribuir os comportamentos do Caso de Uso para as Classes de Análise 
 3. Para cada Classe de Análise
 1. Descrever responsabilidades
 2. Descrever atributos e associações
 3. Qualificar mecanismos de análise
 4. Avaliar resultado da análise

2.2 Distribuir os Comportamentos dos Casos de Uso para as Classes de Análise

- ✓ Definir como os comportamentos descritos nos Casos de Uso serão realizados pelas Classes de Análise descobertas anteriormente, ou seja, definir as **responsabilidades** das classes de análise.
- ✓ Durante essa tarefa novas Classes de Análise podem ser descobertas e a descrição do próprio Caso de Uso pode necessitar de maior detalhamento.
- ✓ Para realizar essa distribuição serão usados os **diagramas de interação**.

- Para representar as interações entre objetos, a UML define dois diagramas:
 - ✓ Diagrama de Comunicação (antigo Diagrama de Colaboração)
 - ✓ Diagrama de Sequência
- Principais características:

Diagrama de Comunicação	Diagrama de Sequência
A ordem das mensagens é definida somente no rótulo da mensagem (menos explícito).	Mostra a sequência explícita de mensagens.
Visualização do fluxo de execução é prejudicada.	Melhor para visualizar o fluxo de execução.
Mais simples. Melhor para ser usado em reuniões.	Melhor para cenários complexos.

- Para entender como usar o Diagrama de Sequência para realizar essa tarefa vamos usar um exemplo:

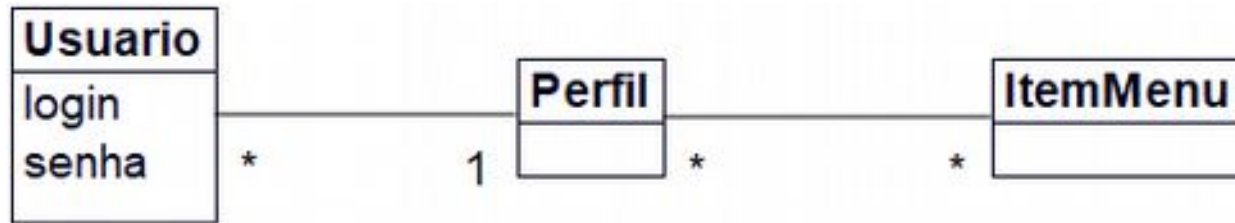
Caso de Uso: Autenticar

Objetivo: permitir ao **usuário** acessar as **funções privadas** do sistema

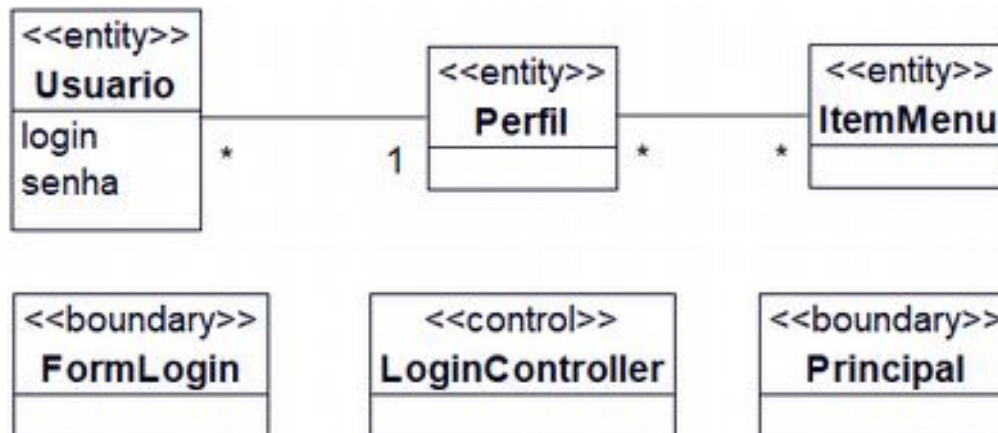
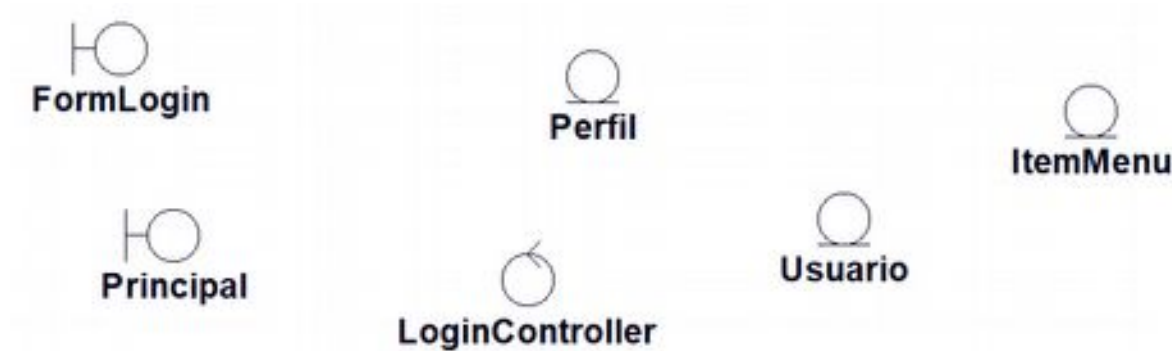
Trigger: usuário acessa a **página de login**

1. Sistema apresenta o **formulário** com **login** e **senha**
2. Usuário preenche o formulário e aciona a opção de "entrar"
3. Sistema valida login e a senha
4. Sistema monta o **menu** de acordo com o **perfil** do usuário
5. Sistema apresenta a **página principal** com o menu.

- Inicialmente a especificação (casos de uso) deve ser analisada para criar o **modelo de domínio** (esse modelo pode ter sido definido anteriormente).



- Analise o caso de uso em questão e defina as suas **classes de análise**.



- De acordo com a filosofia proposta pelos casos de uso, o ator dialoga com o sistema.
- Podemos visualizar esse diálogo como uma troca de mensagens entre o ator e o sistema.
- Sob essa ótica, as ações do caso de uso podem ser categorizadas como:
 1. Mensagens que o ator envia para o sistema.
 2. Mensagens internas do sistema.
 3. Mensagens que o sistema envia para o ator.

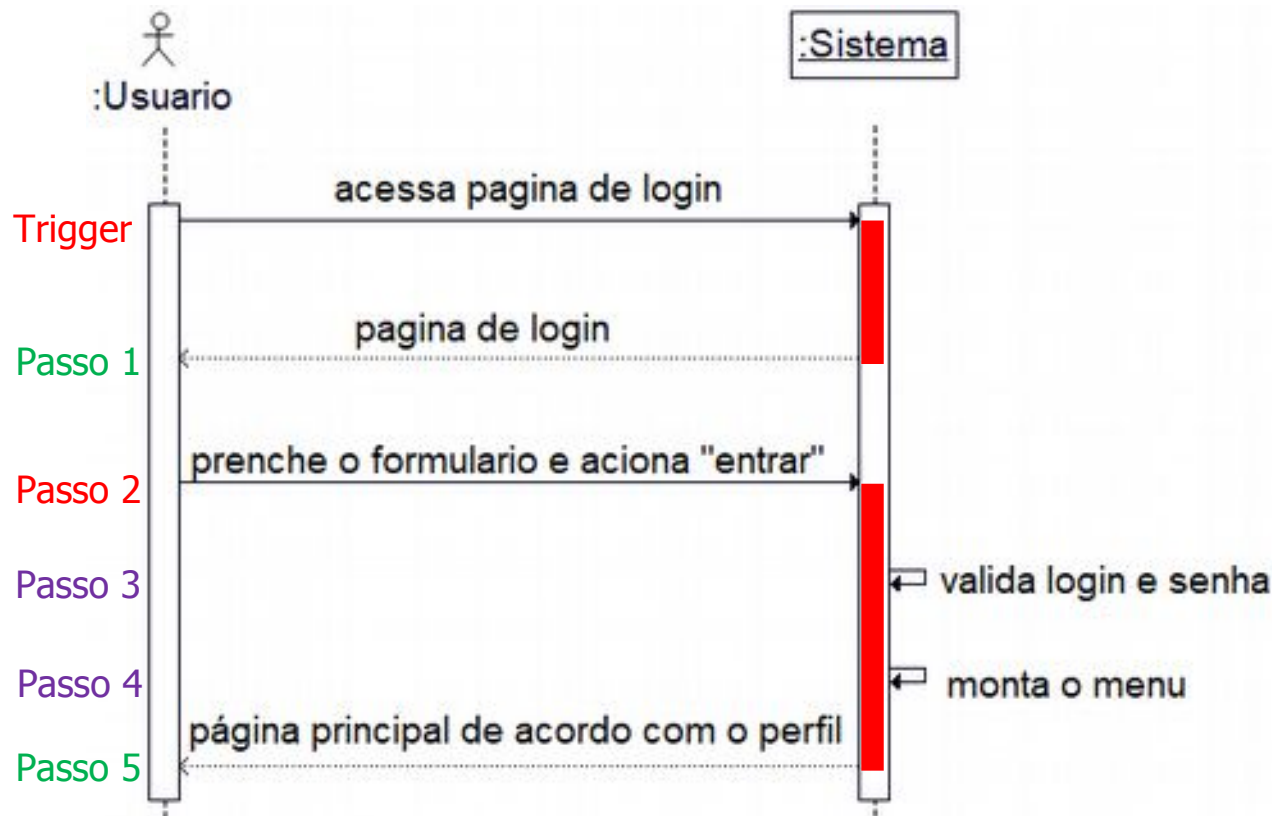
Exemplo 1: Caso de Uso: Autenticar

Objetivo: permitir ao usuário acessar as funções privadas do sistema

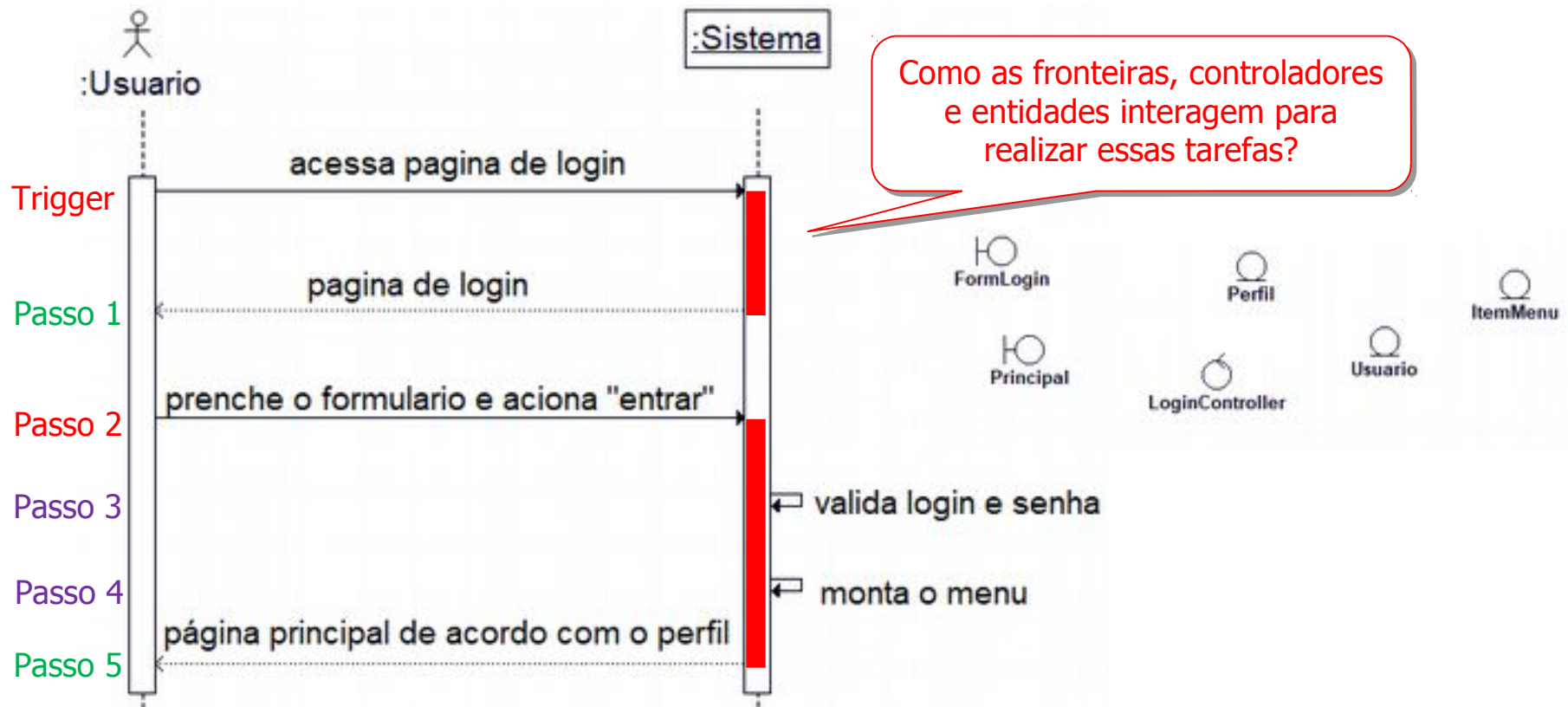
Trigger: usuário acessa a página de login

1. Sistema apresenta o formulário com login e senha
2. Usuário preenche o formulário e aciona a opção de "entrar"
3. Sistema valida login e a senha
4. Sistema monta o menu de acordo com o perfil do usuário
5. Sistema apresenta a página principal com o menu.

- Podemos, então, descrever o caso de uso como uma **troca de mensagens entre o ator e o sistema**.

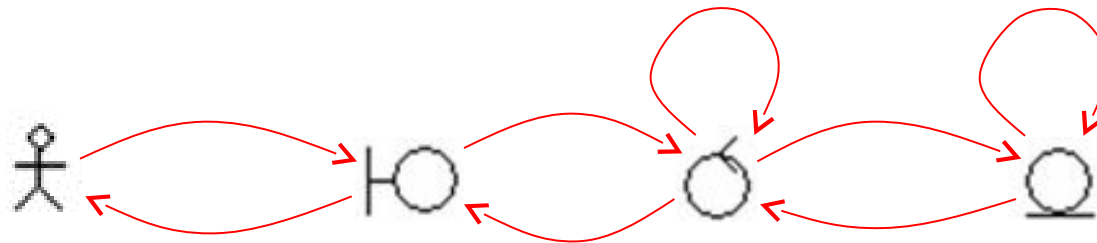


- Podemos, então, descrever o caso de uso como uma **troca de mensagens entre o ator e o sistema**.



- Nos Diagramas de Sequência criados durante a Análise de Casos de Uso, devemos atentar para o seguinte:
 - ✓ As mensagens enviadas entre os objetos significam **intenção**:
 - Não são métodos a serem implementados e nem interfaces.
 - Maiores detalhes serão acrescentados durante a fase de Design.
 - ✓ Seguem os fluxos definidos no caso de uso, ou seja, o diagrama mostra como os objetos **realizam** o caso de uso.
 - ✓ Todos os objetos são "**eternos**", ou seja, não devem ser modeladas a **criação ou destruição dos objetos** nesse momento.
 - ✓ Da mesma forma, não devem ser modelados, nesse momento, a **recuperação ou armazenamento de dados** (em um banco de dados ou outro mecanismo de armazenamento).

- Como distribuir as **responsabilidades** entre as Classes de Análise?
 1. As mensagens entre as classes de análise **só podem ser trocadas** de acordo com o seguinte esquema:



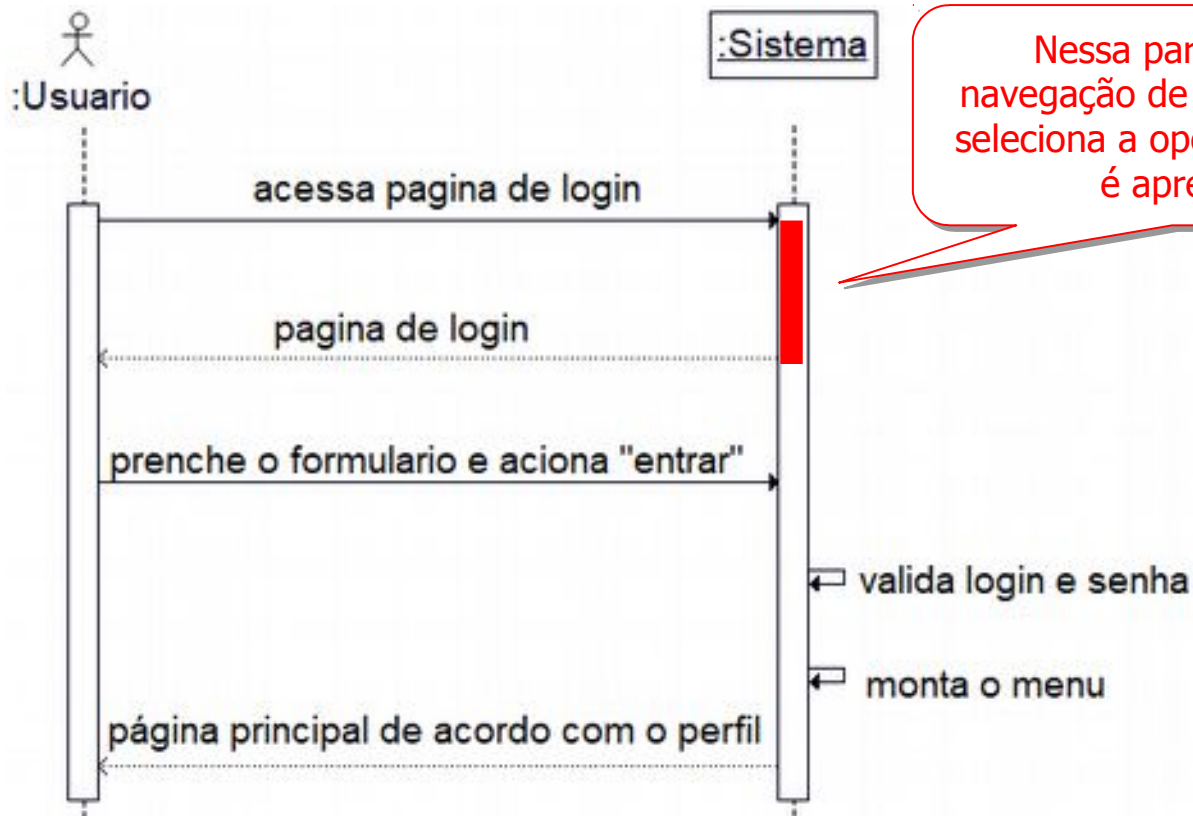
- ✓ As setas indicam quem pode enviar mensagem para quem.

- Como distribuir as **responsabilidades** entre as Classes de Análise?
 2. Use o próprio estereótipo de cada classe como um guia:
 - ✓ **Classes de fronteira**: comportamentos que envolvem comunicação com os atores.
 - ✓ **Classes de controle**: comportamentos que envolvem o controle do fluxo do caso de uso e a coordenação dos demais objetos (fronteira e entidade).
 - ✓ **Classes de entidade**: comportamentos que envolvem a manipulação dos dados que a classe encapsula.

- Como distribuir as **responsabilidades** entre as Classes de Análise?
 3. Avalie a quem pertence os dados necessários para executar a responsabilidade:
 - ✓ Exemplo: "Sistema apresenta os dados do Paciente"
 - ✓ **Onde** estão os dados? Na classe **Paciente**, então ela é responsável por prover esses dados.
 - ✓ **Quem coordena**, ou seja, quem envia para o Paciente a mensagem para obter esses dados? A classe de **Controle**.
 - ✓ Uma vez obtidos, **quem apresenta** os dados? A classe de **Fronteira**.

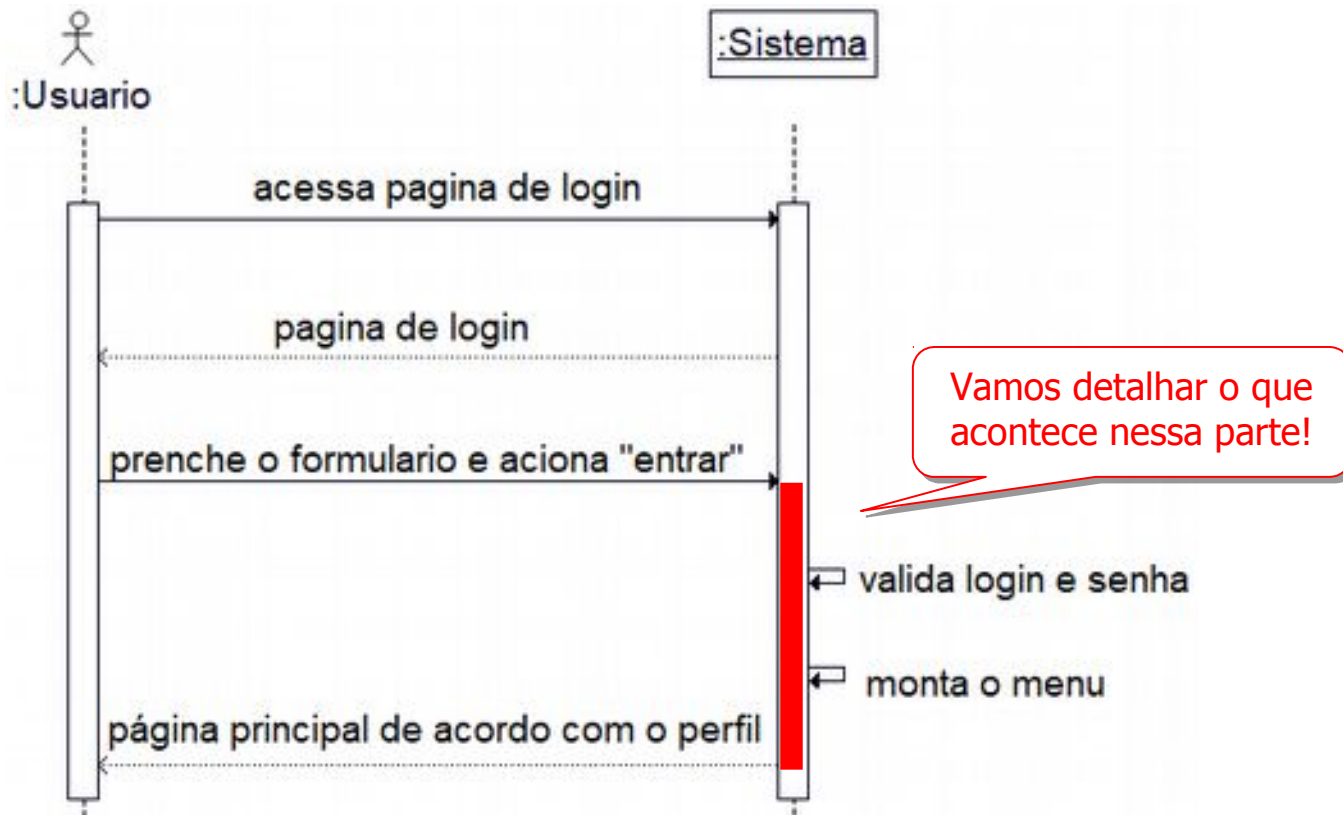
- Como distribuir as **responsabilidades** entre as Classes de Análise?
 3. Avalie a quem pertence os dados necessários para executar a responsabilidade:
 - ✓ Duas opções:
 - a) Se apenas **uma classe** possui os dados, a responsabilidade por fornecer, manipular, alterar, etc., esses dados é da **própria classe** (melhor opção).
 - b) Se os dados estão distribuídos em **duas ou mais classes**:
 - i. Aloque a responsabilidade **em uma dessas classes** e crie relacionamentos entre elas, se necessário.
 - ii. Crie uma **nova classe** e aloque a responsabilidade nela. Crie relacionamentos entre essa classe e as classes que possuem os dados, se necessário.
 - iii. Aloque a responsabilidade na classe de **Controle**.

➤ Exemplo 1: Caso de Uso: Autenticar

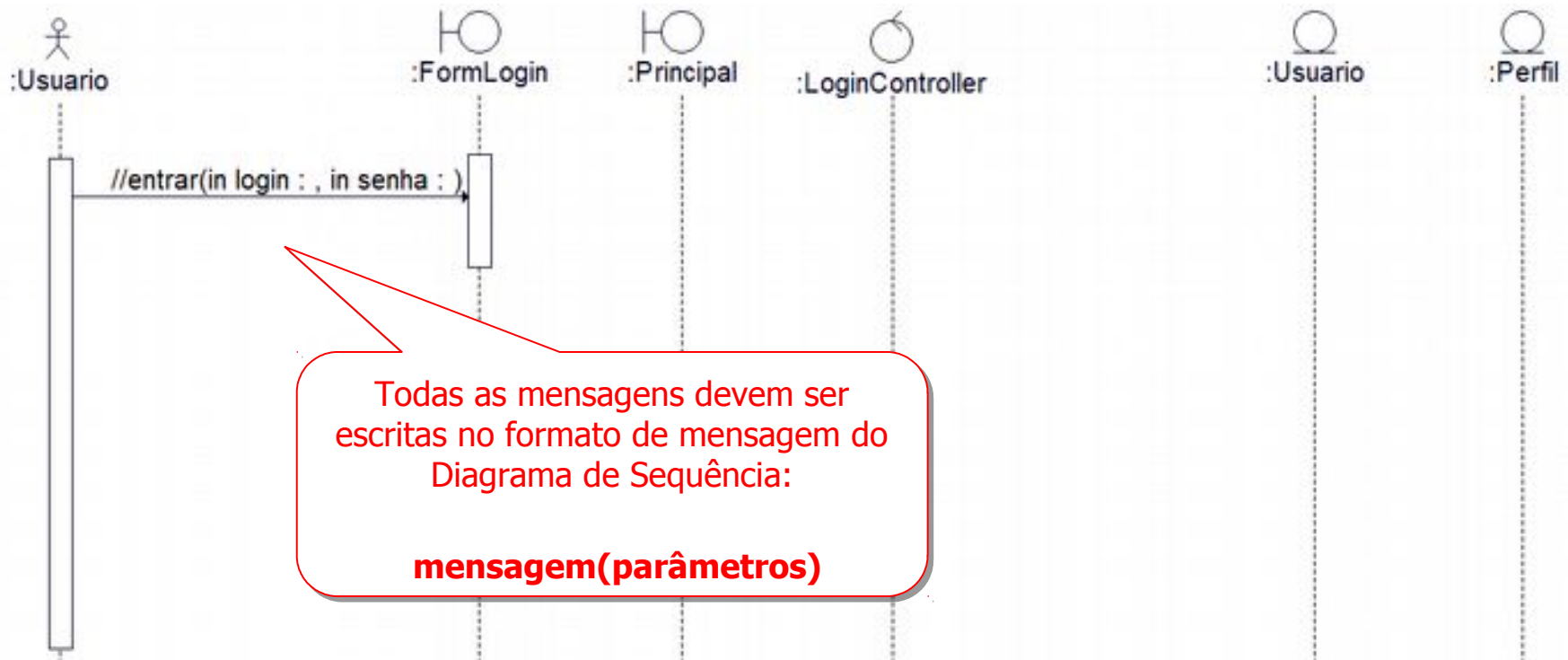


Nessa parte só há uma navegação de página: o usuário seleciona a opção e o formulário é apresentado.

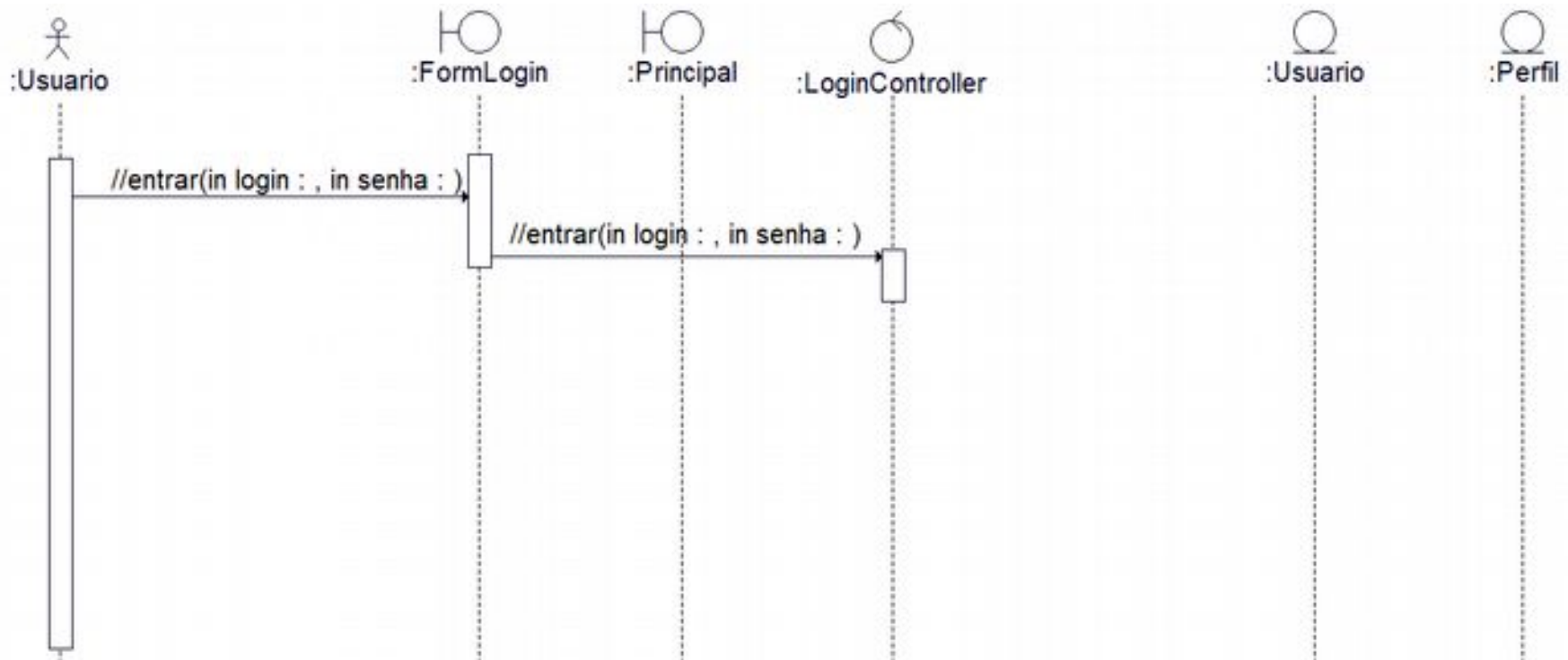
➤ Exemplo 1: Caso de Uso: Autenticar



- Exemplo 1: Caso de Uso: Autenticar
1. Quem recebe as mensagens do ator? As classes de **fronteira**!



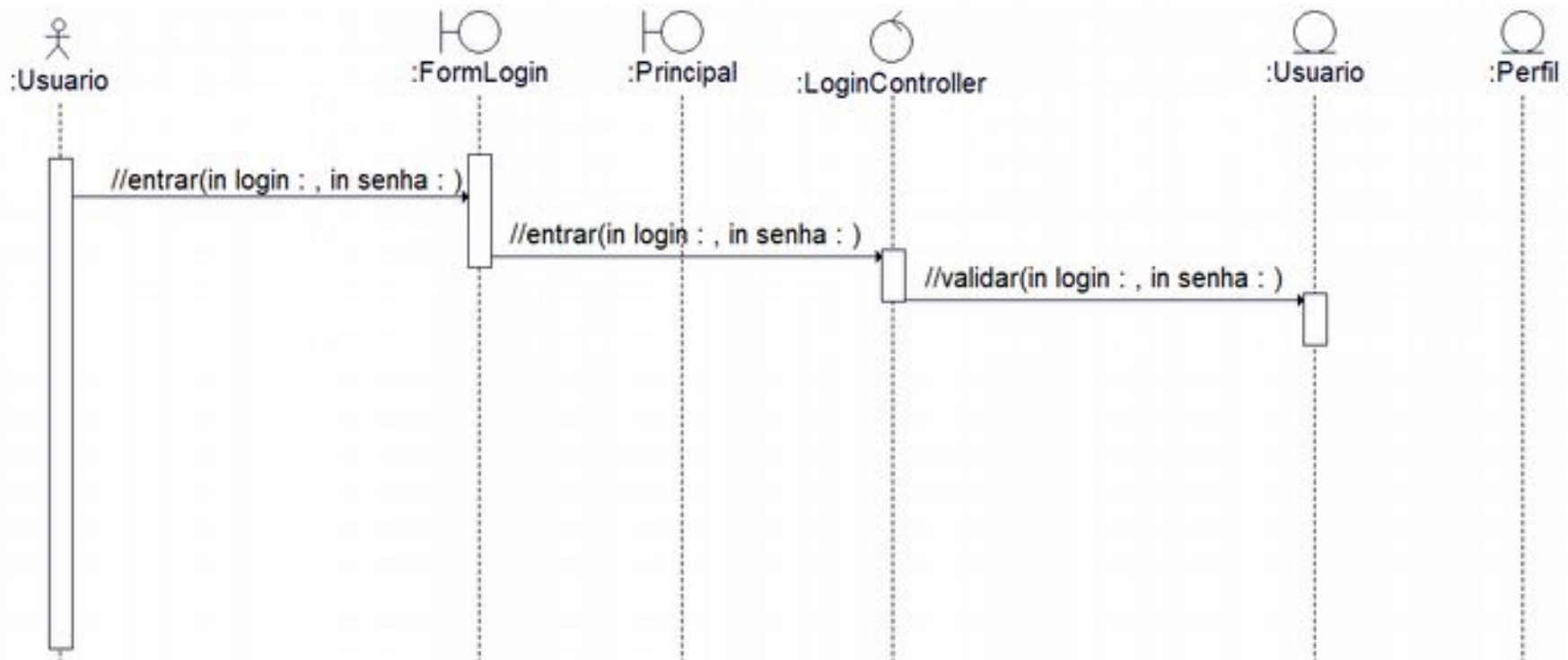
- Exemplo 1: Caso de Uso: Autenticar
2. Quem deve realizar o comportamento `entrar(login, senha)`?
- `LoginController`, porque, por definição, a classe de controle é responsável por coordenar os demais objetos na realização de um comportamento.



➤ Exemplo 1: Caso de Uso: Autenticar

3. Quem deve realizar o comportamento `validar(login, senha)`?

- `Usuário`, porque as informações de login e senha, necessárias para a validação, pertencem à classe `Usuário`.

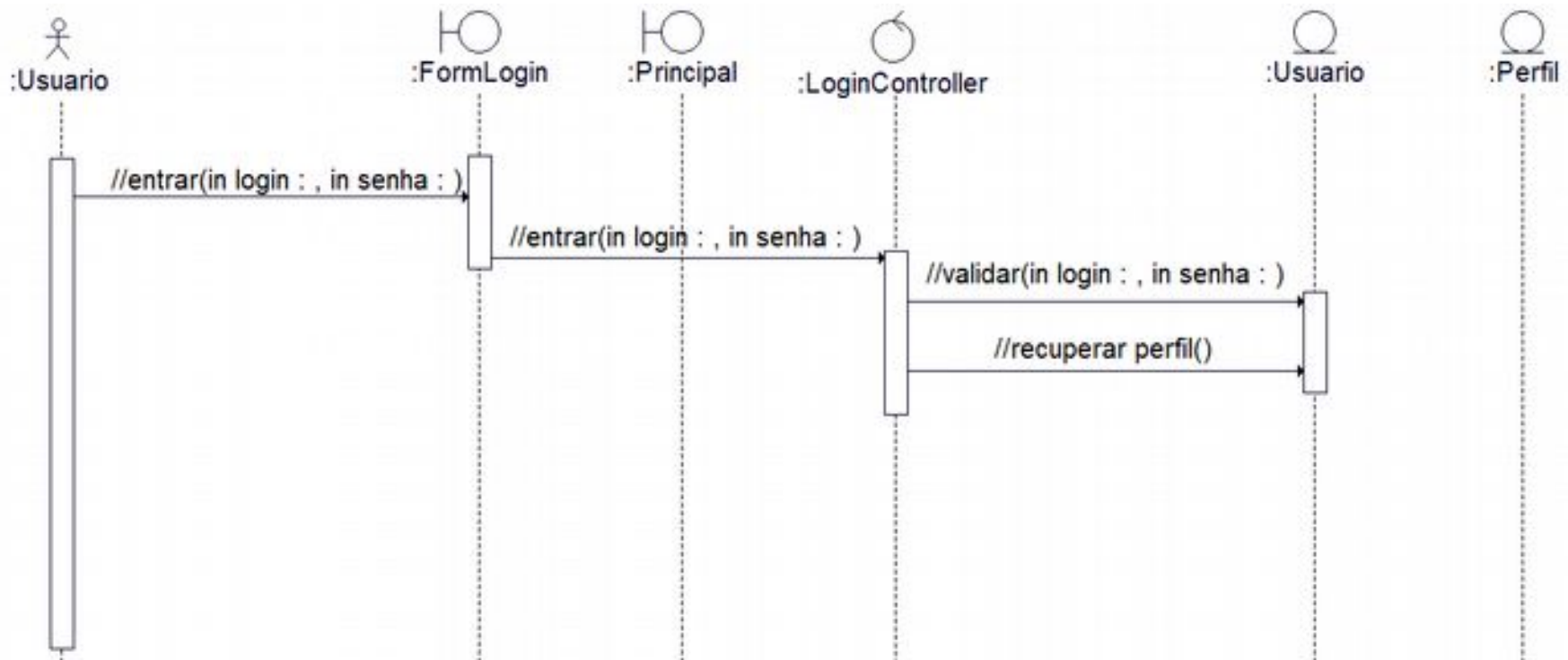


➤ Exemplo 1: Caso de Uso: Autenticar

4. Quem deve realizar o comportamento **montar o menu**?

- **Perfil**, porque a partir do Perfil é possível recuperar o Menu associado ao mesmo (vide navegação **Perfil → Menu** no modelo de classes).
- Entretanto, note que precisamos de um perfil específico: o **perfil do usuário**.
- Como recuperar o perfil do usuário?
- Usando a própria classe **Usuário**, pois cada usuário tem uma referência para o seu perfil (vide navegação **Usuário → Perfil** no modelo de classes).

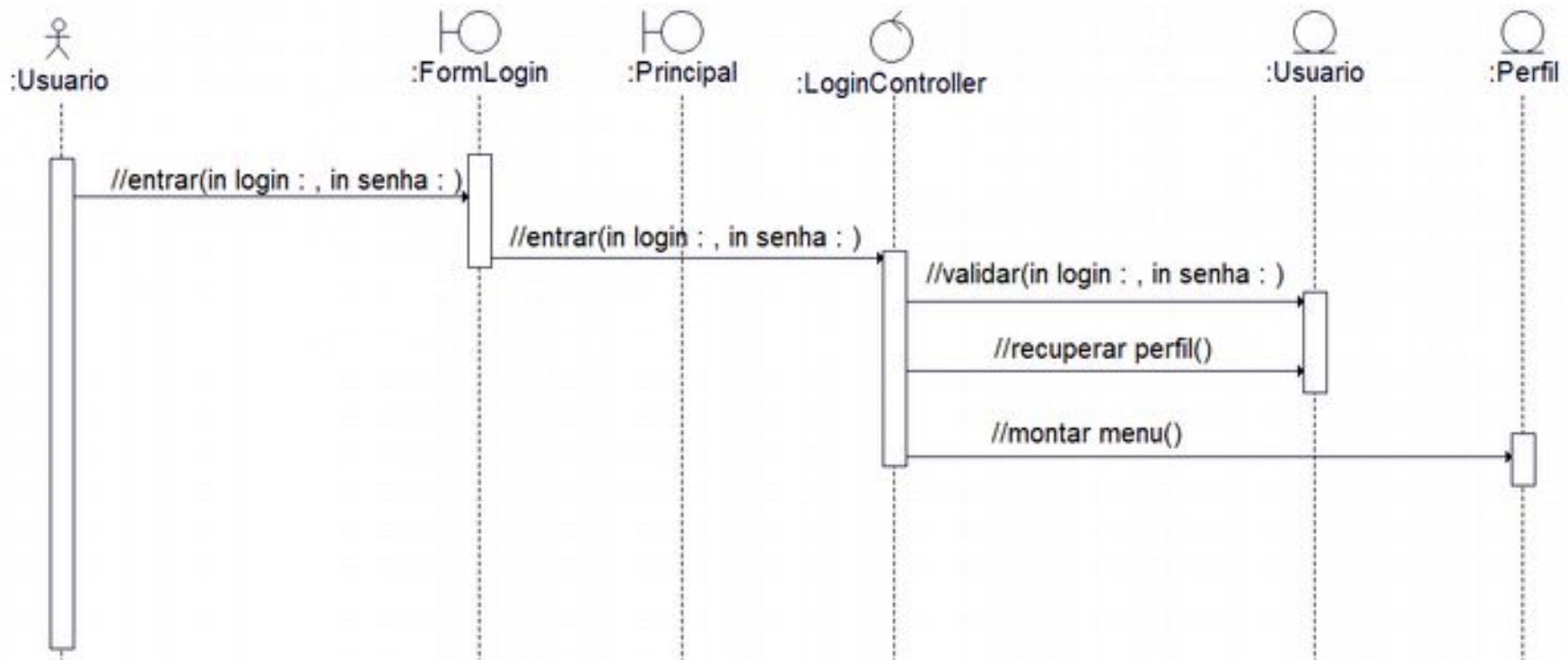
- Exemplo 1: Caso de Uso: Autenticar
4. Quem deve realizar o comportamento **recuperar o perfil**?
- **Usuário**, pois cada usuário tem uma referência para o seu perfil (vide navegação **Usuário** → **Perfil** no modelo de classes).



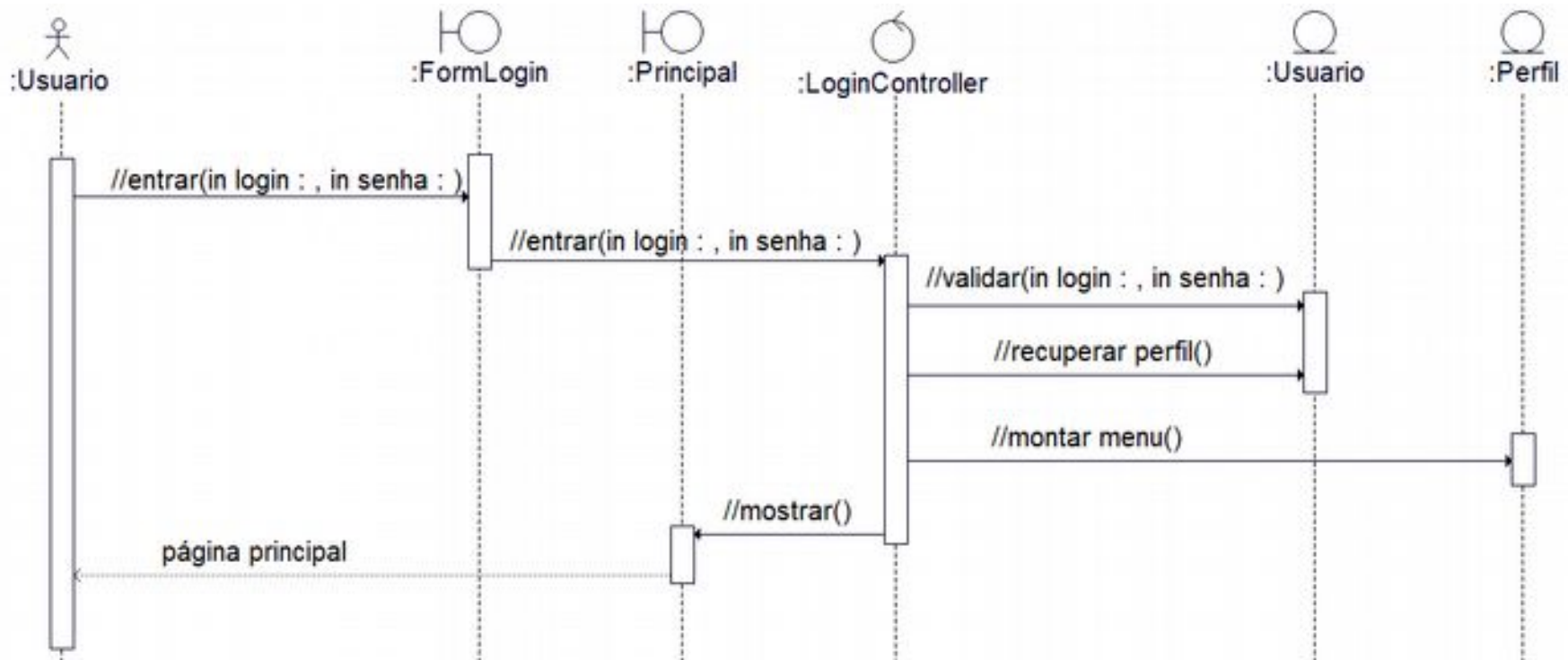
➤ Exemplo 1: Caso de Uso: Autenticar

5. Quem deve realizar o comportamento **montar o menu**?

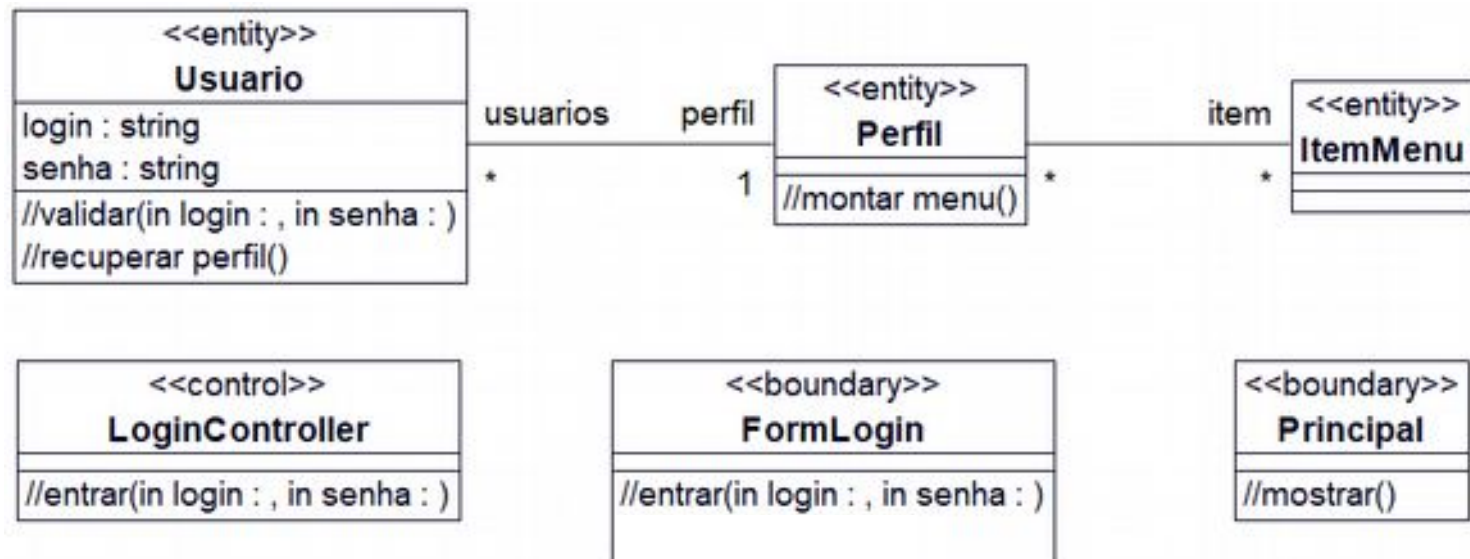
- **Perfil**, porque a partir do Perfil é possível recuperar o Menu associado ao mesmo (vide navegação Perfil → Menu no modelo de classes).



- Exemplo 1: Caso de Uso: Autenticar
6. Quem deve realizar o comportamento **mostrar o menu**?
- **Principal**, porque, por definição, a classe de fronteira é responsável pela comunicação com o ator e apresentação dos dados.



- Nova versão do modelo de classes após a Análise do UC:



➤ Exemplo 2:

- ✓ Em um trecho do caso de uso **Inscriver Aluno em Turma** são escritos os seguintes passos:
 1. ...
 2. Aluno seleciona uma turma e aciona "inscrever"
 3. Sistema inscreve aluno na turma
 4. ...
- ✓ O modelo conceitual extraído desse caso de uso é:

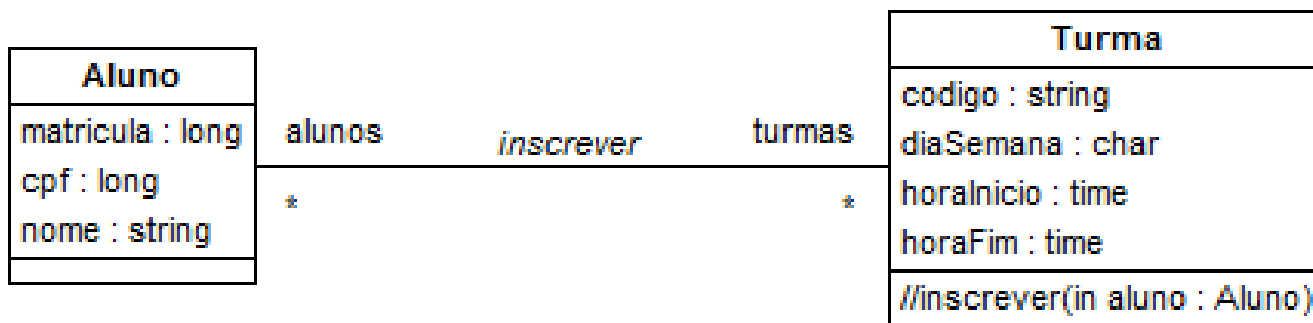
Aluno
matricula : long
cpf : long
nome : string

Turma
codigo : string
diaSemana : char
horaInicio : time
horaFim : time

- A pergunta é: quem é responsável por inscrever o aluno na turma?
 - ✓ A classe Turma?
 - ✓ A classe Aluno?
 - ✓ A classe de Controle?
 - ✓ Outra classe?

- Repare que para realizar essa alocação é preciso ter o dados do aluno e da turma na qual ele vai se inscrever, ou seja, dados que estão em duas classes distintas.

- **Solução:** Aloque a responsabilidade **em uma dessas classes** e crie um relacionamento entre elas, se necessário.
 - ✓ Qual delas? Qualquer uma!
 - a) Na classe **Turma** criar a responsabilidade **inscrever(aluno)**
OU
 - b) Na classe **Aluno** criar a responsabilidade **inscrever(turma)**

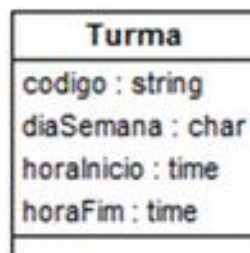
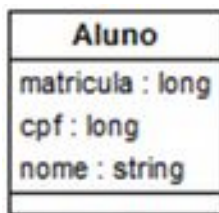


- ✓ Por que não criar uma nova classe ou usar a classe de Controle?

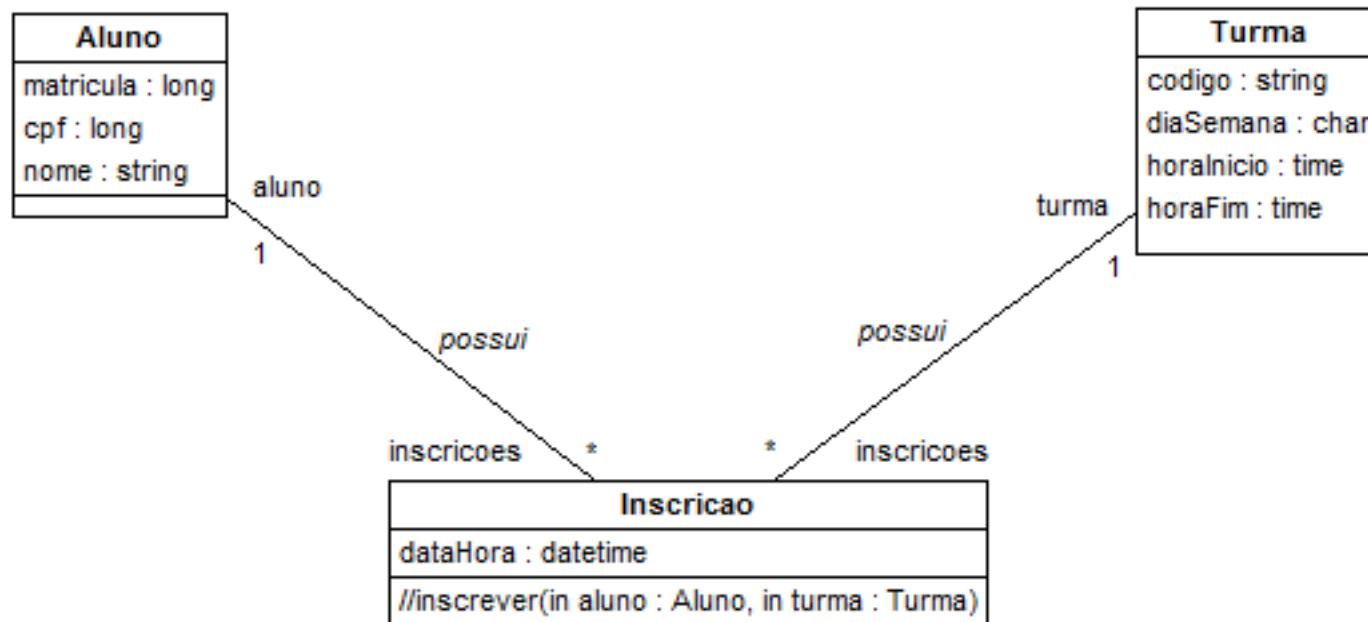
- Porque :
 - a) Criaríamos uma classe "artificial" que teria somente essa responsabilidade e nada mais. Normalmente, isso é indício de um mau projeto.
 - b) A classe de Controle não deve ser responsável por comportamentos que estão estreitamente relacionados com o domínio do problema. Além disso, se esse comportamento fosse necessário em outro caso de uso, teríamos que defini-lo novamente em outra classe de controle.

➤ Exemplo 3:

- ✓ Alterando o caso de uso **Inscriver Aluno em Turma** definido anteriormente:
 1. ...
 2. Aluno seleciona uma turma e aciona "inscrever"
 3. Sistema inscreve aluno na turma e armazena a data/hora
 4. ...
- ✓ Partindo do mesmo modelo conceitual definido antes, pergunta-se: a solução adotada anteriormente é viável?



- Não! Porque precisamos de um local para guardar a **data/hora da inscrição** e esse local não pode ser nem o Aluno nem a Turma.
- **Solução:** criar a classe Inscrição para armazenar a data/hora da inscrição. Essa classe terá um relacionamento com Aluno e Turma e também será responsável por inscrever o aluno na turma.



➤ Exemplo 4:

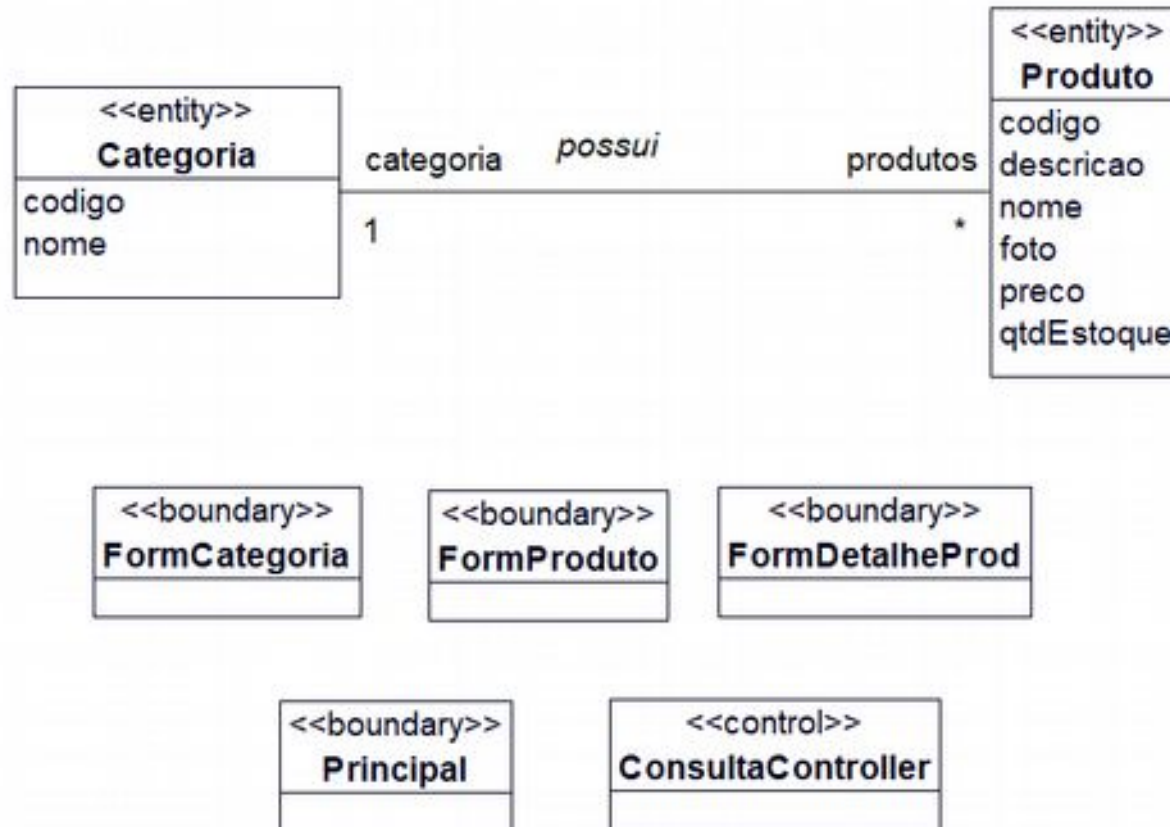
Caso de Uso: Consultar produto

Objetivo: permitir ao ator consultar os dados de um produto

Trigger: a partir da página principal o cliente acessa a página de consulta de produtos

1. O Sistema recupera a lista de categorias de produtos.
2. O Sistema apresenta as categorias.
3. O Cliente seleciona uma categoria.
4. O Sistema recupera os produtos da categoria selecionada.
5. O Sistema apresenta a lista de produtos da categoria selecionada e o total de produtos.
6. O Cliente seleciona um produto.
7. O sistema recupera os dados do produto.
8. O Sistema apresenta o código, o nome, a descrição, a foto, o preço e a quantidade em estoque do produto selecionado.

- Definindo fronteiras, controladoras e entidades:



- Exemplo 4: distribuição de **responsabilidades** entre as Classes de Análise:

Caso de Uso: Consultar produto

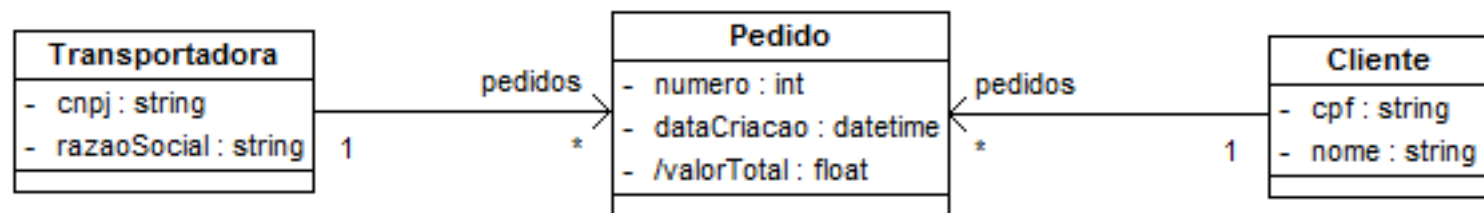
Objetivo: permitir ao ator consultar os dados de um produto

Trigger: **a partir da página principal o cliente acessa a página de consulta de produtos**

1. O Sistema recupera a lista de categorias de produtos.
 2. O Sistema apresenta as categorias.
 3. **O Cliente seleciona uma categoria.**
 4. O Sistema recupera os produtos da categoria selecionada.
 5. O Sistema apresenta a lista de produtos da categoria selecionada e o total de produtos.
 6. **O Cliente seleciona um produto.**
 7. O sistema recupera os dados do produto.
 8. O Sistema apresenta o código, o nome, a descrição, a foto, o preço e a quantidade em estoque do produto selecionado.
- 

- Quem deve ser responsável por recuperar a [lista de categorias](#)?
 - ✓ Não faz sentido alocar essa responsabilidade nas classes Produto ou Categoria. Por que?
 - A classe Produto representa um único produto e está associada a uma única categoria (vide modelo de classes).
 - A classe Categoria representa uma única categoria e seria um desvio de função fazê-la tratar uma lista de categorias.
 - A classe Controle teria que fazer acesso ao banco de dados, o que não faz parte das responsabilidades de um controller.
 - ✓ Vamos examinar a possibilidade de criar uma nova classe, mas como seria essa classe?

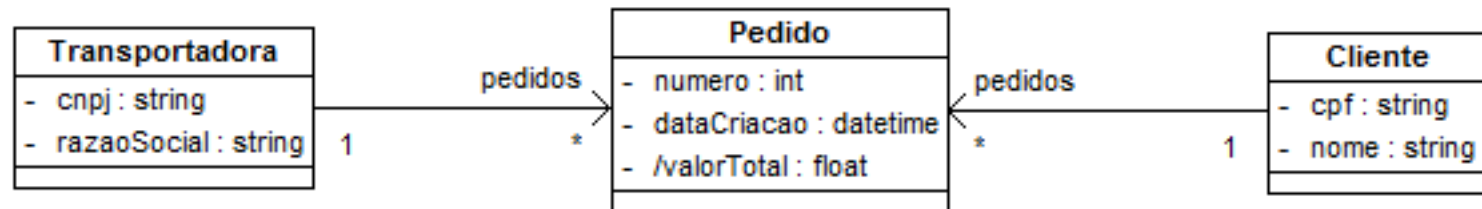
- Observe o seguinte modelo conceitual:



```
public class Transportadora {
    private String cnpj;
    private String razaoSocial;
    private List<Pedido> pedidos;
}
```

```
public class Cliente {
    private String cpf;
    private String nome;
    private List<Pedido> pedidos;
}
```

- Imagine que precisamos obter o somatório dos pedidos transportados por uma transportadora e o somatório dos pedidos feitos por um cliente:



- Então devemos:
1. Implementar o somatório de pedidos da transportadora na classe **Transportadora**.
 2. Implementar o somatório de pedidos do cliente na classe **Cliente**.

- Como a lista de Pedidos pertence a cada uma das classes, temos que implementar dois métodos para realizar o somatório:

```
public class Transportadora {  
    private List<Pedido> pedidos;  
  
    public float calcularValorTotalPedidos() {  
        // Percorre a lista de pedidos e faz o somatório  
    }  
}
```

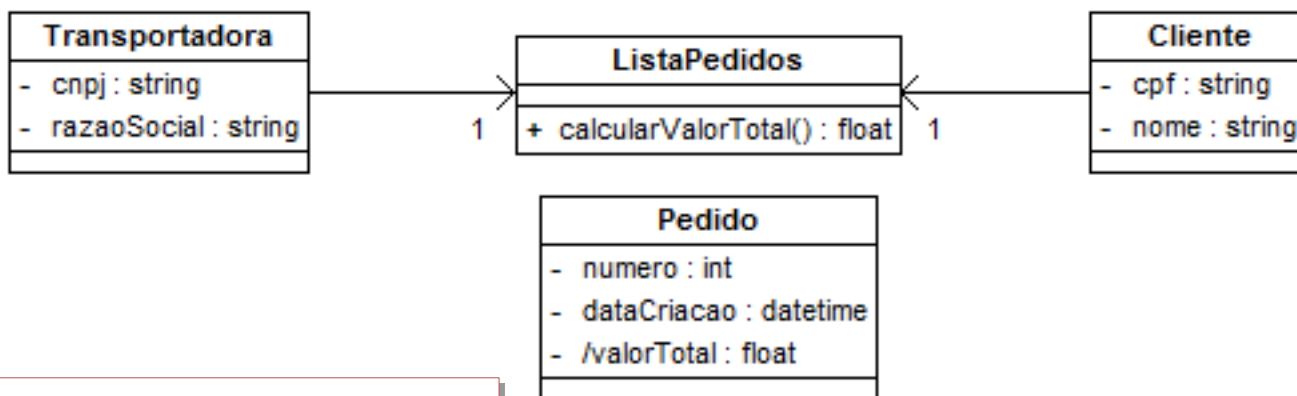
```
public class Cliente {  
    private List<Pedido> pedidos;  
  
    public float calcularValorTotalPedidos() {  
        // Percorre a lista de pedidos e faz o somatório  
    }  
}
```

Onde está o
problema
agora?



- Note que, nesse caso, temos uma estrutura de dados (lista) e precisamos implementar operações sobre essa estrutura.
- Faz sentido, então, **criar uma classe para representar essa estrutura**.
- De uma forma geral, podemos afirmar que:
 - ✓ **Coleções de objetos** podem ser tratadas como **elementos de primeira categoria**, porque elas também podem possuir **atributos e comportamentos**.
 - ✓ Ou seja, podem existir atributos ou comportamentos que são inerentes a uma **coleção de objetos** e não a um elemento individualmente.

- Alterando o modelo para incorporar a lista de pedidos:



```

public class Transportadora {
    private String cnpj;
    private String razaoSocial;
    private ListaPedidos pedidos;
}
    
```

```

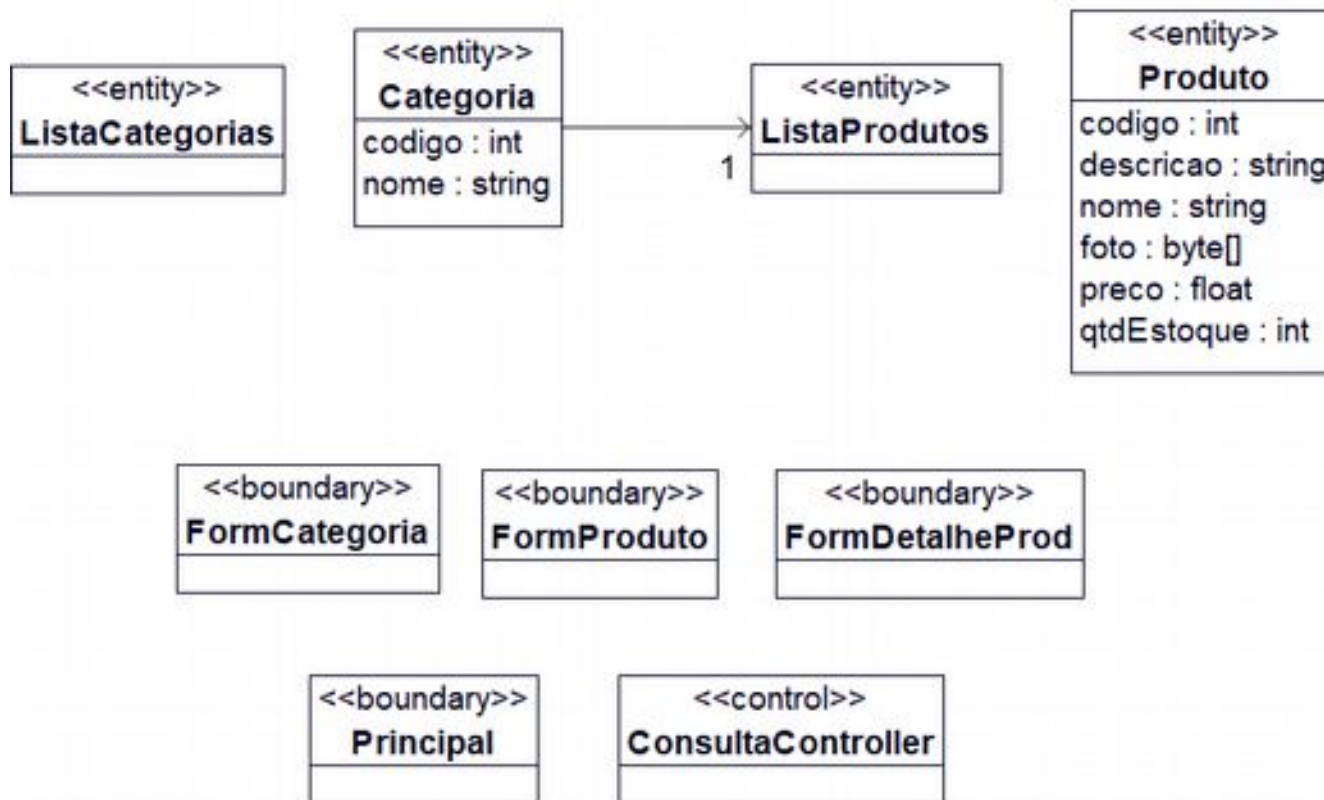
public class Cliente {
    private String cpf;
    private String nome;
    private ListaPedidos pedidos;
}
    
```

```

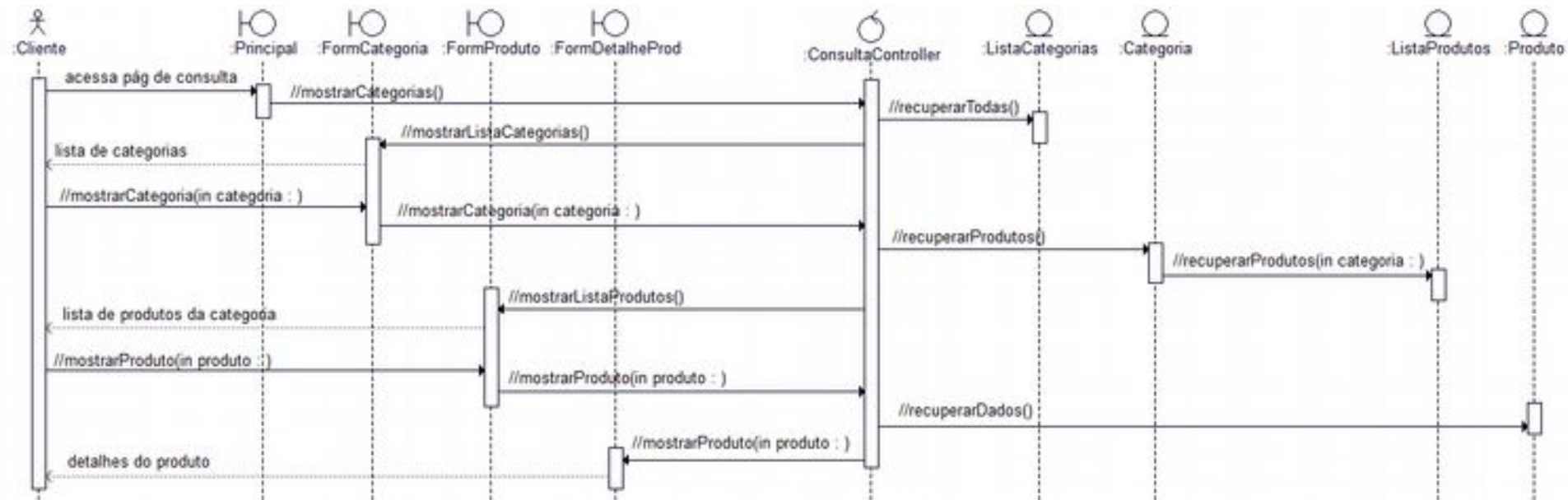
public class ListaPedidos {
    private List<Pedidos> pedidos;

    public float calcularValorTotal() {
        // Percorre a lista de pedidos e
        // faz o somatorio
    }
}
    
```

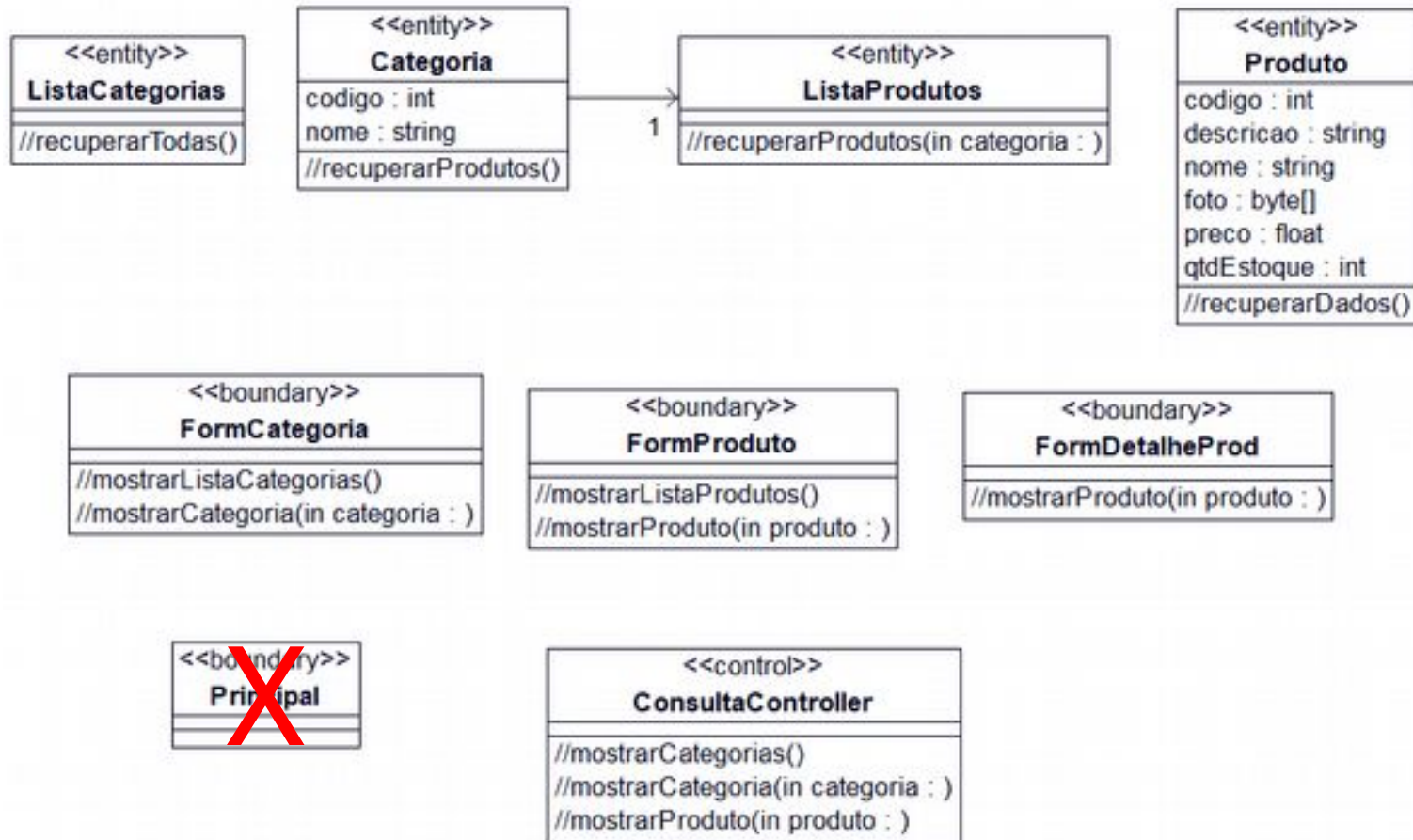
- Voltando para o caso de uso Consultar Produto, vamos criar uma entidade para a **lista de pedidos** e outra para a **lista de produtos** e refazer os relacionamentos.



- Distribuição de responsabilidades pelas classes de análise:



- Modelo de Classes após a Análise do UC:



➤ Tratamento de Fluxos Alternativos e de Exceção

- ✓ Nos casos de uso apresentados até agora temos apenas o Fluxo Principal.
- ✓ Para representar Fluxos Alternativos e de Exceção no Diagrama de Sequência usaremos os frames **opt**, **alt** ou **break**.

- Alterando o caso de uso anterior para acrescentar um **Fluxo Alternativo**:

Caso de Uso: Autenticar

Objetivo: permitir ao usuário acessar as funções privadas do sistema

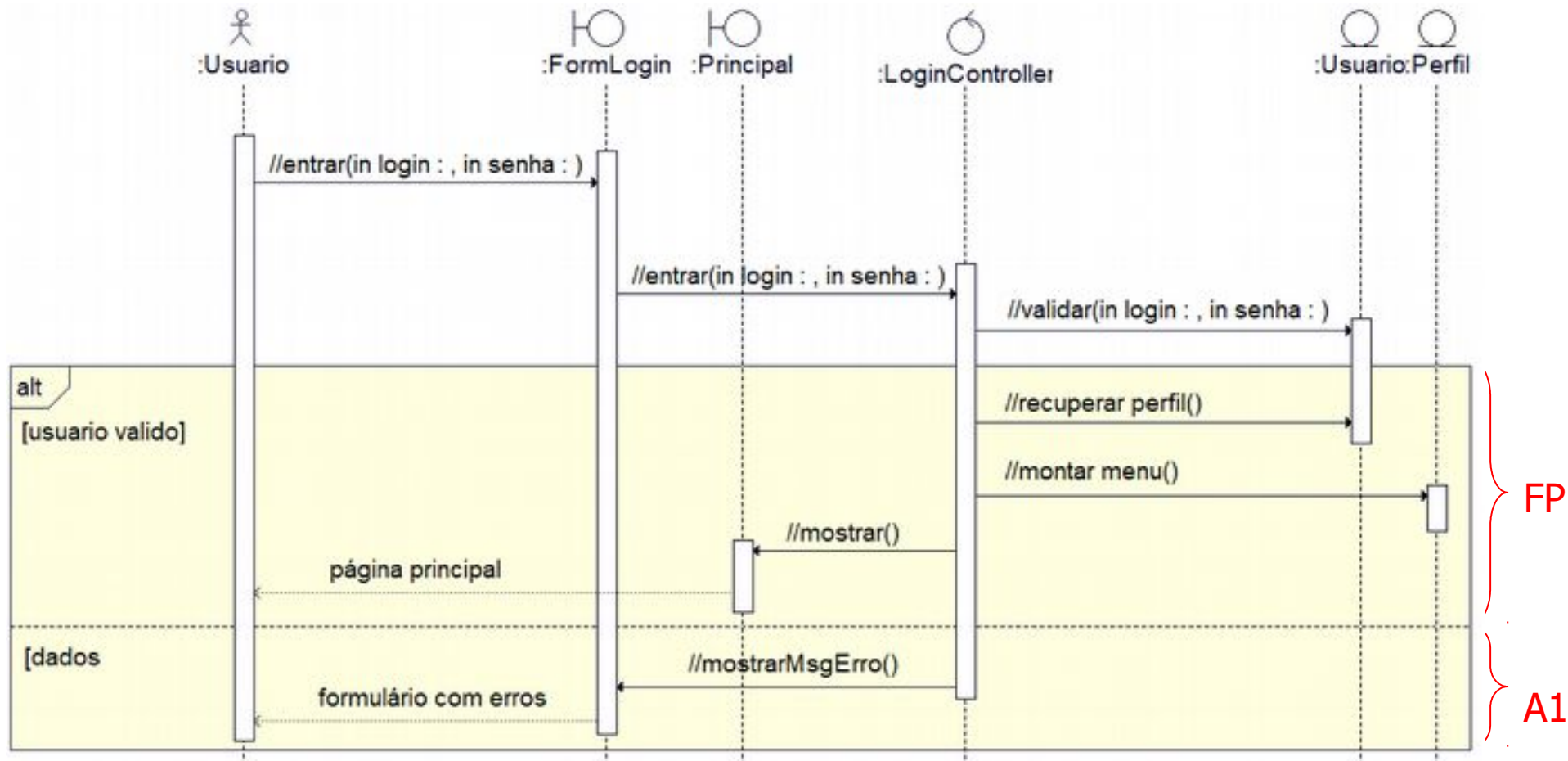
Trigger: usuário acessa a página de login.

1. Sistema apresenta o formulário com login e senha.
2. Usuário preenche o formulário e aciona a opção de "entrar".
3. Sistema valida login e a senha.
4. Sistema apresenta a página principal de acordo com o perfil do usuário [A1].

Fluxo Alternativo A1: usuário inválido

A1.1 Sistema apresenta mensagem de login/senha inválido.

A1.2 Volta para o passo 1.



- Alterando o caso de uso anterior para acrescentar um **Fluxo de Exceção**:

Caso de Uso: Autenticar

Objetivo: permitir ao usuário acessar as funções privadas do sistema.

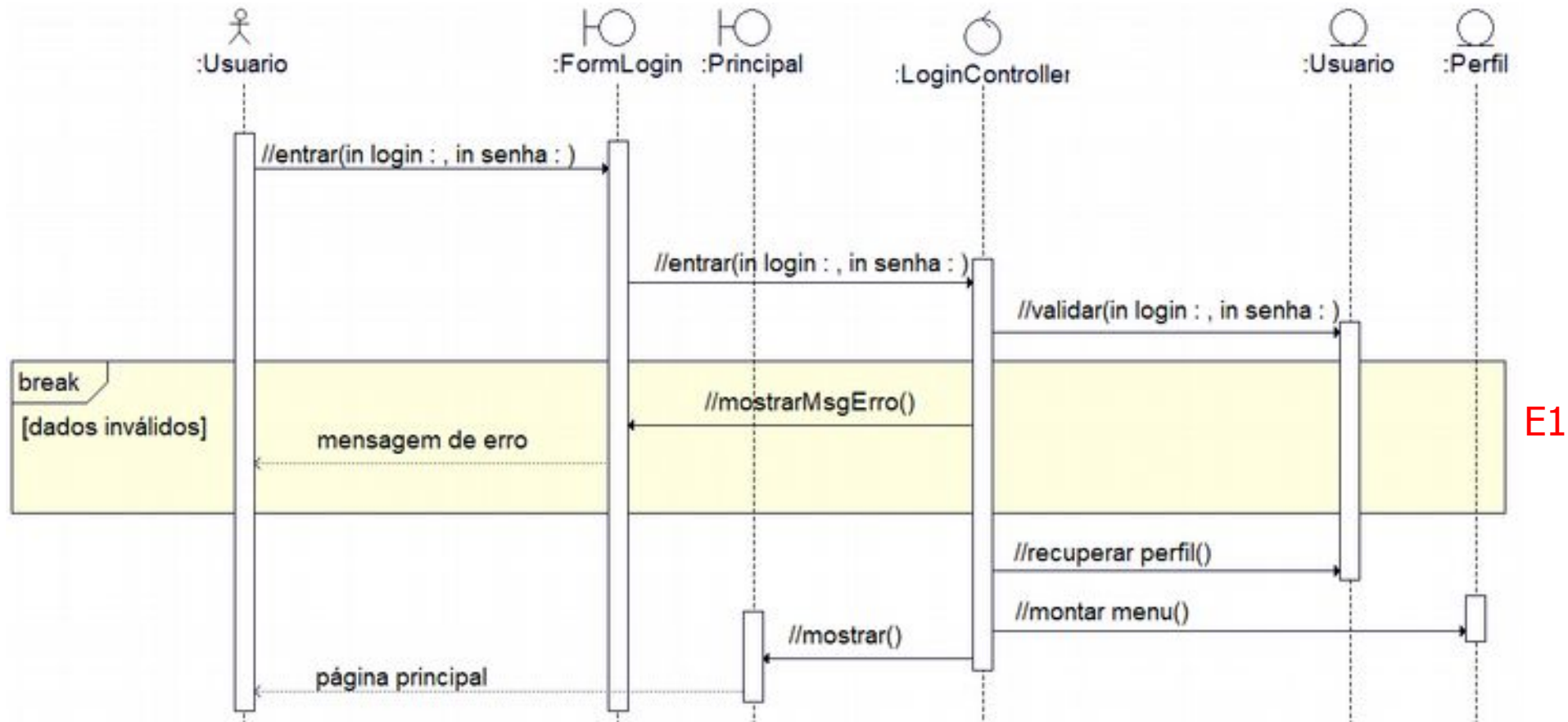
Trigger: usuário acessa a página de login.

1. Sistema apresenta o formulário com login e senha.
2. Usuário preenche o formulário e aciona a opção de "entrar".
3. Sistema valida login e a senha.
4. Sistema apresenta a página principal de acordo com o perfil do usuário [E1].

Fluxo de Exceção E1: usuário inválido

E1.1 Sistema apresenta mensagem de login/senha inválido.

E1.2 Fim do Caso de Uso.



- De forma resumida, envolve os seguintes passos:
 1. Suplementar as descrições dos casos de uso
 2. Para cada Realização de Caso de Uso
 1. Localizar Classes de Análise
 2. Distribuir os comportamentos do Caso de Uso para as Classes de Análise
 3. Para cada Classe de Análise
 1. **Descrever responsabilidades** 
 2. Descrever atributos e associações
 3. Qualificar mecanismos de análise
 4. Avaliar resultado da análise

3.1 Descrever Responsabilidades

- Responsabilidade é a descrição de algo que um objeto pode realizar ou de alguma informação que ele pode prestar.
- No diagrama de sequência que descreve a realização do caso de uso, as mensagens enviadas de um objeto a outro são as responsabilidades de cada classe.
- Note que nos diagramas de sequência anteriores foi usado o símbolo **//** nos métodos das classes. Esse símbolo indica que esse elemento não é um método, e sim uma responsabilidade.
- Dependendo da ferramenta UML utilizada, ao montar o Diagrama de Sequência, as responsabilidades serão automaticamente criadas nas classes.

- De forma resumida, envolve os seguintes passos:
 1. Suplementar as descrições dos casos de uso
 2. Para cada Realização de Caso de Uso
 1. Localizar Classes de Análise
 2. Distribuir os comportamentos do Caso de Uso para as Classes de Análise
 3. Para cada Classe de Análise
 1. Descrever responsabilidades
 2. Descrever atributos e associações
 3. Qualificar mecanismos de análise
 4. Avaliar resultado da análise



3.2 Descrever Atributos e Associações

- Descrever os **atributos** das classes do domínio (entidades):
 - ✓ Defina um nome que seja autoexplicativo e corresponda à informação que será armazenada no atributo.
 - ✓ Defina o **tipo** do atributo.
 - ✓ Concentre-se nas classes de domínio (entidades), pois as classes de fronteira e controle poderão ser alteradas posteriormente durante o Design.

3.2 Descrever Atributos e Associações (continuação)

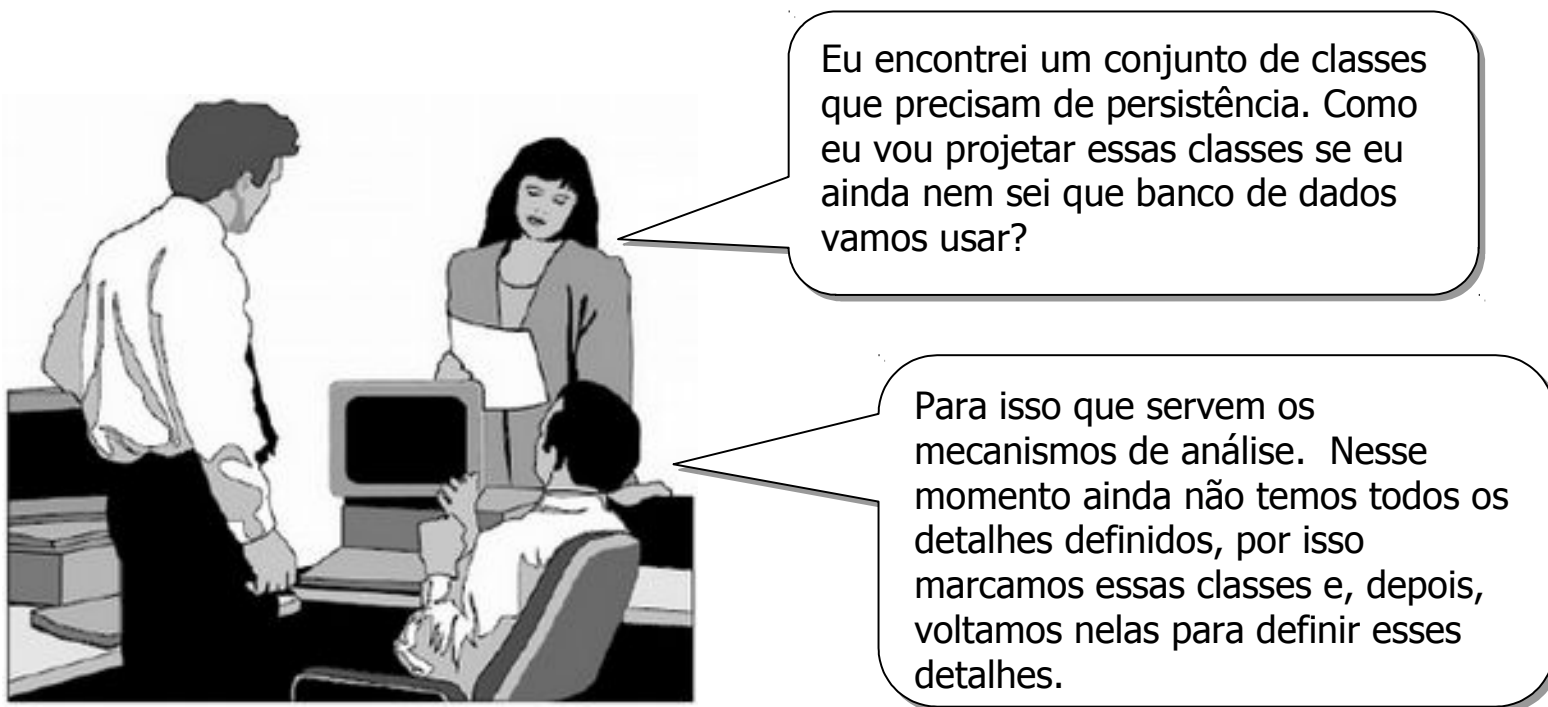
- Descrever as **associações** entre as classes do domínio (entidades):
 - ✓ Define as multiplicidades das associações.
 - ✓ Defina os papéis de cada participante na associação.
 - ✓ Não precisa definir a navegabilidade nesse momento.
 - ✓ Caso uma entidade envie mensagem para outra no Diagrama de Sequência, isso deve ser representado como uma **associação entre elas** no Diagrama de Classes.
 - ✓ Avalie se a associação é uma **agregação**, **composição** ou **herança**. Em caso de dúvida, use uma simplesmente associação.

- De forma resumida, envolve os seguintes passos:
 1. Suplementar as descrições dos casos de uso
 2. Para cada Realização de Caso de Uso
 1. Localizar Classes de Análise
 2. Distribuir os comportamentos do Caso de Uso para as Classes de Análise
 3. Para cada Classe de Análise
 1. Descrever responsabilidades
 2. Descrever atributos e associações
 3. Qualificar mecanismos de análise
 4. Avaliar resultado da análise



3.3 Qualificar Mecanismos de Análise

- Identificar os mecanismos de análise (se houver algum) usados pelas classes de análise.



3.3 Qualificar Mecanismos de Análise (continuação)

- Crie uma lista com as classes de análise e os respectivos mecanismos de análise.
- Nesse ponto, isso é o suficiente. Detalhamentos sobre como projetar e implementar esses mecanismos serão acrescentados posteriormente.
- Exemplo:

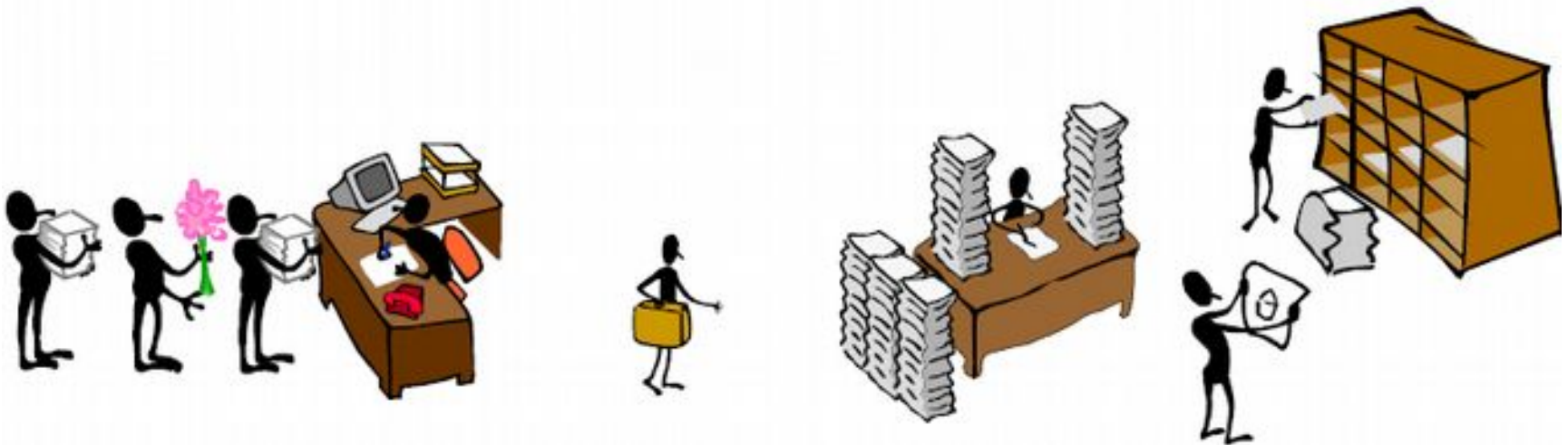
Classe	Mecanismos de Análise
Usuário	Persistência e Criptografia (senha de acesso)
Aluno	Persistência
Conta	Persistência e Interface com sistema externo (interoperabilidade)
FormConsulta	Localização
SaqueController	Log de atividades

Projeto de Software Orientado a Objetos

8ª parte

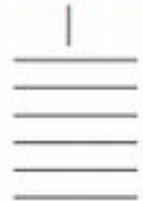
Prof. Jobson Massollar

- Mecanismo utilizado para lidar com o aumento de complexidade.
- Consiste em exibir "o que" pode ser feito sem informar "como" é feito.
- Permite que o grau da abstração do sistema seja alterado, criando estruturas mais abstratas.



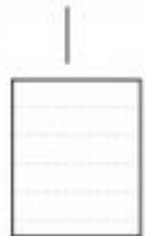
- Existem vários níveis de encapsulamento:
 - ✓ **Encapsulamento nível 0:** completa inexistência de encapsulamento
 - Linhas de código efetuando todas as ações sem nenhum tipo de estruturação.
Exemplos: COBOL, Basic.
 - ✓ **Encapsulamento nível 1:** módulos
 - Procedimentos, rotinas ou funções permitindo a criação de ações complexas.
Exemplos: Fortran, Pascal, C.

linhas primárias
de código



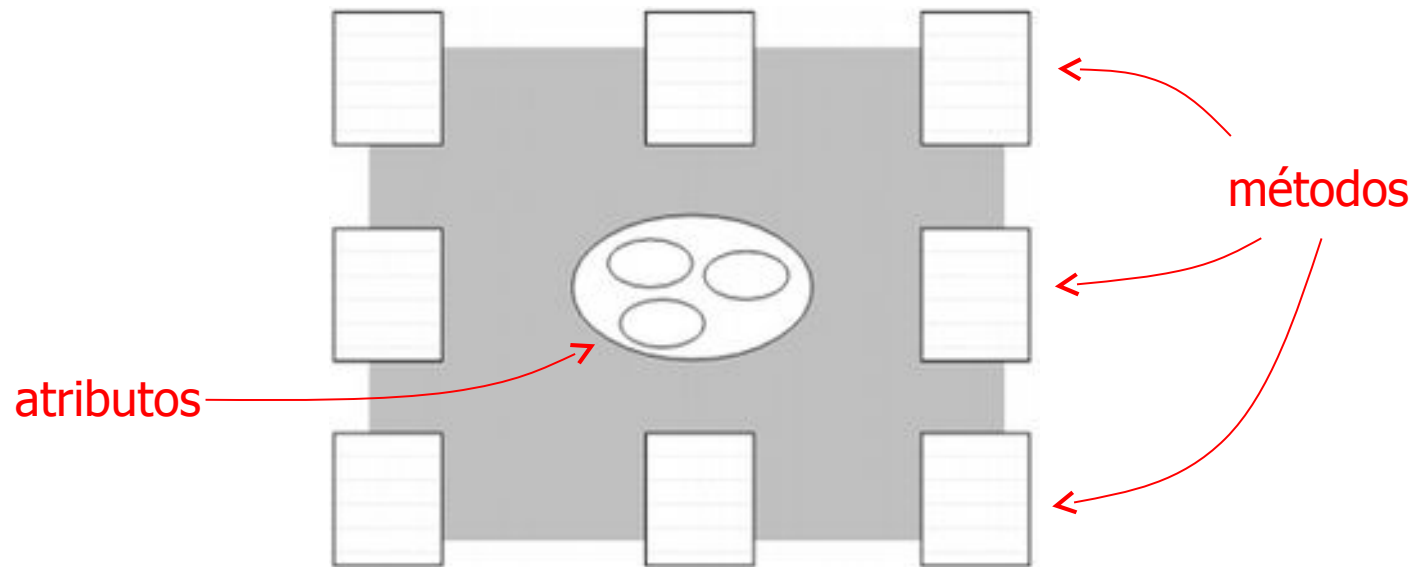
Nível 0

módulo procedural

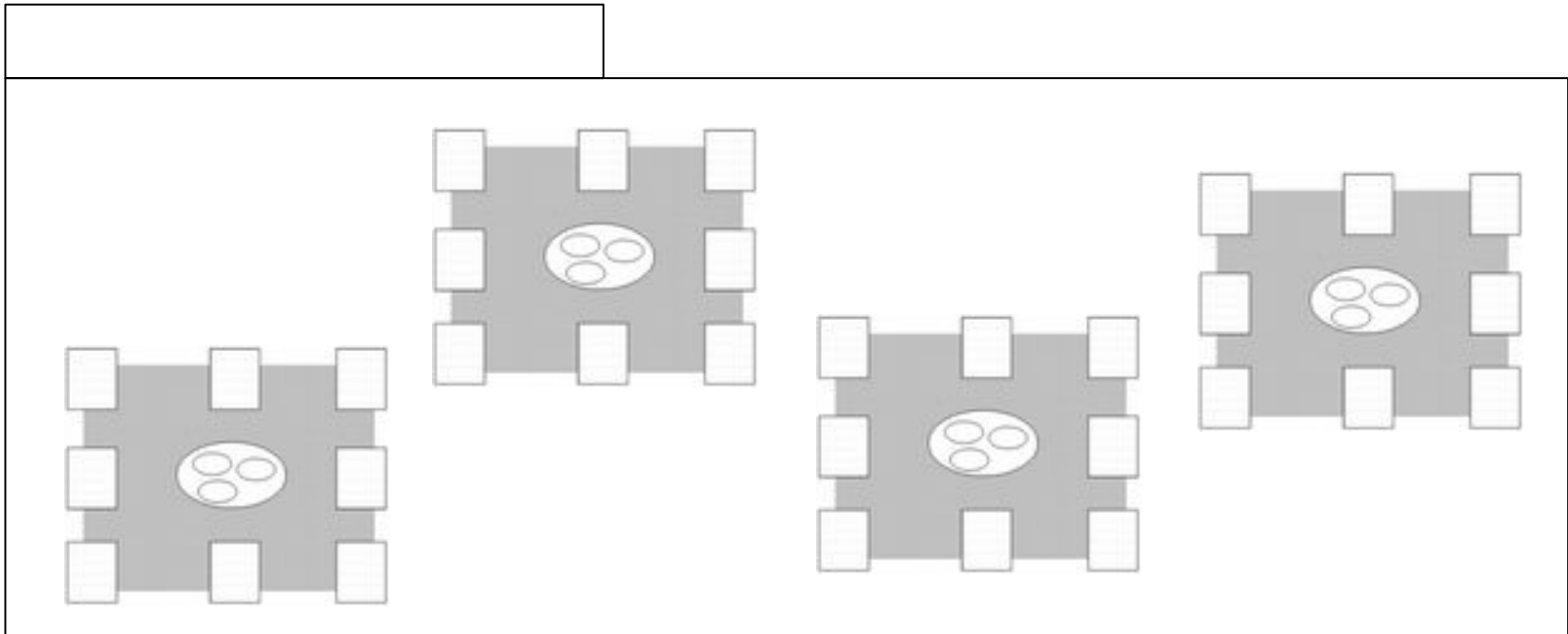


Nível 1

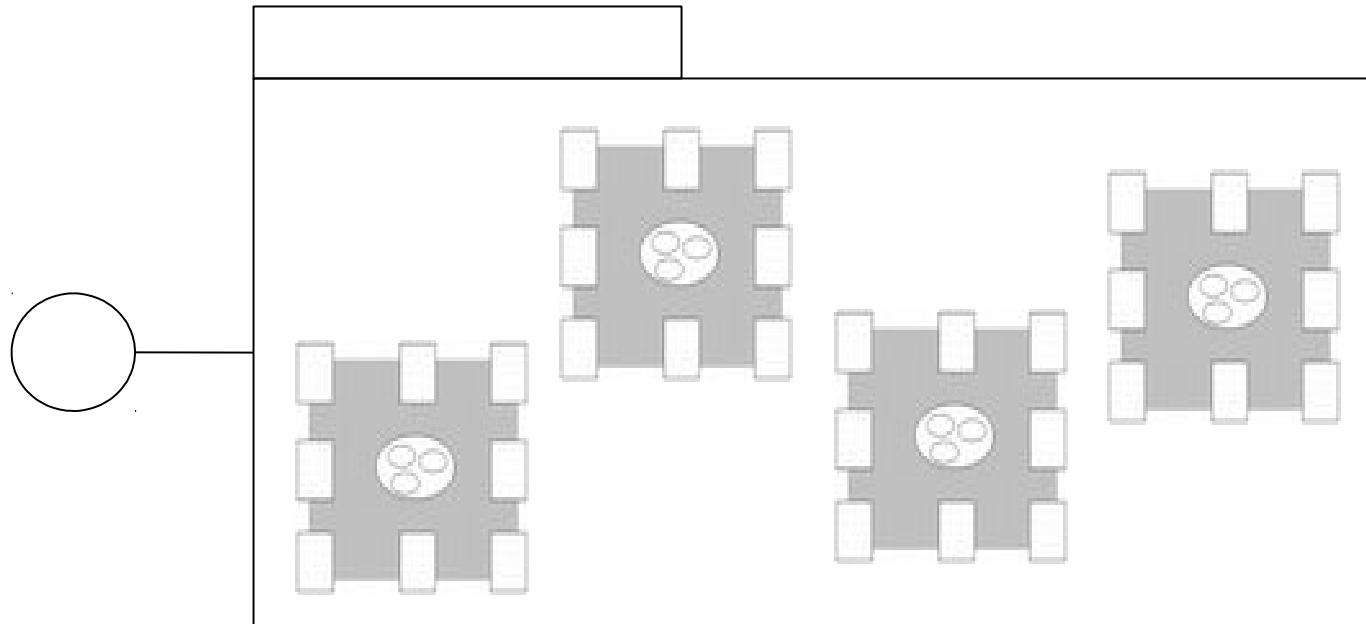
- Existem vários níveis de encapsulamento:
 - ✓ Encapsulamento nível 2: classes
 - Métodos isolam o acesso aos atributos e definem comportamentos complexos dos objetos. Exemplos: C++, Java, C#.



- Existem vários níveis de encapsulamento:
 - ✓ Encapsulamento nível 3: pacotes de classes
 - Grupamento de classes que colaboram para fornecer um conjunto de funcionalidades.

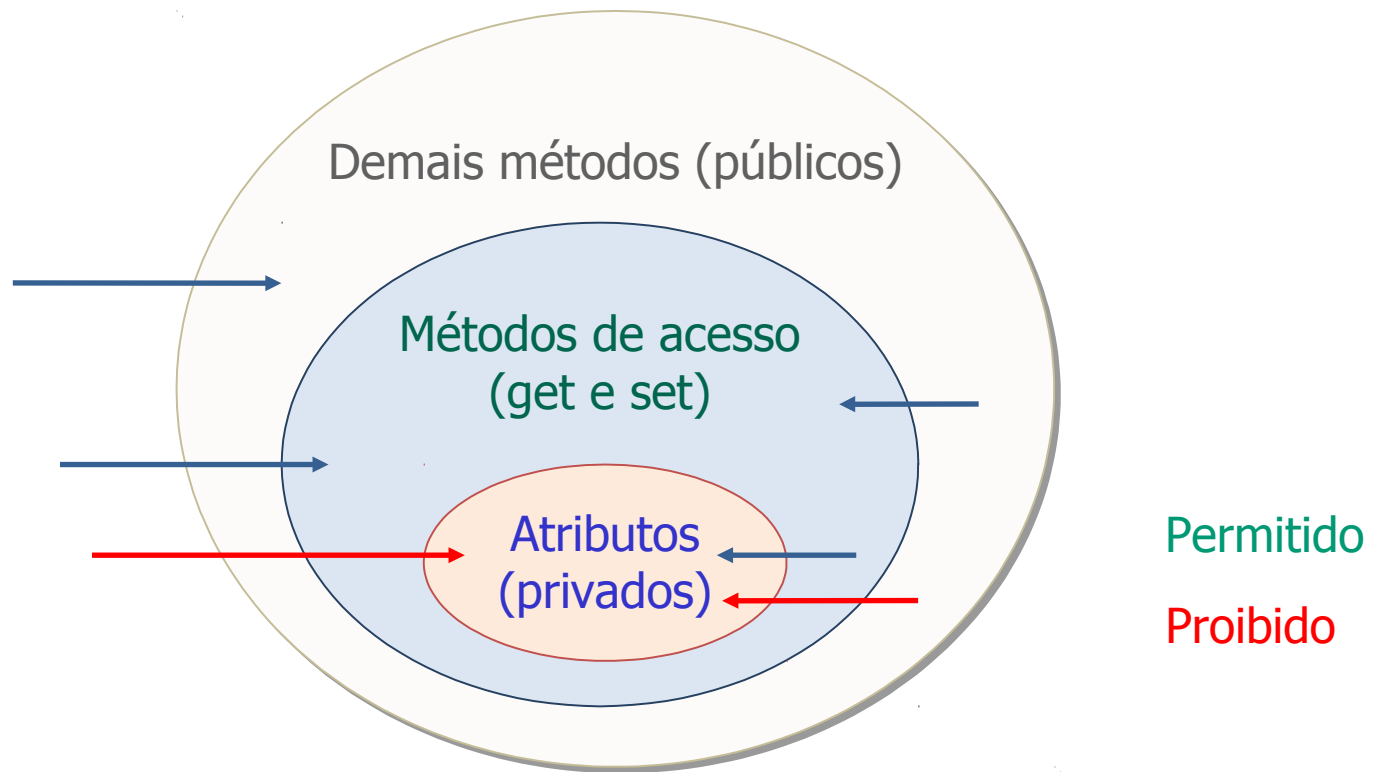


- Existem vários níveis de encapsulamento:
 - ✓ Encapsulamento nível 4: componentes
 - Conjunto de classes acessadas através de uma interface **bem definida** e com um propósito **bem estabelecido**.



- O Projeto OO tem foco principal em estruturas de nível 2 de encapsulamento, ou seja, as classes.
- A técnica de **Anéis de Operações** ou **Anéis de Encapsulamento** ajuda a manter um bom encapsulamento interno da classe:
 - ✓ O uso dessa técnica não afeta o acesso externo (que continua sendo regido por modificadores de visibilidade).
 - ✓ Nessa técnica são criados **três anéis fictícios** na classe.
 - ✓ Os métodos de anéis externos acessam **sempre** métodos (ou atributos) de anéis internos **consecutivos**.

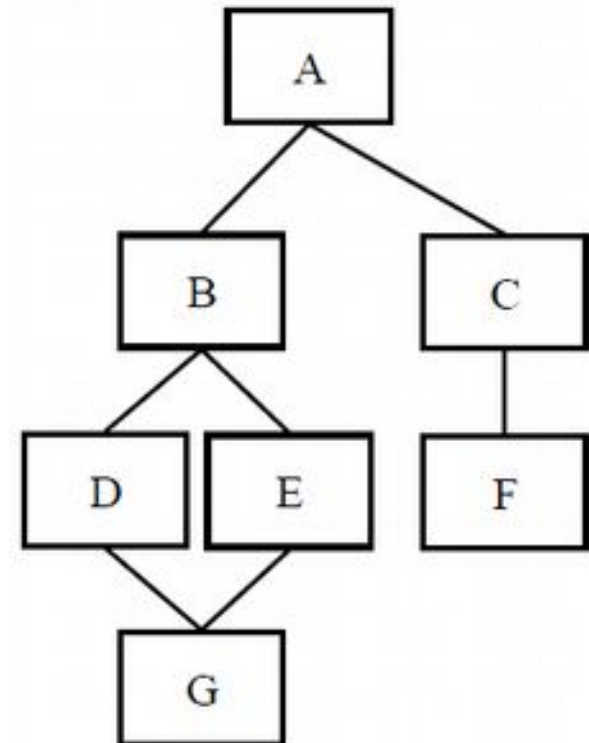
- Note que nem mesmo os métodos ordinários da classe (públicos ou privados) acessam diretamente os atributos, **somente** os métodos de acesso (get/set).



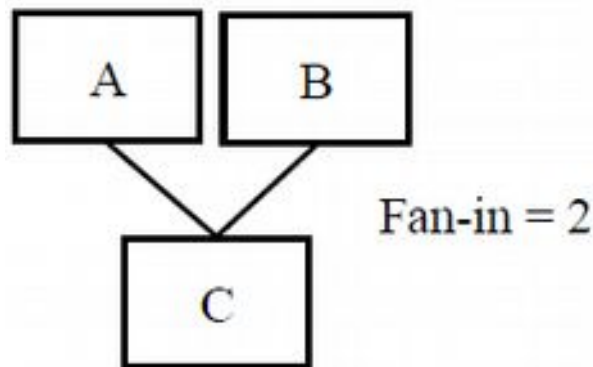
- Com o uso dessa técnica podem ser definidos **atributos derivados**, ou seja, atributos calculados a partir dos atributos reais.
 - ✓ Exemplo 1: método **double getVolume()** na classe **Cubo** retorna **lado³**.
 - ✓ Exemplo 2: método **void setNome(String nome)** poderia quebrar a string do parâmetro **nome** e armazenar os valores nos atributos **primeiroNome**, **iniciaisMeio**, **ultimoNome**.

- O Projeto Estruturado definiu alguns conceitos que são importantes até hoje quando tratamos de projeto de sistemas.
- Esses mesmos conceitos ainda são válidos quando lidamos com o projeto OO.
- **Árvore de Dependência**: estrutura que descreve a dependência entre os módulos do sistema.

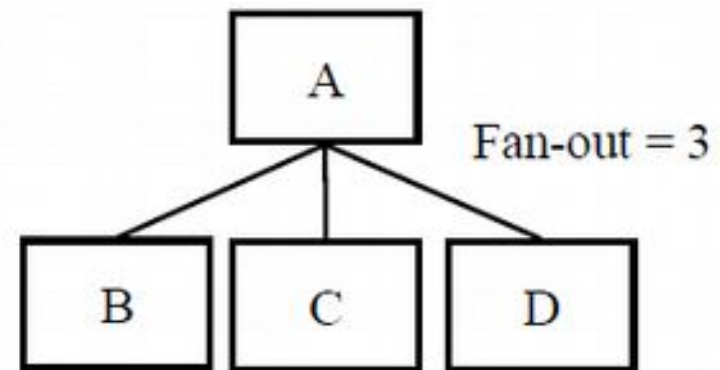
Observação: o conceito de módulo é usado em Projeto Estruturado. Como estamos trabalhando com Projeto OO, podemos substituir módulo por classe.



- **Fan-in**: indica quantos módulos acessam determinado módulo.
- **Fan-out**: indica quantos módulos são acessados por determinado módulo.



A classe C tem seus métodos acessados pelas classes A e B.
Fan-in de C = 2



A classe A acessa os métodos das classes B, C e D.
Fan-out de A = 3

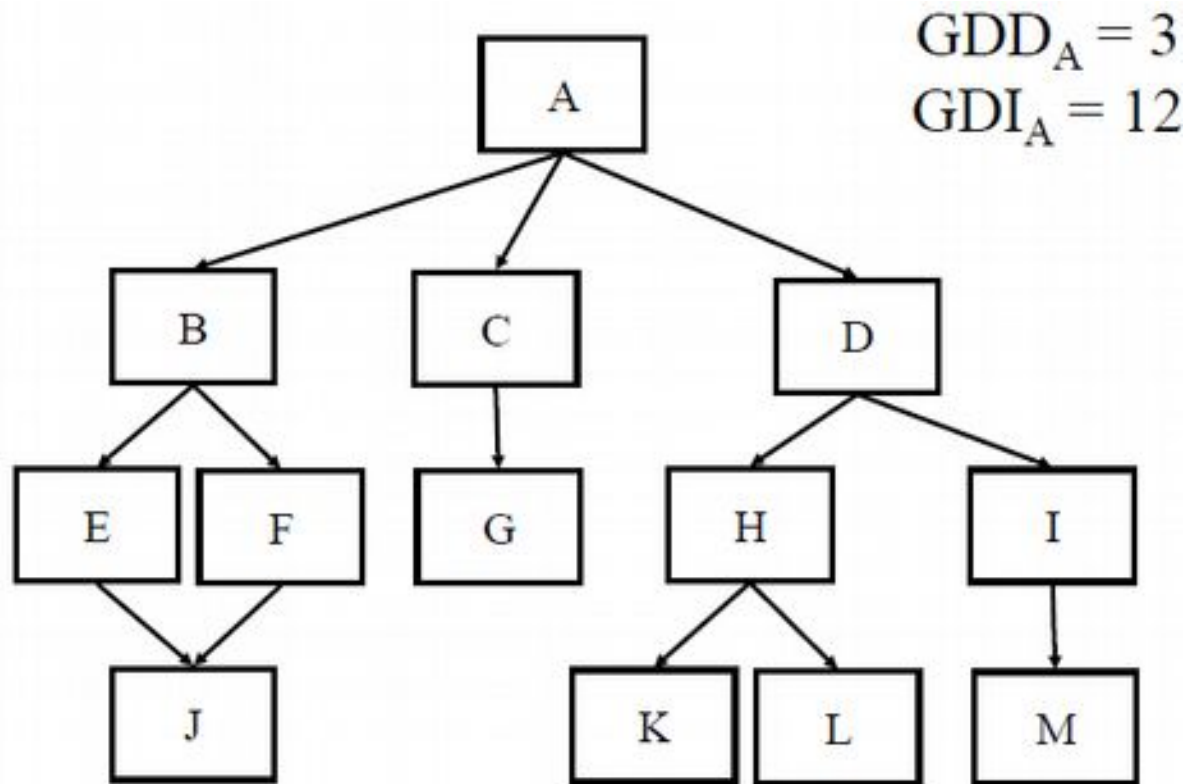
- Do ponto de vista da qualidade do projeto, devemos tratar o **fan-in** e o **fan-out** da seguinte forma:
 1. Fan-out deve ser sempre **minimizado ao máximo**, pois:
 - ✓ **Alto fan-out** indica que o módulo (ou classe) depende de muitos outros, o que o torna "frágil" ou suscetível a falhas que ocorrem nos demais.
 - ✓ **Alto fan-out** indica que o módulo (ou classe) pode ser complexo, devido à complexidade relacionada à coordenação dos demais módulos.

- Do ponto de vista da qualidade do projeto, devemos tratar o **fan-in** e o **fan-out** da seguinte forma:
 2. **Alto fan-in** indica reuso do módulo (ou classe) pelos demais, o que pode indicar um bom projeto.
 3. Módulos (ou classes) com **alto fan-in** devem:
 - ✓ Ser construídos e mantidos pelos profissionais **mais experientes** da equipe.
 - ✓ Ter **prioridade** nos testes unitários, pois muitos outros dependem dele.

- Grau de dependência é uma métrica semelhante ao *fan-out* do Projeto Estruturado.
- O Grau de dependência pode ser :
 - ✓ **Direto**: indica quantas classes são referenciadas **diretamente** por uma determinada classe.
 - ✓ **Indireto**: indica quantas classes são referenciadas **direta ou indiretamente** por uma determinada classe.

- Uma classe **A** referencia **diretamente** uma classe **B** se:
 - ✓ A é subclasse direta de B
 - ✓ A tem atributo do tipo B
 - ✓ A tem métodos com parâmetros do tipo B
 - ✓ A tem métodos com variáveis do tipo B
 - ✓ A chama métodos que retornam objetos do tipo B

➤ Exemplo:



- O Projeto Estruturado definiu outros conceitos que são importantes até hoje quando tratamos de design de sistemas:
 - ✓ **Acoplamento**
 - Mede as **interconexões** entre os módulos de um sistema, ou seja, mede o quanto um módulo está **conectado** a outros ou **depende** de outros módulos.
 - ✓ **Coesão**
 - Mede a **afinidade** dos procedimentos que compõem um módulo do sistema, ou seja, mede o quão relacionadas ou focadas estão as responsabilidades de um módulo.
- Acoplamento e coesão variam em uma escala relativa: **muito baixo, baixo, alto, muito alto**, ou ainda, **muito fraco, fraco, forte, muito forte**.

- O principal objetivo do projeto estruturado é criar módulos com **baixo acoplamento** e **alta coesão**.
- Esse objetivo se mantém no paradigma OO.
- Assim, um dos princípios que rege o projeto OO se baseia em definir classes com:
 - ✓ **Baixo acoplamento**: classes independentes de forma a minimizar o impacto de alterações de uma classe em outras.
 - ✓ **Alta coesão**: classes com características e responsabilidades relacionadas, que contribuem para a abstração implementada pela classe.

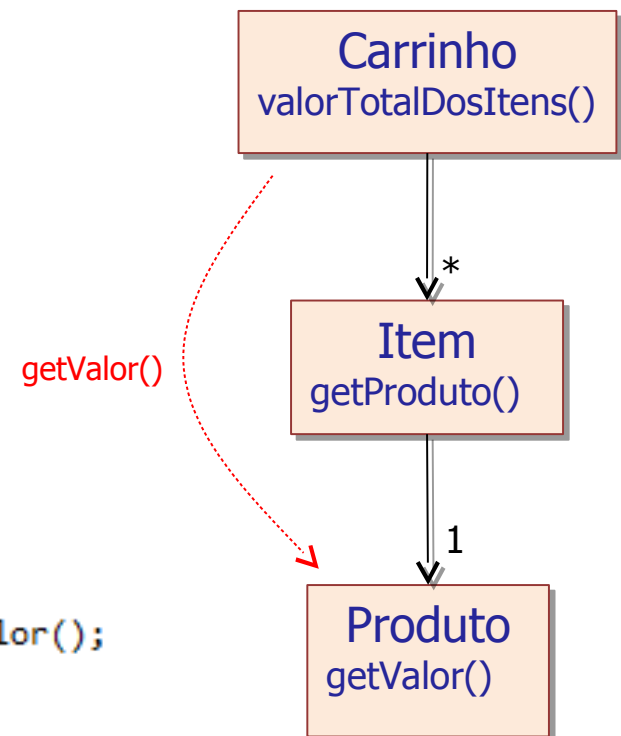
- O que fazer para diminuir:
 - ✓ O grau de dependência de uma classe C (principalmente o grau de dependência indireto)?
 - ✓ O nível de acoplamento de uma classe C?

- Solução:
 - ✓ Adotar abstrações e interfaces (será visto posteriormente);
 - ✓ Adotar a Lei de Demeter.

- A **Lei de Demeter** serve para limitar o grau de dependência ou acoplamento e segue o princípio do "*nunca fale com estranhos*".
- De acordo com essa lei, cada classe deve conhecer apenas as demais classes **intimamente** (diretamente) relacionadas a ela.
- Assim, a **Lei de Demeter** define que um **método M** da **classe C** só deve chamar métodos:
 - ✓ da própria classe C;
 - ✓ de objetos definidos como atributos de C;
 - ✓ de objetos recebidos como parâmetros de M;
 - ✓ de objetos criados por métodos da classe C;
 - ✓ de objetos globais (estáticos ou *singletons*).

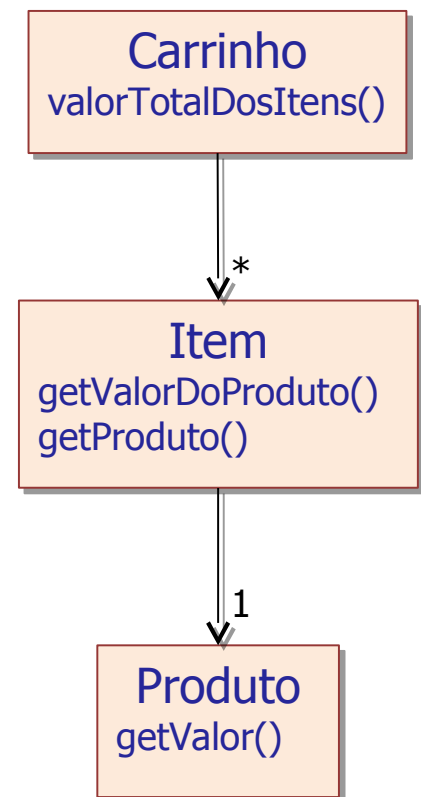
- Nesse exemplo, a lei de Demeter é **violada**, pois o **CarrinhoCompras** envia uma mensagem para **Produto**.

```
1 package rs.com.teste.model;
2
3 import java.util.ArrayList;
4 import java.util.List;
5
6 public class CarrinhoCompras {
7
8     private List<Itens> itens;
9
10    public CarrinhoCompras() {
11        this.itens = new ArrayList<Itens>();
12    }
13
14    public Double valorTotalDosItens(){
15        Double valorTotal = 0.0;
16        for (Itens item : itens) {
17            valorTotal += item.getProduto().getValor();
18        }
19        return valorTotal;
20    }
21
22 }
```



- Para obedecer a lei de Demeter o **CarrinhoCompras** deve enviar uma mensagem para **Itens** (que é um atributo da classe).

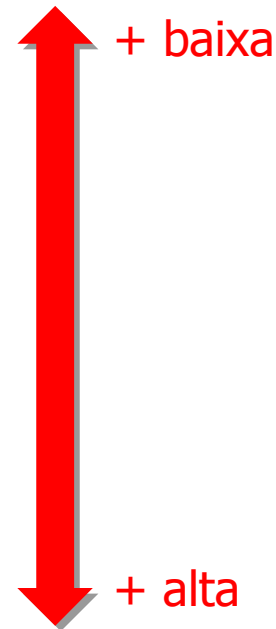
```
1 package rs.com.teste.model;
2
3 import java.util.ArrayList;
4 import java.util.List;
5
6 public class CarrinhoCompras {
7
8     private List<Itens> itens;
9
10    public CarrinhoCompras() {
11        this.itens = new ArrayList<Itens>();
12    }
13
14    public Double valorTotalDosItens(){
15        Double valorTotal = 0.0;
16        for (Itens item : itens) {
17            valorTotal += item.getValorDoProduto();
18        }
19        return valorTotal;
20    }
21 }
22
23
```



- Classe com **Alta Coesão**:
 - ✓ Possui características e responsabilidades relacionadas, que contribuem para a abstração implementada pela classe.

- Classe com **Baixa Coesão**:
 - ✓ Possui características e responsabilidades dissociadas, que levam ao não entendimento da abstração implementada pela classe.

- A coesão pode ser classificada em:
 - ✓ Coesão de instância mista
 - ✓ Coesão de domínio misto
 - ✓ Coesão de papel misto
 - ✓ Coesão alternada
 - ✓ Coesão múltipla
 - ✓ Coesão funcional



- Ocorre quando alguns atributos ou métodos da classe são válidas apenas para **alguns** objetos da classe.
- Exemplo:

```
public class Funcionario {  
    private int matricula;  
    private String nome;  
    private float valorHoraAula;  
    private int qtdHorasAula;  
  
    public int calculaSalarioBruto() {  
        return valorHoraAula * qtdHorasAula;  
    }  
}
```

Esses atributos e esse método só fazem sentido se o funcionário for um Professor !

- A correção da coesão de instância mista normalmente envolve a criação de uma **herança**.

```
public class Funcionario {  
    protected int matricula;  
    protected String nome;  
}  
  
public class Professor extends Funcionario {  
    private float valorHoraAula;  
    private int qtdHorasAula;  
  
    public int calculaSalarioBruto() {  
        return valorHoraAula * qtdHorasAula;  
    }  
}
```

- Ocorre quando alguns atributos ou métodos da classe não estão associados ao **domínio da classe**.
- Quando isso ocorre a classe tende a perder o foco, pois são introduzidos **elementos estranhos** ao domínio da classe.
- Exemplo:

```
public class CarrinhoCompras {  
  
    public float calcularTotalCompras() {  
    }  
    public float calcularFrete() {  
    }  
    public float calcularValorPedido() {  
    }  
}
```

Essa classe está
sobrecarregada com
métodos que não estão
associados ao seu objetivo.

- A correção da coesão de domínio misto normalmente envolve a criação de **novas classes** e a **redistribuição de responsabilidades**, tirando a sobrecarga da classe originalmente projetada.
- Exemplo:

```
public class CarrinhoCompras {  
    public float calcularTotalCompras() {  
    }  
}
```

```
public class Pedido {  
    public float calcularFrete() {  
    }  
    public float calcularValor() {  
    }  
}
```

- Ocorre quando alguns atributos ou métodos da classe só fazem sentido quando a classe é usada em um **contexto específico**.
- São os problemas menos críticos relacionados à coesão.
- Normalmente esses problemas estão relacionados com as dificuldades em reutilizar essa classe em contextos distintos.
- Exemplo:

```
public class Funcionario {  
  
    private int matricula;  
    private String nome;  
    private String tipoSanguineo;  
  
}
```

Esse atributo faz sentido somente se a classe Funcionário for usada no contexto de um sistema de saúde ou doação de sangue.
No contexto de um sistema de RH ele provavelmente não fará sentido.

- A correção da coesão de papel misto normalmente envolve a **refatoração** da classe original para que sejam criadas outras classes adequadas a cada contexto de uso.
- Nesse caso, tanto o mecanismo de **herança** quanto de **delegação** pode ser empregado.

- Ocorre quando a lógica de funcionamento de um método é determinada por um parâmetro.
- Internamente o método usa um **if** ou **switch** para decidir qual lógica deve ser executada.
- Exemplo:

```
public class Mensagem {  
    public static final int SMS = 1;  
    public static final int EMAIL = 2;  
  
    public int envia(int meio) {  
        if (meio == SMS) {  
            ...  
        } else if (meio == EMAIL) {  
            ...  
        } else {  
            ...  
        }  
    }  
}
```

O método envia() recebe por parâmetro um indicador de qual meio deve ser usado para enviar a mensagem.

- A correção da coesão alternada envolve a **divisão do método original** em vários métodos (um para cada comportamento distinto).
- Exemplo:

```
public class Mensagem {  
  
    public int enviaSMS() {  
    }  
  
    public int enviaEMAIL() {  
    }  
}
```

Note que a decisão de qual tipo de mensagem deve ser enviada foi transferida para outro lugar, como um *controller* por exemplo.

- Ocorre quando a lógica de funcionamento de um método contém mais de um comportamento que são **sempre** executados.
- Exemplo:

```
public class Email {  
  
    public int anexaEnvia(String[] arquivos) {  
  
        // Anexa os arquivos passados como parâmetro ao email  
  
        // Envia o email  
    }  
}
```

O método `anexaEnvia()` executa dois comportamentos distintos em sequência.

- A correção da coesão múltipla também envolve a **divisão do método original** em vários métodos (um para cada comportamento distinto).
- Exemplo:

```
public class Email {  
  
    public int anexa(String[] arquivos) {  
    }  
  
    public int envia() {  
    }  
}
```

- É o nível de coesão ideal.
 - ✓ A classe representa um **único conceito coeso**.
 - ✓ Os atributos e métodos estão **conceitualmente relacionados** com a classe.
 - ✓ Os métodos da classe tem um **único e claro objetivo**.
 - ✓ Tanto os atributos quanto os métodos podem ser definidos usando um termo **único e coerente**.

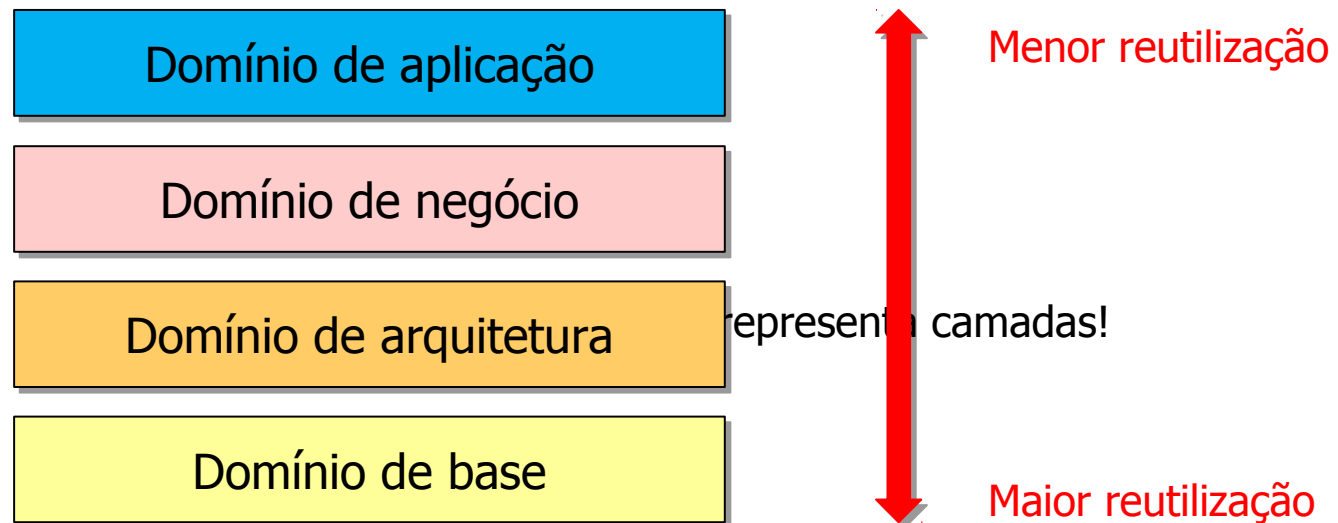
- Um Domínio pode ser visto como uma estrutura de categorização de classes.
- Em sistemas OO, as classes podem pertencer a um dos seguintes domínios:
 - ✓ Domínio de aplicação
 - ✓ Domínio de negócio
 - ✓ Domínio de arquitetura
 - ✓ Domínio de base
- Regra geral: para ser considerada coesa, uma classe deve pertencer a um único domínio!

- O **domínio de base** descreve as classes fundamentais, estruturais e semânticas.
- Usualmente as classes do domínio de base já fazem parte das **bibliotecas** da linguagem de programação.
 - ✓ **Classes fundamentais:** representam tipos primitivos das linguagens OO (ex.: Integer, Float e Boolean)
 - ✓ **Classes estruturais:** implementam estruturas de dados consagradas (ex.: HashMap, ArrayList e HashSet)
 - ✓ **Classes semânticas:** implementam elementos semânticos corriqueiros (ex.: Date, Time e Color)

- O **domínio de arquitetura** fornece abstrações para apoiar a arquitetura de hardware e software utilizada.
- As linguagens de programação atuais também incluem classes do domínio de arquitetura.
 - ✓ **Classes de comunicação:** implementam mecanismos que possibilitam a comunicação com outros sistemas (ex.: Sockets, HTTP e RMI)
 - ✓ **Classes de manipulação de banco de dados:** criam abstrações para acesso aos SGBDs (ex.: JDBC e JDO)
 - ✓ **Classes de interface com o usuário:** permitem a criação de sistema interativos (ex.: JavaFX e Swing)

- O **domínio de negócio** descreve classes inerentes a uma determinada área do conhecimento (ex.: Cliente, Produto, Venda e Caixa no domínio de um supermercado).
- O **domínio de aplicação** descreve classes "cola", que servem para fazer as classes dos demais domínios funcionarem em um sistema.

- Cada domínio faz uso das classes dos domínios inferiores.
- Desta forma, o domínio de base é o mais reutilizável, enquanto o domínio de aplicação torna-se praticamente não reutilizável.



- Um indicativo de **baixa coesão** é quando uma classe pertence a **mais de um domínio**:

Classe Faturai está no domínio de negócio e no domínio da arquitetura.

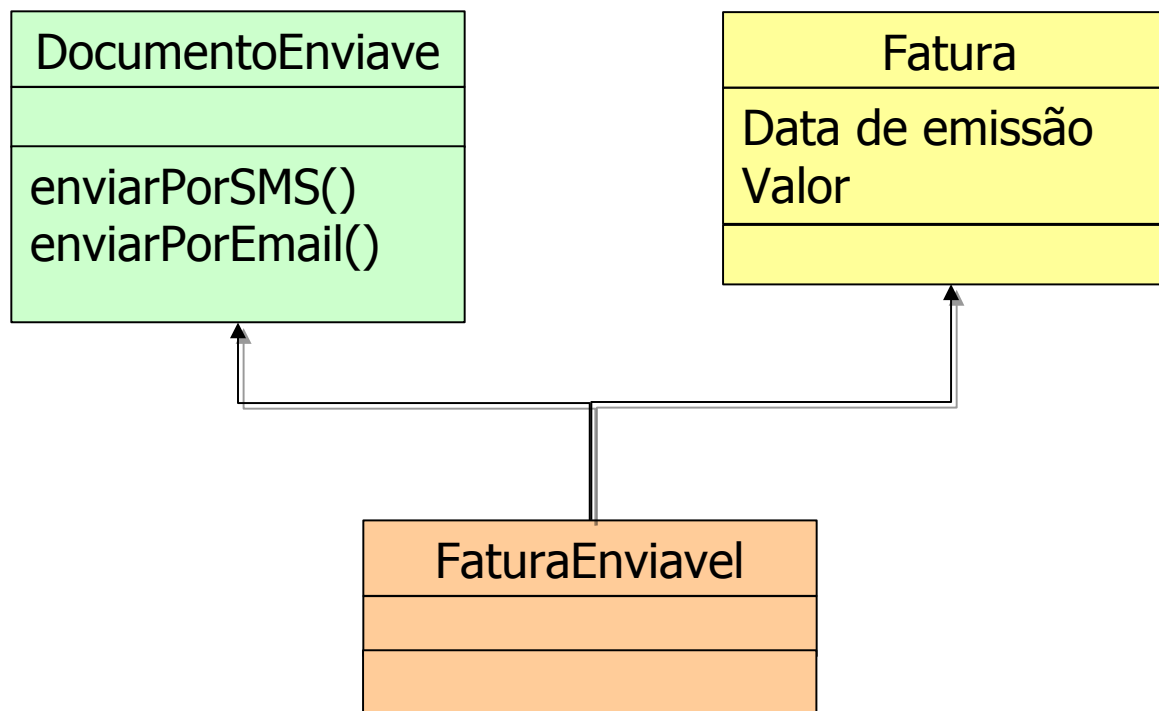
Fatura
Data de emissão Valor
enviarPorSMS() enviarPorEmail()

- Classes do domínio de negócio **não devem** ser dependentes de tecnologia.
- Caso isso ocorra, tanto a classe do domínio quanto a tecnologia implementada nela serão dificilmente reutilizáveis.
- Para contornar esse problema podem ser utilizadas **classes mistas**, pertencentes ao domínio de aplicação.
- Classes mistas são úteis para misturar conceitos de domínios diferentes, sem afetar as classes originais.

- Exemplo: tecnologia de transmissão de dados (domínio da arquitetura) foi inserida na classe **Fatura** (domínio de negócio).

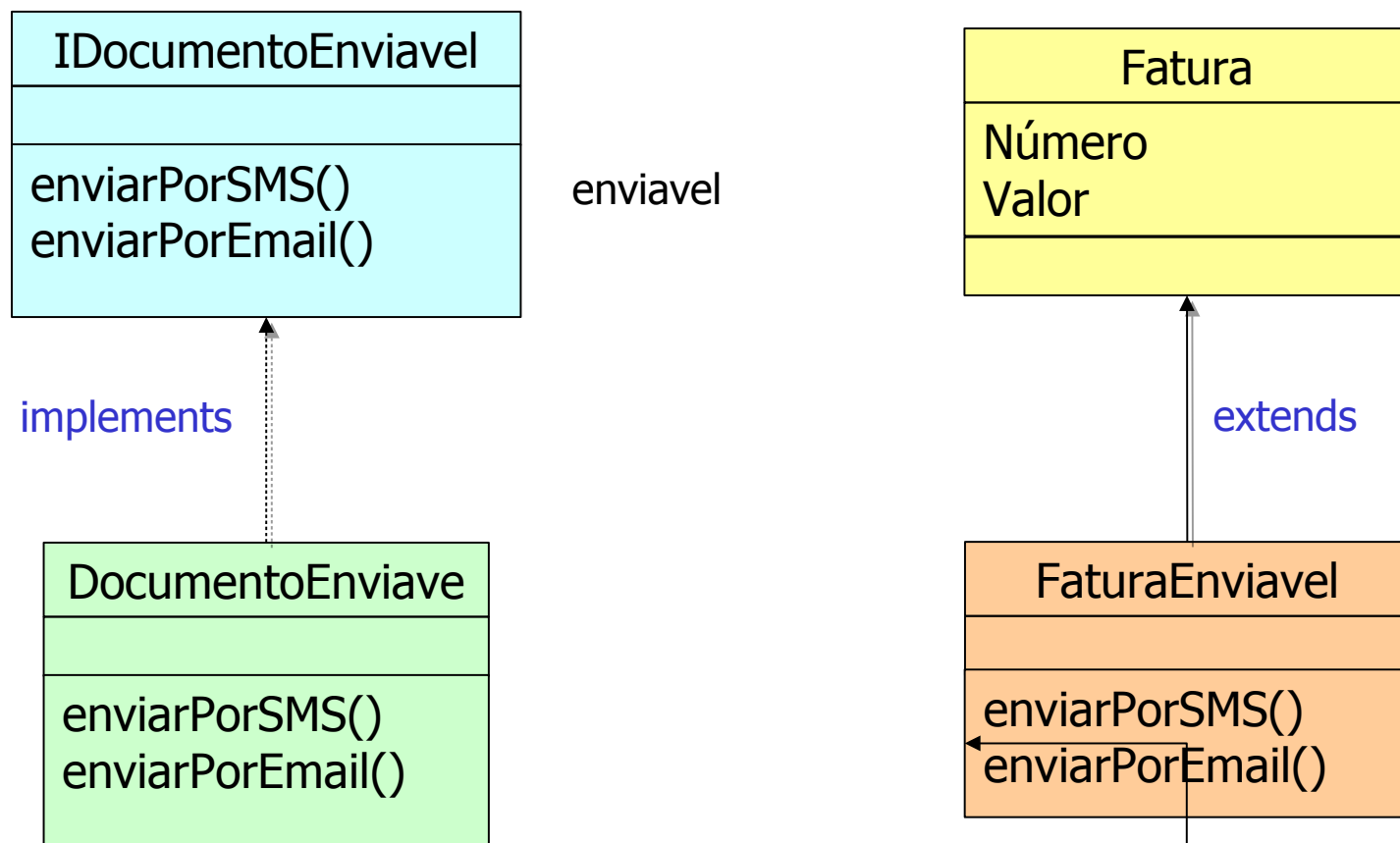
Fatura
Data de emissão Valor
enviarPorSMS() enviarPorEmail()

- Exemplo: criação da classe `DocumentoEnviavel` (domínio de arquitetura) e da classe mista `FaturaEnviavel` (domínio da aplicação)



- Dúvida: "Eu uso Java! Como poderei implementar essa solução já que a minha linguagem de programação não aceita herança múltipla?"
- É possível **simular** herança múltipla utilizando os conceitos de **interface** e **delegação**!
- A ideia consiste na classe mista repassar (delegar) todas as requisições que chegarem a ela através da interface utilizada.

- Solução com **interface** e **delegação**:



Os métodos de **FaturaEnviavel** chamam os métodos de **DocumentoEnviavel**, ou seja, delegam a execução das tarefas para esses métodos.

➤ Solução com **interface** e **delegação**:

```
public interface IDocumentoEnviavel<T> {  
    public void enviarPorSMS(T doc);  
    public void enviarPorEmail(T doc);  
}
```

```
public class DocumentoEnviavel  
    implements IDocumentoEnviavel<FaturaEnviavel> {  
  
    public void enviarPorSMS(FaturaEnviavel f) {  
        System.out.printf("SMS: %d R$ %.2f", f.getNumero(), f.getValor());  
    }  
    public void enviarPorEmail(FaturaEnviavel f) {  
        System.out.printf("Email: %d R$ %.2f", f.getNumero(),  
f.getValor());  
    }  
}
```

➤ Solução com **interface** e **delegação**:

```
public class FaturaEnviavel extends Fatura {  
  
    private IDocumentoEnviavel<FaturaEnviavel> enviavel;  
  
    public FaturaEnviavel(IDocumentoEnviavel <FaturaEnviavel> enviavel) {  
        this.enviavel = enviavel;  
    }  
  
    public void enviarPorSMS() {  
        // delega  
        enviavel.enviarPorSMS(this);  
    }  
  
    public void enviarPorEmail() {  
        // delega  
        enviavel.enviarPorEmail(this)  
    }  
}
```

- Solução com **interface** e **delegação**:

```
public class TesteDelegacao {  
  
    public static void main(String [] args) {  
        DocumentoEnviavel doc = new DocumentoEnviavel();  
        FaturaEnviavel fatura;  
  
        fatura = new FaturaEnviavel(doc);  
  
        fatura.enviarPorSMS();  
        fatura.enviarPorEmail();  
    }  
}
```

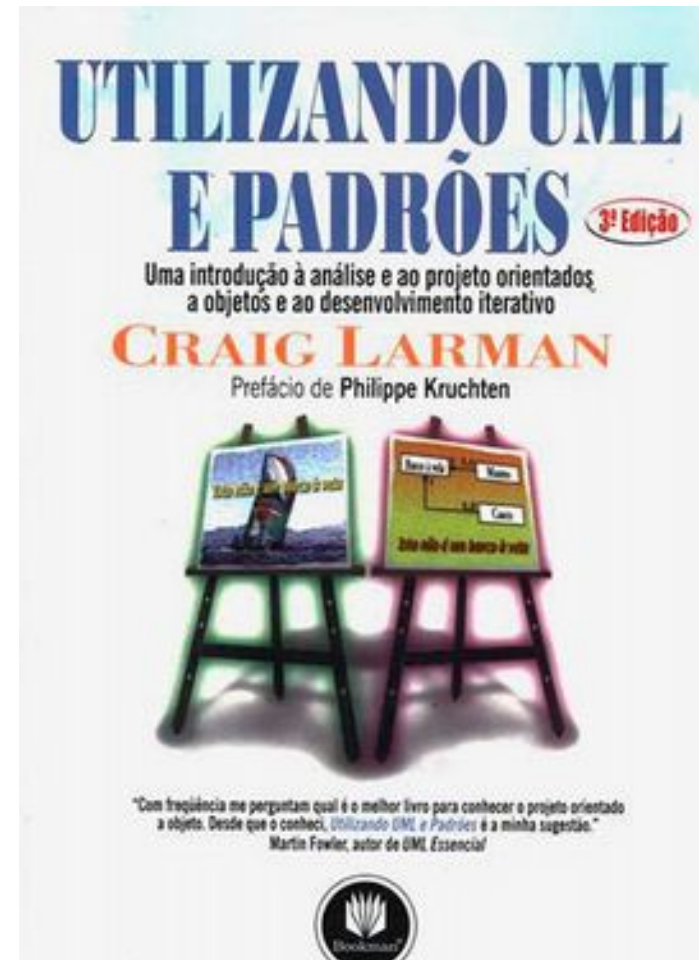

Projeto de Software Orientado a Objetos

9ª parte

Prof. Jobson Massollar

➤ GRASP: **G**eneral **R**esponsibility and **A**ssignment **S**oftware **P**atterns

➤ Eles foram definidos por Craig Larman no livro *Utilizando UML e Padrões* (disponível na biblioteca virtual)



- GRASP: **G**eneral **R**esponsibility and **A**ssignment **S**oftware **P**atterns
- Descrevem os princípios fundamentais da atribuição de responsabilidades a objetos, expressas na forma de padrões.
- Ajudam a compreender melhor a utilização do paradigma OO em projetos mais complexos.
- O que são responsabilidades?
 - "São contratos ou obrigações de um tipo ou classe"
 - [Booch e Rumbaugh]
- Responsabilidade não é um método, é uma abstração!

- Existem dois tipos de responsabilidades que os objetos podem ter:
 1. **Conhecer** ou **Saber** (*knowing*):
 - ✓ Seus atributos privados e encapsulados;
 - ✓ Os objetos relacionados a ele;
 - ✓ Coisas que ele pode calcular ou derivar a partir de seus atributos.
 2. **Fazer** (*doing*):
 - ✓ Fazer alguma coisa, como criar um objeto ou realizar um cálculo;
 - ✓ Iniciar uma ação em outro objeto;
 - ✓ Controlar e coordenar atividades em outros objetos.

- Atribuição de responsabilidades durante o design:
 - ✓ Responsabilidades do tipo **Conhecer** ou **Saber**:
 - Inferidas a partir do modelo conceitual (são os atributos e relacionamentos do modelo).
 - ✓ Responsabilidades do tipo **Fazer**:
 - São realizadas por um único método ou uma coleção de métodos (de uma ou mais classes) trabalhando em conjunto.

- Responsabilidades de **Conhecer** ou **Saber**:
 - ✓ Um Pedido é responsável por saber seu total.
 - ✓ Um Aluno é responsável por saber em que turmas ele está inscrito.
 - ✓ Um Carro é responsável por conhecer o motor que está instalado nele.
 - ✓ Um Time é responsável por saber quanto ele gasta com os salários dos jogadores.
 - ✓ Um Elevador é responsável por saber em que andar está.
 - ✓ Uma Conta Corrente é responsável por saber seu saldo.

- Responsabilidades de **Fazer**:
 - ✓ Uma Nota Fiscal é responsável por criar seus itens.
 - ✓ Um Sinal de Trânsito é responsável por fechar ou abrir.
 - ✓ Um Funcionário é responsável por criar seus dependentes.
 - ✓ Um Visitante é responsável por registrar sua entrada e saída no prédio.
 - ✓ Uma Conta Corrente é responsável por sacar dinheiro de si mesma.

- São representados por:
 - ✓ 5 padrões fundamentais:
 - Information Expert -(Especialista da Informação)
 - Creator (Criador)
 - High Cohesion (Alta Coesão)
 - Low Coupling (Fraco Acoplamento)
 - Controller (Controladora)
 - ✓ 4 padrões avançados:
 - Polymorphism (Polimorfismo)
 - Pure Fabrication (Invenção Pura)
 - Indirection (Indireção)
 - Protected Variations (Não Fale com Estranhos)

➤ Problema:

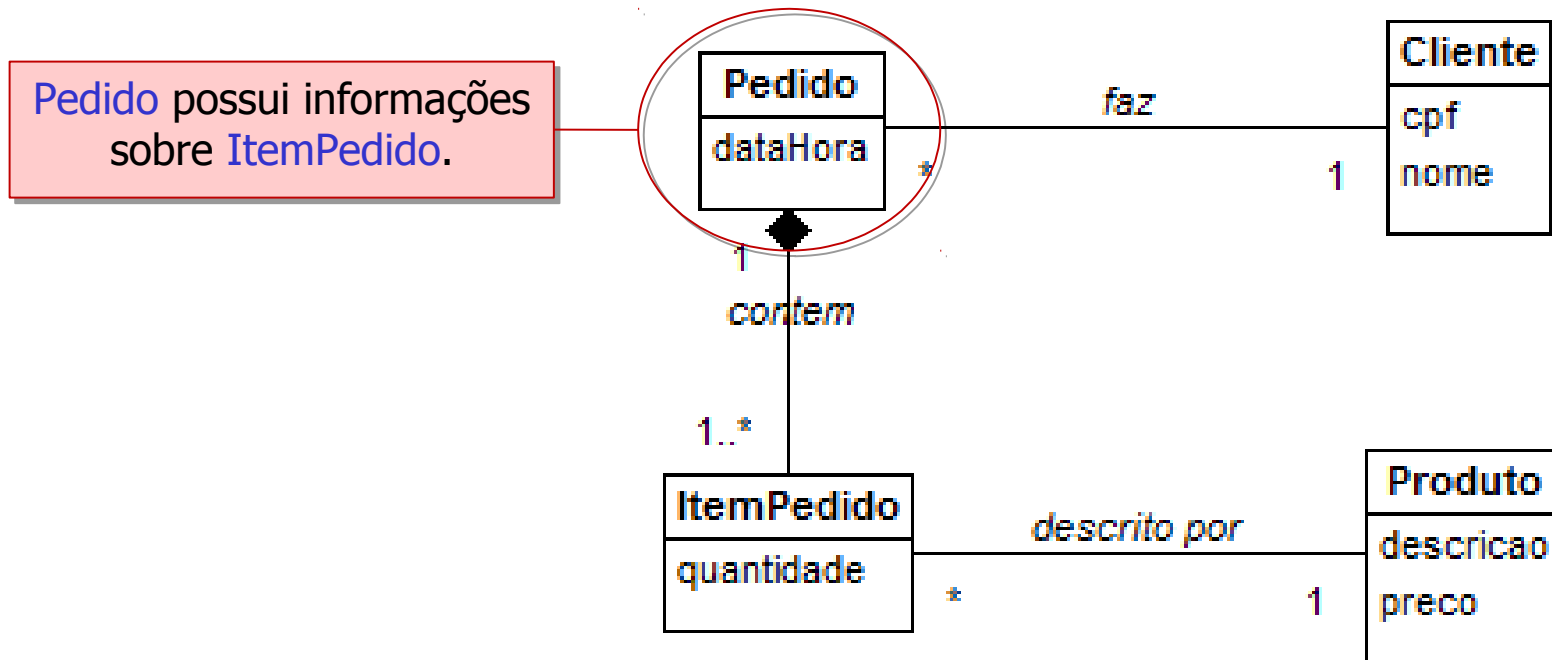
- ✓ Qual o princípio básico para atribuição de responsabilidades para as classes?

➤ Solução:

- ✓ Atribuir responsabilidade ao **especialista da informação**, ou seja, a classe que possui a informação necessária para cumprir com determinada responsabilidade-.

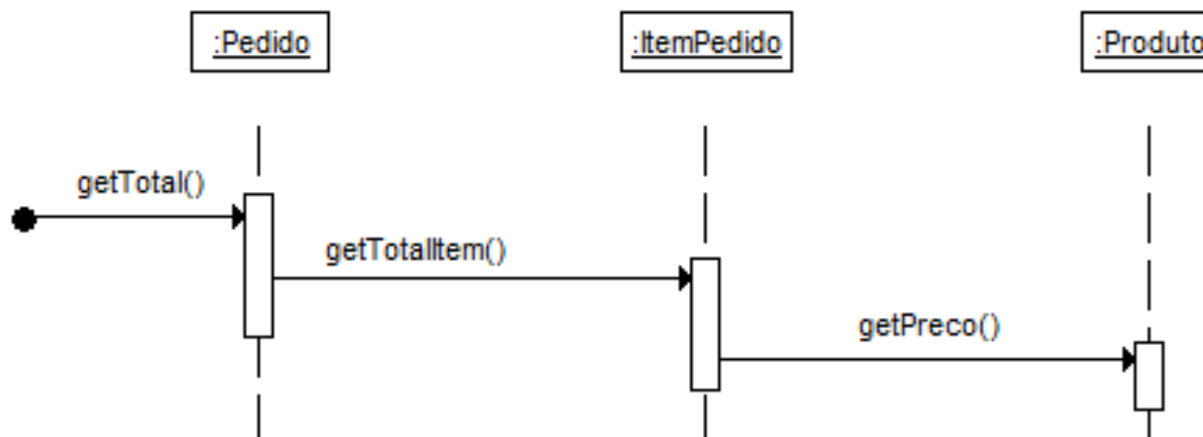
➤ Exemplo:

- ✓ A partir do modelo abaixo, deseja-se saber o **valor total de um pedido**. Qual classe deve ser a responsável por esse cálculo?

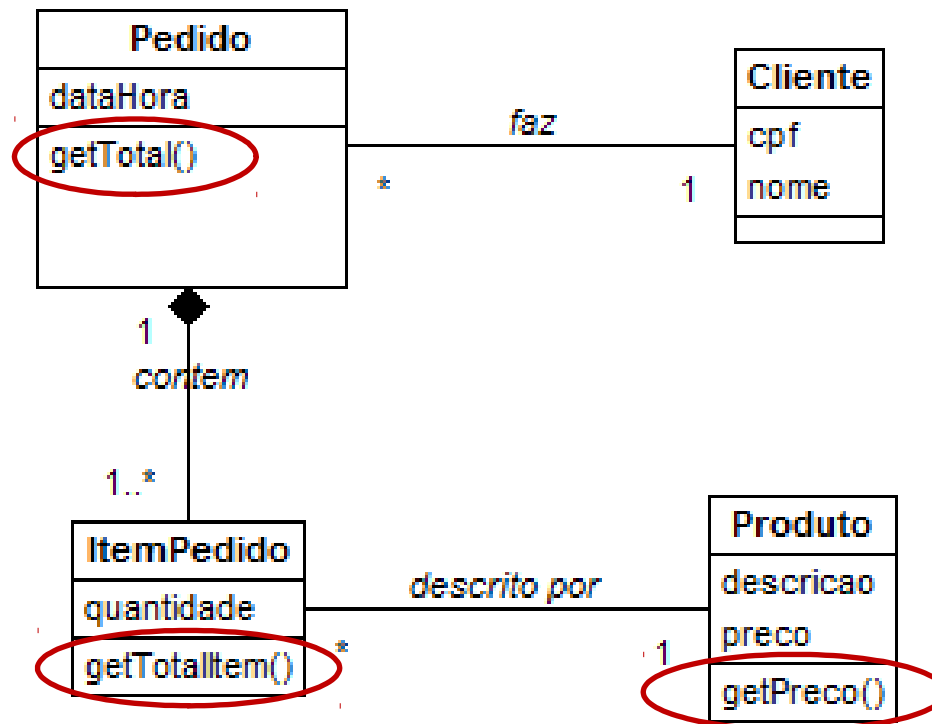


➤ Exemplo:

- ✓ Mas como a classe **Pedido** vai calcular o valor total?
 - Somando o valor de cada item.
- ✓ Mas como saber o valor de cada item?
 - A responsabilidade de calcular o valor de cada item é atribuída ao objeto **ItemPedido**, porque ele conhece a quantidade.
- ✓ Para calcular o valor de cada item é preciso saber o preço do produto. Quem fornece essa informação?
 - O preço do produto é de responsabilidade do **Produto**.



- Exemplo:
 - Agora, o modelo fica assim:



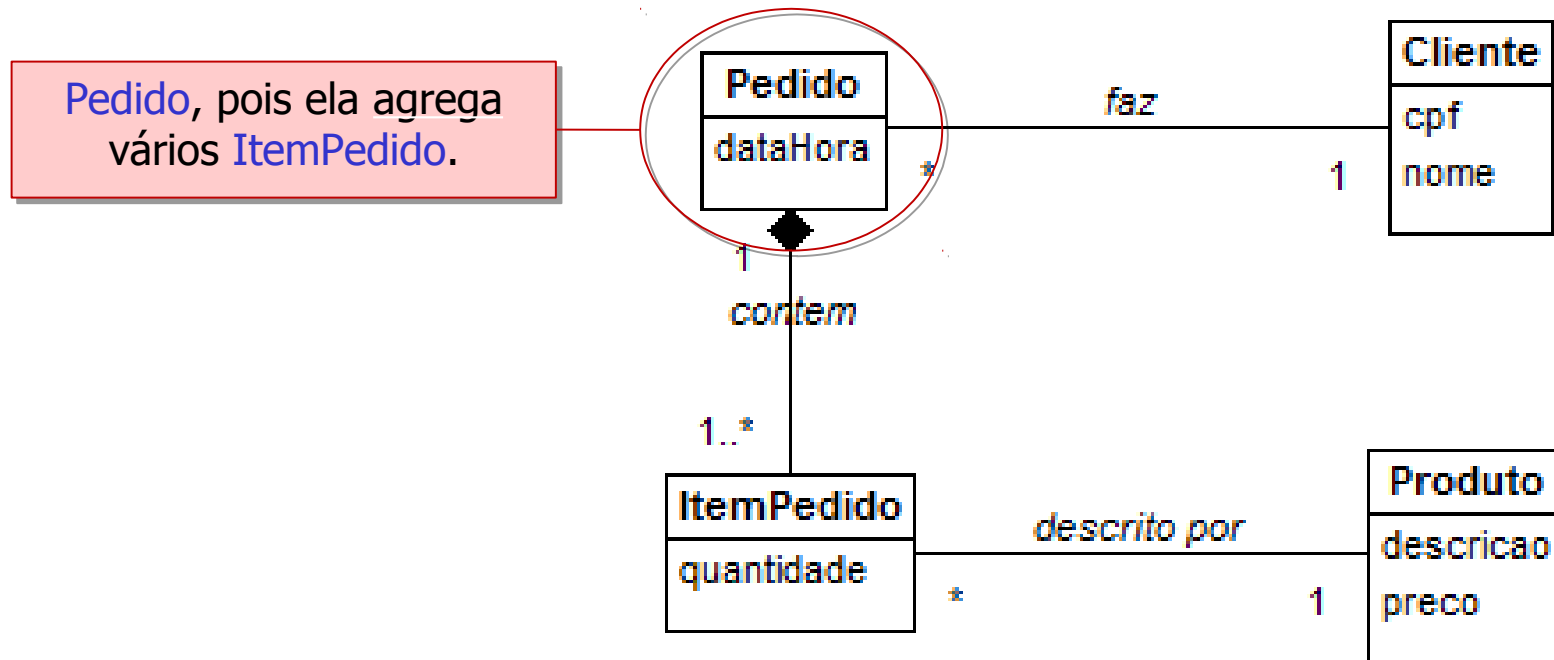
➤ **Problema:**

- ✓ Quem deve ser responsável pela criação de uma nova instância (objeto) de uma classe?

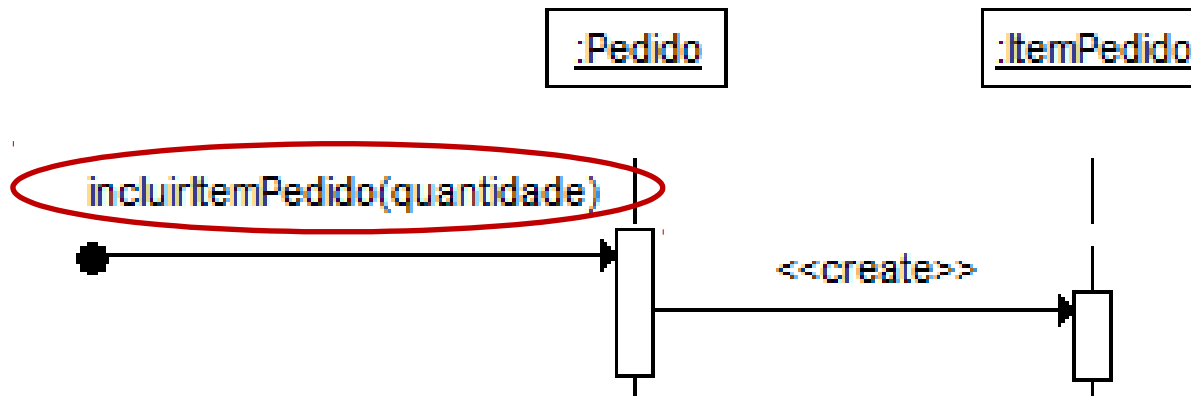
➤ **Solução:** atribua à classe A a responsabilidade de criar objetos B se:

- ✓ A tiver alguma agregação ou composição com B;
- ✓ A registra objetos de B;
- ✓ A usa B de forma intensiva;
- ✓ A possui dados para construção de B.

- Exemplo:
- ✓ Quem classe deve ser responsável pela criação de uma nova instância de **ItemPedido**?

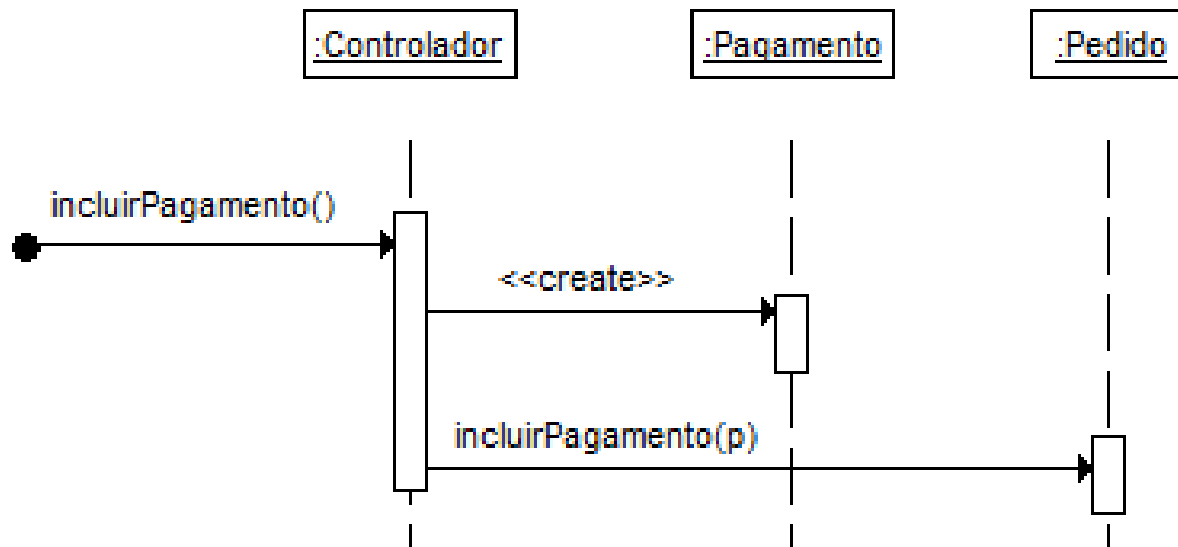


- Exemplo:
 - ✓ Um novo método deve ser criado em `Pedido` para realizar essa operação: `incluirItemPedido()`



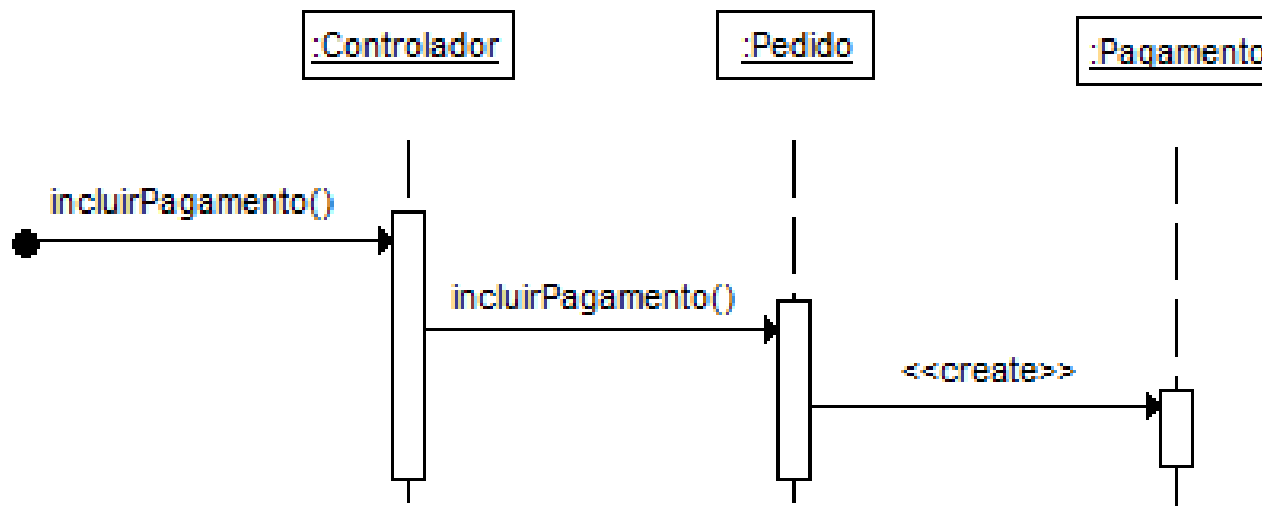
- **Problema:**
 - ✓ Como manter os objetos bem focados, inteligíveis e gerenciáveis?
- **Solução:**
 - ✓ Atribuir uma responsabilidade para que a **coesão se mantenha alta**.
- Uma classe com coesão baixa faz muitas coisas não relacionadas e trabalha demais. Tais classes são indesejáveis, pois elas sofrem dos seguintes problemas:
 - ✓ São difíceis de compreender;
 - ✓ São difíceis de reutilizar;
 - ✓ São difíceis de manter;
 - ✓ São constantemente afetadas por modificações.

- Exemplo:
 - ✓ Que classe é responsável por criar um **Pagamento** e associá-lo a um **Pedido**?

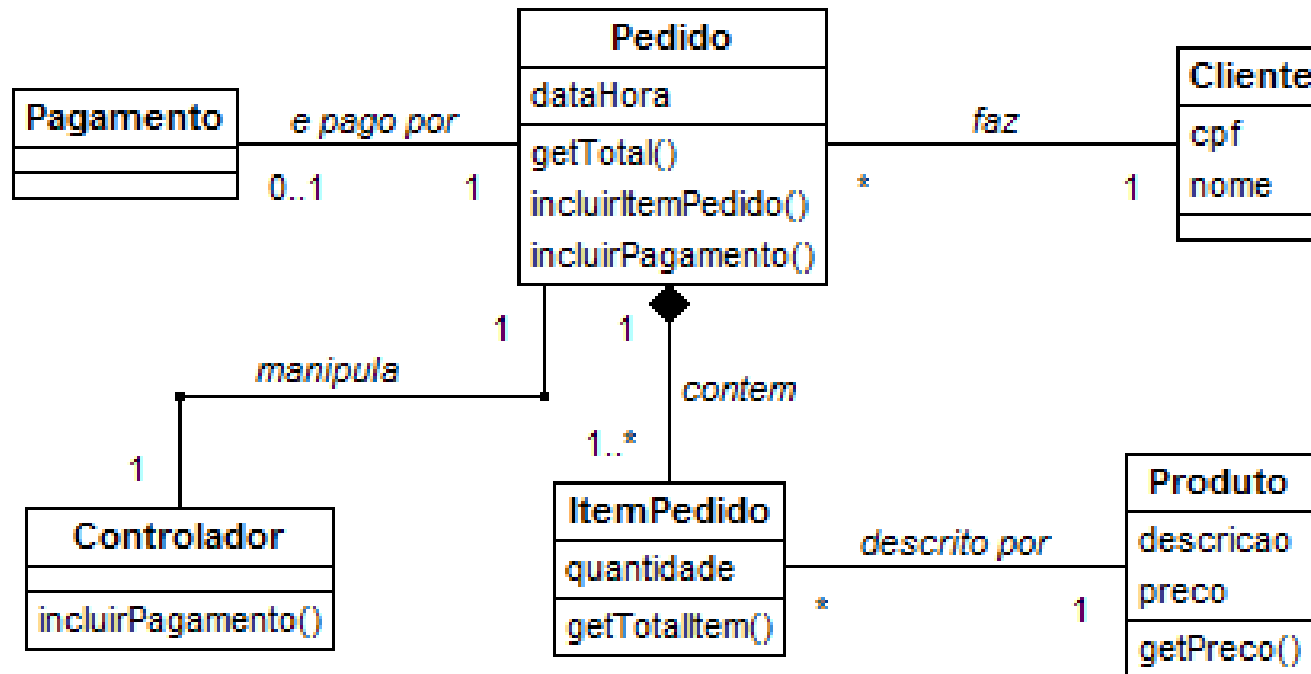


- ✓ Repare que Controlador assumiu a responsabilidade por uma coisa que é parte do Pedido (criar um pagamento **não** é responsabilidade do Controlador!).

- Exemplo:
- ✓ Nesta solução, o **Controlador** delega a responsabilidade ao **Pedido**, aumentando a coesão do **Controlador**.
 - A criação do processo de pagamento agora é responsabilidade do **Pedido** e não do **Controlador**.
 - Faz mais sentido pois o pagamento está associado ao **Pedido**.



- Exemplo:
- ✓ Agora, o modelo fica assim:



➤ Problema:

- ✓ Como obter baixa dependência e baixo impacto relacionado à mudanças?

➤ Solução:

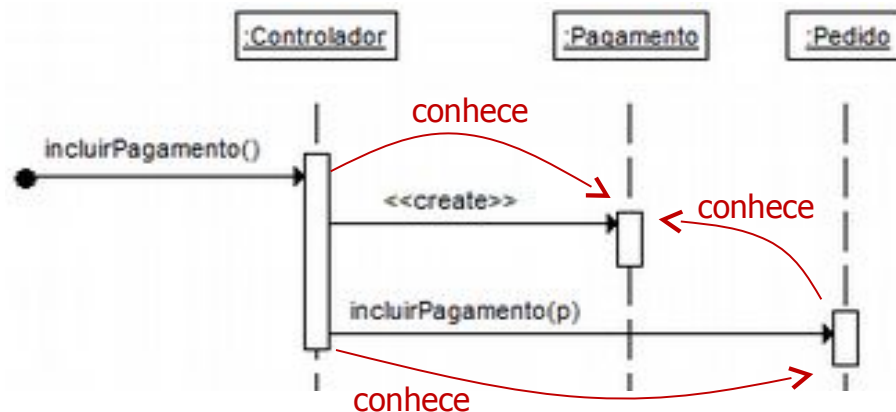
- ✓ Atribuir uma responsabilidade para que o **acoplamento** mantenha-se baixo.

➤ Acoplamento:

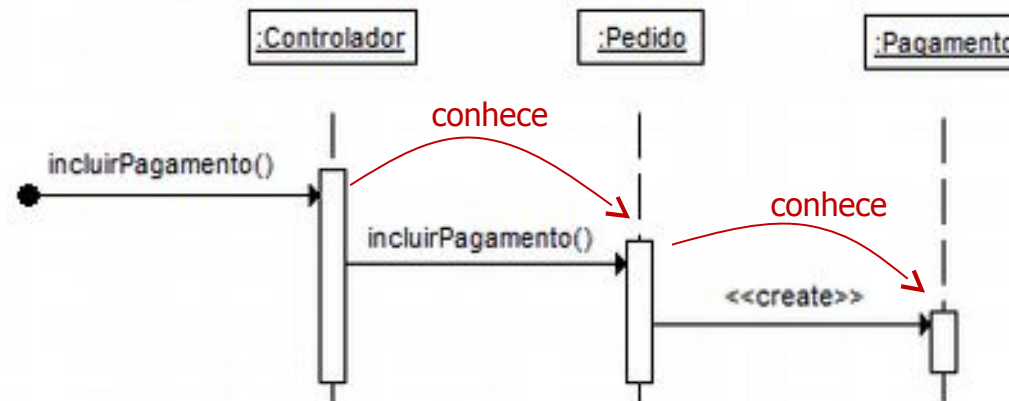
- ✓ É uma medida de quanto um elemento está conectado a ou depende de outros elementos.
- ✓ Uma classe com acoplamento forte depende de muitas outras classes.

➤ Exemplo:

- ✓ Voltando à questão: que classe é responsável por criar um **Pagamento** e associá-lo a um **Pedido**?



1ª alternativa



2ª alternativa

Essa alternativa possui acoplamento mais fraco !

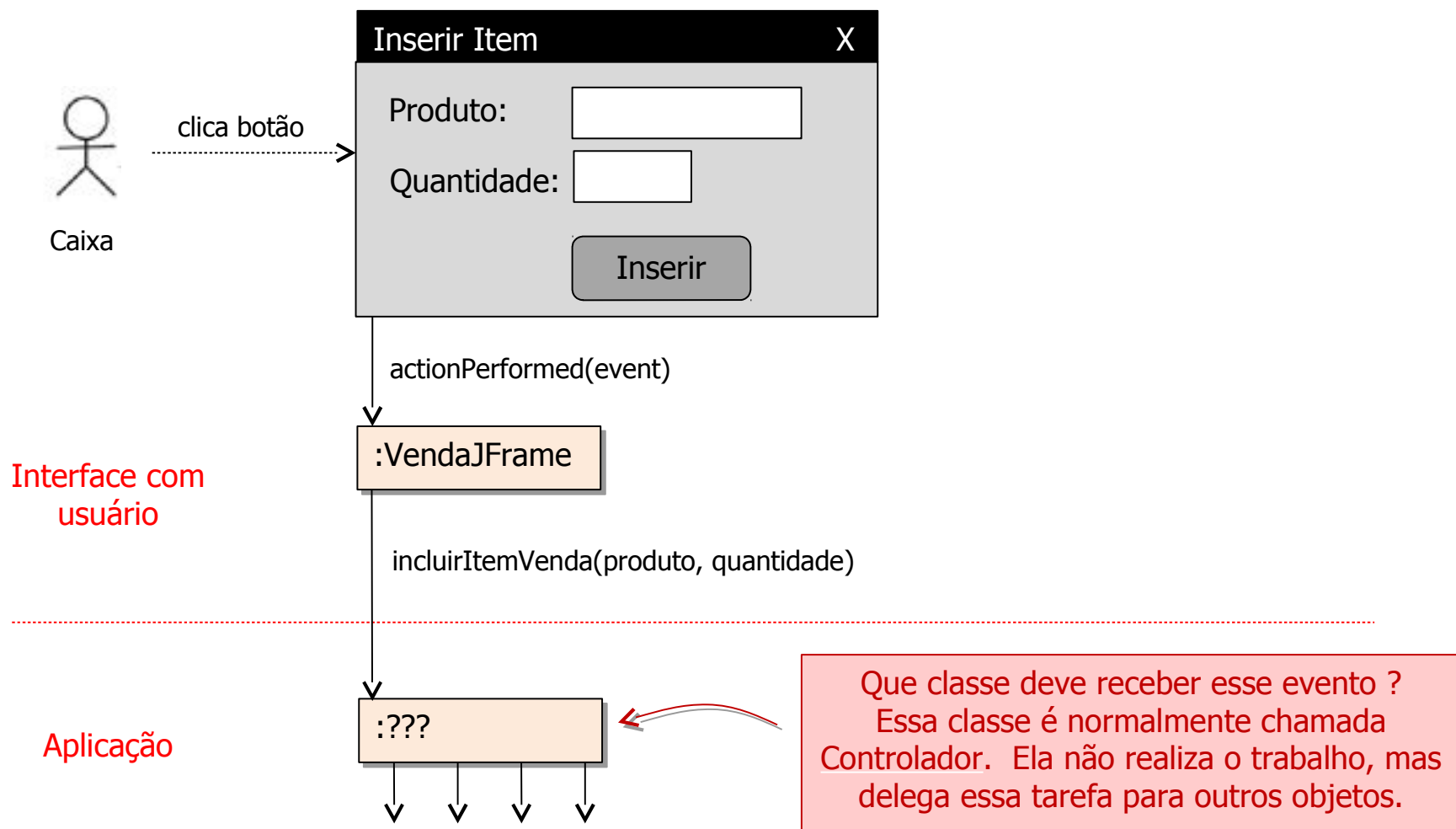
➤ Problema:

- ✓ Quem deve ser responsável por tratar os eventos gerados por atores externos?

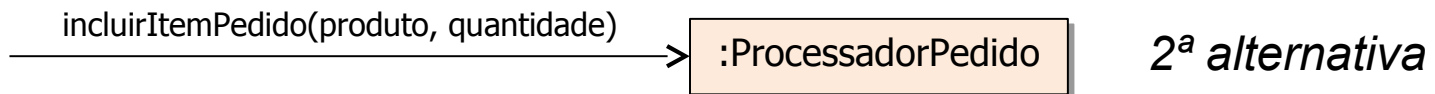
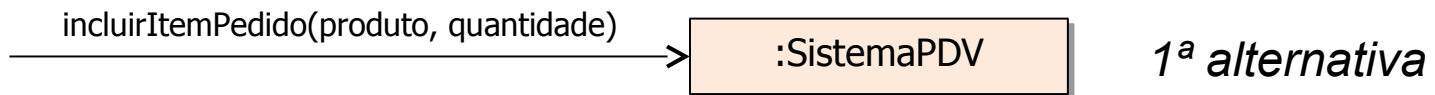
➤ Solução:

- ✓ Atribuir a responsabilidade para tratar um evento para:
 - uma classe que representa todo o sistema ou um subsistema (**controlador façade**);
 - uma classe que representa o caso de uso ou cenário de caso de uso (**controlador de caso de uso**).

➤ Exemplo de Modelo-Visão-Controle:



- Possíveis escolhas:
 1. Um controlador para todo o sistema ou um subsistema.
 2. Um controlador para os eventos de um caso de uso.



- A escolha dependerá de outros fatores.
- Por exemplo: em um sistema com muitos eventos, um controlador único fatalmente terá baixa coesão.

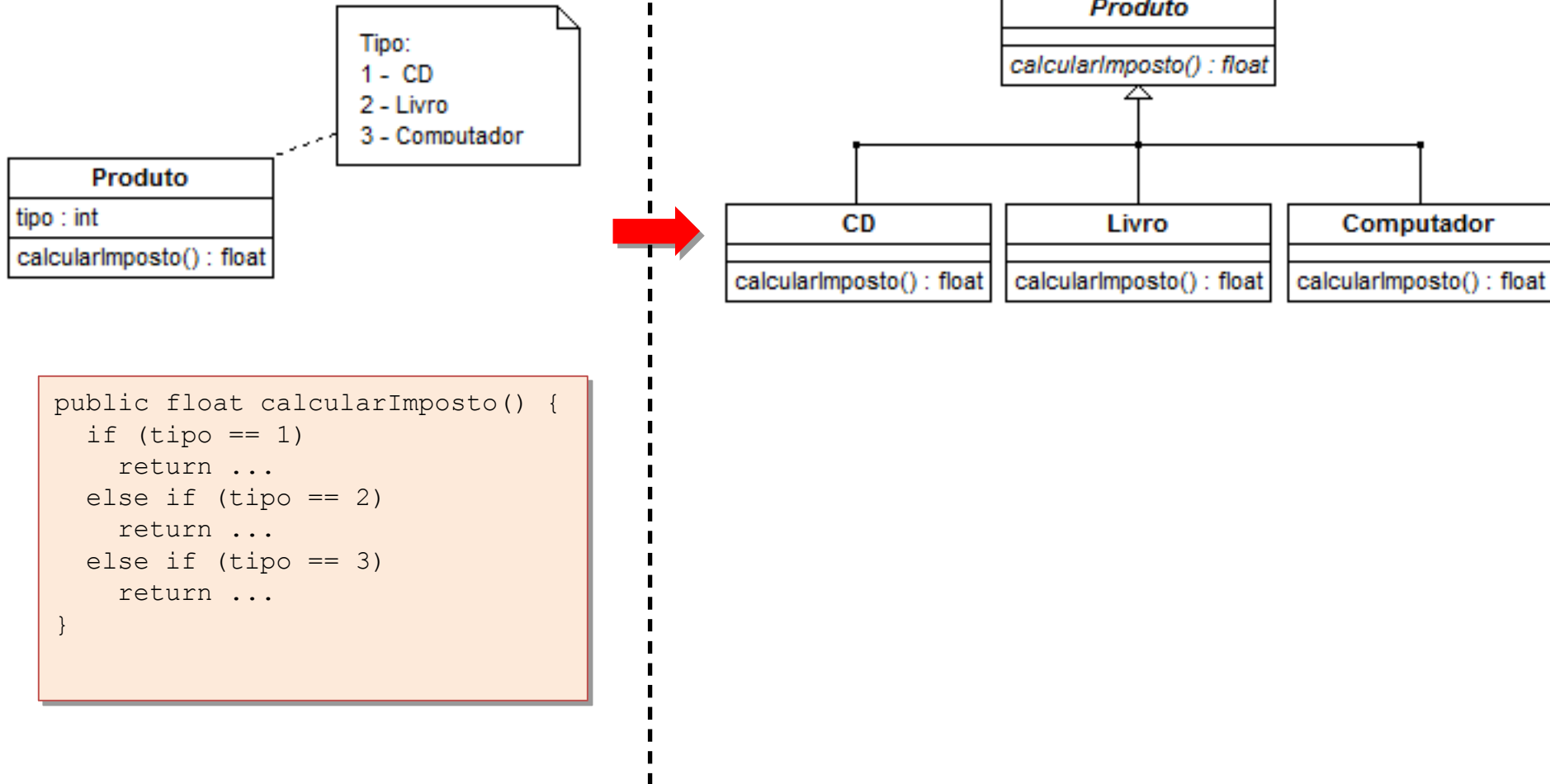
➤ Problema:

- ✓ Como lidar com alternativas que se baseiam na classe?
- ✓ Como criar componentes de software plugáveis?

➤ Solução:

- ✓ Não use lógica condicional (*if-then-else*, *switch*) para executar alternativas de acordo com as classes.
- ✓ Atribua responsabilidades às classes usando operações polimórficas.
- ✓ Refatore!

➤ Exemplo:



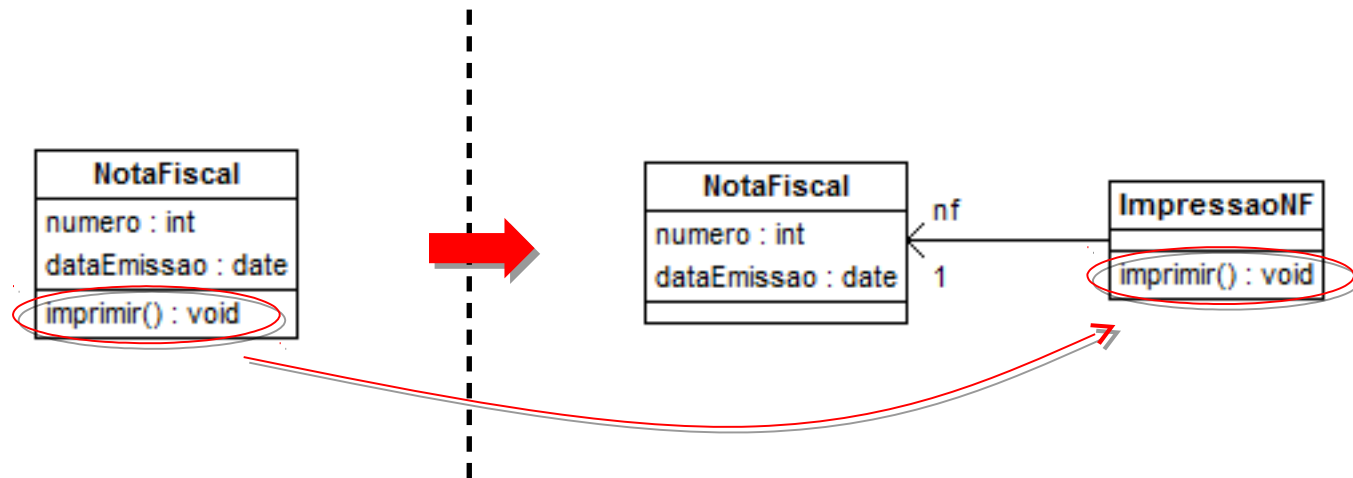
➤ Problema:

- ✓ Atribuir responsabilidades apenas para classes do domínio pode levar a situações de maior acoplamento e menor coesão.
- ✓ Que objeto deve ter a responsabilidade quando você não quer violar **alta coesão** e **baixo acoplamento**, mas as soluções oferecidas por **Expert** não são adequadas?

➤ Solução:

- ✓ Atribuir um conjunto altamente coeso de responsabilidades a uma **classe artificial**, criada pela sua **imaginação**, que não representa um conceito do domínio do problema.

- Exemplo:
- ✓ Apesar de **NotaFiscal** ser a candidata natural a imprimir seus próprios dados, essa decisão levaria à **baixa coesão** e **baixo reuso**.
 - ✓ Assim, criamos uma classe artificial que ficará responsável pela impressão da NF.



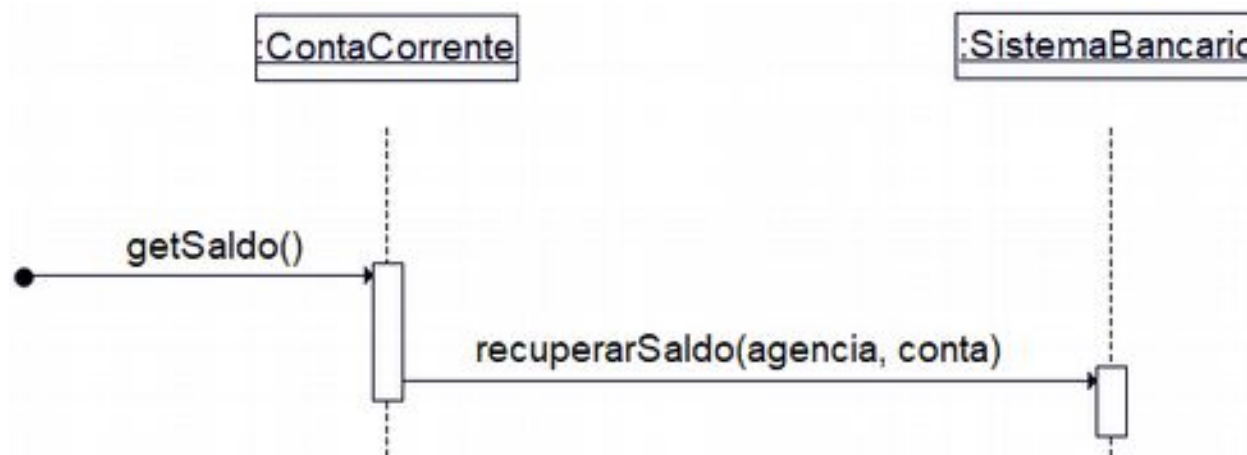
➤ Problema:

- ✓ Pra quem atribuir uma responsabilidade de forma a evitar o acoplamento direto entre dois ou mais objetos?
- ✓ Como desacoplar objetos para que se tenha **baixo acoplamento** e elevado potencial de reuso?

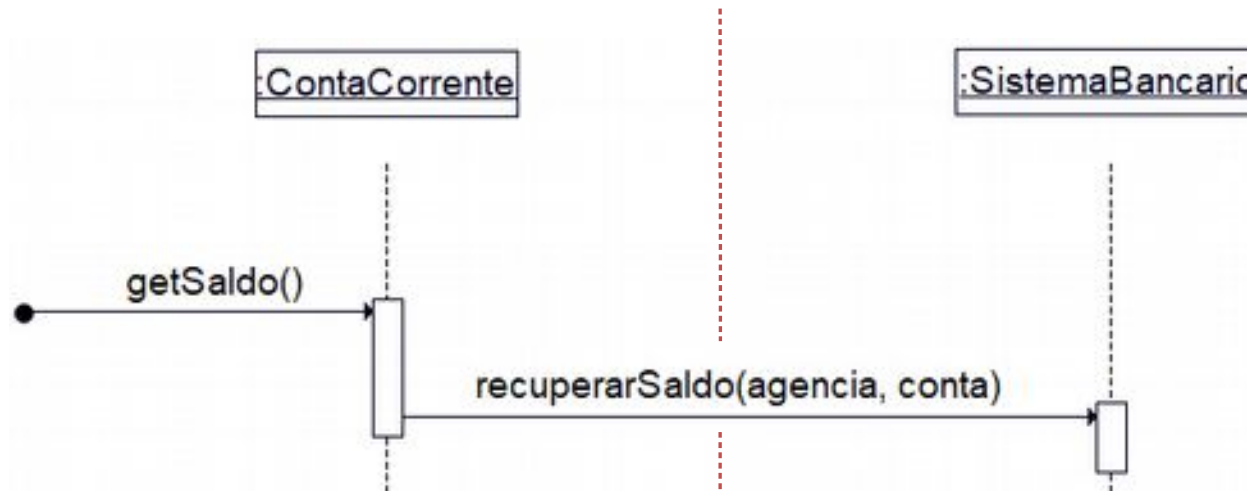
➤ Solução:

- ✓ Atribua a responsabilidade a um **objeto intermediário** que vai mediar as mensagens entre os vários objetos, causando desacoplamento.
- ✓ O objeto intermediário cria uma camada de **indireção** entre os dois componentes que não mais dependem um do outro: agora ambos dependem da indireção.

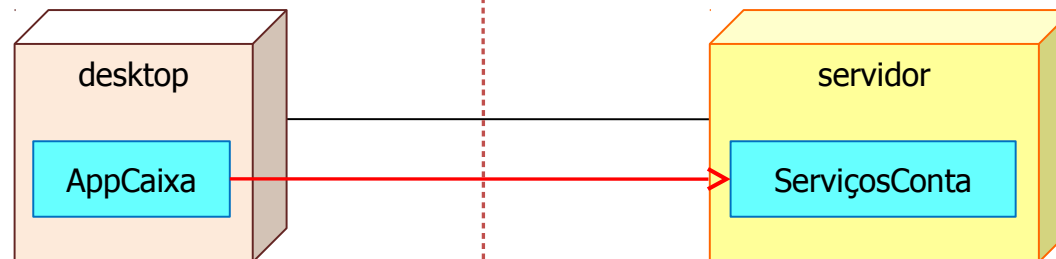
- Exemplo: em um sistema de caixa eletrônico.



- Exemplo: em um sistema de caixa eletrônico.



Essa comunicação com certeza vai precisar de um protocolo, mas eu não quero que a minha classe `ContaCorrente` seja responsável por lidar com os detalhes desse protocolo.



- Exemplo: em um sistema de caixa eletrônico.



A classe **SistemaBancarioAdapter** acrescenta uma camada a mais entre a **ContaCorrente** e o **SistemaBancario**, criando uma indireção e tornando a **ContaCorrente** independente do **SistemaBancário**.

➤ Problema:

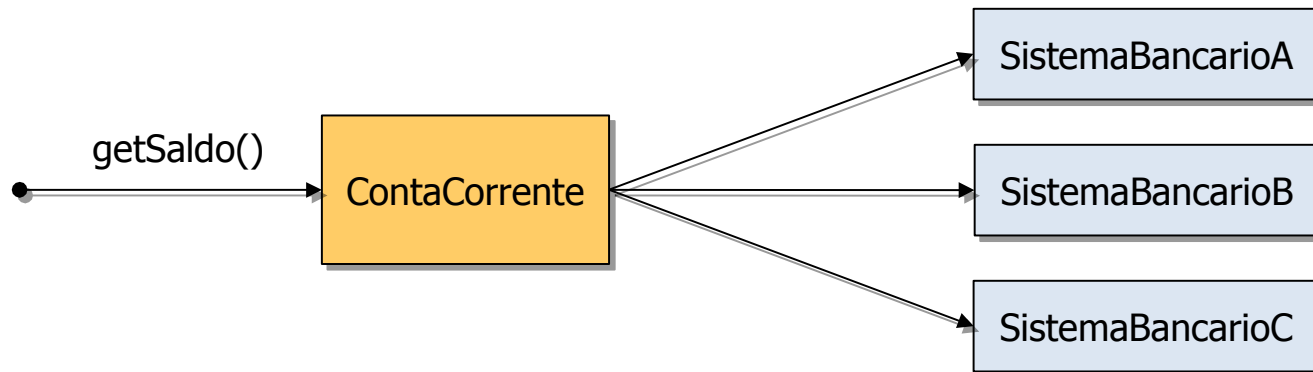
- ✓ Como projetar componentes, subsistemas e sistemas para que as **variações*** nesses elementos não tenham um impacto indesejável nos outros elementos?

➤ Solução:

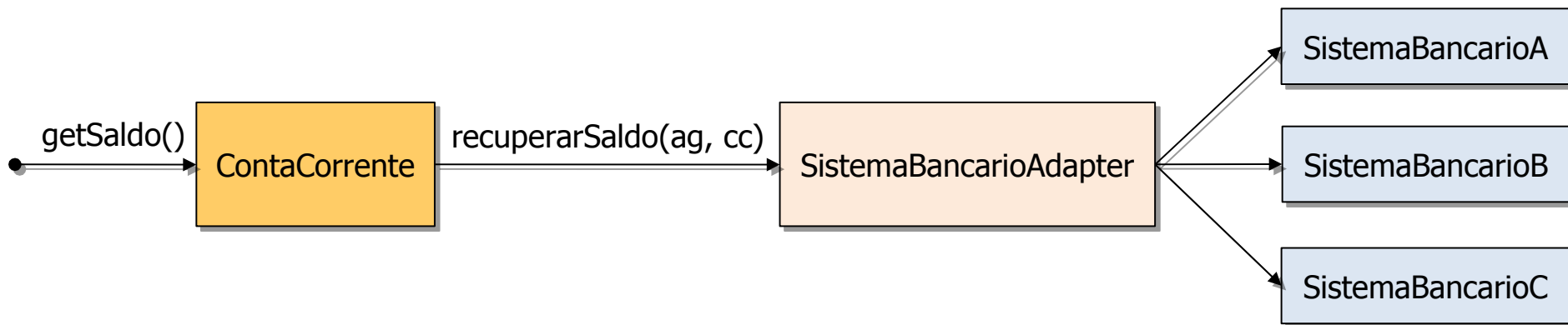
- ✓ Identificar pontos de variação ou instabilidade potenciais e atribuir responsabilidades para criar uma interface estável em torno desses pontos.
- ✓ Encapsulamento, interfaces, polimorfismo e indireção são mecanismos usados para atingir Protected Variation.

* **Variações** nesse contexto estão associadas à evolução ou mudança.

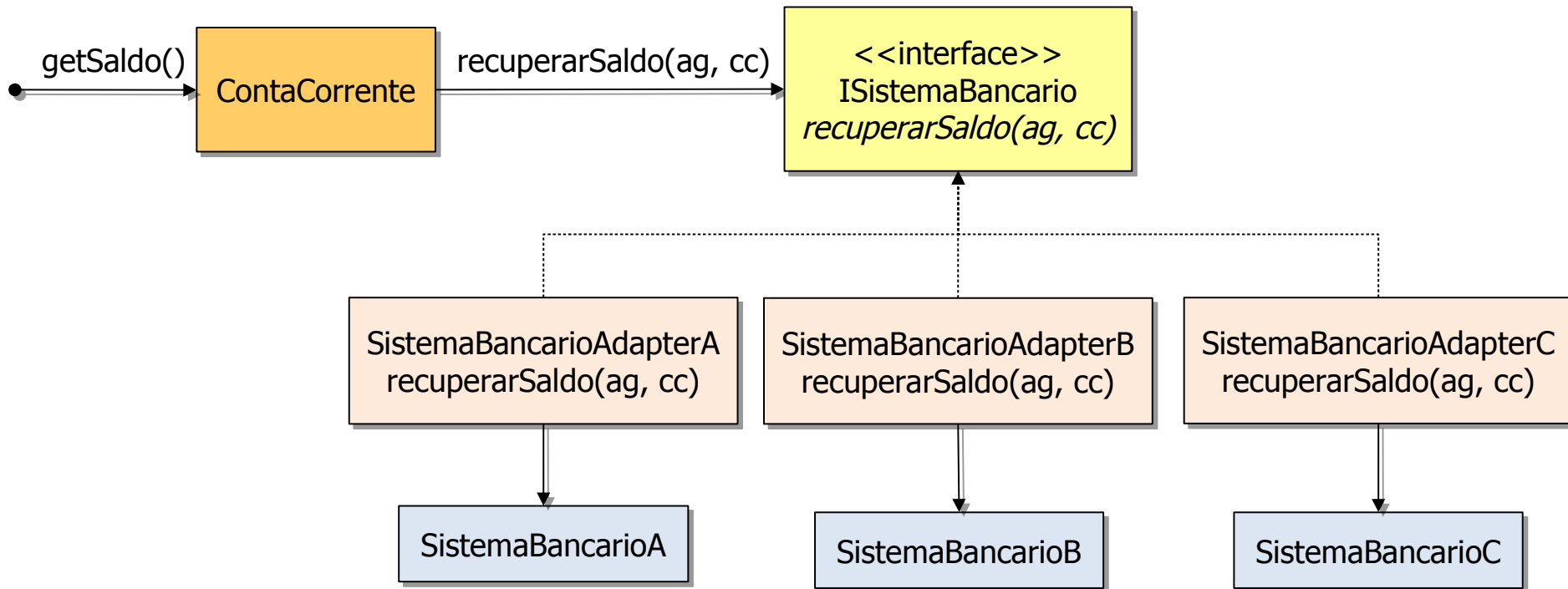
- Exemplo: caixa eletrônico multibancos
 - ✓ Em um caixa eletrônico que trabalha com diversos bancos distintos, cada um dos sistemas bancários tem as suas particularidades.
 - ✓ A classe `ContaCorrente` não deve tratar cada uma dessas particularidades.



- Exemplo: caixa eletrônico multibancos
 - ✓ A aplicação do padrão **Indirection** somente não resolve completamente o problema, pois a classe **SistemaBancarioAdapter** tem que fazer a interface com diversos sistemas bancário distintos.



- Exemplo: caixa eletrônico multibancos
 - ✓ A solução passa pelo uso de **Interface** e **Polimorfismo**.



- Os princípios SOLID foram definidos por Robert Martin em 2000.
- Esses princípios são **recomendações** para organização de classes e seus relacionamentos, de forma que tais classes:
 - ✓ Sejam **menos rígidas**, ou seja, que as mudanças em uma classe tenham menor impacto possível nas demais classes do projeto;
 - ✓ Sejam mais **fáceis de entender**, e;
 - ✓ Sejam a base de componentes que podem ser usados em vários sistemas (**reuso**).

➤ **SOLID** é um acrônimo formado por 5 princípios de projeto:

Single Responsibility Principle (SRP)

✓ Princípio da Responsabilidade Única

Open-Closed Principle (OCP)

✓ Princípio do Aberto-Fechado

Liskov Substitution Principle (LSP)

✓ Princípio de Substituição de Liskov

Interface Segregation Principle (ISP)

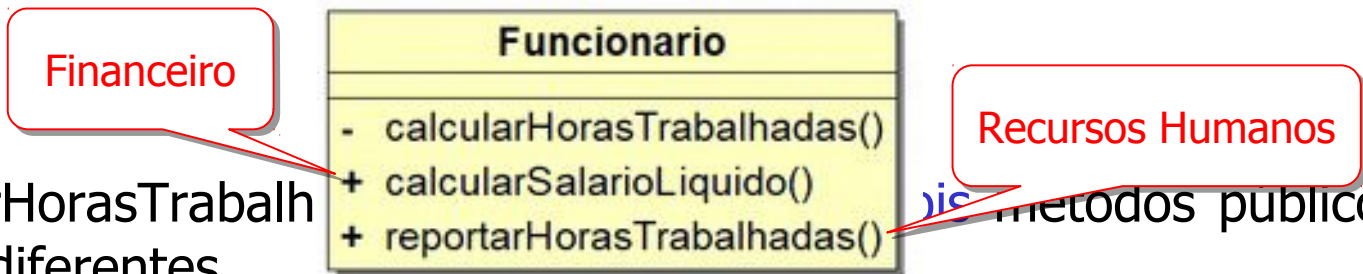
✓ Princípio da Segregação de Interface

Dependency Inversion Principle (DIP)

✓ Princípio da Inversão de Dependência

- *Single Responsibility Principle (SRP)*
- Historicamente, esse princípio tem sido definido como:
Uma classe deve ter uma, e somente uma, razão para mudar
- Entretanto, mudanças em softwares ocorrem para satisfazer às demandas dos *stakeholders*.
- Assim, esse princípio também pode ser descrito como:
Uma classe deve ser responsável por um, e somente um, ator
- Note que o SRP oferece uma perspectiva diferente para o conceito de *coesão*.

➤ Problema:



- calcularHorasTrabalhadas() para diferentes departamentos.
- Que problemas podem ocorrer, caso esse método deixe de contar as horas extras por uma solicitação do Financeiro?

➤ Solução:



- Quando uma classe tem mais de uma responsabilidade, ela pode ser dividida em duas classes originais, cada uma com uma responsabilidade.
- Se existirem dados complexos, eles podem ser divididos em duas classes:
 - a) Ficar em uma classe separada
 - b) Ficar em uma das classes

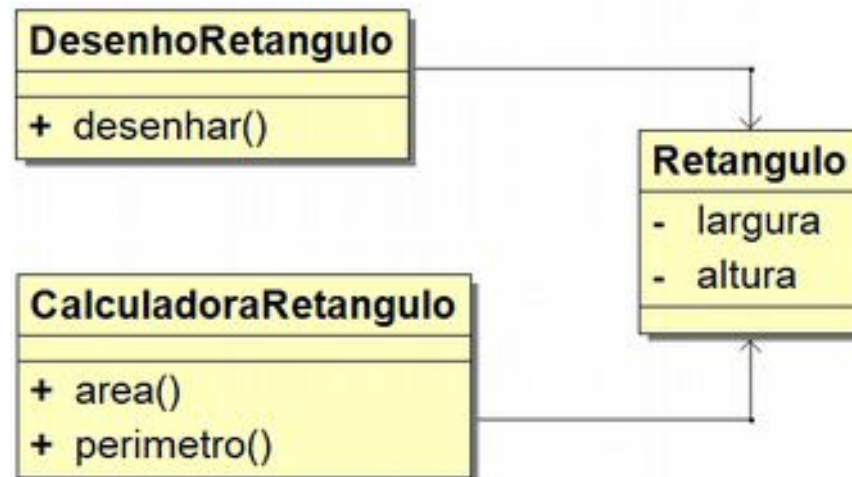


A aplicação indiscriminada do SRP pode provocar um aumento desnecessário na complexidade do software.

➤ Aplicação do SRP:

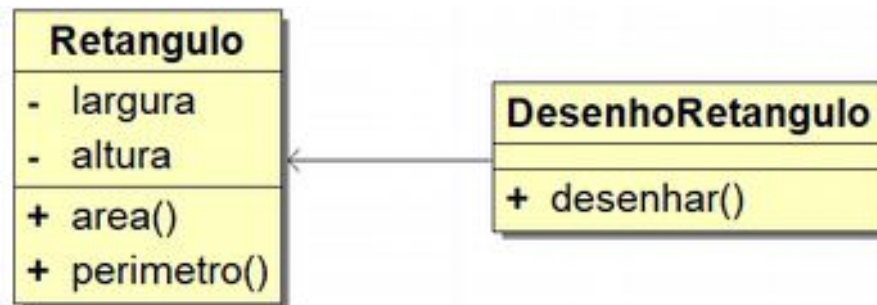
Retangulo
- largura - altura
+ area() + perimetro() + desenhar()

➤ Aplicação do SRP:



1ª versão

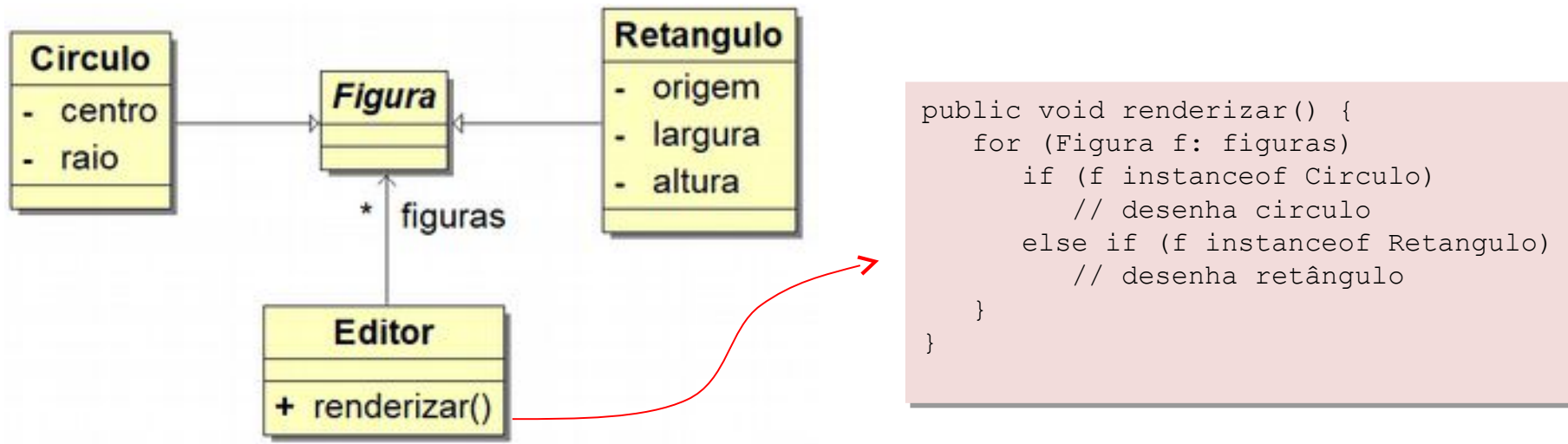
➤ Aplicação do SRP:



2ª versão

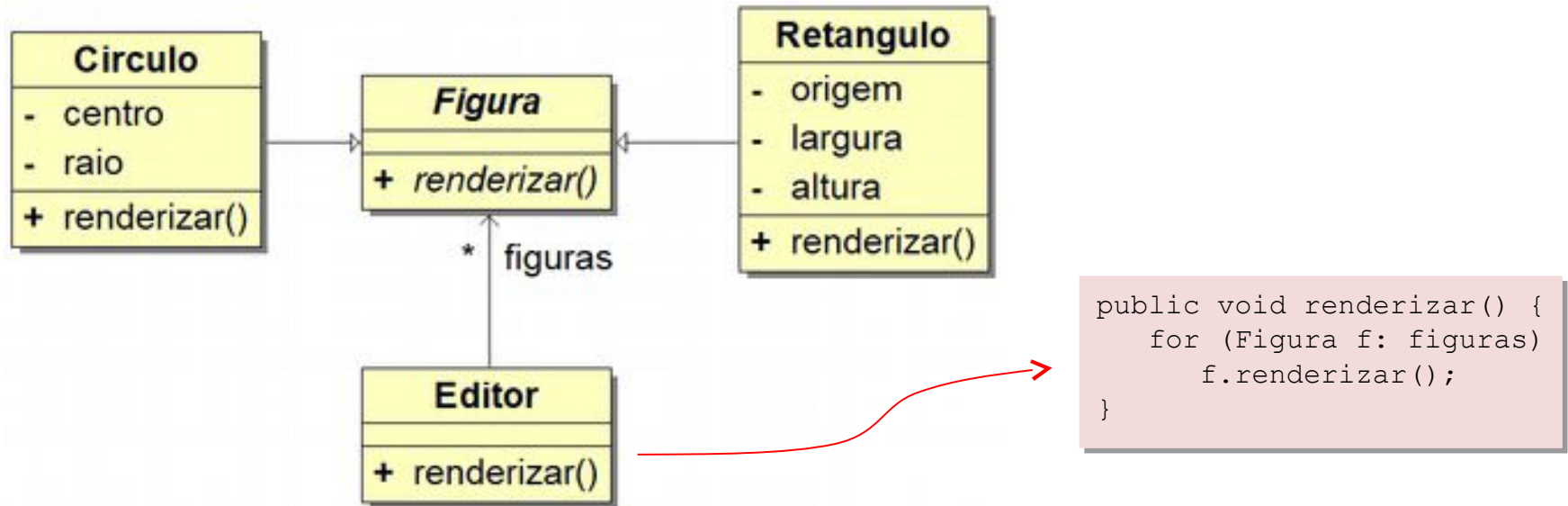
- *Open-Closed Principle (OCP)*
- Esse princípio foi definido por Bertrand Meyer em 1988:
Classes devem estar abertas para extensão e fechadas para modificação
- Quando os requisitos mudam (e eles mudam!) o projeto deve permitir que o comportamento das classes seja alterado pela **adição** de novos códigos e não pela **alteração** do comportamento já existente.

➤ Problema:



➤ Caso seja necessário acrescentar a **Reta** ao Editor, que alterações deverão ser feitas?

➤ Solução:

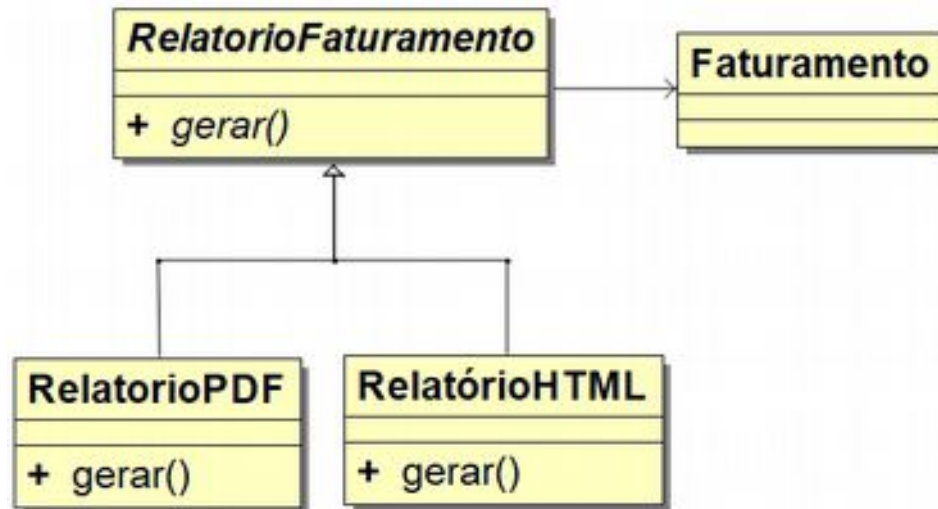


- A solução geralmente passa pelo uso de interfaces/classes abstratas e polimorfismo.
- Antes um método era alterado, agora um método novo (polimórfico) é adicionado.

- Aplicação do OCP: relatório é gerado em HTML e é necessário implementá-lo também em PDF.



- Aplicação do OCP:

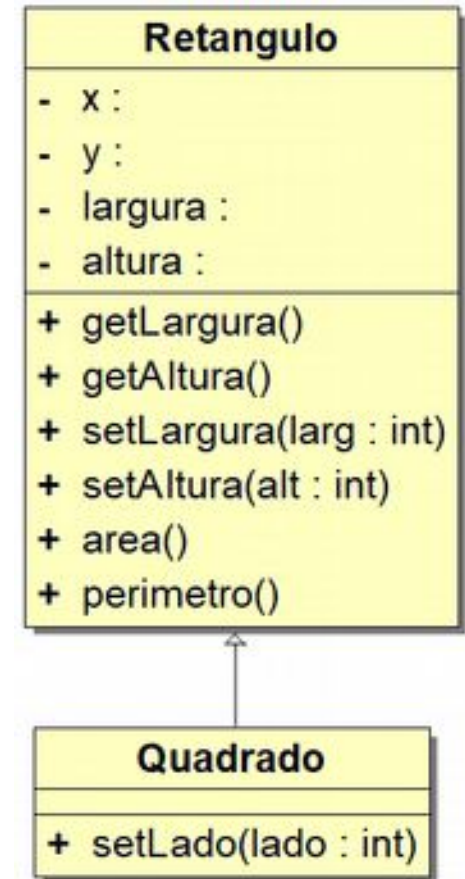


- *Liskov Substitution Principle (LSP)*
- Esse princípio está baseado na definição de subtipo de Barbara Liskov (1988):
 - Em todo lugar onde é usado um objeto da classe X, outro objeto da classe Y, derivada de X, também deve poder ser usado.*
- Esse princípio afeta diretamente como projetamos as hierarquias de classes, evitando que sejam criadas hierarquias sem sentido ou forçadas.

➤ Problema:

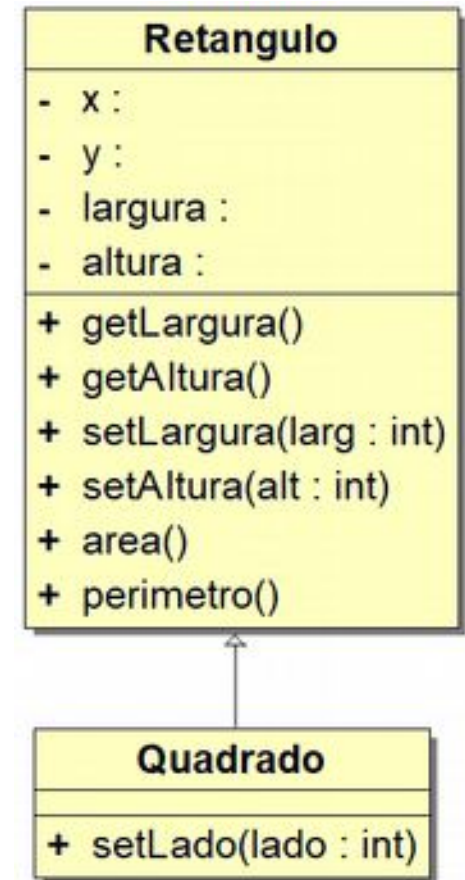
- ✓ Para criar a classe Quadrado poderíamos, por **comodidade**, herdar da classe Retangulo.

```
public class Quadrado extends Retangulo {  
    public float getLado() {  
        return largura;  
    }  
  
    public void setLado(float lado) {  
        largura = lado;  
        altura = lado;  
    }  
}
```



- Problema:
- ✓ Porém, essa classe tem um problema sério!

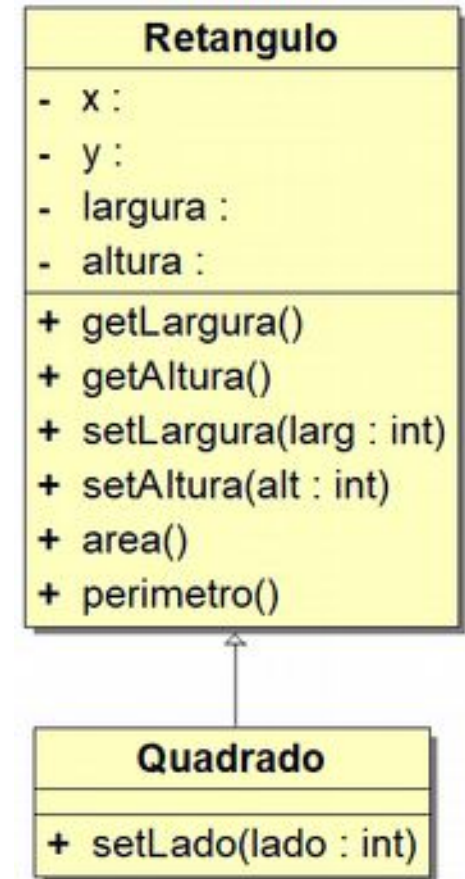
```
Quadrado q = new Quadrado();  
  
q.setLado(5);  
q.setLargura(10); // ?!?!?!?
```



➤ Problema:

- ✓ Para corrigi-lo devemos sobrescrever os métodos `setLargura()` e `setAltura()`:

```
public class Quadrado extends Retangulo {  
    public float getLado() {  
        return largura;  
    }  
  
    public void setLado(float lado) {  
        largura = lado;  
        altura = lado;  
    }  
  
    public void setLargura(float valor) {  
        setLado(valor);  
    }  
  
    public void setAltura(float valor) {  
        setLado(valor);  
    }  
}
```



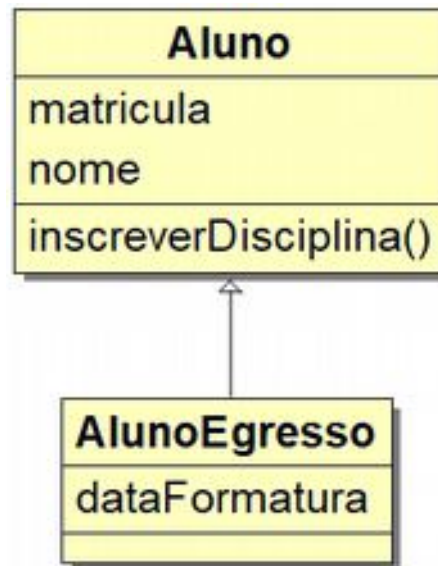
➤ Problema:

- ✓ Matematicamente o quadrado agora está consistente, mas é no mínimo estranho poder alterar a **largura** e a **altura** de um quadrado!

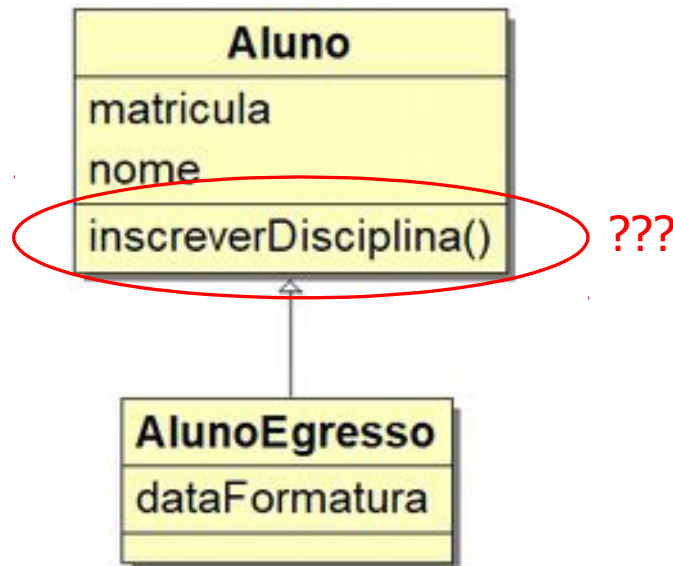
```
Quadrado q = new Quadrado();  
  
q.setLado(5);      // altera a Largura e a Altura para 5!  
q.setLargura(10);  // altera a Largura e a Altura para 10!
```

- Isso acontece porque, conceitualmente, um **Quadrado** não é um **Retângulo**, porém criamos uma herança apenas para economizar algumas linhas de código.

- Aplicação do LSP: essa hierarquia viola o LSP?

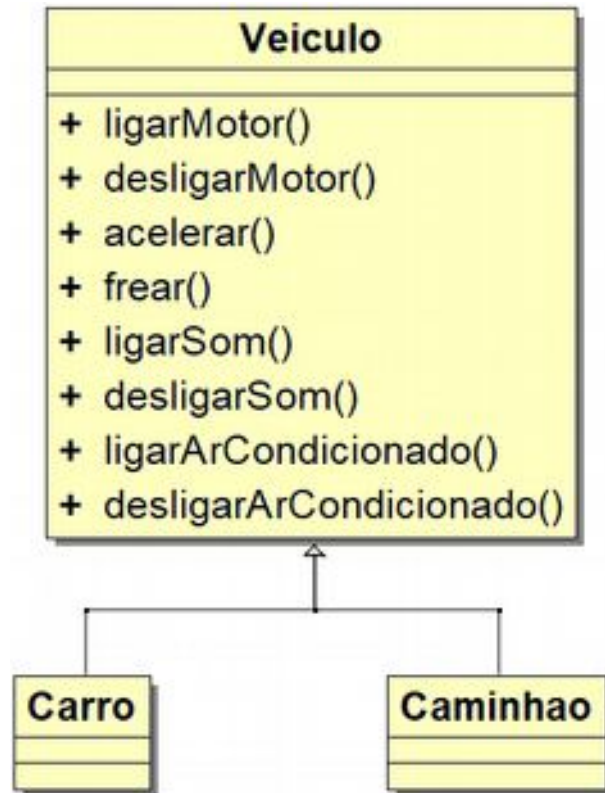


- Aplicação do LSP: essa hierarquia viola o LSP?



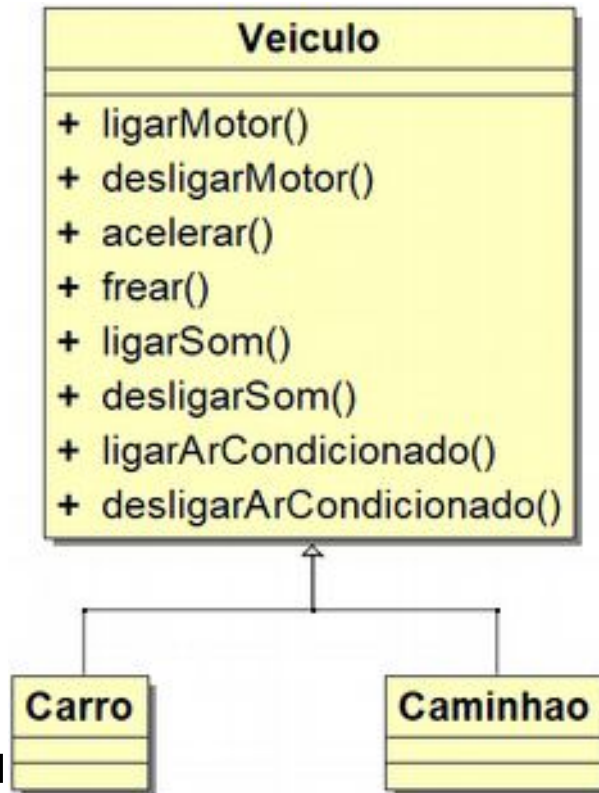
- *Interface Segregation Principle (ISP)*
- Esse princípio recomenda que:
 - Classes cliente não devem ser forçadas a depender de interfaces que elas não utilizam.
- Isso sugere que os projetistas devem evitar criar classes que dependam de coisas que elas não usam.
- Esse tipo de dependência não é recomendado, porque as classes cliente terão acesso a serviços que podem não fazer sentido no seu contexto.

➤ Problema:



➤ O que acontece se incluirmos a classe **Moto** na hierarquia?

➤ Problema:



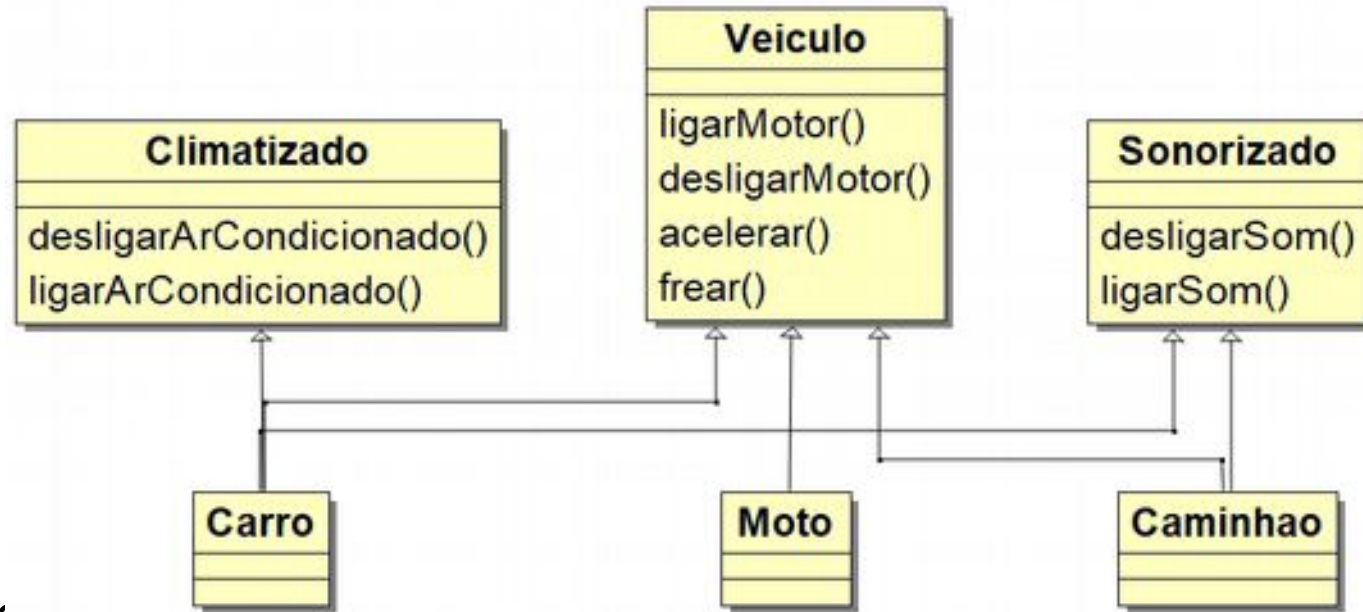
```

Moto m = new Moto();

m.ligarArCondicionado();
    
```

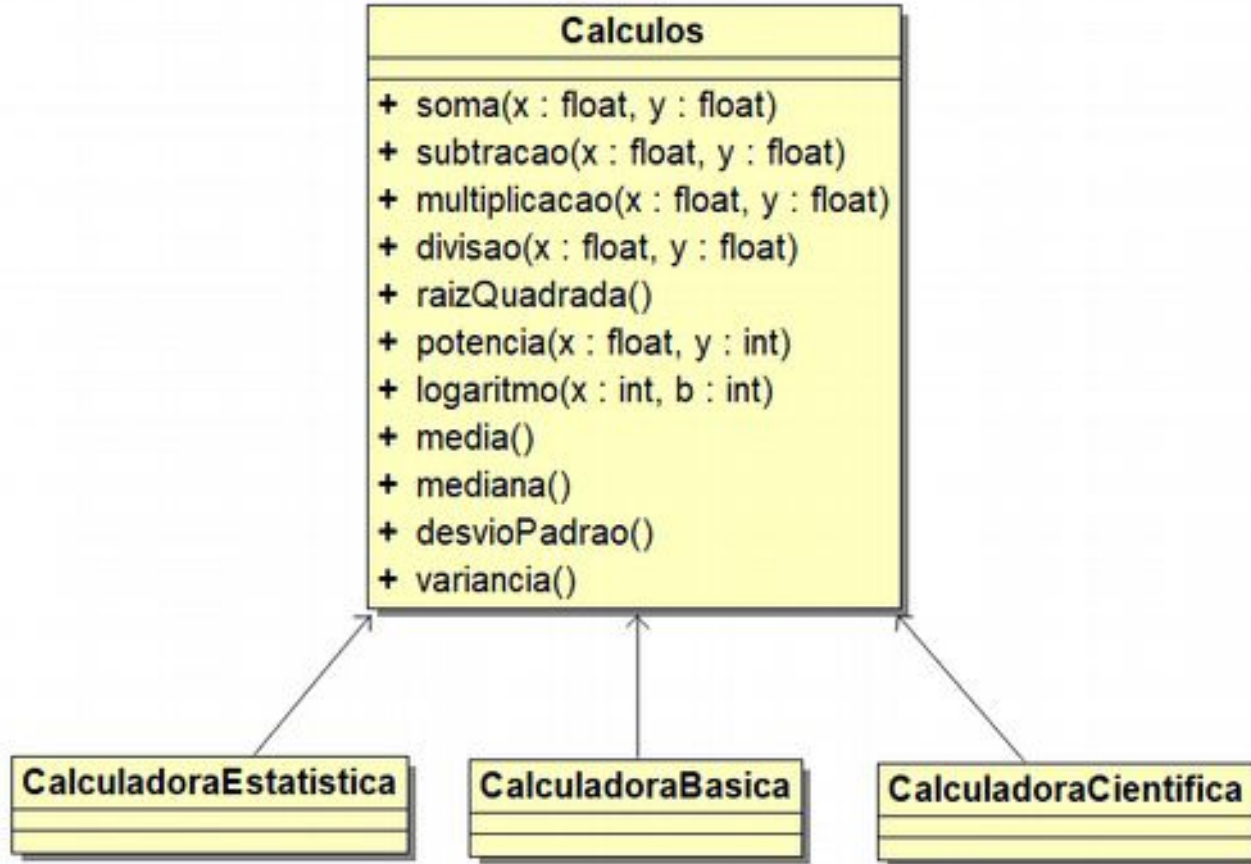
➤ Ao incluir **Carro** e **Caminhao** em **Veiculo**, teremos sérias inconsistências!

➤ Solução:

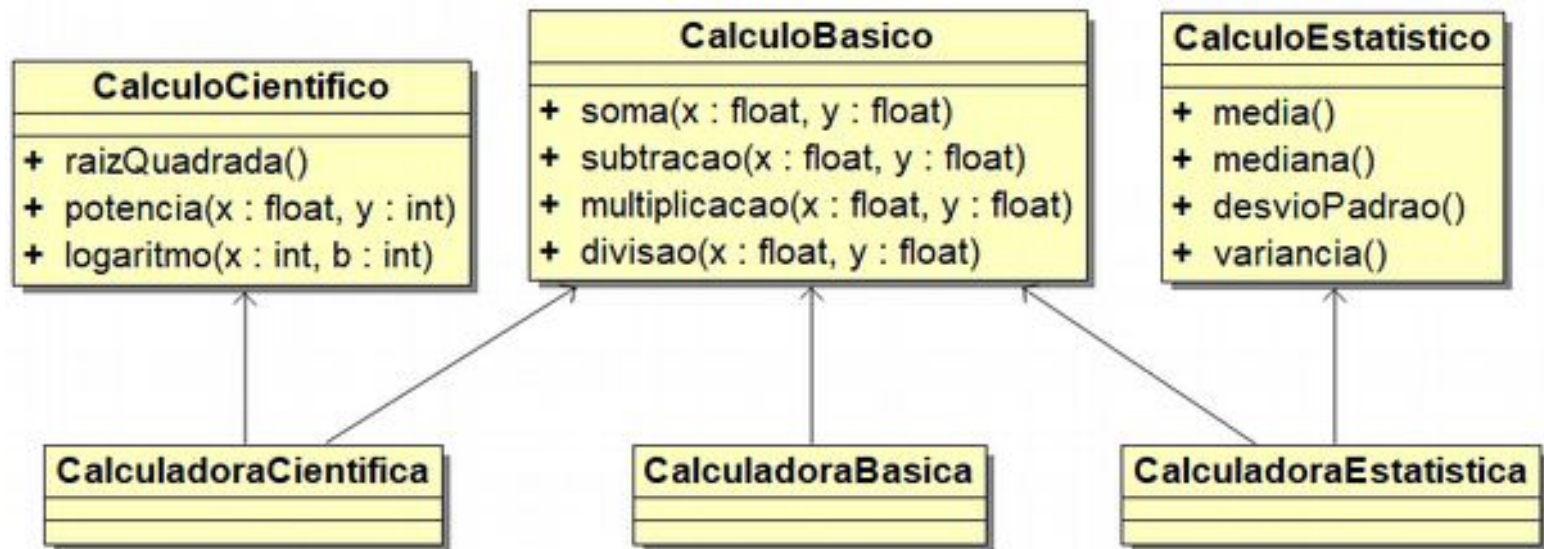


- Segregue interfaces que contêm métodos que não fazem sentido para todas as classes que implementam a interface (veja aqui!).
- Nas linguagens de programação onde não existe herança múltipla, o desenho acima pode ser implementado usando **delegação**.

➤ Aplicação do ISP:



➤ Aplicação do ISP:

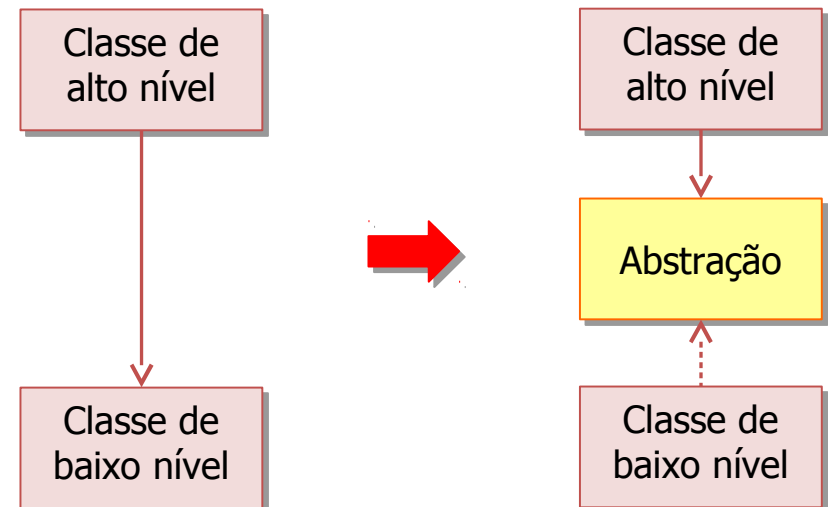


- *Dependency Inversion Principle (DIP)*
- Esse princípio recomenda que:
 - Classes de alto nível não devem depender de classes de baixo nível.*
 - Ambas devem depender de abstrações.*
- Dois conceitos importantes:
 - ✓ Classe de **alto nível**: é aquela onde normalmente se encontra encapsulada a lógica de negócio do software.
 - ✓ Classe de **baixo nível**: é aquela que se distancia dos objetivos do software e está mais relacionada com a tecnologia adotada.

- *Dependency Inversion Principle (DIP)*
- O DIP recomenda que, quando existem **dependências** entre classes, elas devem ser definidas usando **abstrações** (interfaces ou classes abstratas), ao invés de referenciar diretamente as classes.

O DIP:

- ✓ Reduz a fragilidade das classes de alto nível, já que elas não ficam expostas à mudanças na implementação.
- ✓ Aumenta a flexibilidade, pois é possível substituir as implementações sem que as classes de alto nível sejam afetadas.

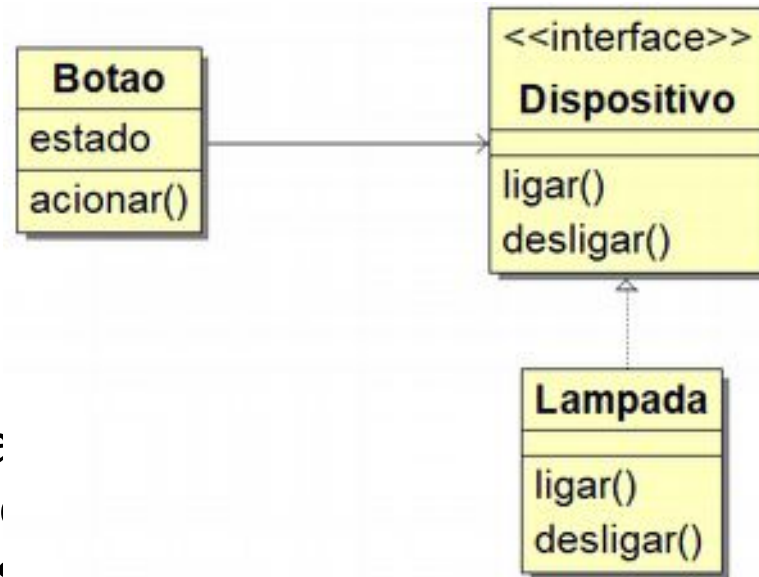


➤ Problema:



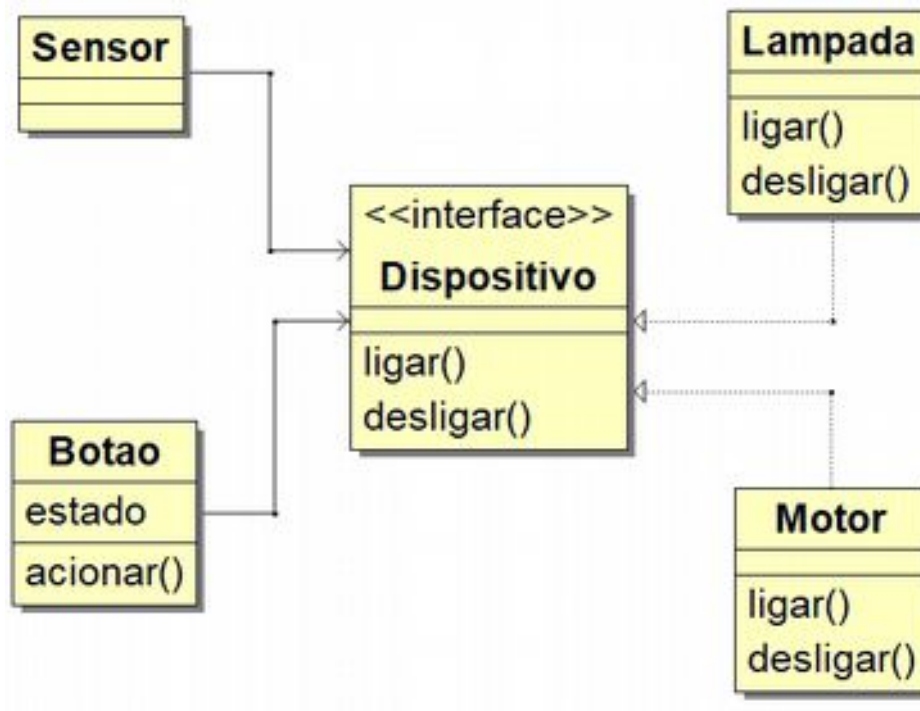
- Esse *design* viola o DIP. Por que?
- ✓ A classe **Botão** depende de uma classe concreta **Lâmpada** ao invés de depender de uma abstração.

➤ Solução:



- Criar uma abstração
- Que abstração seja usada por todos os dispositivos!
 - ✓ Botão deve ser tratado como um dispositivo, seja ele qual for: uma lâmpada, um motor, um alarme, etc.

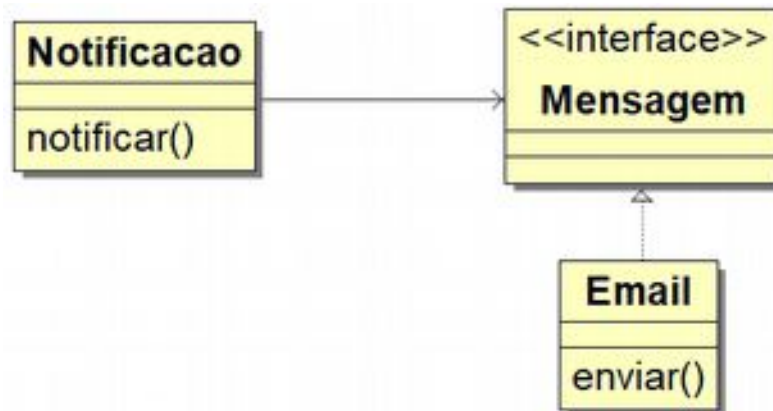
- O DIP permite que outras classes clientes ou servidoras sejam adicionados ao projeto.



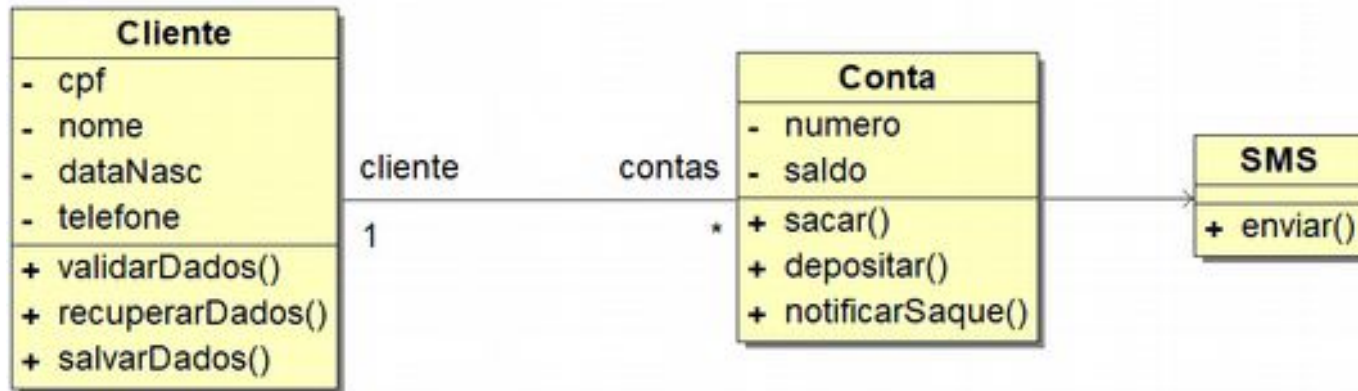
- Aplicação do DIP:



➤ Aplicação do DIP:

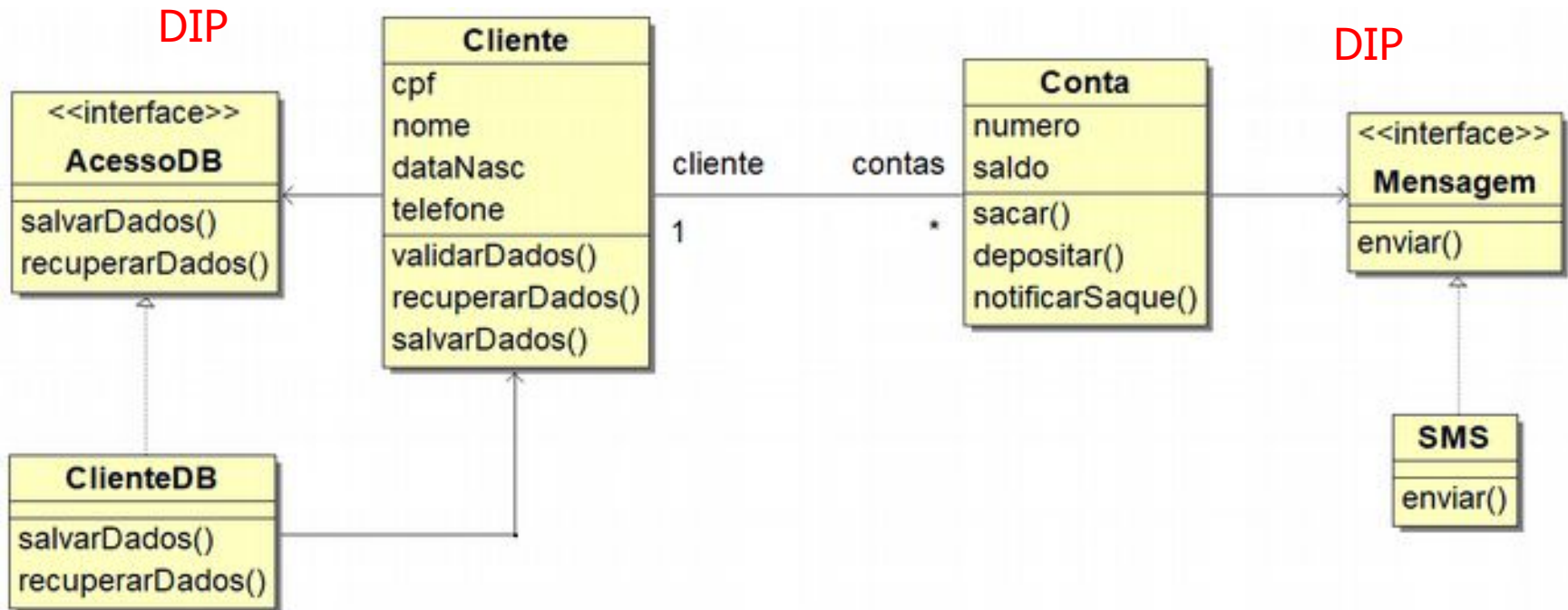


- Aplique os padrões SOLID no modelo abaixo, onde for adequado:

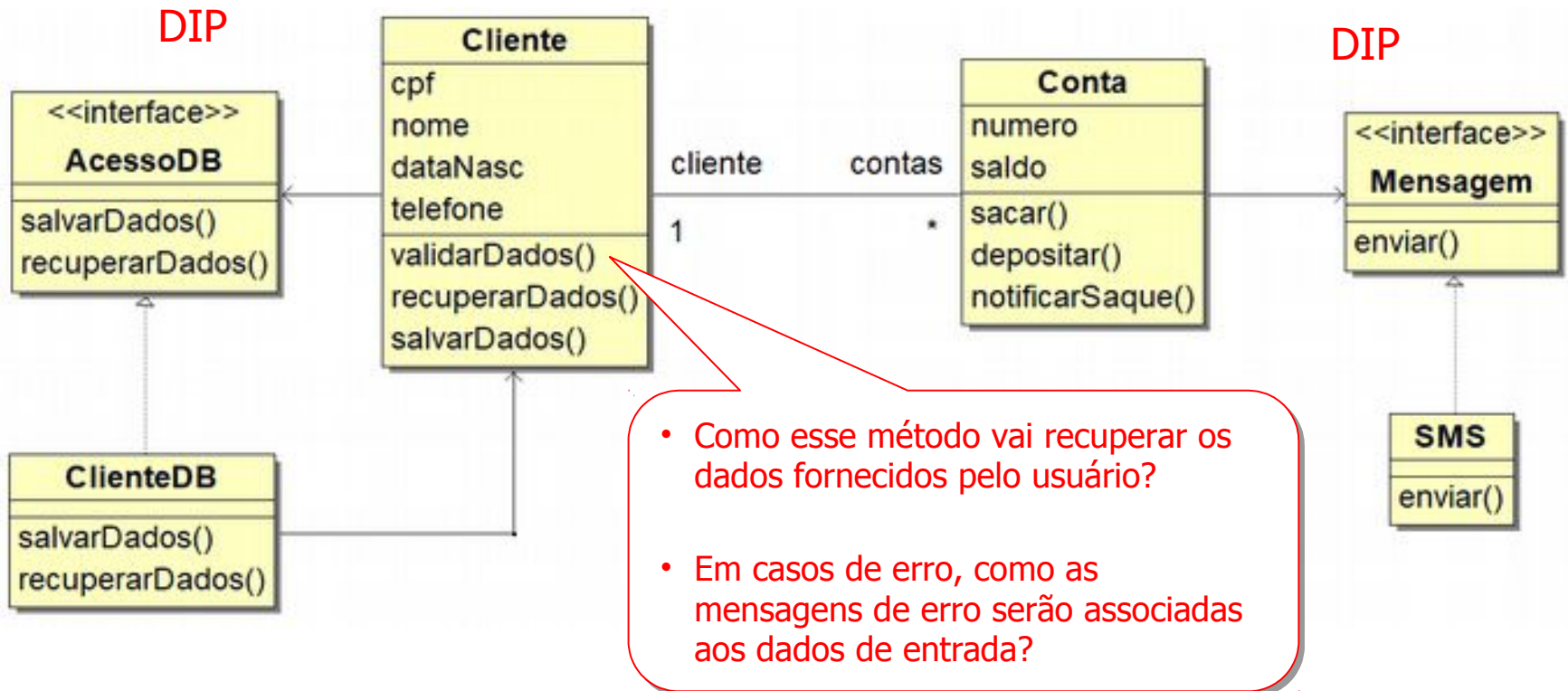


- ✓ **validarDados()**: valida os dados do cliente no cadastro e verifica se já existe outro cliente com o mesmo CPF no banco de dados. O formulário de cadastro pode estar na web ou em uma aplicação móvel.
- ✓ **recuperarDados()**: recupera os dados do cliente de um banco de dados relacional.
- ✓ **salvarDados()**: salva os dados do cliente em um banco relacional.
- ✓ **notificarSaque()**: notifica o cliente, via SMS, de saques realizados em suas contas.

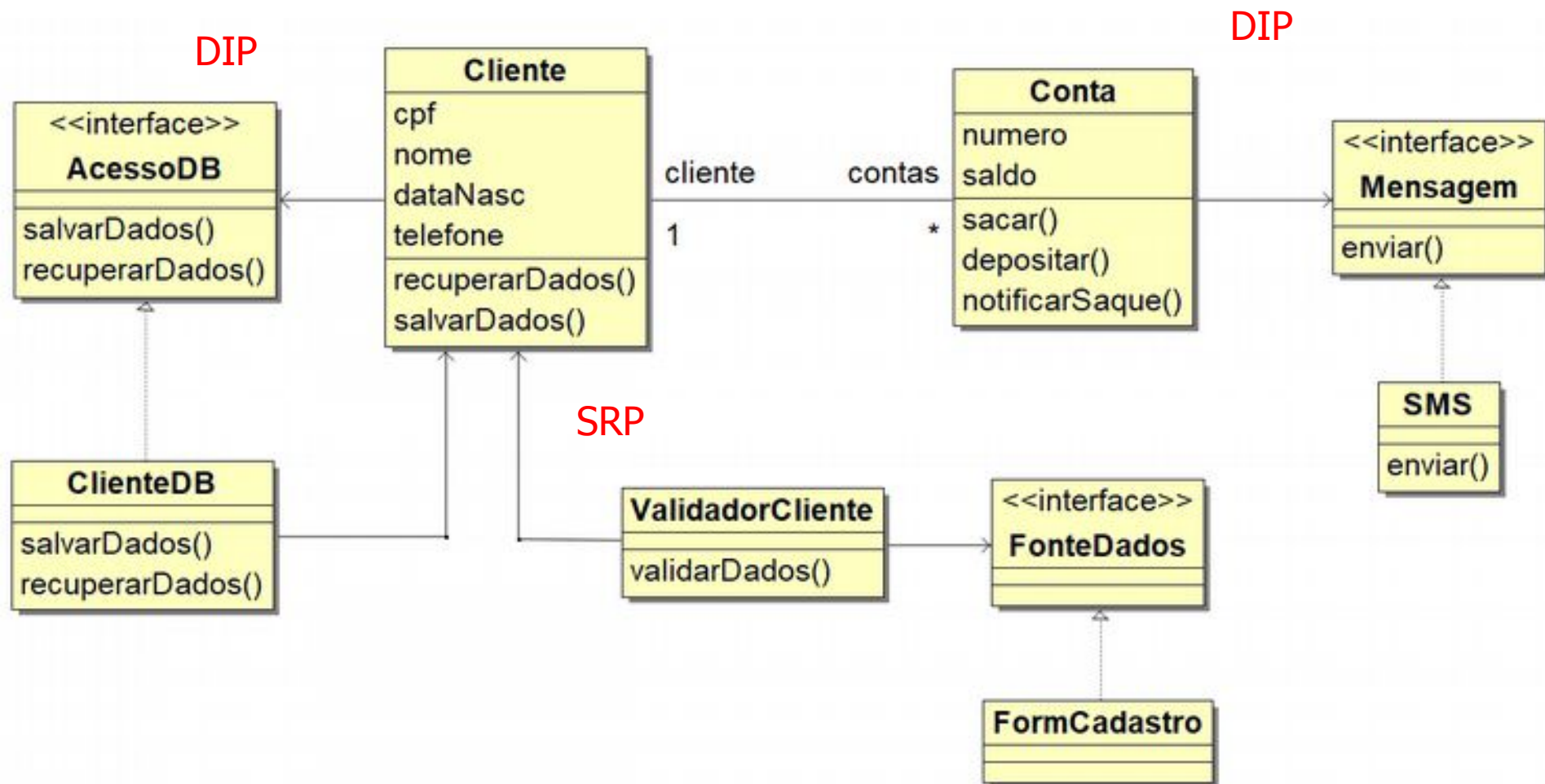
➤ Versão 1:



➤ Versão 1:



➤ Versão 2:



Projeto de Software Orientado a Objetos

10ª parte

Prof. Jobson Massollar

- A ideia de padrões foi apresentada por Christopher Alexander em 1977 no contexto de Arquitetura (de prédios e cidades):

"Cada padrão **descreve um problema** que ocorre **repetidas vezes** em nosso ambiente e o **núcleo da sua solução** para aquele problema, de tal maneira que seja possível **usar essa solução milhões de vezes** sem nunca fazê-lo da mesma maneira"

[Christopher Alexander]

- Um padrão encerra o conhecimento de uma pessoa ou de um grupo de pessoas muito experientes em um determinado assunto, de forma que este conhecimento pode ser transmitido para outras pessoas menos experientes.
- Por que adotar padrões para software?
 - ✓ Problemas no desenvolvimento de software se repetem.
 - ✓ Bons desenvolvedores adotam boas soluções.
 - ✓ Por que não reutilizar as boas soluções em outros projetos?

- O que são padrões de software?
 - ✓ A descrição de um problema que ocorre com frequência e;
 - ✓ A base de uma solução para este problema com um nome.
- Padrões de projeto são formas **não triviais** de se resolver problemas de projeto que foram colecionados e documentados ao longo do tempo.
- Padrões de projeto representam a reutilização de ideias... não de código!

- Existem diversos tipos de padrões na área de software:
 - ✓ Padrões de análise
 - ✓ Padrões de testes
 - ✓ Padrões de negócio
 - ✓ Padrões arquiteturais
 - ✓ Padrões de design ou Padrões de projeto (*Design Patterns*)

➤ Exemplos:

Classificação	Título	Autores
Análise OO	Analysis Patterns: Reusable Object Models	Martin Fowler
Arquitetura	Pattern-Oriented Software Architecture: A System of Patterns	Buschmann et al.
Projeto	Design Patterns: Elements of Reusable Object-Oriented Software	Gamma et al.
	Anti-Patterns: Refactoring Software, Architectures, and Projects in Crisis	William J. Brown et al.
	Design Patterns Java™ Workbook	Steven John Metsker

➤ Todo padrão deve incluir:

✓ Nome

- Deve ser sugestivo/representativo do problema ou solução.
- Apenas pelo nome deve-se entender a "natureza" do padrão.

✓ Problema

- Define o problema que se pretende resolver com a utilização do padrão.

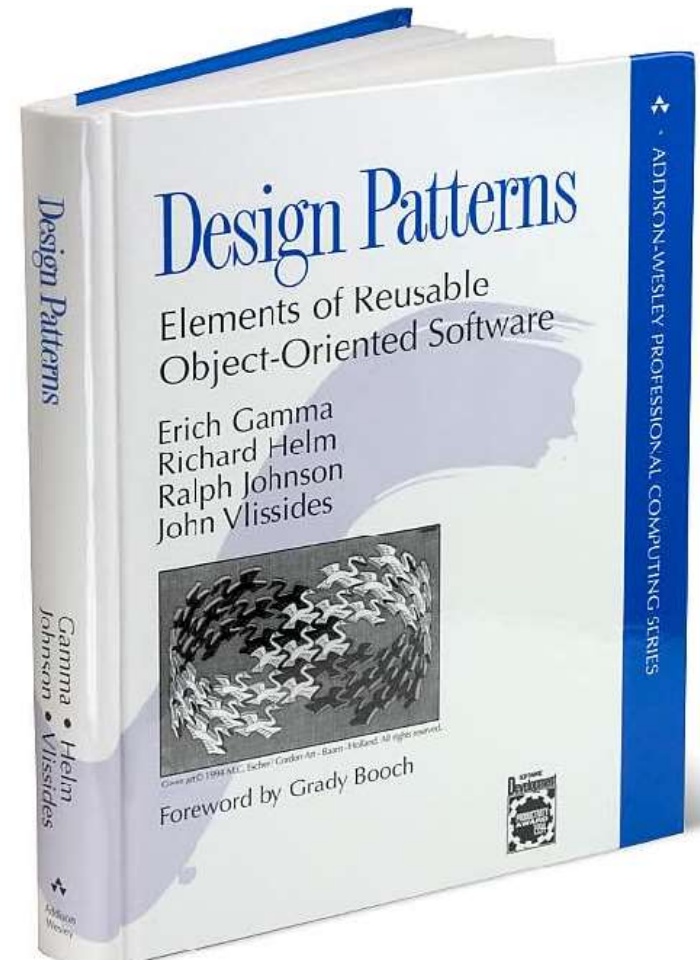
✓ Solução

- Descreve os elementos que compõem a solução, suas responsabilidades e colaborações. Pode adotar modelos conceituais ou diagramas de classes ou, ainda, diagramas de interação.
- A solução **não descreve** uma implementação concreta, apenas um modelo genérico que pode ser reutilizado e adaptado.

✓ Consequências

- Vantagens e desvantagens do uso do padrão.

- Em 1995, Erich Gamma, Richard Helm, Ralph Johnson e John Vlissides escreveram o livro *Design Patterns - Elements of Reusable Object-Oriented Software*.
- Os padrões apresentados no livro surgiram a partir da documentação de soluções que obtiveram sucesso em projetos com C++ e SmallTalk.
- Eles fizeram tanto sucesso que ficaram conhecidos como *Gang of Four* (GoF) e os padrões propostos passaram a ser chamados **Padrões GoF**.



- São 23 padrões divididos em 3 propósitos e 2 escopos.
- Os 3 propósitos são:
 - ✓ **Criação**: tratam da inicialização e configuração de objetos;
 - ✓ **Estrutura**: tratam do desacoplamento entre a interface da classe e a implementação da classe;
 - ✓ **Comportamento**: tratam da interação (comunicação) entre classes/objetos e a distribuição de responsabilidades.

- São 23 padrões divididos em 3 propósitos e 2 escopos:

		Propósito		
		1. Criação	2. Estrutura	3. Comportamento
Escopo	Classe	Factory Method	Class Adapter	Interpreter Template Method
	Objeto	Abstract Factory Builder Prototype Singleton	Object Adapter Bridge Composite Decorator Facade Flyweight Proxy	Chain of Responsibility Command Iterator Mediator Memento Observer State Strategy Visitor

- Nesse quadro existem 24 padrões porque *Class Adapter* e *Object Adapter* contam como um só padrão chamado *Adapter*.

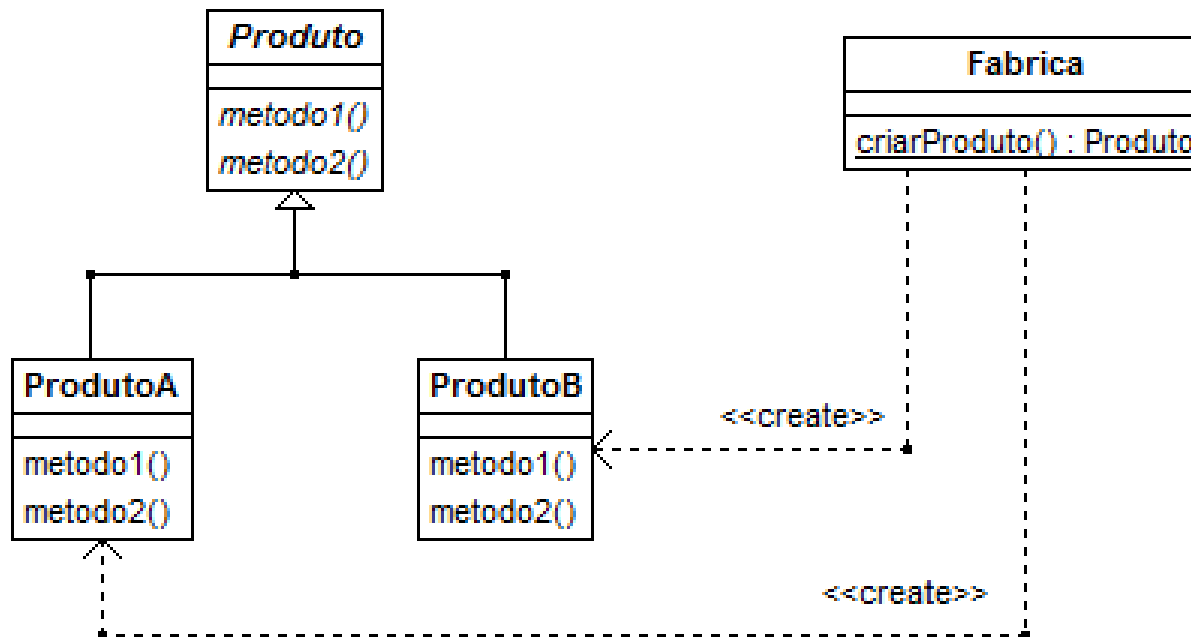
➤ Padrões Criacionais:

		Propósito		
		1. Criação	2. Estrutura	3. Comportamento
Escopo	Classe	Factory Method ←	Class Adapter	Interpreter Template Method
	Objeto	Abstract Factory ← Builder Prototype Singleton ←	Object Adapter Bridge Composite Decorator Facade Flyweight Proxy	Chain of Responsibility Command Iterator Mediator Memento Observer State Strategy Visitor

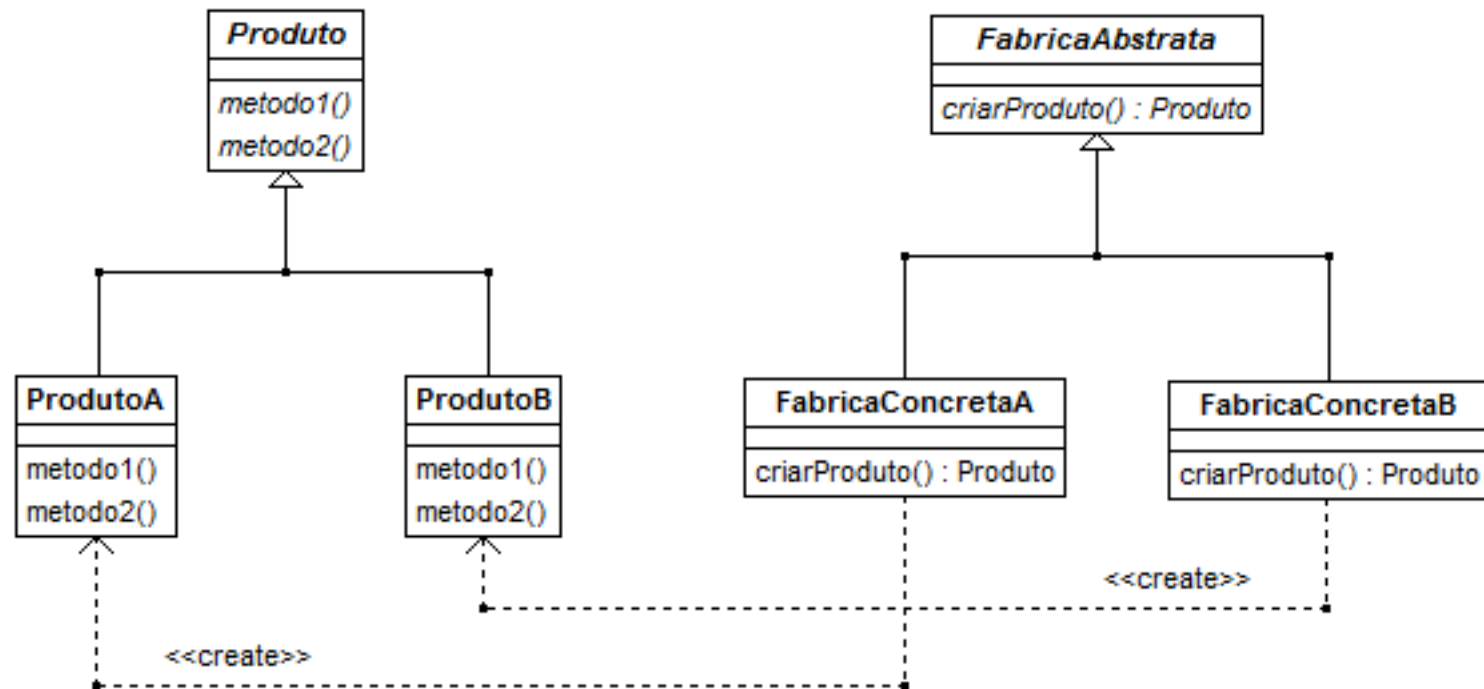
Nome:	Factory Method
Objetivo:	"Definir uma interface para criar um objeto, mas deixar que as subclasses decidam qual classe instanciar. Factory Method permite que uma classe delegue a responsabilidade de instanciação às subclasses." [GoF]
Motivação:	Dada uma hierarquia de classe, temos um ponto do código em que gostaríamos de construir um objeto dessa hierarquia. Entretanto, qual objeto devemos construir depende de alguma condição. Exemplos: • Precisamos criar um objeto que armazena a forma de pagamento de um pedido, que pode ser cartão de crédito, débito, boleto, pay-pal, etc., dependendo do tipo desejado pelo cliente. • Precisamos criar um objeto que representa um protocolo de atendimento ao cliente de uma operadora de TV, que pode ser uma reclamação, uma dúvida, uma solicitação de visita técnica, etc.

➤ Existem duas soluções possíveis:

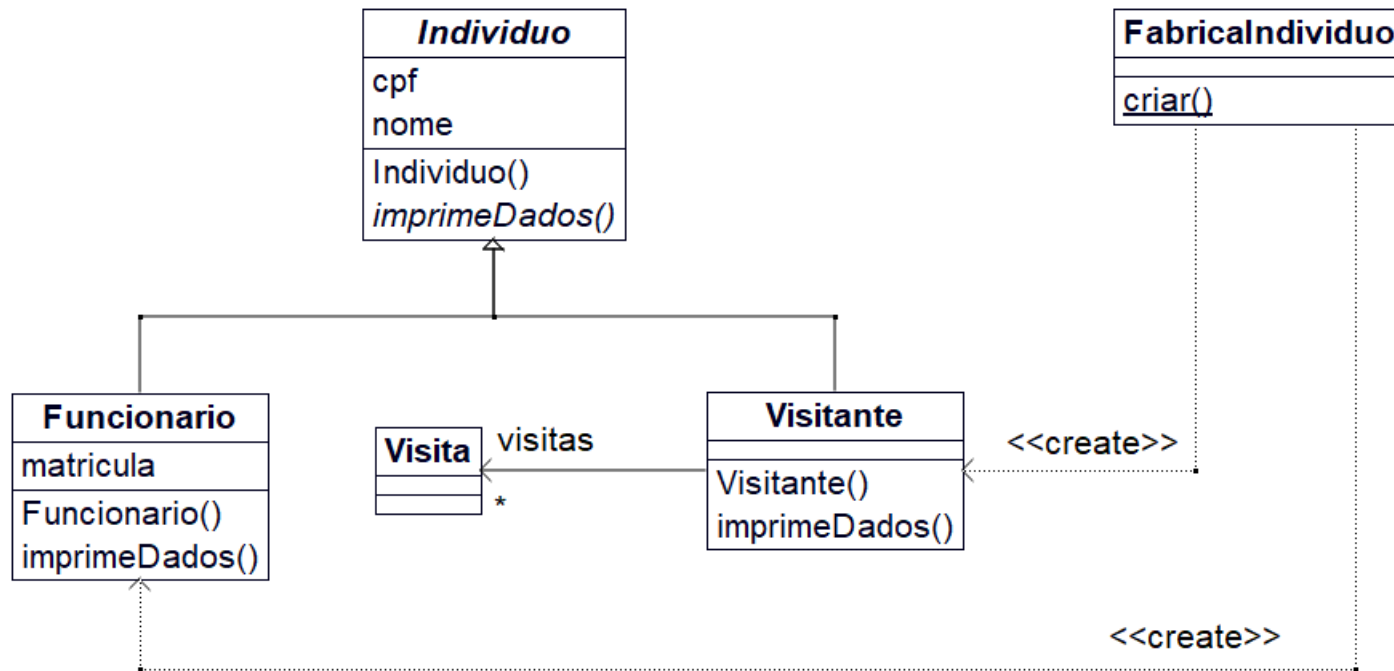
1. Fábrica é uma **classe concreta** que cria os objetos:



- Existem duas soluções possíveis:
2. Fábrica é uma **classe abstrata** ou **interface** que delega a criação dos objetos às subclasses:



➤ Exemplo usando solução 1:



➤ Exemplo usando solução 1:

```
public abstract class Indivíduo {  
    protected long cpf;  
    protected String nome;  
  
    public Indivíduo(long cpf, String nome) {  
        this.cpf = cpf;  
        this.nome = nome;  
    }  
  
    public abstract void imprimeDados();  
}
```

➤ Exemplo usando solução 1:

```
public class Funcionario extends Indivíduo {  
    private int matricula;  
  
    public Funcionario(long cpf, String nome, int matricula) {  
        super(cpf, nome);  
        this.matricula = matricula;  
    }  
  
    @Override  
    public void imprimeDados() {  
        System.out.printf("Eu sou um funcionário: %d - %s\n", cpf, nome);  
    }  
}
```

➤ Exemplo usando solução 1:

```
public class Visitante extends Indivíduo {
    private ArrayList<Visita> visitas;

    public Visitante(long cpf, String nome) {
        super(cpf, nome);
    }

    @Override
    public void imprimeDados() {
        System.out.printf("Eu sou um visitante: %d - %s\n", cpf, nome);
    }
}
```

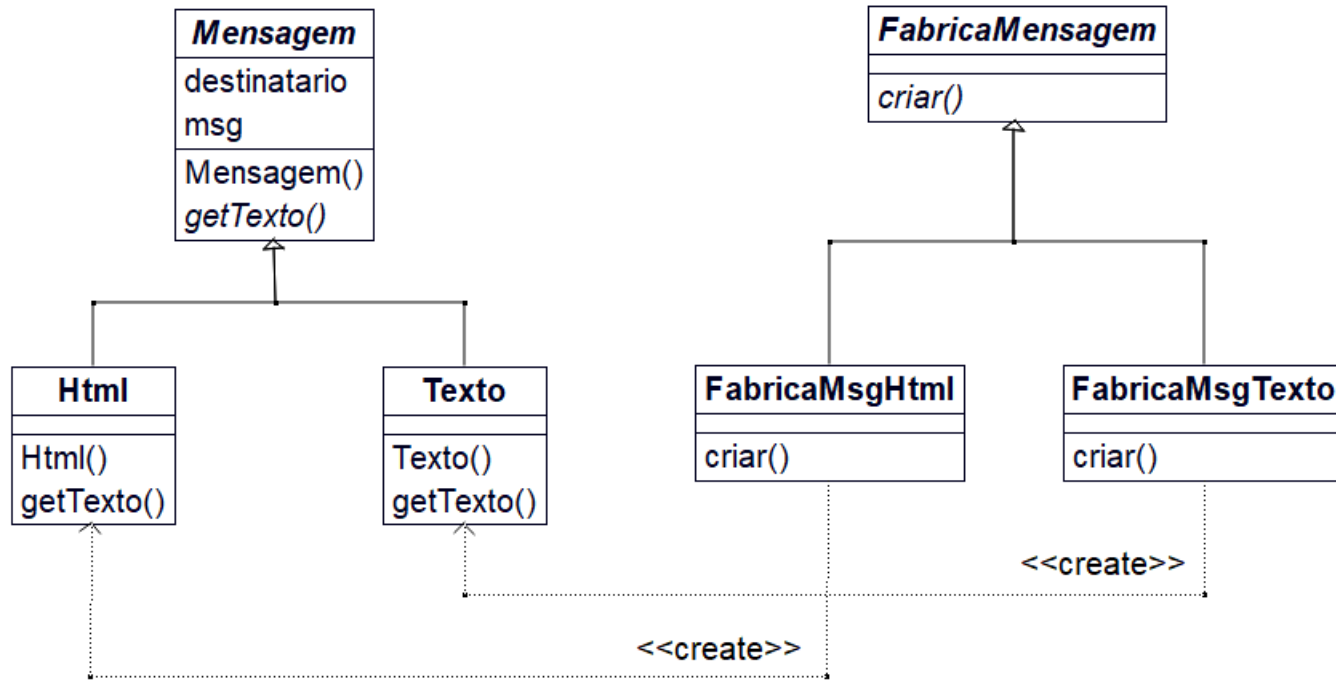
➤ Exemplo usando solução 1:

```
public class FabricaIndividuo {  
  
    public static Individuo criar(long cpf) {  
        // Isso é uma simulação!!!  
        // Em uma aplicação real a fábrica deveria acessar o BD  
        // para recuperar o objeto pelo CPF  
        if (cpf <= 55555555555L)  
            return new Funcionario(cpf, "João da Silva", 123456);  
        else  
            return new Visitante(cpf, "Ana de Almeida");  
    }  
}
```

➤ Exemplo usando solução 1:

```
public class TesteFactoryMethod {  
  
    public static void main(String[] args) {  
        Scanner t = new Scanner(System.in);  
        long cpf;  
        Indivíduo p;  
  
        System.out.print("CPF: ");  
        cpf = t.nextLong();  
  
        p = FabricaIndivíduo.criar(cpf);  
  
        p.imprimeDados();  
    }  
}
```

➤ Exemplo usando solução 2:



➤ Exemplo usando solução 2:

```
public abstract class Mensagem {  
    protected String destinatario;  
    protected String msg;  
  
    public Mensagem(String destinatario, String msg) {  
        this.destinatario = destinatario;  
        this.msg = msg;  
    }  
  
    public abstract String getTexto();  
}
```

➤ Exemplo usando solução 2:

```
public class Texto extends Mensagem {  
  
    public Texto(String destinatario, String msg) {  
        super(destinatario, msg);  
    }  
  
    @Override  
    public String getTexto() {  
        return "Caro Sr(a). " + destinatario + "\n" + msg;  
    }  
}
```


➤ Exemplo usando solução 2:

```
public class Html extends Mensagem {  
  
    public Html(String destinatario, String msg) {  
        super(destinatario, msg);  
    }  
  
    @Override  
    public String getTexto() {  
        return "<html><body>Caro Sr(a). " + destinatario + "<p>" +  
            msg + "</p></body></html>";  
    }  
}
```

➤ Exemplo usando solução 2:

```
public abstract class FabricaMensagem {  
    public abstract Mensagem criar(String destinatario, String msg).  
}
```

```
public class FabricaMsgTexto extends FabricaMensagem {  
  
    @Override  
    public Mensagem criar(String destinatario, String msg) {  
        return new Texto(destinatario, msg);  
    }  
}
```

```
public class FabricaMsgHtml extends FabricaMensagem {  
  
    @Override  
    public Mensagem criar(String destinatario, String msg) {  
        return new Html(destinatario, msg);  
    }  
}
```

➤ Exemplo usando solução 2:

```
public class TesteFactoryMethod {
    public static void main(String[] args) {
        Scanner t = new Scanner(System.in);
        String nome, texto;
        int opcao;
        FabricaMensagem f = null;
        Mensagem msg;

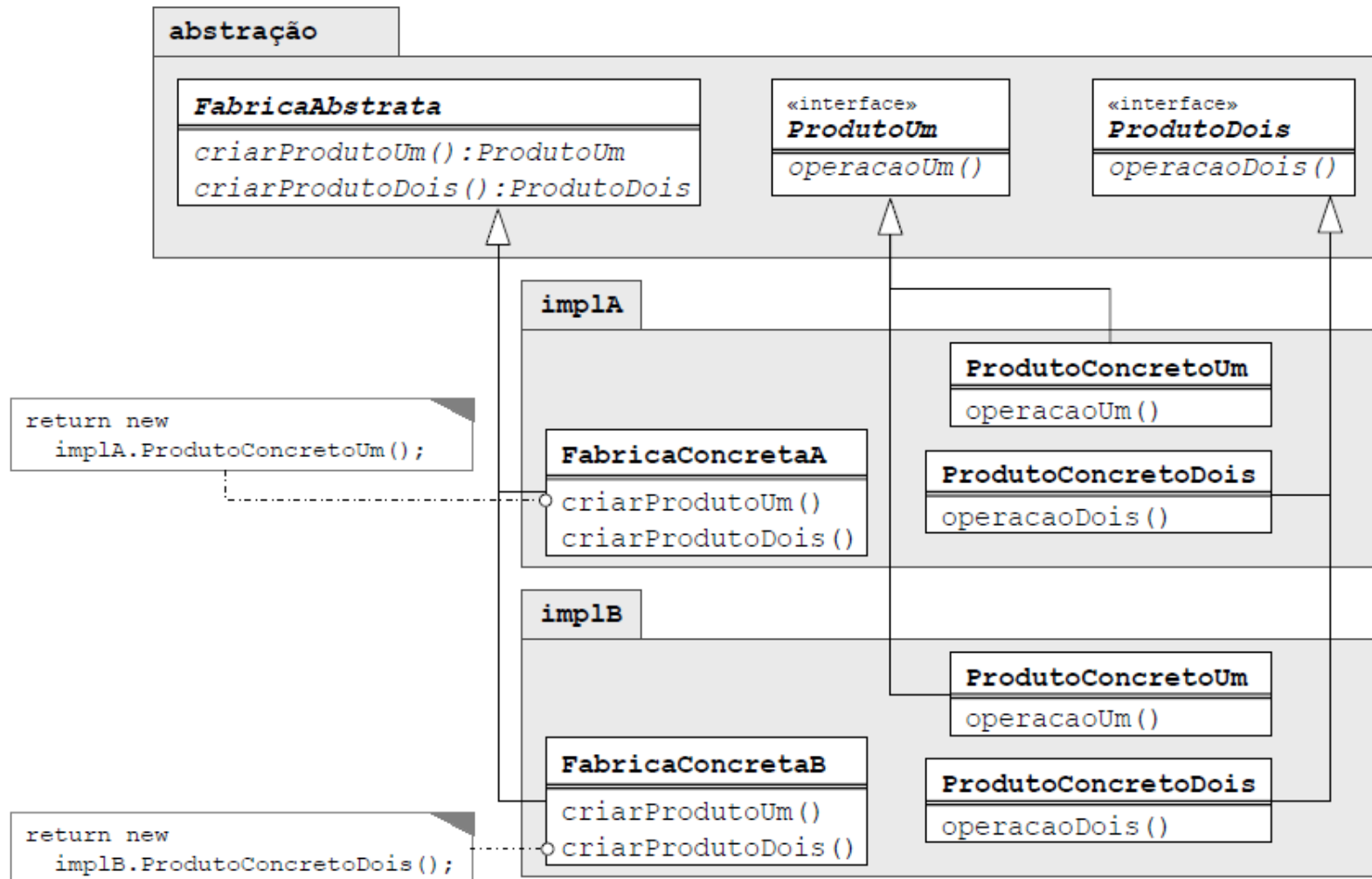
        System.out.print("Destinatario: ");
        nome = t.nextLine();
        System.out.print("Mensagem: ");
        texto = t.nextLine();
        System.out.print("Digite 1-Texto ou 2-HTML: ");
        opcao = t.nextInt();

        if (opcao == 1)
            f = new FabricaMsgTexto();
        else if (opcao == 2)
            f = new FabricaMsgHtml();

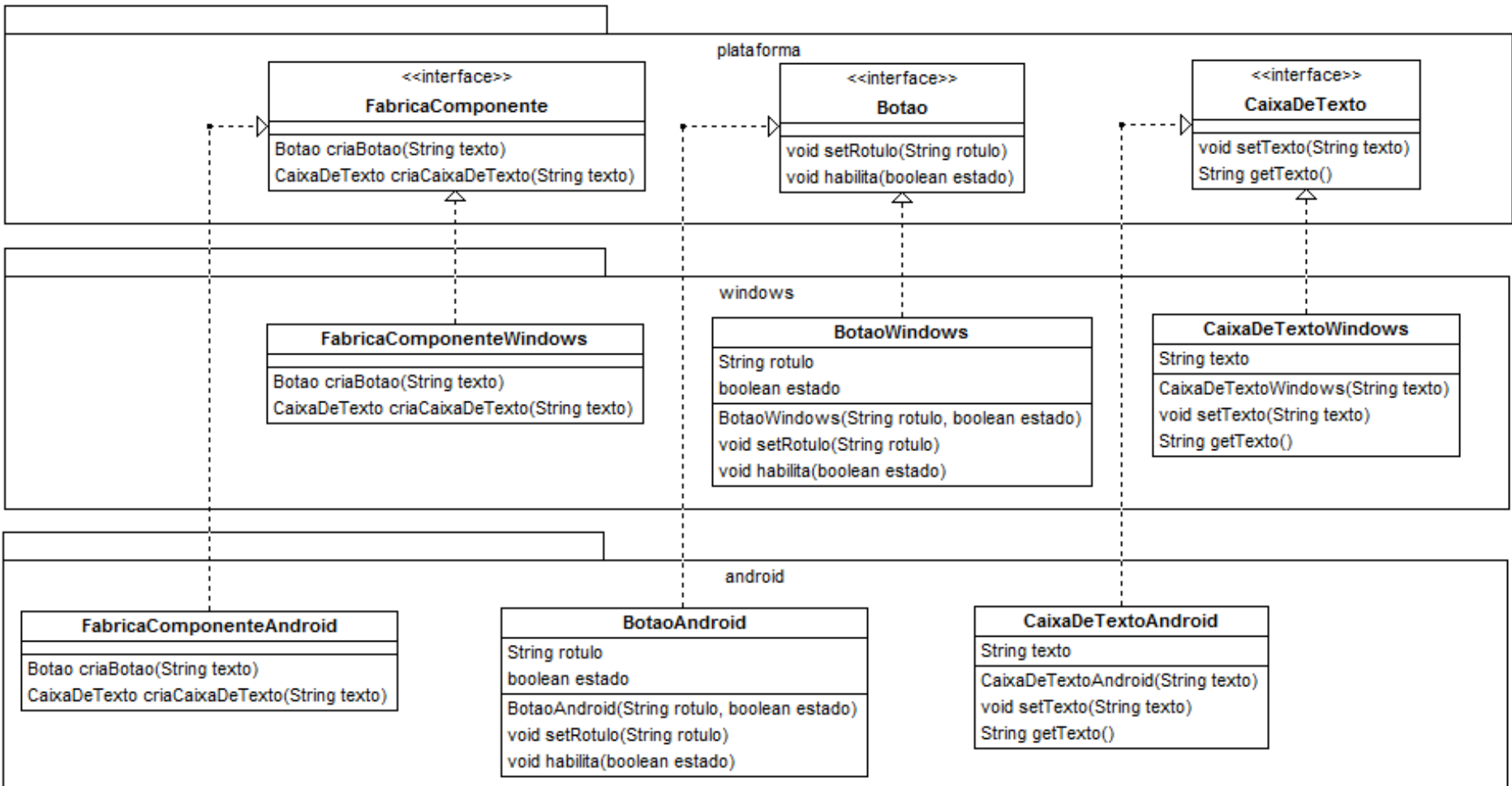
        if (f != null) {
            msg = f.criar(nome, texto);
            System.out.println(msg.getTexto());
        }
    }
}
```

Nome:	Abstract Factory
Objetivo:	"Prover uma interface para criar famílias de objetos relacionados ou dependentes sem especificar suas classes concretas." [GoF]
Motivação:	Existem situações em que precisamos criar famílias de objetos dependendo de determinada condição. Nesse caso, podemos criar fábricas que, por sua vez, vão criar os objetos desejados. Exemplo: • Criar uma interface gráfica com botões e caixas de texto que funcione tanto no Windows quanto no Android. Nesse caso, teremos uma fábrica para os botões e caixas de texto do Windows e outra fábrica para os botões e caixas de texto do Android. A classe cliente que vai usar os botões e caixas de texto não fica ciente que fábrica foi usada.

➤ Solução:



➤ Exemplo: interface gráfica multiplataforma



➤ Exemplo:

```
public interface Botao {  
    public void setRotulo(String rotulo);  
    public void habilita(boolean estado);  
}
```

```
public interface CaixaDeTexto {  
    public void setTexto(String texto);  
    public String getTexto();  
}
```

```
public interface FabricaComponente {  
    public Botao criaBotao(String texto);  
    public CaixaDeTexto criaCaixaDeTexto(String texto);  
}
```

➤ Exemplo:

```
public class BotaoWindows implements Botao {  
    private String rotulo;  
    private boolean estado;  
  
    public BotaoWindows(String rotulo, boolean estado) {  
        this.rotulo = rotulo;  
        this.estado = estado;  
        System.out.println("Botão windows criado...");  
    }  
  
    public void setRotulo(String rotulo) {  
        System.out.printf("Rotulo do botão windows: %s\n", rotulo);  
    }  
  
    public void habilita(boolean estado) {  
        System.out.printf("Botão windows habilitado: %s\n", estado);  
    }  
}
```


➤ Exemplo:

```
public class CaixaDeTextoWindows implements CaixaDeTexto {
    private String texto;

    public CaixaDeTextoWindows(String texto) {
        this.texto = texto;
        System.out.println("Caixa de texto windows criada...");
    }

    public void setTexto(String texto) {
        this.texto = texto;
        System.out.printf("Caixa de texto windows: %s\n", texto);
    }

    public String getTexto() {
        return texto;
    }
}
```

➤ Exemplo:

```
public class BotaoAndroid implements Botao {
    private String rotulo;
    private boolean estado;

    public BotaoAndroid(String rotulo, boolean estado) {
        this.rotulo = rotulo;
        this.estado = estado;
        System.out.println("Botão android criado...");
    }

    public void setRotulo(String rotulo) {
        System.out.printf("Rotulo do botão android: %s\n", rotulo);
    }

    public void habilita(boolean estado) {
        System.out.printf("Botão android habilitado: %s\n", estado);
    }
}
```

➤ Exemplo:

```
public class CaixaDeTextoAndroid implements CaixaDeTexto {
    private String texto;

    public CaixaDeTextoAndroid(String texto) {
        this.texto = texto;
        System.out.println("Caixa de texto android criada...");
    }

    public void setTexto(String texto) {
        this.texto = texto;
        System.out.printf("Caixa de texto android: %s\n", texto);
    }

    public String getTexto() {
        return texto;
    }
}
```

➤ Exemplo:

```
public class FabricaComponenteWindows implements FabricaComponente {  
    public Botao criaBotao(String texto) {  
        return new BotaoWindows(texto, true);  
    }  
  
    public CaixaDeTexto criaCaixaDeTexto(String texto) {  
        return new CaixaDeTextoWindows(texto);  
    }  
}
```

```
public class FabricaComponenteAndroid implements FabricaComponente {  
    public Botao criaBotao(String texto) {  
        return new BotaoAndroid(texto, true);  
    }  
  
    public CaixaDeTexto criaCaixaDeTexto(String texto) {  
        return new CaixaDeTextoAndroid(texto);  
    }  
}
```

➤ Exemplo:

```
public class ConstrutorInterface {  
    public void construir(FabricaComponente fabrica) {  
        Botao botao = fabrica.criaBotao("Ok");  
        CaixaDeTexto cx = fabrica.criaCaixaDeTexto("texto");  
  
        botao.habilita(true);  
        cx.setTexto("Digite algo aqui");  
    }  
}
```

➤ Exemplo:

```
public class TesteAbstractFactory {
    public static void main(String[] args) {
        Scanner t = new Scanner(System.in);
        ConstrutorInterface construtor = new ConstrutorInterface();
        FabricaComponente fabrica;

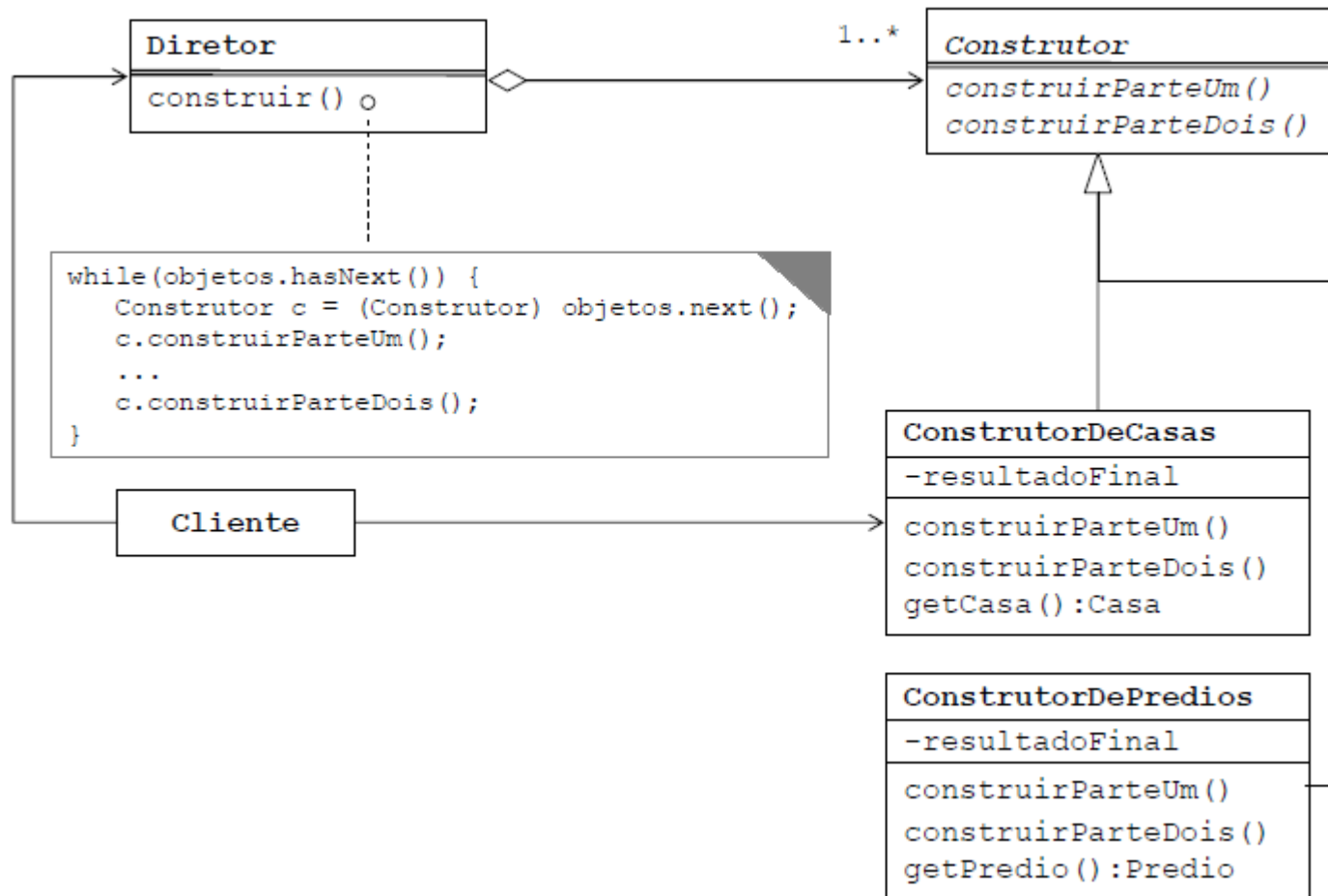
        System.out.print("1-Windows\n2-Android\nOpção: ");
        int op = t.nextInt();

        if (op == 1)
            fabrica = new FabricaComponenteWindows();
        else if (op == 2)
            fabrica = new FabricaComponenteAndroid();
        else
            return;

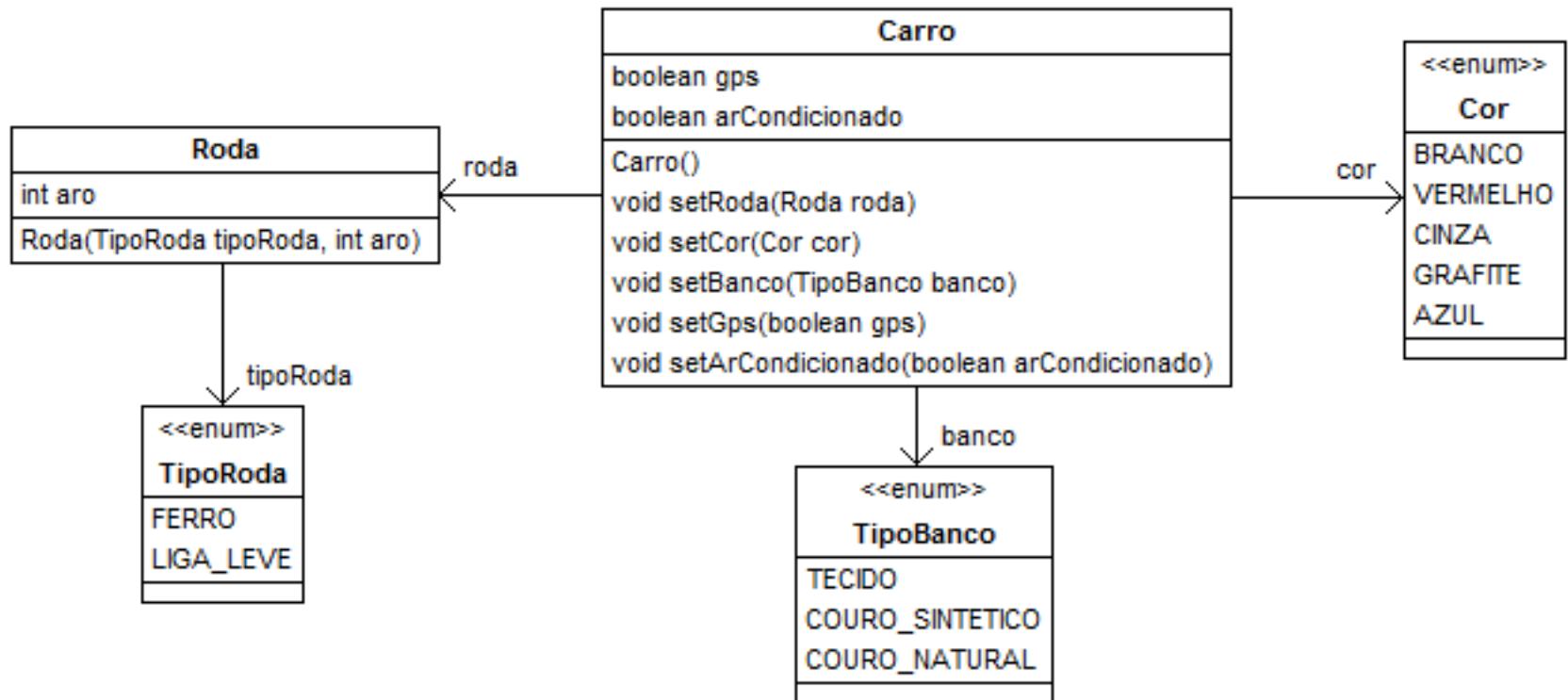
        construtor.construir(fabrica);
    }
}
```

Nome:	Builder
Objetivo:	"Separar a construção de um objeto complexo de sua representação para que o mesmo processo de construção possa criar representações diferentes" [GoF]
Motivação:	Em algumas situações necessitamos definir diversos algoritmos diferentes para criação de um objeto complexo e, a mesmo tempo, gostaríamos de desacoplar esses algoritmos do cliente que demanda a criação desse objeto. Além disso, gostaríamos de ter controle sobre a criação ou não de partes desse objeto complexo.

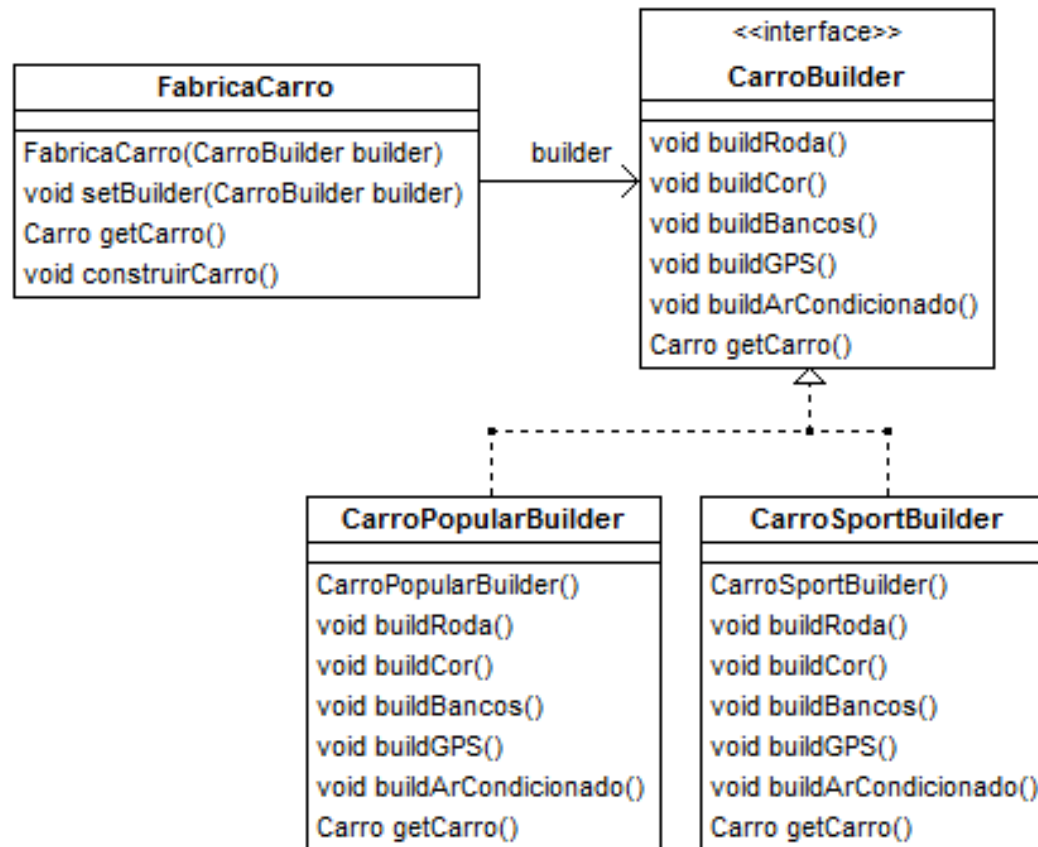
➤ Solução:



- Exemplo 1: ter a possibilidade de construir diversos tipos de carro, combinando suas partes.



- Exemplo 1: aplicando o padrão Builder para definir dois tipos de carro: esporte e popular.



➤ Exemplo 1:

```
public interface CarroBuilder {  
  
    public void buildRoda();  
    public void buildCor();  
    public void buildBancos();  
    public void buildGPS();  
    public void buildArCondicionado();  
  
    public Carro getCarro();  
}
```

➤ Exemplo 1:

```
public class FabricaCarro {
    private CarroBuilder builder;

    public FabricaCarro(CarroBuilder builder) {
        this.builder = builder;
    }

    public void setBuilder(CarroBuilder builder) {
        this.builder = builder;
    }

    public Carro getCarro() {
        return builder.getCarro();
    }

    public void construirCarro() {
        builder.buildRoda();
        builder.buildCor();
        builder.buildBancos();
        builder.buildGPS();
        builder.buildArCondicionado();
    }
}
```

➤ Exemplo 1:

```
public class CarroSportBuilder implements CarroBuilder {  
  
    public Carro carro;  
  
    public CarroSportBuilder() { carro = new Carro(); }  
  
    public void buildRoda() { carro.setRoda(new Roda(TipoRoda.LIGA_LEVE, 17)); }  
  
    public void buildCor() { carro.setCor(Cor.AZUL); }  
  
    public void buildBancos() { carro.setBanco(TipoBanco.COURO_NATURAL); }  
  
    public void buildGPS() { carro.setGps(true); }  
  
    public void buildArCondicionado() { carro.setArCondicionado(true); }  
  
    public Carro getCarro() { return carro; }  
}
```

➤ Exemplo 1:

```
public class CarroPopularBuilder implements CarroBuilder {  
  
    public Carro carro;  
  
    public CarroSportBuilder() { carro = new Carro(); }  
  
    public void buildRoda() { carro.setRoda(new Roda(TipoRoda.FERRO, 15)); }  
  
    public void buildCor() { carro.setCor(Cor.BRANCA); }  
  
    public void buildBancos() { carro.setBanco(TipoBanco.TECIDO); }  
  
    public void buildGPS() { carro.setGps(false); }  
  
    public void buildArCondicionado() { carro.setArCondicionado(false); }  
  
    public Carro getCarro() { return carro; }  
}
```

➤ Exemplo 1:

```
public class Carro {
    private Roda roda;
    private Cor cor;
    private TipoBanco banco;
    private boolean gps;
    private boolean arCondicionado;

    public Carro() {}

    public void setRoda(Roda roda) { this.roda = roda; }
    public void setCor(Cor cor) { this.cor = cor; }
    public void setBanco(TipoBanco banco) { this.banco = banco; }
    public void setGps(boolean gps) { this.gps = gps; }
    public void setArCondicionado(boolean arCondicionado) {
        this.arCondicionado = arCondicionado;
    }

    public String toString() {
        return String.format("Roda:%s Cor:%s Banco:%s GPS:%s Ar:%s",
            roda, cor, banco, gps, arCondicionado);
    }
}
```

➤ Exemplo 1:

```
public class Roda {  
    private TipoRoda tipoRoda ;  
    private int aro;  
  
    public Roda(TipoRoda tipoRoda, int aro) {  
        this.tipoRoda = tipoRoda;  
        this.aro = aro;  
    }  
  
    public String toString() {  
        return String.format("(%s  Aro:%s)", tipoRoda, aro);  
    }  
}
```

```
public enum Cor { BRANCO, VERMELHO, CINZA, GRAFITE, AZUL }
```

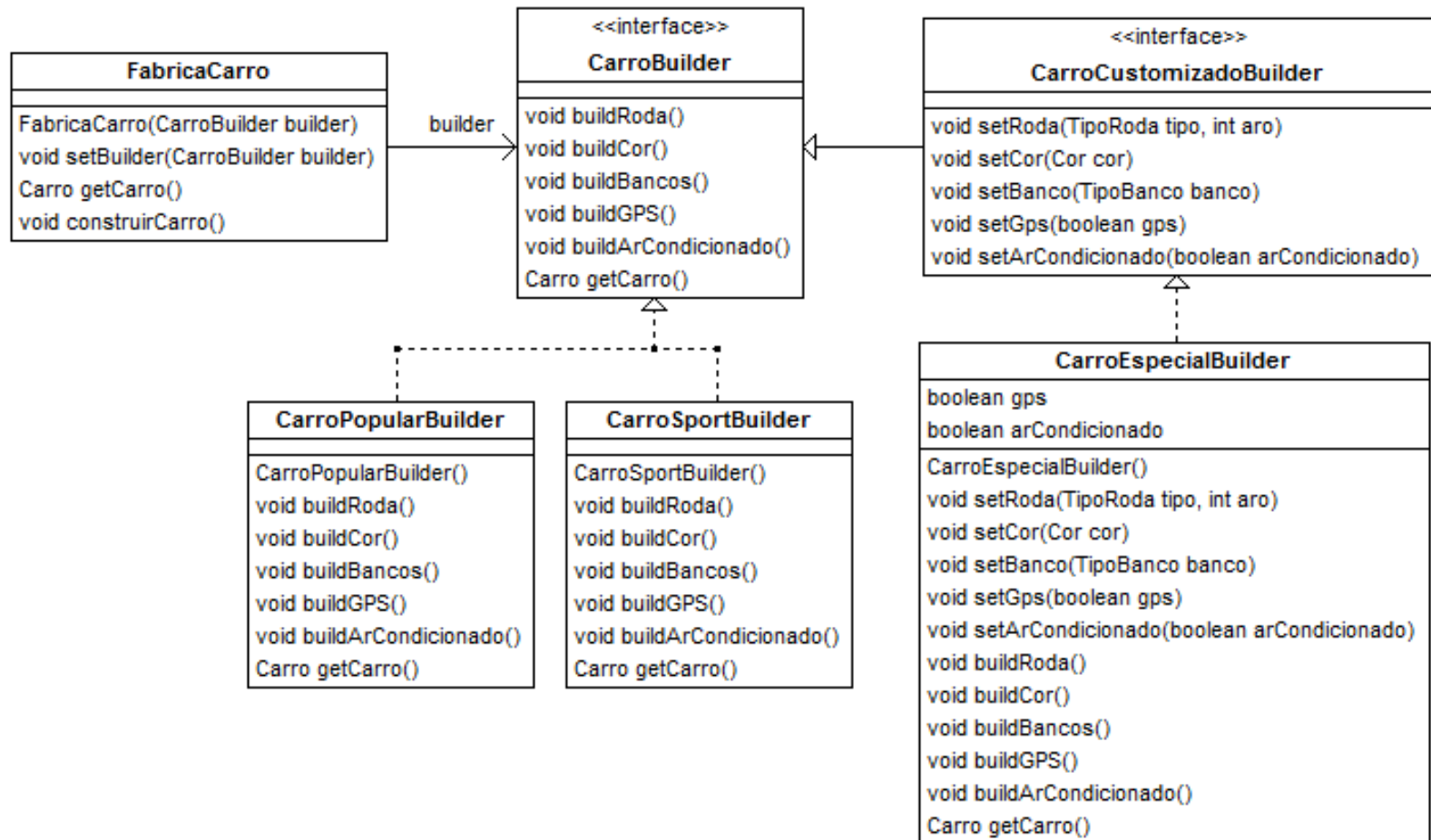
```
public enum TipoBanco {TECIDO, COURO_SINTETICO, COURO_NATURAL}
```

```
public enum TipoRoda {FERRO, LIGA_LEVE}
```


➤ Exemplo 1:

```
public class TesteBuilder {  
    public static void main(String[] args) {  
        Carro c1, c2;  
  
        CarroBuilder sport = new CarroSportBuilder();  
        CarroBuilder pop    = new CarroPopularBuilder();  
  
        FabricaCarro fabrica = new FabricaCarro(sport);  
  
        fabrica.construirCarro();  
  
        c1 = fabrica.getCarro();  
  
        fabrica.setBuilder(pop);  
        fabrica.construirCarro();  
  
        c2 = fabrica.getCarro();  
  
        System.out.println("C1 = " + c1);  
        System.out.println("C2 = " + c2);  
    }  
}
```

- Exemplo 2: estendendo a versão anterior para permitir a **customização** das partes do carro.



➤ Exemplo 2:

```
public interface CarroCustomizadoBuilder extends CarroBuilder {  
  
    public void setRoda(TipoRoda tipo, int aro);  
    public void setCor(Cor cor);  
    public void setBanco(TipoBanco banco);  
    public void setGps(boolean gps);  
    public void setArCondicionado(boolean arCondicionado);  
  
}
```

➤ Exemplo 2:

```
public class CarroEspecialBuilder implements CarroCustomizadoBuilder {  
  
    public Carro carro;  
    private Roda roda;  
    private Cor cor;  
    private TipoBanco banco;  
    private boolean gps;  
    private boolean arCondicionado;  
  
    public CarroEspecialBuilder() {  
        carro = new Carro();  
  
        // Valores DEFAULT  
        roda = new Roda(TipoRoda.FERRO, 15);  
        cor = Cor.BRANCO;  
        banco = TipoBanco.TECIDO;  
        gps = false;  
        arCondicionado = false;  
    }  
}
```

➤ Exemplo 2:

```
public void setRoda(TipoRoda tipo, int aro) { this.roda = new Roda(tipo, aro); }

public void setCor(Cor cor) { this.cor = cor; }

public void setBanco(TipoBanco banco) { this.banco = banco; }

public void setGps(boolean gps) { this.gps = gps; }

public void setArCondicionado(boolean ar) { this.arCondicionado = ar; }

public void buildRoda() { carro.setRoda(roda); }

public void buildCor() { carro.setCor(cor); }

public void buildBancos() { carro.setBanco(banco); }

public void buildGPS() { carro.setGps(gps); }

public void buildArCondicionado() { carro.setArCondicionado(arCondicionado); }

public Carro getCarro() { return carro; }

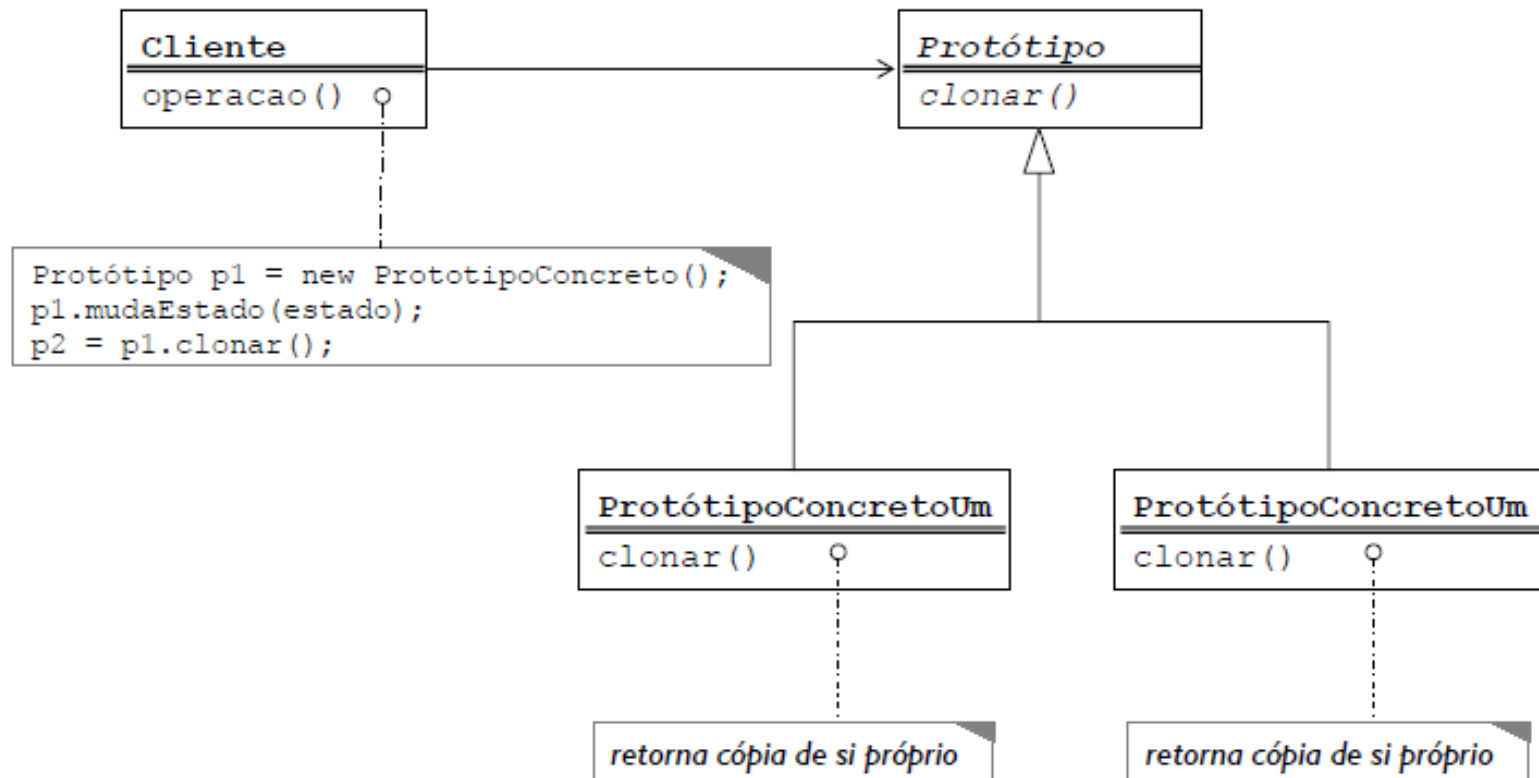
}
```

➤ Exemplo 2:

```
public class TesteBuilder {  
    public static void main(String[] args) {  
        Carro c;  
  
        CarroEspecialBuilder especial = new CarroEspecialBuilder();  
  
        FabricaCarro fabrica = new FabricaCarro(especial);  
  
        especial.setCor(Cor.GRAFITE);  
        especial.setGps(true);  
        especial.setBanco(TipoBanco.COURO_NATURAL);  
        especial.setArCondicionado(true);  
        especial.setRoda(TipoRoda.LIGA_LEVE, 16);  
  
        fabrica.setBuilder(especial);  
        fabrica.construirCarro();  
  
        c = fabrica.getCarro();  
  
        System.out.println("C = " + c);  
    }  
}
```

Nome:	Prototype
Objetivo:	"Especificar os tipos de objetos a serem criados usando uma instância como protótipo e criar novos objetos ao copiar este protótipo" [GoF]
Motivação:	Em algumas situações precisamos criar um objeto a partir de outro já existente, ou seja, clonar um objeto. Em todos os sistemas onde existe a opção de copiar e colar um objeto, podemos aplicar o conceito de clonagem.

➤ Solução:

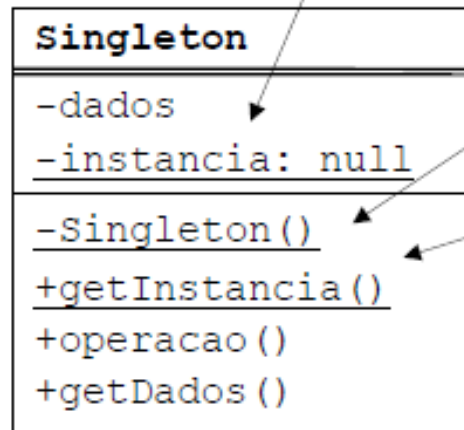


- Algumas linguagens de programação possuem mecanismos pré-definidos para clonagem de objetos. A aplicação do padrão Prototype pode ser realizada através do uso desses mecanismos.
- Em outras linguagens de programação, o próprio programador tem que se encarregar de implementar o método de clonagem. Nesse caso, o importante é definir um padrão para esse método (através de uma interface ou classe abstrata), de forma que ele possa ser chamado de forma consistente.

Nome:	Singleton
Objetivo:	"Garantir que uma classe só tenha uma única instância, e prover um ponto de acesso global a ela" [GoF]
Motivação:	Em diversas situações necessitamos que uma, e apenas uma, instância de uma classe seja criada, independente de quantas requisições de criação possam ser feitas. Exemplos: • Uma única conexão com o banco de dados • Um único arquivo de log de erros • Uma única fila de impressão • Um único arquivo de configuração

➤ Solução:

Objeto com acesso privativo



Construtor privativo (nem subclasses têm acesso)

Ponto de acesso simples, estático e global

➤ Exemplo: duas possibilidades em Java

```
public class ConfigFile {  
  
    private static ConfigFile instancia = null;  
  
    private ConfigFile() {  
        System.out.println("Criando arquivo de configuração...");  
    }  
  
    public static synchronized ConfigFile getInstancia() {  
        if (instancia == null)  
            instancia = new ConfigFile();  
        return instancia;  
    }  
}
```

➤ Exemplo: duas possibilidades em Java

```
public class ConfigFile {  
  
    private static ConfigFile instancia = new ConfigFile();  
  
    private ConfigFile() {  
        System.out.println("Criando arquivo de configuração...");  
    }  
  
    public static ConfigFile getInstancia() {  
        return instancia;  
    }  
}
```

➤ Exemplo:

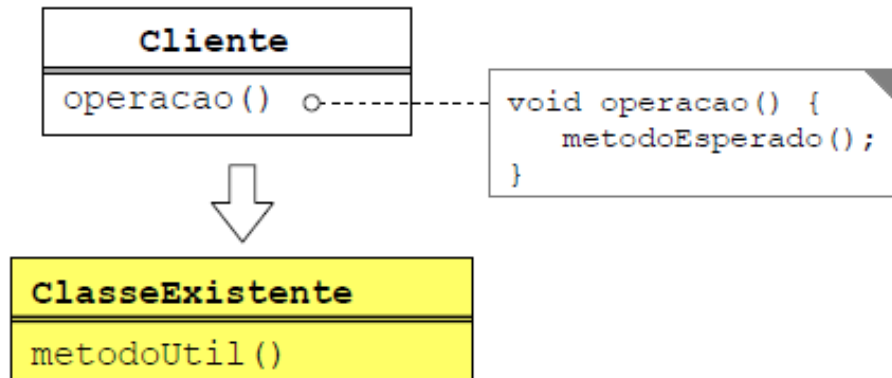
```
public class TesteSingleton {  
    public static void main(String[] args) {  
  
        // O CODIGO ABAIXO APRESENTARIA UM ERRO DE COMPILAÇÃO,  
        // POIS O CONSTRUTOR É PRIVADO !  
        // ConfigFile cfg = new ConfigFile();  
  
        ConfigFile cfg1 = ConfigFile.getInstancia();  
        ConfigFile cfg2 = ConfigFile.getInstancia();  
  
        if (cfg1 == cfg2)  
            System.out.println("Mesmo objeto !");  
        else  
            System.out.println("Objetos diferentes !");  
    }  
}
```

➤ Padrões Estruturais:

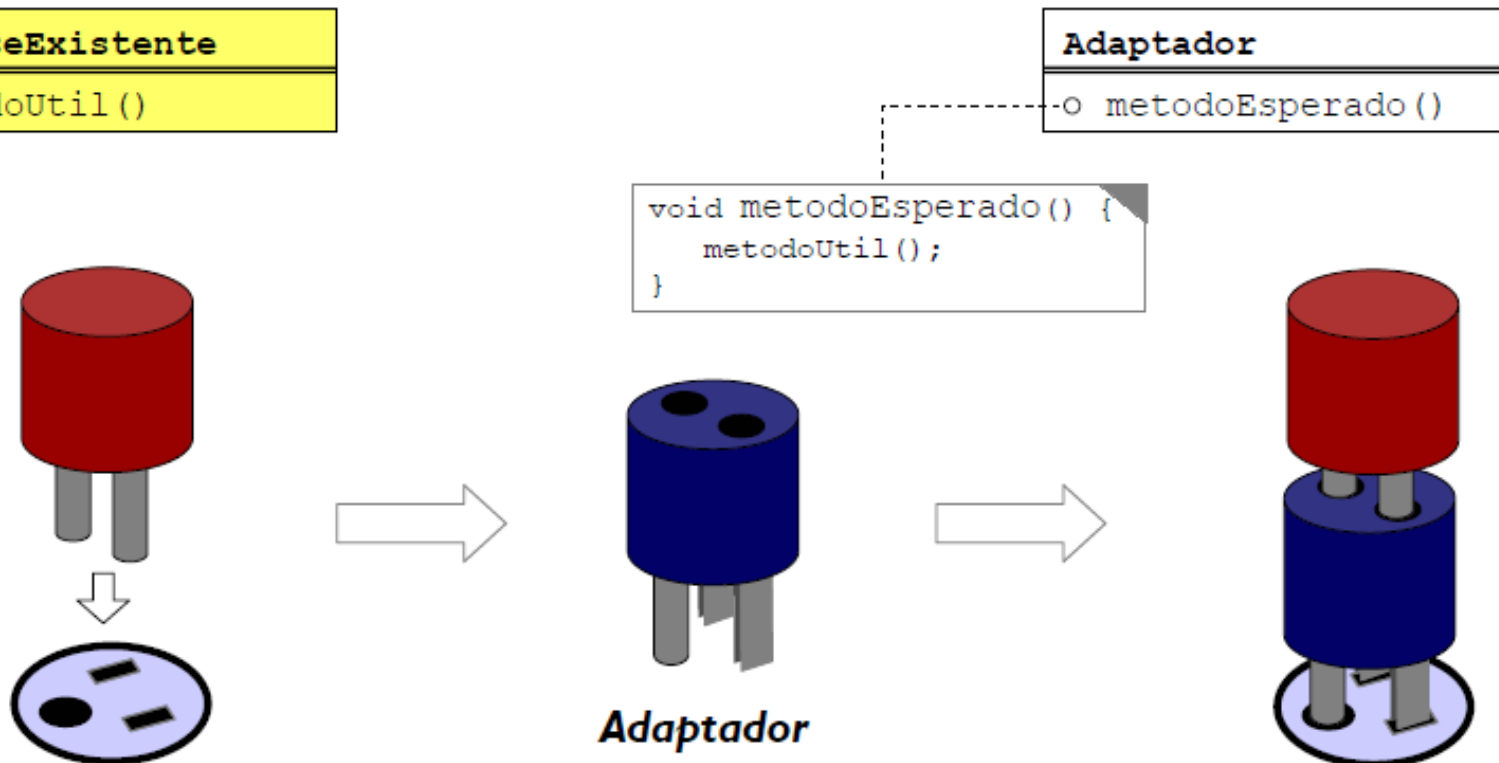
		Propósito		
		1. Criação	2. Estrutura	3. Comportamento
Escopo	Classe	Factory Method	Class Adapter ←	Interpreter Template Method
	Objeto	Abstract Factory Builder Prototype Singleton	Object Adapter ← Bridge Composite Decorator Facade ← Flyweight Proxy	Chain of Responsibility Command Iterator Mediator Memento Observer State Strategy Visitor

Nome:	Adapter
Objetivo:	"Converter a interface de uma classe em outra interface esperada pelos clientes. Adapter permite a comunicação entre classes que não poderiam trabalhar juntas devido à incompatibilidade de suas interfaces" [GoF]
Motivação:	Algumas vezes precisamos que dois objetos se comuniquem, mas a interface de um é incompatível com a interface do outro. Nesse caso precisamos de um adaptador que permita a comunicação desses dois objetos. Exemplos: • Dois objetos que trabalham com medida de área, mas um usa m^2 e outro usa Km^2. • Dois objetos que trabalham com uma lista de números, mas um trabalha com lista encadeada e outro com vetor.

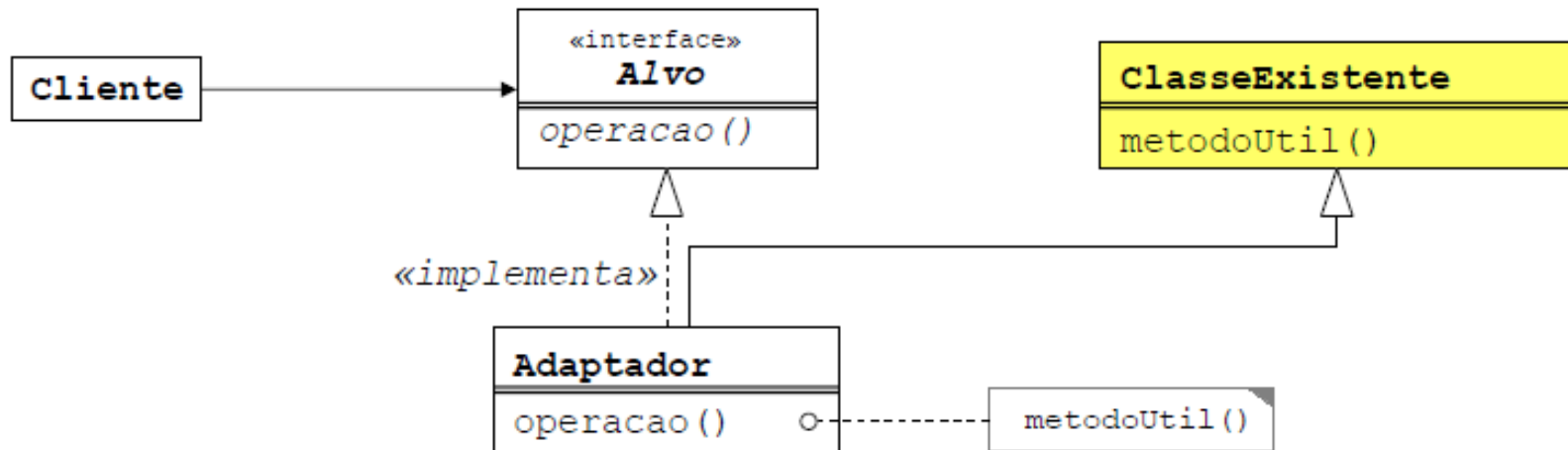
Problema



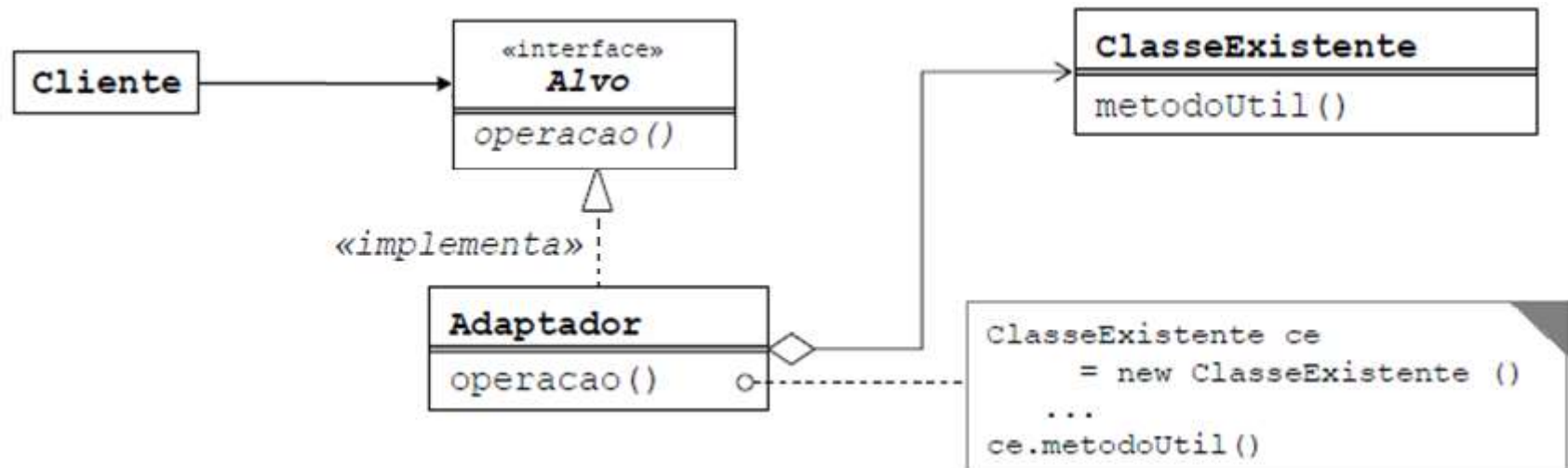
Solução



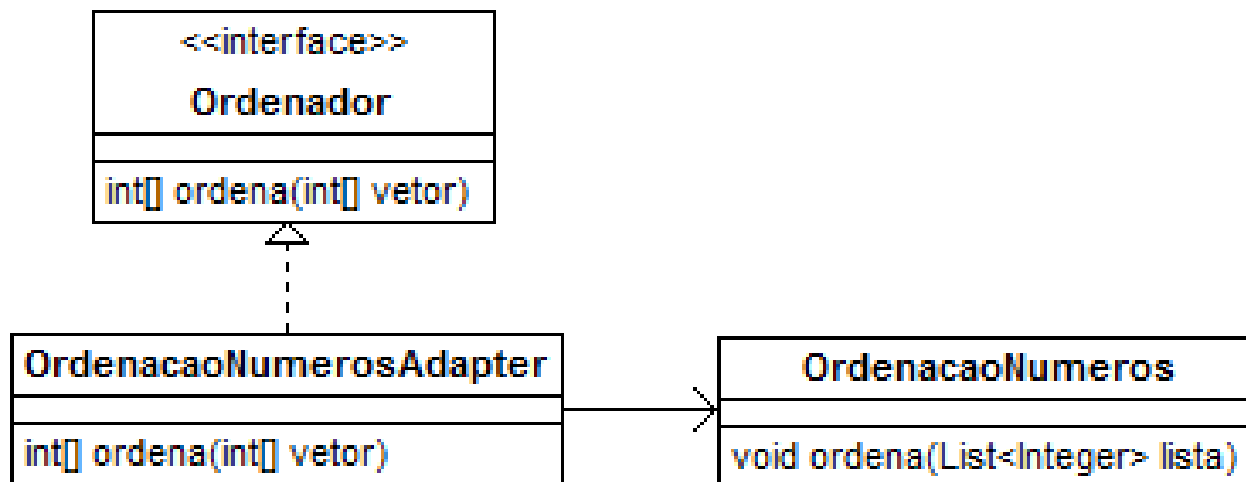
- Duas soluções: Class Adapter
- Adaptador usa herança para acessar o método a ser adaptado (`metodoUtil()` nesse exemplo).



- Duas soluções: **Object Adapter**
- Adaptador usa **delegação** para acessar método a ser adaptado (**metodoUtil()** nesse exemplo)



- Exemplo de Object Adapter: ordenação de um **vetor** de números inteiros em Java. A classe **Collections** possui um método para ordenar **listas**.



➤ Exemplo de Object Adapter:

```
public class OrdenacaoNumeros {  
    public void ordena(List<Integer> lista)  
    {  
        Collections.sort(lista);  
    }  
}
```

```
public interface Ordenador {  
    public int[] ordena(int[] vetor);  
}
```

➤ Exemplo de Object Adapter:

```
public class OrdenacaoNumerosAdapter implements Ordenador {
    public int[] ordena(int[] vetor)
    {
        // Converte int[] para Integer[]
        Integer[] aux = new Integer[vetor.length];
        int i = 0;
        for (int valor : vetor)
            aux[i++] = valor;

        // Converte Integer[] para lista
        List<Integer> lista = Arrays.asList(aux);

        // Ordena
        OrdenacaoNumeros o = new OrdenacaoNumeros();
        o.ordena(lista);

        // Converte lista para int[]
        int[] v = new int[lista.size()];
        for (i = 0; i < lista.size(); i++)
            v[i] = lista.get(i);

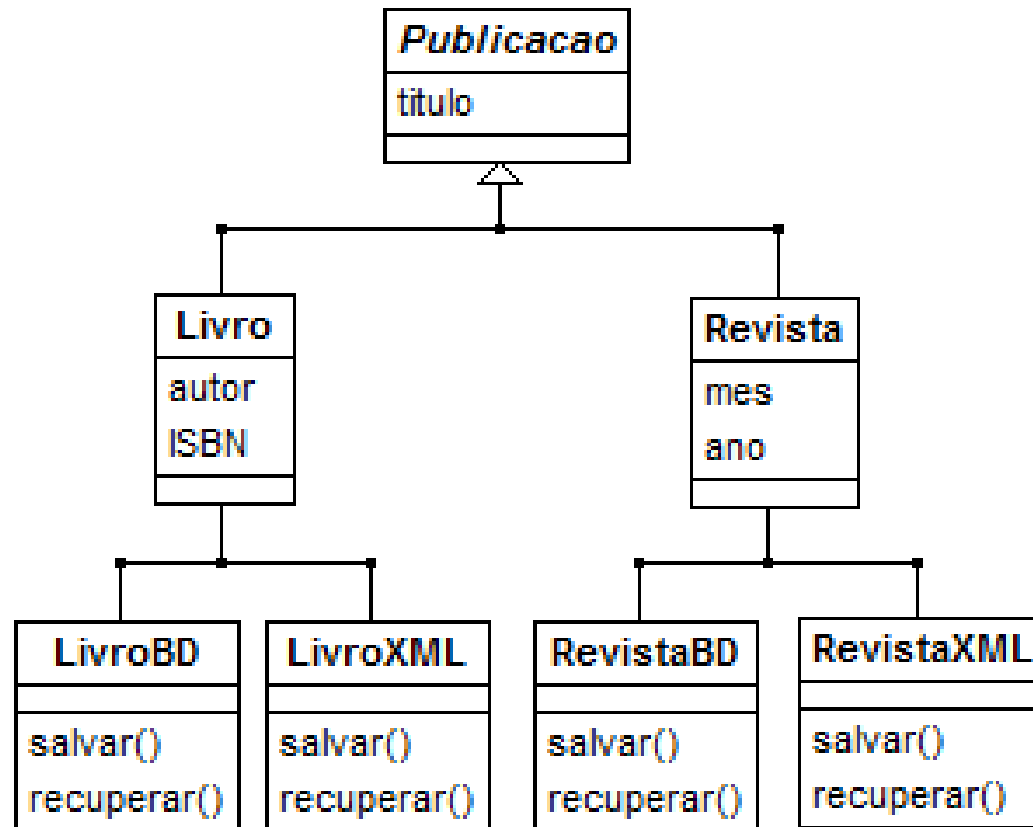
        return v;
    }
}
```

➤ Exemplo de Object Adapter:

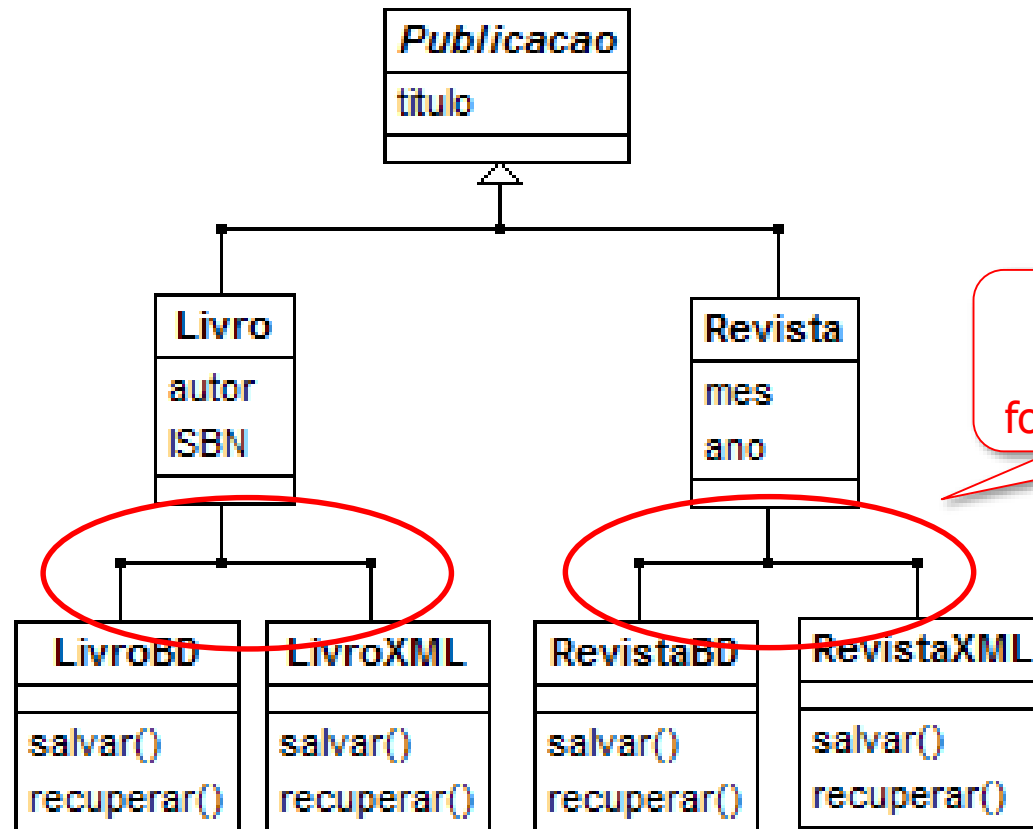
```
public class TesteObjectAdapter {  
    public static void main(String[] args) {  
        int[] numeros = {34, 2, 4, 12, 1, 15, 6, 3, 10};  
  
        Ordenador o = new OrdenacaoNumerosAdapter();  
  
        numeros = o.ordena(numeros);  
  
        for (int v : numeros)  
            System.out.printf("%d ", v);  
    }  
}
```

Nome:	Bridge
Objetivo:	"Desacoplar uma abstração da sua implementação para que os dois possam variar independentemente" [GoF]
Motivação:	Quando uma abstração pode ter várias implementações possíveis, a maneira usual de acomodá-la é usando herança. Entretanto, essa abordagem nem sempre é suficientemente flexível, pois ela leva a um forte acoplamento entre a implementação e a abstração. Precisamos, então, criar uma estrutura hierárquica para a abstração e criar outra estrutura para a implementação, de forma a torná-las independentes, mas mantendo uma ligação entre elas.

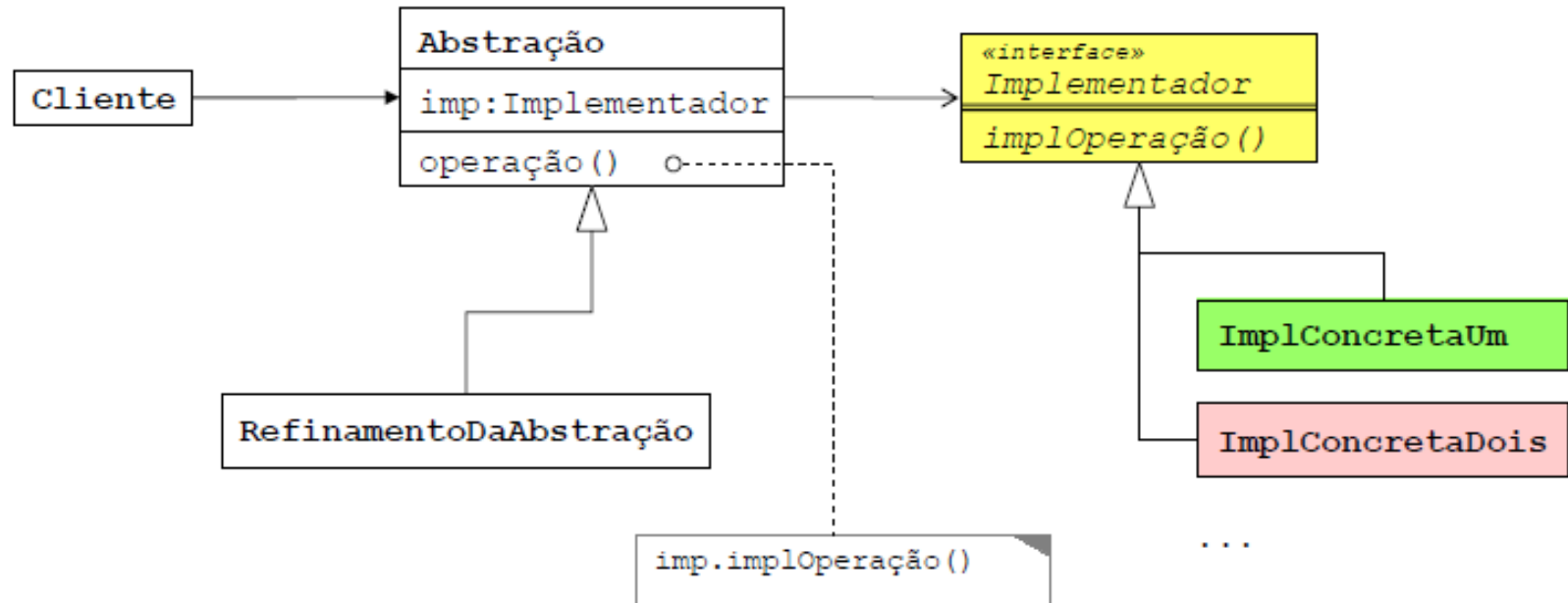
- Problema: Livros e Revistas podem ser salvos ou recuperados de um banco de dados ou de um arquivo XML.



- Problema: Livros e Revistas podem ser salvos ou recuperados de um banco de dados ou de um arquivo XML.



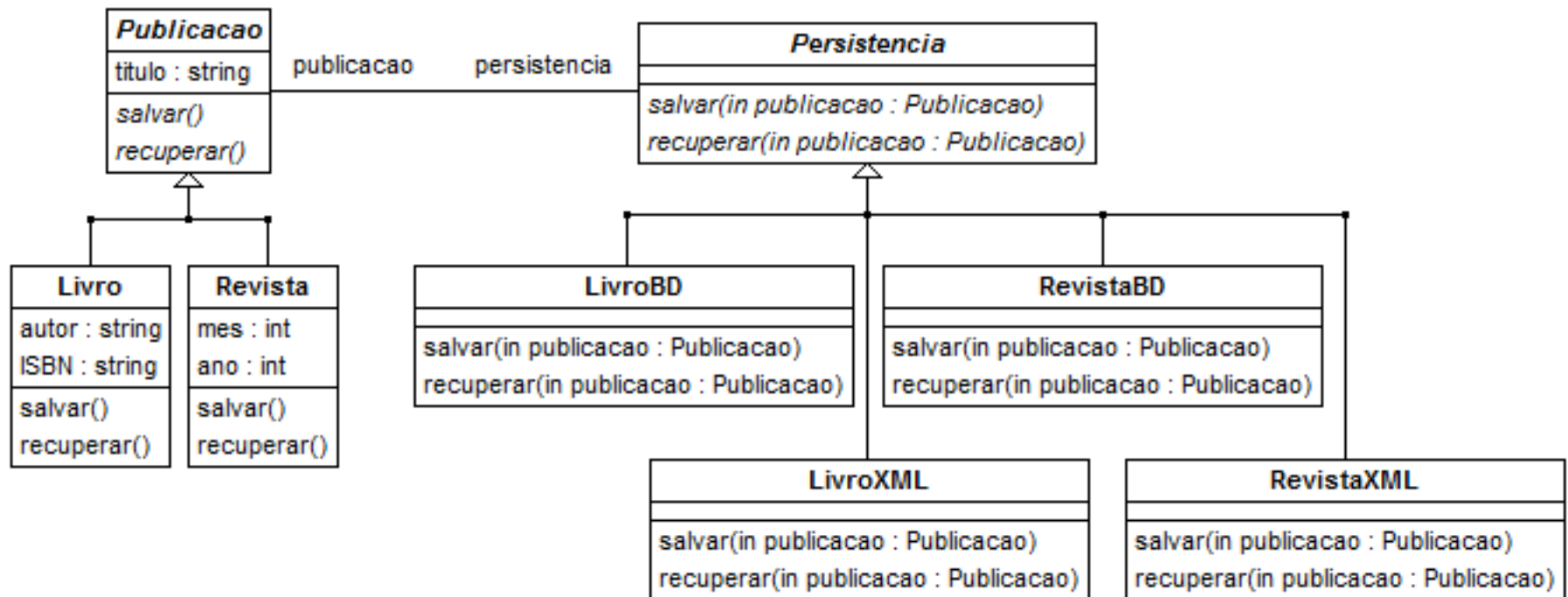
➤ Solução:



A Abstração define operações de nível mais elevado baseadas nas operações primitivas do Implementador

A interface Implementador define operações primitivas

- Exemplo 1: desacoplamento de Livros e Revistas das classes que implementam os mecanismos de persistência.



➤ Exemplo 1:

```
public abstract class Publicacao {  
    protected String titulo;  
    protected Persistencia persistencia;  
  
    public Publicacao(String titulo, Persistencia persistencia) {  
        this.titulo = titulo;  
        this.persistencia = persistencia;  
    }  
  
    public String getTitulo() {  
        return titulo;  
    }  
  
    public abstract void salvar();  
    public abstract void recuperar();  
}
```

➤ Exemplo 1:

```
public class Livro extends Publicacao {
    private String autor;
    private String isbn;

    public Publicacao(String titulo, String autor, String isbn,
                      Persistencia persistencia) {
        super(titulo, persistencia);
        this.autor = autor;
        this.isbn = isbn;
    }

    public int getMes() { return mes; }
    public int getAno() { return ano; }

    public void salvar() { persistencia.salvar(this); }
    public void recuperar() { persistencia.recuperar(this); }
}
```

➤ Exemplo 1:

```
public class Revista extends Publicacao {  
    private int mes;  
    private int ano;  
  
    public Publicacao(String titulo, int mes, int ano,  
                      Persistencia persistencia) {  
        super(titulo, persistencia);  
        this.mes = mes;  
        this.ano = ano;  
    }  
  
    public int getMes() { return mes; }  
    public int getAno() { return ano; }  
  
    public void salvar() { persistencia.salvar(this); }  
    public void recuperar() { persistencia.recuperar(this); }  
}
```

➤ Exemplo 1:

```
public interface Persistencia {  
    public void salvar(Publicacao publicacao);  
    public void recuperar(Publicacao publicacao);  
}
```


➤ Exemplo 1:

```
public class LivroBD implements Persistencia {

    public void salvar(Publicacao publicacao) {
        System.out.printf("Salvando LIVRO no BD...\n%s\n%s\n%s\n",
            ((Livro) publicacao).getTitulo(),
            ((Livro) publicacao).getAutor(),
            ((Livro) publicacao).getIsbn());
    }

    public void recuperar(Publicacao publicacao) {
        System.out.println("Recuperando LIVRO do BD...");
    }
}
```

➤ Exemplo 1:

```
public class LivroXML implements Persistencia {

    public void salvar(Publicacao publicacao) {
        System.out.printf("Salvando LIVRO no XML...\n%s\n%s\n%s\n",
            ((Livro) publicacao).getTitulo(),
            ((Livro) publicacao).getAutor(),
            ((Livro) publicacao).getIsbn());
    }

    public void recuperar(Publicacao publicacao) {
        System.out.println("Recuperando LIVRO do XML...");
    }
}
```

➤ Exemplo 1:

```
public class RevistaBD implements Persistencia {

    public void salvar(Publicacao publicacao) {
        System.out.printf("Salvando REVISTA no BD...\n%s\n%d\n%d\n",
            ((Revista) publicacao).getTitulo(),
            ((Revista) publicacao).getMes(),
            ((Revista) publicacao).getAno());
    }

    public void recuperar(Publicacao publicacao) {
        System.out.println("Recuperando REVISTA do BD...");
    }
}
```

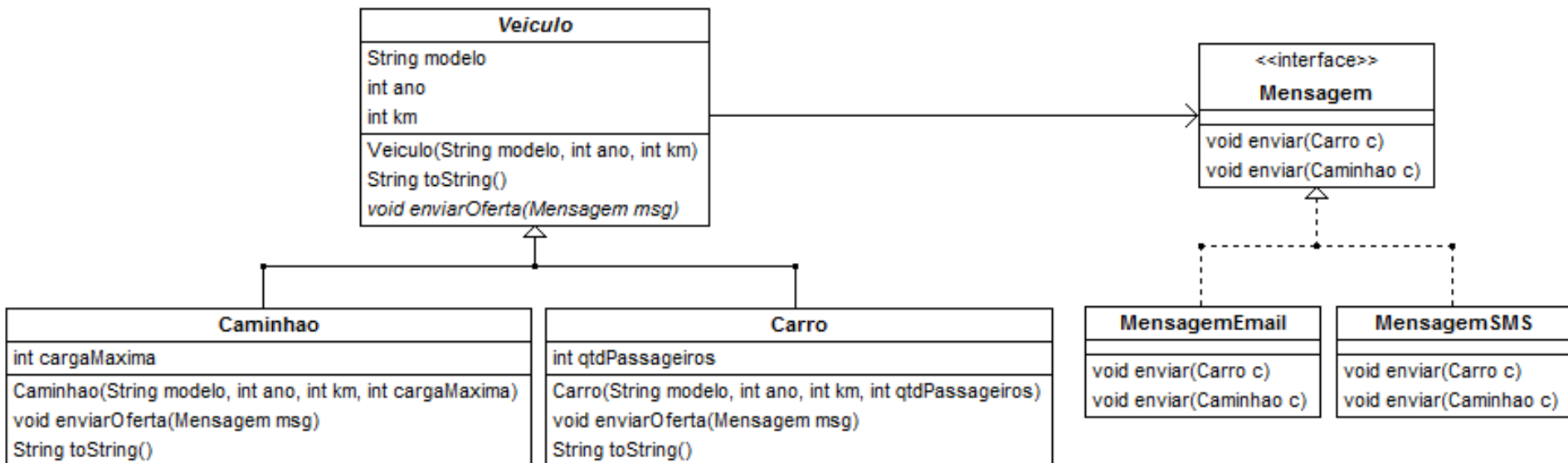
➤ Exemplo 1:

```
public class RevistaXML implements Persistencia {  
  
    public void salvar(Publicacao publicacao) {  
        System.out.printf("Salvando REVISTA no XML...\n%s\n%d\n%d\n",  
                           ((Revista) publicacao).getTitulo(),  
                           ((Revista) publicacao).getMes(),  
                           ((Revista) publicacao).getAno());  
    }  
  
    public void recuperar(Publicacao publicacao) {  
        System.out.println("Recuperando REVISTA do XML...");  
    }  
}
```

➤ Exemplo 1:

```
public class TesteBridge {  
    public static void main(String[] args) {  
        Persistencia bd = new LivroBD();  
        Persistencia xml = new LivroXML();  
  
        Livro lv1 = new Livro ("Design Patterns", "GoF", "234567", bd);  
        Livro lv2 = new Livro ("GRASP", "Larman", "543210", bd);  
        Livro lv3 = new Livro ("Programação OO", "Deitel", "453287", xml);  
  
        lv1.salvar();  
        lv2.salvar();  
        lv3.salvar();  
    }  
}
```

- Exemplo 2: desacoplamento de Caminhão e Carro das classes que enviam ofertas por email e SMS sobre esses veículos.



➤ Exemplo 2:

```
public abstract class Veiculo {
    private String modelo;
    private int ano;
    private int km;

    public Veiculo(String modelo, int ano, int km) {
        this.modelo = modelo;
        this.ano = ano;
        this.km = km;
    }

    public String toString() {
        return String.format("Modelo: %s  Ano: %d  Km: %d",
                               modelo, ano, km);
    }

    public abstract void enviarOferta(Mensagem msg);
}
```

➤ Exemplo 2:

```
public class Carro extends Veiculo {
    private int qtdPassageiros;

    public Carro(String modelo, int ano, int km, int qtdPassageiros) {
        super(modelo, ano, km);
        this.qtdPassageiros = qtdPassageiros;
    }

    public void enviarOferta(Mensagem msg) {
        msg.enviar(this);
    }

    public String toString() {
        return super.toString() + String.format(" Passageiros: %d",
                                                qtdPassageiros);
    }
}
```


➤ Exemplo 2:

```
public class Caminhao extends Veiculo {
    private int cargaMaxima;

    public Caminhao(String modelo, int ano, int km, int cargaMaxima) {
        super(modelo, ano, km);
        this.cargaMaxima = cargaMaxima;
    }

    public void enviarOferta(Mensagem msg) {
        msg.enviar(this);
    }

    public String toString() {
        return super.toString() + String.format(" Carga: %d",
                                                    cargaMaxima);
    }
}
```

➤ Exemplo 2:

```
public interface Mensagem {  
    public void enviar(Carro c);  
    public void enviar(Caminhao c);  
}
```

```
public class MensagemEmail implements Mensagem {  
    public void enviar(Carro c) {  
        System.out.printf("Enviando EMAIL de %s\n", c);  
    }  
  
    public void enviar(Caminhao c) {  
        System.out.printf("Enviando EMAIL de %s\n", c);  
    }  
}
```

➤ Exemplo 2:

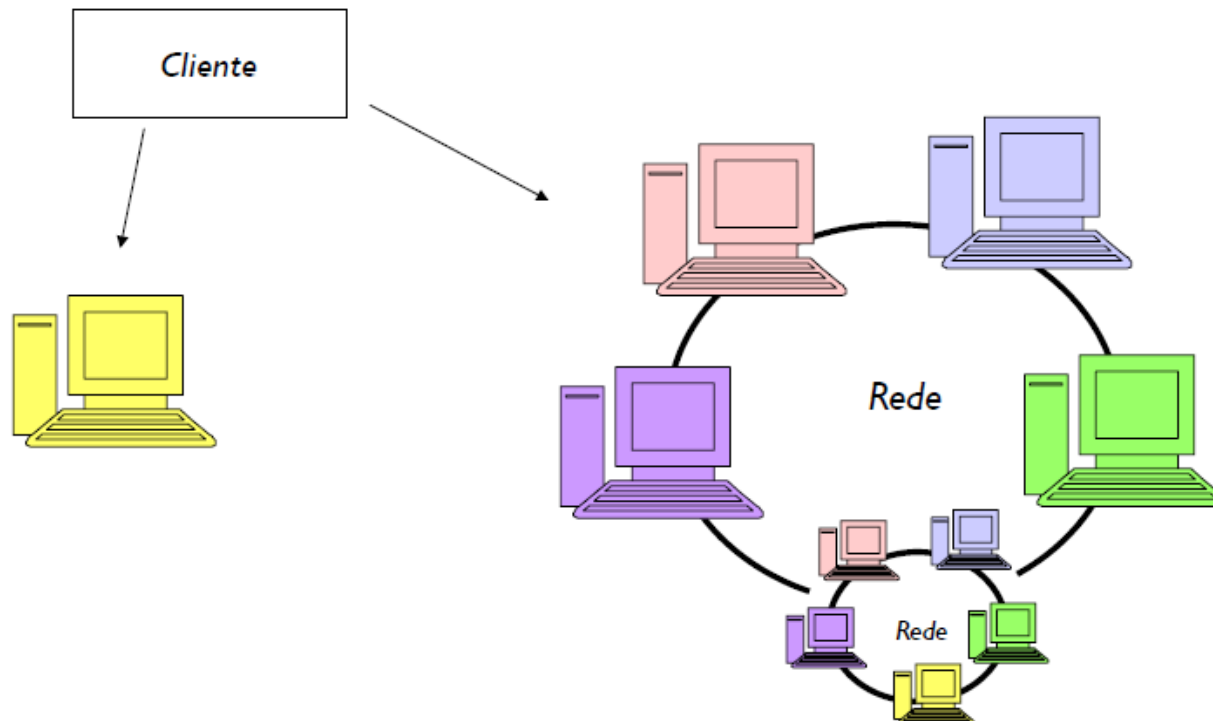
```
public class MensagemSMS implements Mensagem {  
    public void enviar(Carro c) {  
        System.out.printf("Enviando SMS de %s\n", c);  
    }  
  
    public void enviar(Caminhao c) {  
        System.out.printf("Enviando SMS de %s\n", c);  
    }  
}
```

➤ Exemplo 2:

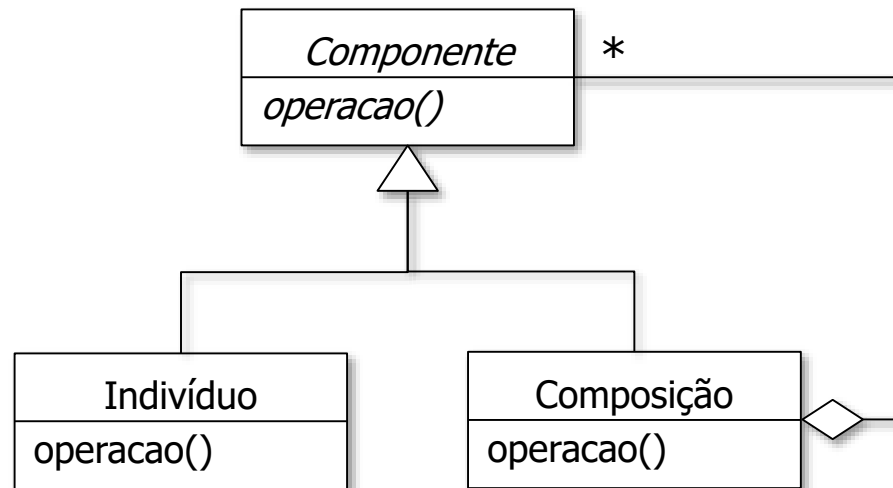
```
public class TesteBridge {  
  
    public static void main(String[] args) {  
        Carro cr = new Carro("Gol", 2013, 12000, 4);  
        Caminhao ca = new Caminhao("Mercedes", 2010, 250000, 30);  
  
        MensagemSMS msgSMS = new MensagemSMS();  
        MensagemEmail msgEmail = new MensagemEmail();  
  
        cr.enviarOferta(msgSMS);  
        cr.enviarOferta(msgEmail);  
        ca.enviarOferta(msgSMS);  
    }  
}
```

Nome:	Composite
Objetivo:	"Compor objetos em estruturas de árvore para representar hierarquias todo-parte. Composite permite que clientes tratem objetos individuais e composições de objetos de maneira uniforme" [GoF]
Motivação:	Em alguns casos, um objeto é composto de vários objetos, que por sua vez, podem ser compostos de outros objetos. Algumas operações precisam ser realizadas nesses objetos como um grupo, independente dele ser simples ou composto. Exemplos: • Em um editor gráfico, o usuário desenha formas geométricas primitivas (retângulo, triângulo, círculo, etc.) ou cria formas mais complexas a partir dessas primitivas. As operações de redimensionamento, rotação, deslocamento, etc., precisam ser aplicadas de forma consistente e uniforme tanto nas figuras primitivas quanto nas figuras complexas. • Em uma rede de computadores, as operações de bloquear ou liberar o acesso à internet deve ser feito de forma consistente tanto para um único computador quanto para uma sub-rede de computadores.

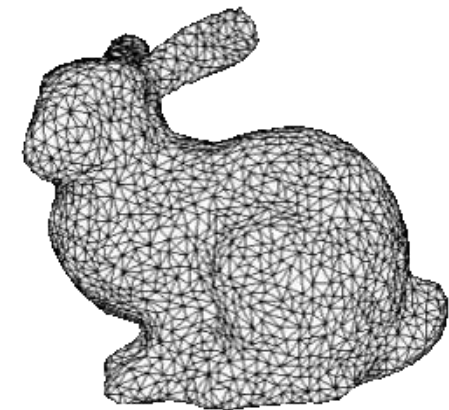
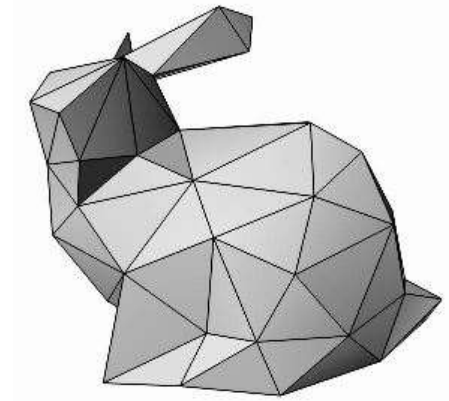
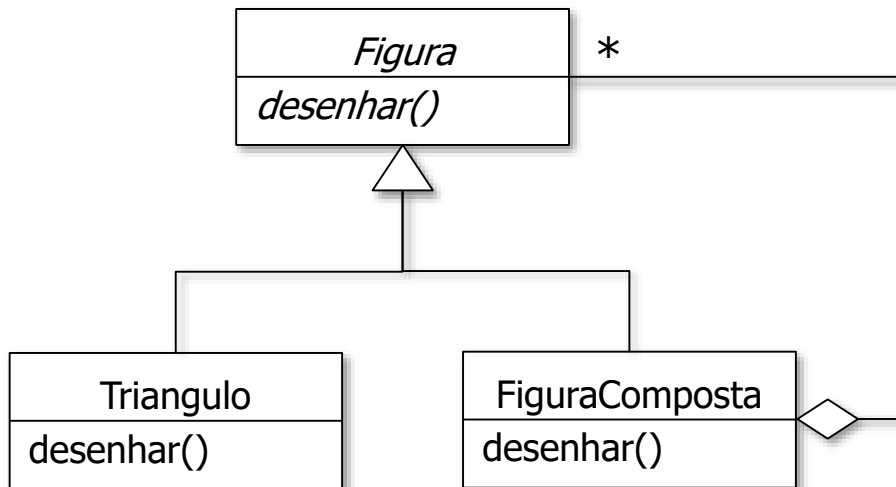
- Exemplo: rede de computadores
 - ✓ As operações para **bloquear internet** e **liberar internet** devem ser aplicadas de forma consistente tanto em um **nó específico** da rede, quanto na **rede toda** ou em uma **sub-rede**.



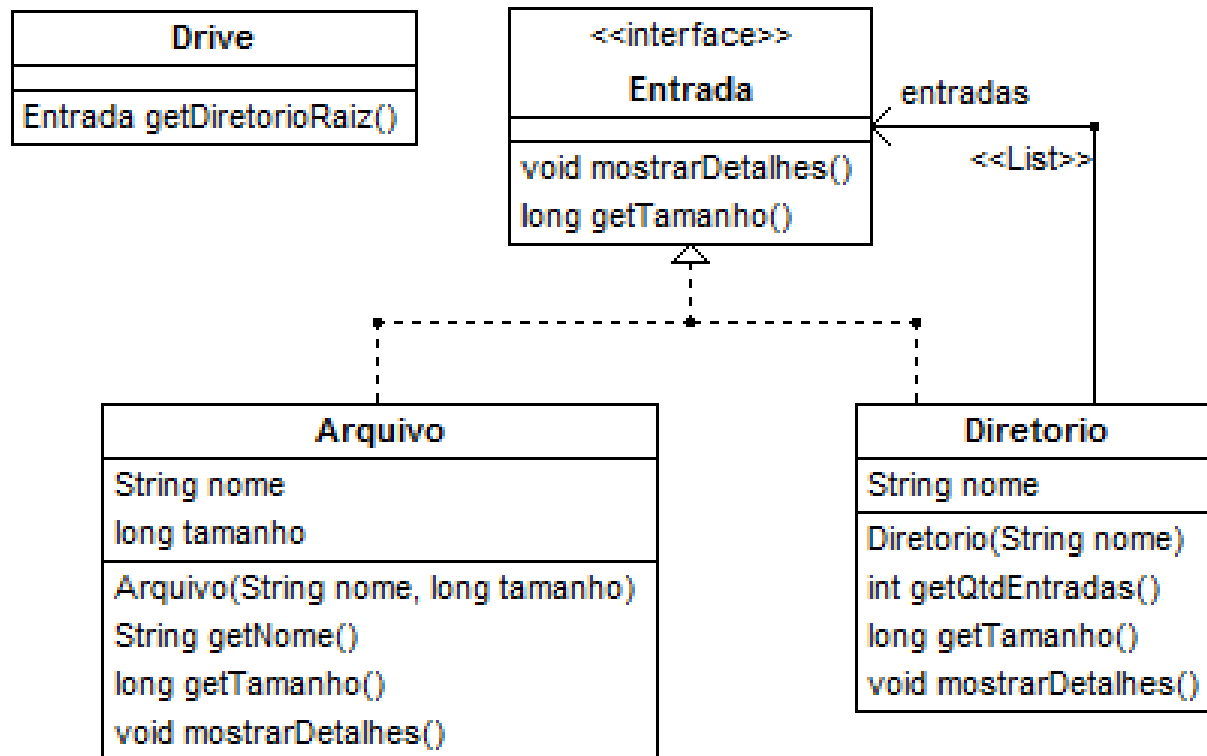
- Solução: usar uma **única interface** para tratar **indivíduos** ou **composições** (grupos de indivíduos).



- Exemplo: qualquer figura geométrica pode ser formada por uma composição de triângulos (tesselação).



- Exemplo: estrutura de diretórios.



➤ Exemplo: estrutura de diretórios.

```
public interface Entrada {  
    public void mostrarDetalhes();  
    public long getTamanho();  
}
```

```
public class Arquivo implements Entrada {  
    private String nome;  
    private long tamanho;  
  
    public Arquivo(String nome, long tamanho) {  
        this.nome = nome;  
        this.tamanho = tamanho;  
    }  
  
    public String getNome() { return nome; }  
    public long getTamanho() { return tamanho; }  
  
    public void mostrarDetalhes() {  
        System.out.printf("%s %dKb\n", nome, tamanho);  
    }  
}
```

➤ Exemplo: estrutura de diretórios.

```
public class Diretorio implements Entrada {
    private String nome;
    private List<Entrada> entradas;

    public Diretorio(String nome) {
        this.nome = nome;
        entradas = new ArrayList<>();
    }

    public int getQtdEntradas() { return entradas.size(); }
    public void add(Entrada e) { entradas.add(e); }

    public long getTamanho() {
        long tam = 0;
        for (Entrada e : entradas)
            tam += e.getTamanho();
        return tam;
    }

    public void mostrarDetalhes() {
        System.out.printf("%s %d %d Kb\n", nome, getQtdEntradas(), getTamanho());
        for (Entrada e : entradas)
            e.mostrarDetalhes();
    }
}
```

➤ Exemplo: estrutura de diretórios.

```
public class Drive {  
    private Diretorio raiz;  
  
    public Drive() {  
        raiz = new Diretorio("\\");  
    }  
  
    public Diretorio getRaiz() {  
        return raiz;  
    }  
  
    public void add(Entrada e) {  
        raiz.add(e);  
    }  
}
```

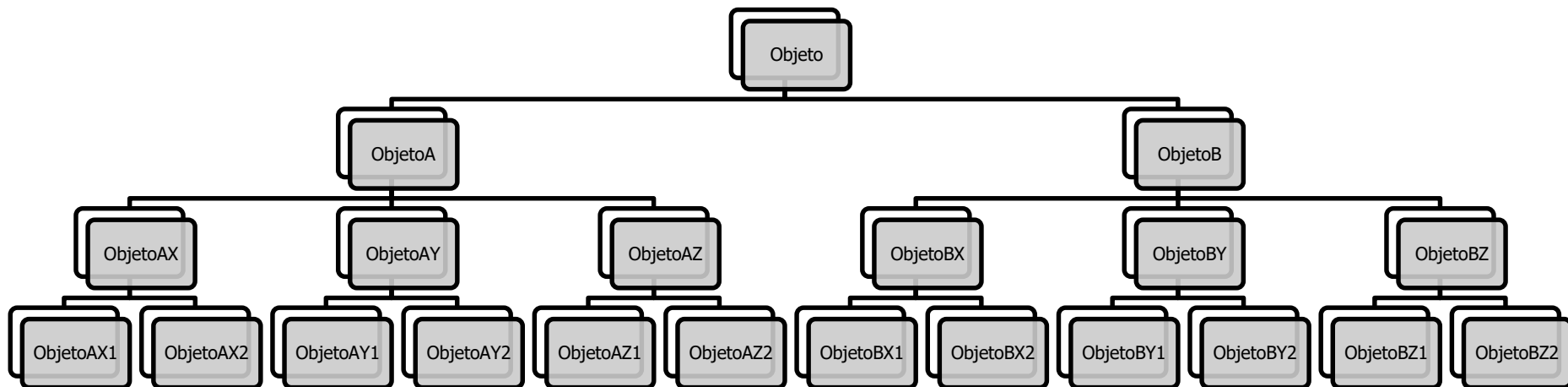
➤ Exemplo: estrutura de diretórios.

```
public class TesteComposite {  
    public static void main(String[] args) {  
        Drive d = new Drive();  
        Diretorio r = d.getRaiz();  
        Diretorio pessoal = new Diretorio("pessoal");  
        Diretorio musica = new Diretorio("musica");  
  
        r.add(new Arquivo("err.log", 125));  
        r.add(new Arquivo("conta.txt", 8));  
        r.add(pessoal);  
        r.add(musica);  
  
        pessoal.add(new Arquivo("cv.doc", 58));  
        pessoal.add(new Arquivo("foto.jpg", 642));  
        pessoal.add(new Arquivo("pagamento.pdf", 468));  
  
        musica.add(new Arquivo("time.mp3", 3547));  
        musica.add(new Arquivo("sultans of swing.mp3", 5479));  
  
        r.mostrarDetalhes();  
    }  
}
```

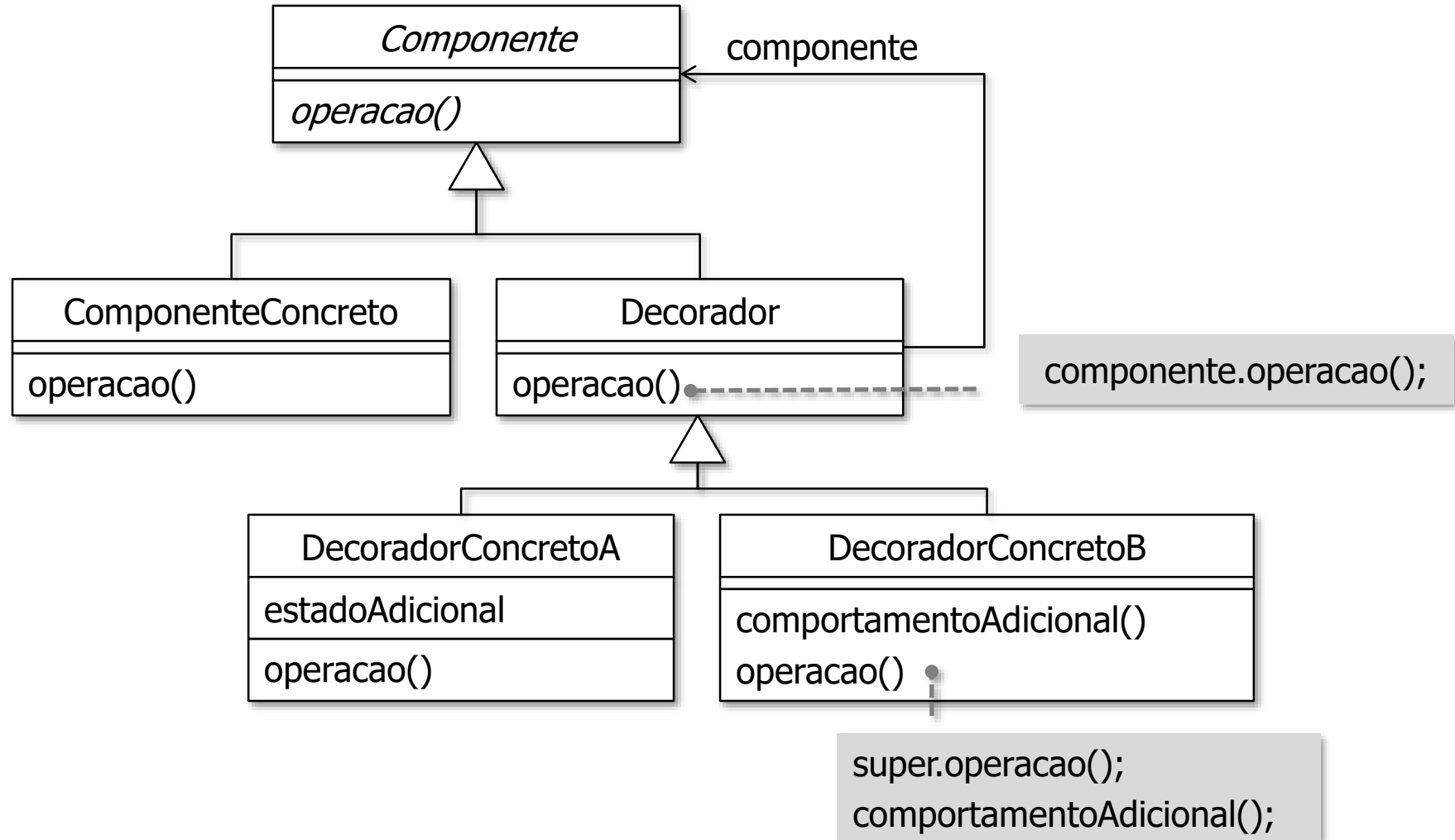
Nome:	Decorator
Objetivo:	"Anexar responsabilidades adicionais a um objeto dinamicamente. Decorator oferece uma alternativa flexível ao uso de herança para estender uma funcionalidade" [GoF]
Motivação:	Em algumas situações precisamos estender as funcionalidades de um objeto, mas não podemos ou não queremos usar herança. Exemplo: • Uma janela pode ter uma borda ou não, pode ter uma barra de rolagem vertical, horizontal ou ambas e pode ser ou não redimensionável.

➤ Problema:

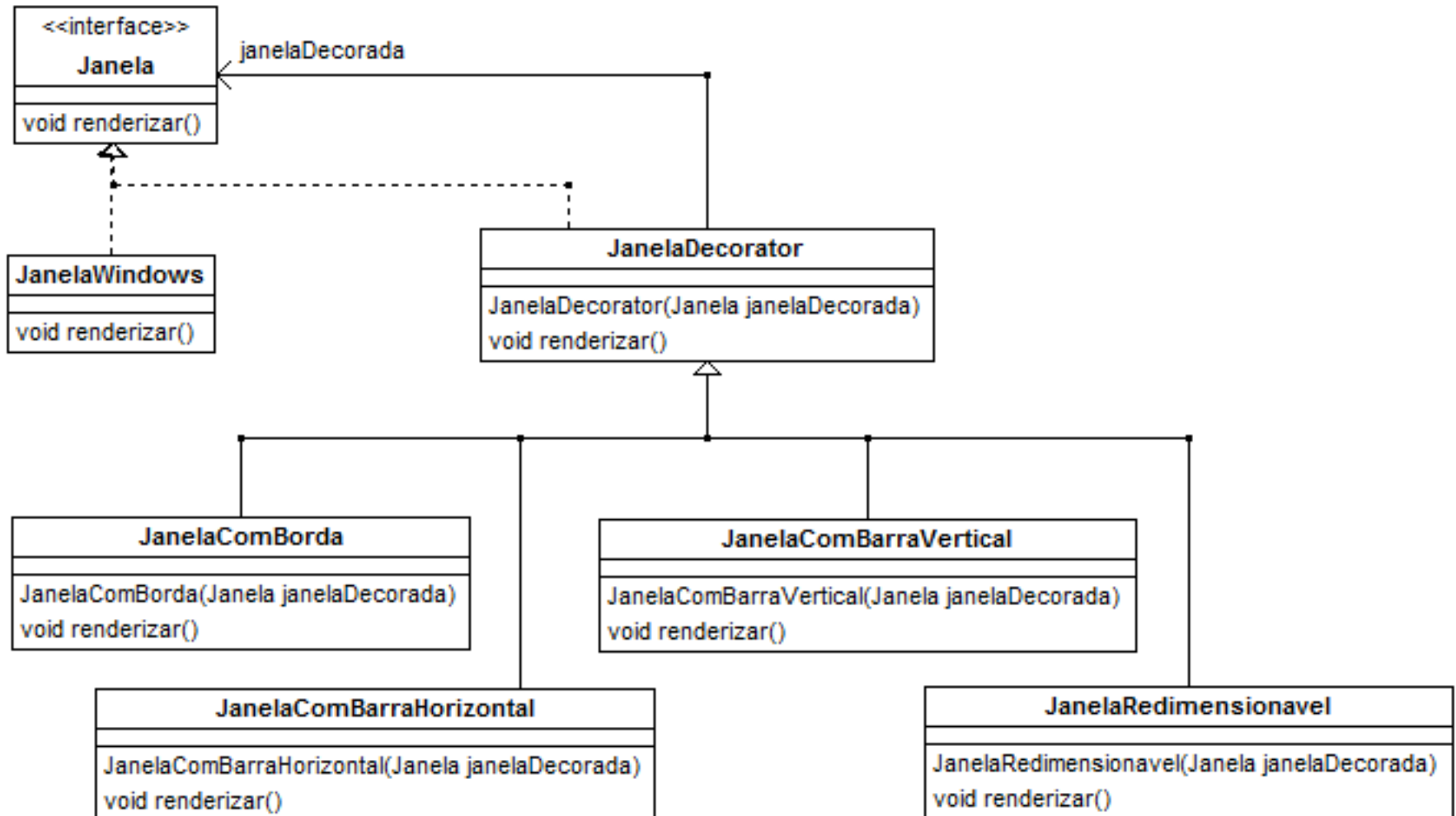
- ✓ A adoção de uma estrutura hierárquica para representar as combinação de diferentes funcionalidades de determinado objeto pode levar a um conjunto muito grande de classes.



➤ Solução:



➤ Exemplo:



➤ Exemplo:

```
public interface Janela {  
    public void renderizar();  
}
```

```
public class JanelaWindows implements Janela {  
    public void renderizar() {  
        System.out.println("Renderizando janela do windows...");  
    }  
}
```

➤ Exemplo:

```
public class JanelaDecorator implements Janela {  
  
    protected Janela janelaDecorada;  
  
    public JanelaDecorator(Janela janelaDecorada) {  
        this.janelaDecorada = janelaDecorada;  
    }  
  
    public void renderizar() {  
        janelaDecorada.renderizar();  
    }  
}
```

➤ Exemplo:

```
public class JanelaComBorda extends JanelaDecorator {  
  
    public JanelaComBorda(Janela janelaDecorada) {  
        super(janelaDecorada);  
    }  
  
    public void renderizar() {  
        super.renderizar();  
        System.out.println("Renderizando borda...");  
    }  
}
```

➤ Exemplo:

```
public class JanelaComBarraHorizontal extends JanelaDecorator {  
  
    public JanelaComBarraHorizontal(Janela janelaDecorada) {  
        super(janelaDecorada);  
    }  
  
    public void renderizar() {  
        super.renderizar();  
        System.out.println("Renderizando barra horizontal...");  
    }  
}
```

➤ Exemplo:

```
public class JanelaComBarraVertical extends JanelaDecorator {  
  
    public JanelaComBarraVertical(Janela janelaDecorada) {  
        super(janelaDecorada);  
    }  
  
    public void renderizar() {  
        super.renderizar();  
        System.out.println("Renderizando barra vertical...");  
    }  
}
```

➤ Exemplo:

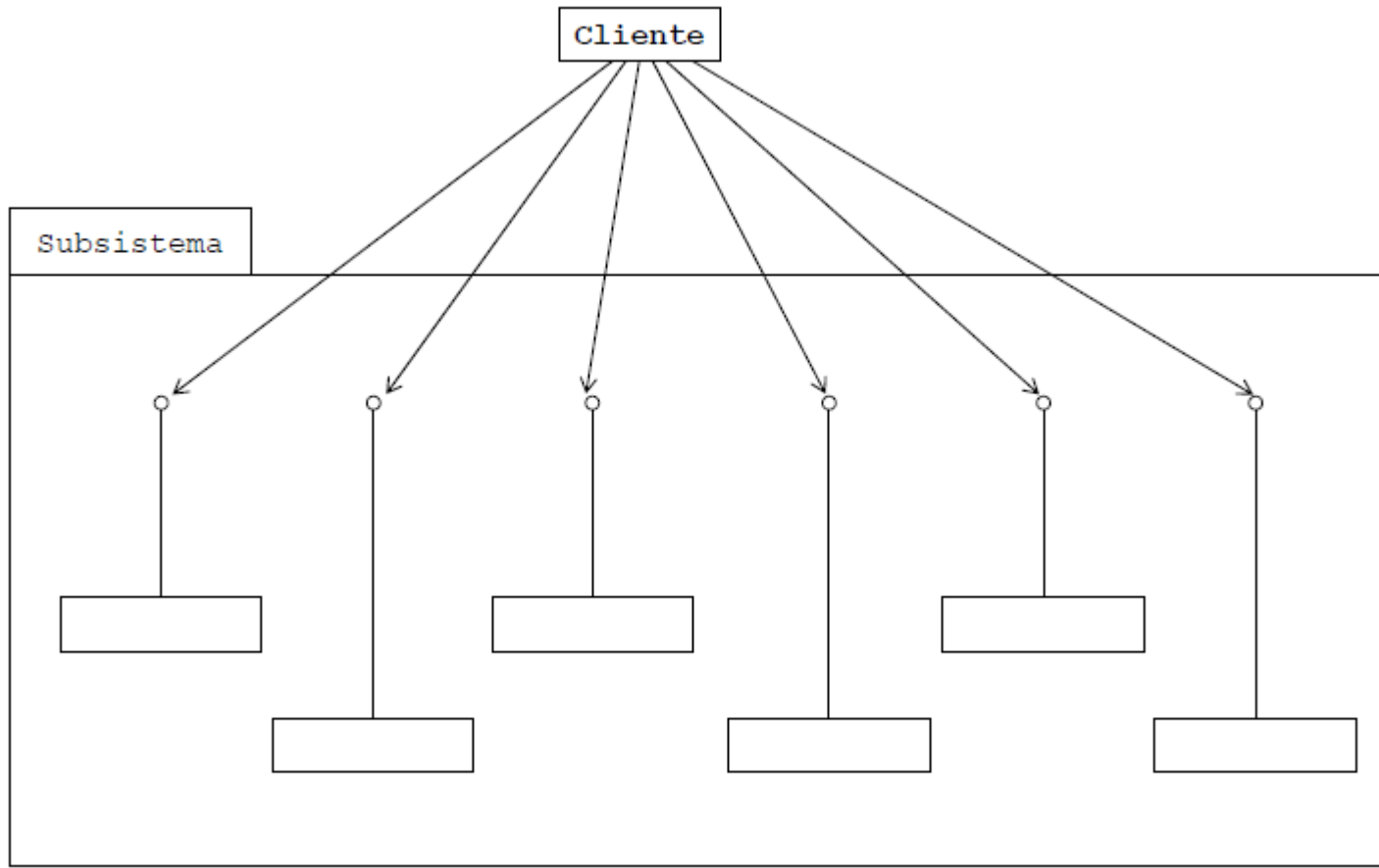
```
public class JanelaRedimensionavel extends JanelaDecorator {  
  
    public JanelaRedimensionavel(Janela janelaDecorada) {  
        super(janelaDecorada);  
    }  
  
    public void renderizar() {  
        super.renderizar();  
        System.out.println("Renderizando redimensionável...");  
    }  
}
```

➤ Exemplo:

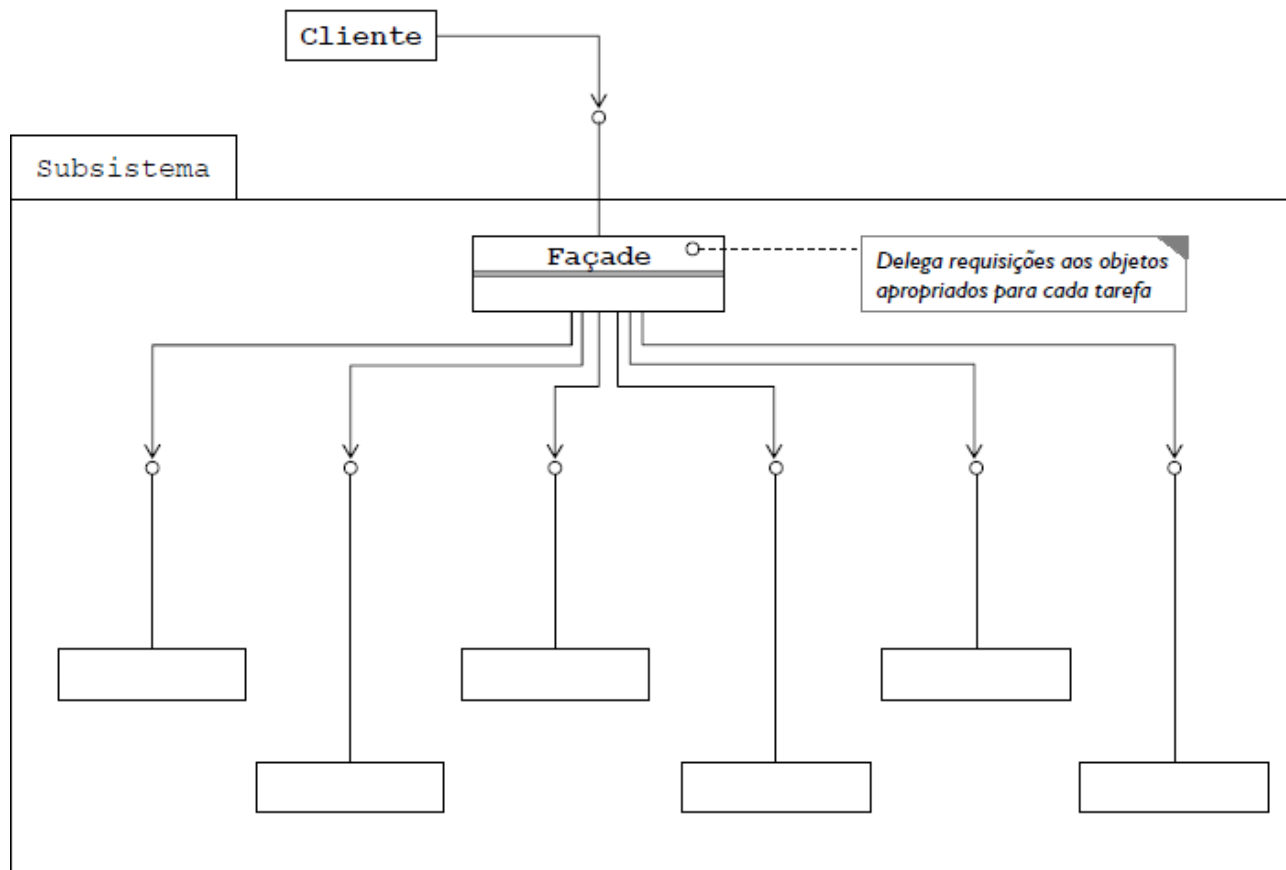
```
public class TesteDecorator {  
    public static void main(String[] args) {  
        Janela j1 = new JanelaComBarraVertical(  
            new JanelaComBorda(  
                new JanelaWindows())));  
  
        Janela j2 = new JanelaComBorda(  
            new JanelaRedimensionavel(  
                new JanelaWindows())));  
  
        j1.renderizar();  
        j2.renderizar();  
    }  
}
```


Nome:	Façade
Objetivo:	"Oferecer uma interface única para um conjunto de interfaces de um subsistema. Façade define uma interface de nível mais elevado que torna o subsistema mais fácil de usar" [GoF]
Motivação:	Um subsistema pode conter várias classes, mas geralmente não estamos interessados em detalhes de como essas classes se organizam ou como elas funcionam. Estamos somente preocupados em usar os serviços que esse subsistema oferece. Para esconder a complexidade desse subsistema, podemos criar uma classe que vai oferecer uma interface simples para acessar somente os serviços relevantes do subsistema. Exemplo: • Um subsistema que controla uma impressora multifuncional provavelmente será composto de diversas classes que representam os diversos componentes dessa impressora. Entretanto, do ponto de vista de uso dessa impressora, estamos interessados apenas nas operações de imprimir e digitalizar.

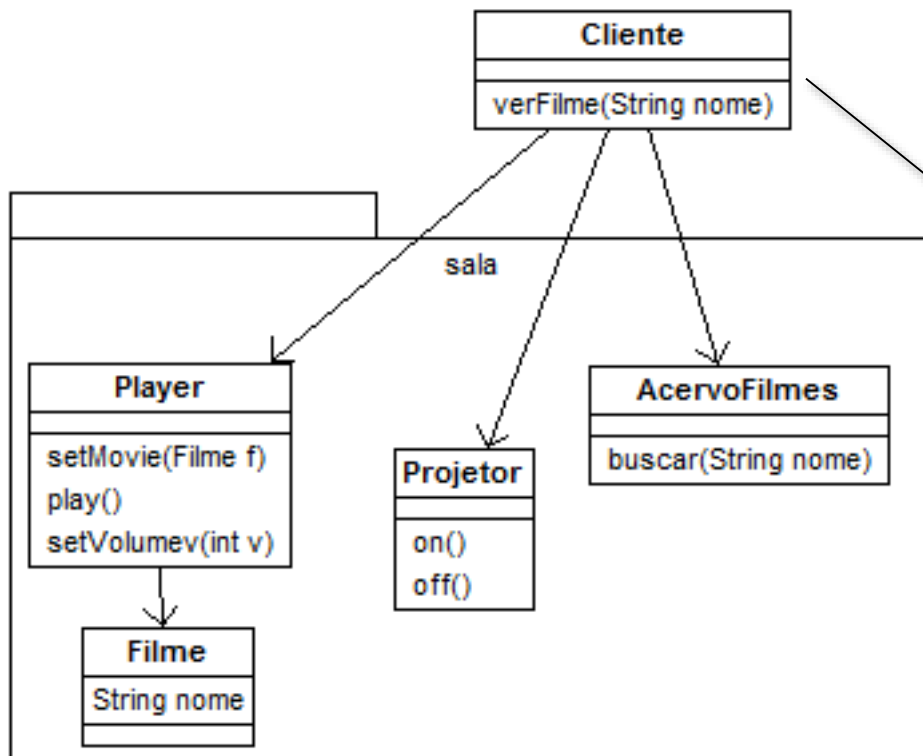
- Cliente é obrigado a lidar com a complexidade do subsistema:



- Solução: cliente interage com o **Façade**, que esconde a complexidade do subsistema. O **Façade** expõe, na sua interface, métodos úteis de acordo com os serviços oferecidos pelo subsistema.



- Exemplo: sala de projeção **sem** façade

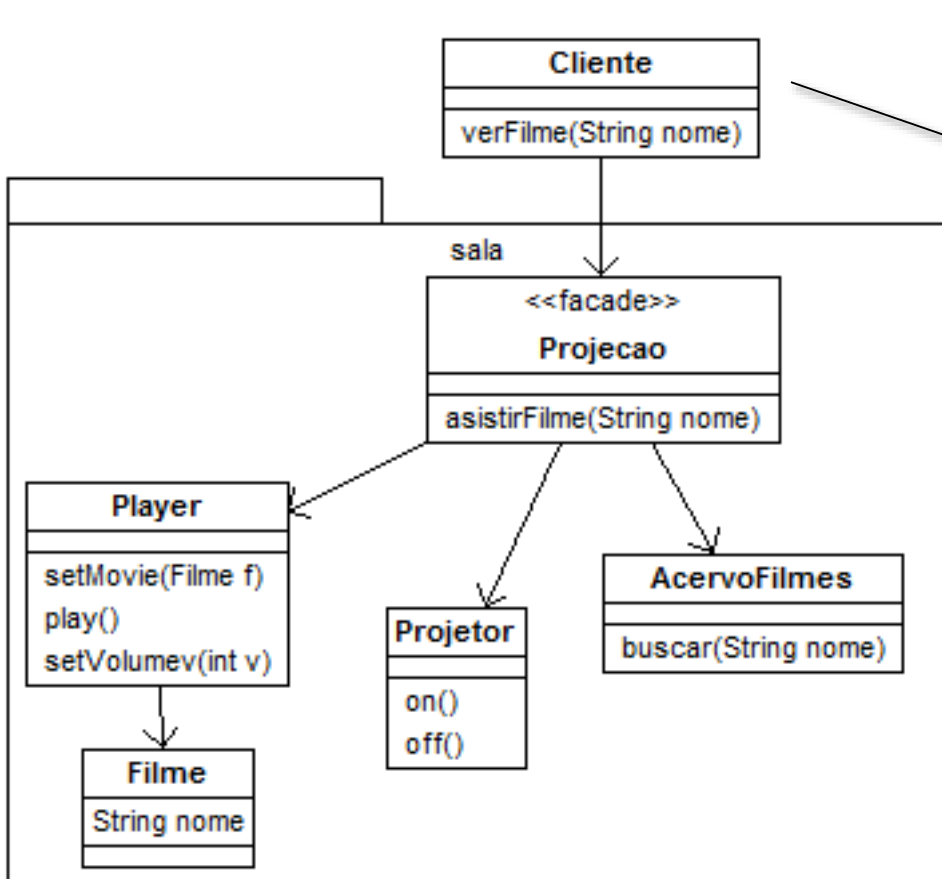


```

public void verFilme(nome: String)
{
    AcervoFilme a = new AcervoFilme();
    Player p = new Player();
    Projeto pj = new Projeto();

    Filme f = a.buscar(nome);
    pj.on();
    p.setMovie(f);
    p.setVolume(5);
    p.play();
    pj.off();
}
    
```

- Exemplo: sala de projeção **com** façade



```
public void verFilme(nome: String)
{
    Projecao p = new Projecao();

    p.assistirFilme(nome);
}
```

➤ Exemplo:

```
public class Filme {  
    private String nome;  
  
    public Filme(String nome) {  
        this.nome = nome;  
    }  
}
```

```
public class AcervoFilme {  
    public Filme buscar(String nome) {  
        System.out.printf("Buscando filme \"%s\"...\n", nome);  
  
        return new Filme(nome);  
    }  
}
```

➤ Exemplo:

```
public class Player {  
    private Filme filme;  
  
    public void setMovie(Filme f) {  
        this.filme = f;  
    }  
  
    public void play() {  
        System.out.println("Play...");  
    }  
  
    public void setVolume(int v) {  
        System.out.printf("Ajustando volume para %d...\n", v);  
    }  
}
```

➤ Exemplo:

```
public class Projetor {  
    public void on() {  
        System.out.println("Ligando projetor...");  
    }  
  
    public void off() {  
        System.out.println("Desligando projetor...");  
    }  
}
```


➤ Exemplo:

```
public class Projecao {  
    public void assistirFilme(String nome) {  
        AcervoFilme a = new AcervoFilme();  
        Player p = new Player();  
        Projetor pj = new Projetor();  
  
        Filme f = a.buscar(nome);  
        pj.on();  
        p.setMovie(f);  
        p.setVolume(5);  
        p.play();  
        pj.off();  
    }  
}
```

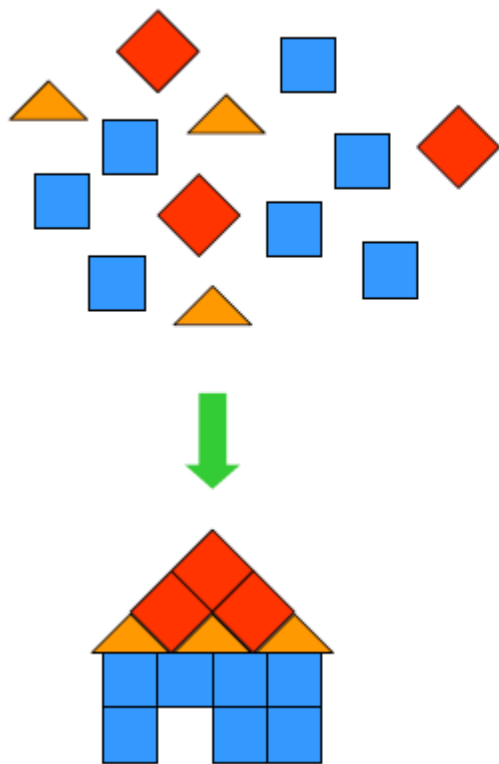
➤ Exemplo:

```
public class Cliente {  
    public void verFilme(String nome) {  
        Projecao p = new Projecao();  
  
        p.assistirFilme(nome);  
    }  
}
```

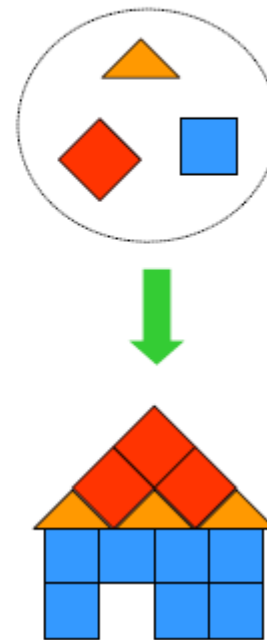
```
public class TesteFacade {  
    public static void main(String[] args) {  
        Cliente c = new Cliente();  
  
        c.verFilme("Jornada nas Estrelas");  
    }  
}
```

Nome:	Flyweight
Objetivo:	"Usar compartilhamento para suportar grandes quantidades de objetos refinados de forma eficiente" [GoF]
Motivação:	Alguns programas necessitam de centenas ou milhares de objetos que compartilham as mesmas características com estados distintos, podendo também possuir características específicas. Exemplo: • Em um jogo de guerra são renderizados centenas ou milhares de soldados. Todos eles possuem um conjunto de características comuns (roupa, capacete, arma, etc.), mas cada um deles também possui características específicas (posição, cor da pele, nível de saúde, etc.).

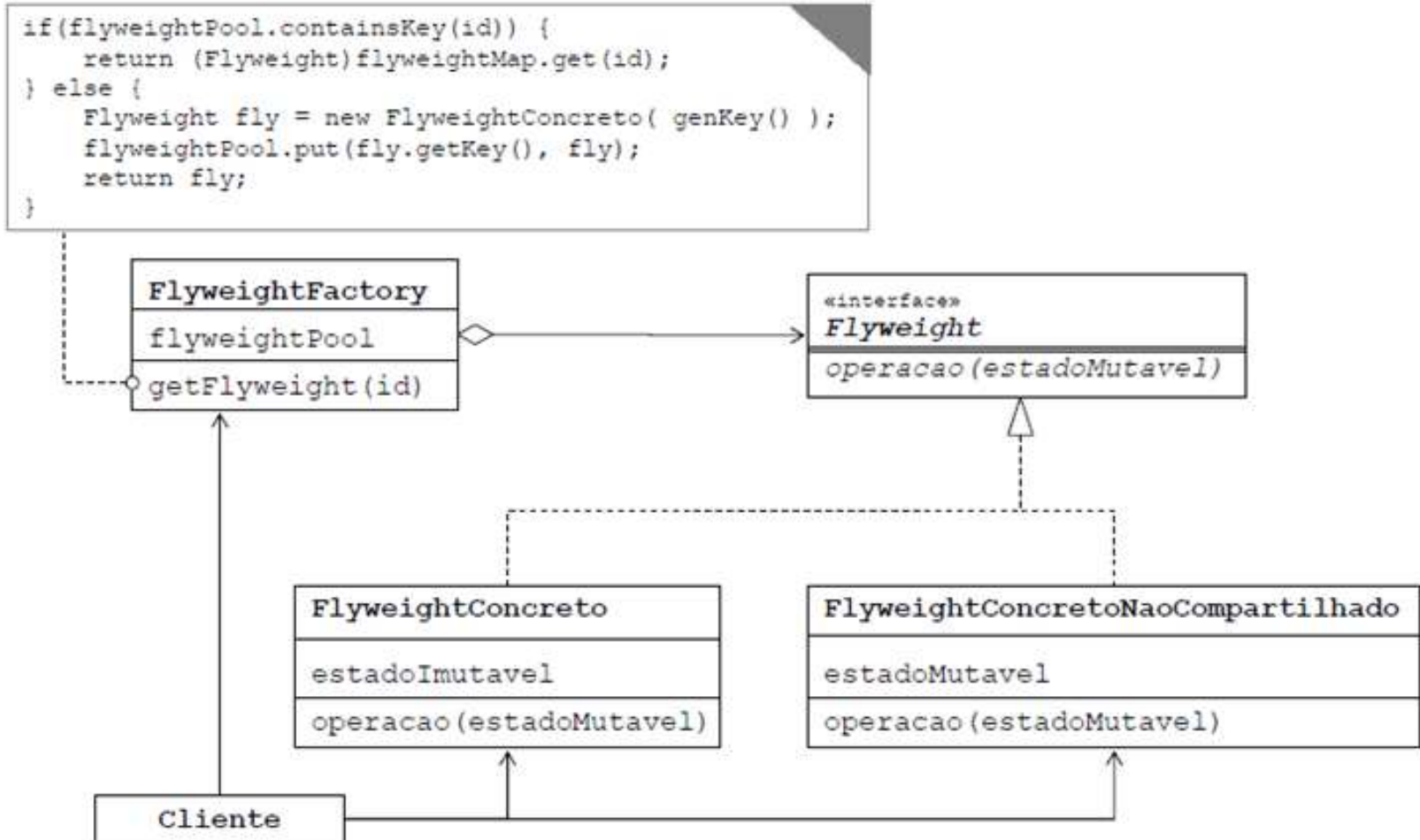
- Problema: para representar um cenário ou objeto complexo são necessários centenas ou milhares de outros objetos, o que pode levar a problemas de memória ou desempenho. Seria interessante poder representar as características comuns **uma única vez**.



*Pool de objetos
imutáveis
compartilhados*

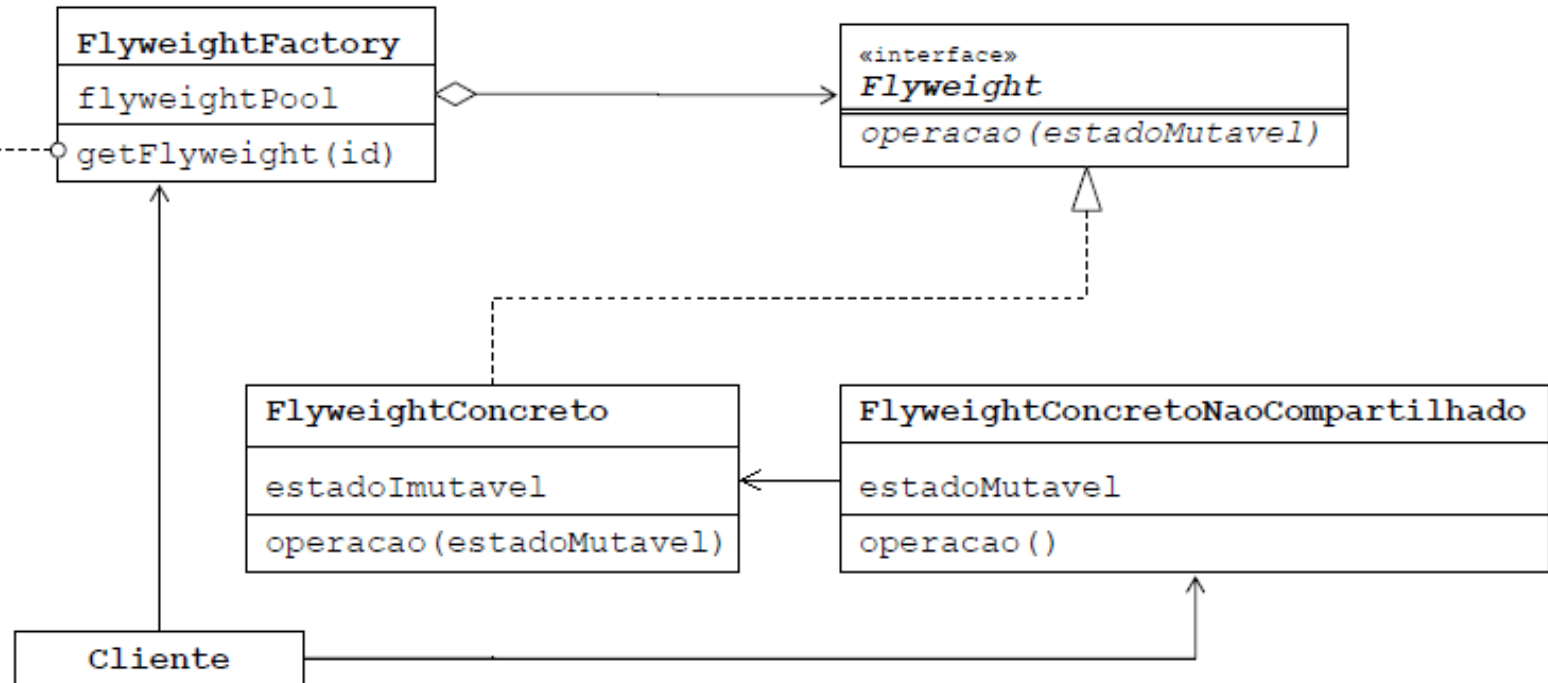


➤ Solução: duas possibilidades

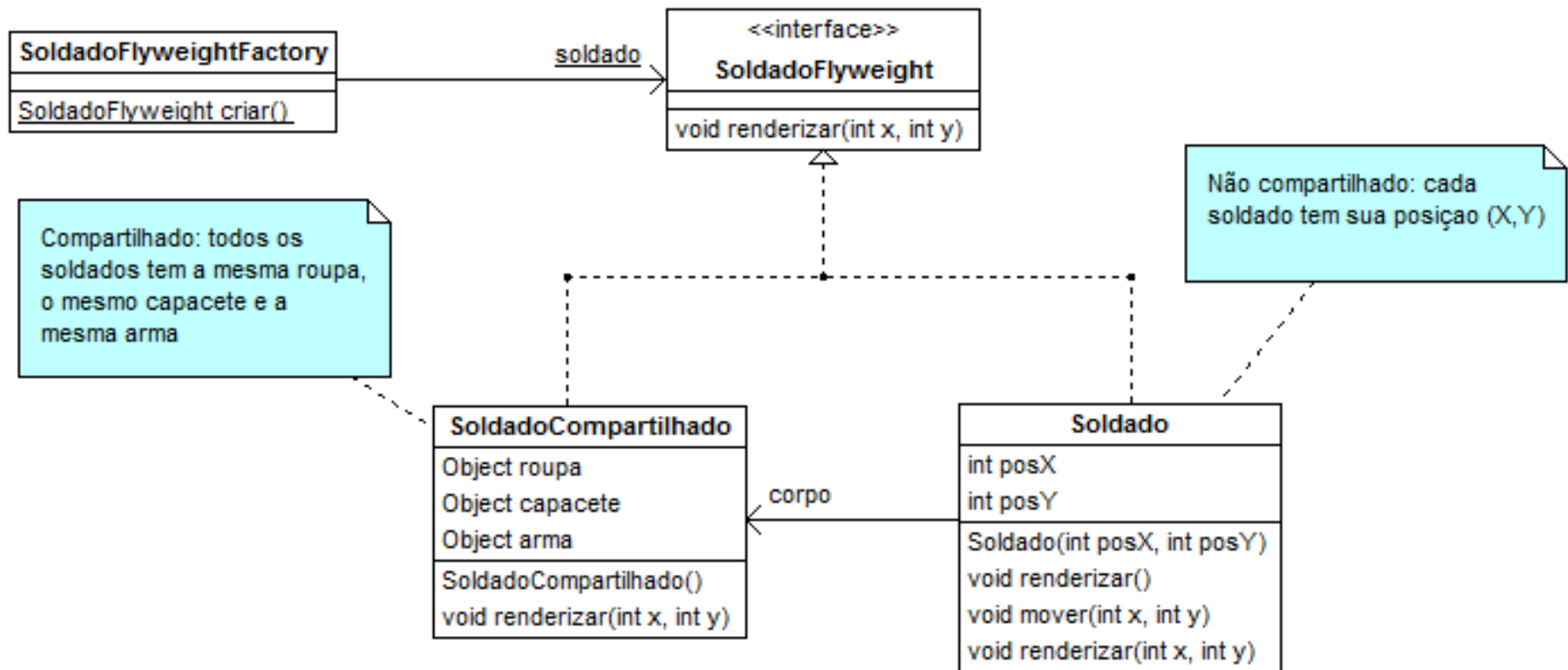


➤ Solução: duas possibilidades

```
if(flyweightPool.containsKey(id)) {  
    return (Flyweight)flyweightMap.get(id);  
} else {  
    Flyweight fly = new FlyweightConcreto( genKey() );  
    flyweightPool.put(fly.getKey(), fly);  
    return fly;  
}
```



➤ Exemplo:



➤ Exemplo:

```
public interface SoldadoFlyweight {  
    public void renderizar(int x, int y);  
}
```

```
public class SoldadoCompartilhado implements SoldadoFlyweight {  
  
    // Atributos comuns (intrínsecos - aqueles que todo soldado tem)  
    private Object roupa;  
    private Object capacete;  
    private Object arma;  
  
    public SoldadoCompartilhado() {  
        // Define os atributos intrínsecos - aqueles que todo soldado tem  
    }  
  
    @Override  
    public void renderizar(int x, int y) {  
        System.out.printf("Renderizando soldado em (%d, %d)...\n", x, y);  
    }  
}
```


- Exemplo: aqui foi usado o padrão Singleton, para garantir que apenas **UM** SoldadoCompartilhado será criado.

```
public class SoldadoFlyweightFactory {  
  
    private static SoldadoFlyweight soldado = null;  
  
    public static SoldadoFlyweight criar() {  
  
        if (soldado == null)  
            soldado = new SoldadoCompartilhado();  
  
        return soldado;  
    }  
}
```

➤ Exemplo:

```
public class Soldado implements SoldadoFlyweight {  
  
    private int posX;  
    private int posY;  
    private SoldadoCompartilhado corpo;  
  
    public Soldado(int posX, int posY) {  
        this.posX = posX;  
        this.posY = posY;  
        this.corpo = (SoldadoCompartilhado) SoldadoFlyweightFactory.criar();  
    }  
  
    public void renderizar() { corpo.renderizar(posX, posY); }  
  
    public void mover(int x, int y) { renderizar(posX + x, posY + y); }  
  
    public void renderizar(int x, int y) {  
        posX = x;  
        posY = y;  
        corpo.renderizar(posX, posY);  
    }  
}
```

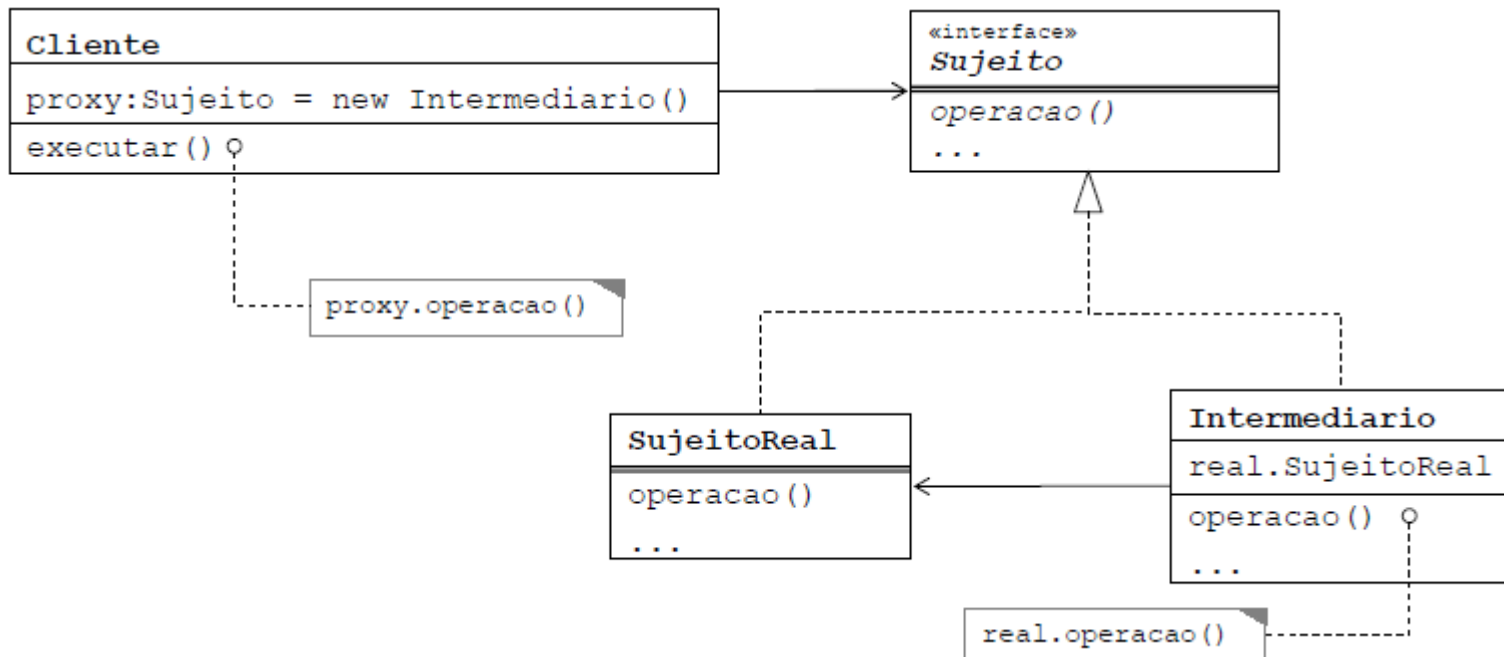
➤ Exemplo:

```
public class TesteFlyweight {  
    public static void main(String[] args) {  
        Random r = new Random();  
        int numeroSoldados = r.nextInt(50) + 50; // Cria de 50 a 99 soldados  
        List<Soldado> soldados = new ArrayList();  
  
        for (int i = 0; i < numeroSoldados; i++)  
            soldados.add(new Soldado(r.nextInt(100), r.nextInt(100)));  
  
        for (Soldado s : soldados)  
            s.renderizar();  
  
        for (Soldado s : soldados)  
            s.mover(-2+r.nextInt(5), r.nextInt(3));  
    }  
}
```

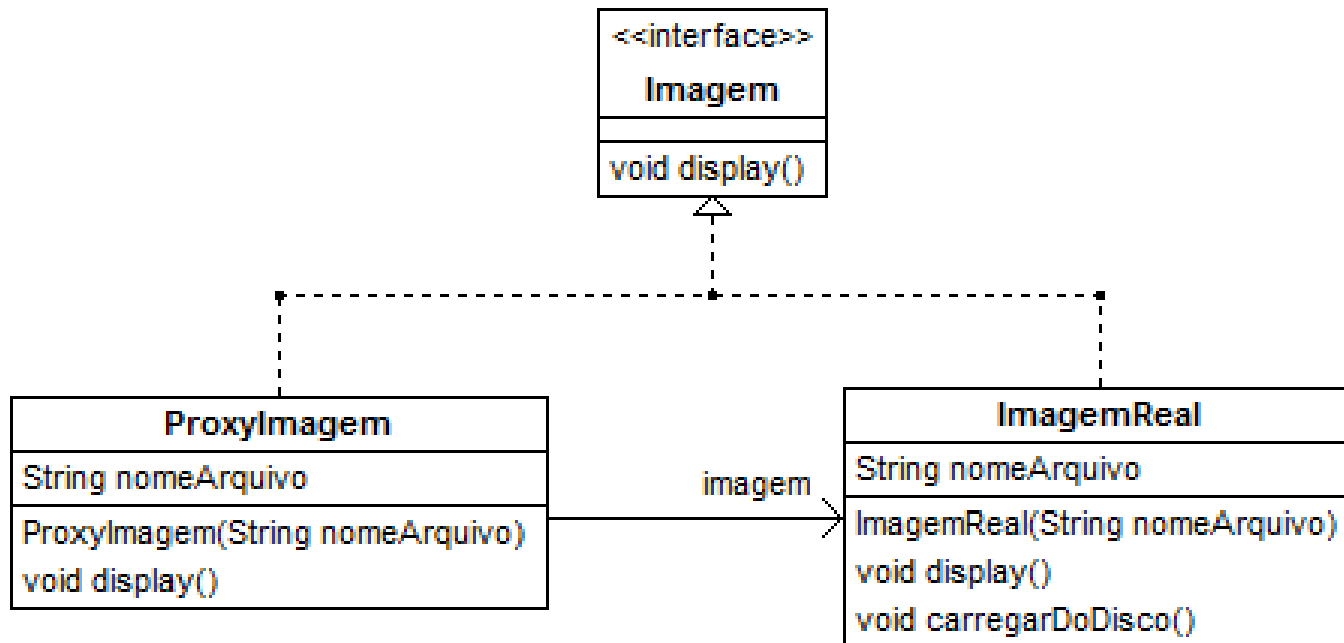
Nome:	Proxy
Objetivo:	"Prover um substituto ou ponto através do qual um objeto possa controlar o acesso a outro" [GoF]
Motivação:	Existem diversas situações nas quais precisamos controlar o acesso a determinado objeto. Exemplos: • Um objeto complexo é composto de vários outros objetos e só queremos criar essas partes, quando elas forem acessados. • Um objeto cujas características ou funcionalidades necessitam de permissões específicas para serem acessadas. • Um objeto que reside remotamente, mas queremos passar a sensação de que ele reside localmente. • Um objeto não reentrante, ou seja, suas características ou funcionalidades só podem ser acessadas por um único objeto de cada vez.

➤ Solução:

- ✓ Cliente usa intermediário em vez de sujeito real.
- ✓ Intermediário suporta a mesma interface que sujeito real.
- ✓ Intermediário contém uma referência para o sujeito real e repassa chamadas, possivelmente, acrescentando informações ou filtrando dados no processo.



➤ Exemplo:



➤ Exemplo:

```
public interface Imagem {  
    public void display();  
}
```

```
public class ProxyImagem implements Imagem {  
  
    private ImagemReal imagem;  
    private String nomeArquivo;  
  
    public ProxyImagem(String nomeArquivo) {  
        this.nomeArquivo = nomeArquivo;  
    }  
  
    @Override  
    public void display() {  
        if (imagem == null)  
            imagem = new ImagemReal(nomeArquivo);  
  
        imagem.display();  
    }  
}
```

➤ Exemplo:

```
public class ImagemReal implements Imagem {

    private String nomeArquivo;

    public ImagemReal(String nomeArquivo) {
        this.nomeArquivo = nomeArquivo;
        carregarDoDisco();
    }

    @Override
    public void display() {
        System.out.printf("Apresentando imagem %s...\n", nomeArquivo);
    }

    private void carregarDoDisco() {
        System.out.printf("Carregando imagem %s...\n", nomeArquivo);
    }
}
```


➤ Exemplo:

```
public class TesteProxy {  
    public static void main(String[] args) {  
        Imagem imagem1 = new ProxyImagem("teste1_10mb.jpg");  
        Imagem imagem2 = new ProxyImagem("teste2_10mb.jpg");  
  
        // A imagem1 só é carregada (uma única vez) quando for apresentada  
        imagem1.display();  
        imagem1.display();  
  
        // A imagem2 nunca é carregada para a memória...  
    }  
}
```

➤ Padrões Comportamentais:

		Propósito		
		1. Criação	2. Estrutura	3. Comportamento
Escopo	Classe	Factory Method	Class Adapter	Interpreter Template Method
	Objeto	Abstract Factory Builder Prototype Singleton	Object Adapter Bridge Composite Decorator Facade Flyweight Proxy	Chain of Responsibility Command Iterator Mediator Memento Observer State Strategy Visitor



Nome:	Template Method
Objetivo:	"Definir o esqueleto de um algoritmo dentro de uma operação, deixando alguns passos a serem preenchidos pelas subclasses. Template Method permite que suas subclasses redefinam certos passos de um algoritmo sem mudar sua estrutura" [GoF]
Motivação:	Em algumas situações, determinado algoritmo oferece uma estrutura geral de solução para um problema. Entretanto, ele precisa ser especializado em alguns pontos para que seja aplicado a um problema específico. Exemplos: • Uma determinada lista de itens precisa ser apresentada, mas a ordem das colunas e a ordem das linhas precisa ser adaptada de acordo com algum critério.

- Problema: um algoritmo que oferece uma estrutura geral, mas que precisa ser especializado em algumas partes de acordo com uma ou mais condições.

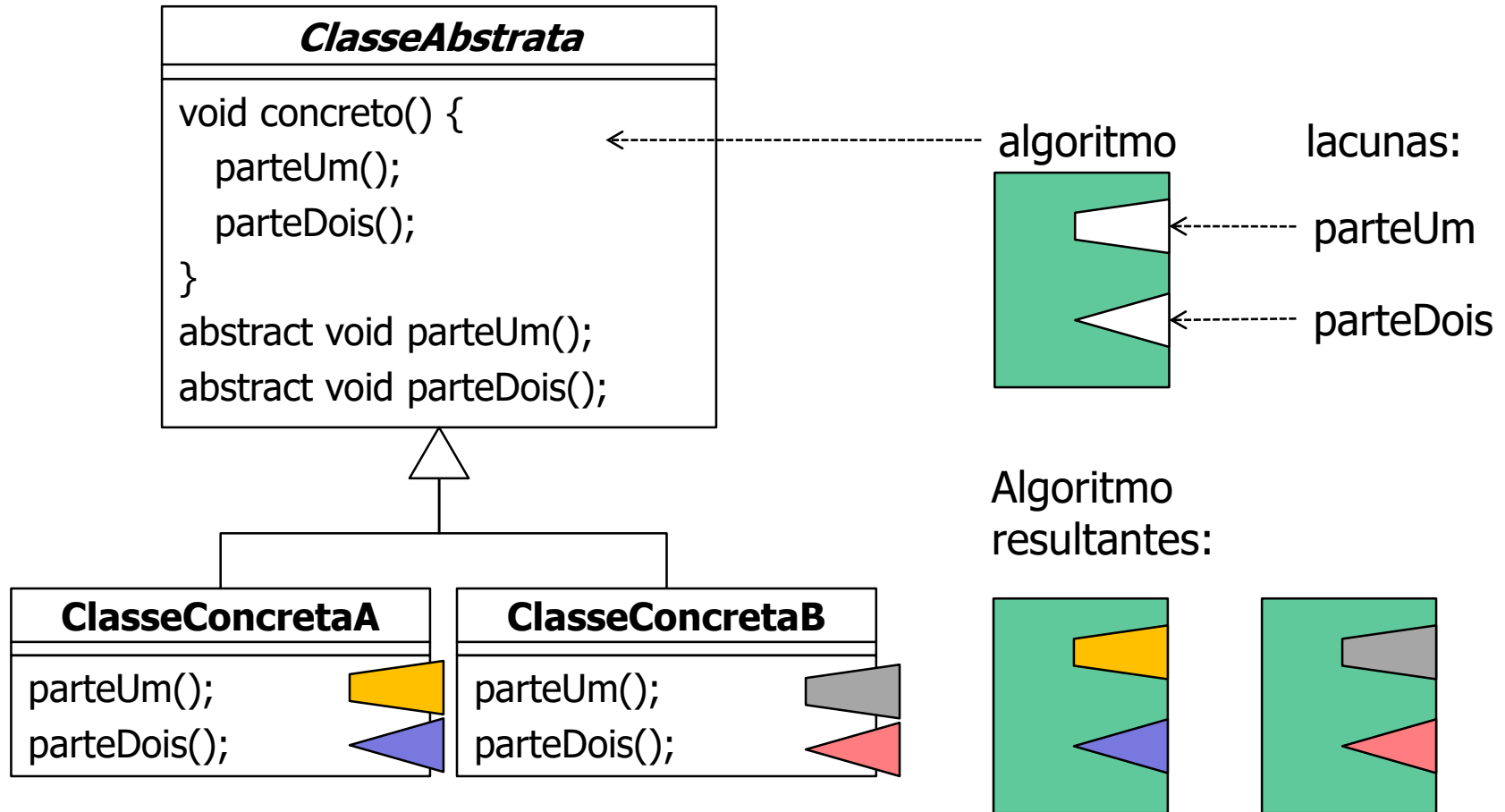
```
processarA() {  
    passo1();  
    passo2();  
    passo3();  
    passo5();  
    passo6();  
    passo7();  
    passo9();  
}
```

```
processarB() {  
    passo1();  
    passo2();  
    passo4();  
    passo5();  
    passo6();  
    passo8();  
    passo9();  
}
```

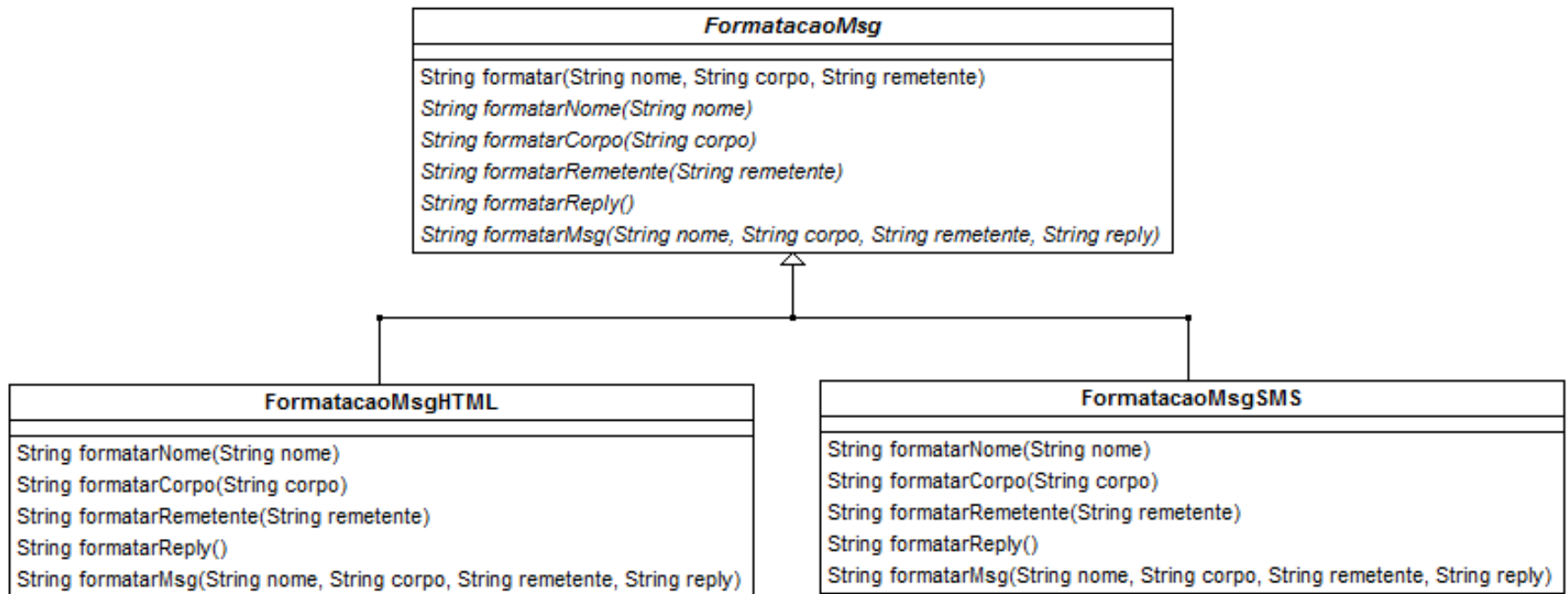
- Problema: um algoritmo que oferece uma estrutura geral, mas que precisa ser especializado em algumas partes de acordo com uma ou mais condições.

```
processar(condiçãoA, condiçãoB) {  
    passo1();  
    passo2();  
    if (condiçãoA)  
        passo3();  
    else  
        passo4();  
    passo5();  
    passo6();  
    if (condiçãoB)  
        passo7();  
    else  
        passo8();  
    passo9();  
}
```

➤ Solução:



➤ Exemplo:



➤ Exemplo:

```
public abstract class FormatacaoMsg {  
    public String formatar(String nome, String corpo, String remetente) {  
  
        String n  = formatarNome(nome);  
        String c  = formatarCorpo(corpo);  
        String r  = formatarRemetente(remetente);  
        String rp = formatarReply();  
  
        return formatarMsg(n, c, r, rp);  
    }  
  
    protected abstract String formatarNome(String nome);  
    protected abstract String formatarCorpo(String corpo);  
    protected abstract String formatarRemetente(String remetente);  
    protected abstract String formatarReply();  
    protected abstract String formatarMsg(String nome,  
                                           String corpo,  
                                           String remetente,  
                                           String reply);  
}
```


➤ Exemplo:

```
public class FormatacaoMsgHTML extends FormatacaoMsg {  
  
    protected String formatarNome(String nome) {  
        return "<h1>Caro " + nome + ",</h1>";  
    }  
  
    protected String formatarCorpo(String corpo) {  
        return "<p>" + corpo + "</p>";  
    }  
  
    protected String formatarRemetente(String remetente) {  
        return "<b>" + remetente + "</b>";  
    }  
  
    protected String formatarReply() {  
        return "<small>Não dê replay</small>";  
    }  
  
    protected String formatarMsg(String nome, String corpo, String remetente, String reply) {  
        return "<html><body>" + nome + corpo + remetente + reply + "</body></html>";  
    }  
}
```

➤ Exemplo:

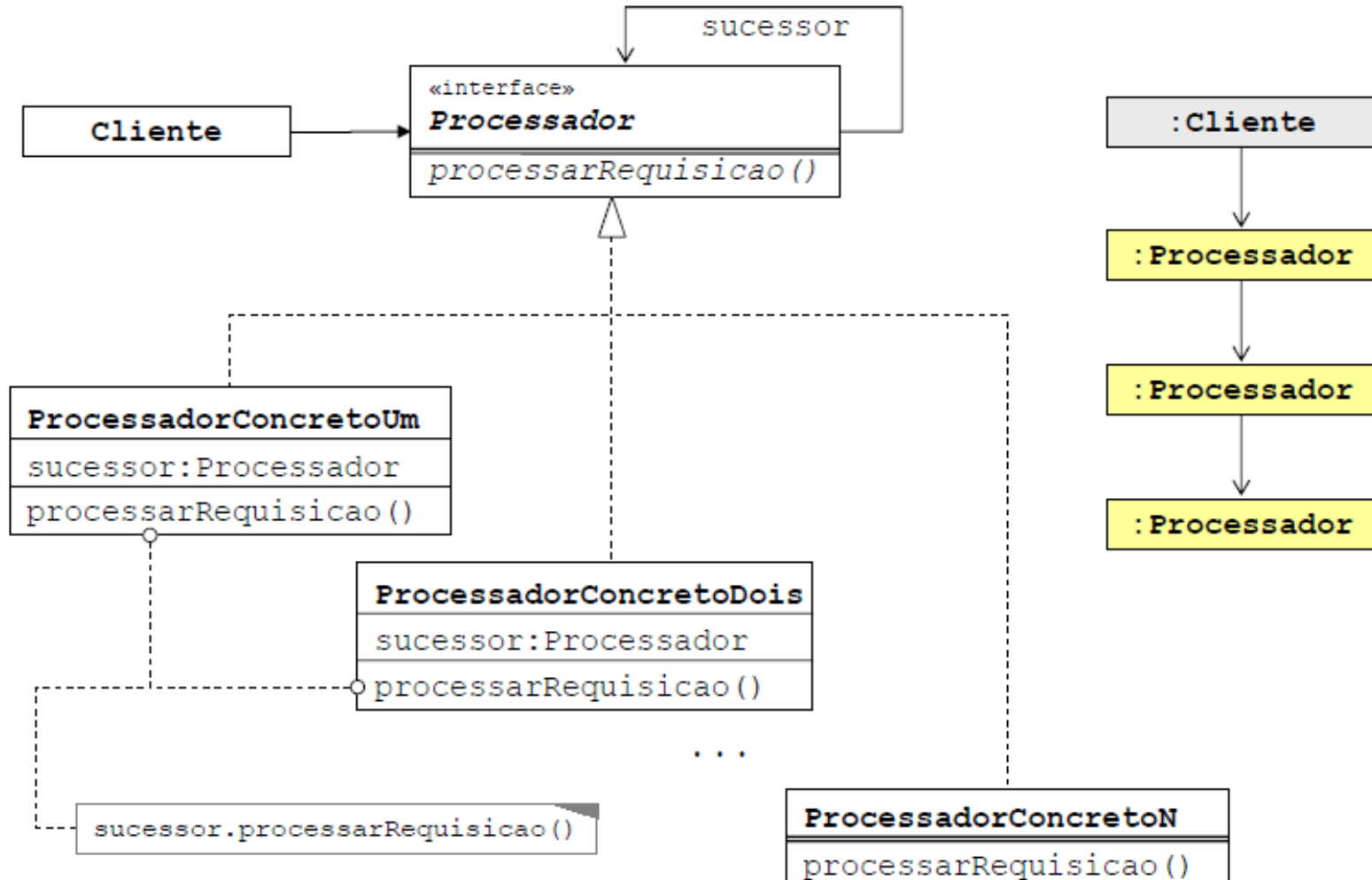
```
public class FormatacaoMsgSMS extends FormatacaoMsg {  
  
    protected String formatarNome(String nome) {  
        return "Caro " + nome + "\n";  
    }  
  
    protected String formatarCorpo(String corpo) {  
        return corpo+"\n";  
    }  
  
    protected String formatarRemetente(String remetente) {  
        return remetente + "\n";  
    }  
  
    protected String formatarReply() {  
        return "";  
    }  
  
    protected String formatarMsg(String nome, String corpo, String remetente, String reply) {  
        return nome+corpo+remetente+reply;  
    }  
}
```

➤ Exemplo:

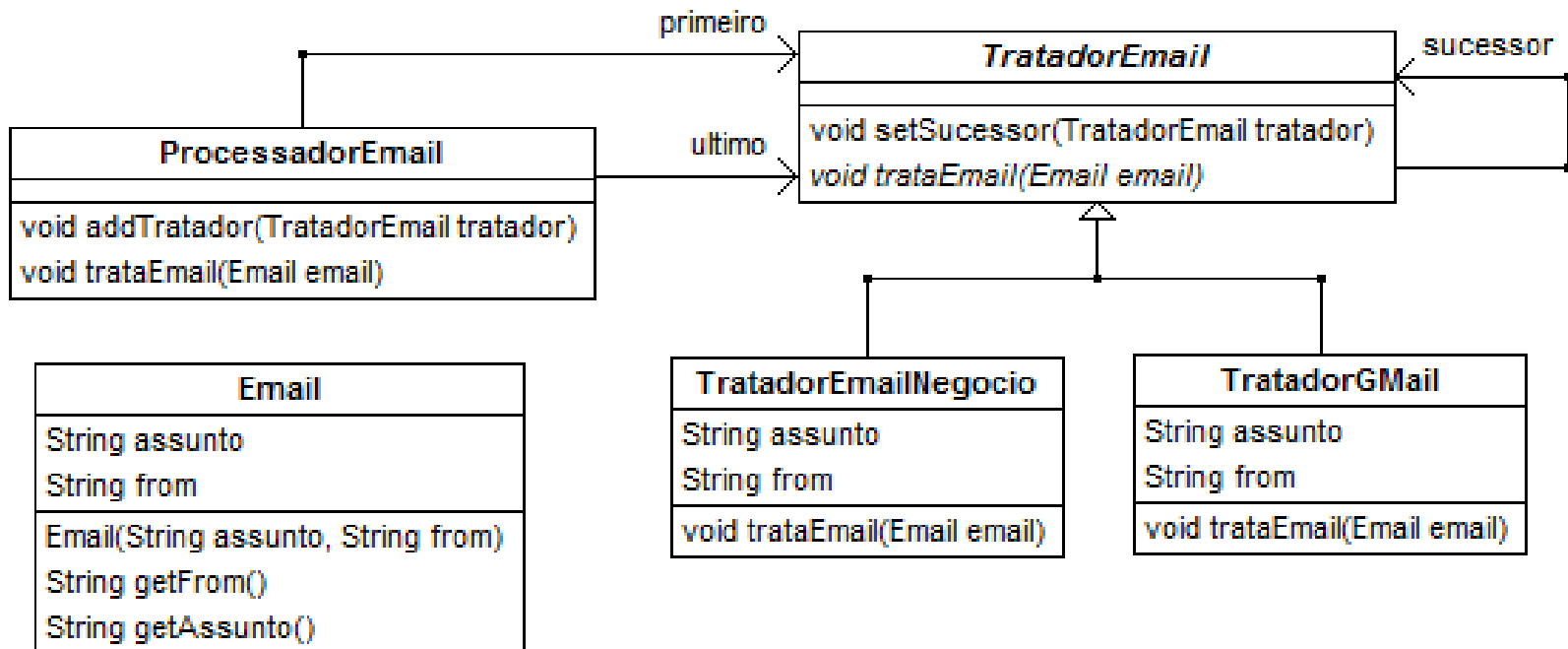
```
public class TesteTemplateMethod {  
  
    public static void main(String[] args) {  
  
        FormatacaoMsg fmt1 = new FormatacaoMsgHTML();  
        FormatacaoMsg fmt2 = new FormatacaoMsgSMS();  
  
        String msg1 = fmt1.formatar("João da Silva",  
                                    "Entre em contato urgente com o SAC",  
                                    "Serviço de Atendimento ao Consumidor: sac@empresa.br");  
  
        String msg2 = fmt2.formatar("João da Silva",  
                                    "Entre em contato urgente com o SAC",  
                                    "Serviço de Atendimento ao Consumidor: sac@empresa.br");  
  
        System.out.println(msg1);  
        System.out.println(msg2);  
    }  
}
```

Nome:	Chain of Responsibility
Objetivo:	"Evita acoplar o remetente de uma requisição ao seu destinatário ao dar a mais de um objeto a chance de servir a requisição. Compõe os objetos em cascata e passa a requisição pela corrente até que um objeto a atenda" [GoF]
Motivação:	Em algumas situações, uma requisição enviada por um remetente pode ter vários destinatários distintos, dependendo do tipo de requisição. A ideia é organizar os possíveis destinatários em uma cadeia, para que o destinatário trate a requisição ou a passe adiante para o próximo da cadeia. Exemplos: • Um servidor que atende a diferentes requisições como http, ftp, https, etc. • Um email recebido pode ter como destino a caixa de spam, a caixa de entrada geral, ou uma caixa específica. • A confirmação de um recebimento deve ser tratada de acordo com a forma de pagamento (cartão de crédito, cartão de débito, boleto, pay-pal, etc.)

➤ Solução:



➤ Exemplo:



➤ Exemplo:

```
public class Email {  
    private String assunto, from;  
  
    public Email(String assunto, String from) {  
        this.assunto = assunto;  
        this.from = from;  
    }  
    public String getFrom() { return from; }  
    public String getAssunto() { return assunto; }  
}
```

```
public abstract class TratadorEmail{  
    protected TratadorEmail sucessor;  
  
    public void setSucessor(TratadorEmail tratador) {  
        sucessor = tratador;  
    }  
  
    public abstract void trataEmail(Email email);  
}
```

➤ Exemplo:

```
public class TratadorEmailNegocio extends TratadorEmail {  
    private String assunto, from;  
  
    public void trataEmail(Email email) {  
        if (email.getFrom().endsWith("@empresa.com"))  
            System.out.printf("Email [%s] será copiado para caixa de entrada\n",  
                               email.getAssunto());  
        else if (sucessor != null)  
            sucessor.trataEmail(email);  
    }  
}
```


➤ Exemplo:

```
public class TratadorGMail extends TratadorEmail {  
    private String assunto, from;  
  
    public void trataEmail(Email email) {  
        if (email.getFrom().endsWith("@gmail.com"))  
            System.out.printf("Email [%s] será copiado para caixa pessoal\n",  
                               email.getAssunto());  
        else if (sucessor != null)  
            sucessor.trataEmail(email);  
    }  
}
```

➤ Exemplo:

```
public class ProcessadorEmail {  
    private TratadorEmail primeiro, ultimo;  
  
    public void addTratador(TratadorEmail tratador) {  
        if (primeiro == null)  
            primeiro = tratador;  
        else  
            ultimo.setSucessor(tratador);  
  
        ultimo = tratador;  
    }  
  
    public void trataEmail(Email email) {  
        if (primeiro != null)  
            primeiro.trataEmail(email);  
    }  
}
```

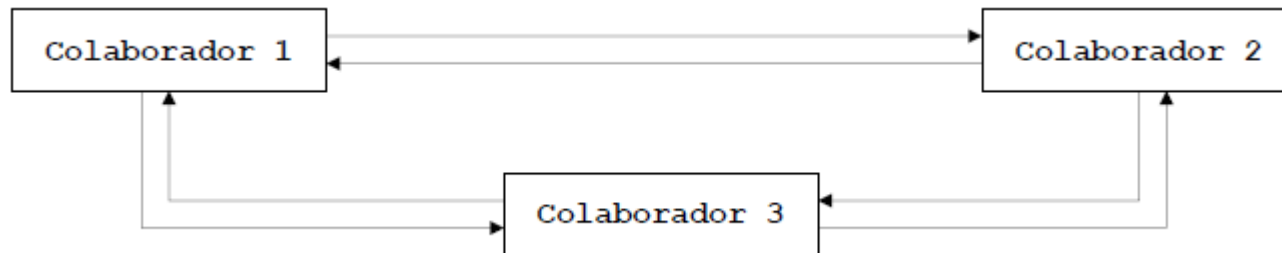
➤ Exemplo:

```
public class TesteChainOfResponsability {  
    public static void main(String[] args) {  
        ProcessadorEmail proc = new ProcessadorEmail();  
  
        proc.addTratador(new TratadorEmailNegocio());  
        proc.addTratador(new TratadorGMail());  
  
        Email email1 = new Email("Pendencia da entrega",  
                                   "analista@empresa.com");  
        Email email2 = new Email("Festa", "amigo@gmail.com");  
  
        proc.trataEmail(email1);  
        proc.trataEmail(email2);  
    }  
}
```

Nome:	Mediator
Objetivo:	"Definir um objeto que encapsula como um conjunto de objetos interagem. O Mediator promove acoplamento fraco ao manter objetos que não se referem um ao outro explicitamente, permitindo variar sua interação independentemente" [GoF]
Motivação:	Quando um conjunto de objetos precisa se relacionar, temos uma situação de forte acoplamento entre esses objetos, pois cada um tem que conhecer o demais. Além disso, a inclusão de um novo objeto ao conjunto faz com que os demais tenham que ser atualizados para conhecer o novo objeto. Exemplo: • No controle de tráfego aéreo temos diversas aeronaves se aproximando de um aeroporto. As aeronaves não se comunicam diretamente umas com as outras. Todas se comunicam com a torre de controle que faz a mediação entre elas.

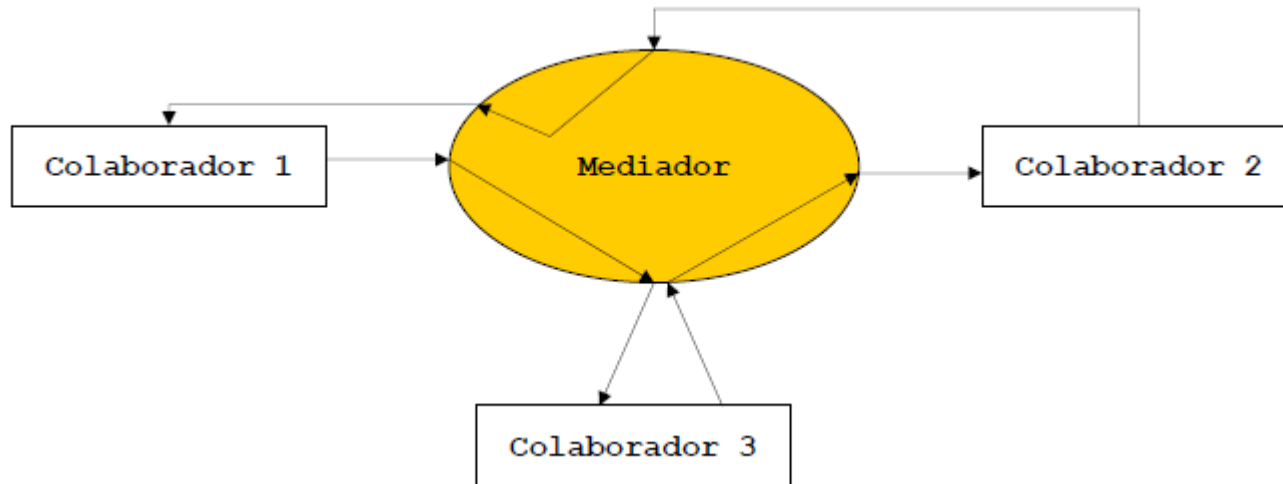
➤ Problema:

- ✓ Como permitir que um grupo de objetos se comunique entre si sem que haja acoplamento entre eles?
- ✓ Como remover o forte acoplamento presente em relacionamentos muitos para muitos?
- ✓ Como permitir que novos participantes sejam ligados ao grupo facilmente?

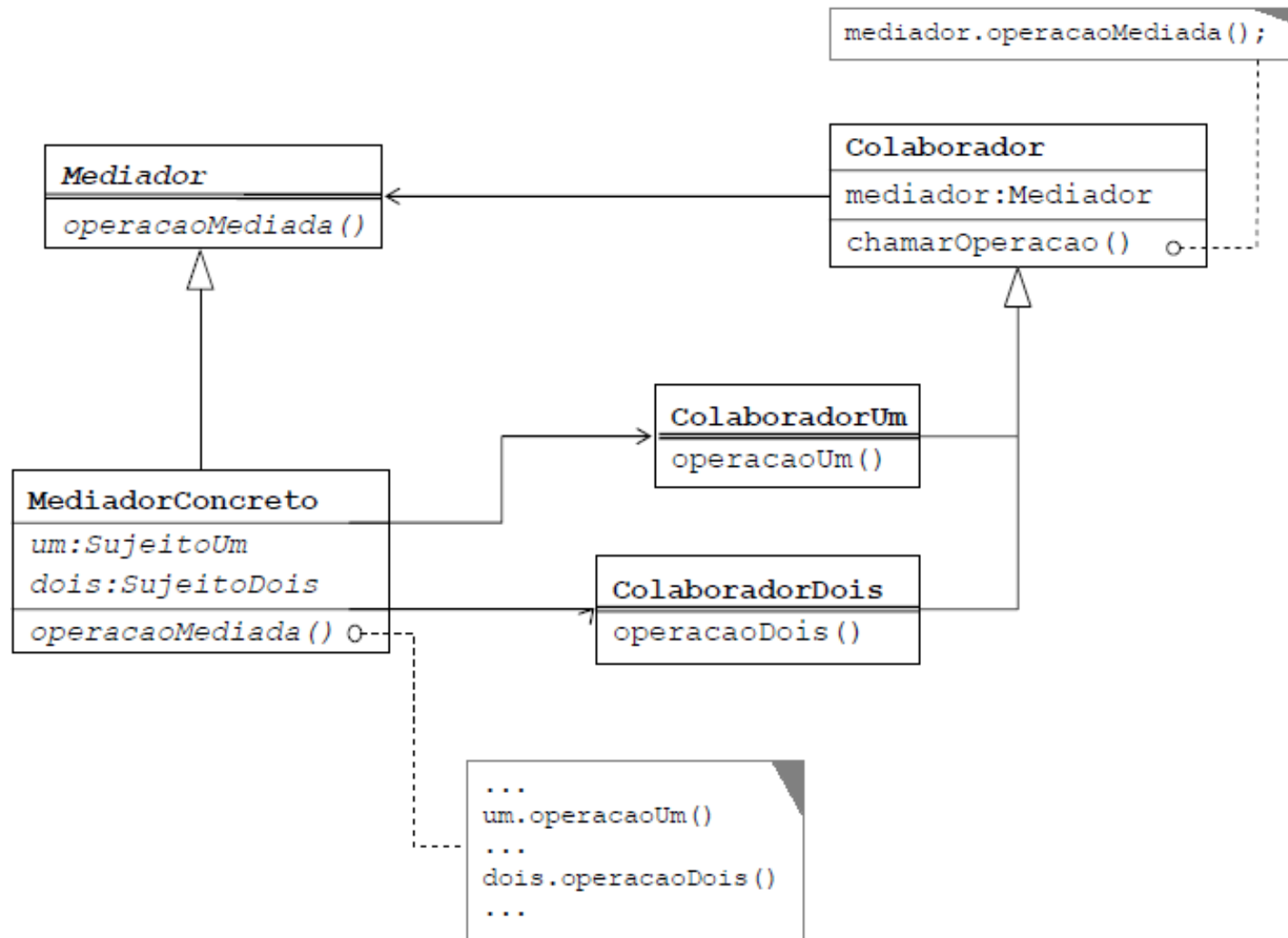


➤ Solução:

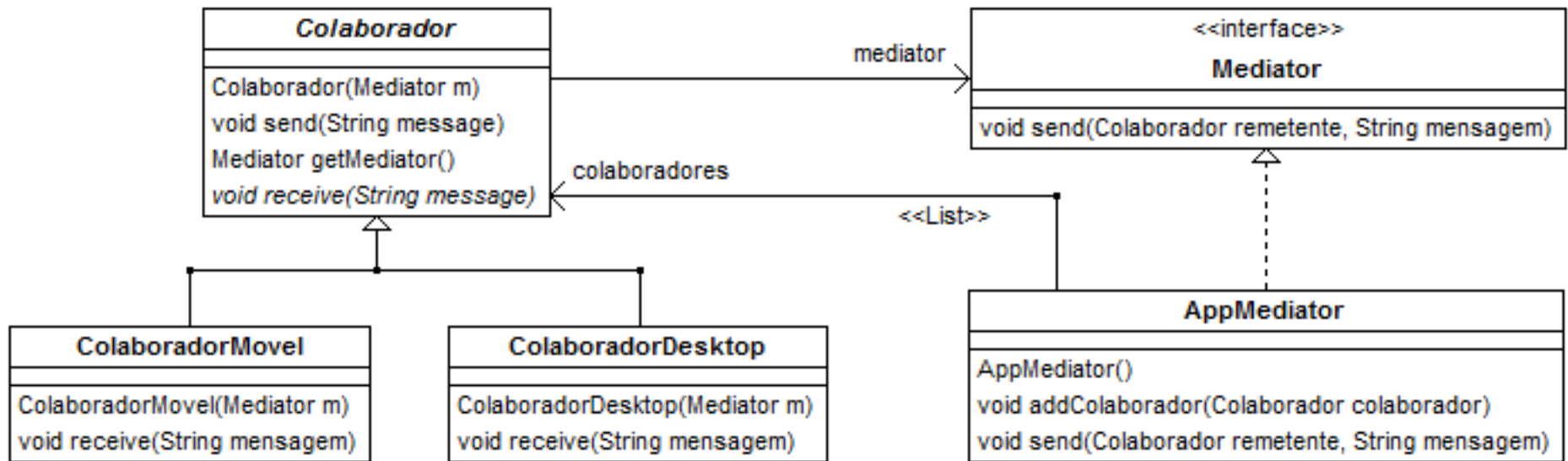
- ✓ Introduzir um mediador para que os objetos se comuniquem sem se conhecerem



➤ Solução:



➤ Exemplo:



➤ Exemplo:

```
public abstract class Colaborador {  
  
    private Mediator mediator;  
  
    public Colaborador(Mediator m) { mediator = m; }  
  
    public void send(String message) { mediator.send(this, message); }  
  
    public Mediator getMediator() { return mediator; }  
  
    public abstract void receive(String message);  
  
}
```

```
public interface Mediator {  
    public void send(Colaborador remetente, String mensagem);  
}
```

➤ Exemplo:

```
public class ColaboradorDesktop extends Colaborador {  
    public ColaboradorDesktop(Mediator m) {  
        super(m);  
    }  
  
    public void receive(String mensagem) {  
        System.out.println("Desktop recebeu: " + mensagem);  
    }  
}
```

```
public class ColaboradorMovel extends Colaborador {  
    public ColaboradorMovel(Mediator m) {  
        super(m);  
    }  
  
    public void receive(String mensagem) {  
        System.out.println("Móvel recebeu: " + mensagem);  
    }  
}
```

➤ Exemplo:

```
public class AppMediator implements Mediator {

    private List<Colaborador> colaboradores;

    public AppMediator() {
        colaboradores = new ArrayList<Colaborador>();
    }

    public void addColaborador(Colaborador colaborador) {
        colaboradores.add(colaborador);
    }

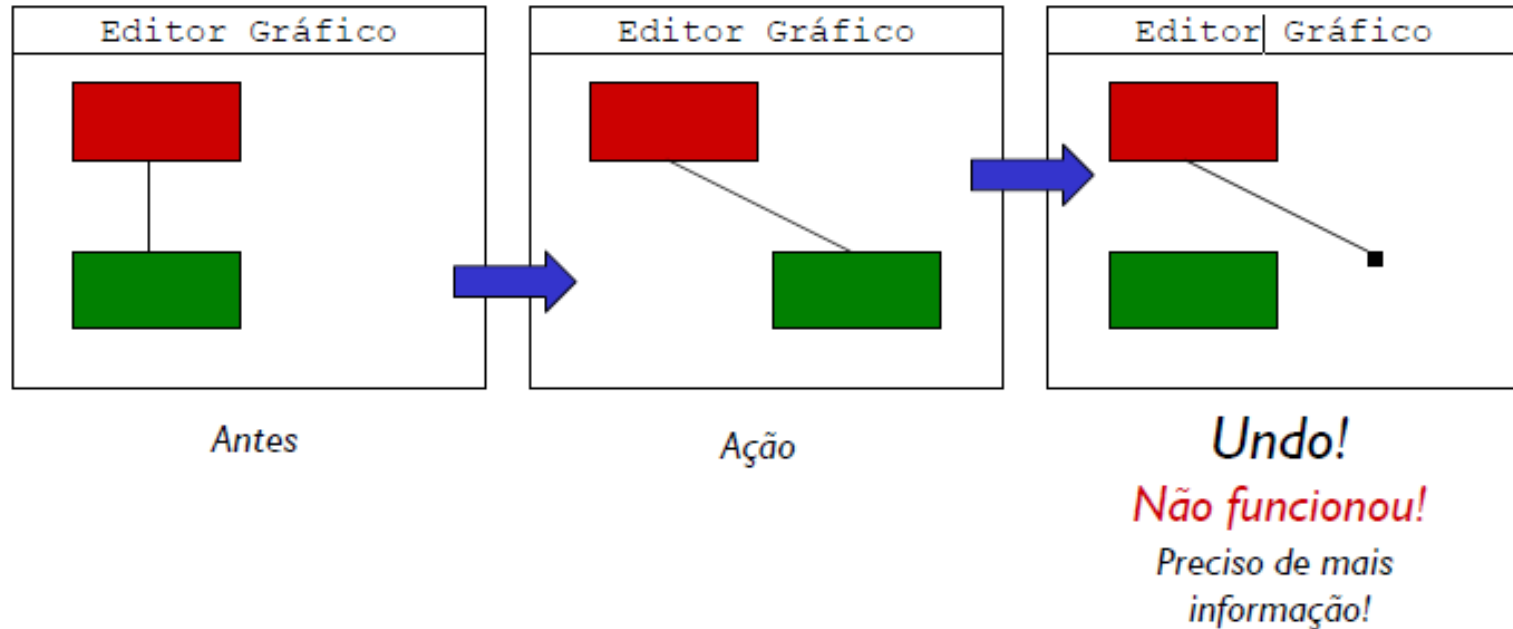
    public void send(Colaborador remetente, String mensagem) {
        for (Colaborador colaborador : colaboradores) {
            // nao envia mensagem para ele mesmo
            if (colaborador != remetente) {
                colaborador.receive(mensagem);
            }
        }
    }
}
```

➤ Exemplo:

```
public class TesteMediator {  
    public static void main(String[] args) {  
        AppMediator mediator = new AppMediator();  
        Colaborador desktop  = new ColaboradorDesktop(mediator);  
        Colaborador movel    = new ColaboradorMovel(mediator);  
  
        mediator.addColaborador(desktop);  
        mediator.addColaborador(movel);  
  
        desktop.send("Alô mundo ! Eu sou um desktop.");  
        movel.send("Alô mundo ! Eu sou um móvel.");  
    }  
}
```

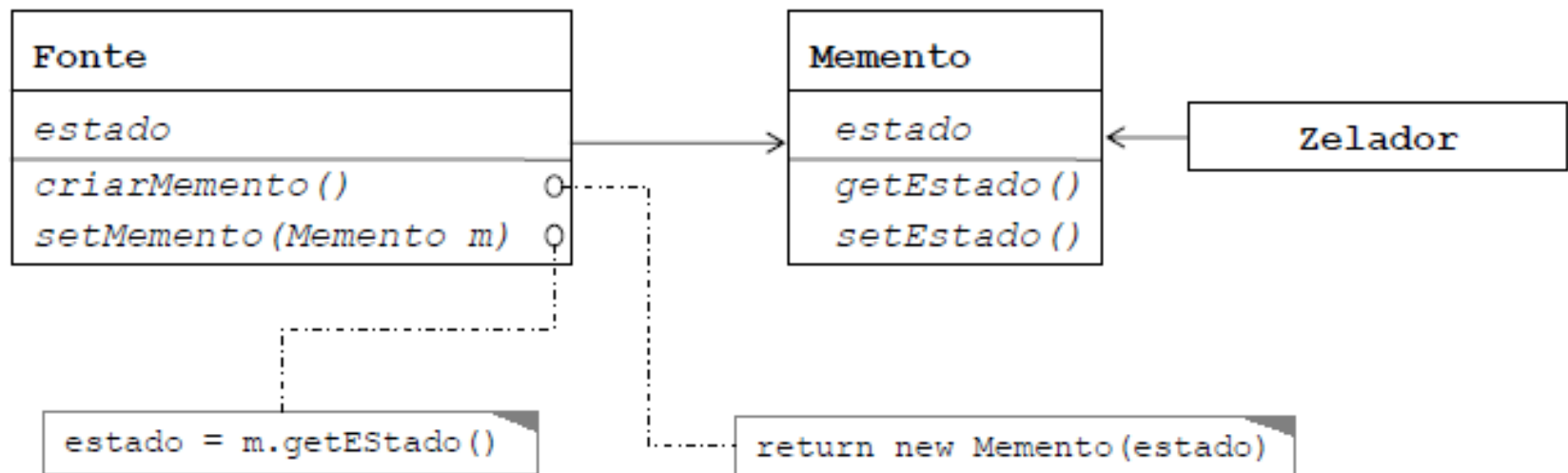
Nome:	Memento
Objetivo:	"Sem violar o encapsulamento, capturar e externalizar o estado interno de um objeto para que o objeto possa ter esse estado restaurado posteriormente" [GoF]
Motivação:	Operações de cancelamento e <i>undo</i> são comuns em sistemas computacionais. Para realizar essas operações é preciso salvar o estado de determinado objeto em um repositório para que ele possa ser restaurado posteriormente, caso seja necessário cancelar ou desfazer uma ou mais operações. Exemplo: • Operações em um editor de textos ou gráfico que podem ser desfeitas pelo usuário (<i>undo</i>). • Voltar um ou mais passos em um software de instalação.

- Problema: é preciso guardar informações suficientes sobre um objeto para desfazer uma operação. Contudo, essas informações não devem ser públicas.

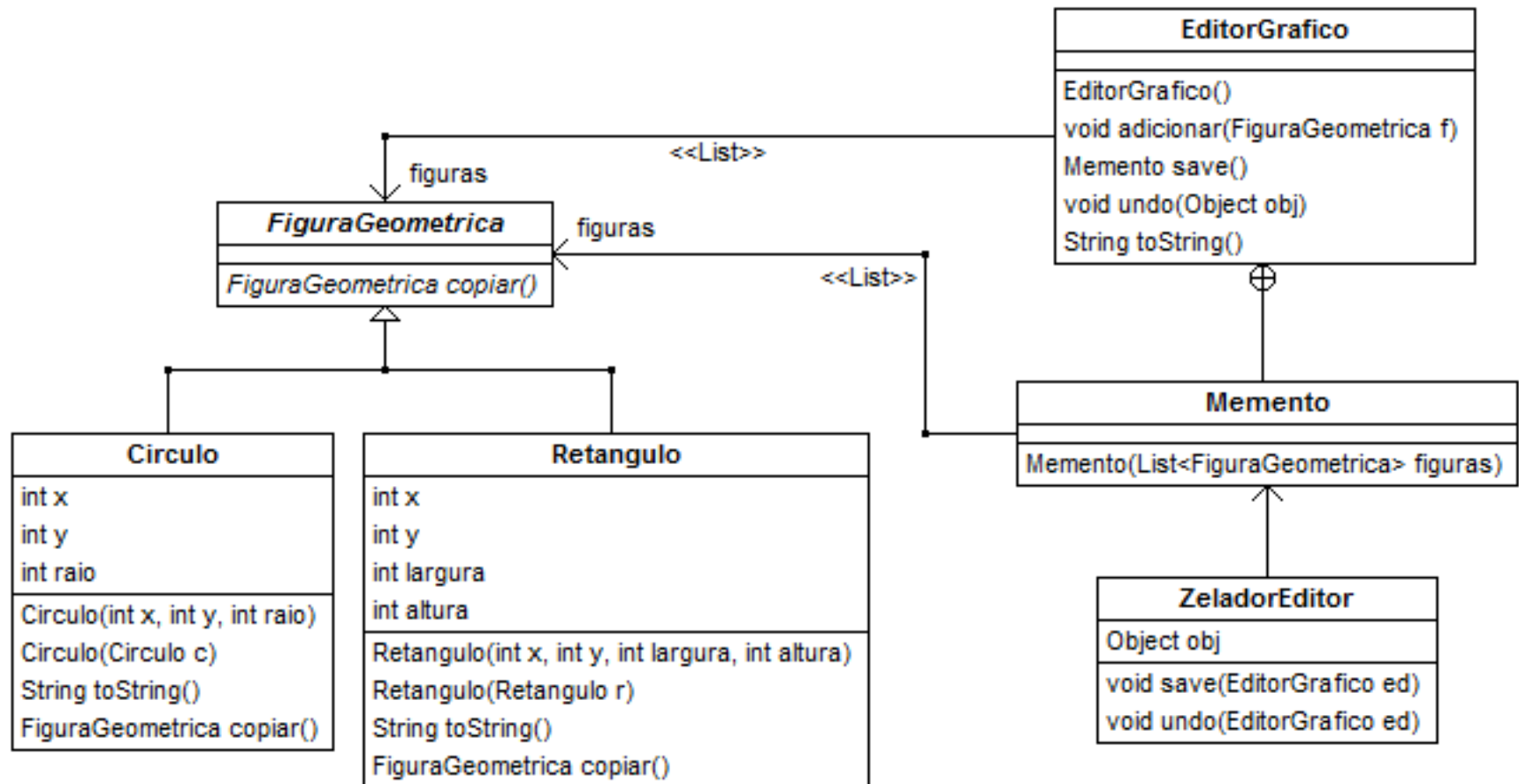


- Solução:
- Um memento é um repositório que guarda um *snapshot* do estado interno de outro objeto, chamado de Fonte:
 - ✓ A Fonte inicializa o memento com informações que caracterizam seu estado atual;
 - ✓ Um mecanismo de *undo* irá requisitar um memento (*snapshot*) da Fonte quando ele necessitar acessar o estado desse objeto;
 - ✓ Só a Fonte tem permissão para recuperar as informações do memento.

➤ Solução:



➤ Exemplo:



➤ Exemplo:

```
public abstract class FiguraGeometrica {  
    protected abstract FiguraGeometrica copiar();  
}
```

```
public class Circulo extends FiguraGeometrica {  
  
    int x, y, raio;  
  
    public Circulo(int x, int y, int raio) {  
        this.x = x;  
        this.y = y;  
        this.raio = raio;  
    }  
  
    public String toString() {  
        return String.format("Circulo(%d, %d, %d)", x, y, raio);  
    }  
  
    protected FiguraGeometrica copiar() {  
        return new Circulo(this.x, this.y, this.raio);  
    }  
}
```

➤ Exemplo:

```
public class Retangulo extends FiguraGeometrica {  
  
    private int x, y, largura, altura;  
  
    public Retangulo(int x, int y, int largura, int altura) {  
        this.x = x;  
        this.y = y;  
        this.largura = largura;  
        this.altura = altura;  
    }  
  
    public String toString() {  
        return String.format("Retangulo(%d, %d, %d, %d)", x, y, largura, altura);  
    }  
  
    protected FiguraGeometrica copiar() {  
        return new Retangulo(this.x, thi.y, this.largura, this.altura);  
    }  
}
```

➤ Exemplo:

```
public class EditorGrafico {  
  
    List<FiguraGeometrica> figuras;  
  
    public EditorGrafico() { figuras = new ArrayList(); }  
  
    public void adicionar(FiguraGeometrica f) { figuras.add(f); }  
  
    public Memento save() { return new Memento(figuras); }  
  
    public void undo(Object obj) {  
        Memento memento = (Memento) obj;  
  
        figuras.clear();  
        for (FiguraGeometrica f : memento.figuras)  
            figuras.add(f.copiar());  
    }  
  
    public String toString() {  
        String str = "";  
        for (FiguraGeometrica f : figuras)  
            str += f.toString() + "\n";  
        return str;  
    }  
}
```

➤ Exemplo: (continuação da classe EditorGrafico)

```
private class Memento {  
    private List<FiguraGeometrica> figuras;  
  
    public Memento(List<FiguraGeometrica> figuras) {  
        // Copia cada figura do editor para o memento  
        this.figuras = new ArrayList();  
        for (FiguraGeometrica f : figuras)  
            this.figuras.add(f.copiar());  
    }  
}
```

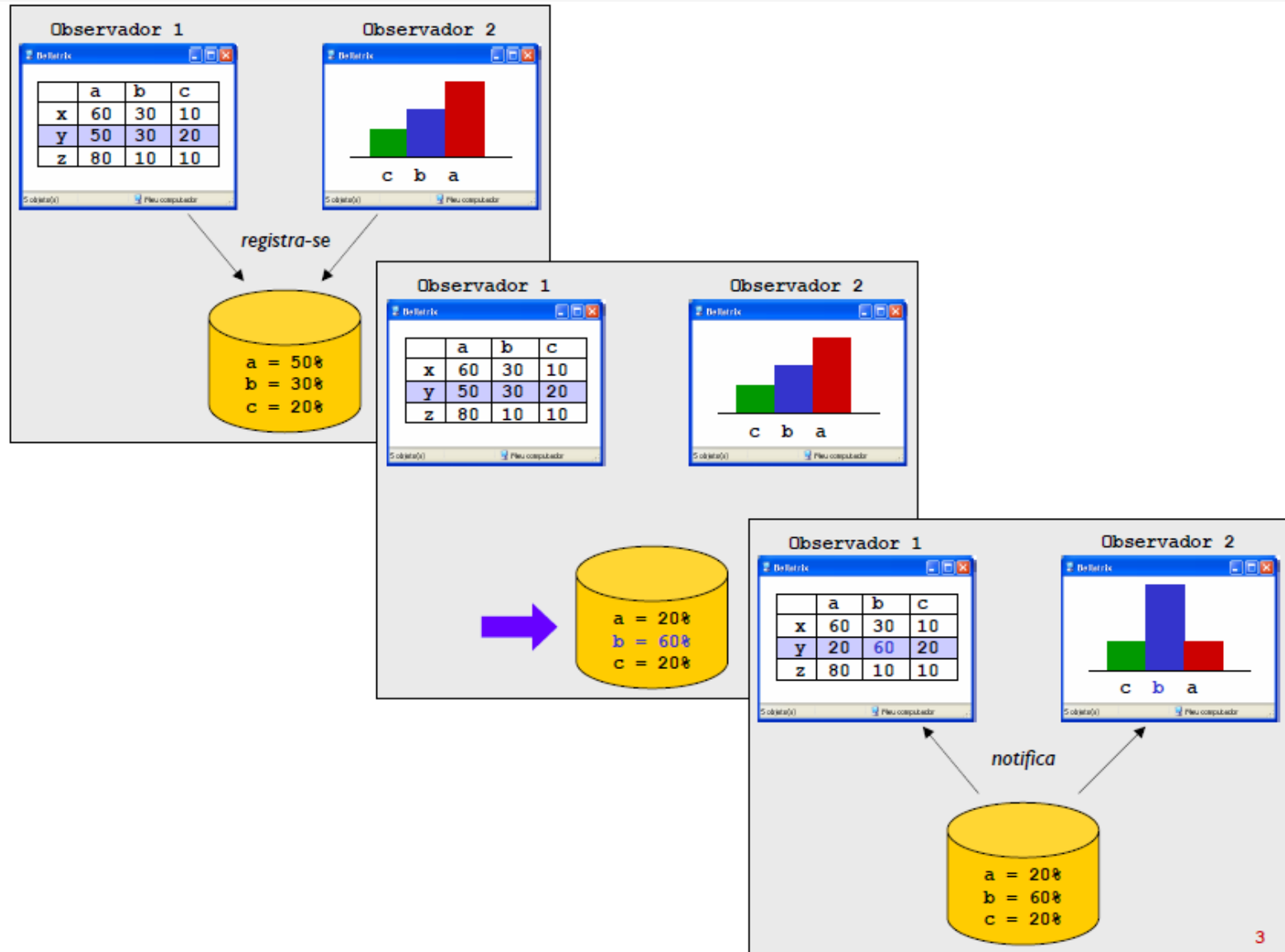
➤ Exemplo:

```
public class ZeladorEditor {  
  
    private Object obj;  
  
    public void save(EditorGrafico ed) {  
        obj = ed.save();  
    }  
  
    public void undo(EditorGrafico ed) {  
        ed.undo(obj);  
    }  
}
```

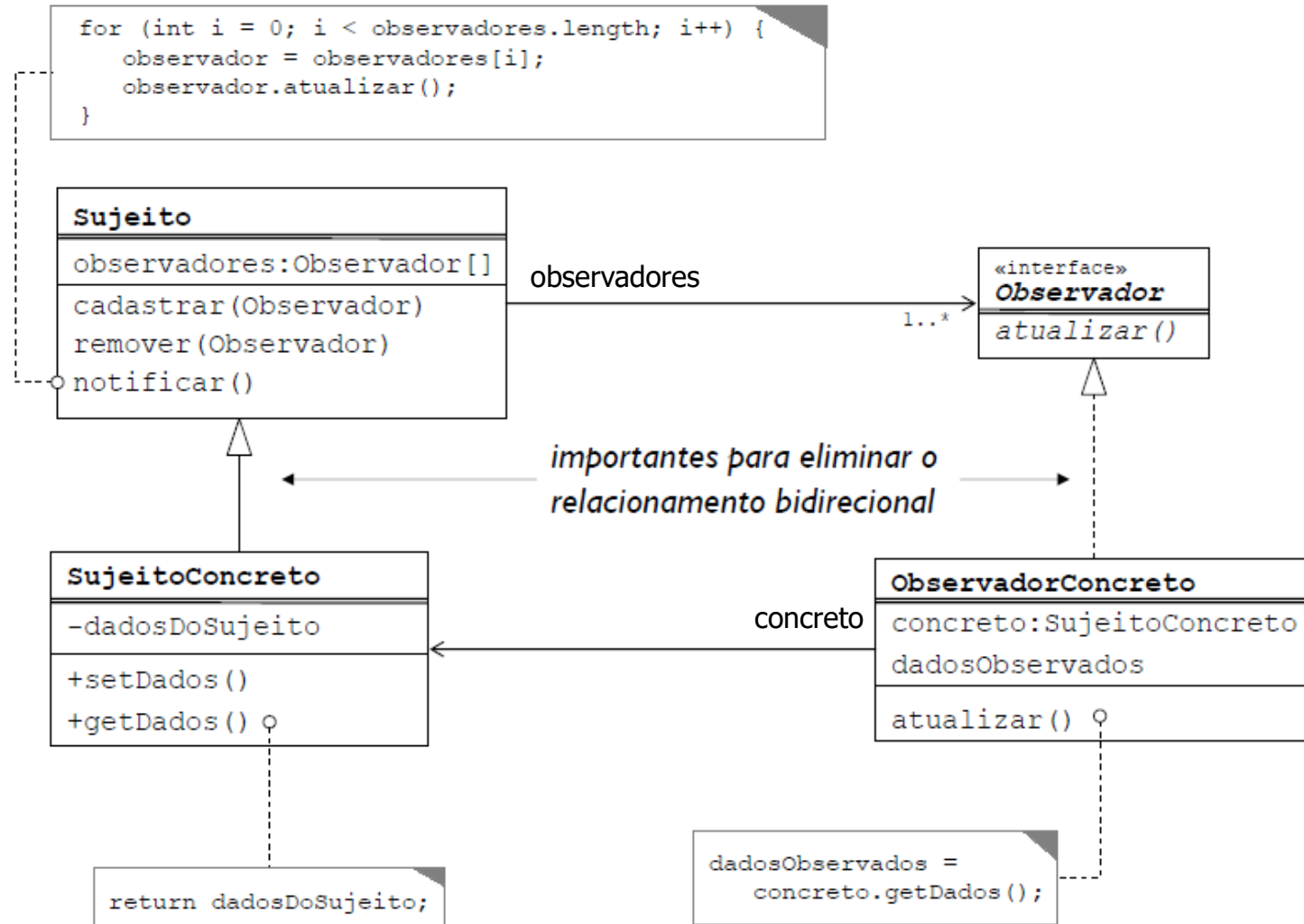
➤ Exemplo:

```
public class TesteMemento {  
    public static void main(String[] args) {  
  
        ZeladorEditor zelador = new ZeladorEditor();  
        EditorGrafico ed = new EditorGrafico();  
  
        ed.adicionar(new Retangulo(5, 10, 20, 50));  
        ed.adicionar(new Circulo(25, 25, 80));  
  
        System.out.print(ed);  
        zelador.save(ed);  
  
        ed.adicionar(new Retangulo(15, 18, 25, 70));  
        ed.adicionar(new Circulo(35, 5, 40));  
  
        System.out.print(ed);  
  
        zelador.undo(ed);  
  
        System.out.print(ed);  
    }  
}
```

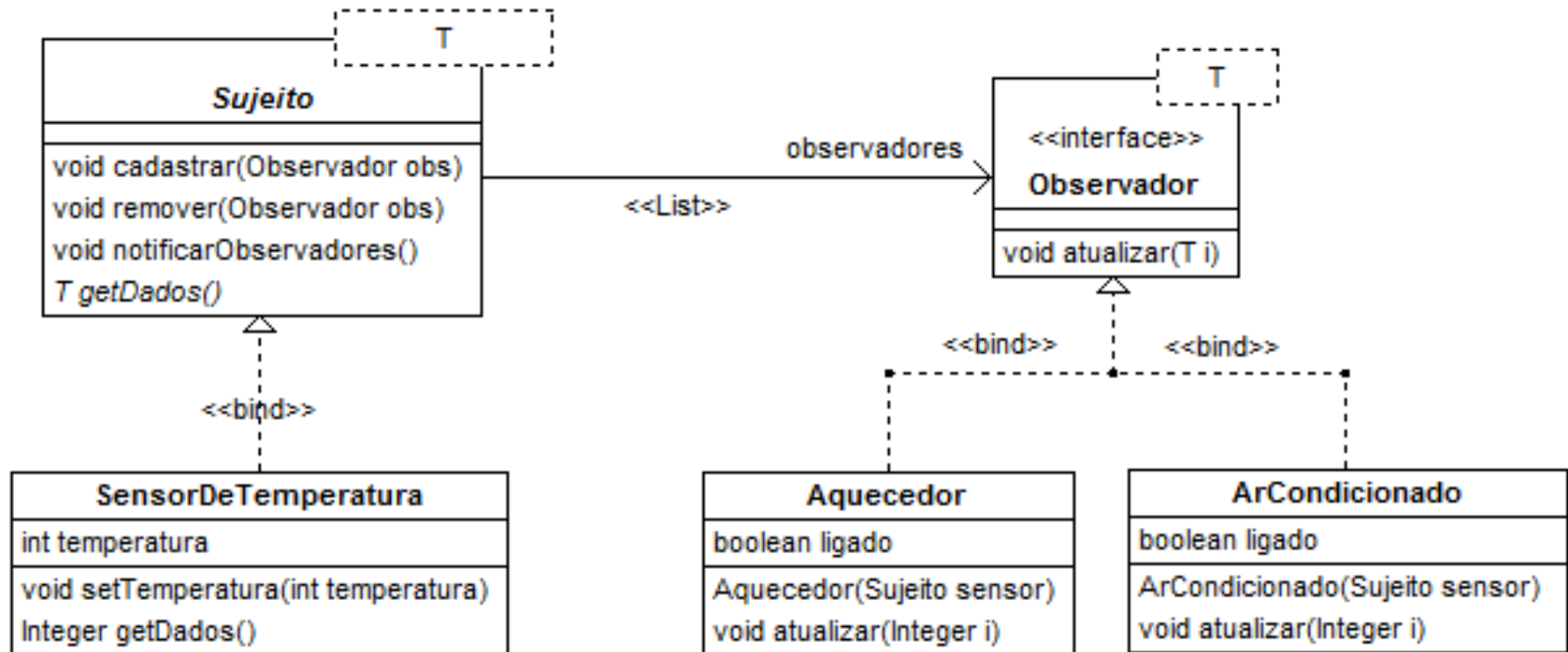
Nome:	Observer
Objetivo:	"Definir uma dependência um-para-muitos entre objetos para que, quando um objeto mudar de estado, todos os seus dependentes sejam notificados e atualizados automaticamente" [GoF]
Motivação:	Em várias situações temos um objeto que possui dependentes e estes dependentes tem que ser notificados caso haja alguma mudança no estado do objeto principal. Exemplos: • Visões distintas sobre um mesmo conjunto de dados precisam ser notificadas caso os dados sejam alterados. • Aplicações em execução precisam ser notificadas caso o SO mude seu estado (desligar display, suspender, hibernar, reiniciar, desligar, etc.)



➤ Solução:



➤ Exemplo:



➤ Exemplo:

```
public abstract class Sujeito<T> {  
    private List<Observador> observadores = new ArrayList<Observador>();  
  
    public void cadastrar(Observador obs) {  
        this.observadores.add(obs);  
    }  
  
    public void remover(Observador obs) {  
        this.observadores.remove(obs);  
    }  
  
    public void notificarObservadores() {  
        for (Observador o : this.observadores) {  
            o.atualizar(getDados());  
        }  
    }  
  
    public abstract T getDados();  
}
```

➤ Exemplo:

```
public interface Observador<T> {  
    void atualizar(T i);  
}
```

➤ Exemplo:

```
public class SensorDeTemperatura extends Sujeito<Integer> {  
    private int temperatura;  
  
    public void setTemperatura(int temperatura) {  
        if (this.temperatura != temperatura) {  
            this.temperatura = temperatura;  
            notificarObservadores();  
        }  
    }  
  
    public Integer getDados() {  
        return temperatura;  
    }  
}
```

➤ Exemplo:

```
public class Aquecedor implements Observador<Integer> {
    private boolean ligado;

    public Aquecedor(Sujeito sensor) {
        this.ligado = false;
        sensor.cadastrar(this);
    }

    public void atualizar(Integer i) {
        if (i <= 16 && !ligado) {
            System.out.printf("Temp: %d -> Ligando o aquecedor...\n", i);
            this.ligado = true;
        } else if (i > 16 && ligado) {
            System.out.printf("Temp: %d -> Desligando o aquecedor...\n", i);
            this.ligado = false;
        }
    }
}
```

➤ Exemplo:

```
public class ArCondicionado implements Observador<Integer> {
    private boolean ligado;

    public ArCondicionado(Sujeito sensor) {
        this.ligado = false;
        sensor.cadastrar(this);
    }

    public void atualizar(Integer i) {
        if (i > 24 && !ligado) {
            System.out.printf("Temp: %d -> Ligando o ar...\n", i);
            this.ligado = true;
        } else if (i <= 16 && ligado) {
            System.out.printf("Temp: %d -> Desligando o ar...\n", i);
            this.ligado = false;
        }
    }
}
```


➤ Exemplo:

```
public class TesteObserver {  
    public static void main(String[] args) {  
        SensorDeTemperatura sensor = new SensorDeTemperatura();  
        ArCondicionado ar = new ArCondicionado(sensor);  
        Aquecedor aquecedor = new Aquecedor(sensor);  
  
        sensor.setTemperatura(22);  
        sensor.setTemperatura(29);  
        sensor.setTemperatura(15);  
        sensor.setTemperatura(22);  
    }  
}
```

➤ Saída:

Temp: 29 -> Ligando o ar...
Temp: 15 -> Desligando o ar...
Temp: 15 -> Ligando o aquecedor...
Temp: 22 -> Desligando o aquecedor...

➤ Diferenças entre Chain of Responsibility, Mediator e Observer:

✓ Chain of Responsibility

- Um conjunto de objetos é responsável por atender requisições de diversos tipos.
- Quando uma requisição é gerada é preciso descobrir qual objeto do conjunto é responsável por atendê-la e repassar essa requisição ao objeto.
- O objeto que gera a requisição não faz parte do conjunto de objetos que deve atendê-la.

✓ Mediator

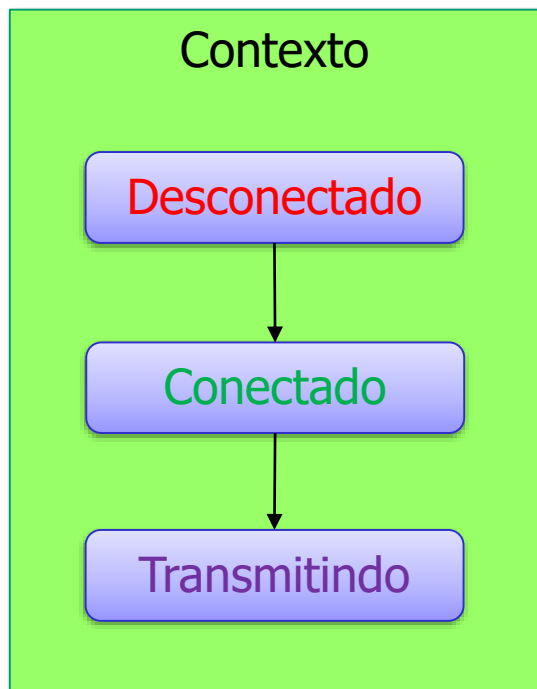
- Vários objetos de um conjunto precisam se comunicar uns com os outros.
- Quando um objeto envia uma mensagem, ela deve ser recebida por todos os demais objetos do conjunto.
- O objeto remetente da mensagem faz parte do conjunto de objetos.

✓ Observer

- Um conjunto de objetos está interessado em um evento.
- Quando um evento é gerado, todos os objetos interessados são avisados da sua ocorrência.
- O objeto que gera o evento não faz parte do conjunto de objetos interessados.

Nome:	State
Objetivo:	"Permitir a um objeto alterar o seu comportamento quando o seu estado interno mudar. O objeto irá aparentar mudar de classe" [GoF]
Motivação:	Objetos que mudam de estado, podem ter comportamentos que também mudam de acordo com o estado. Exemplos: • Acionar a opção de ligar quando a TV está desligada fará com que ela seja ligada, porém se a TV já estiver ligada, nada acontecerá. • Quando o usuário pressiona Ctrl-C e a área de transferência contem algum dado, esse dado é copiado, senão nada acontece. • Quando o aluno aciona a opção de "matricular em disciplina" e a matrícula está ativa, o sistema apresenta a lista de disciplinas. Se a matrícula estiver inativa, será apresentada uma mensagem de erro.

➤ Problema:



operação

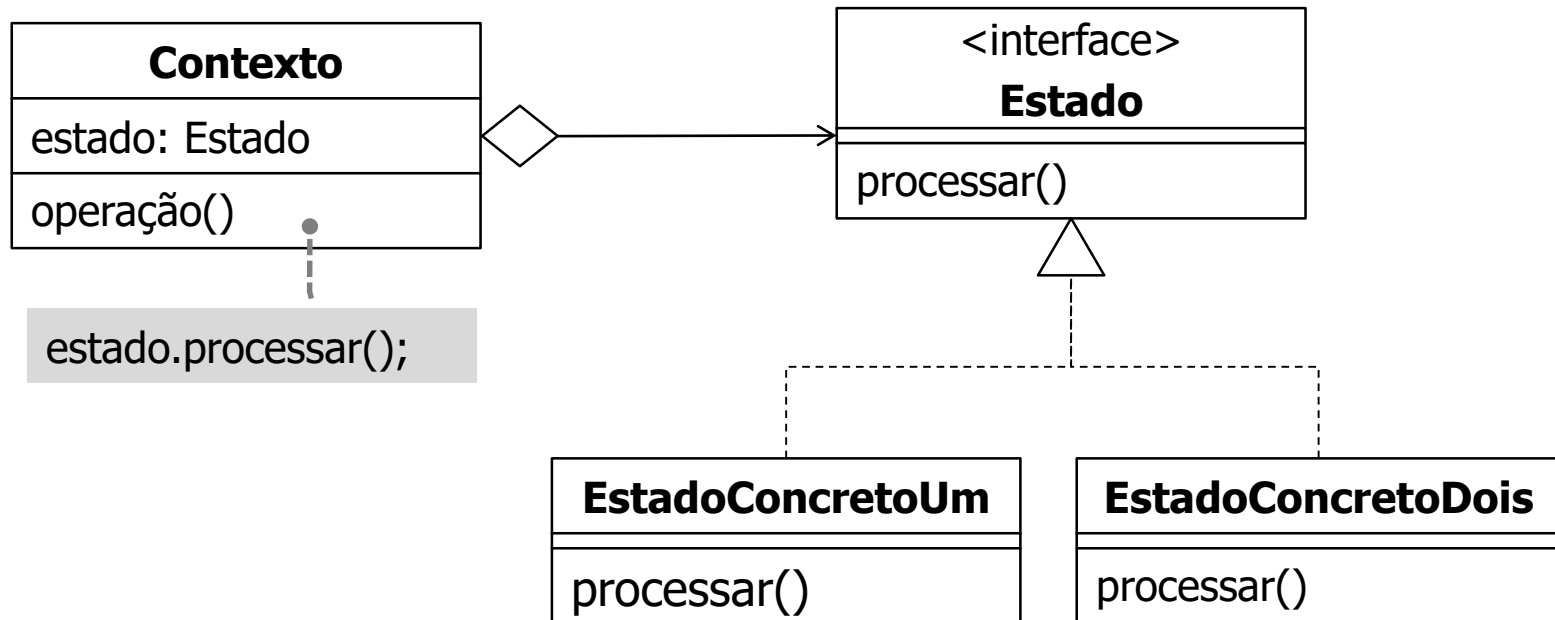
```
if (estado == desconectado)
    façaIsto();
else if (estado == conectado)
    façaAquilo();
else
    façaOutraCoisa();
```



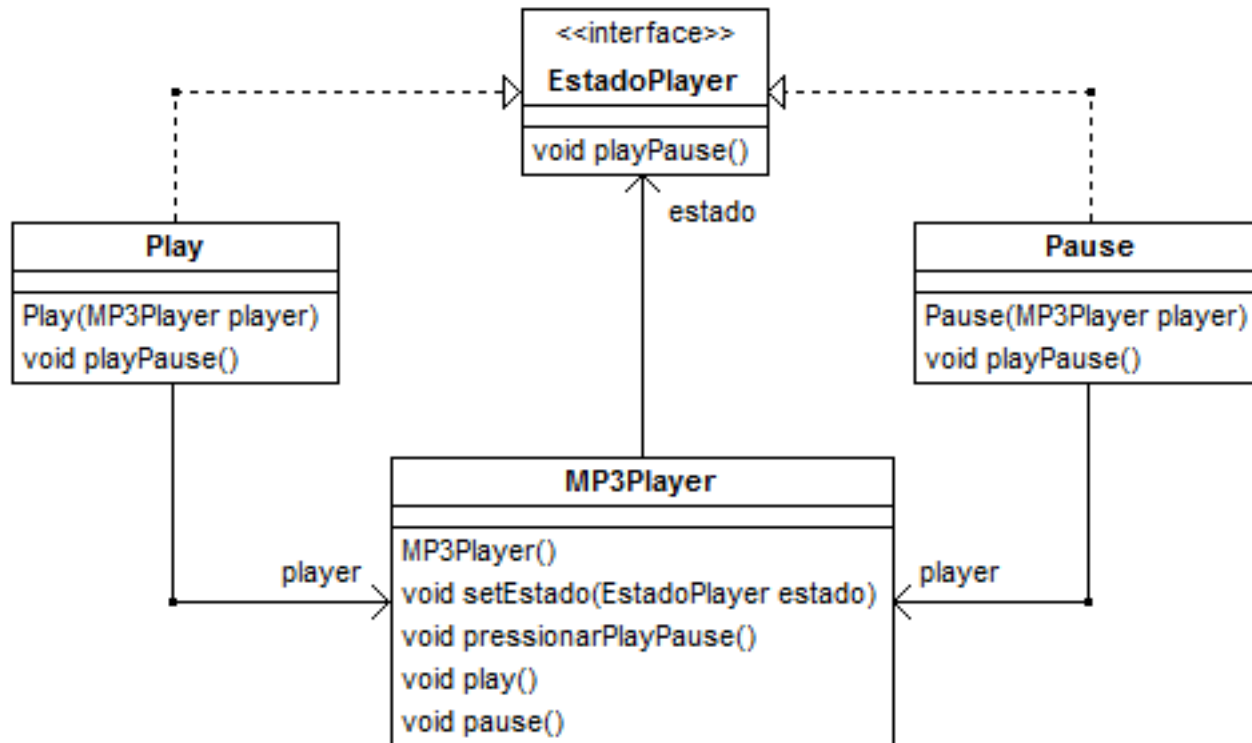
operação

```
estado.fazça();
```

➤ Solução:



➤ Exemplo:



➤ Exemplo:

```
public interface EstadoPlayer {  
    public void playPause();  
}
```

```
public class MP3Player {  
    private EstadoPlayer estado;  
  
    public MP3Player() {  
        setEstado(new Pause(this));  
    }  
  
    public void setEstado(EstadoPlayer estado) { this.estado = estado; }  
  
    public void pressionarPlayPause() { estado.playPause(); }  
  
    public void play() { System.out.println("Play..."); }  
  
    public void pause() { System.out.println("Pause..."); }  
}
```

➤ Exemplo:

```
public class Pause implements EstadoPlayer {
    MP3Player player;

    public Pause(MP3Player player) {
        this.player = player;
    }

    public void playPause() {
        player.play();
        player.setEstado(new Play(player));
    }
}
```


➤ Exemplo:

```
public class Play implements EstadoPlayer {
    MP3Player player;

    public Play(MP3Player player) {
        this.player = player;
    }

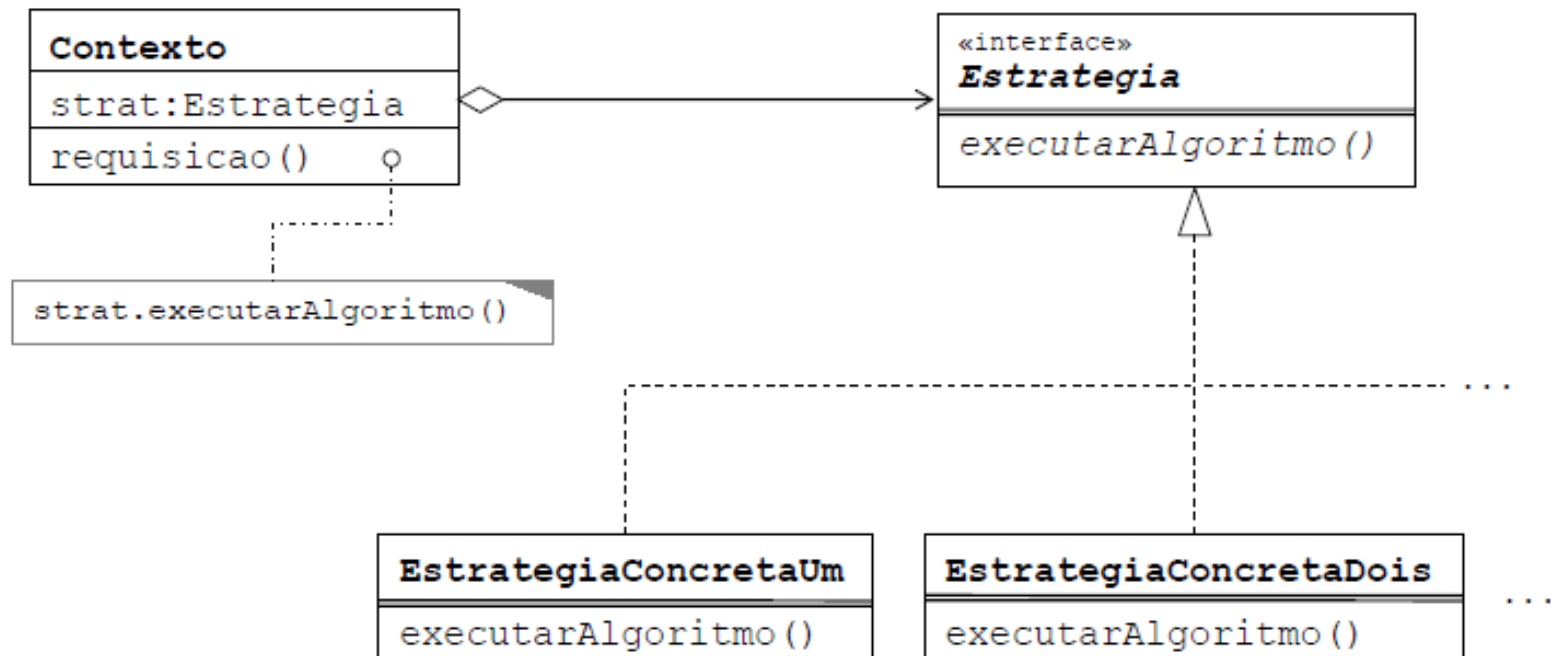
    public void playPause() {
        player.pause();
        player.setEstado(new Pause(player));
    }
}
```

➤ Exemplo:

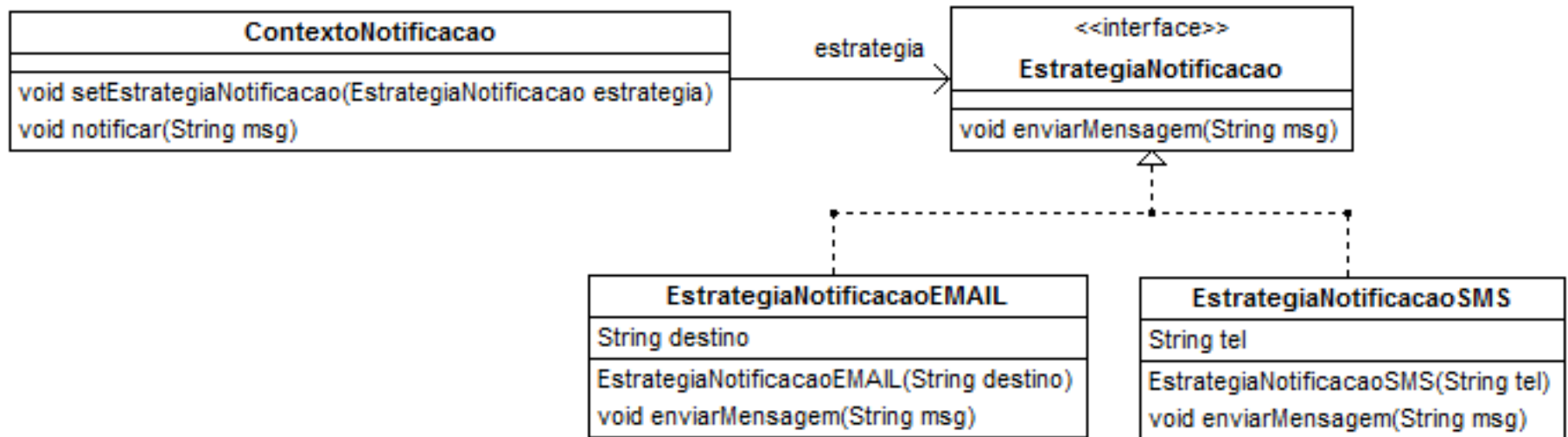
```
public class TesteState {  
    public static void main(String[] args) {  
        MP3Player player = new MP3Player();  
  
        player.pressionarPlayPause();  
        player.pressionarPlayPause();  
        player.pressionarPlayPause();  
        player.pressionarPlayPause();  
    }  
}
```

Nome:	Strategy
Objetivo:	"Definir uma família de algoritmos, encapsular cada um e fazê-los intercambiáveis. Strategy permite que algoritmos mudem independentemente dos clientes que os utilizam" [GoF]
Motivação:	Algumas vezes precisamos alterar o algoritmo a ser executado em um método ou os dados manipulados por um algoritmo baseados em alguma condição . Exemplos: • Ordenar uma lista de números inteiros ou uma lista de caracteres. • Criar uma notificação e enviá-la por email ou por SMS

➤ Solução:



➤ Exemplo:



➤ Exemplo:

```
public interface EstrategiaNotificacao {  
    public void enviarMensagem(String msg);  
}
```

```
public class EstrategiaNotificacaoSMS implements EstrategiaNotificacao {  
    private String tel;  
  
    public EstrategiaNotificacaoSMS(String tel) {  
        this.tel = tel;  
    }  
  
    public void enviarMensagem(String msg)  
    {  
        System.out.printf("Enviar por SMS para %s: %s\n", tel, msg);  
    }  
}
```

➤ Exemplo:

```
public class EstrategiaNotificacaoEMAIL implements EstrategiaNotificacao {
    private String destino;

    public EstrategiaNotificacaoEMAIL (String destino) {
        this.destino = destino;
    }

    public void enviarMensagem(String msg)
    {
        System.out.printf("Enviar por EMAIL para %s: %s\n", destino, msg);
    }
}
```

➤ Exemplo:

```
public class ContextoNotificacao {  
    private EstrategiaNotificacao estrategia;  
  
    public void setEstrategiaNotificacao(EstrategiaNotificacao estrategia) {  
        this.estrategia = estrategia;  
    }  
  
    public void notificar(String msg) {  
        msg = "Caro cliente, " + msg +  
            ". Qualquer dúvida entre em contato com o SAC";  
  
        estrategia.enviarMensagem(msg);  
    }  
}
```

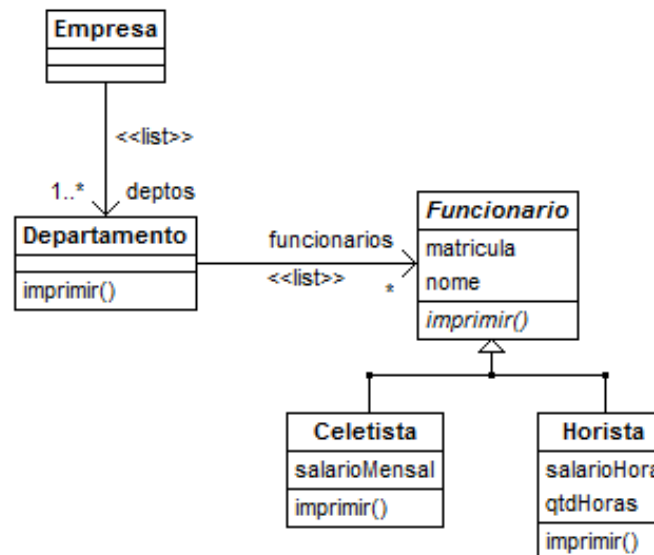

➤ Exemplo:

```
public class TesteStrategy {  
  
    public static void main(String[] args) {  
        ContextoNotificacao ctx = new ContextoNotificacao();  
  
        // Envia por SMS  
        ctx.setEstrategiaNotificacao(  
            new EstrategiaNotificacaoSMS("993328888"));  
  
        ctx.notificar("sua encomenda foi despachada hoje");  
  
        // Envia por Email  
        ctx.setEstrategiaNotificacao(  
            new EstrategiaNotificacaoEMAIL("destino@Hotmail.com"));  
  
        ctx.notificar("sua encomenda foi despachada hoje");  
    }  
}
```

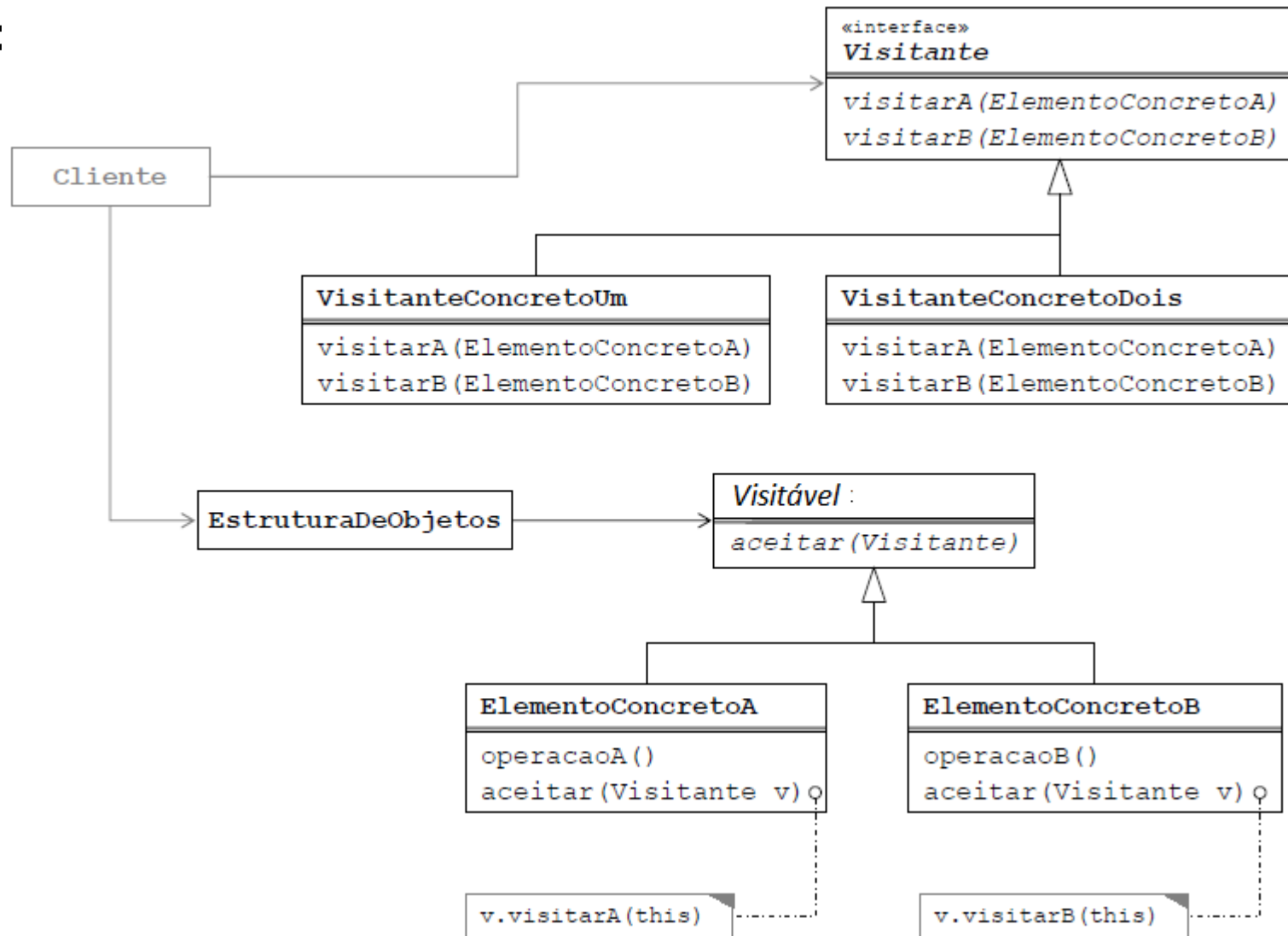
Nome:	Visitor
Objetivo:	"Representar uma operação a ser realizada sobre os elementos de uma estrutura de objetos. Visitor permite definir uma nova operação sem mudar as classes dos elementos nos quais opera" [GoF]
Motivação:	Estruturas de objetos são amplamente usado em programação OO. Em alguns casos, essa estrutura possui objetos de diferentes tipos e uma mesma operação precisa ser executada sobre esses objetos, sem distinguir seu tipo. Exemplo: • Imprimir um relatório de funcionários, por departamento, onde alguns são celetistas e outros horistas. A impressão é diferente para cada tipo de funcionário.

➤ Problema:

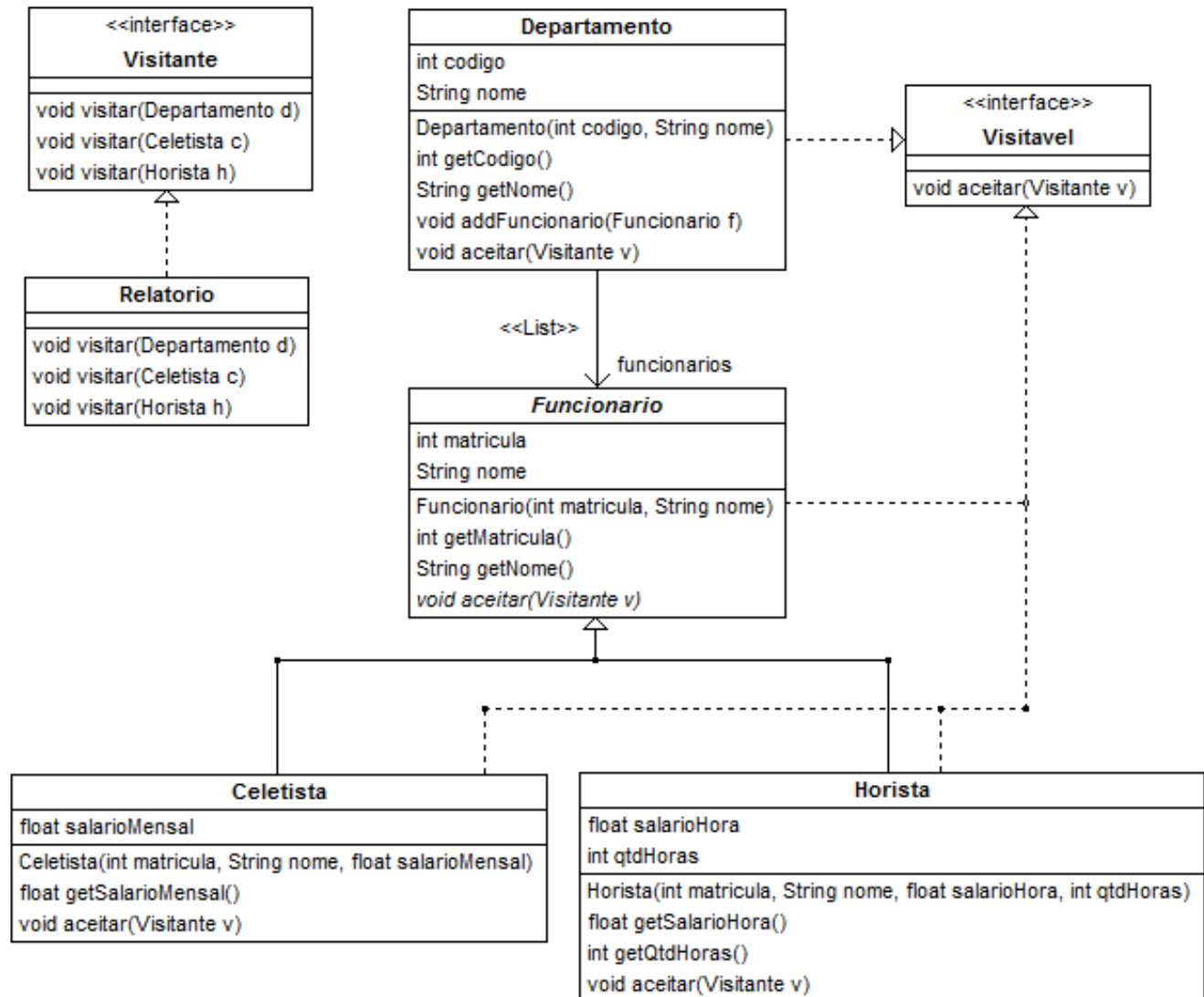
- ✓ Ao percorrer uma estrutura de objetos não podemos usar o conceito de polimorfismo, porque os objetos não pertencem necessariamente à mesma hierarquia.
- ✓ Por outro lado, não desejamos implementar a operação desejada em cada objeto para não fugir do Princípio da Responsabilidade Única.
- ✓ Também é questionável o uso do operador *instanceof* para decidir qual operação deve ser realizada.



➤ Solução:



➤ Exemplo:



➤ Exemplo:

```
public interface Visitante {  
    public void visitar(Departamento d);  
    public void visitar(Celetista c);  
    public void visitar(Horista h);  
}
```

```
public interface Visitavel {  
    public void aceitar(Visitante v);  
}
```

➤ Exemplo:

```
public abstract class Funcionario implements Visitavel {  
  
    private int matricula;  
    private String nome;  
  
    public Funcionario(int matricula, String nome) {  
        this.matricula = matricula;  
        this.nome = nome;  
    }  
  
    public int getMatricula() { return matricula; }  
  
    public String getNome() { return nome; }  
  
    public abstract void aceitar(Visitante v);  
}
```

➤ Exemplo:

```
public class Celetista extends Funcionario implements Visitavel {  
  
    private float salarioMensal;  
  
    public Celetista(int matricula, String nome, float salarioMensal) {  
        super(matricula, nome);  
        this.salarioMensal = salarioMensal;  
    }  
  
    public float getSalarioMensal() { return salarioMensal; }  
  
    public void aceitar(Visitante v) {  
        v.visitar(this);  
    }  
}
```


➤ Exemplo:

```
public class Horista extends Funcionario implements Visitavel {

    private float salarioHora;
    private int qtdHoras;

    public Horista(int matricula, String nome, float salarioHora,
                  int qtdHoras) {
        super(matricula, nome);
        this.salarioHora = salarioHora;
        this.qtdHoras = qtdHoras;
    }

    public float getSalarioHora() { return salarioHora; }

    public int getQtdHoras() { return qtdHoras; }

    public void aceitar(Visitante v) {
        v.visitar(this);
    }
}
```

➤ Exemplo:

```
public class Departamento implements Visitavel {
    private int codigo;
    private String nome;
    private List<Funcionario> funcionarios;

    public Departamento(int codigo, String nome) {
        this.codigo = codigo;
        this.nome = nome;
        this.funcionarios = new ArrayList();
    }

    public int getCodigo() { return codigo; }
    public String getNome() { return nome; }
    public void addFuncionario(Funcionario f) { funcionarios.add(f); }

    public void aceitar(Visitante v) {
        v.visitar(this);
        for (Funcionario f : funcionarios)
            f.aceitar(v);
    }
}
```

➤ Exemplo:

```
public class Relatorio implements Visitante {
    public void visitar(Departamento d) {
        System.out.printf("Departamento: %d - %s\n",
                           d.getCodigo(), d.getNome());
        System.out.println("-----");
        System.out.println("Matricula Nome      Mensal      Hora   Qtd Horas");
        System.out.println("-----");
    }

    public void visitar(Celetista c) {
        System.out.printf(" %7d %-15s %8.2f      -      -\n",
                           c.getMatricula(), c.getNome(), c.getSalarioMensal());
    }

    public void visitar(Horista h) {
        System.out.printf(" %7d %-15s      -      %8.2f      %3d\n",
                           h.getMatricula(), h.getNome(),
                           h.getSalarioHora(), h.getQtdHoras());
    }
}
```

➤ Exemplo:

```
public class TesteVisitor {  
    public static void main(String[] args) {  
        Visitante r = new Relatorio();  
        Departamento d = new Departamento(100, "RH");  
  
        d.addFuncionario(new Celetista(100, "João", 1500));  
        d.addFuncionario(new Celetista(200, "André", 3500));  
        d.addFuncionario(new Horista( 300, "Beatriz", 100, 160));  
        d.addFuncionario(new Horista( 400, "Carlos", 80, 120));  
        d.addFuncionario(new Celetista(500, "Ana", 5500));  
        d.addFuncionario(new Horista( 600, "Paula", 120, 180));  
  
        d.aceitar(r);  
    }  
}
```

➤ Diferenças entre **Iterator** e **Visitor**:

✓ **Iterator**

- Percorre uma coleção de objetos.
- Simplesmente retorna os objetos da coleção.
- Todos os objetos da coleção são do mesmo tipo ou possuem um ancestral em comum, ou seja, fazem parte da mesma hierarquia.

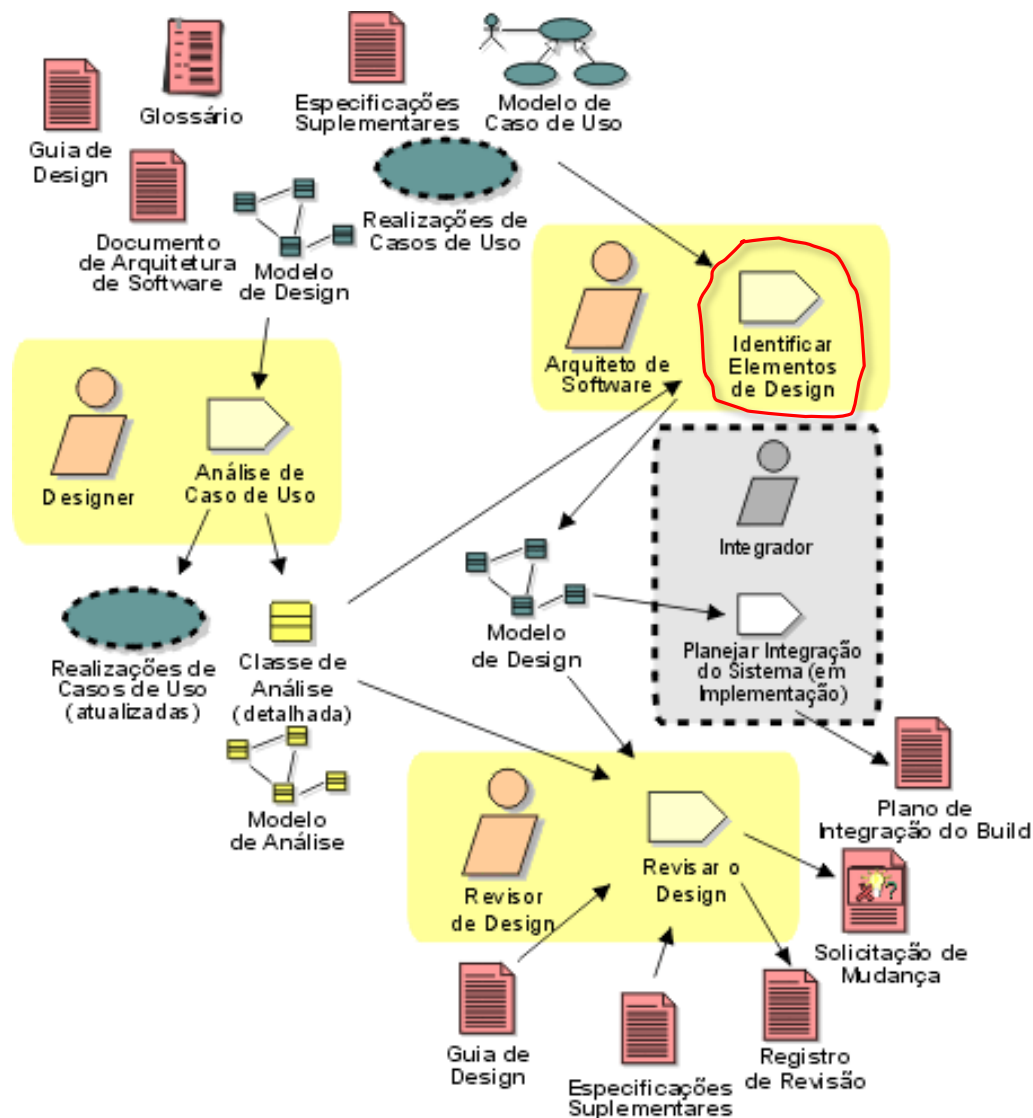
✓ **Visitor**

- Percorre uma estrutura de objetos (pode ser uma coleção ou não).
- Permite executar uma operação sobre os objetos da estrutura.
- Os objetos da estrutura não precisam ser do mesmo tipo ou pertencerem a uma mesma hierarquia. É possível trabalhar com objetos completamente distintos.

Projeto de Software Orientado a Objetos

11ª parte

Prof. Jobson Massollar



- O resultado da Análise de Casos de Uso (etapa anterior) apresenta uma **visão local**, ou seja, as responsabilidades de um grupo de classes para realizar **um** caso de uso específico.
- É preciso agora **unificar** essas visões para obter um modelo de classes que atenda a **todos os casos de uso do sistema**.
- Essa unificação começa pela atividade **Identificar Elementos de Design**.
- Nessa atividade as Classes de Análise serão refinadas em **Classes de Design** e **Subsistemas**.
- Classes de Design e Subsistemas são elementos de *design* que podem ser **implementados** em um ambiente de execução, ou seja, Classes de Design e Subsistemas são representações de **elementos implementáveis**

➤ De forma resumida, essa atividade envolve os seguintes passos:

1. Identificar Classes e Subsistemas
2. Identificar interfaces dos subsistemas
3. Identificar oportunidades de reuso



- Nesse passo, o projetista irá decidir se uma **Classe de Análise** será transformada em uma ou mais **Classes de Design** ou então em um **Subsistema**.
- O diagrama usado nessa atividade é o **Diagrama de Sequência**, ou seja, os diagramas elaborados na atividade de Análise do Caso de Uso serão **detalhados** e as classes de Análise será **substituídas** por **classes implementáveis** ou **subsistemas**.
- Vários detalhes referentes à arquitetura da aplicação e às tecnologias adotadas para a implementação deverão ser levados em consideração, uma vez que as classes geradas ao final do design serão efetivamente implementadas posteriormente.

➤ Fatores que influenciam as decisões de design:

✓ Princípios de Projeto OO:

- Dependência
- Congeneridade
- Acoplamento
- Coesão
- SOLID

✓ Padrão arquitetural adotado:

- MVC
- Cliente-Servidor
- 3 ou mais camadas
- etc.

✓ Padrões de Projeto:

- GRASP
- GoF

✓ Tipo da aplicação:

- Aplicação gráfica "convencional" (WIMP)
- Aplicação móvel
- Aplicação web
- etc.

✓ Tecnologias adotadas:

- Para persistência dos dados
- Para comunicação com sistemas legados
- Para a interface gráfica
- Para o envio de mensagens
- etc.

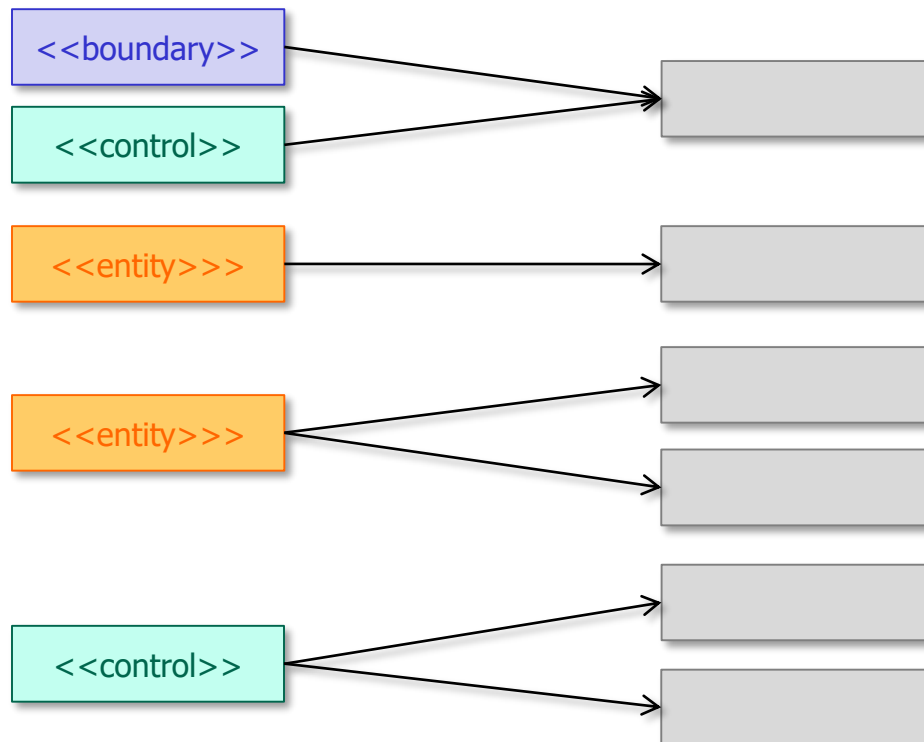
✓ Linguagens/frameworks:

- ASP.NET WinForms
- ASP.NET MVC
- Java Swing ou JavaFX
- JSP/Servlet
- Python/Django
- PHP 5
- Hibernate
- Spring
- etc.

- O relacionamento entre Classes de Análise e Classes de Design/Subsistemas é de **muitos-para-muitos**.

Classes de Análise

Classes de Design e Subsistemas



➤ Como realizar esse mapeamento?

1. Uma Classe de Análise **simples**, que representa **uma única abstração**, tende a ser mapeada para uma única Classe de Design.

✓ Exemplos:

- a) Normalmente as classes de **Entidade** seguem essa regra e transformam-se diretamente em Classes de Design.
- b) Classes de **Fronteira** que representam **janelas** ou **páginas** com as quais o usuário interage durante um caso de uso também tendem a seguir essa regra.

- Como realizar esse mapeamento?
 2. Várias Classes de Análise **muito simples** podem ser **combinadas** em uma única Classe de Design.
 - ✓ Exemplos:
 - a) Classes de **Fronteira** e **Controle** associadas a um mesmo caso de uso podem ser **combinadas** em uma única Classe de Design.
 - b) Classes de **Controle** simples associadas a casos de uso **semelhantes** podem ser combinadas em uma única Classe de Design. Exemplo: CRUD, ou seja, uma mesma classe pode ficar responsável pelo controle das operações de inclusão, recuperação, alteração e exclusão.

- Como realizar esse mapeamento?
- 3. Em geral, uma Classe de Análise **complexa** pode:
 - ✓ Ser dividida em várias Classes de Design;
 - ✓ Ser dividida em várias Classes de Design representadas por um Pacote;
 - ✓ Ser transformada em um Subsistema;
 - ✓ Alguma combinação das opções anteriores.
- ✓ O que é uma classe **complexa**?
 - Classe que realiza diversas responsabilidades distintas.
 - Classe que representa mais de um conceito.
 - Classe que representa conceitos de negócio e tecnologia.

Importante: a definição de subsistema e as diferenças entre subsistemas e pacotes será vista posteriormente.

I. Mapeamento de Classes de Fronteira de Interface com o Usuário

- ✓ Esse mapeamento é **muito dependente** da tecnologia adotada para implementação. Exemplos:

Tecnologia	Classe de Fronteira se transforma em...
Java Swing	Classe que estende JFrame ou JDialog
JSP	Página JSP
JSF	Arquivo XHTML combinado com Java Beans
ASP.NET WinForm	Classe que estende Form
ASP.NET MVC	Página ASPX
Aplicativo Android	Classe que estende Activity

- ✓ Em muitos casos, essas classes são geradas pelos Ambientes Integrados de Desenvolvimento (IDEs), através de alguma facilidade para "desenhar" a interface com o usuário.

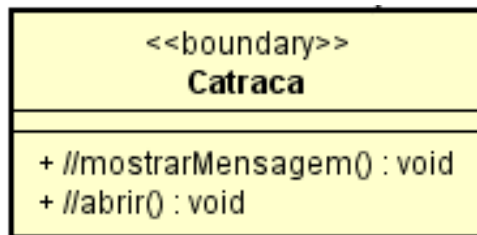
II. Mapeamento de Classes de Fronteira para Interface com Dispositivos ou Sistemas Externos

1. Essas classes são, em geral, modeladas como **Subsistemas**, pois normalmente têm um comportamento interno complexo.
 - ✓ Exemplo: a interface com um sistema legado requer que os dados sejam enviados em um formato específico, usando um protocolo específico e, por questões de segurança, os dados devem ser criptografados segundo um padrão específico.
 - ✓ Nesse caso, o **subsistema** disponibiliza somente uma interface para **enviar os dados**. Entretanto, várias classes internas devem ser implementadas para conversão dos dados, criptografia, transmissão dos dados, tratamento de erros de transmissão, etc.

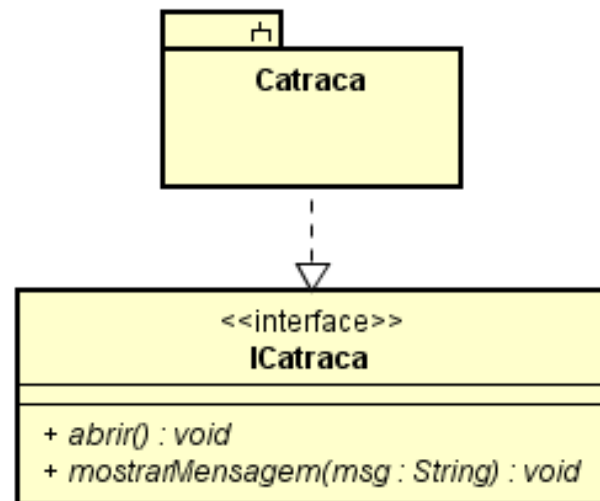
II. Mapeamento de Classes de Fronteira para Interface com Dispositivos ou Sistemas Externos

1. Exemplo:

Classe de Análise



Classes de Design



Existe um padrão muito usado nessa situação:
Façade

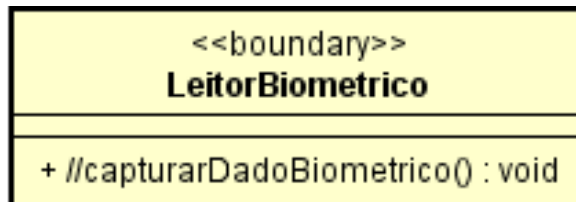
II. Mapeamento de Classes de Fronteira para Interface com Dispositivos ou Sistemas Externos

2. Se o comportamento da classe for simples (talvez atuando apenas como um mecanismo de acesso para a API do dispositivo ou do sistema externo), é possível representá-la com uma única Classe de Design.
 - ✓ Exemplo: classe de fronteira que representa um ar condicionado que pode ser ligado e desligado.
 - ✓ Nesse caso, a classe de fronteira pode ser implementada com uma única classe de design que implementa três métodos públicos: um para **ligar**, outro para **desligar** e outro para **recuperar o estado** (ligado ou desligado). Nessa classe estarão encapsulados os comandos específicos que irão controlar o dispositivo.

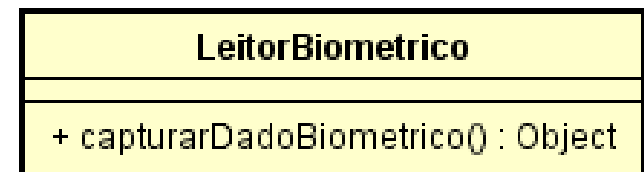
II. Mapeamento de Classes de Fronteira para Interface com Dispositivos ou Sistemas Externos

2. Exemplo:

Classe de Análise



Classe de Design



III. Mapeamento de Classes de Controle

- ✓ Esse mapeamento é muito dependente da tecnologia, do padrão arquitetural adotado e da complexidade do próprio controlador.
Exemplos:

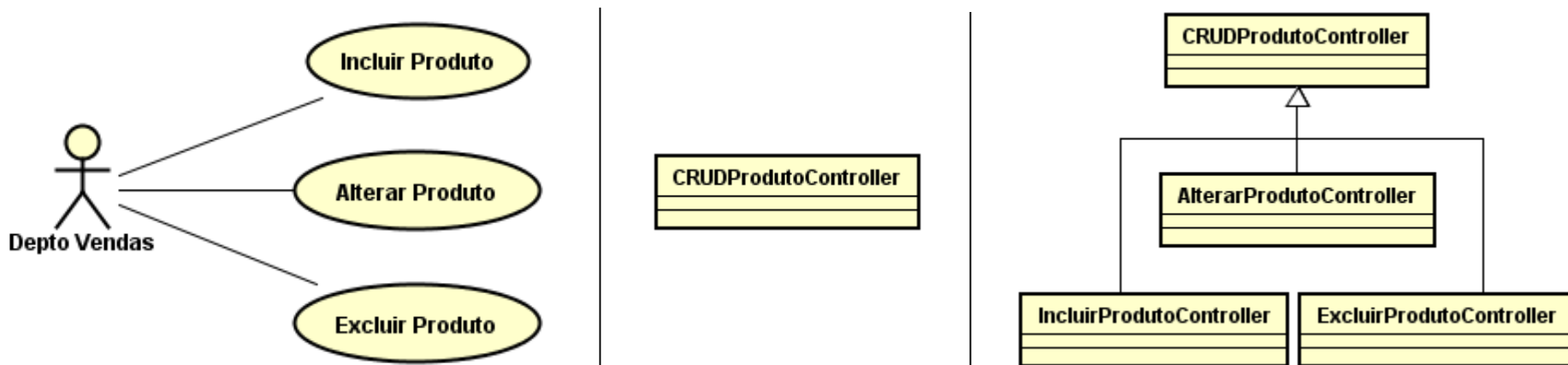
1. A classe de Controle pode desaparecer e ter sua responsabilidade incorporada pela classe de fronteira.
 - ✓ Exemplo: em uma aplicação Java com Swing, o designer pode incorporar o tratador dos eventos do usuário à classe do JFrame (que implementa a fronteira). Esse mesmo design pode ser adotado para aplicações Delphi ou Java para Android.

III. Mapeamento de Classes de Controle

- ✓ Esse mapeamento é muito dependente da tecnologia, do padrão arquitetural adotado e da complexidade do próprio controlador.
Exemplos:

2. Caso sejam definidos vários controladores que possuem responsabilidades semelhantes, o designer pode decidir implementar uma única Classe de Design ou criar uma hierarquia de controladores.

- ✓ Exemplo:

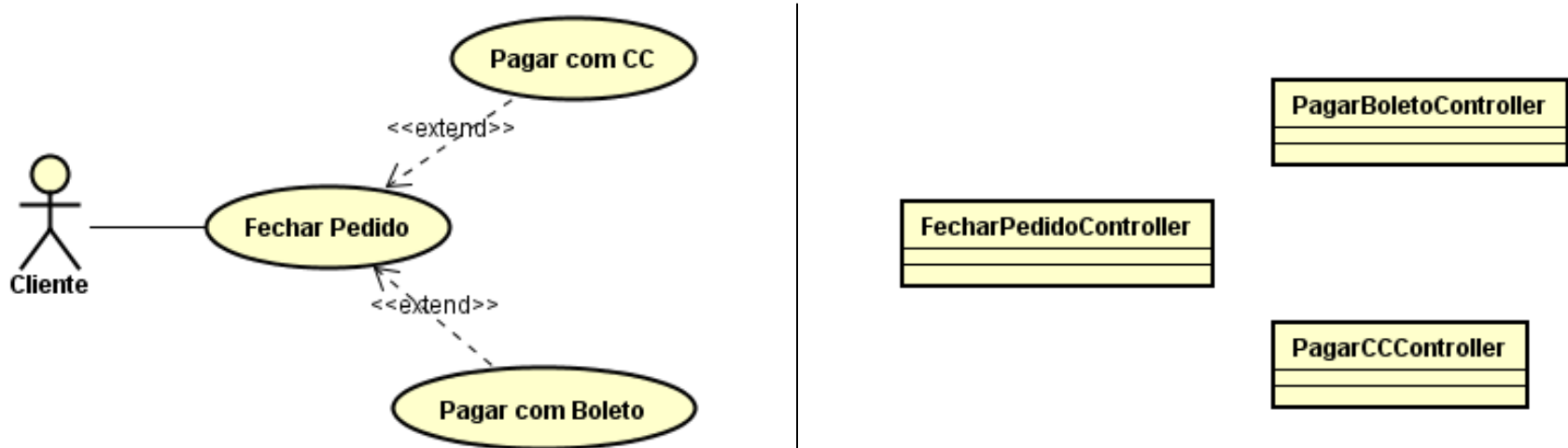


III. Mapeamento de Classes de Controle

- ✓ Esse mapeamento é muito dependente da tecnologia, do padrão arquitetural adotado e da complexidade do próprio controlador. Exemplos:

3. Caso a complexidade da classe de Controle seja alta, o designer pode decidir distribuí-la por diversas Classes de Design.

- ✓ Exemplo:



IV. Mapeamento de Classes de Entidade

- ✓ Inicialmente, é possível assumir que uma classe de Entidade será mapeada para uma **única** classe de Design.
- ✓ Entretanto, conforme o design vai sendo detalhado, esse cenário pode ser alterado em função da:
 - Adoção de padrões e princípios de projeto.
 - Exploração de estruturas hierárquicas ou de delegação.
 - Distribuição de responsabilidades para classes específicas.

IV. Mapeamento de Classes de Entidade

- ✓ A seguir são listados **alguns exemplos** onde dividimos a classe de Entidade em diversas classes de Design:
 - A classe de entidade possui responsabilidades relacionadas à entrada/saída de dados.
 - A classe de entidade possui responsabilidades relacionadas à formatação de dados.
 - A classe de entidade possui responsabilidades onde é necessário se comunicar diretamente com algum dispositivo ou sistema externo.
 - A classe de entidade possui responsabilidades relacionadas à validação de dados que chegam do meio externo.
 - A classe de entidade possui responsabilidades que dependem ou estão associadas diretamente com alguma tecnologia.

➤ Pacote

- ✓ É um elemento de propósito geral que tem o objetivo de **agrupar** outros elementos.
- ✓ Os critérios de empacotamento das classes podem ser vários:
 1. **Agrupar classes de acordo com seu propósito dentro de um sistema:**
 - Exemplos: pacote de transmissão de dados, pacote de validação de dados, pacote de criptografia, pacote para geração de gráficos, pacotes para geração de relatórios, etc.
 - Nesse caso, os pacotes tendem a ser **independentes** de um sistema específico e podem ser **reutilizados** em outros sistemas.

➤ Pacote

2. Agrupar classes de acordo com as camadas do sistema (horizontal):

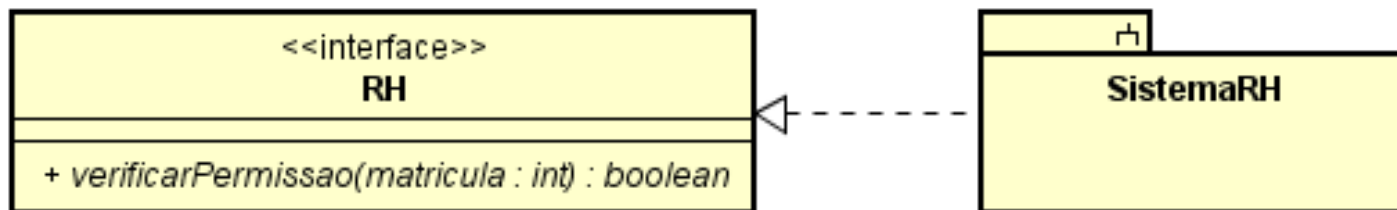
- Exemplos: pacote de interface com o usuário, pacote de lógica da aplicação, pacote de classes de domínio, pacote de persistência de dados, etc.
- Note que, nesse caso, cada pacote corresponde a uma **camada** bem específica do sistema.
- Esse agrupamento é recomendado quando há possibilidade de **troca** de uma dessas camadas.
 - Por exemplo: a camada de apresentação pode ser trocada de uma aplicação web para uma aplicação móvel.

➤ Pacote

3. Agrupar classes de acordo com as funcionalidades que elas implementam (vertical):
 - Exemplos: pacote com as funcionalidades relacionadas à venda, pacote com funcionalidades relacionadas ao estoque, pacote com funcionalidades relacionadas à manutenção de clientes, etc.
 - Esse agrupamento pode ser adotado quando desejamos organizar o sistema do ponto de vista **funcional**, definindo em que pacote determinada funcionalidade está implementada.
- ✓ Na prática é possível adotar diversos ou todos esses critérios **simultaneamente**.

➤ Subsistema

- ✓ Encapsula um conjunto de classes que, juntas, oferecem um **conjunto coerente de responsabilidades**.
- ✓ Para tal, cada subsistema está associado a uma **interface** que define os **métodos providos** pelo subsistema.
- ✓ A interface é a **única parte visível** do subsistema, já que sua estrutura interna fica encapsulada.



- **Subsistemas** podem ser usados de diversas maneiras para dividir o sistema em partes que:
 - ✓ Podem ser projetadas e implementadas de forma independente.
 - ✓ Podem ser implantadas em nós computacionais distribuídos.
 - ✓ Podem ser alteradas ou substituídas sem afetar outras partes do sistema (desde que a interface se mantenha intacta).

➤ Como identificar Subsistemas?

1. **Opcionalidade**: se um conjunto de classes interagem para realizar um comportamento opcional, então elas são candidatas a se tornarem um subsistema. Exemplos:

- ✓ A versão básica de um sistema de contabilidade não permite importar dados de uma planilha eletrônica, mas a versão avançada desse mesmo sistema permite essa importação.

2. **Substituição de componente**: se uma ou mais classes representam um componente da arquitetura que pode ser substituído por outro, então elas são candidatas a se tornarem um subsistema. Exemplo:

- ✓ Sistema de controle de acesso que deve trabalhar com várias opções de leitores biométricos como polegar, palma da mão, íris, face, etc.

➤ Como identificar Subsistemas?

3. **Sistemas externos**: se o sistema depende de alguma funcionalidade oferecida por um sistema externo, então crie um subsistema para representá-lo. Exemplos:

- ✓ Sistema do caixa eletrônico depende do sistema bancário para liberar ou não o saque para o cliente.
- ✓ Sistema de e-commerce depende de vários sistemas de cartão de crédito para autorizar o pagamento.

4. **Distribuição**: se um conjunto de classes deve ser executado em um nó específico da infraestrutura, então ele deve ser encapsulado como um subsistema. Exemplo:

- ✓ Em um sistema qualquer, um serviço de envio de SMS possui alta demanda e, por isso, deve ser executado em um servidor específico.

➤ Como identificar Subsistemas?

5. **Substituição de serviço**: se várias de classes interagem para oferecer um serviço que pode ser substituído por outro, então elas devem ser encapsuladas em um subsistema. Exemplo:

- ✓ Os serviços de Compactação e Criptografia em um sistema podem ser substituídos por outros mais eficientes e seguros.

- Diferenças entre **Subsistema** e **Pacote**:

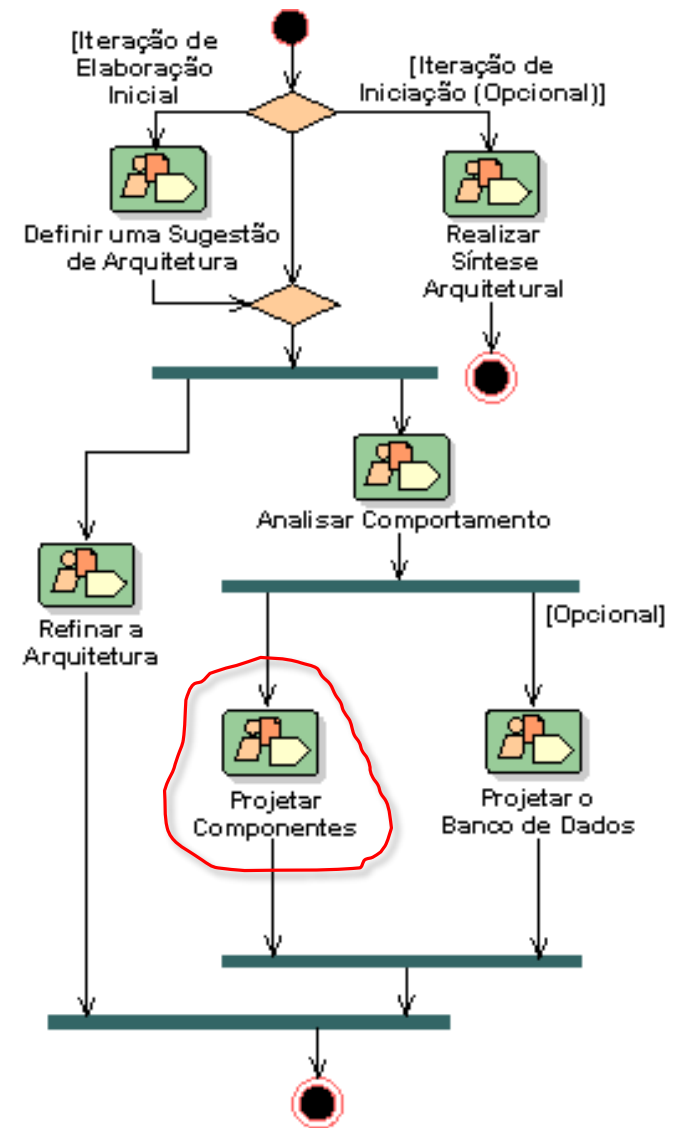
Subsistema	Pacote
Conjunto de classes que, juntas, representam uma abstração bem definida.	Conjunto de classes agrupadas por propósito, camada ou funcionalidade.
Oferece um conjunto de comportamentos bem definidos e coerentes, através da sua interface.	Não oferece, necessariamente, um conjunto de comportamentos coerentes.
Encapsula completamente a estrutura que implementa os comportamentos.	Não encapsula completamente a sua estrutura interna.
São facilmente substituídos (desde que se mantenha a interface).	Nem sempre são facilmente substituídos.

Projeto de Software Orientado a Objetos

12ª parte

Prof. Jobson Massollar

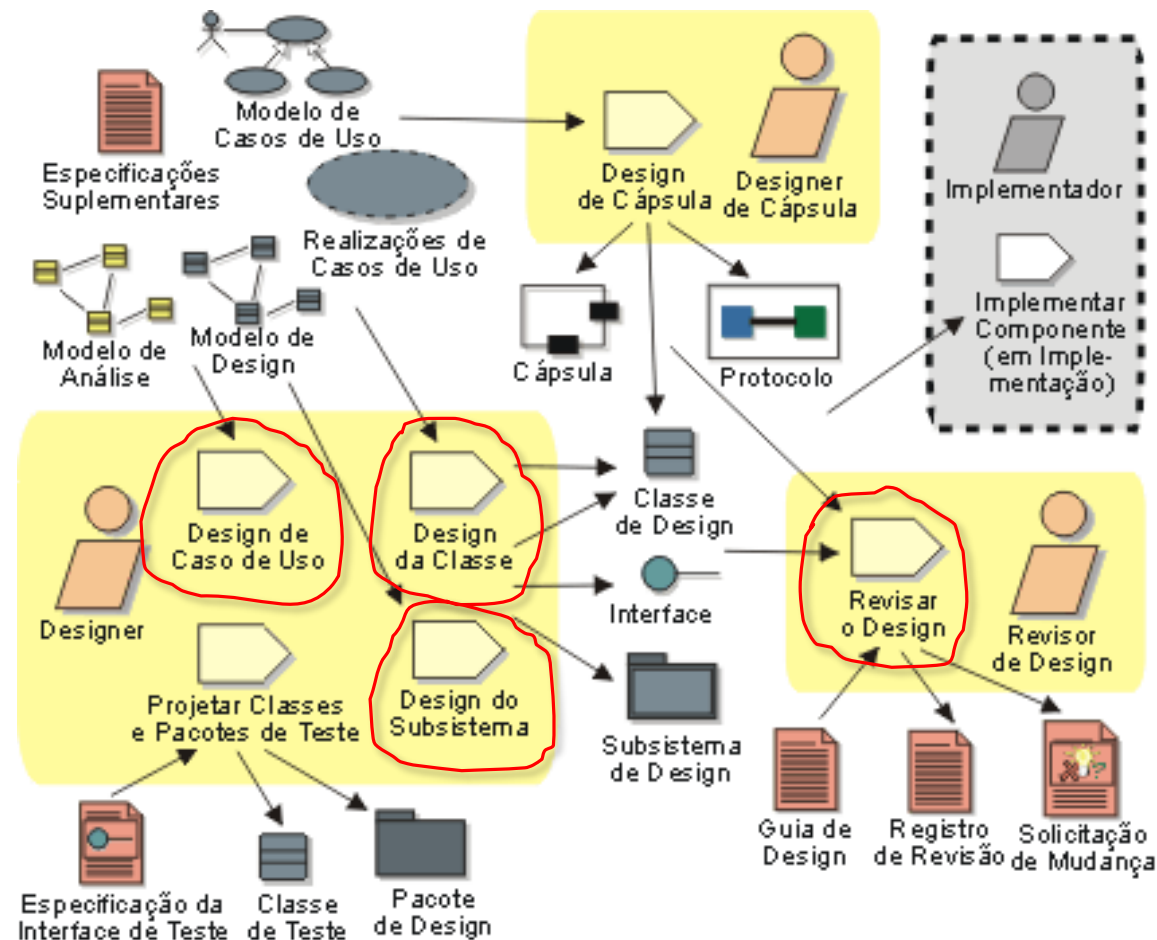
- A disciplina de **Análise e Design** é composta dos seguintes fluxos de trabalho:
1. Definir uma Sugestão de Arquitetura
 2. Analisar Comportamento
 3. **Projetar Componentes**
 4. Projetar BD
 5. Refinar a Arquitetura

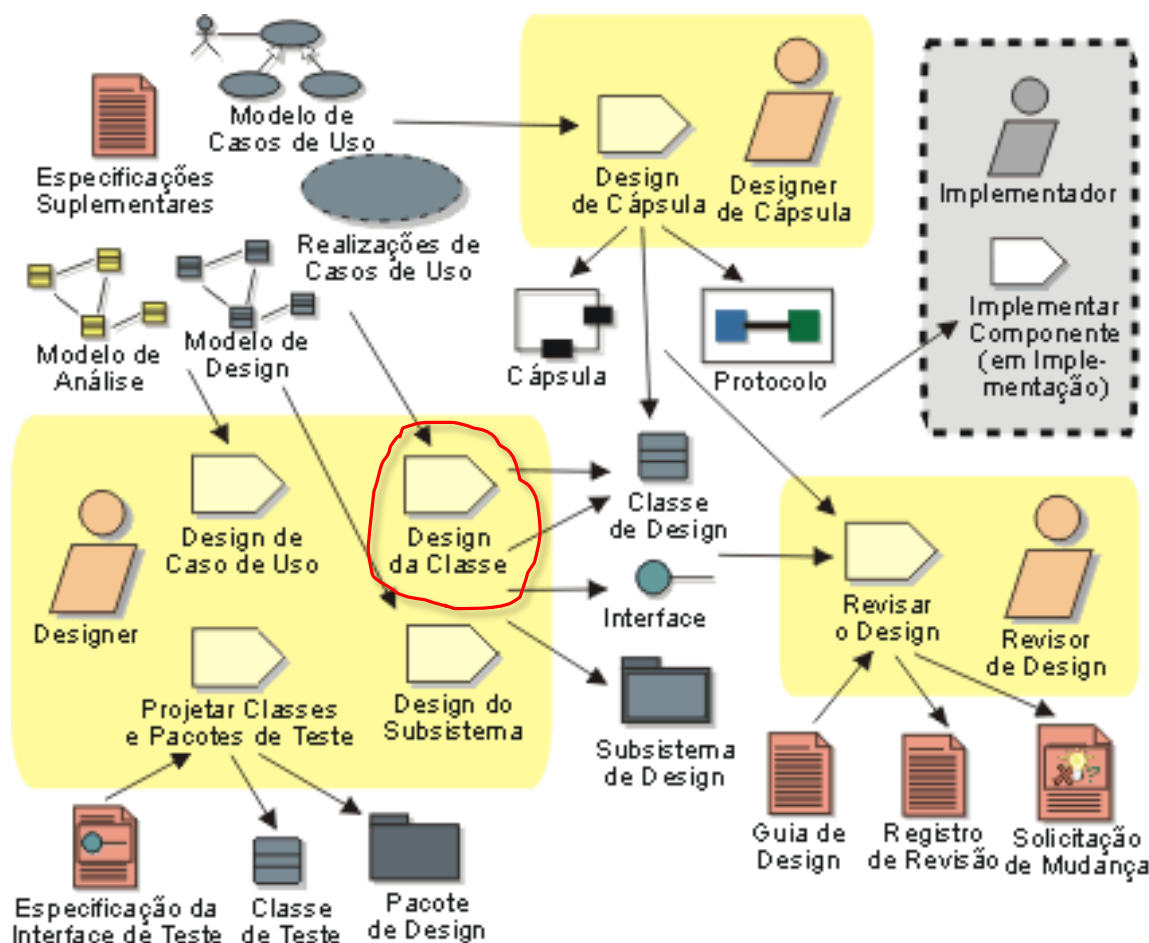


- A finalidade desse fluxo de trabalho é:
 - ✓ Refinar as definições dos elementos de design acrescentando detalhes sobre como estes elementos implementam o comportamento requerido;
 - ✓ Refinar e atualizar as realizações de casos de uso com os novos elementos de design identificados;
 - ✓ Revisar o design conforme ele evolui.

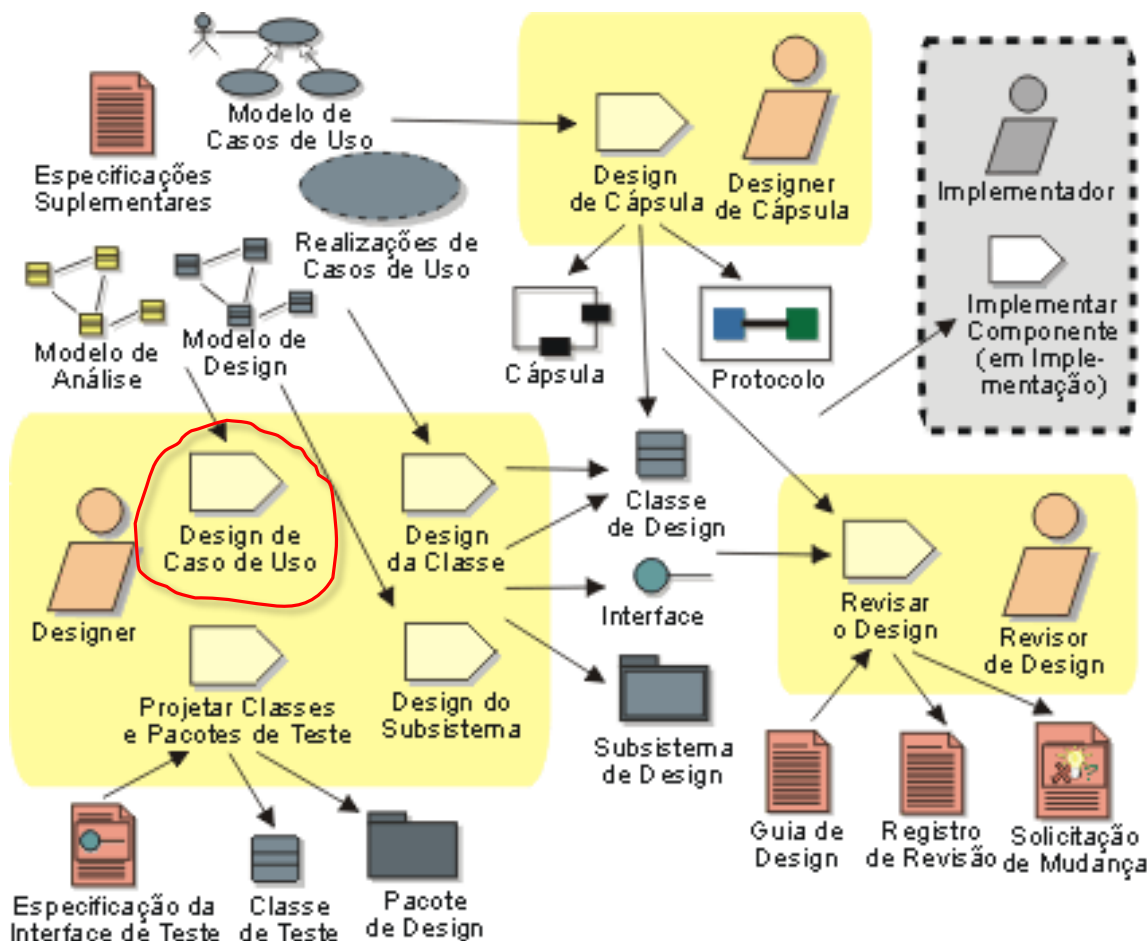
- Esse fluxo de trabalho é composto das atividades:
- ✓ Design da Classe
 - ✓ Design de Caso de Uso
 - ✓ Design do Subsistema
 - ✓ Revisar o Design

As atividades de Design da Classe, Design de Caso de Uso e Design de Subsistemas estão fortemente acopladas e são realizadas de forma alternada. A divisão existe somente para fins didáticos.





- Refinar as definições das Classes de Design acrescentando detalhes sobre como esses elementos implementam as responsabilidades definidas.
- De forma resumida, essa atividade envolve os seguintes passos:
 1. Criar classes iniciais de design.
 2. Definir atributos e seus tipos.
 3. Definir métodos, tipos e parâmetros.
 4. Definir dependências, associações, agregações, composições e generalizações (hierarquias).
- Esses passos são realizados em **paralelo** com o Design do Caso de Uso.



- Refinar e atualizar as realizações de casos de uso (Diagramas de Sequência) com os novos elementos de design identificados (Classes de Design e Subsistemas).
- De forma resumida, envolve os seguintes passos:
 1. Descrever as interações entre as classes de design.
 2. Descrever comportamentos da persistência (criar, recuperar, alterar ou excluir objetos persistentes).
 3. Simplificar os diagramas de sequência usando subsistemas.

- De forma resumida, envolve os seguintes passos:
 1. Descrever as interações entre as classes de design
 2. Descrever comportamentos da persistência
 3. Simplificar os diagramas de sequência usando subsistemas

- **Importante:** para realizar essas atividades serão usados os critérios de mapeamentos entre **Classes de Análise** e **Classes de Design** apresentados na atividade **Identificar Elementos de Design**, vista anteriormente.



1. Descrever as Interações entre as Classes de Design

- ✓ Usando as **Classes de Design** e **Subsistemas** identificados anteriormente, **redefinir** os Diagramas de Sequência que realizam os casos de uso.
- ✓ Nesse processo, é importante inserir detalhes que não eram relevantes anteriormente:
 - Criação/destruição de objetos.
 - Criação/destruição de conjuntos de objetos.
 - Valores que são passados nas mensagens (via parâmetros).
 - Valores que são retornados pelas mensagens.

- Para discutir algumas questões será usado um exemplo simples: **cadastro de um Aluno**.

Cadastrar Aluno

CPF:

Nome:

Dt. Nasc:

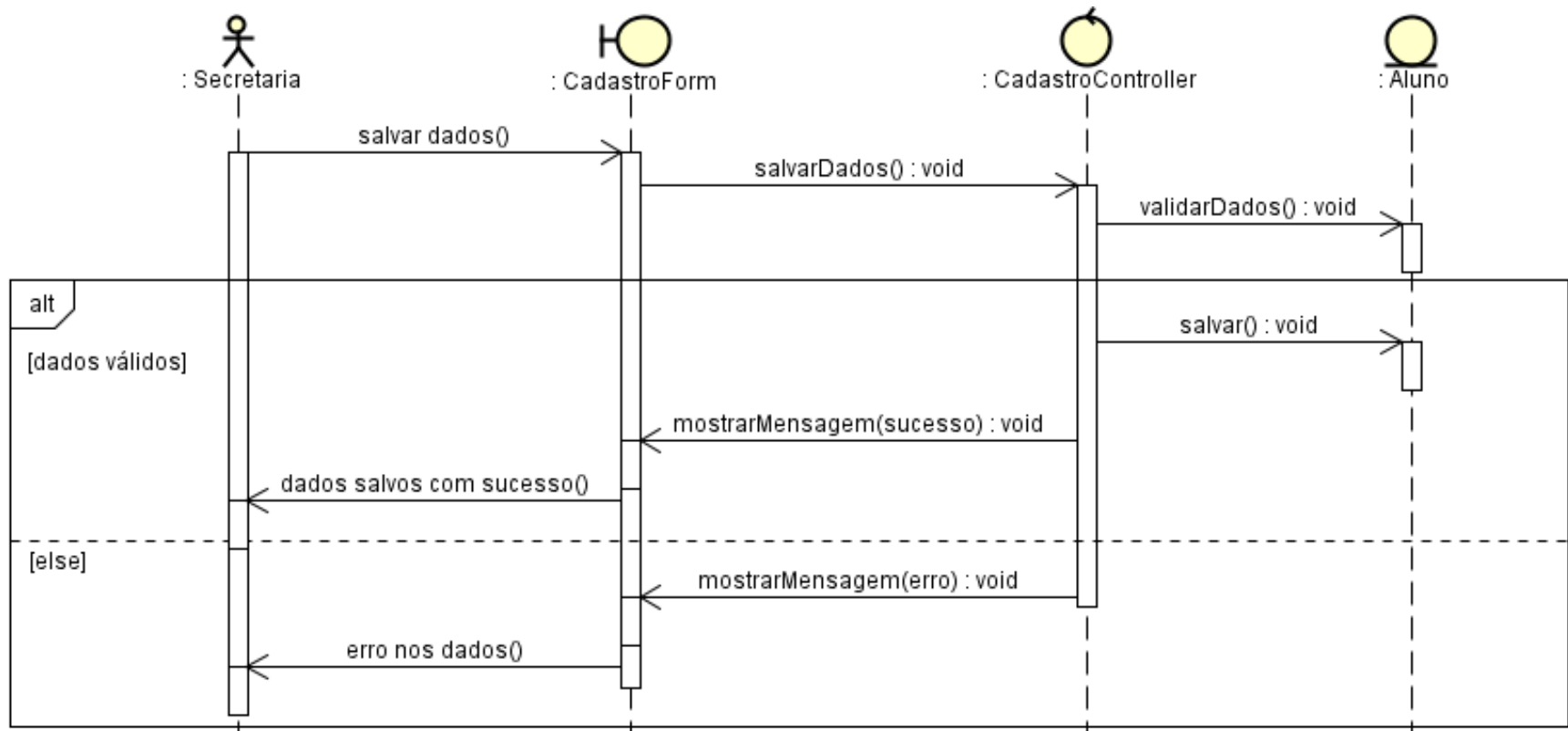
Salvar

Cadastrar Aluno

Aluno cadastrado com sucesso!

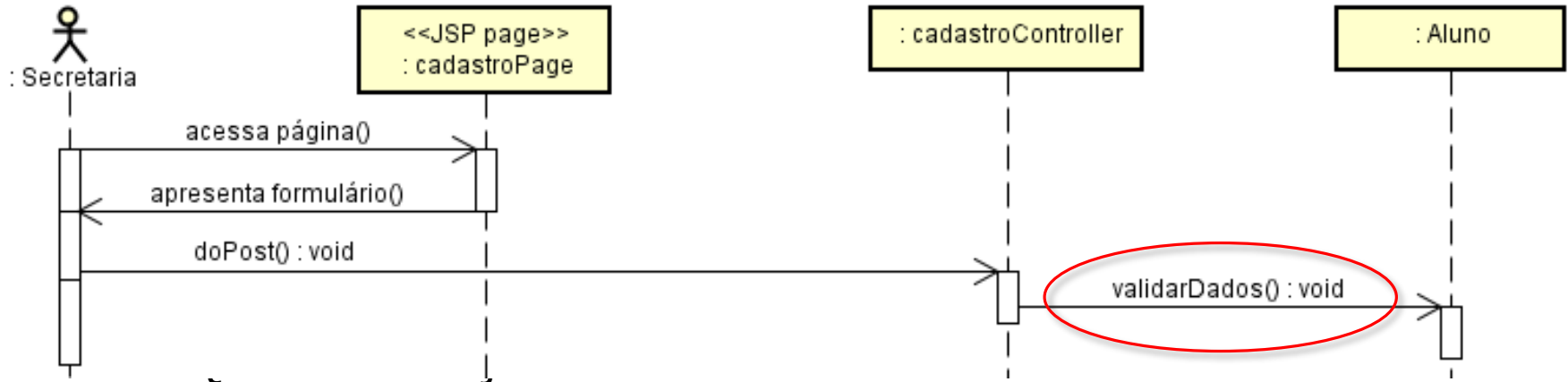
Fechar

- O Diagrama de Sequência da Análise do Caso de Uso poderia ser definido como:



- Assumindo as seguintes tecnologias:
 - ✓ Aplicação JSP/Servlet.
 - ✓ Não será usado nenhum *framework* de persistência.
 - ✓ Acesso ao banco de dados via JDBC.
- Inicialmente serão feitas as seguintes substituições:

Classes de Análise	Classes de Design
Classe de fronteira CadastroForm	Página JSP
Classe de controle CadastroController	Classe que herda de HttpServlet e sobrescreve o método doPost().



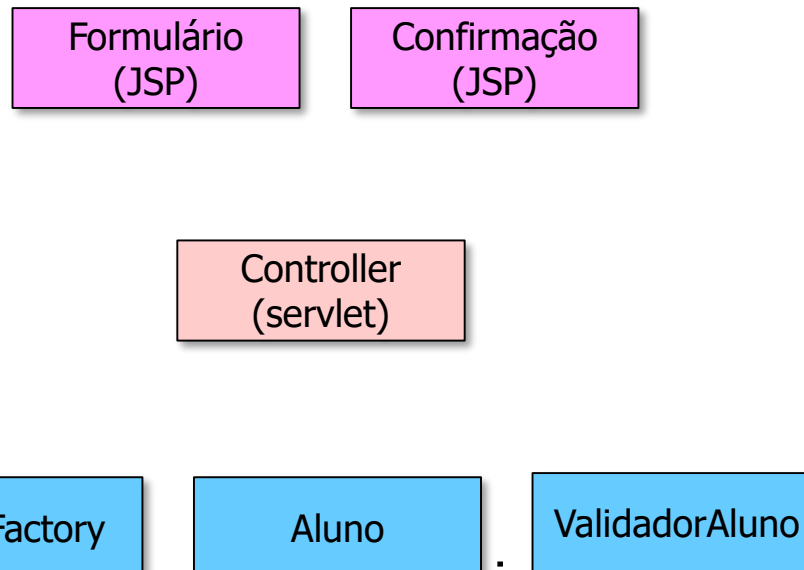
2. Quem realiza a validação? Lembre-se de que para validar é preciso verificar se existe outro aluno com o mesmo CPF.
3. Como retornar para o Controller se houve ou não erro na validação?
4. Como retornar para o Controller as mensagens de erro?

➤ Para a **validação dos dados** temos basicamente duas possibilidades:

Solução	Vantagens	Desvantagens
Definir um construtor que recebe os dados e faz as validações.	A validação dos dados fica na própria classe que detém os dados.	- A classe terá métodos de validação, o que fere o Princípio da Responsabilidade Única. - A classe terá que lidar com estruturas de dados para armazenar as mensagens de erro.
Criar uma outra classe que fica responsável somente pela validação.	- Mantém a classe do domínio coesa. - A validação é realizada por uma classe especificamente projetada para esse fim.	- Os dados são validados por uma classe que não detém esses dados. - Cria uma dependência entre a classe e sua validadora.

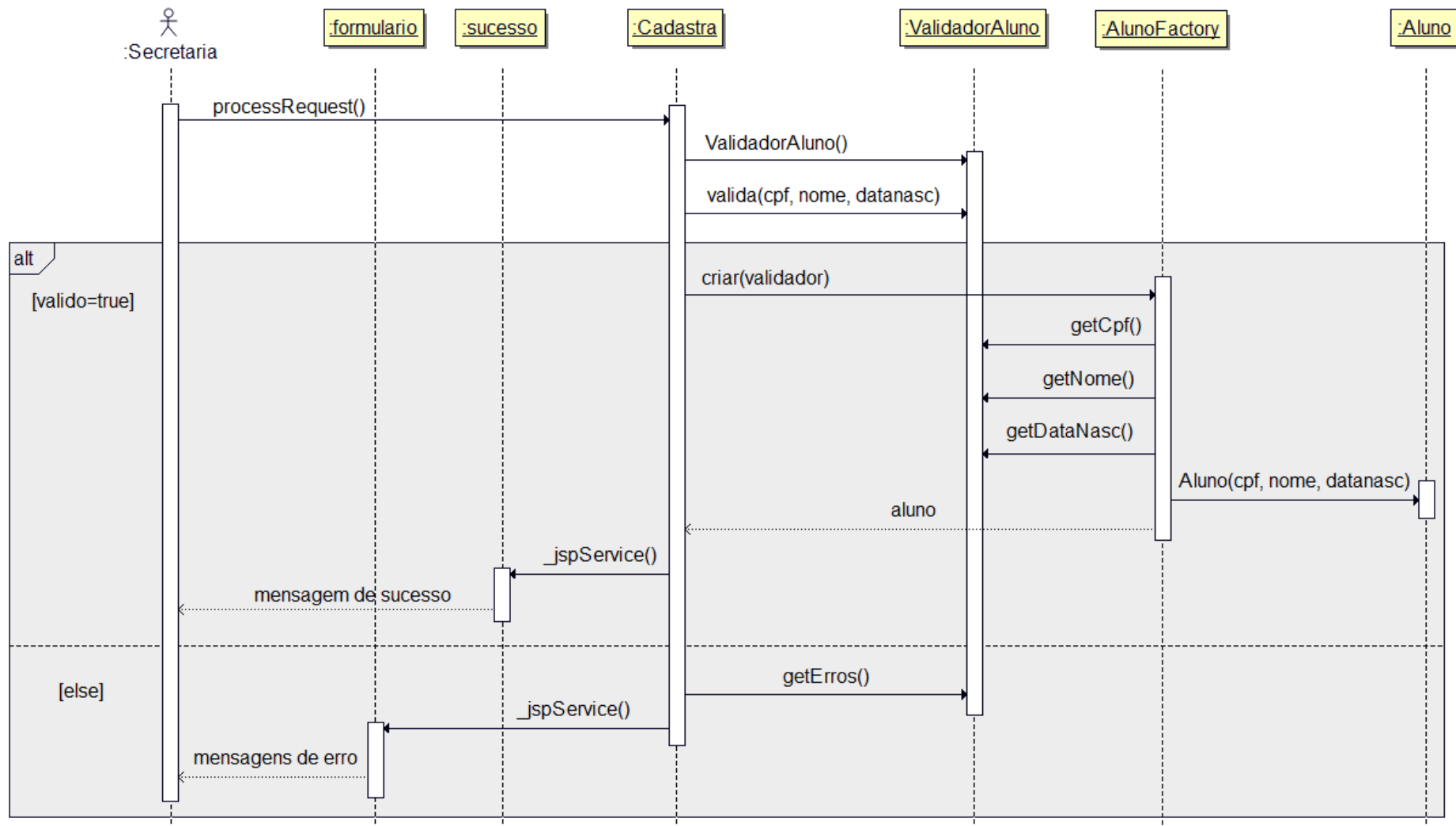
- Para a **criação de objetos** serão adotados os princípios do padrão **Factory Method**, ou seja, será definida uma classe **AlunoFactory** para criar objetos Aluno.
- Note que o padrão **Factory Method** se aplica a uma hierarquia de classes e que, nesse caso, temos somente **uma classe**.
- Mesmo assim, a adoção desses princípios tem vantagens claras:
 - ✓ Define um ponto único para criação dos objetos, evitando que a criação dos objetos fique "espalhada" pelo código.
 - ✓ Permite a criação de objetos de forma consistente e transparente, independente do objeto ser criado a partir de dados de entrada ou a partir de dados em um banco de dados.

- Com base nessas decisões de projeto temos as seguintes **classes de design**:

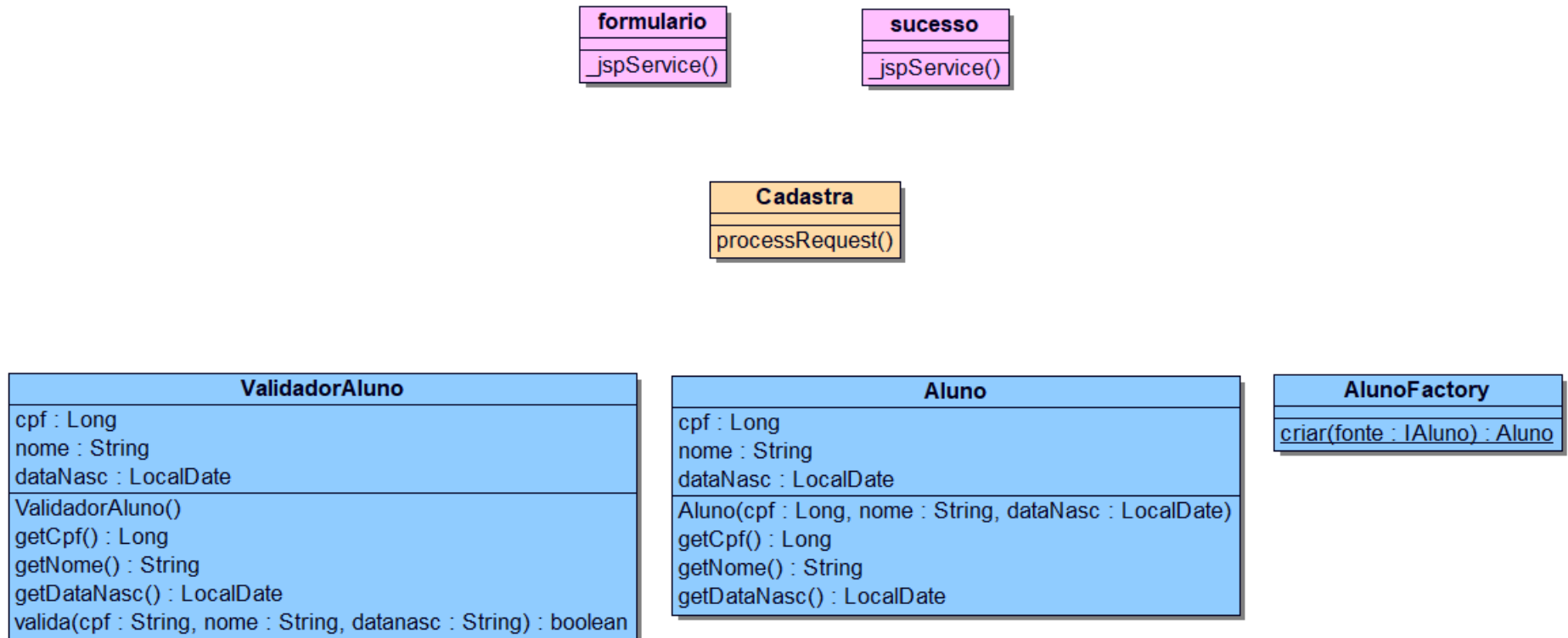


- Vamos redefinir o Diagrama de Sequência com base nessas classes.

➤ Diagrama de Sequência de design:



➤ Diagrama de Classes:

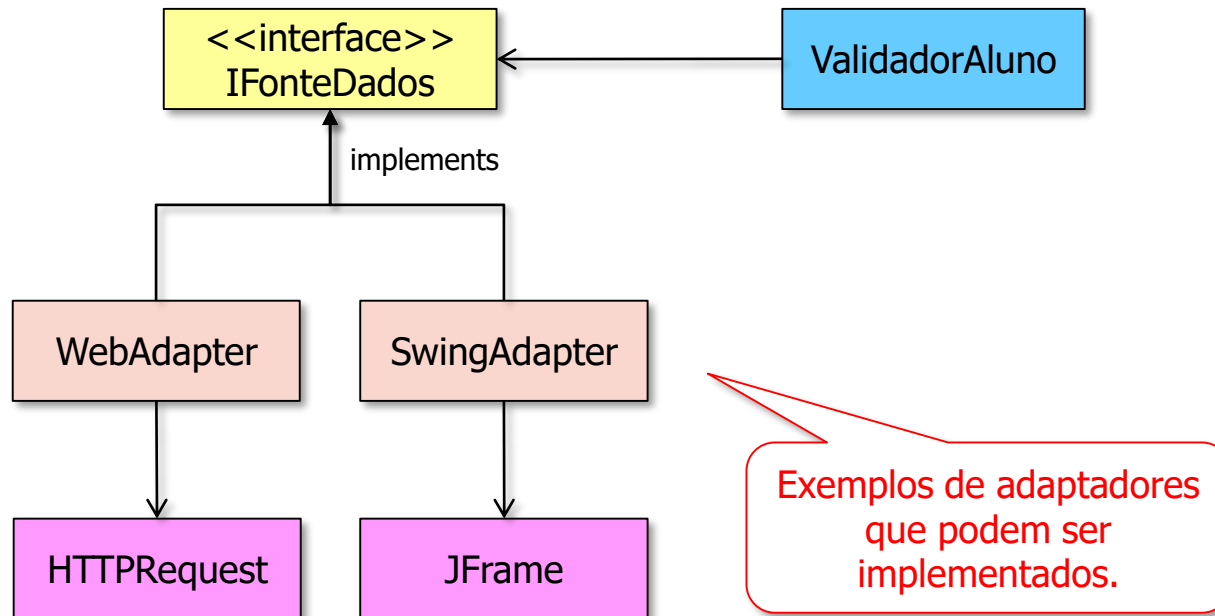


- O Diagrama de Sequência anterior representa a seguinte lógica:

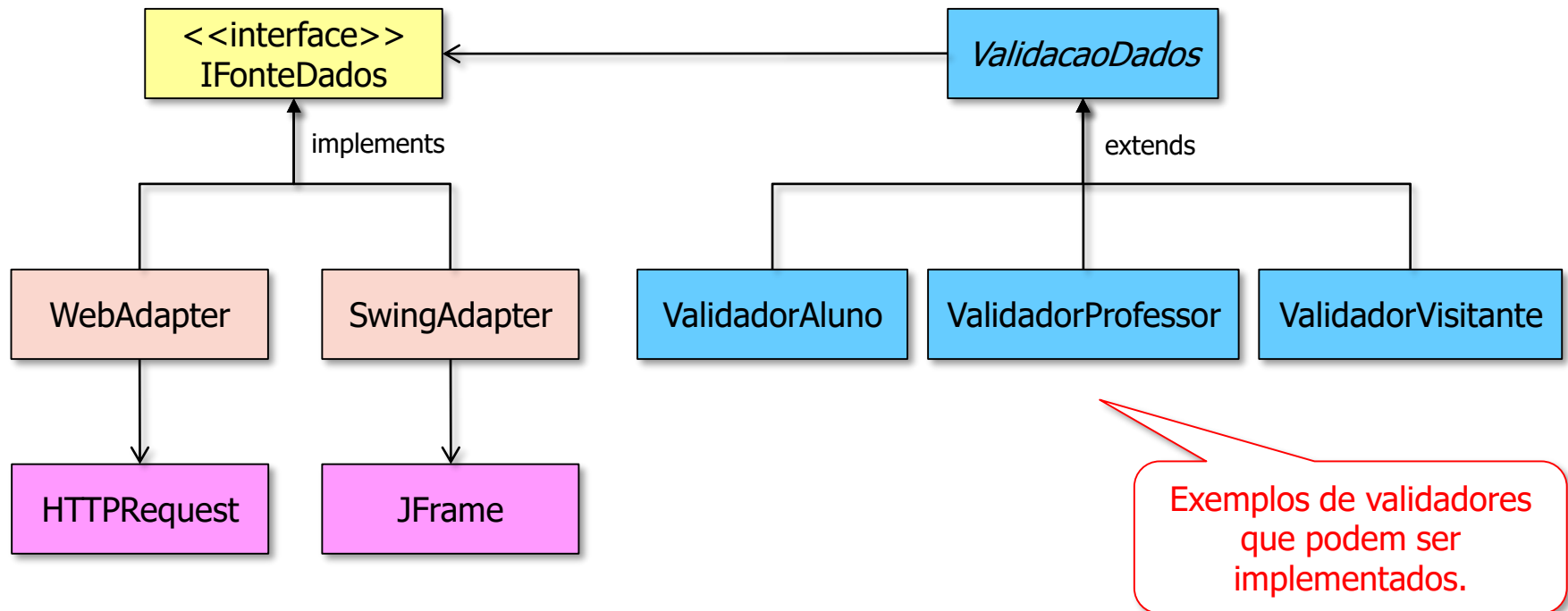
```
processRequest(HttpServletRequest request, HttpServletResponse response) {  
    ValidadorAluno validador = new ValidadorAluno();  
  
    boolean valido = validador.valida(request.getParameter("cpf"),  
                                     request.getParameter("nome"),  
                                     request.getParameter("dataNasc"));  
  
    if (valido) {  
        aluno = AlunoFactory.criar(validador);  
        // Aqui o controller transfere a execução para a página de sucesso  
    }  
    else {  
        // Aqui o controller transfere a execução para a página do formulário  
    }  
}
```

- Esse design possui algumas questões que podem ser melhoradas:
 1. A classe **controller** recupera os dados do request e envia esses dados para o **validador**.
 - ✓ Isso é ruim, porque o controller tem que manipular a estrutura onde os dados estão armazenados.
 2. A classe **controller** poderia repassar a estrutura que armazena os dados para o validador e o próprio validador se encarregaria de recuperar os dados.
 - ✓ Isso também é ruim, porque o validador ficará fortemente acoplado à uma tecnologia específica.
- Vamos adotar o **DIP** (**Princípio da Inversão de Dependência**) e o padrão **Adapter**.

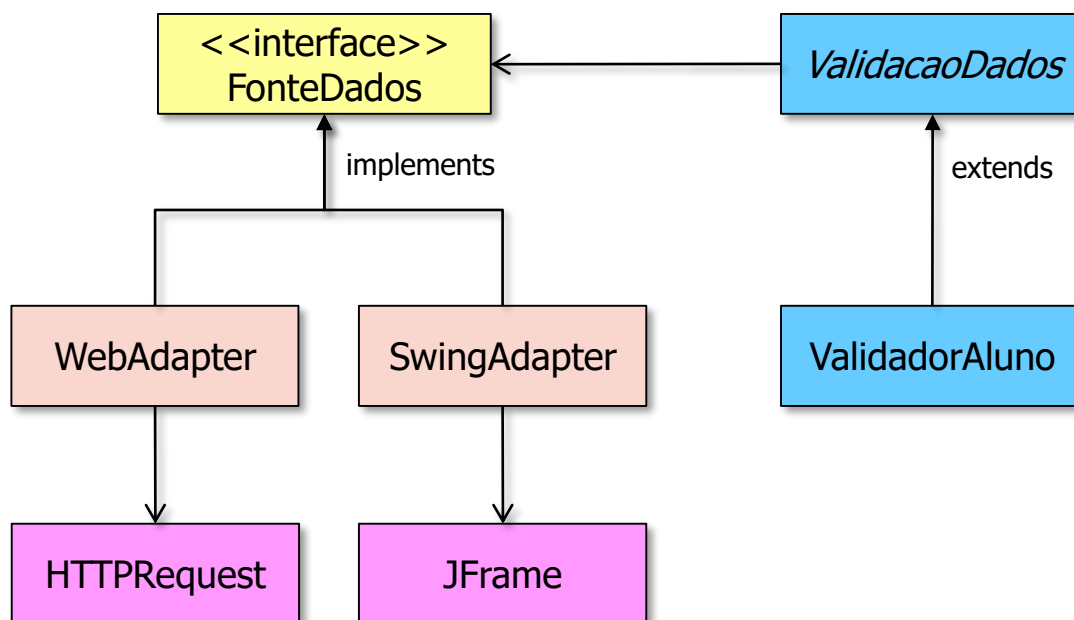
1. Inicialmente devemos tornar o `ValidadorAluno` dependente da interface `IFonteDados`. Em seguida, definimos um ou mais `Adapters` que irão recuperar os dados de uma fonte específica.



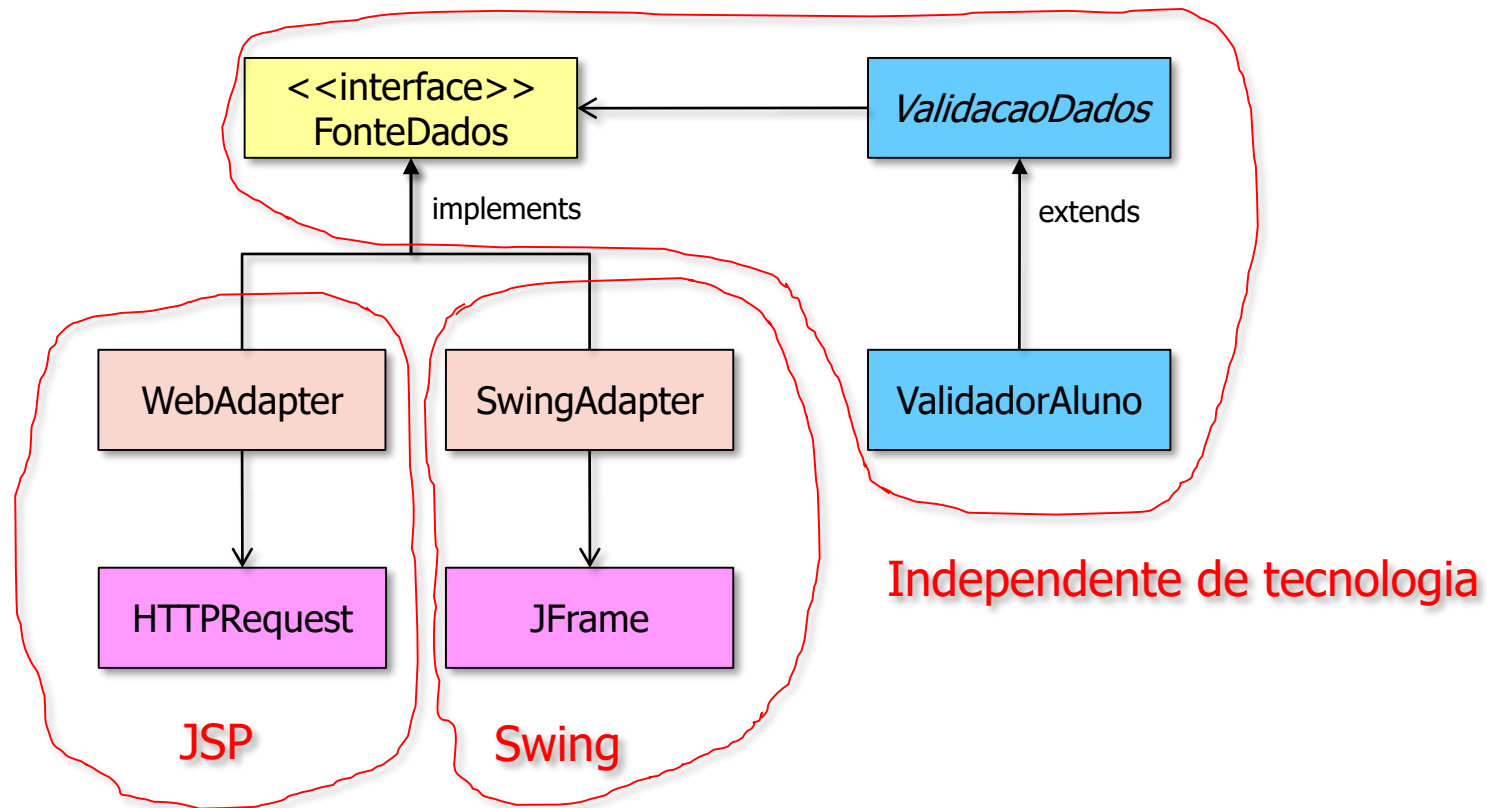
2. Podemos também definir uma superclasse abstrata `ValidacaoDados` com os comportamentos comuns a todos os validadores e criar os validadores específicos através de herança.

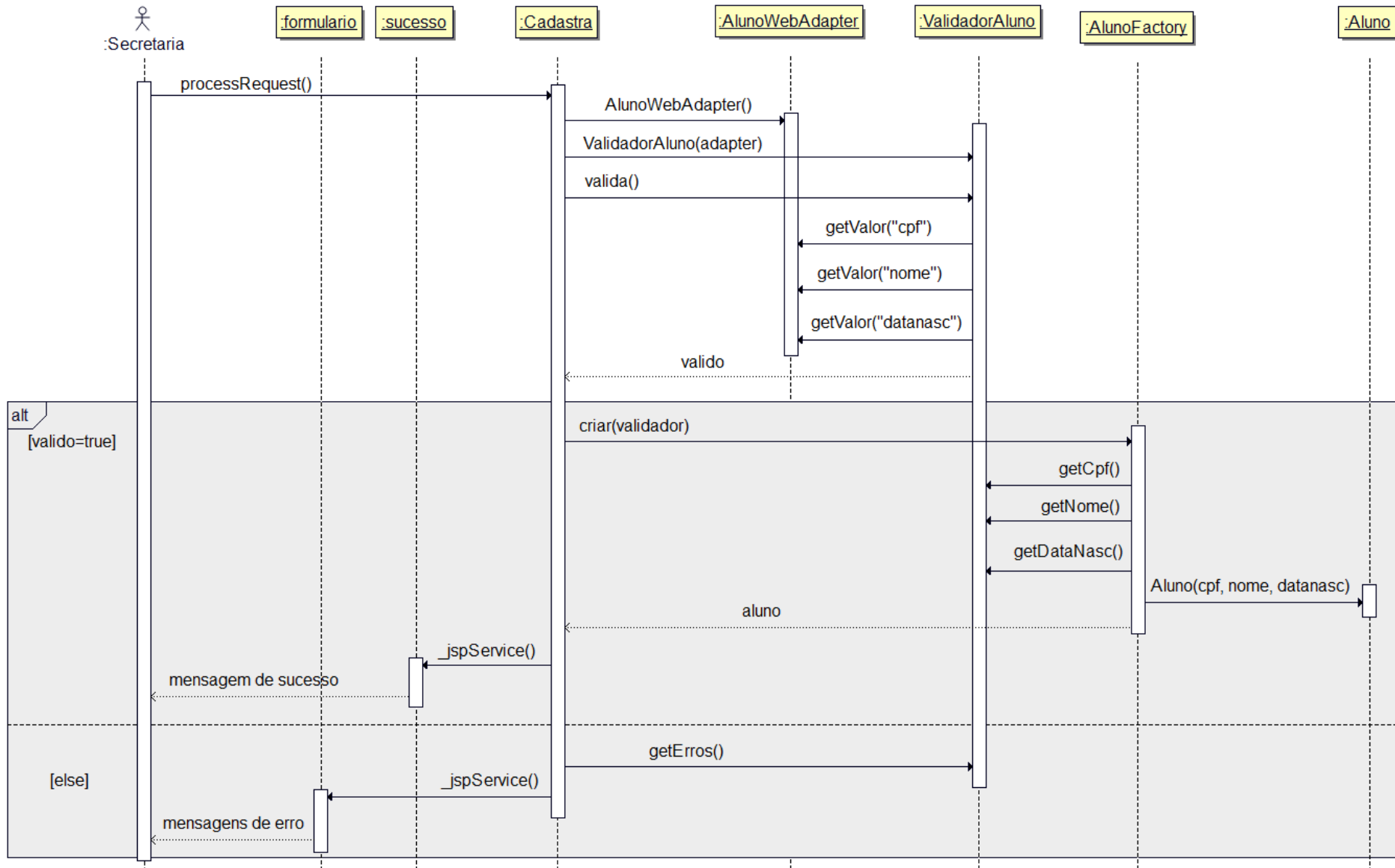


- Adotando esses conceitos, chegamos a um modelo onde temos uma separação clara entre tecnologia e domínio.

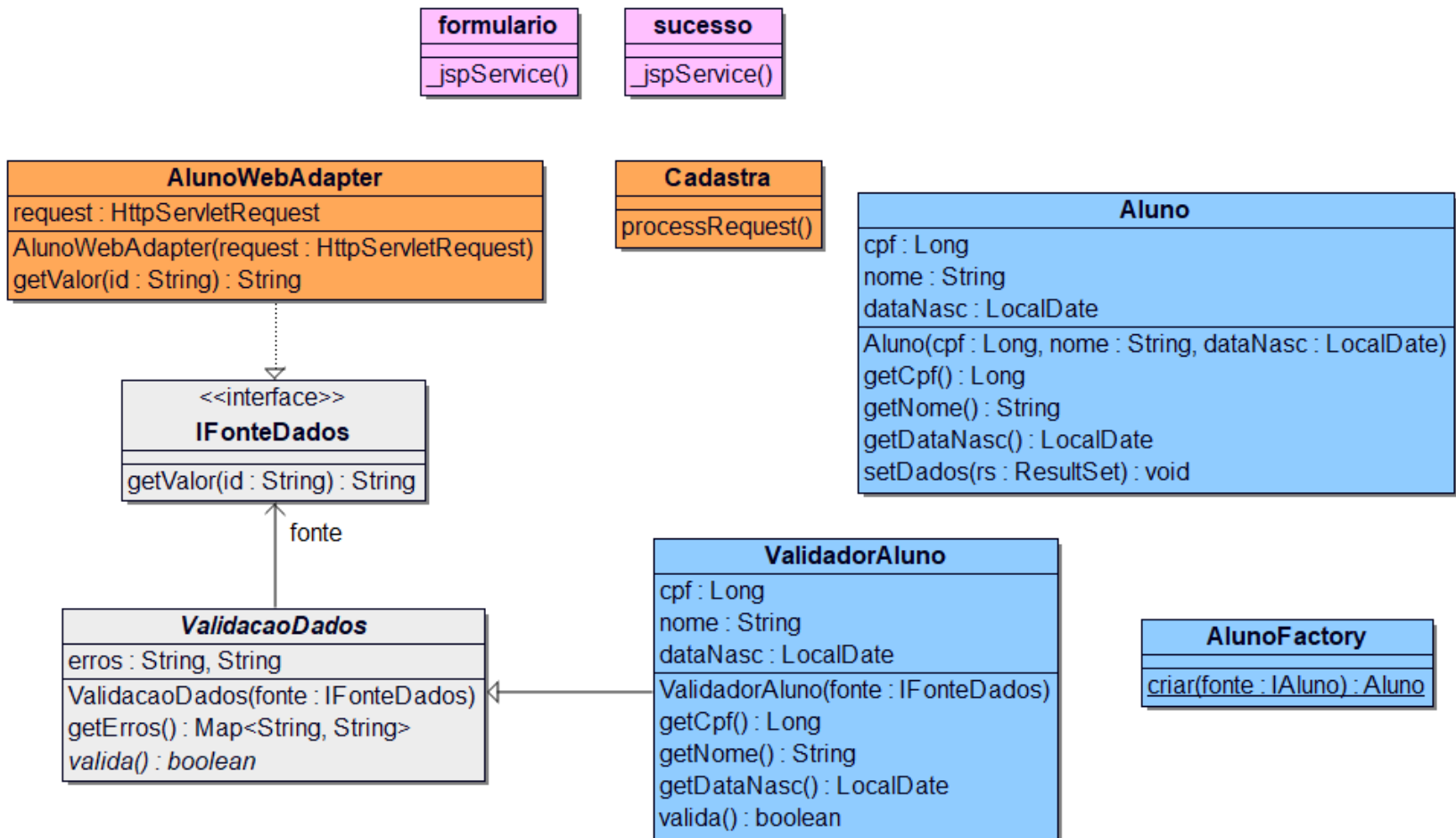


- Adotando esses conceitos, chegamos a um modelo onde temos uma separação clara entre tecnologia e domínio.





➤ Novo Diagrama de Classes:



➤ A nova lógica fica assim:

```
processRequest(HttpServletRequest request, HttpServletResponse response) {  
    AlunoWebAdapter adapter = new AlunoWebAdapter(request);  
    ValidadorAluno validador = new ValidadorAluno(adapter);  
    boolean valido = validador.valida();  
  
    if (valido) {  
        aluno = AlunoFactory.criar(validador);  
        // Aqui o controller transfere a execução para a página de sucesso  
    }  
    else {  
        // Aqui o controller transfere a execução para a página do formulário  
    }  
}
```

- Note que, na solução adotada, os dados para criação do objeto estão no **validador** (dados previamente validados).
- Para criar o objeto Aluno a partir desses dados, temos algumas opções:
 1. O **Controller** recupera os dados do **ValidadorAluno** e cria o **Aluno**.
 2. A própria classe **ValidadorAluno** cria o **Aluno**.
 3. A classe **Aluno** recupera os dados do **ValidadorAluno**.
 4. Define uma **classe específica** para **criar** o Aluno.

1. O **Controller** recupera os dados do **ValidadorAluno** e cria o **Aluno**
 - ✓ Desvantagem: sobrecarga de responsabilidades na classe Controller

// Trecho de código do controller

```
processRequest(HttpServletRequest request, HttpServletResponse response) {  
    AlunoWebAdapter adapter = new AlunoWebAdapter(request);  
    ValidadorAluno validador = new ValidadorAluno(adapter);  
    Aluno aluno;  
  
    if (validador.valida())  
        aluno = new Aluno(validador.getCPF(),  
                           validador.getNome(),  
                           validador.getDtNasc());  
}
```


2. A própria classe `ValidadorAluno` cria o `Aluno`

- ✓ Desvantagem: sobrecarga de responsabilidades na classe `AlunoValidator`

// Trecho de código do controller

```
processRequest(HttpServletRequest request, HttpServletResponse response) {  
    AlunoWebAdapter adapter = new AlunoWebAdapter(request);  
    ValidadorAluno validador = new ValidadorAluno(adapter);  
    Aluno aluno;
```

```
    if (validador.valida())  
        aluno = validador.criar();  
}
```

// Classe `ValidadorAluno`

```
public class ValidadorAluno {  
    public Aluno criar() {  
        return new Aluno(this.getCPF(), this.getNome(), this.getDtnasc());  
    }  
}
```

3. A classe `Aluno` recupera os dados do `ValidadorAluno`

- ✓ Desvantagem: acoplamento da classe `Aluno` com `ValidadorAluno`

// Trecho de código do controller

```
processRequest(HttpServletRequest request, HttpServletResponse response) {  
    AlunoWebAdapter adapter = new AlunoWebAdapter(request);  
    ValidadorAluno validador = new ValidadorAluno(adapter);  
    Aluno aluno;  
  
    if (validador.valida())  
        aluno = new Aluno(validador);  
}
```

// Classe Aluno

```
public class Aluno {  
    public Aluno(ValidadorAluno v) {  
        this.cpf = v.getCpf();  
        this.nome = v.getNome();  
        this.dtNasc = v.getDtNasc();  
    }  
}
```

4. Define uma **classe específica** para **criar** o Aluno.

- ✓ Desvantagem: Controller recupera os dados do ValidadorAluno para repassar ao AlunoFactory

// Trecho de código do controller

```
processRequest(HttpServletRequest request, HttpServletResponse response) {  
    AlunoWebAdapter adapter = new AlunoWebAdapter(request);  
    ValidadorAluno validador = new ValidadorAluno(adapter);  
    Aluno aluno;
```

```
    if (validador.valida())  
        aluno = AlunoFactory.criar(validador.getCpf(),  
                                    validador.getNome(),  
                                    validador.getDtNasc());  
}
```

// Classe AlunoFactory

```
public class AlunoFactory {  
    public static Aluno criar(String cpf, String nome, LocalDate dtNasc) {  
        return new Aluno(cpf, nome, dtNasc);  
    }  
}
```

4. Define uma **classe específica** para **criar** o Aluno.

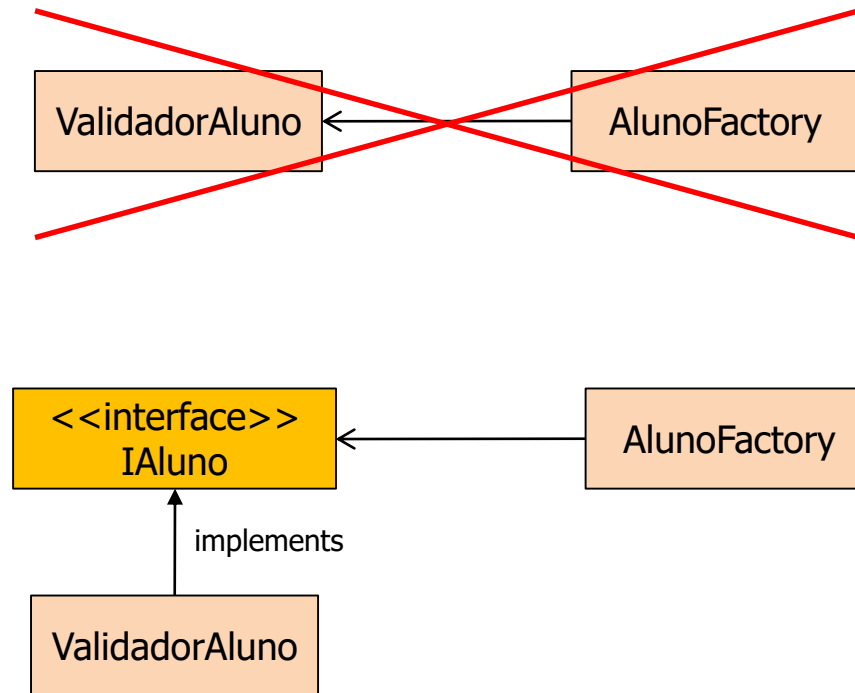
- ✓ Desvantagem: acoplamento entre as classes AlunoFactory e ValidadorAluno.

```
// Trecho de código do controller
processRequest(HttpServletRequest request, HttpServletResponse response) {
    AlunoWebAdapter adapter = new AlunoWebAdapter(request);
    ValidadorAluno validador = new ValidadorAluno(adapter);
    Aluno aluno;

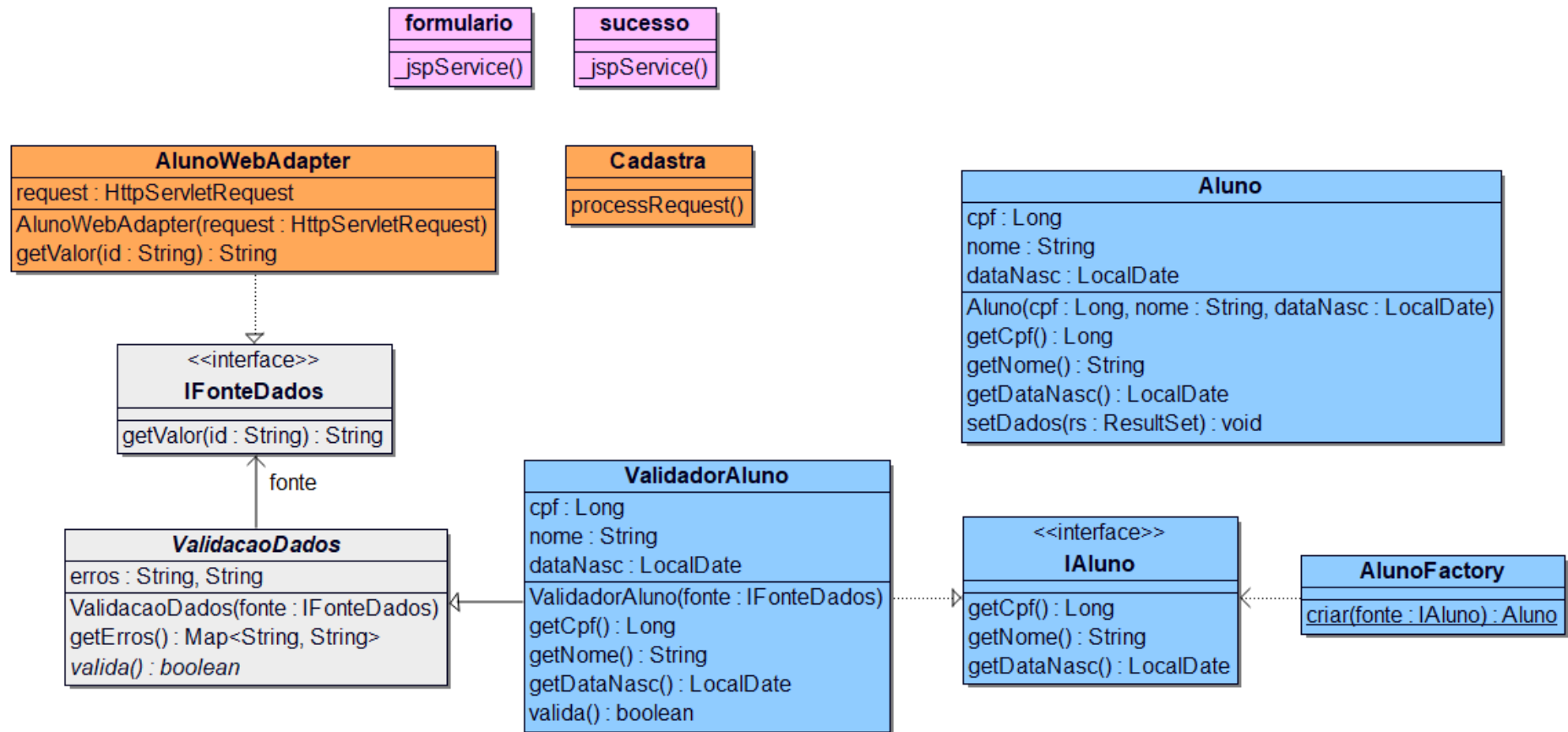
    if (validador.valida())
        aluno = AlunoFactory.criar(validador);
}

// Classe AlunoFactory
public class AlunoFactory {
    public static Aluno criar(ValidadorAluno v) {
        return new Aluno(v.getCpf(), v.getNome(), v.getDtnasc());
    }
}
```

- Para enfraquecer o acoplamento entre `AlunoFactory` e `ValidadorAluno`, vamos aplicar o **Princípio da Inversão de Dependência** (DIP) e tornar a classe dependente de interfaces:



➤ Diagrama final de classes:



- Duas tarefas ainda precisam ser realizadas para completar o design desse caso de uso:
 1. Verificar se existe outro aluno cadastrado com o mesmo CPF;
 2. Salvar os dados do novo aluno no cadastro.
- Esses dois itens serão resolvidos quando falarmos sobre **persistência dos objetos**.

Projeto de Software Orientado a Objetos

13ª parte

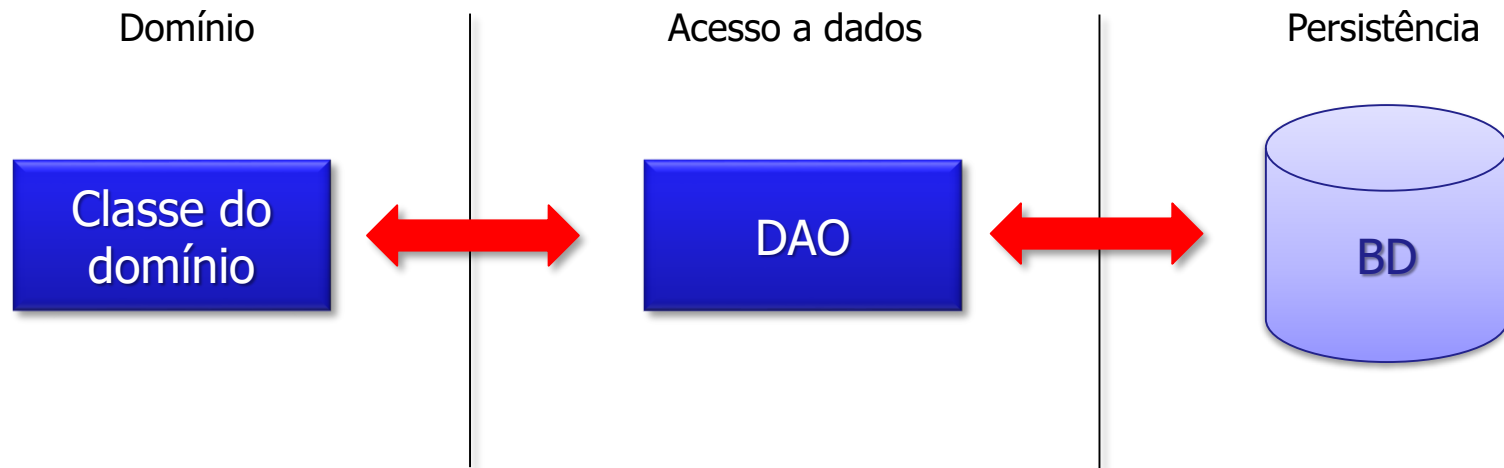
Prof. Jobson Massollar

- De forma resumida, envolve os seguintes passos:
 1. Descrever as interações entre as classes de design
 2. Descrever comportamentos da persistência 
 3. Simplificar os diagramas de sequência usando subsistemas

- **Importante:** para realizar essas atividades serão usados os critérios de mapeamentos entre **Classes de Análise** e **Classes de Design** apresentados na atividade **Identificar Elementos de Design**, vista anteriormente.

- As classes do domínio que precisam ser persistidas não devem implementar os comandos de persistência propriamente ditos, por vários motivos:
 - ✓ **Princípio da Responsabilidade Única**: não podemos sobrecarregar a classe com tarefas que não fazem parte do seu objetivo.
 - ✓ Devemos **desacoplar** o mecanismo de persistência da classe que o utiliza, para não tornar a classe dependente de uma tecnologia específica.
 - ✓ **Todas** as classes persistentes devem adotar uma estratégia **consistente** para implementar esse mecanismo.

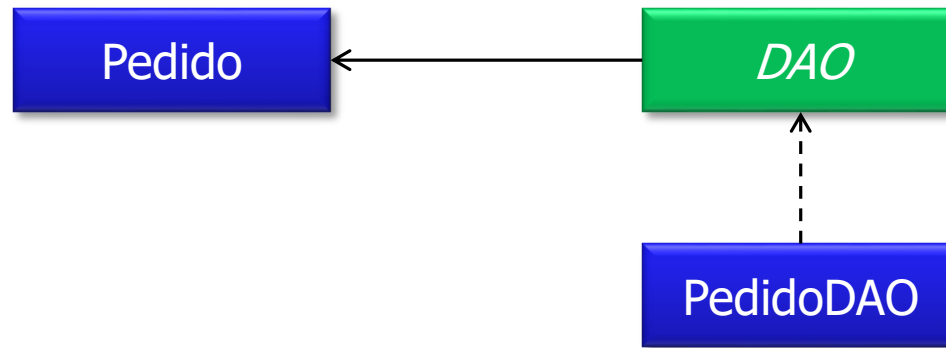
- Solução comumente adotada: criação de um padrão de acesso aos dados com classes DAO (*Data Access Objects*).



- Problemas com essa abordagem:
 - ✓ Cada classe pode implementar esse acesso de uma forma.
 - ✓ Não é possível reaproveitar comportamentos comuns.
 - ✓ Forte acoplamento entre as classes do domínio e os suas classes DAO, que estão em camadas distintas.

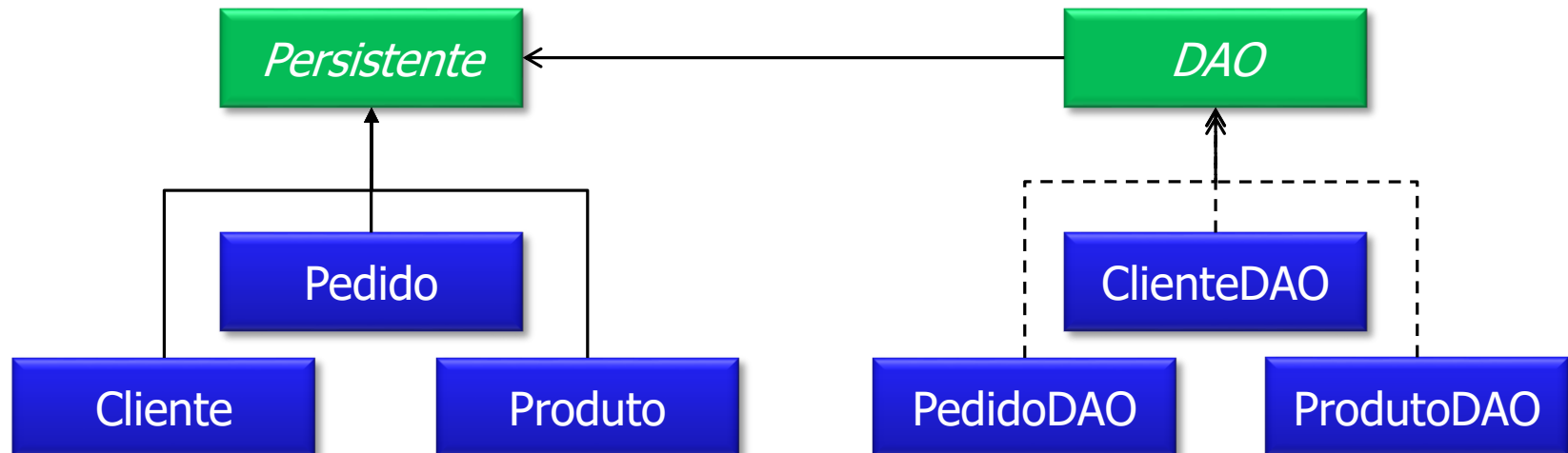


- De acordo com o princípio da **Inversão de Dependência** (DIP), podemos desacoplar as classes de domínio das classes DAO usando abstrações.

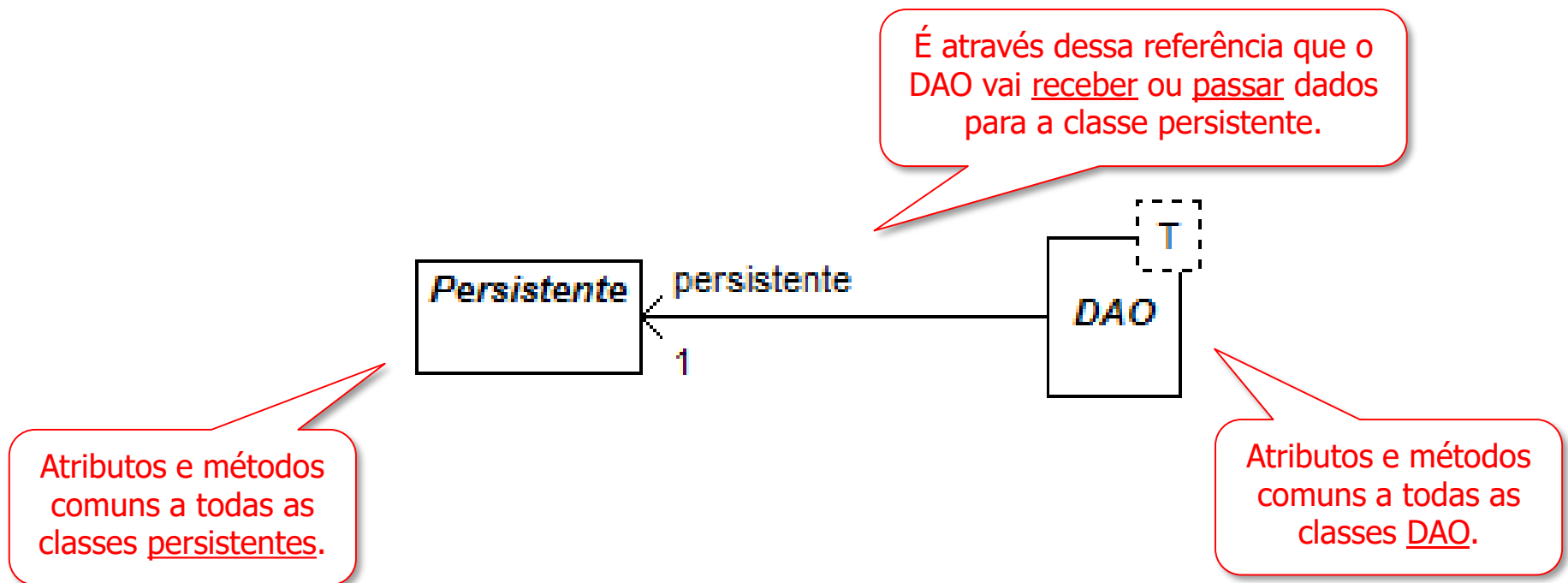


- Note que essa solução faz com que a classe DAO acesse **somente** Pedido. Entretanto, a classe DAO deve acessar **qualquer classe que necessita dos serviços de persistência!**

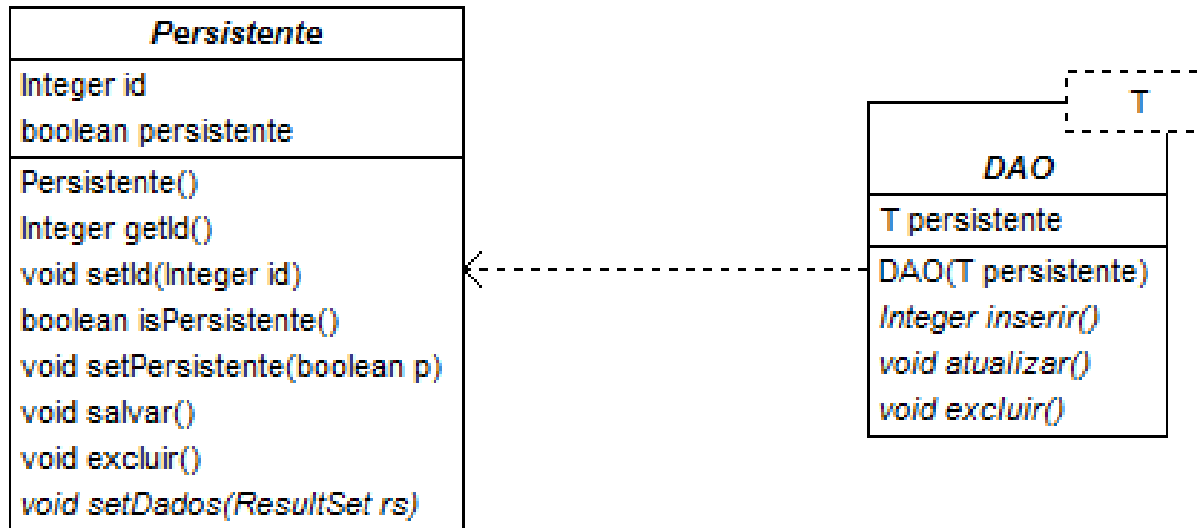
- Vamos aplicar novamente o princípio da **Inversão de Dependência** (DIP) e criar **duas abstrações**:
 - ✓ Uma para representar as classes de domínio que precisam de persistência.
 - ✓ Outra para representar as classes que implementam os serviços de persistência.



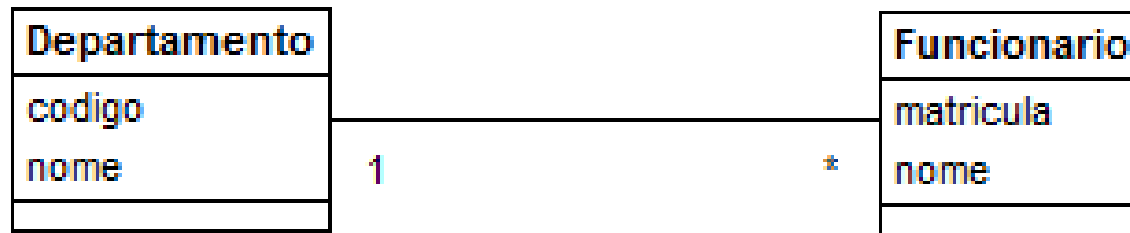
- ✓ As classes abstratas **Persistente** e **DAO** servirão de base para todas as classes persistentes e para as classes que acessam o banco de dados.



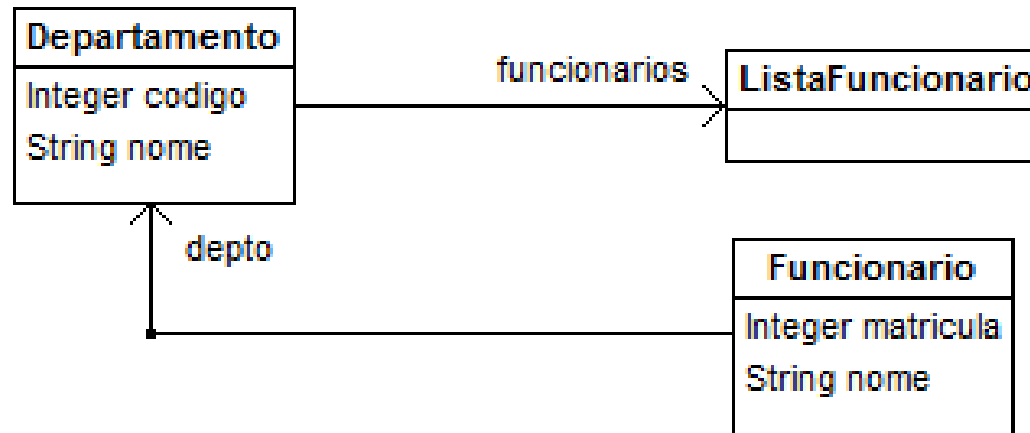
- Esses são os métodos **básicos** das classes **Persistente** e **DAO**.
- Para cada classe persistente específica e para cada classe DAO específica, novos métodos poderão ser acrescentados.



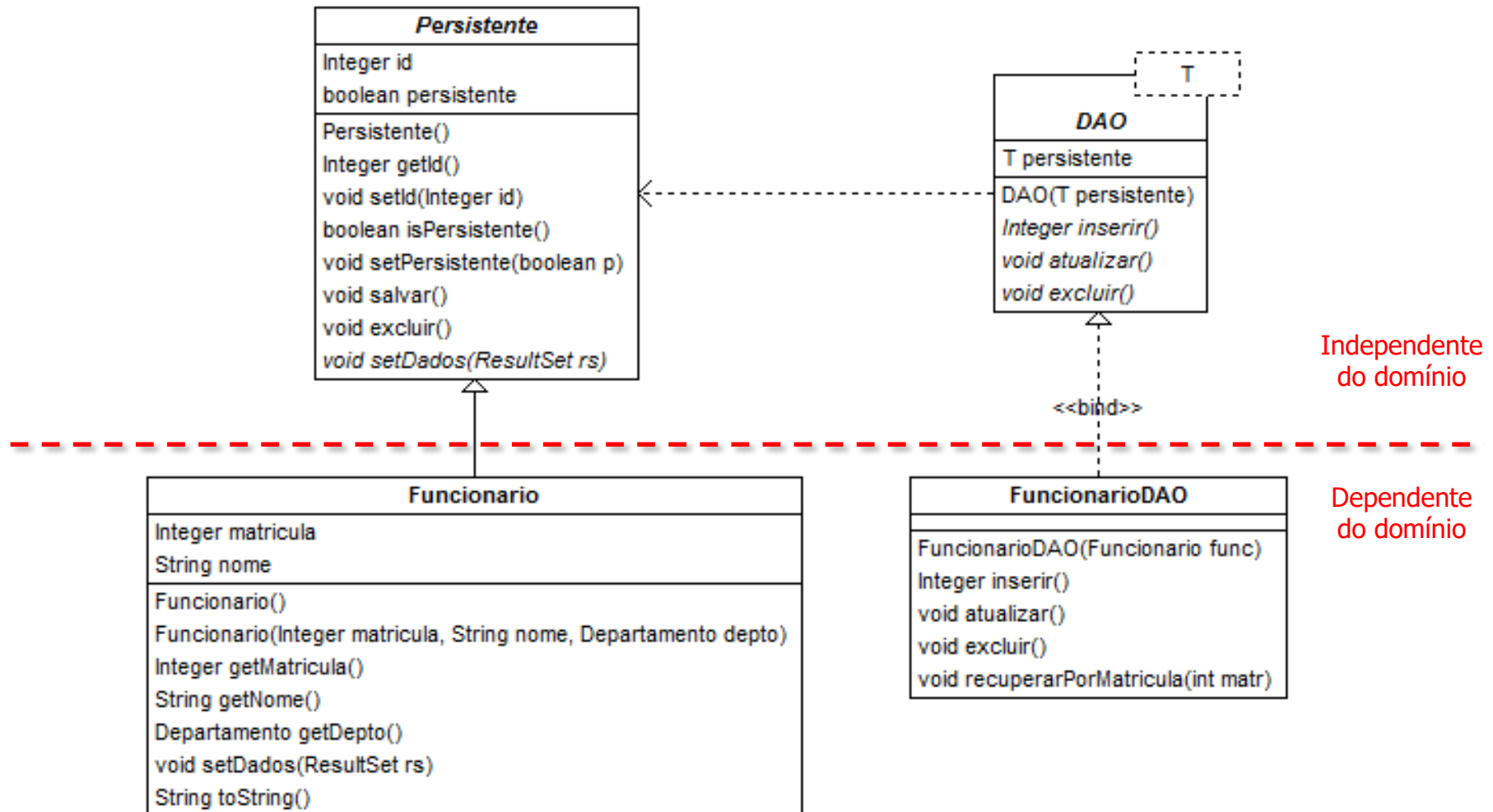
- Para discutir o uso desse padrão será usado um exemplo simples, definido pelo modelo abaixo.
- Nesse modelo **Departamento** e **Funcionario** são classes persistentes.



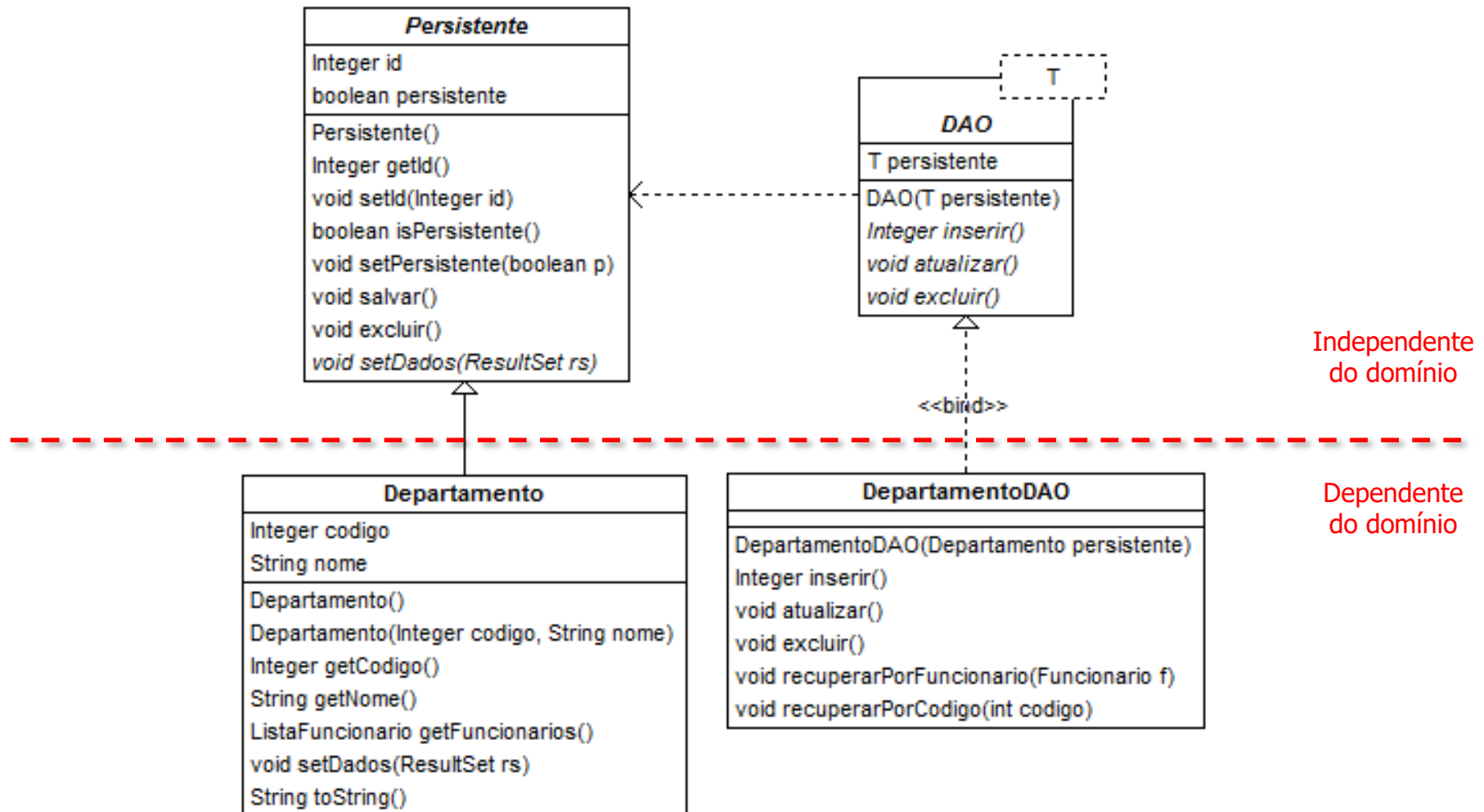
- Inicialmente, é necessário transformar esse modelo de domínio em **modelo de design**:



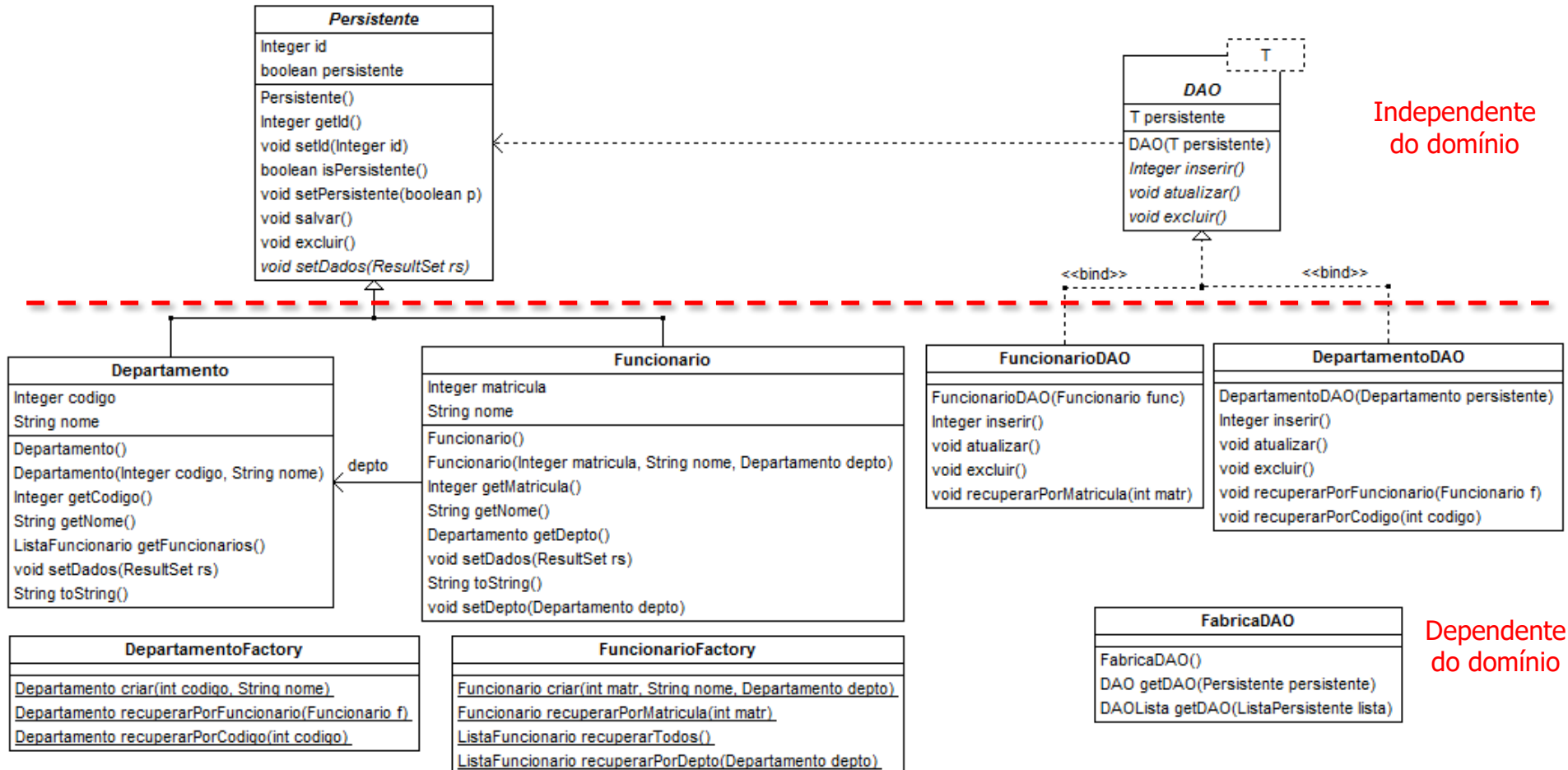
- Aplicando o padrão de acesso à classe **Funcionário**.



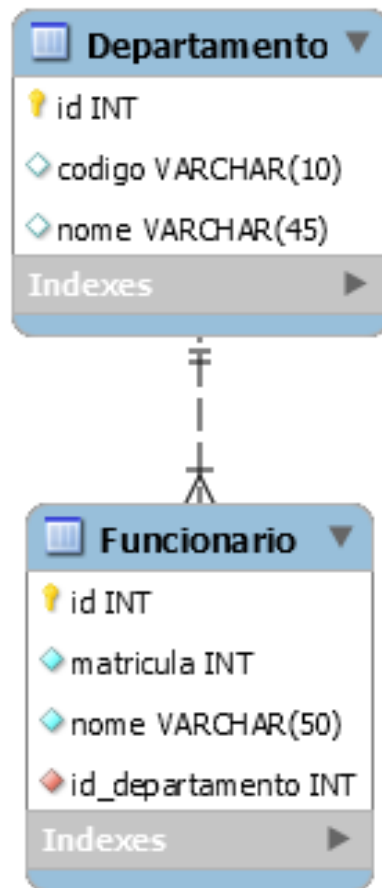
- Aplicando o padrão de acesso à classe **Departamento**.



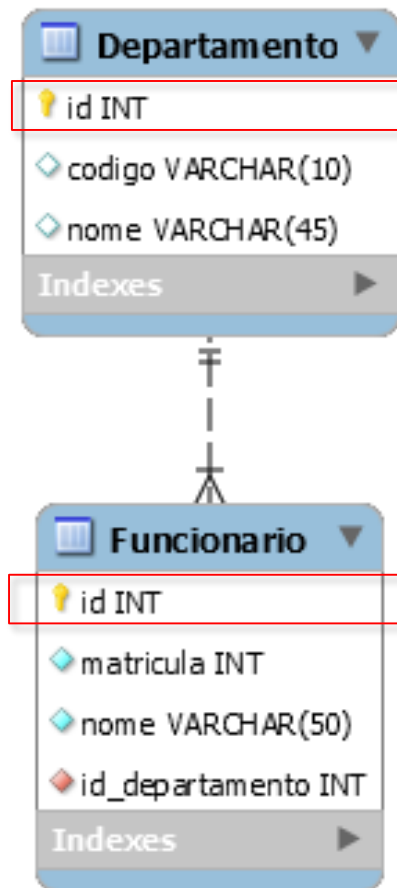
- Para criar objetos da classe **Funcionário** e **Departamento** e **DAO**, serão usadas **factories**, como foi feito no exemplo anterior.



- As classe **Funcionário** e **Departamento** serão mapeadas para as seguintes tabelas:

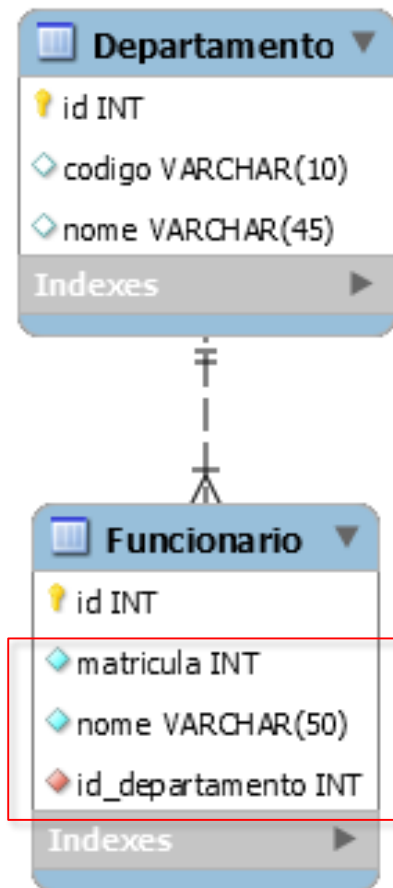


- As classe **Funcionário** e **Departamento** serão mapeadas para as seguintes tabelas:



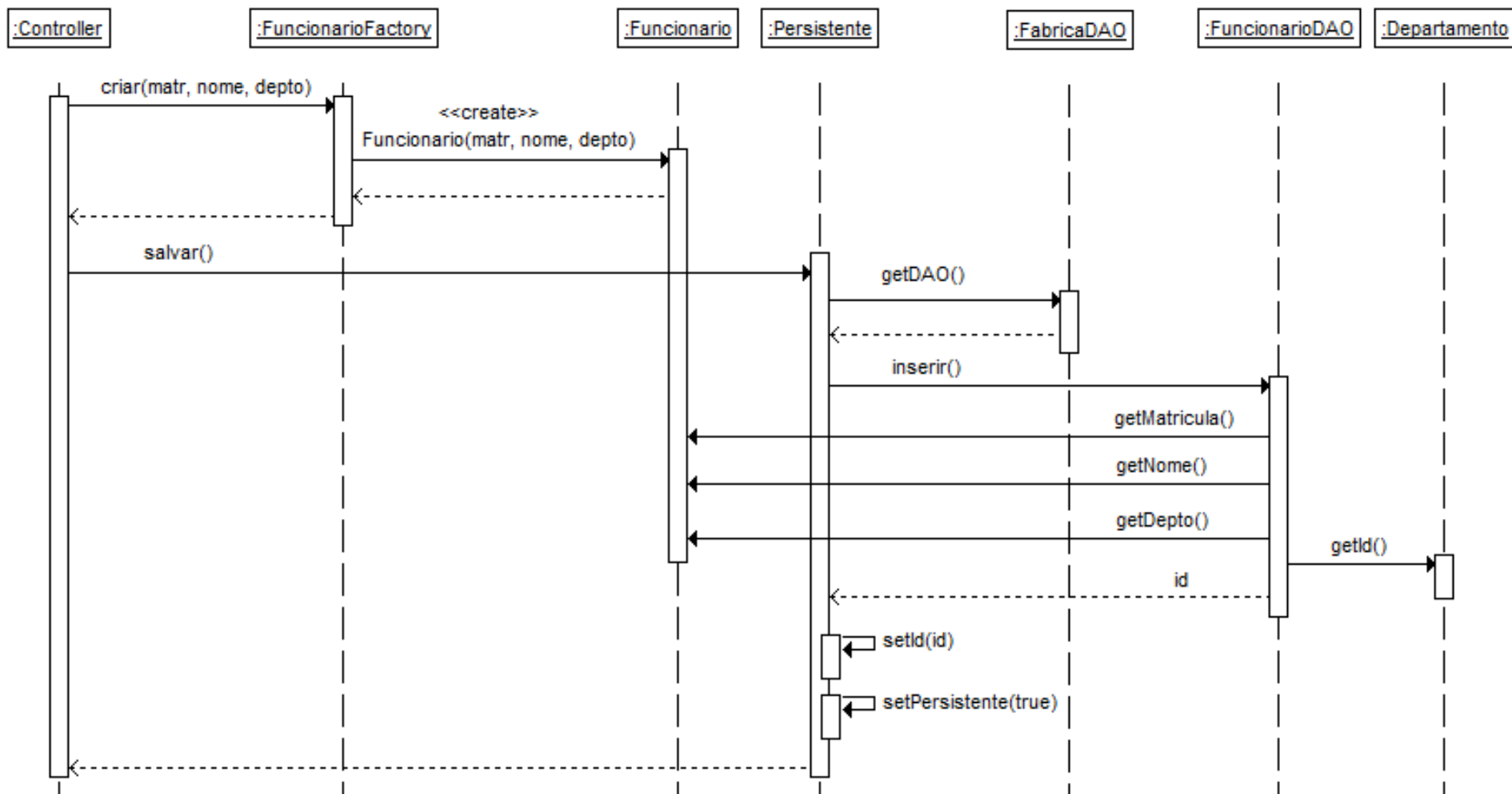
As chaves primárias de todas as tabelas serão geradas pelo próprio BD.

- As classe **Funcionário** e **Departamento** serão mapeadas para as seguintes tabelas:

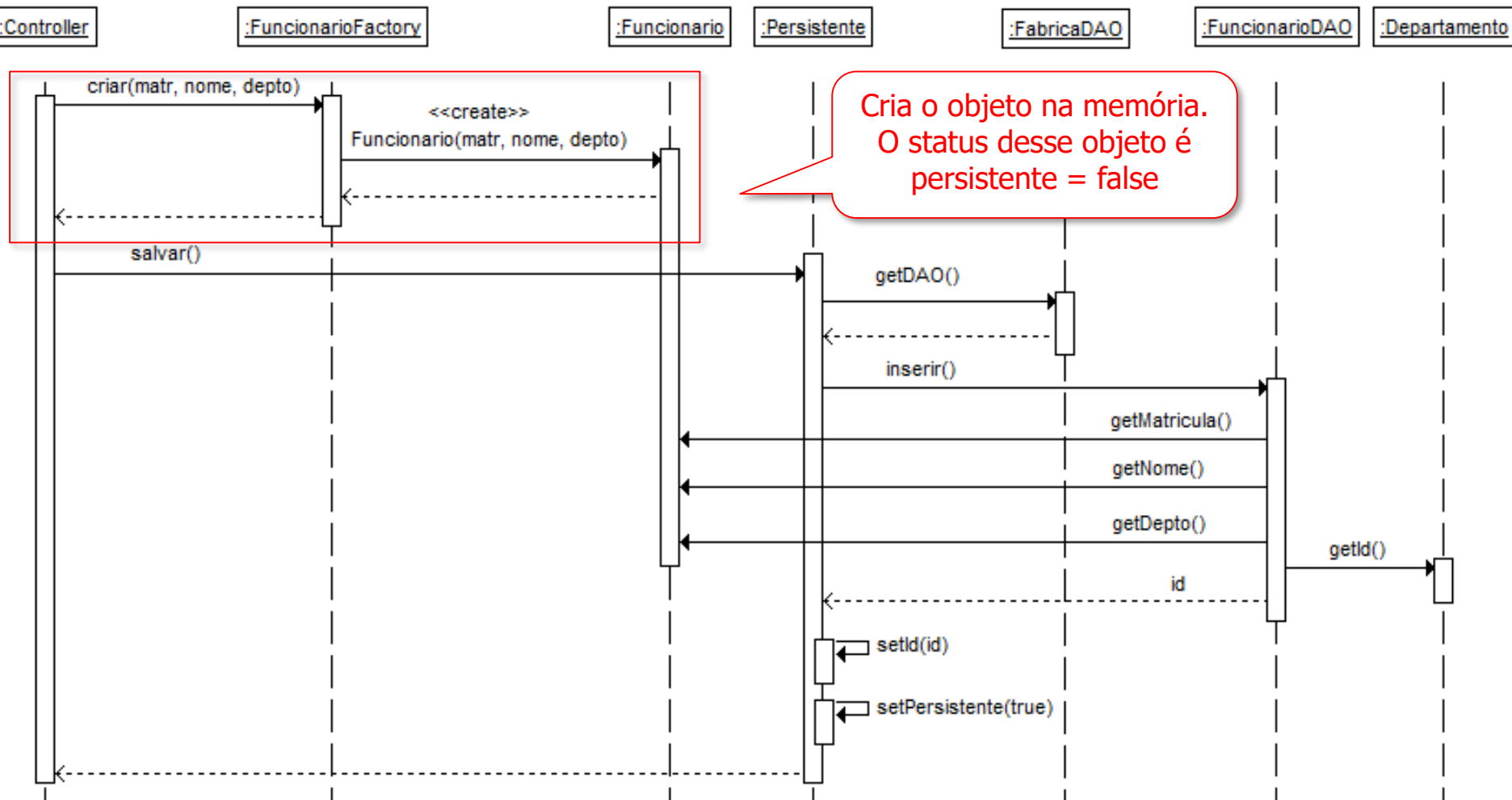


Note que para inserir um novo Funcionario são necessários a matrícula e o nome do funcionário, assim como o id do departamento.

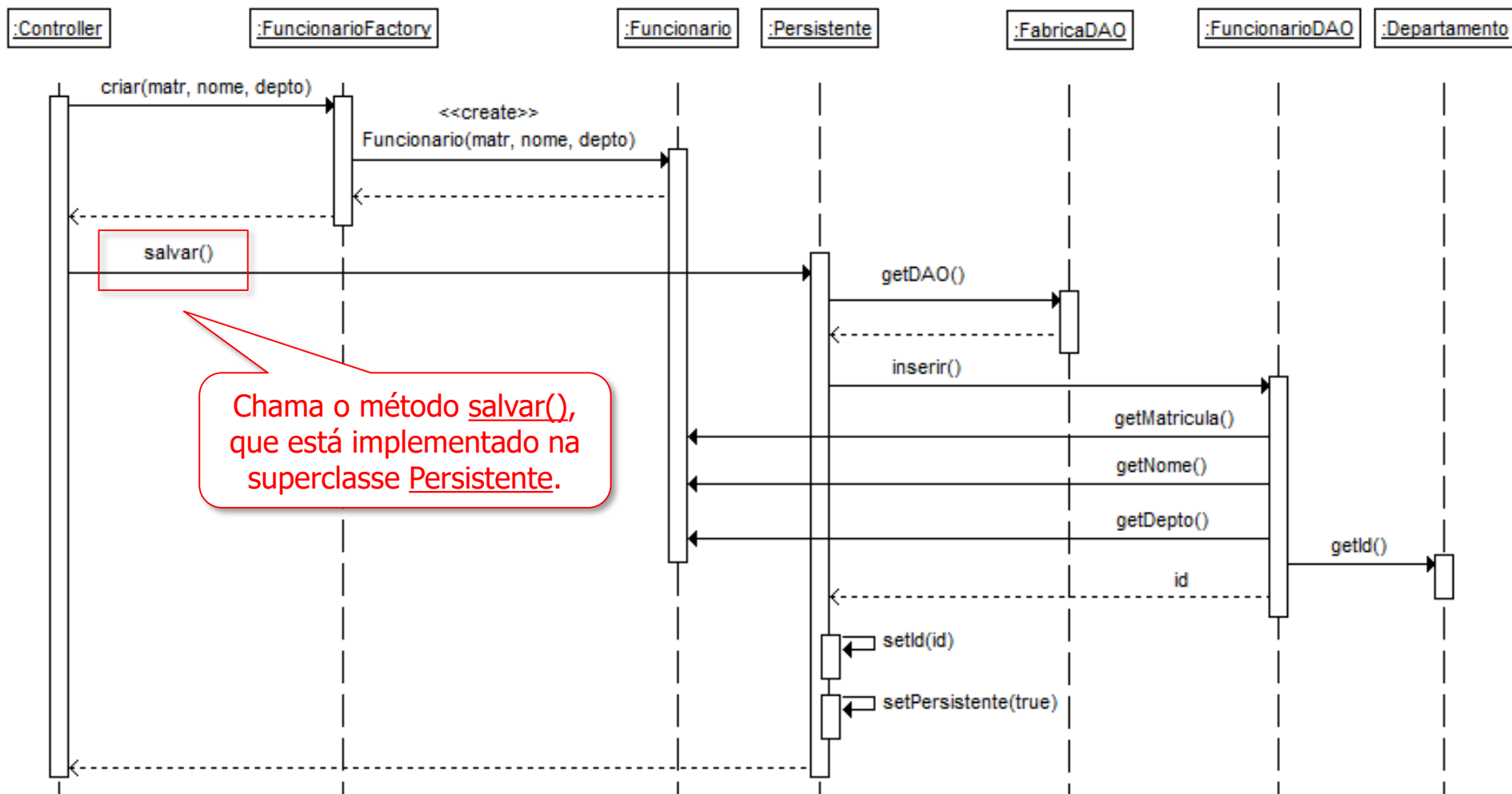
- Sequência para **criação** de um novo **Funcionário**:



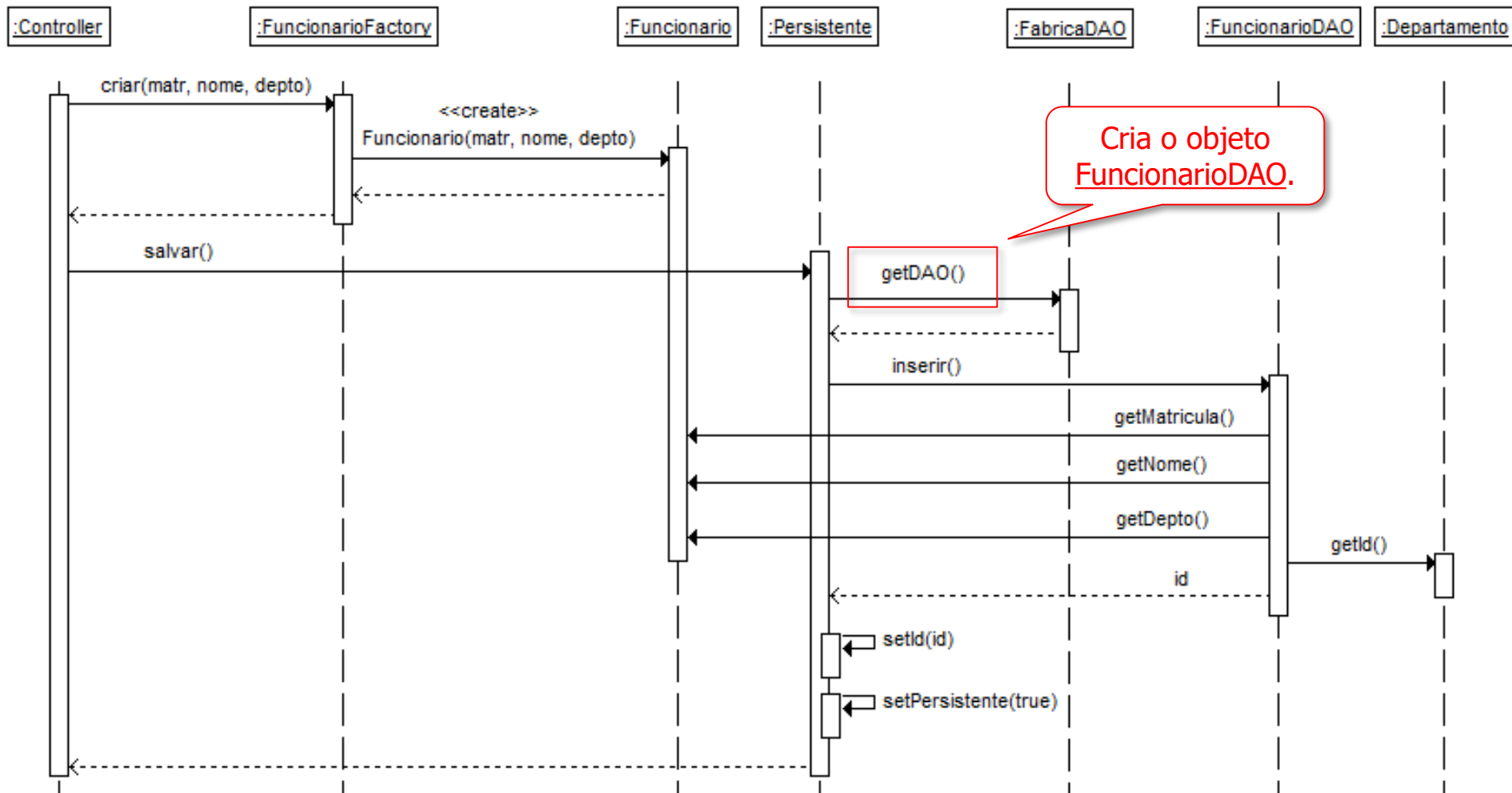
- Sequência para **criação** de um novo **Funcionário**:



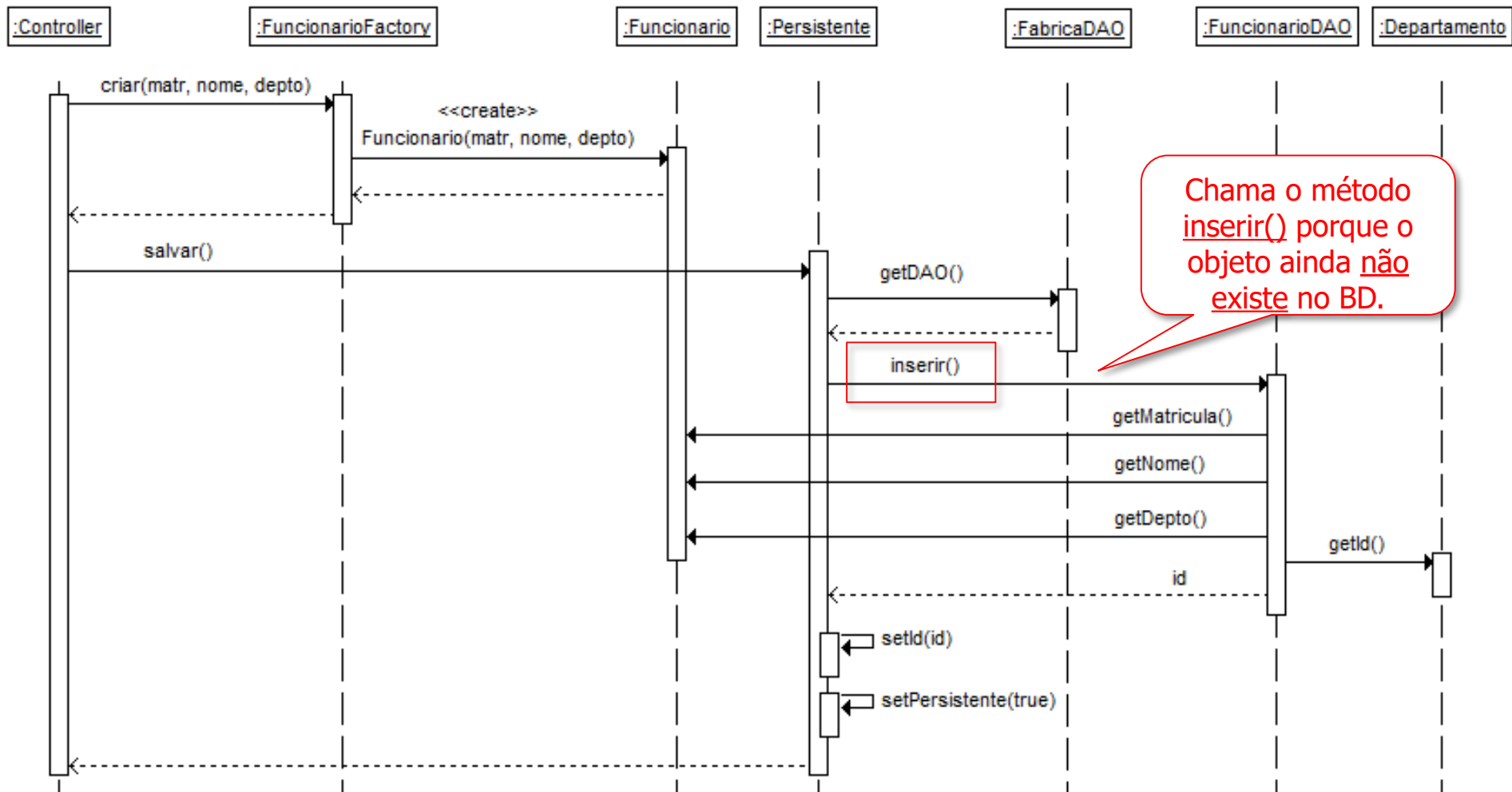
- Sequência para **criação** de um novo **Funcionário**:



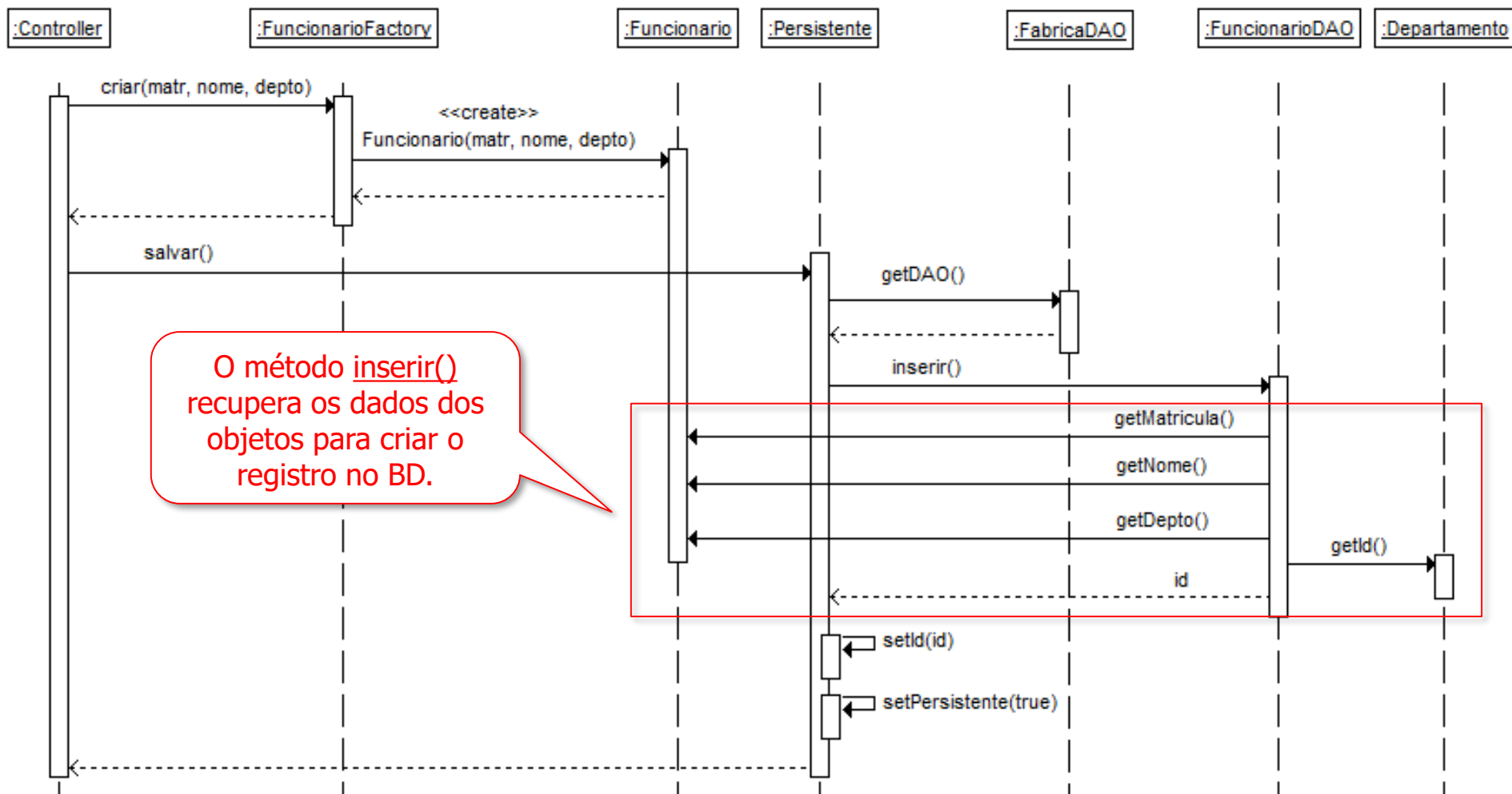
- Sequência para **criação** de um novo **Funcionário**:



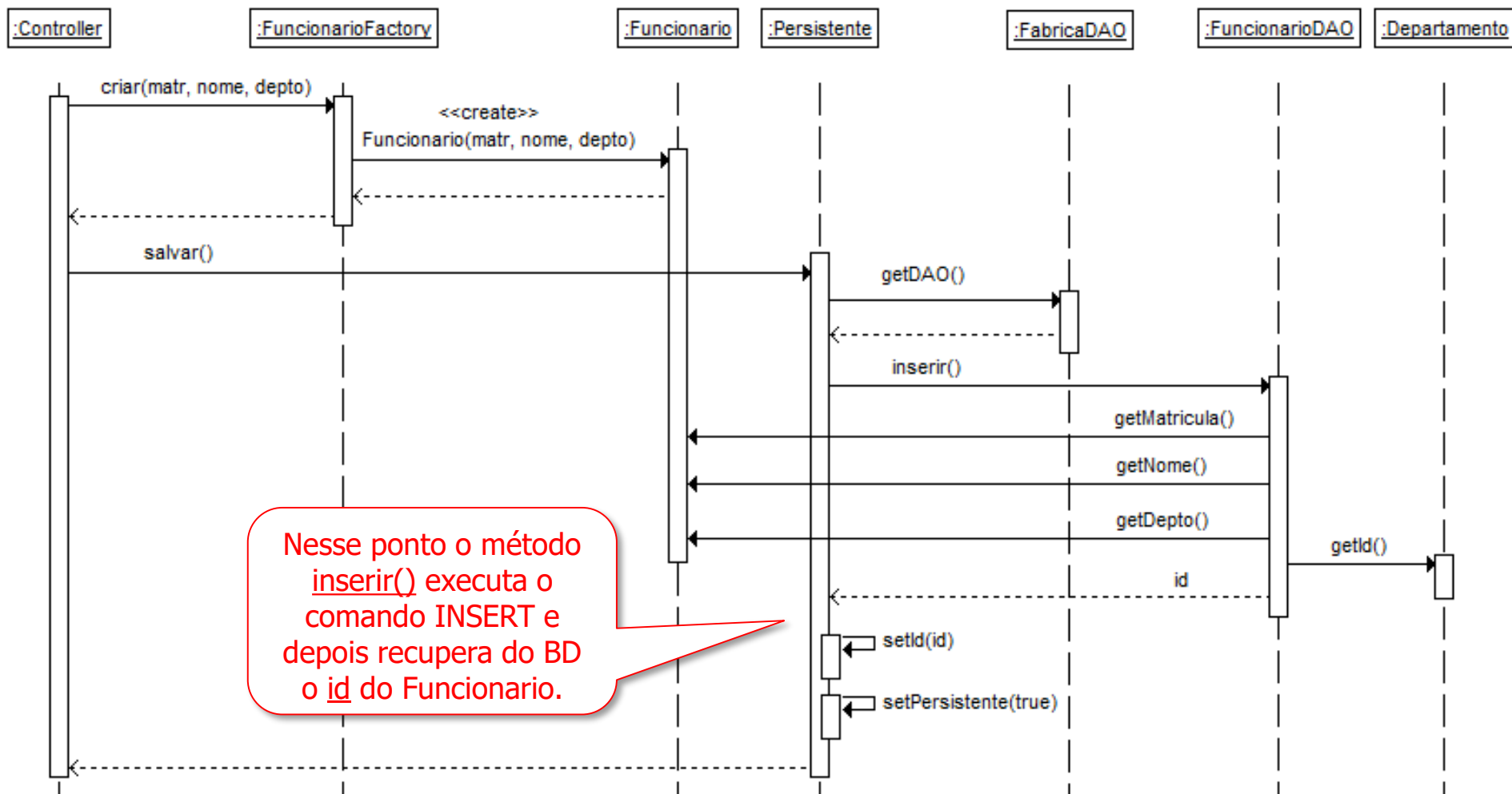
➤ Sequência para **criação** de um novo **Funcionário**:



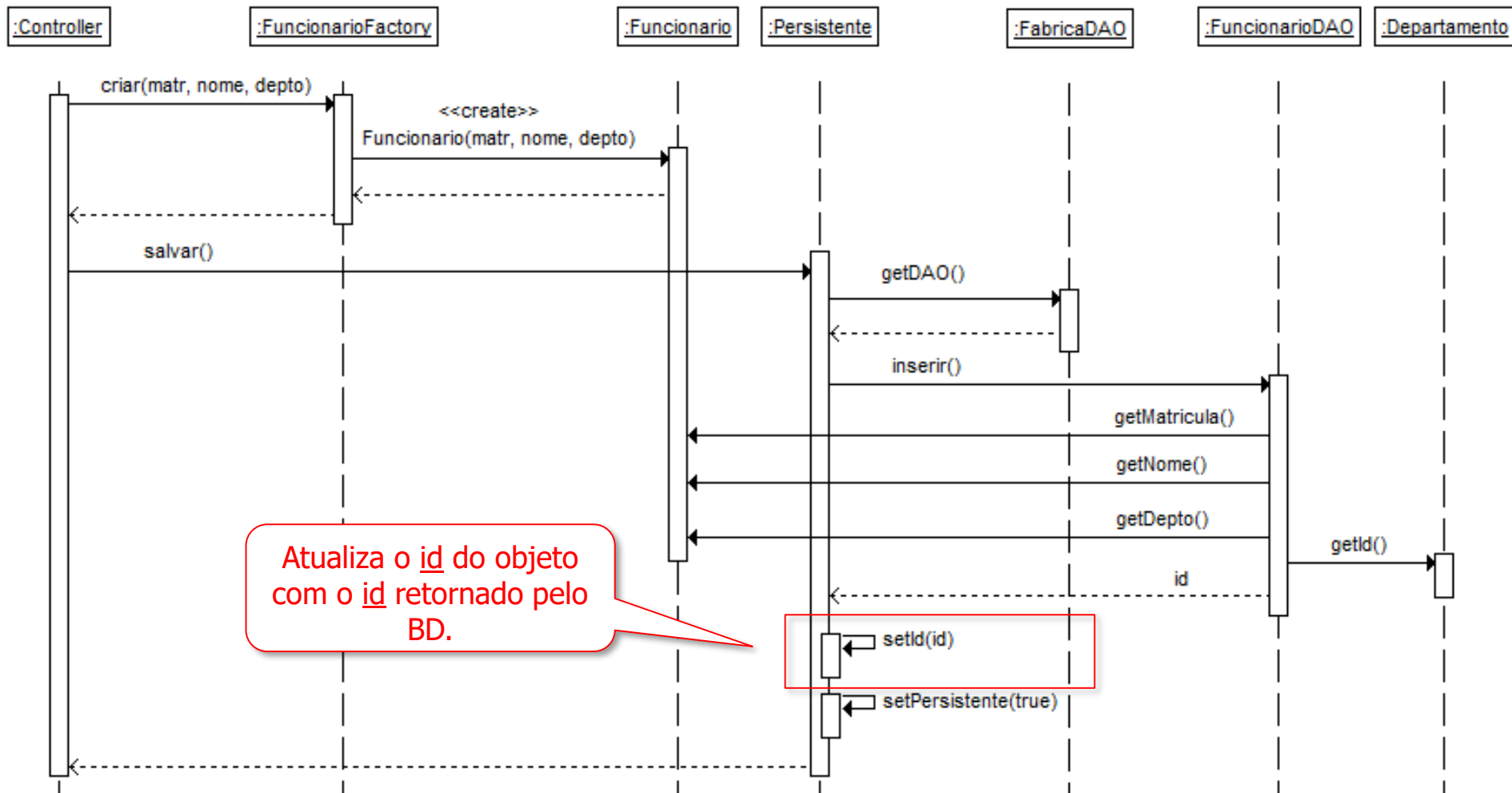
- Sequência para **criação** de um novo **Funcionário**:



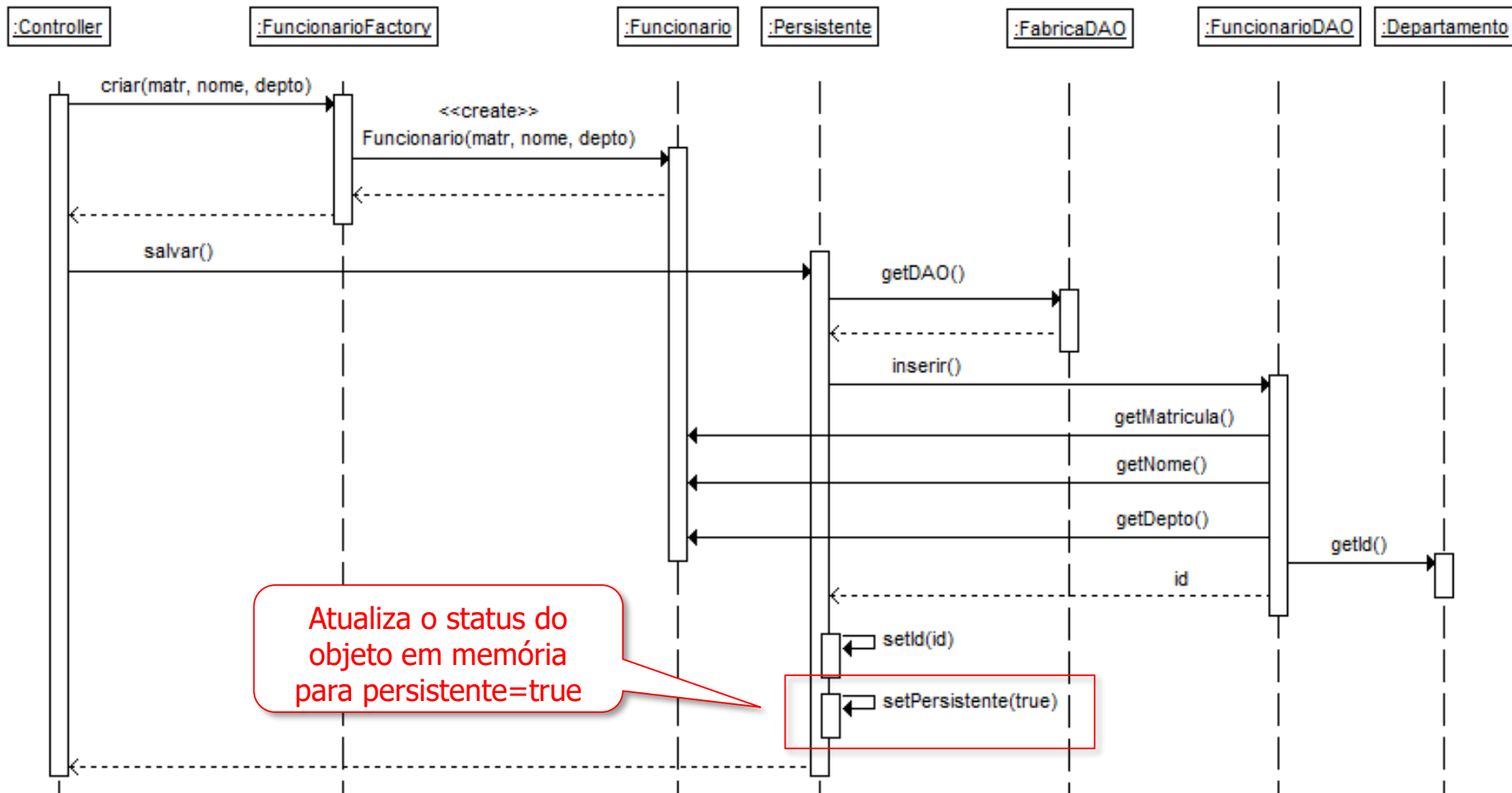
- Sequência para **criação** de um novo **Funcionário**:



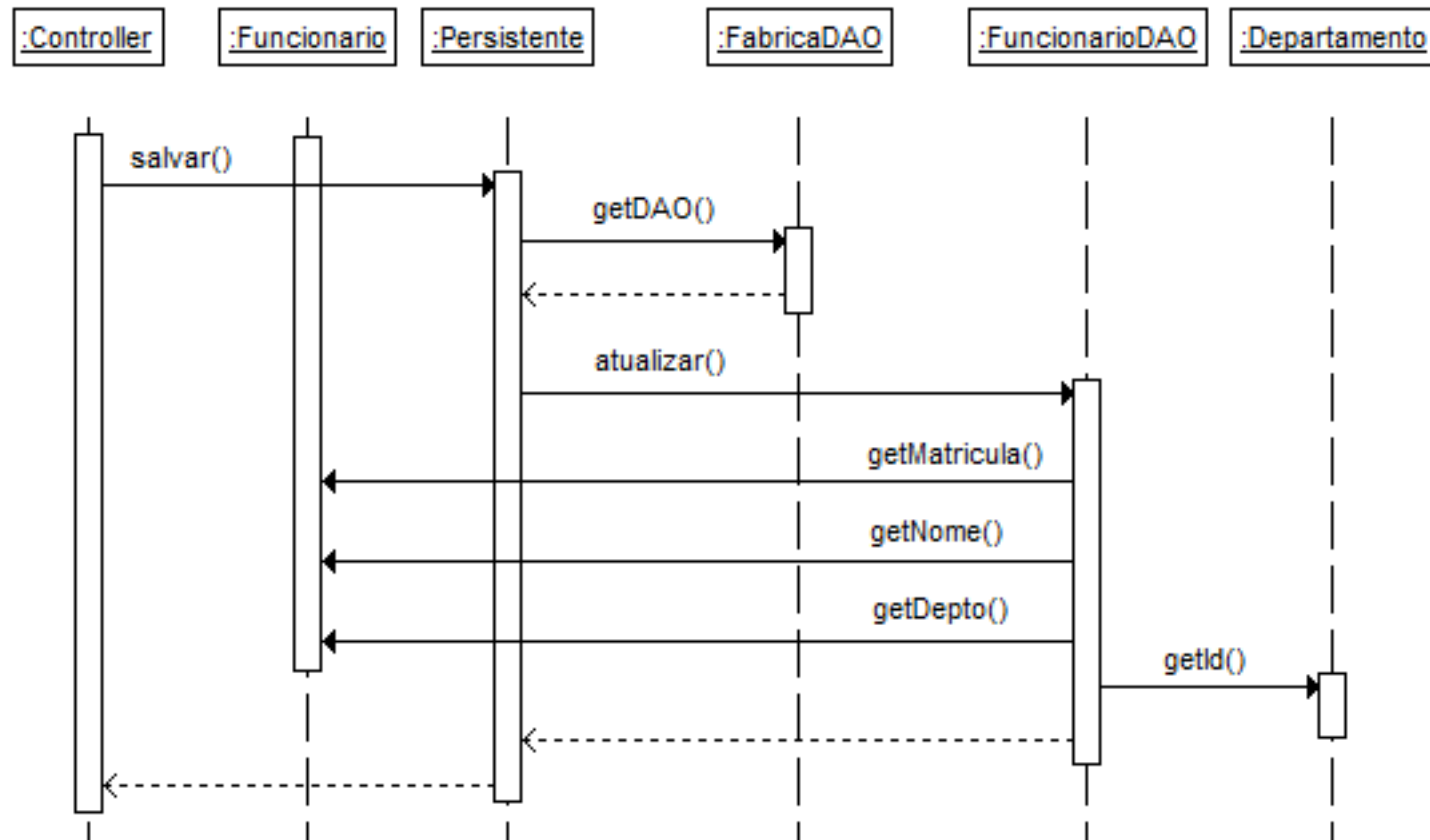
➤ Sequência para **criação** de um novo **Funcionário**:



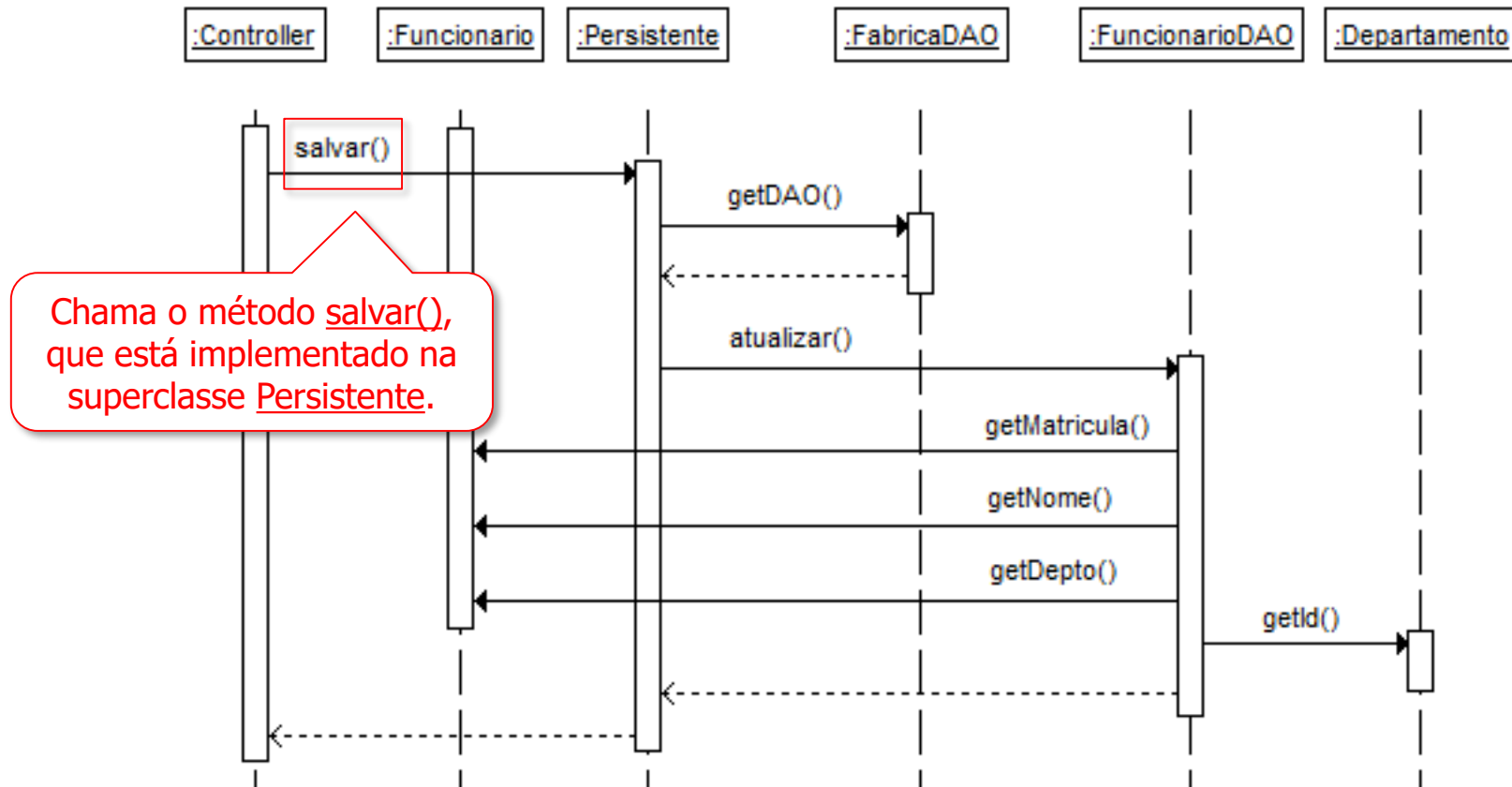
- Sequência para **criação** de um novo **Funcionário**:



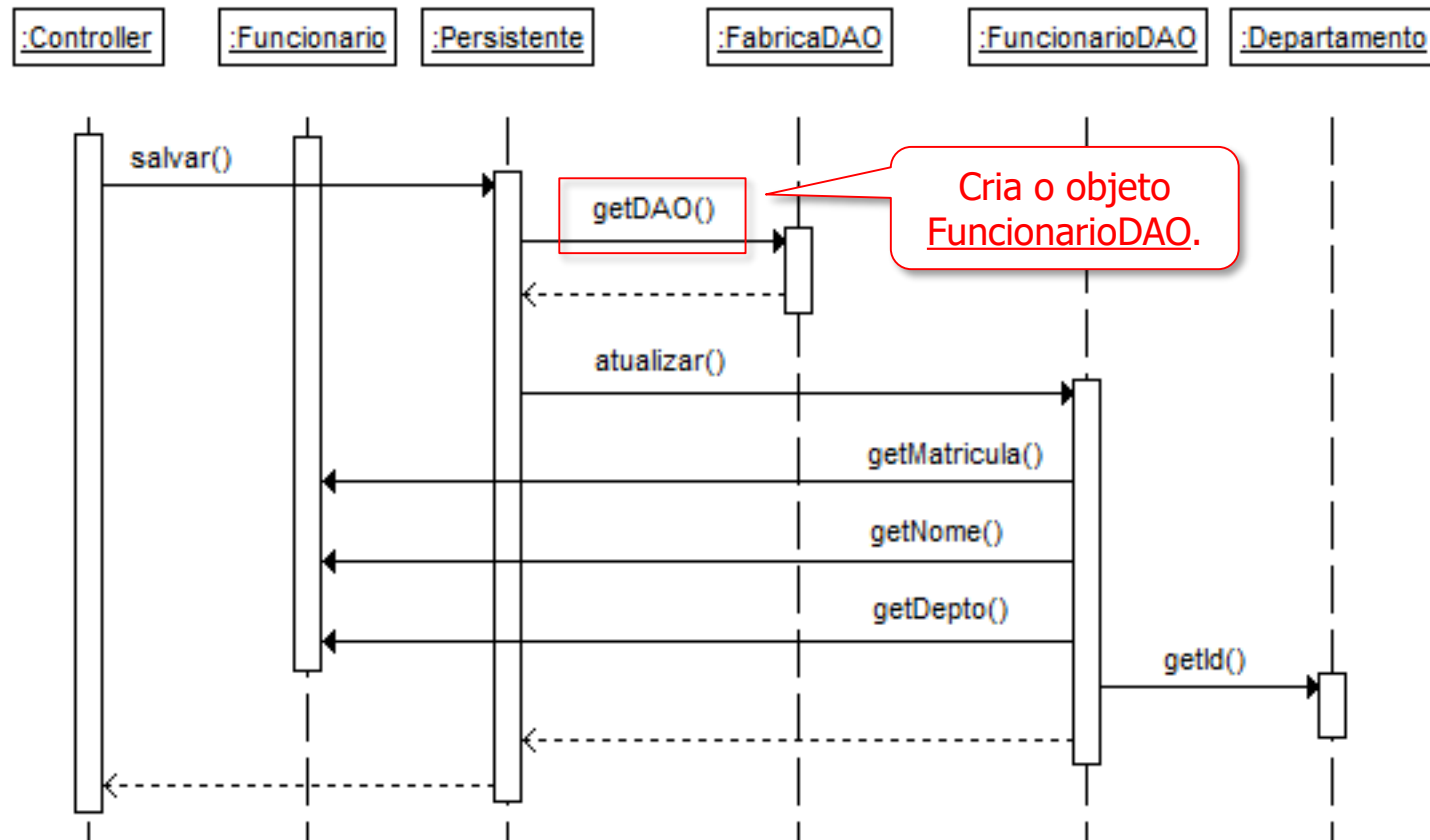
- Sequência para **atualização** dos dados do **Funcionário**:



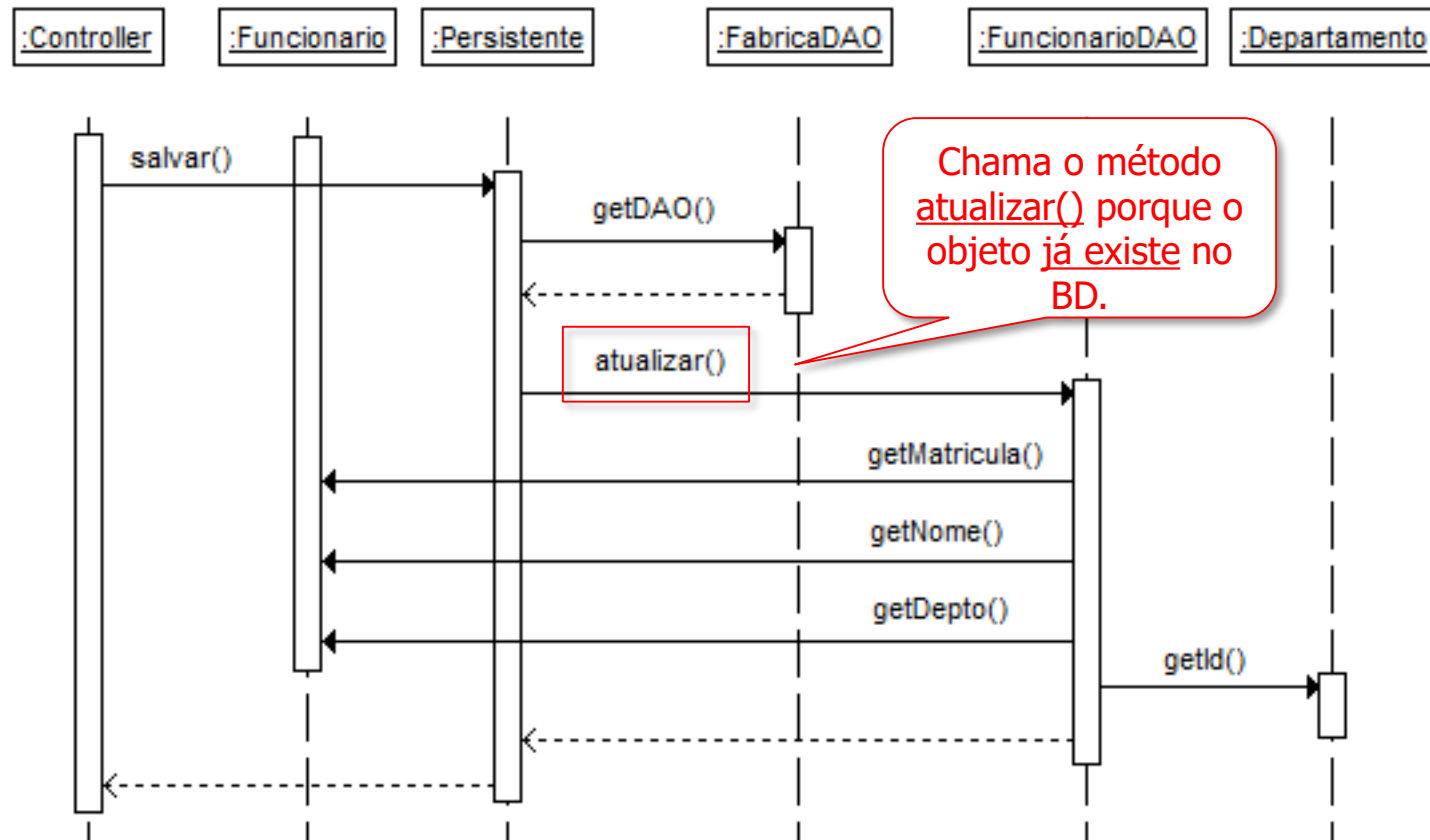
- Sequência para **atualização** dos dados do **Funcionário**:



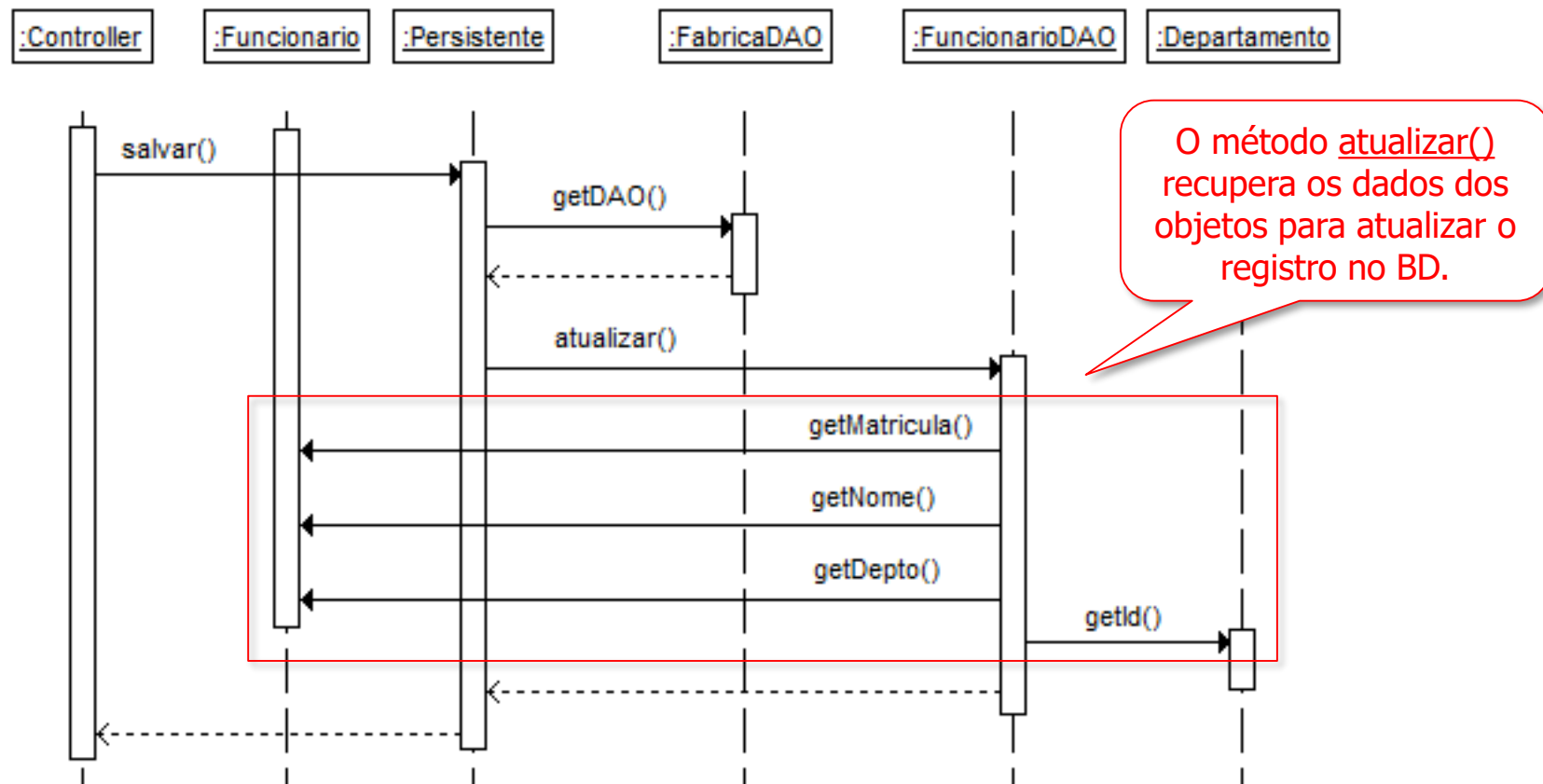
- Sequência para **atualização** dos dados do **Funcionário**:



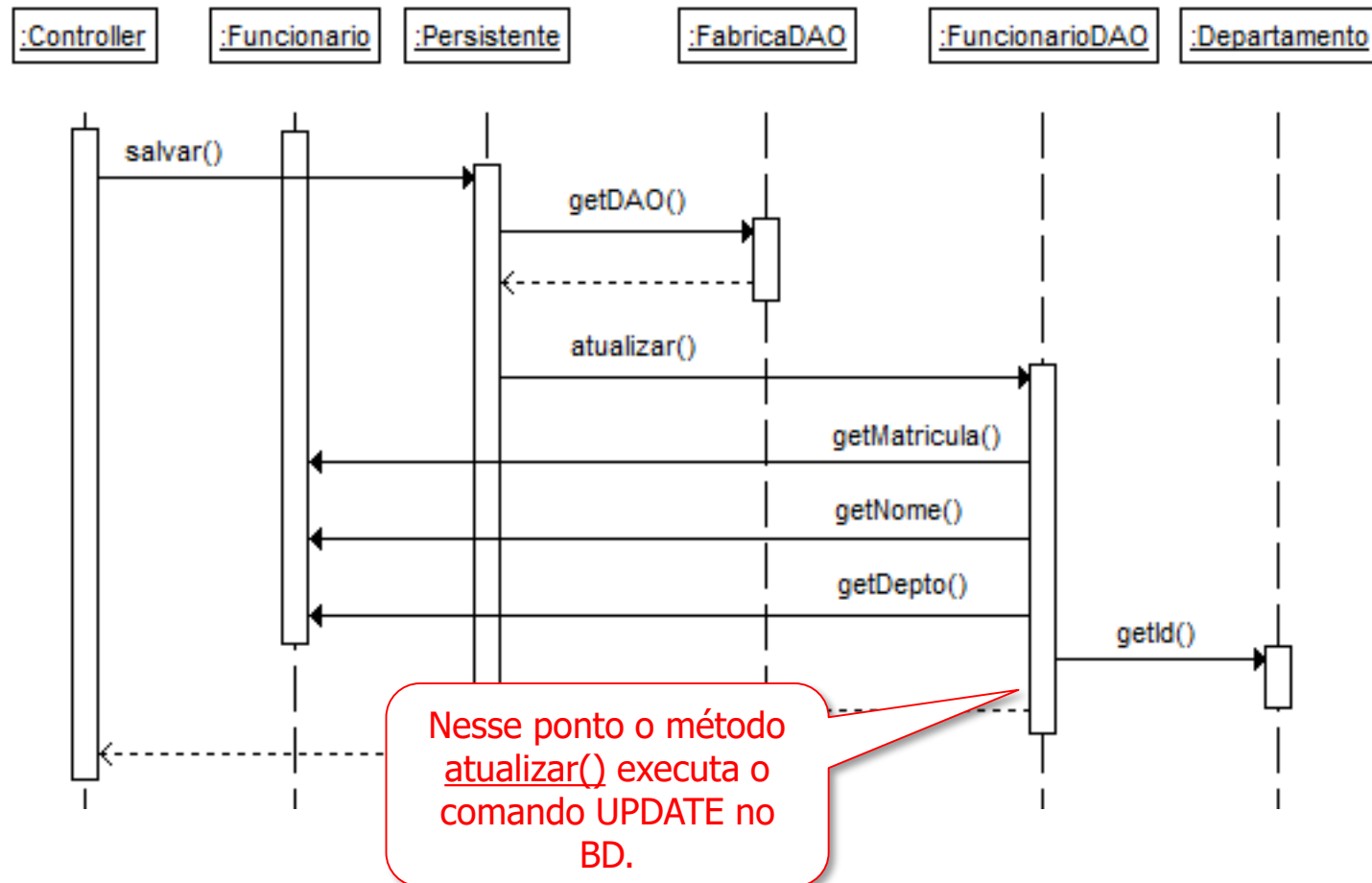
- Sequência para **atualização** dos dados do **Funcionário**:



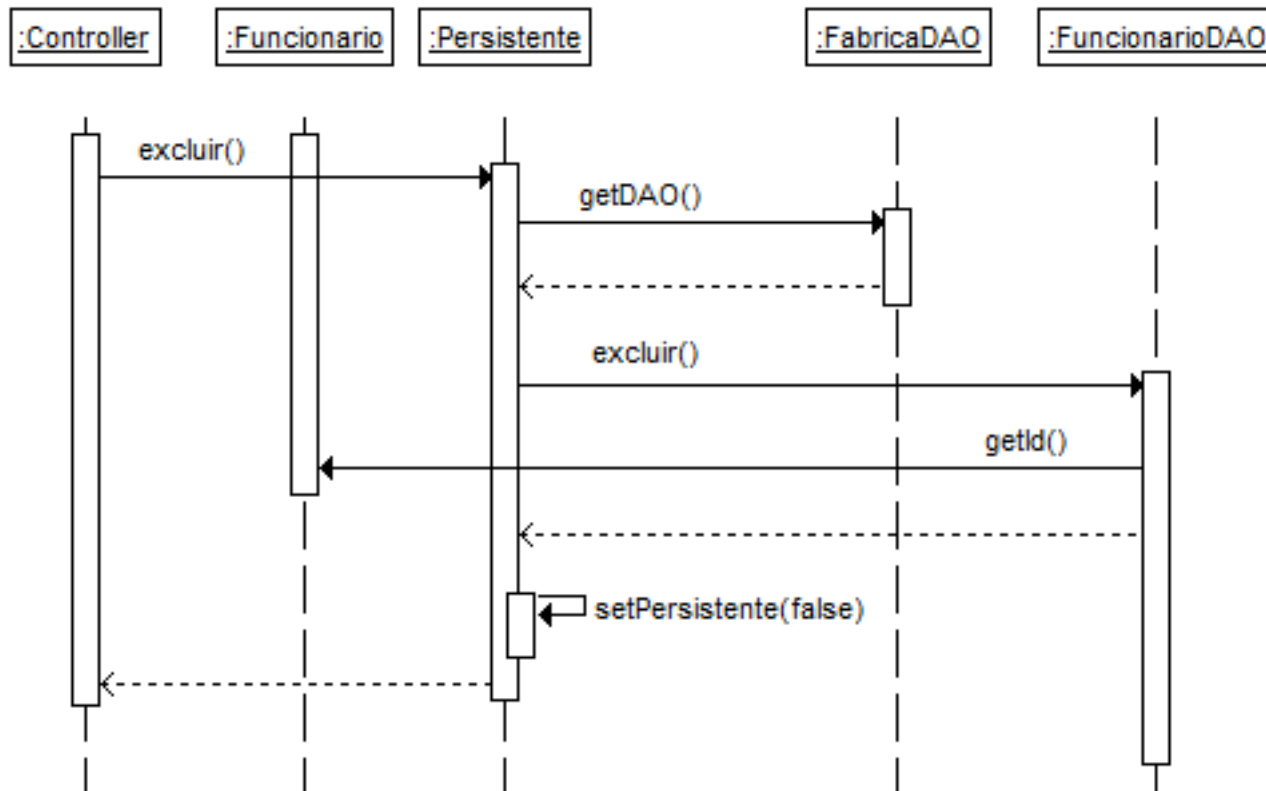
- Sequência para **atualização** dos dados do **Funcionário**:



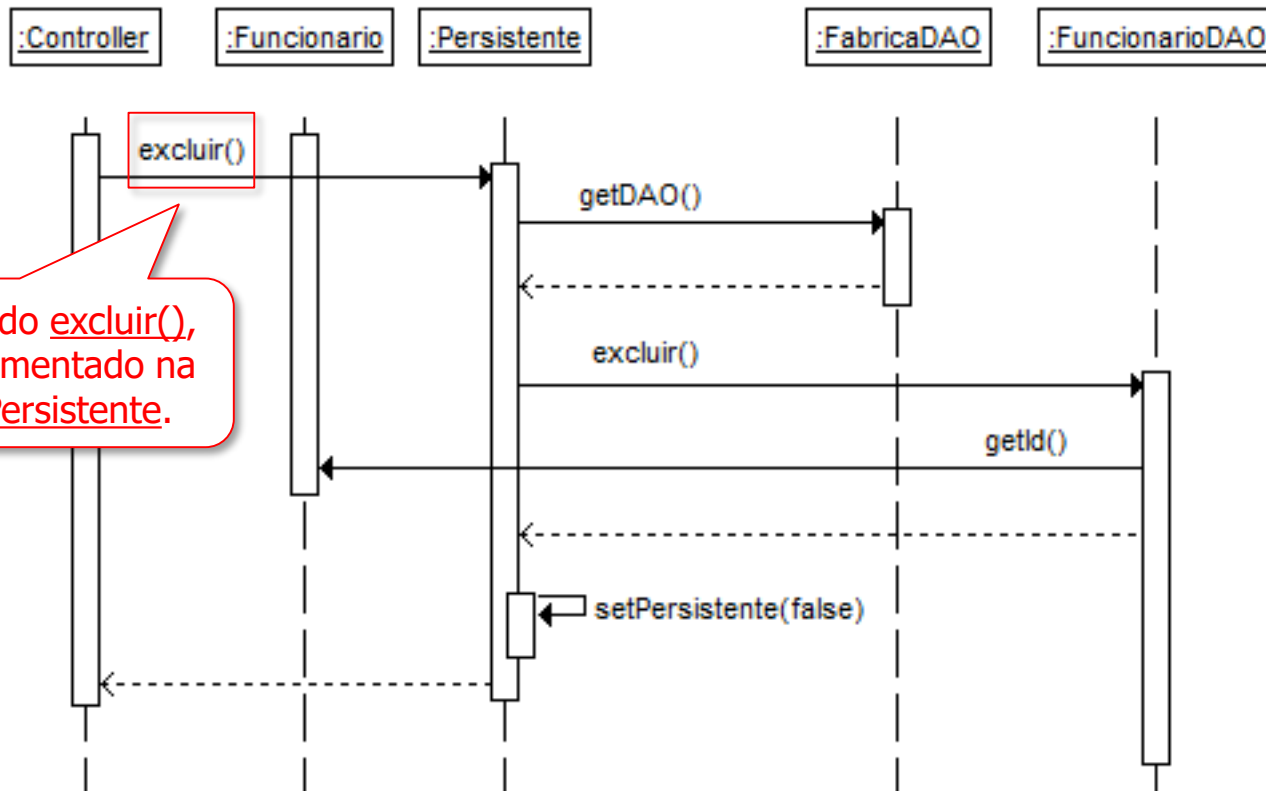
- Sequência para **atualização** dos dados do **Funcionário**:



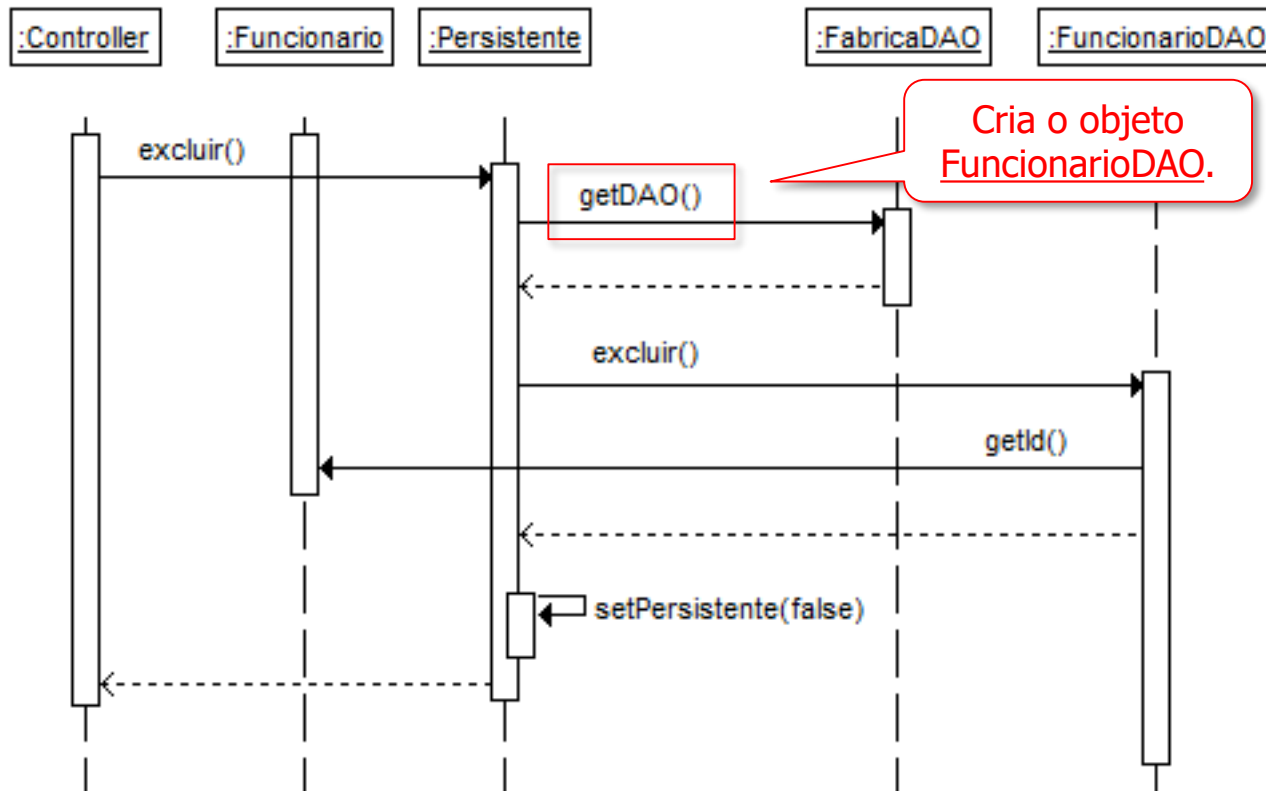
- Sequência para **exclusão** de um **Funcionário**:



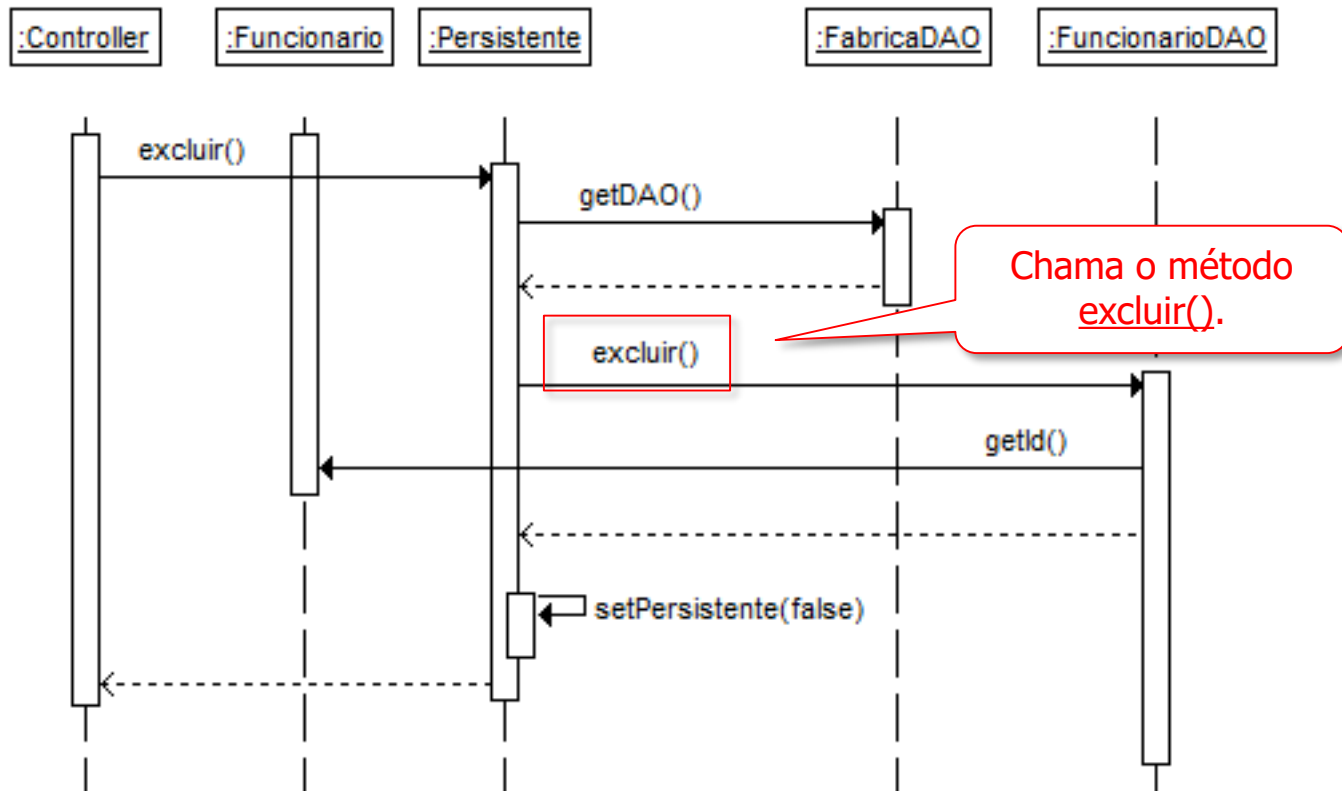
- Sequência para **exclusão** de um **Funcionário**:



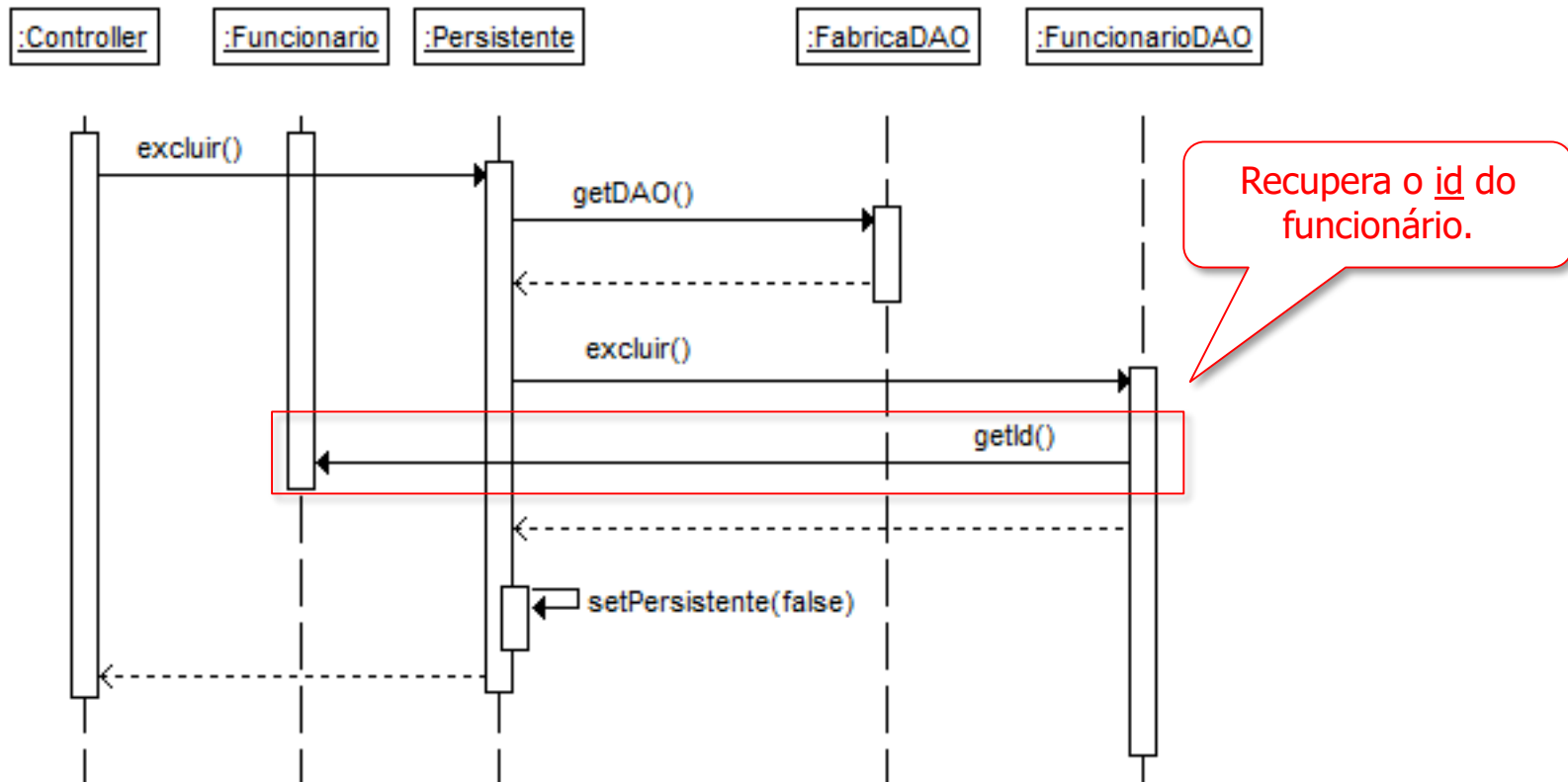
- Sequência para **exclusão** de um **Funcionário**:



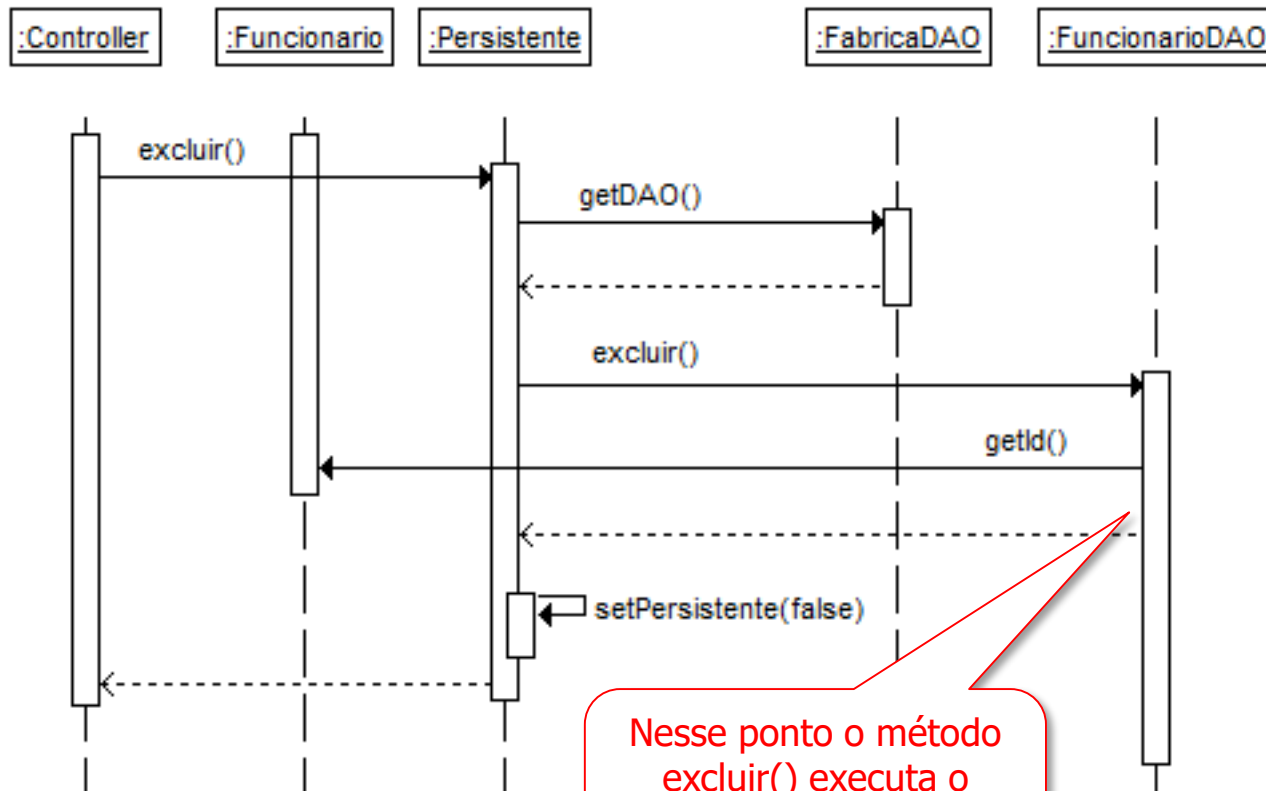
- Sequência para **exclusão** de um **Funcionário**:



- Sequência para **exclusão** de um **Funcionário**:

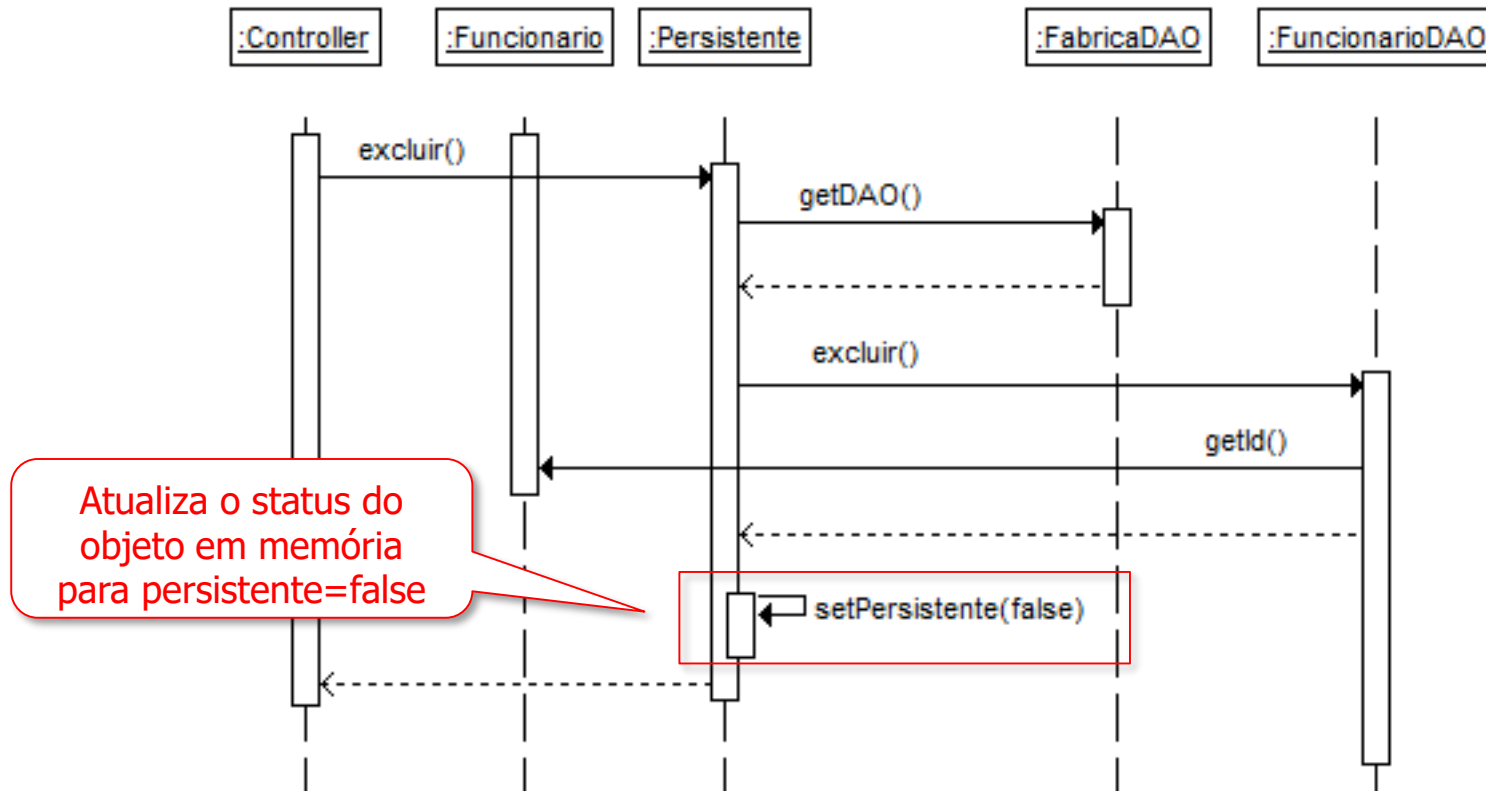


- Sequência para **exclusão** de um **Funcionário**:

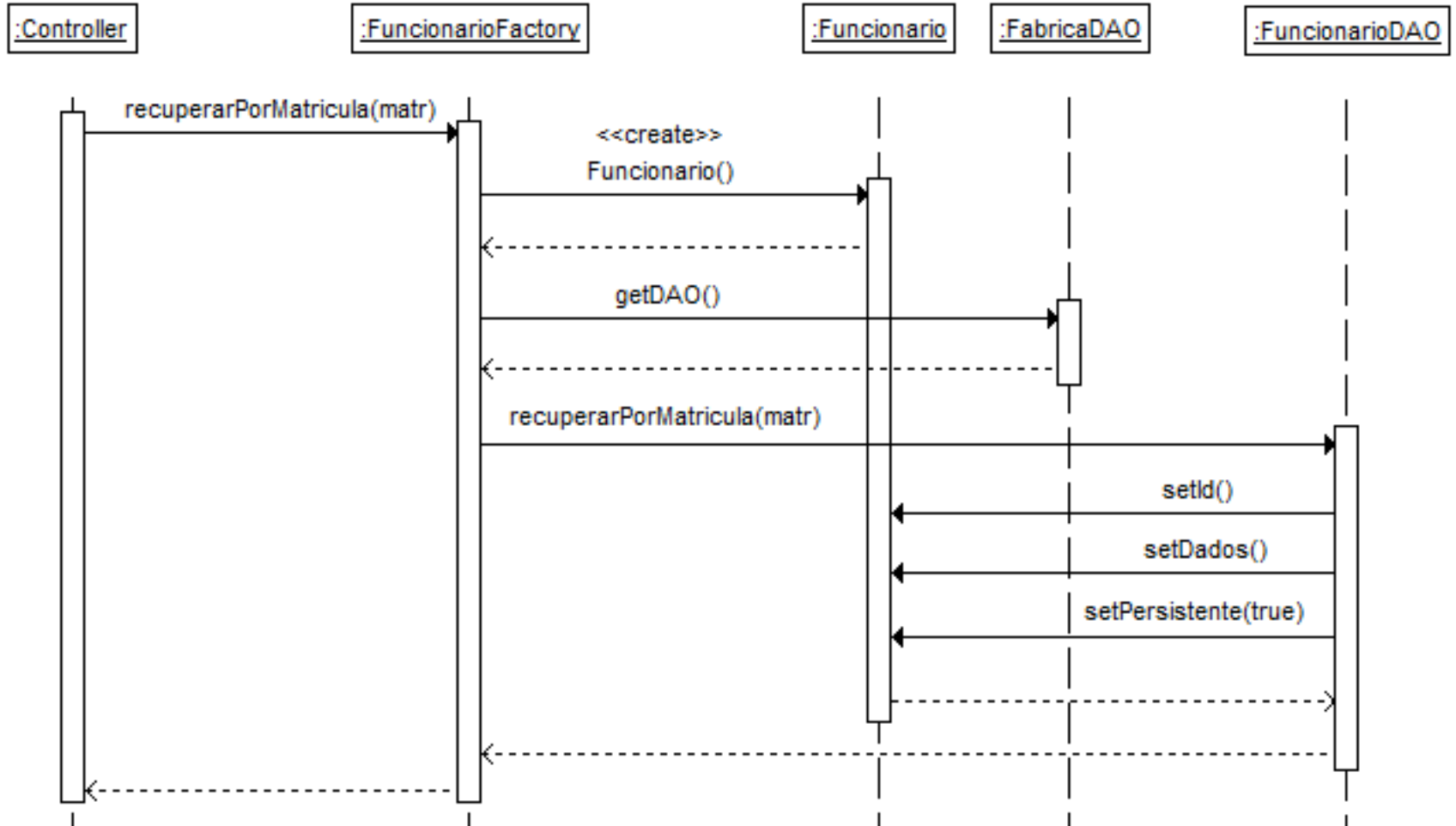


Nesse ponto o método excluir() executa o comando DELETE no BD.

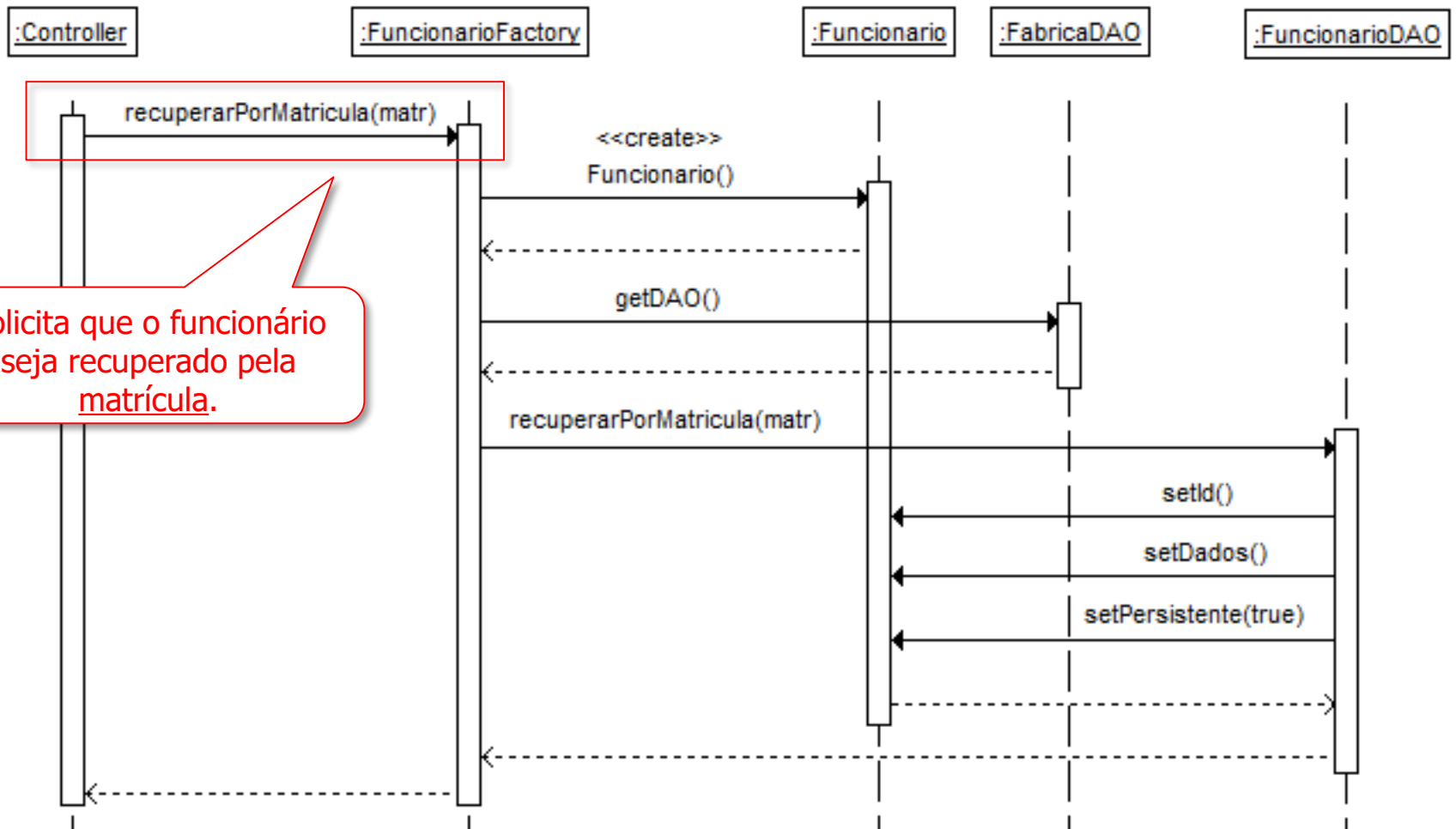
- Sequência para **exclusão** de um **Funcionário**:



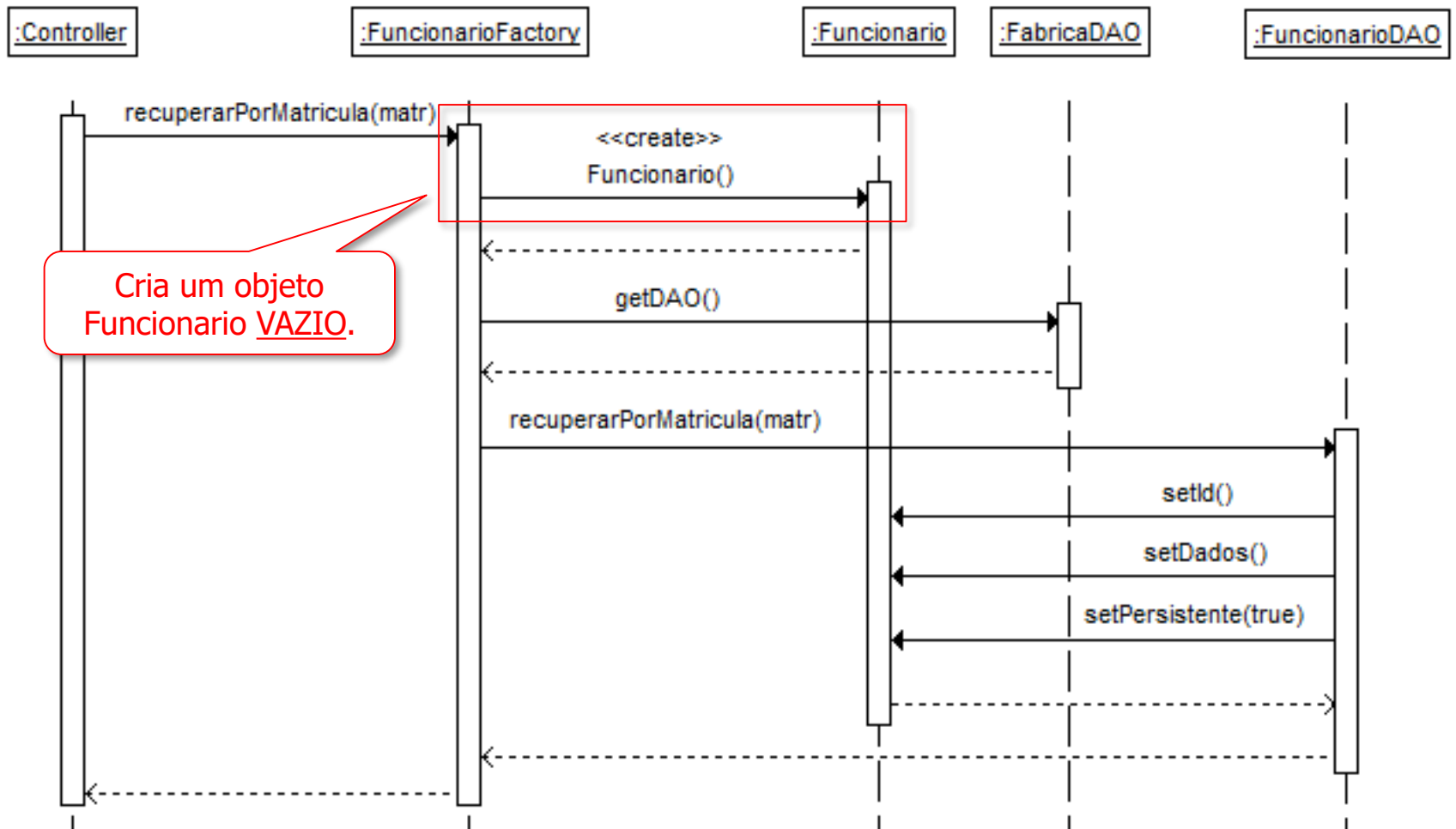
- Sequência para **recuperação** de um Funcionário pela **matrícula**:



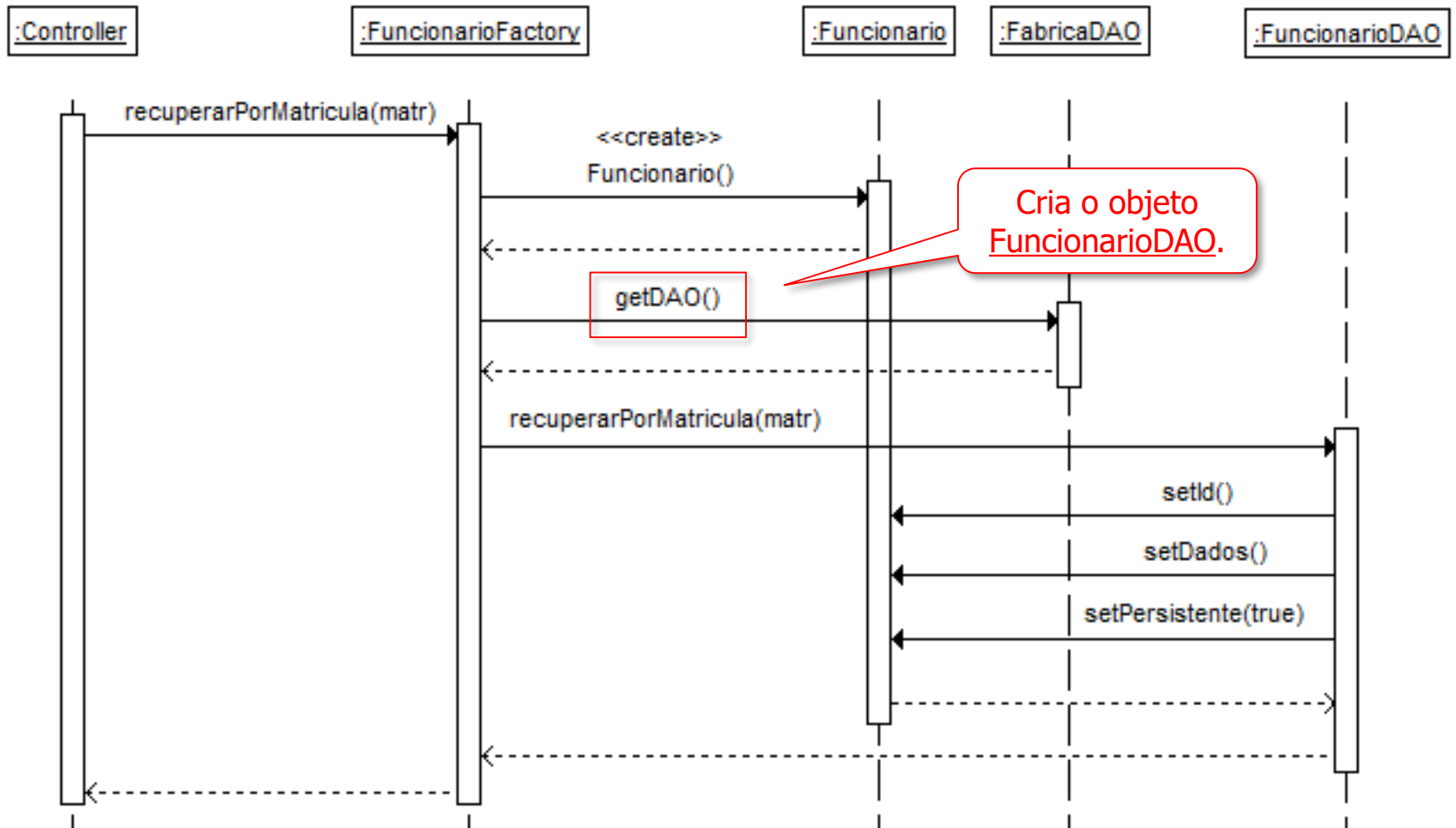
- Sequência para **recuperação** de um Funcionário pela **matrícula**:



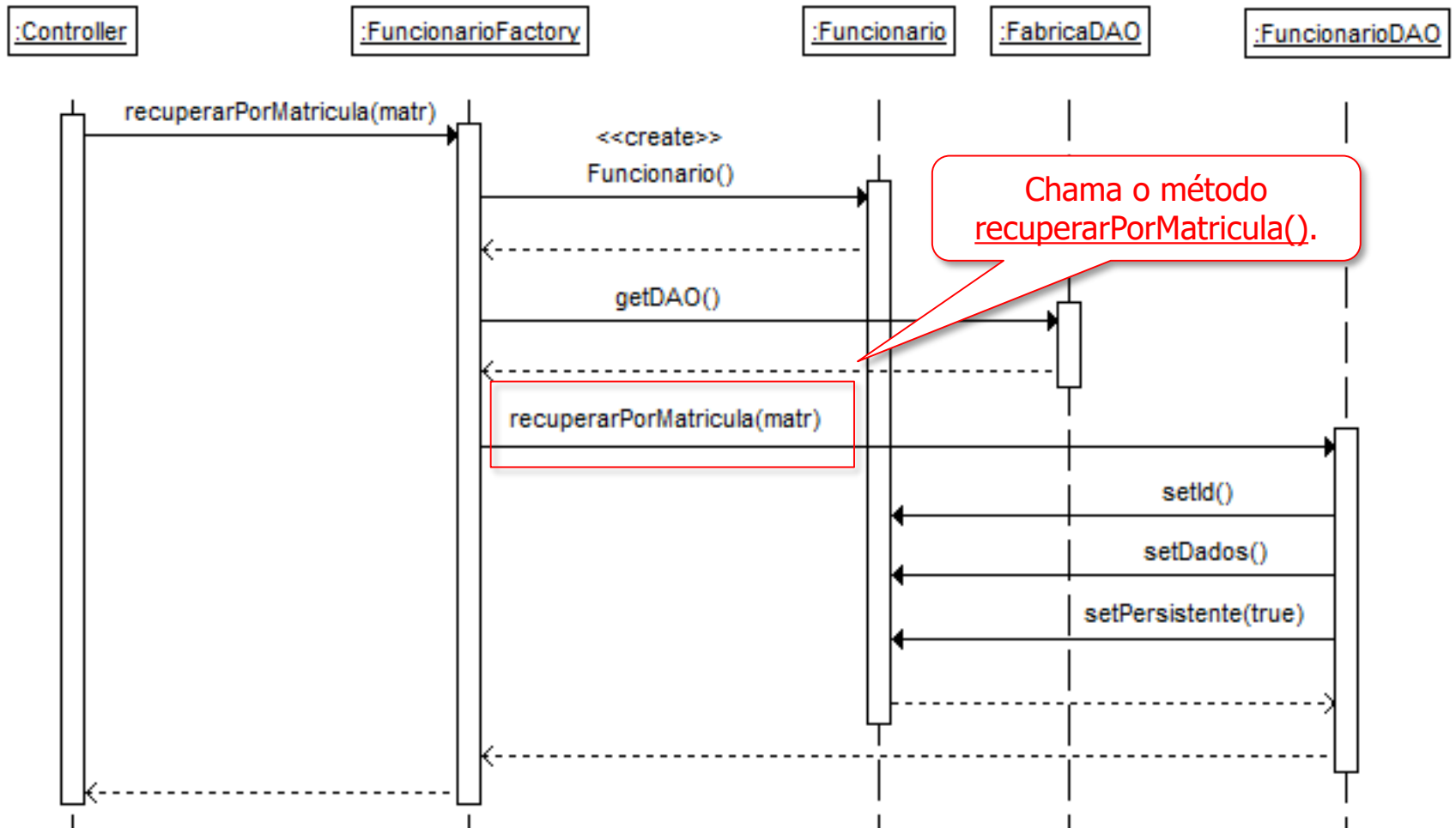
- Sequência para **recuperação** de um Funcionário pela **matrícula**:



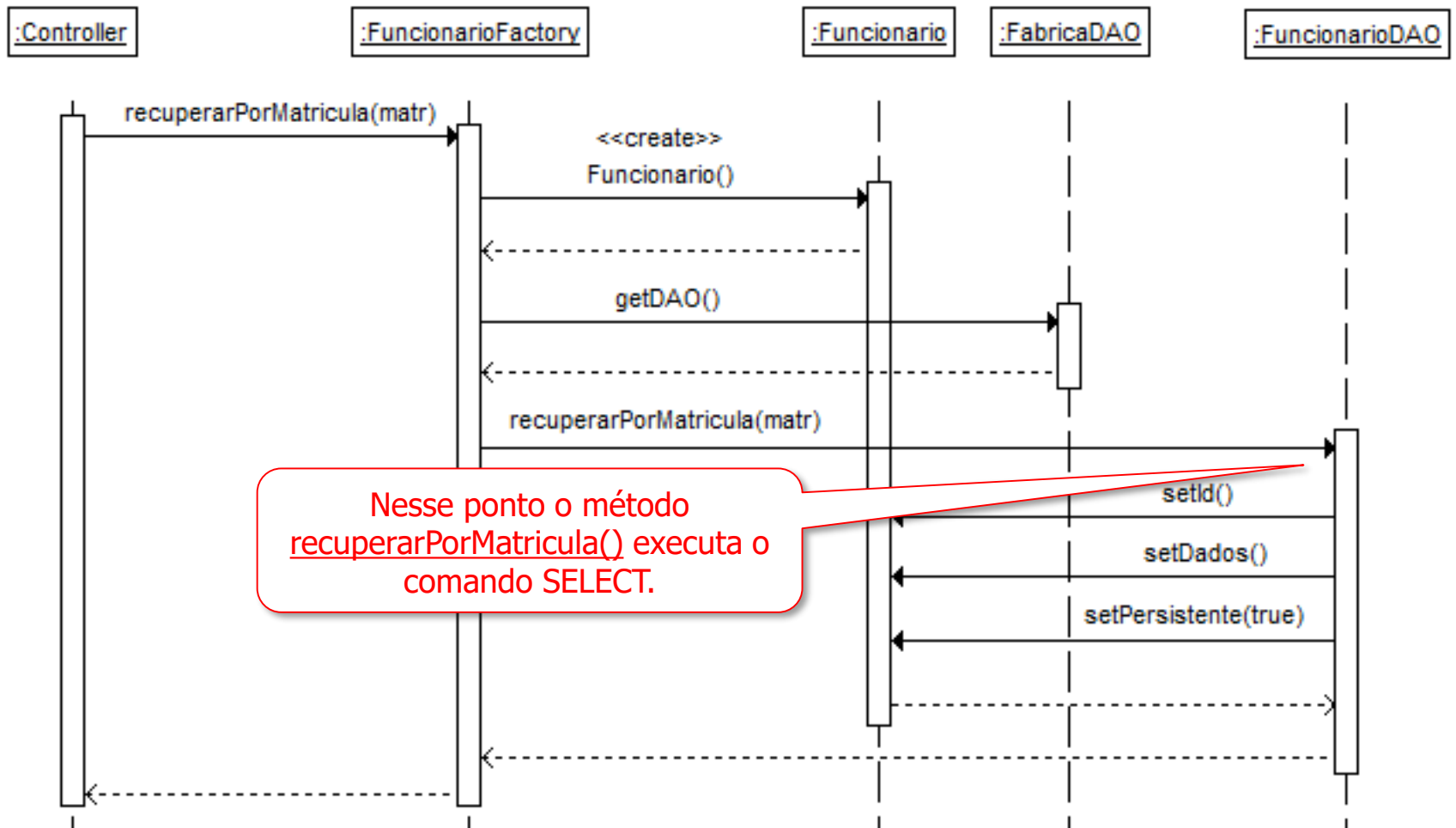
- Sequência para **recuperação** de um Funcionário pela **matrícula**:



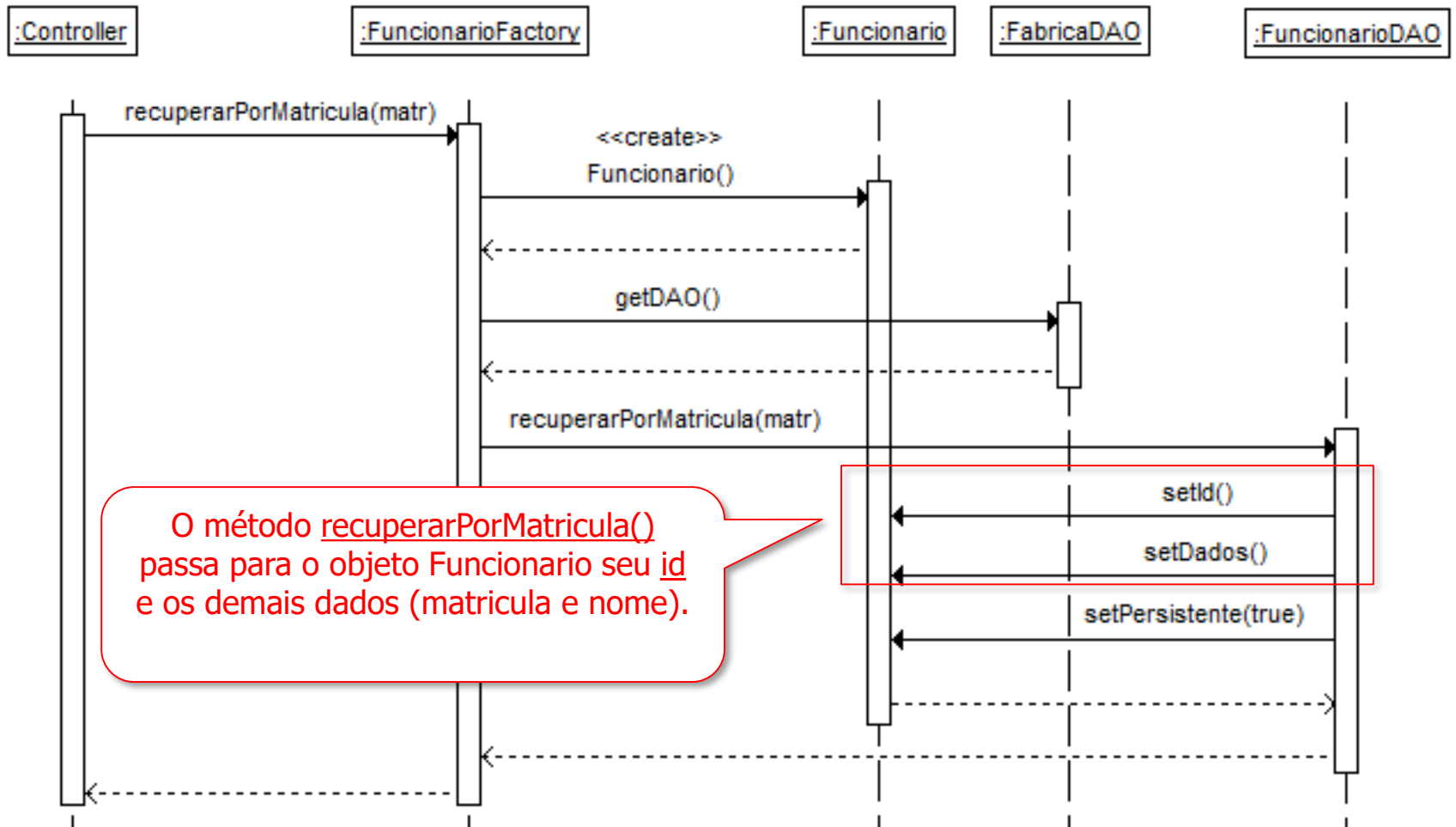
- Sequência para **recuperação** de um Funcionário pela **matrícula**:



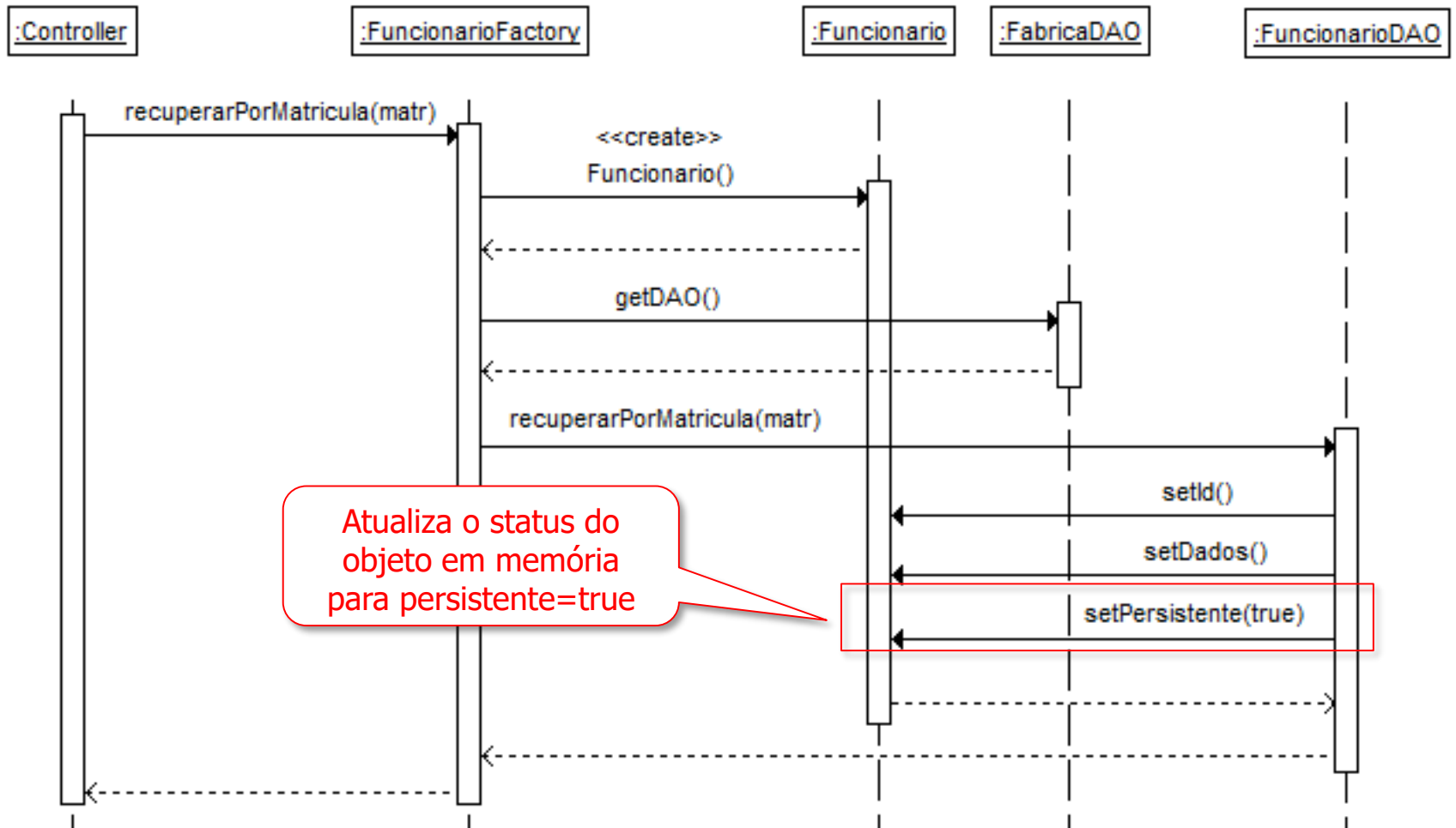
- Sequência para **recuperação** de um Funcionário pela **matrícula**:



- Sequência para **recuperação** de um Funcionário pela **matrícula**:



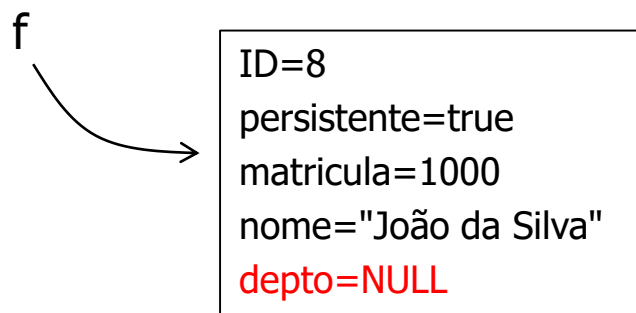
- Sequência para **recuperação** de um Funcionário pela **matrícula**:



- Note que, quando recuperamos um funcionário, **não recuperamos o departamento associado a ele.**

```
Funcionario f;
```

```
f = FuncionarioFactory.recuperarPorMatricula(1000);
```



Essa técnica é chamada de **lazy load**, ou seja, só é carregado o objeto solicitado.

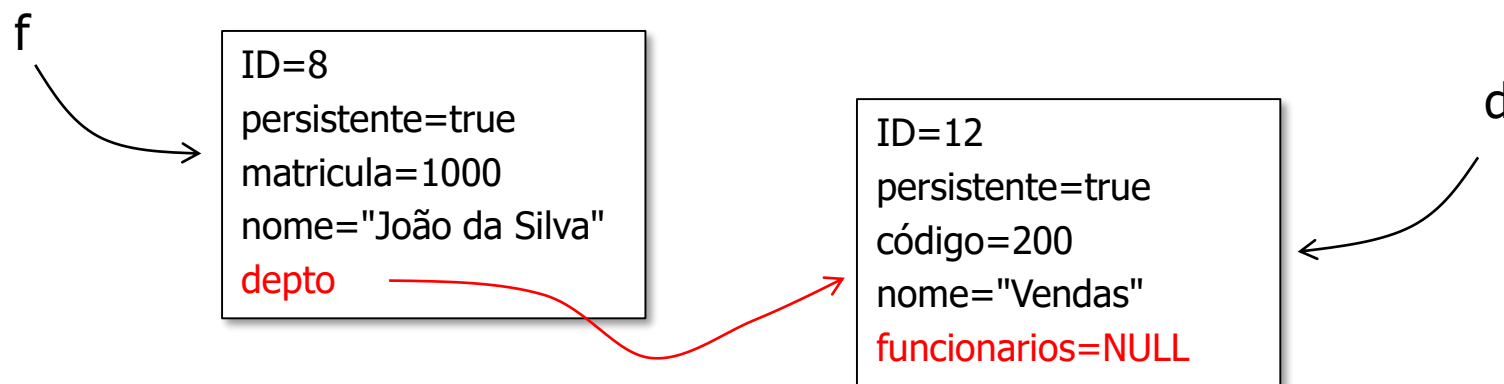
- Em que momento o departamento associado ao funcionário será carregado do BD para a memória?

Resposta: Quando ele for solicitado via **get**!

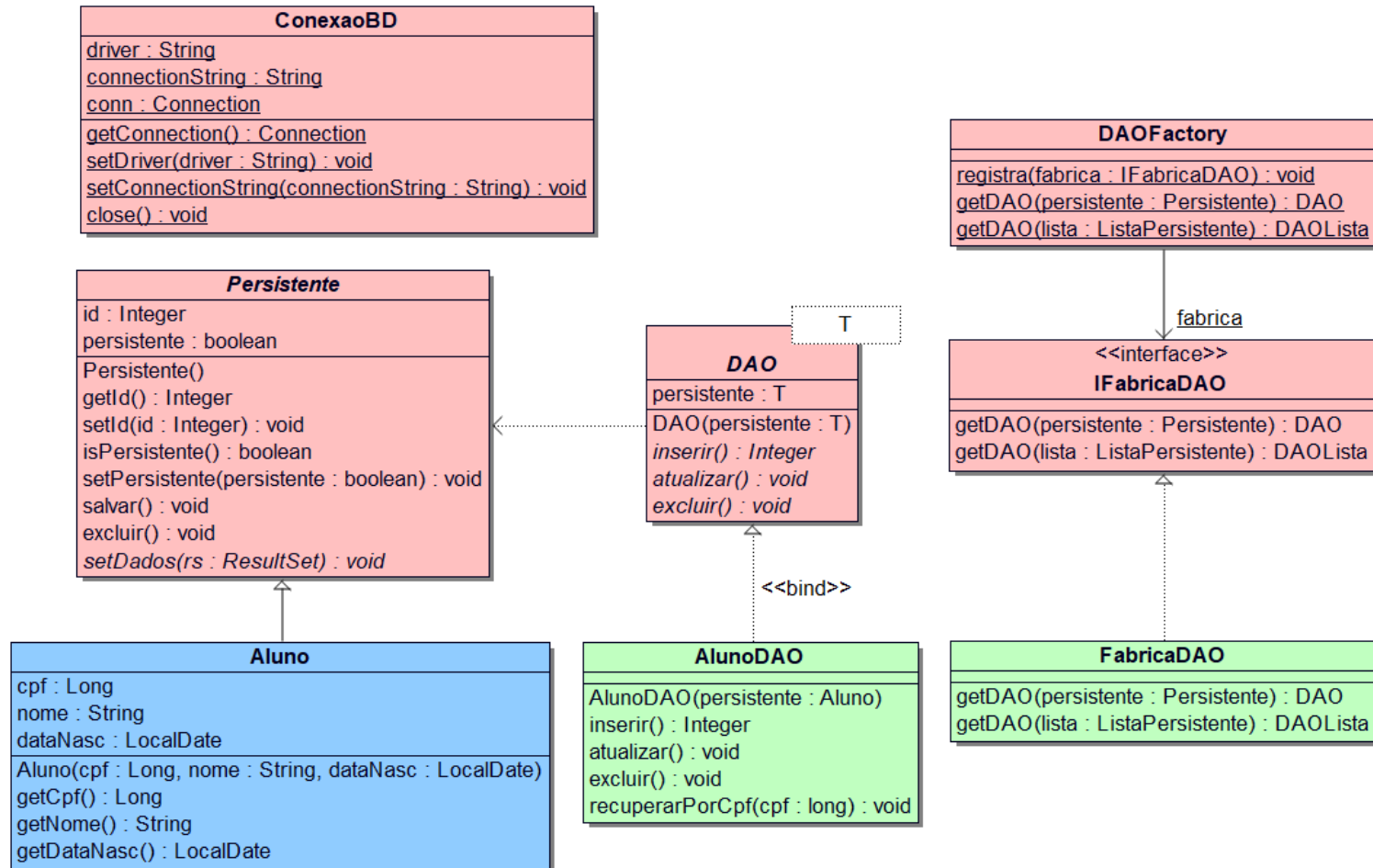
```
Funcionario f;
```

```
f = FuncionarioFactory.recuperarPorMatricula(1000);
```

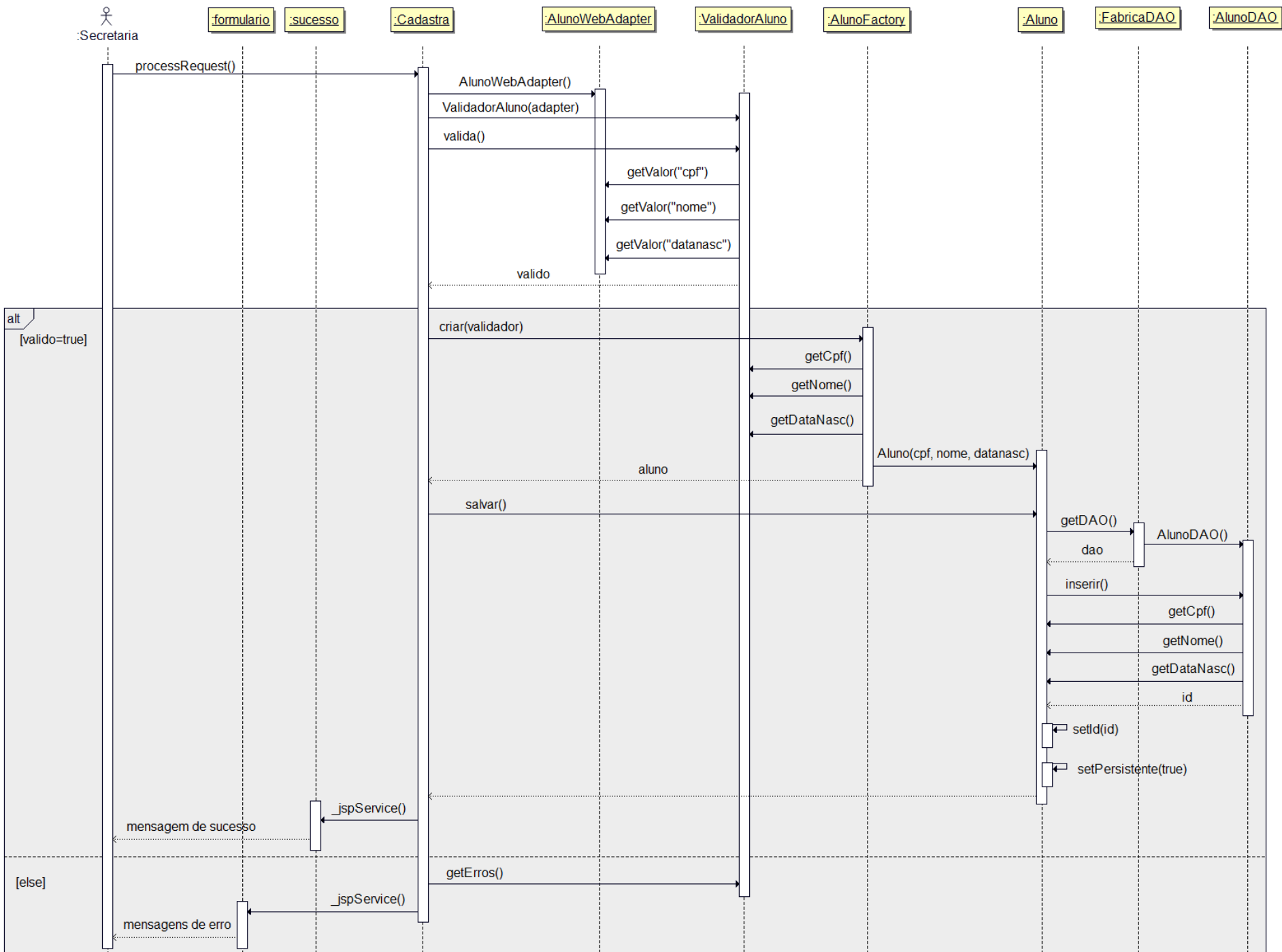
```
Departamento d = f.getDepto();
```

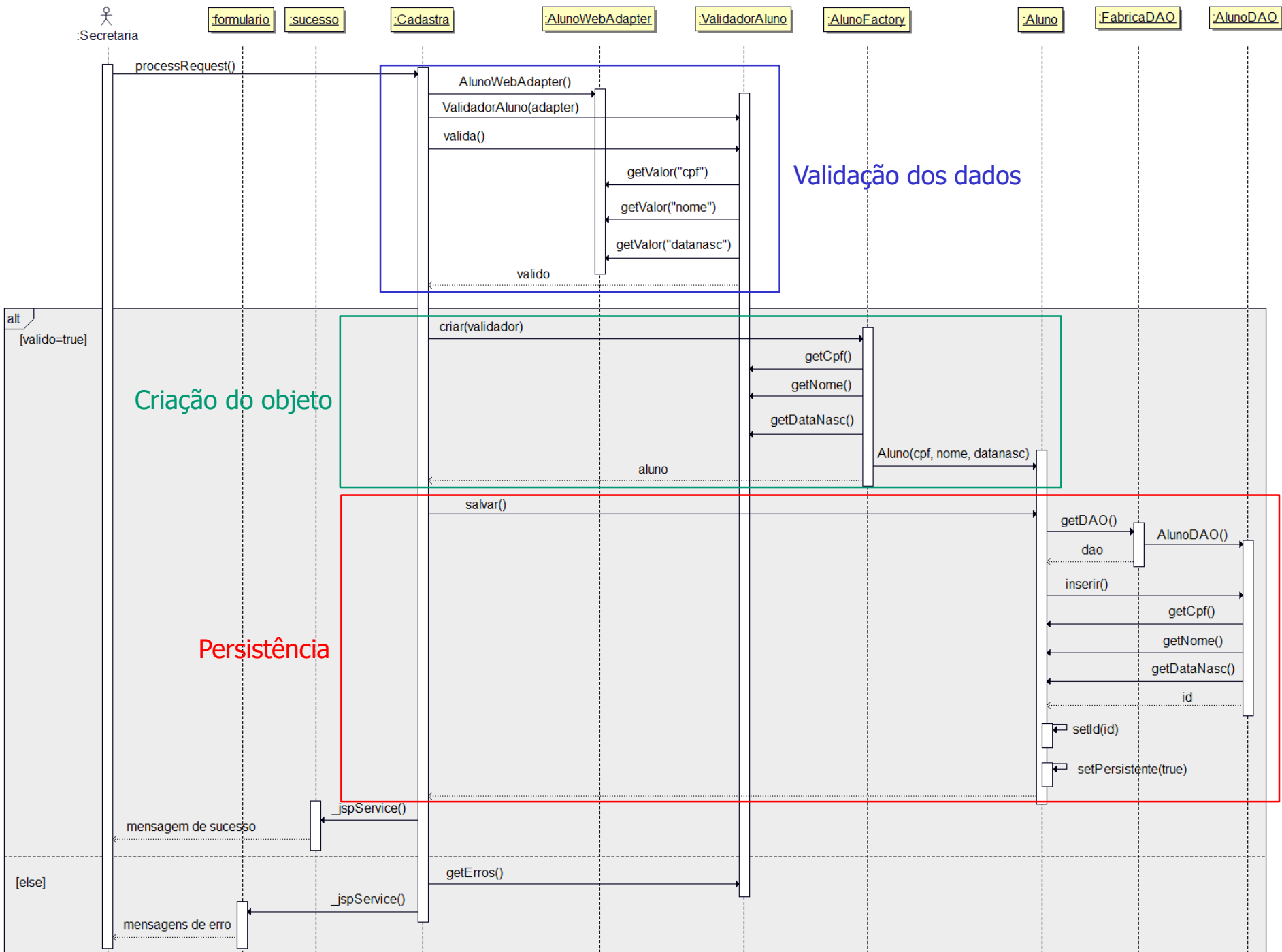


- Para aplicar essa solução de persistência ao cadastramento do aluno, basta implementar as classes **AlunoDAO** e **FabricaDAO**.

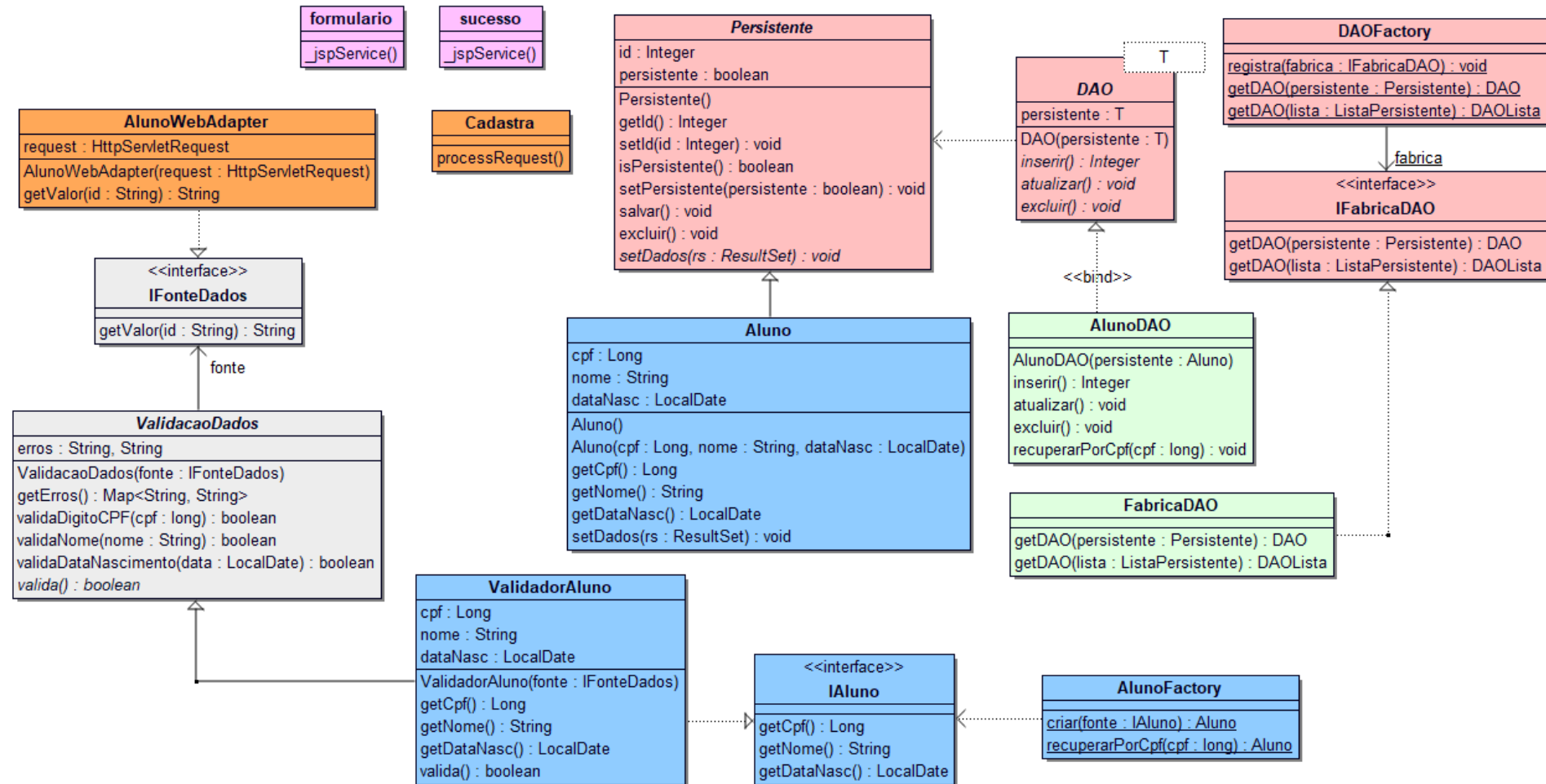


- A seguir é apresentado o Diagrama de Sequência completo e detalhado para o cadastramento do Aluno.

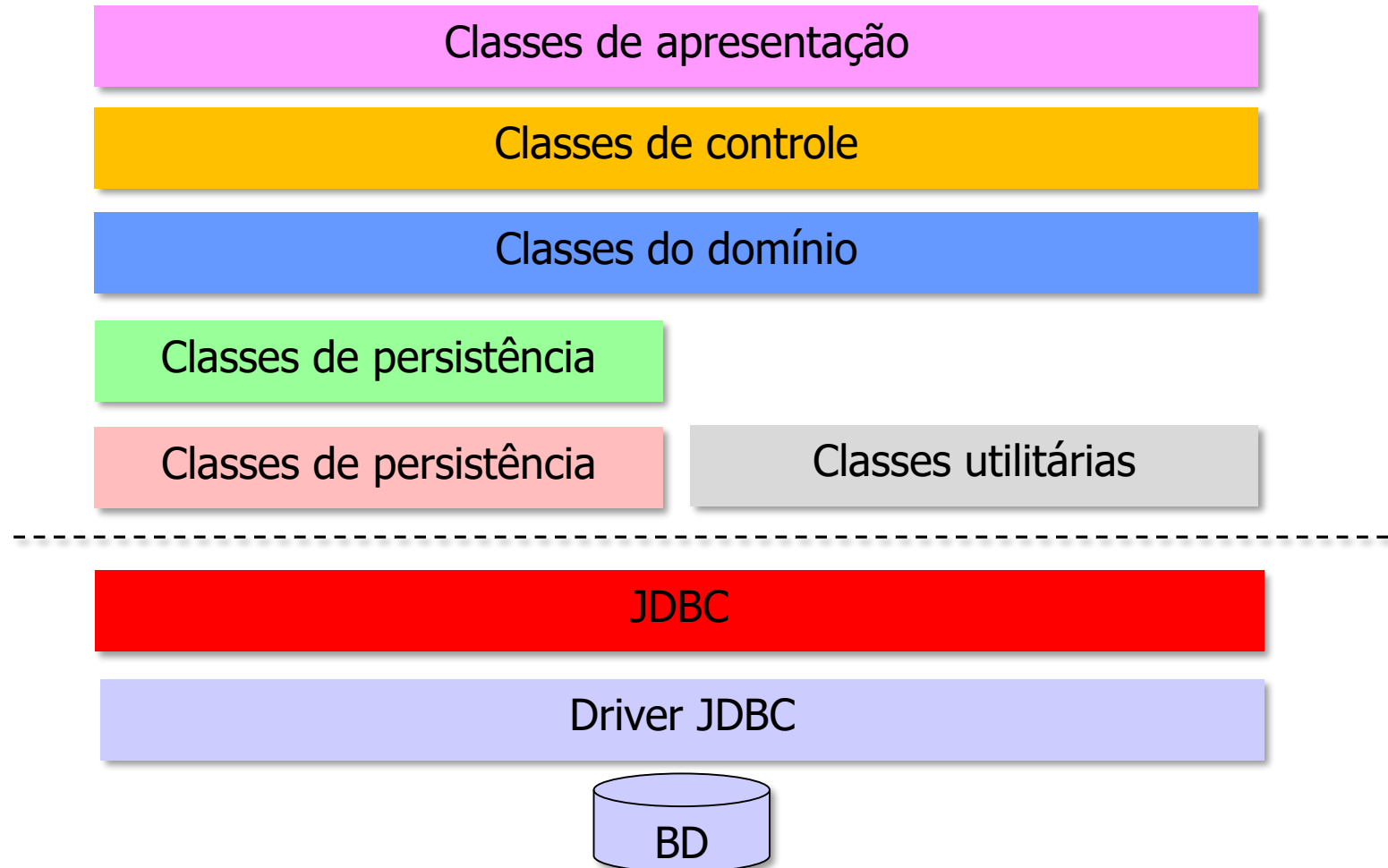




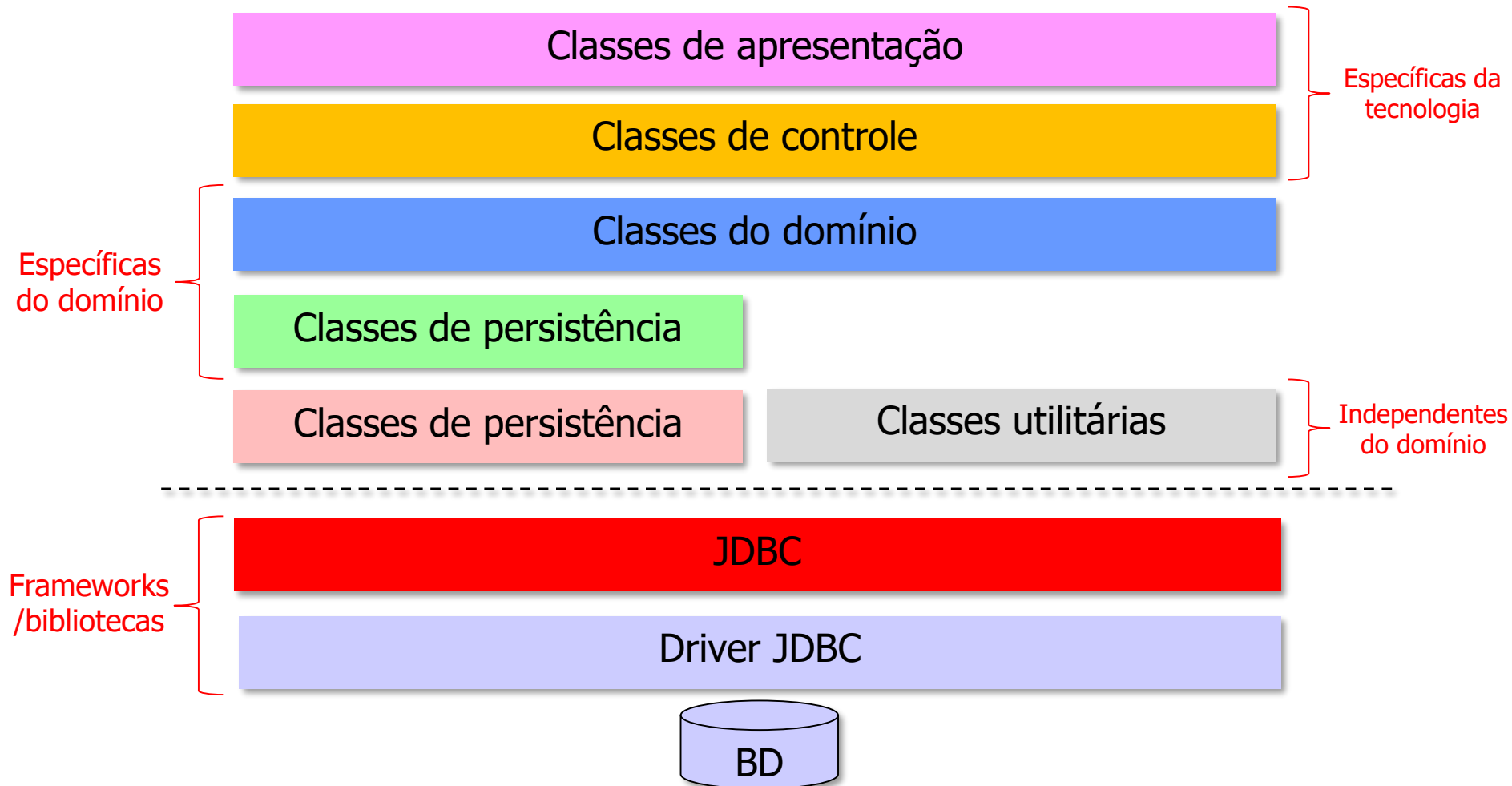
➤ Modelo completo:



- Camadas envolvidas nessa solução (as cores das camadas tem correspondência com as cores do modelo anterior):



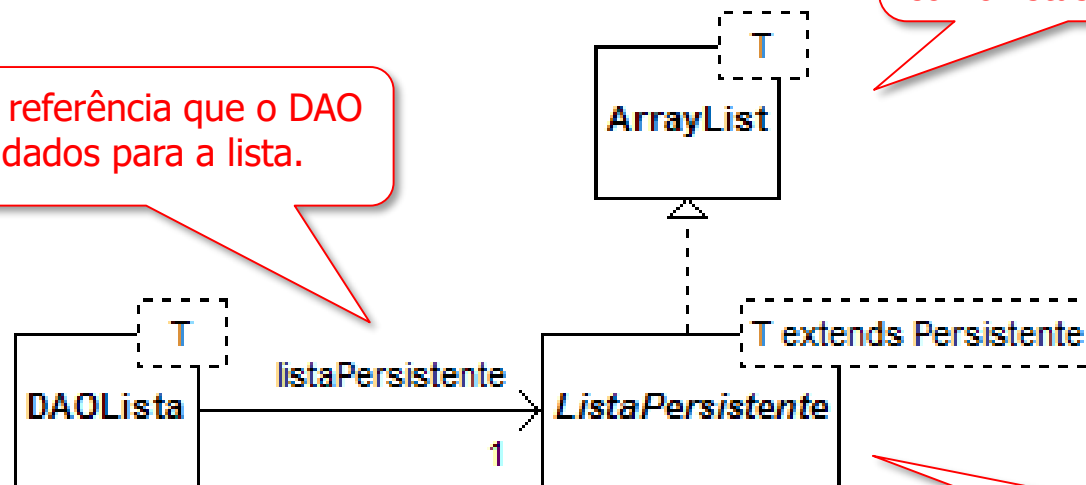
- Camadas envolvidas nessa solução (as cores das camadas tem correspondência com as cores do modelo anterior):



- O padrão definido anteriormente funciona somente para **um único objeto**.
- Entretanto, é possível ampliar esse padrão para tratar **coleções de objetos**.

É por essa referência que o DAO vai passar dados para a lista.

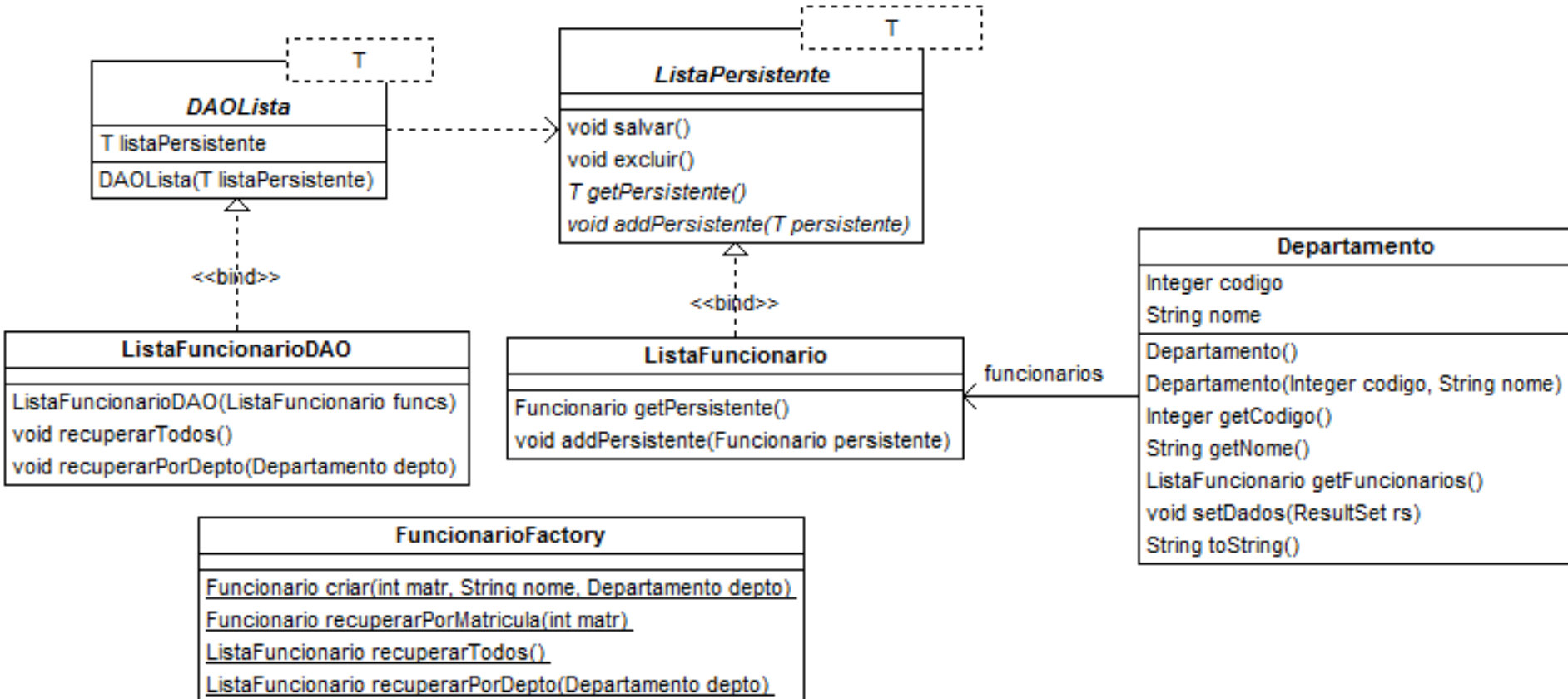
Atributos e métodos comuns a todas as classes DAOLista.



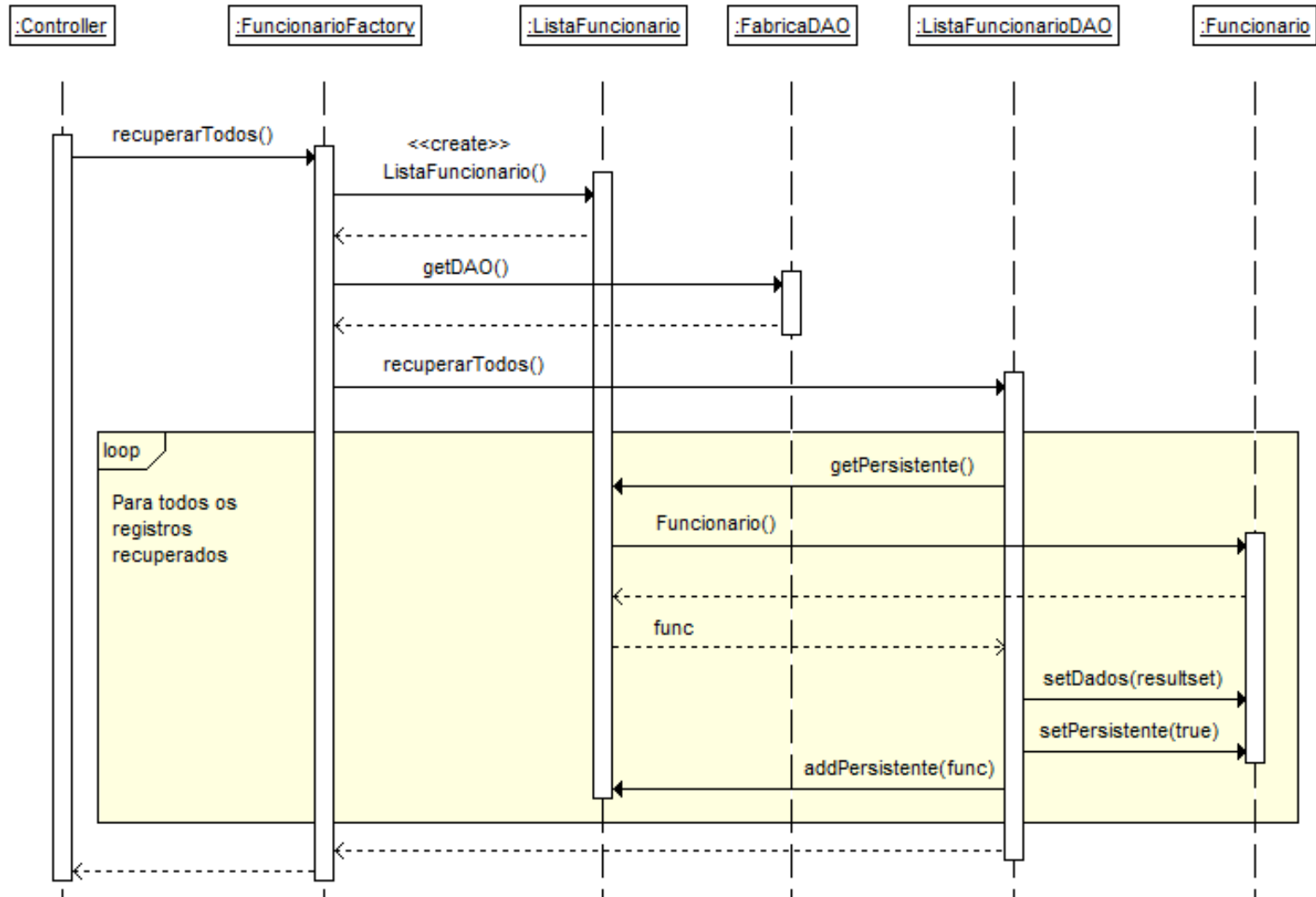
Nesse padrão, coleções são representadas como listas.

As listas persistentes são formadas por objetos persistentes.

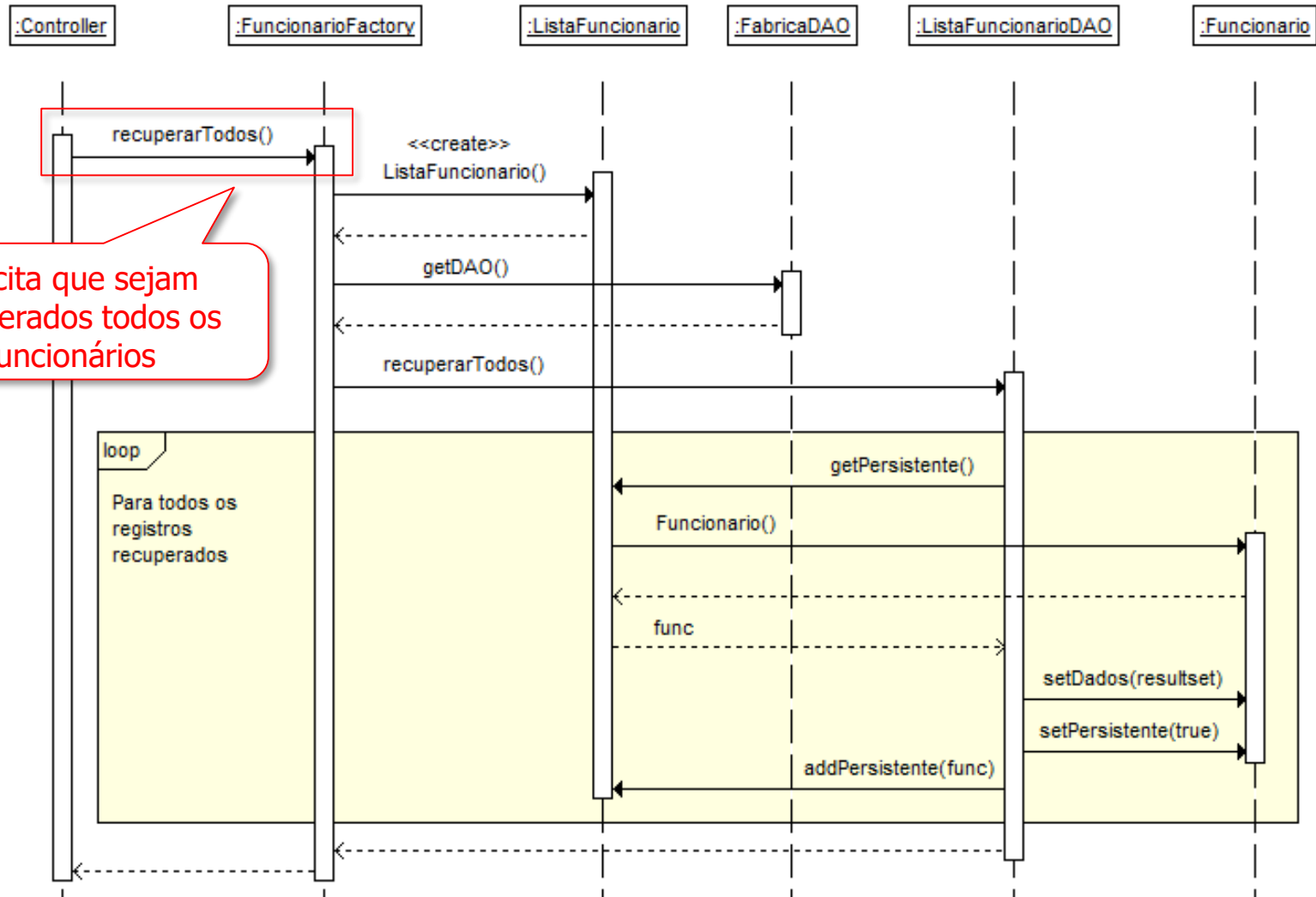
- Aplicando esse padrão para **lista de Funcionários**:



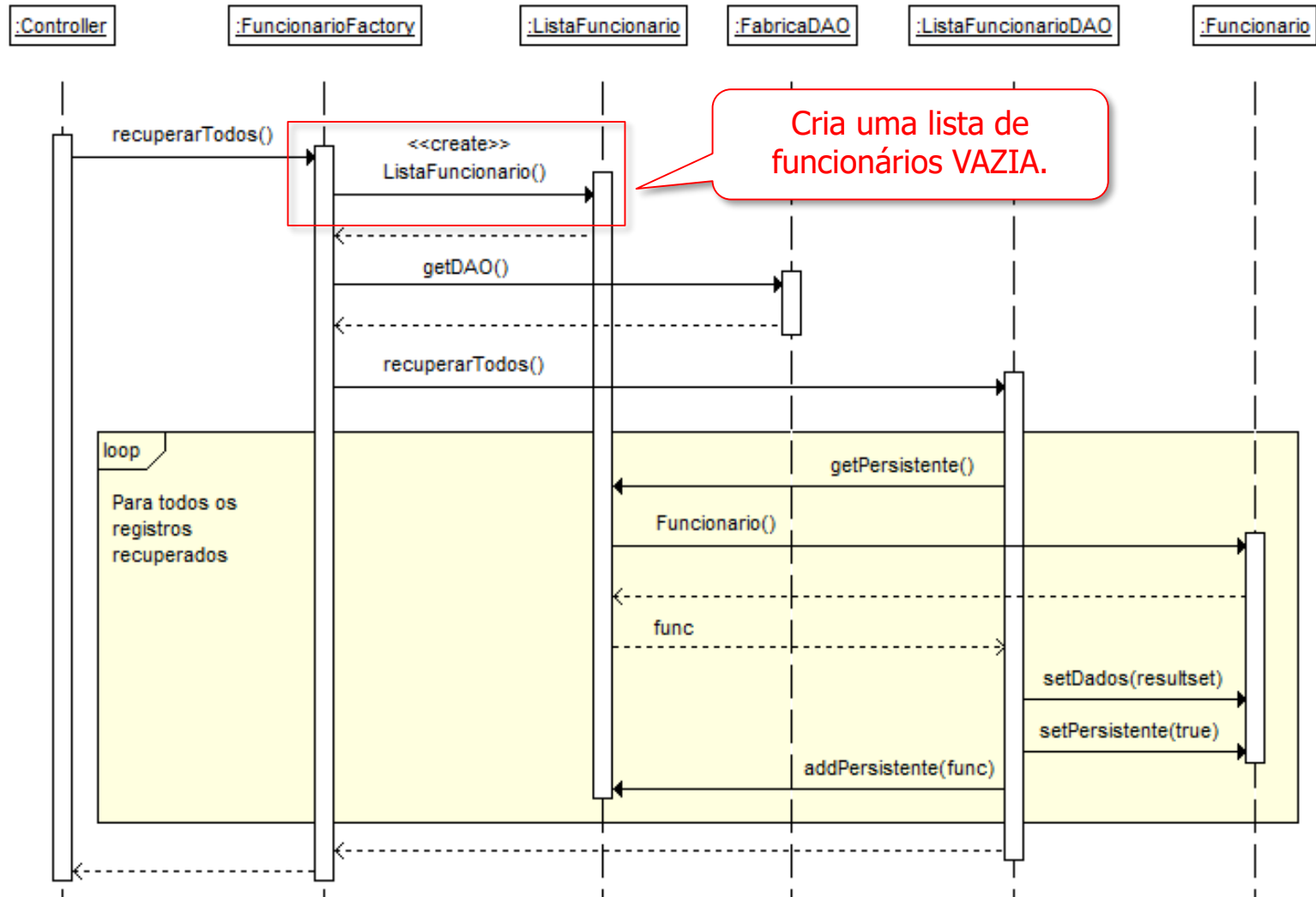
- Sequência para recuperação de **todos** os Funcionários:



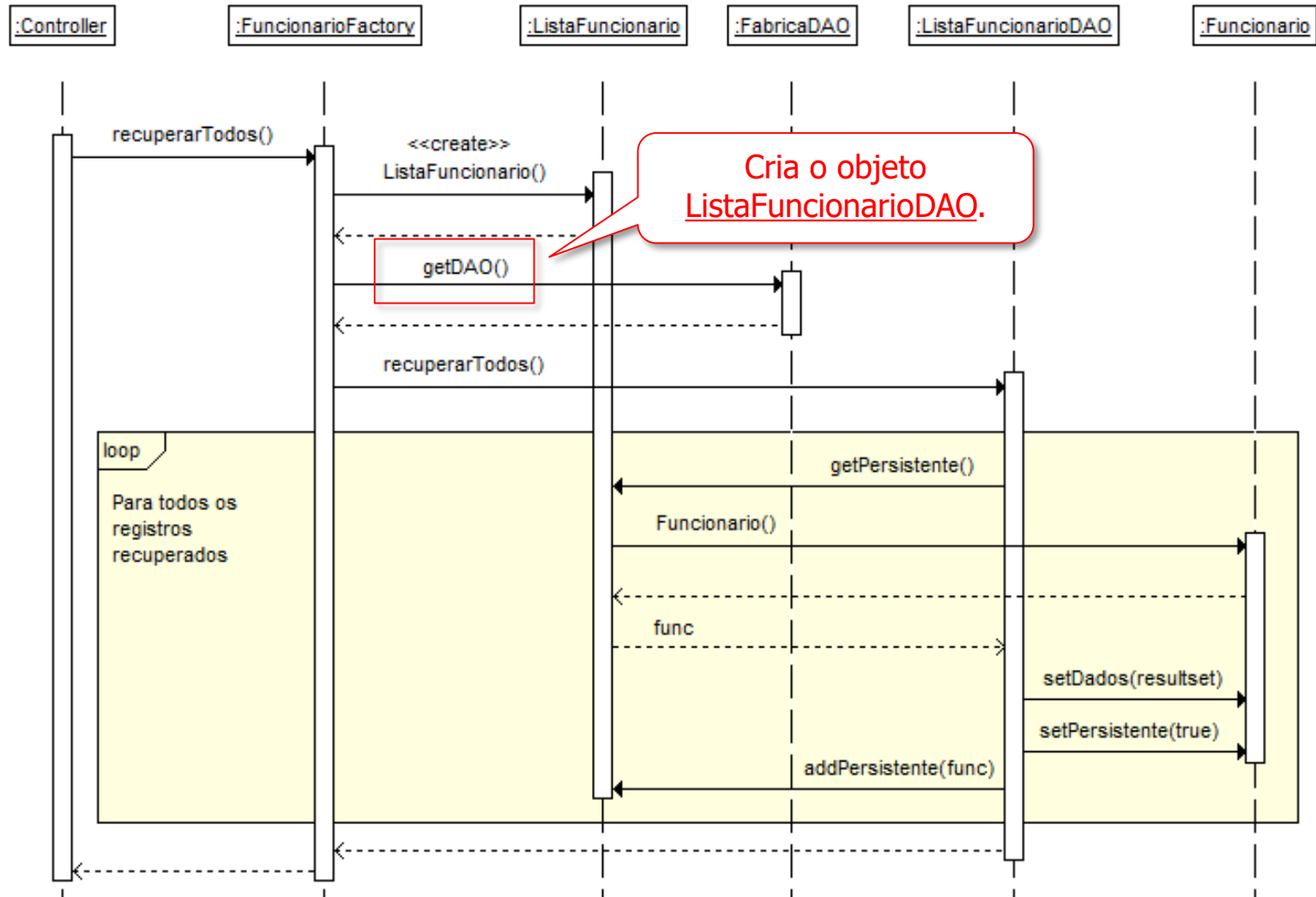
- Sequência para recuperação de **todos** os Funcionários:



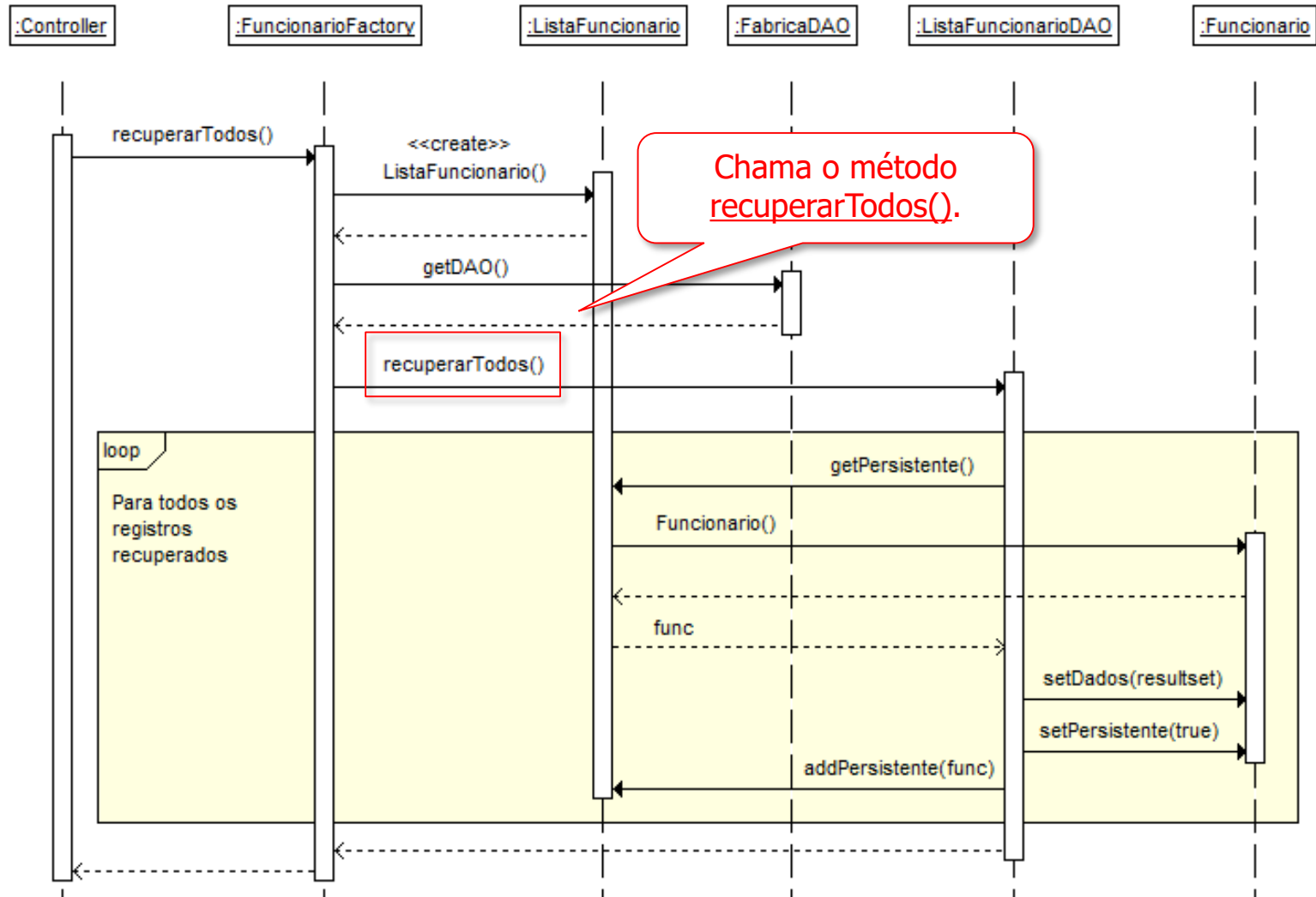
- Sequência para recuperação de **todos** os Funcionários:



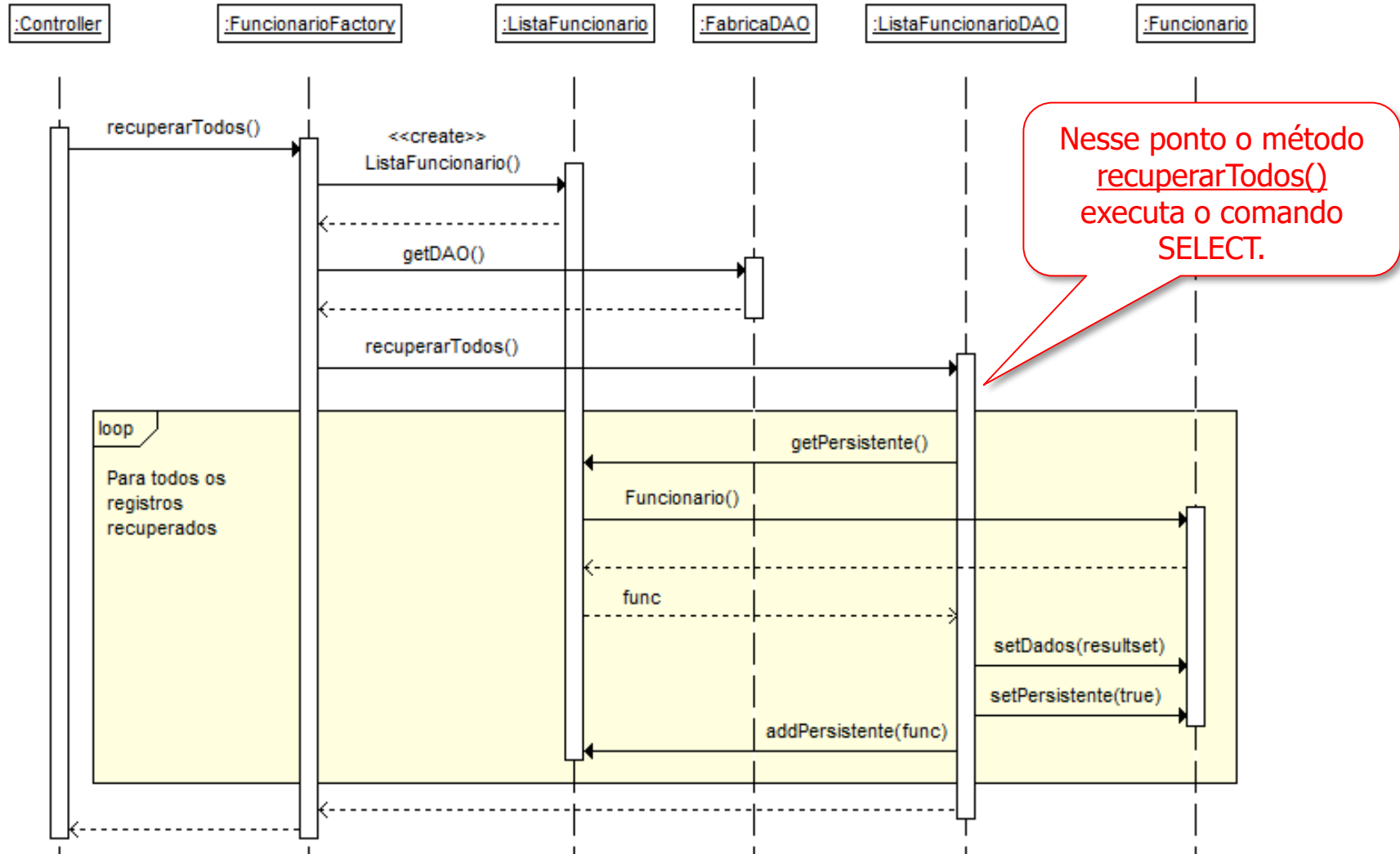
- Sequência para recuperação de **todos** os Funcionários:



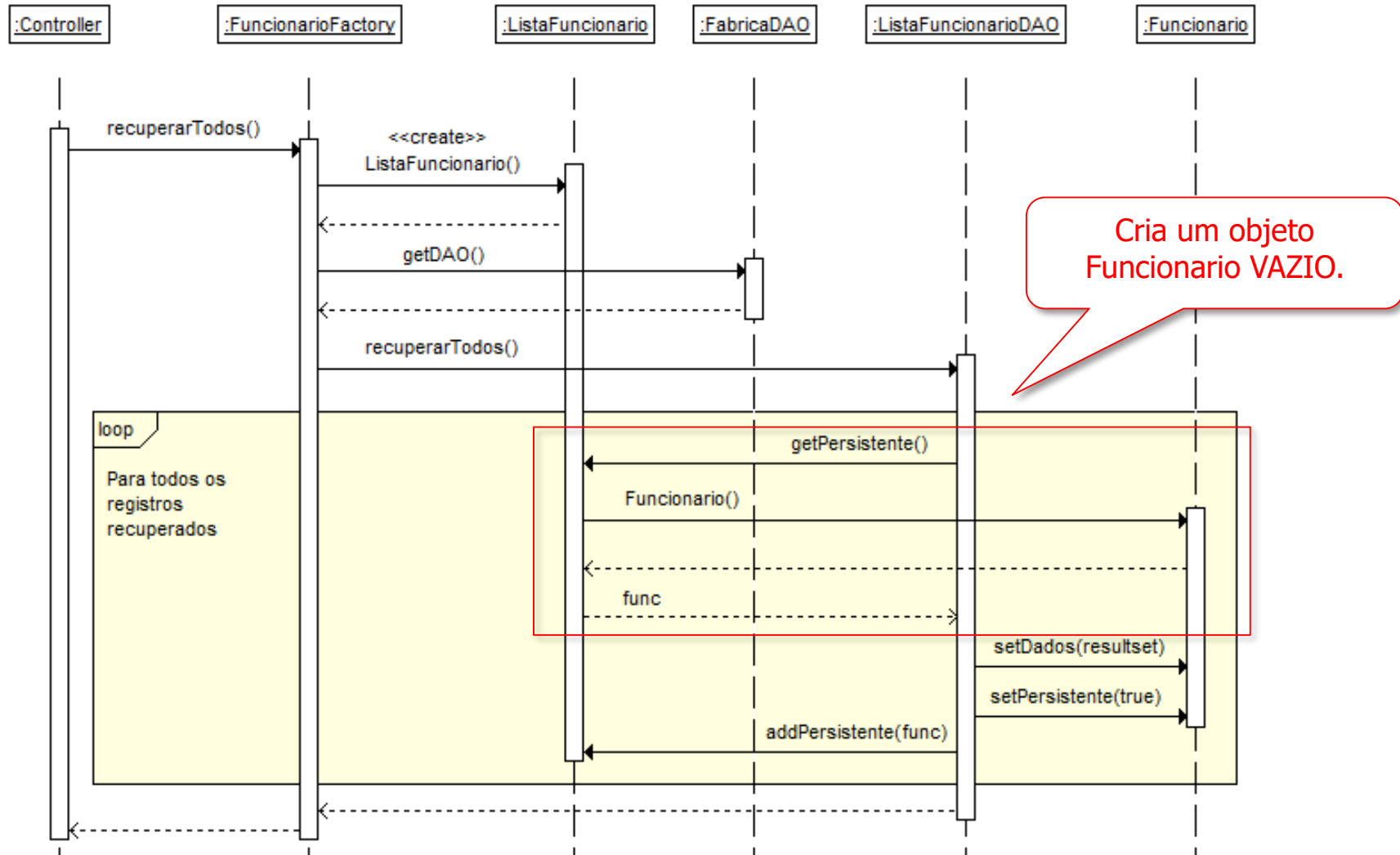
- Sequência para recuperação de **todos** os Funcionários:



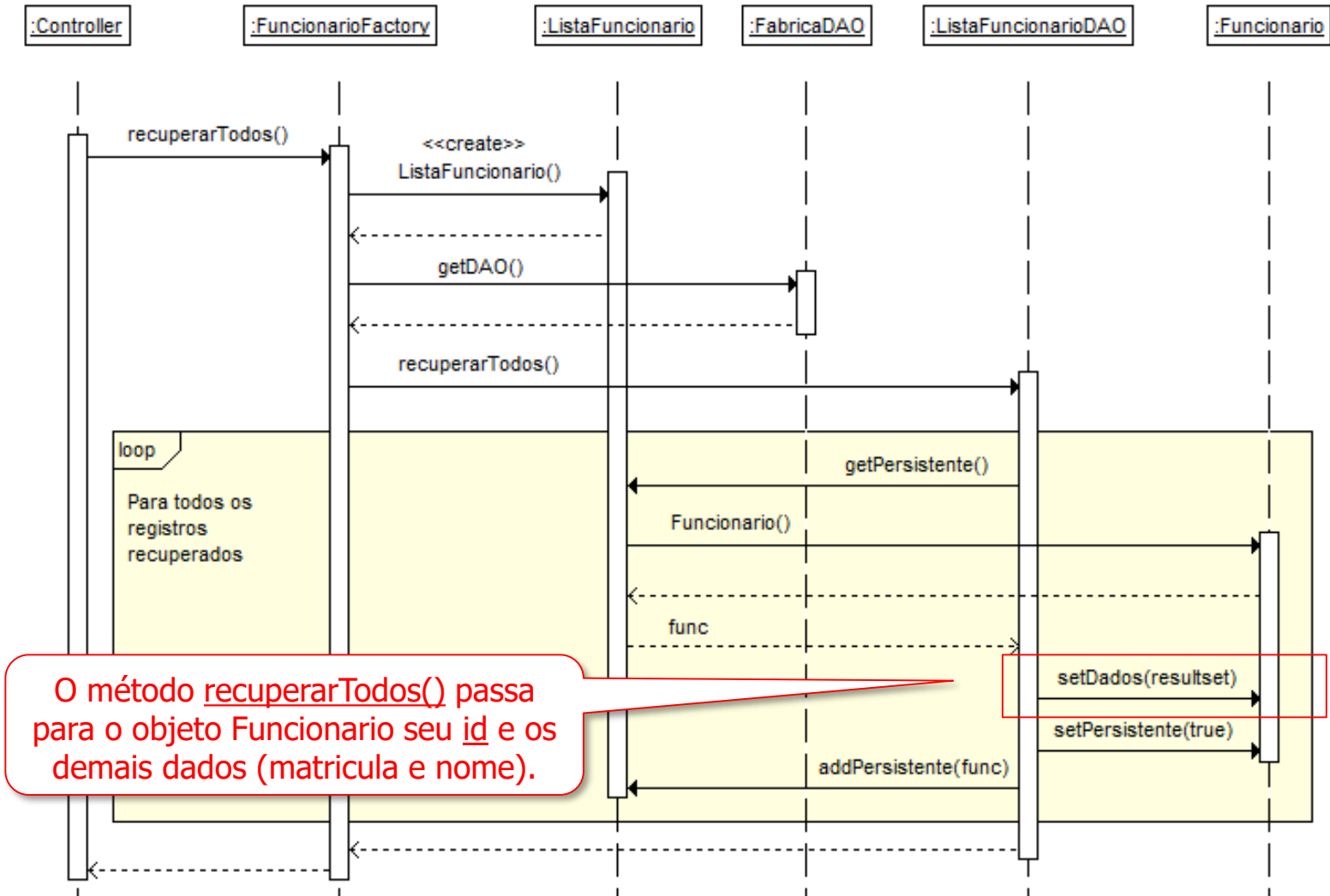
- Sequência para recuperação de **todos** os Funcionários:



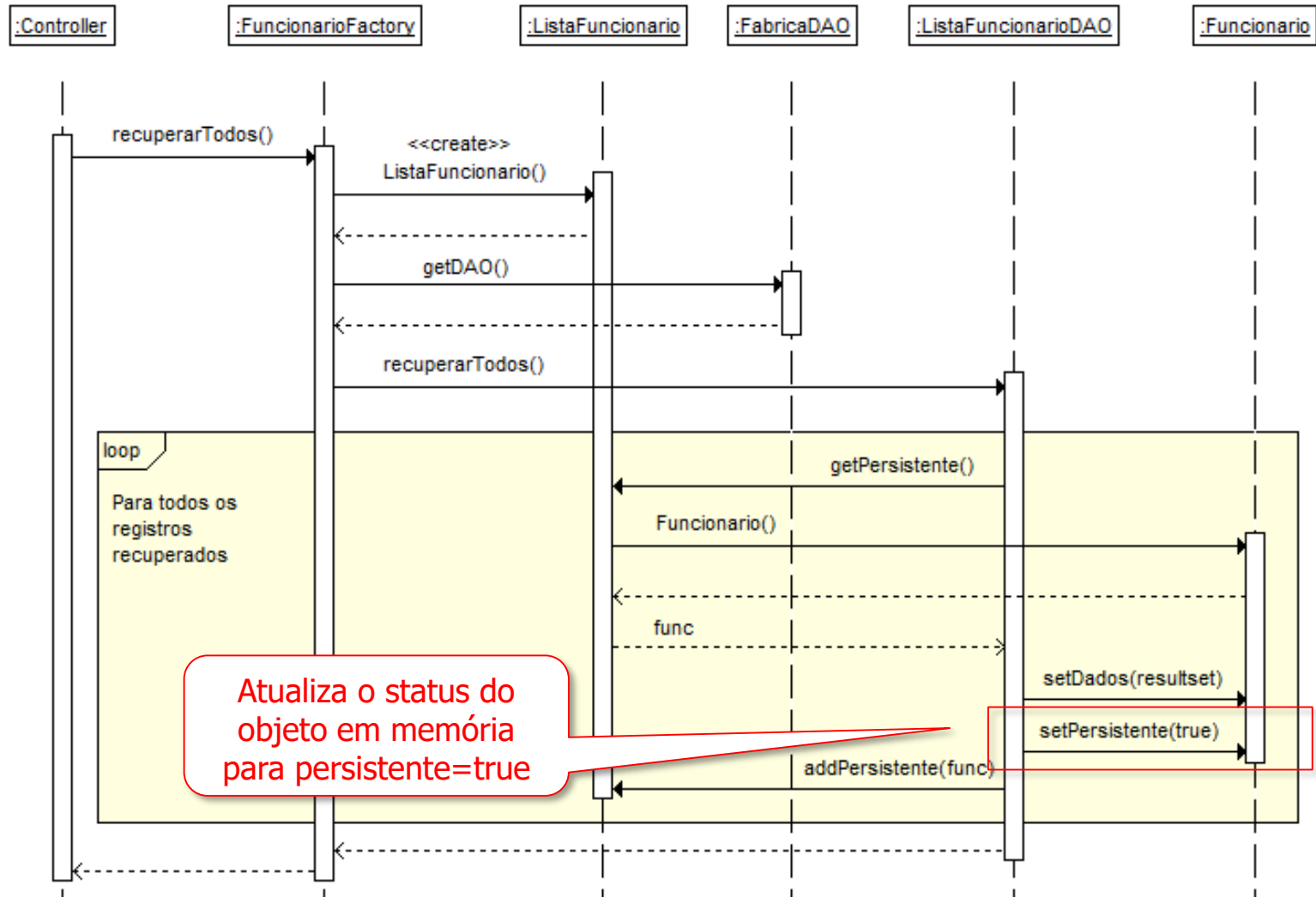
- Sequência para recuperação de **todos** os Funcionários:



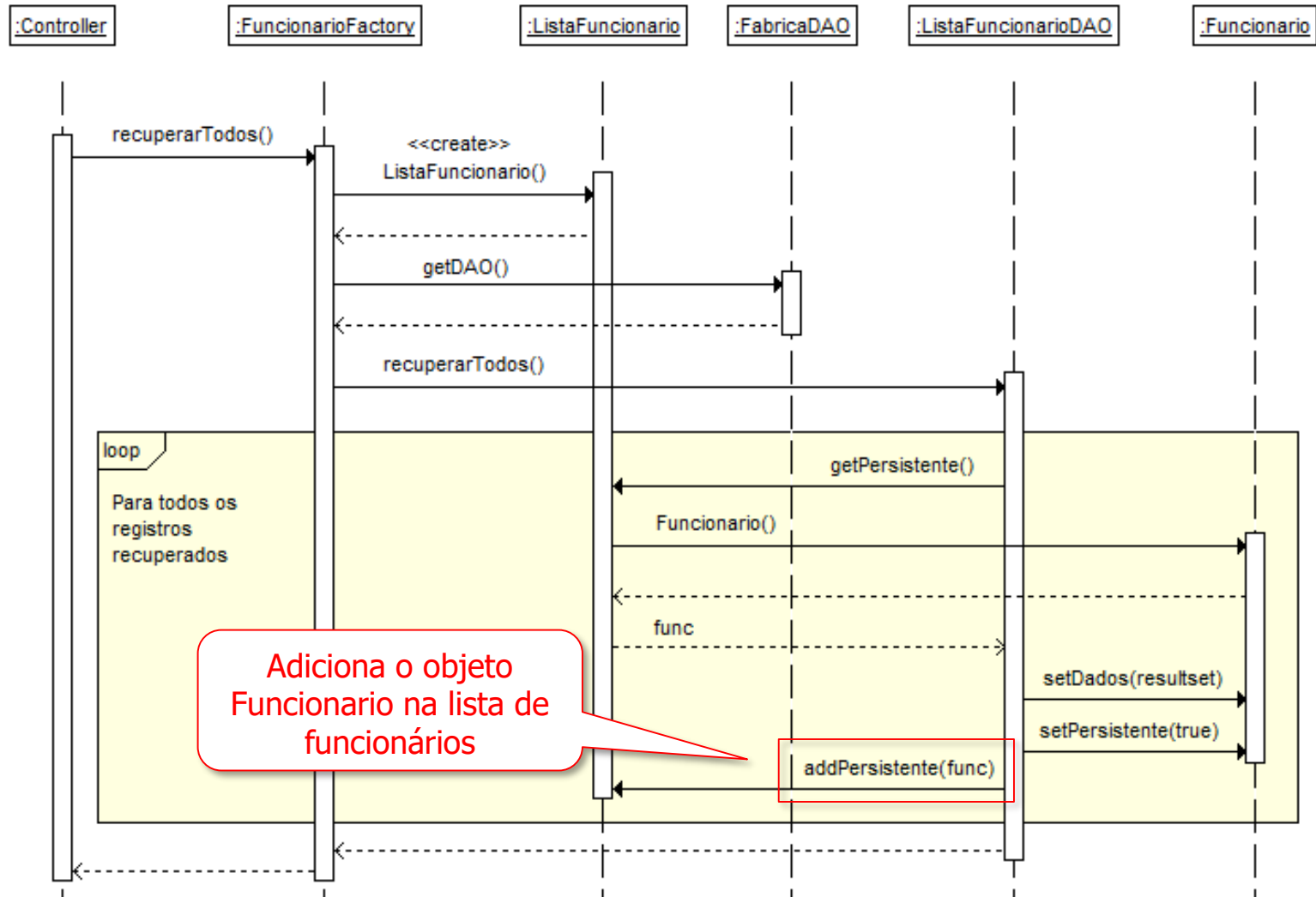
- Sequência para recuperação de **todos** os Funcionários:



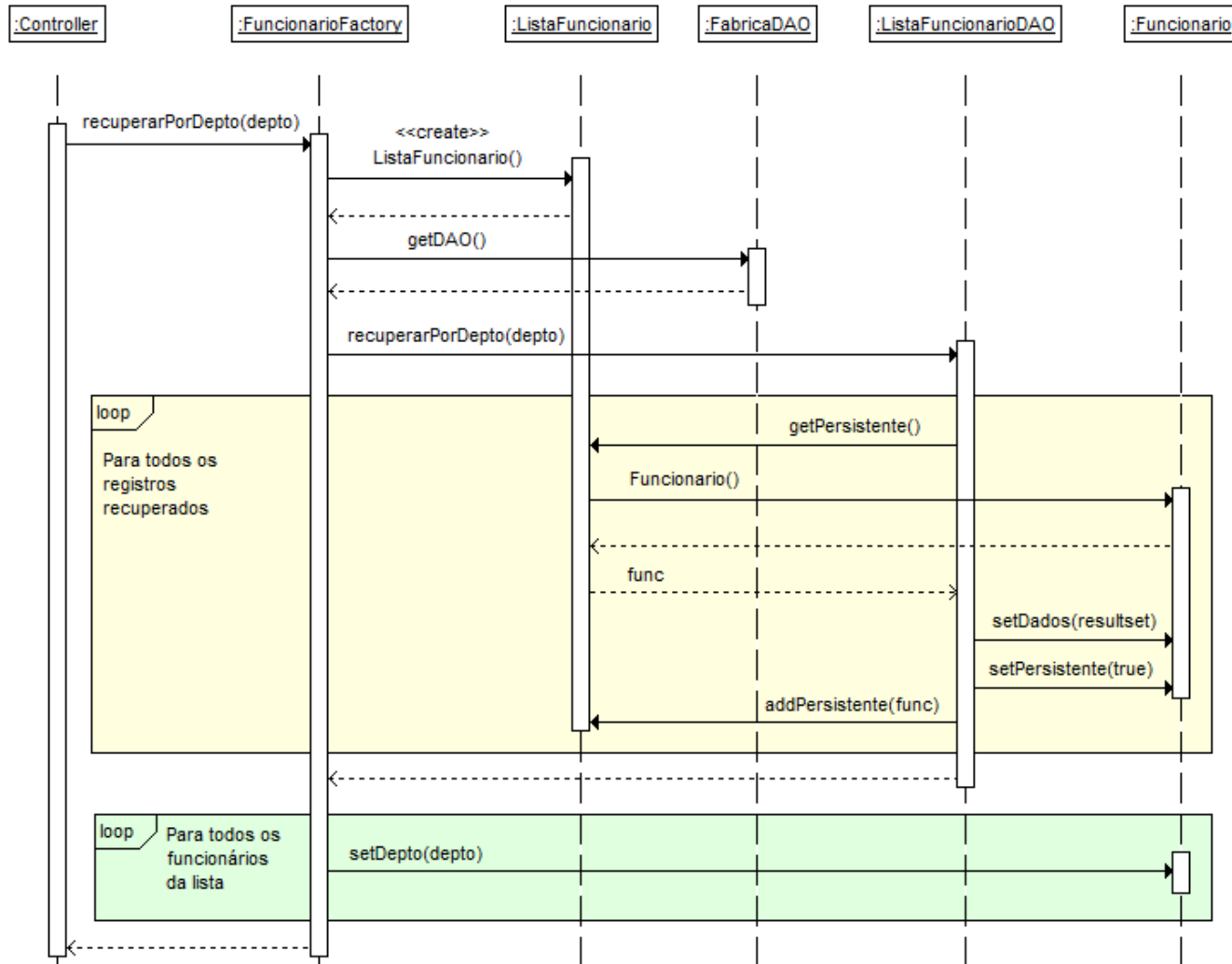
- Sequência para recuperação de **todos** os Funcionários:



- Sequência para recuperação de **todos** os Funcionários:



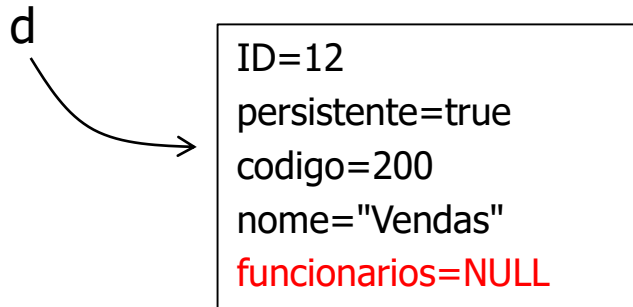
➤ Sequência para recuperação dos **Funcionários de um Departamento**:



- Note que, quando recuperamos um departamento, não recuperamos a lista de funcionários lotados nele.

```
Departamento d;
```

```
d = DepartamentoFactory.recuperarPorCodigo(200);
```



Aqui também é aplicada a técnica de **lazy load**.

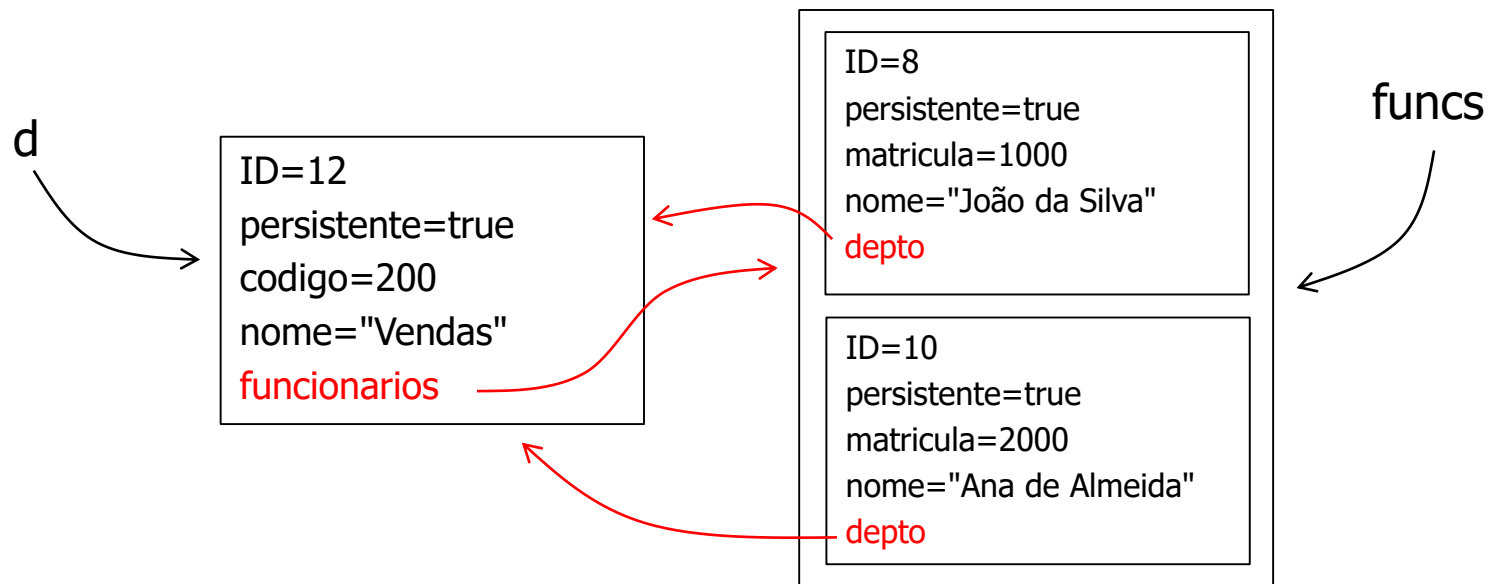
- Em que momento a lista de funcionários lotados em um departamento será carregada do BD para a memória?

R: Quando ela for solicitado via **get**!

```
Departamento d;
```

```
d = DepartamentoFactory.recuperarPorCodigo(200);
```

```
ListaFuncionario funcs = d.getFuncionarios();
```



- De forma resumida, envolve os seguintes passos:
 1. Descrever as interações entre as classes de design
 2. Descrever comportamentos da persistência
 3. Simplificar os diagramas de sequência usando subsistemas 

- Esse passo é **opcional**, mas o conceito é simples:
 - ✓ Caso tenham sido identificados **subsistemas**, simplifique os diagramas de sequência sem entrar em detalhes sobre esses subsistemas, ou seja, defina apenas as **interfaces** que deverão ser implementadas por eles.
 - ✓ Posteriormente, esses subsistemas serão detalhados na atividade **Design de Subistema**.

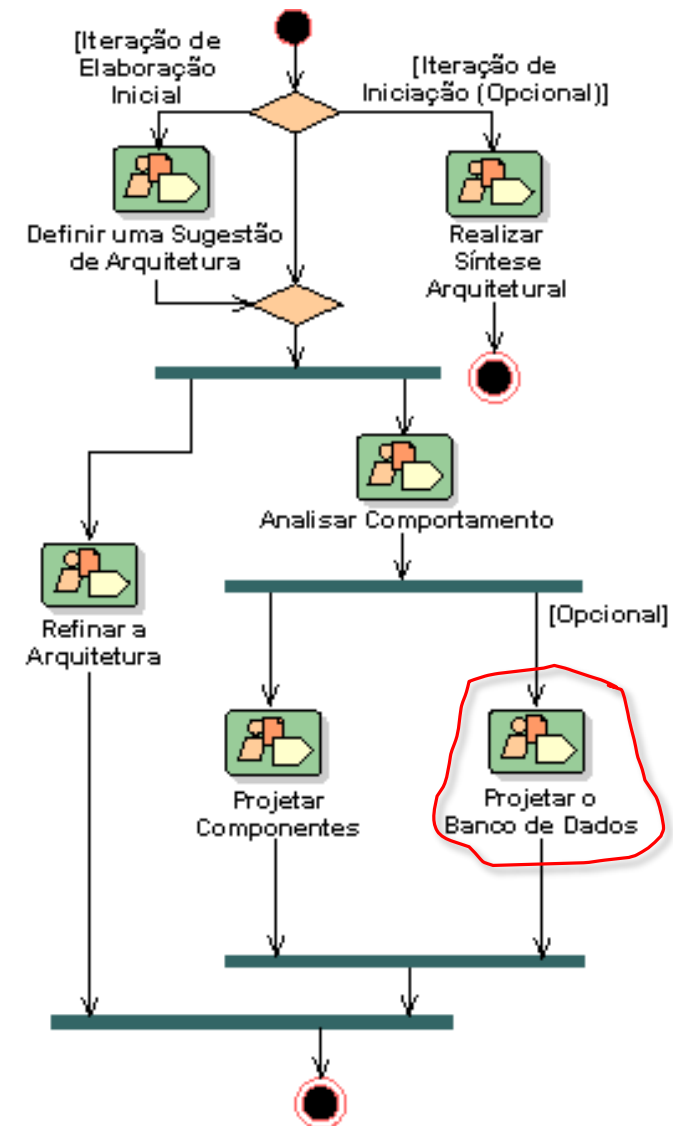
Observação: a discussão sobre o que são subsistemas e pacotes e como identificá-los foi feita no item anterior (Identificar Classes e Subsistemas).

Projeto de Software Orientado a Objetos

14ª parte

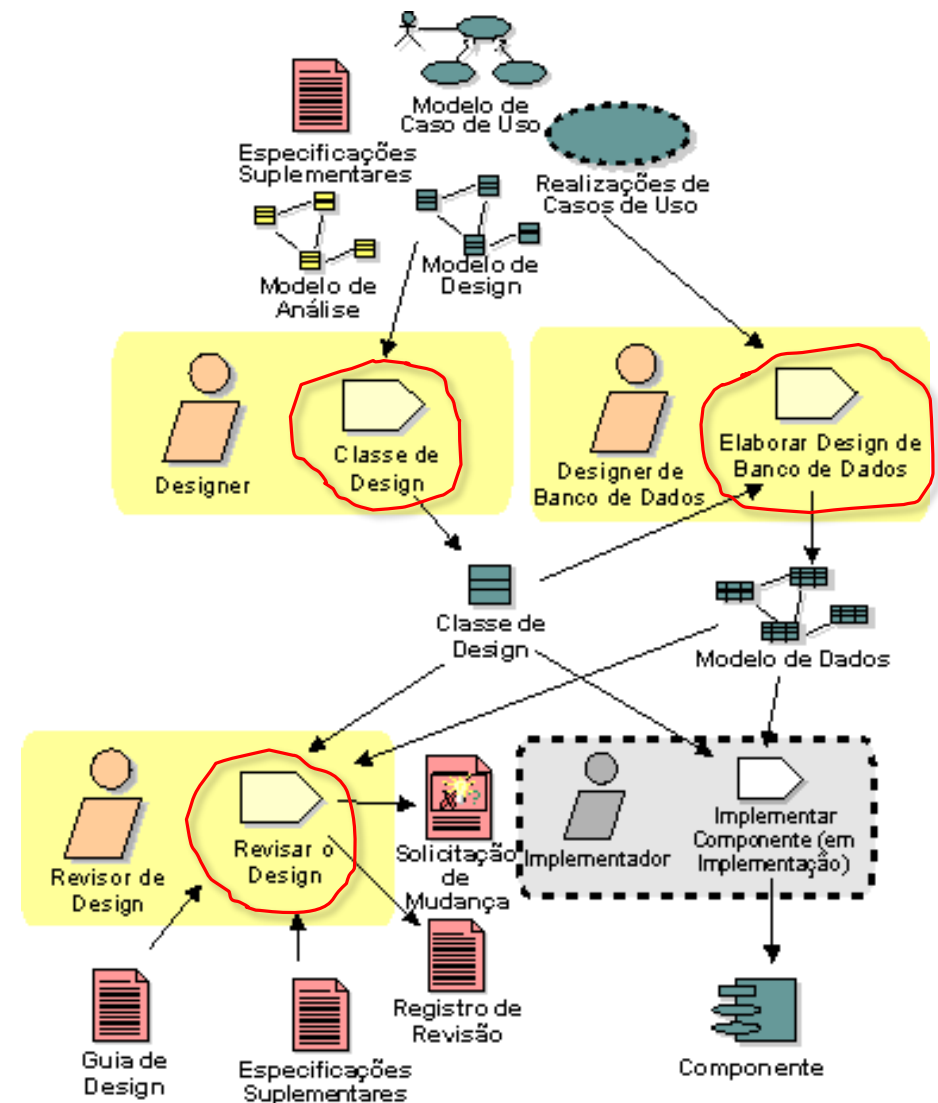
Prof. Jobson Massollar

- A disciplina de **Análise e Design** é composta dos seguintes fluxos de trabalho:
1. Definir uma Sugestão de Arquitetura
 2. Analisar Comportamento
 3. Projetar Componentes
 4. **Projetar BD**
 5. Refinar a Arquitetura



- A finalidade desse fluxo de trabalho é:
 - ✓ Identificar classes persistentes no modelo de design;
 - ✓ Projetar as estruturas de banco de dados apropriadas, a fim de armazenar as classes persistentes;
 - ✓ Definir mecanismos e estratégias para armazenar e recuperar dados de forma eficiente, consistentes com as necessidades do sistema.

- Esse fluxo de trabalho é composto das atividades:
 - ✓ Design da Classe
 - ✓ Elaborar Design do BD
 - ✓ Revisar o Design

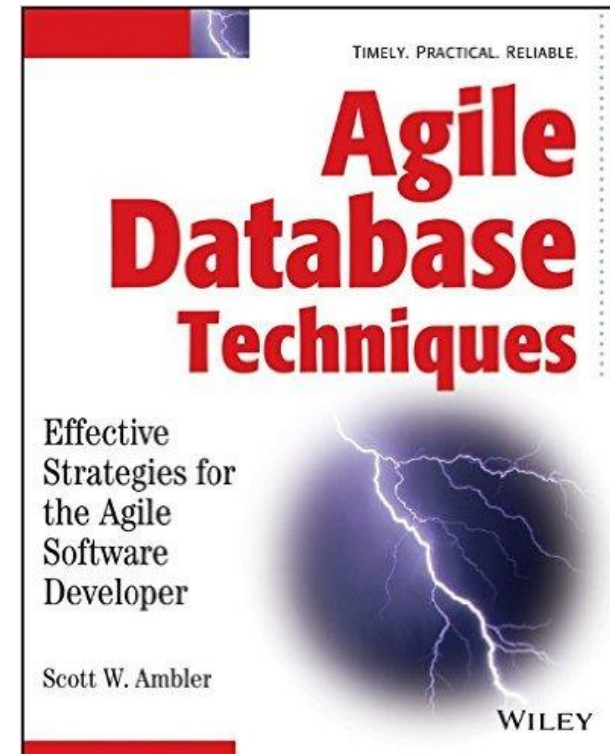


- De forma resumida, envolve os seguintes passos:
 1. Mapear Classes de Design Persistentes para o Modelo de Dados
 2. Otimizar o Desempenho do Modelo de Dados (desnormalização)
 3. Otimizar o Acesso aos Dados (índices)
 4. Definir Tabelas de Referência
 5. Definir Regras para a Integridade Referencial e de Dados
 6. Distribuir o Comportamento de Classes no Banco de Dados
 7. Revisar os Resultados

➤ De forma resumida, envolve os seguintes passos:

1. Mapear Classes de Design Persistentes para o Modelo de Dados 
2. Otimizar o Desempenho do Modelo de Dados (desnormalização)
3. Otimizar o Acesso aos Dados (índices)
4. Definir Tabelas de Referência
5. Definir Regras para a Integridade Referencial e de Dados
6. Distribuir o Comportamento de Classes no Banco de Dados
7. Revisar os Resultados

- Os padrões de mapeamento objeto-relacional que serão apresentados a seguir foram retirados do livro *Agile Database Techniques* de Scott Ambler.
- Apesar de atualmente existirem *frameworks* que realizam esse mapeamento de forma (semi) automática, o conhecimento dessas técnicas é essencial pois:
 - ✓ O *framework* pode não estar disponível na plataforma de desenvolvimento utilizada;
 - ✓ O *framework* não implementa alguma característica necessária;
 - ✓ O mapeamento do *framework* não atende à desnormalização necessária por razões de desempenho;
 - ✓ Alguma decisão estratégica para não adotar um framework.



1. Mapeamento Básico

- ✓ Cada classe persistente é mapeada em uma tabela e cada atributo persistente é mapeado em uma coluna.

ItemPatrimonial
- numeroPatrimonio : int - numeroSerie : int - descricao : String - valorAtual : float
+ ItemPatrimonial() + salvar() : void + setLocal(local : Local) : void + setLocalizacao(localizacao : Localizacao) : void + getHistorico() : ListItemHistorico + getNotificacao() : Notificacao

- Dependendo da linguagem de programação, os atributos persistentes devem adotar tipos que representam **classes** e **não tipos primitivos**.
- Por exemplo, no Java teríamos Integer, Long, Float, Character, Double e Boolean, ao invés de int, long, float char, double e boolean.
- A principal razão é a possibilidade de ter atributos **NULOS**, o que não é possível com tipos primitivos.

ItemPatrimonial	
- numeroPatrimonio : int Integer	
- numeroSerie : int Integer	
- descricao : String	
- valorAtual : float Float	
+ ItemPatrimonial()	
+ salvar() : void	
+ setLocal(local : Local) : void	
+ setLocalizacao(localizacao : Localizacao) : void	
+ getHistorico() : ListItemHistorico	
+ getNotificacao() : Notificacao	

- Chaves primárias (*Primary Keys* - *PK*) são conceitos do mundo relacional, mas não tem equivalente no mundo OO.
- Duas técnicas são normalmente usadas para representar PKs:
 1. Usar chaves naturais: chaves associadas a atributos de negócio.
 2. Usar chaves artificiais: adotar um mecanismo para gerar u valor artificial, ou seja, que não tem nenhuma relação com dados de negócio.

- Duas técnicas são normalmente usadas para representar PKs:
 1. Selecionar um atributo da classe que represente um **valor único** e usar esse atributo como **chave primária**.

ItemPatrimonial
- numeroPatrimonio : Integer - numeroSerie : Integer - descricao : String - valorAtual : Float
+ ItemPatrimonial() + salvar() : void + setLocal(local : Local) : void + setLocalizacao(localizacao : Localizacao) : void + getHistorico() : ListItemHistorico + getNotificacao() : Notificacao

ItemPatrimonial
P * numero_patrimonio INTEGER numero_serie INTEGER descricao VARCHAR2 (100) valor_atual NUMBER (11,2)
 ItemPatrimonial_PK (numero_patrimonio)

- Duas técnicas são normalmente usadas para representar PKs :
2. Criar uma **chave primária artificial** (*surrogate key*) e definir o atributo com valor único como uma **chave alternativa ou chave única** (*Alternate Key – AK ou Unique Key - UK*).

ItemPatrimonial
- id : Integer - numeroPatrimonio : Integer - numeroSerie : Integer - descricao : String - valorAtual : Float
+ ItemPatrimonial() + salvar() : void + setLocal(local : Local) : void + setLocalizacao(localizacao : Localizacao) : void + getHistorico() : ListItemHistorico + getNotificacao() : Notificacao

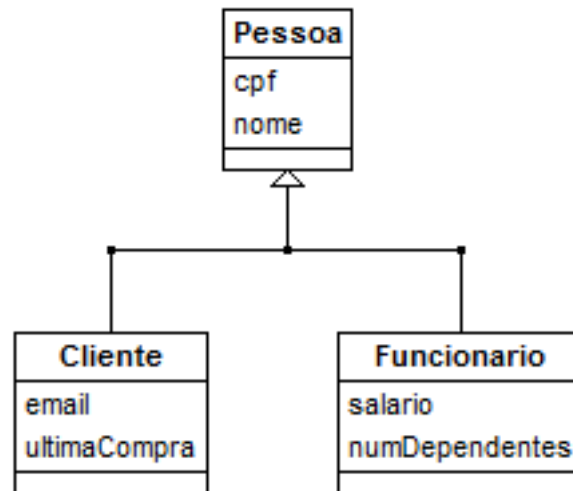
ItemPatrimonial
P * id INTEGER U * numero_patrimonio INTEGER * numero_serie INTEGER * descricao VARCHAR2 (100) * valor_atual NUMBER (11,2)
 ItemPatrimonial_PK (id)  ItemPatrimonial_AK (numero_patrimonio)

- Caso seja adotada a estratégia 2, a chave primária (ID) poderá ser gerada pelo próprio BD (coluna AUTO INCREMENT).
- Repare que, nesse caso, os objetos ainda **não persistidos** no BD teriam **ID = NULO**.
- Isso não é um problema, pois o atributo ID é usado somente pelo modelo relacional e não pelo modelo OO.

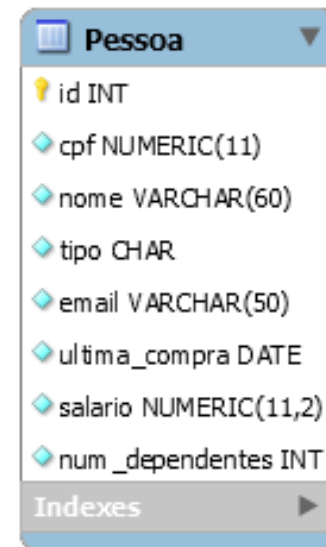
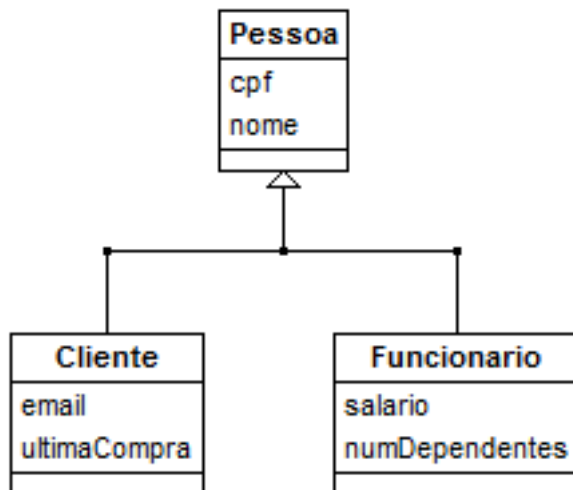
- Outra informação relevante para as classes persistentes é o indicador que define se o objeto foi ou não salvo no BD.
- Normalmente esse indicador é um atributo **booleano**.
- Qual a sua importância?
 - ✓ Ao salvar um objeto ainda não persistido (*indicador = false*), devemos executar um comando **INSERT** no BD.
 - ✓ A salvar um objeto já persistido (*indicador = true*), devemos executar um comando **UPDATE** no BD.
 - ✓ Quando criamos um objeto com dados vindos do usuário, então **indicador ← false**.
 - ✓ Quando criamos um objeto recuperando seus dados do BD, então **indicador ← true**.

2. Mapeamento de Hierarquias

- ✓ Existem 3 formas de mapear hierarquias de classes para tabelas relacionais:
 - a) Uma tabela para **toda** a estrutura hierárquica.
 - b) Uma tabela para cada **classe concreta**.
 - c) Uma tabela para **cada classe** da hierarquia.

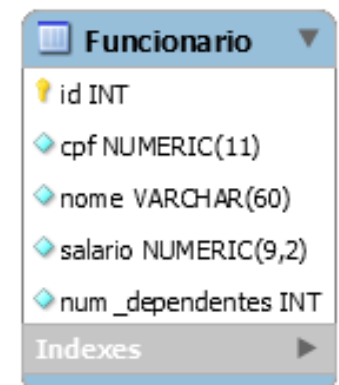
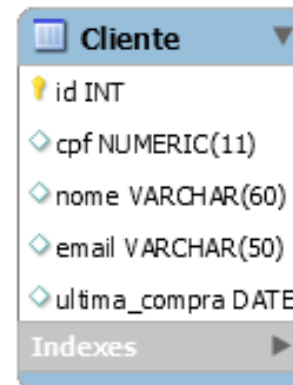
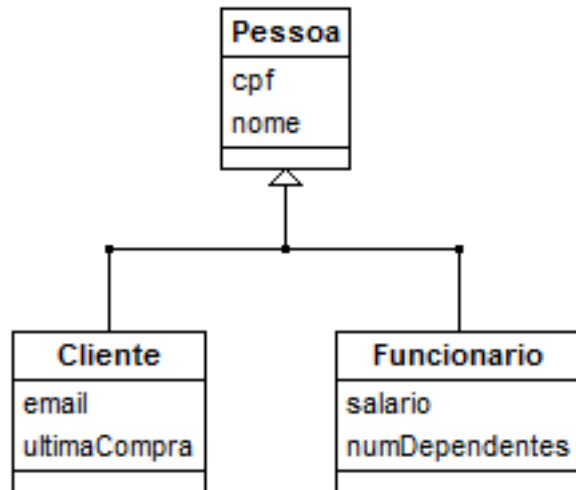


- a) Uma tabela para toda a estrutura hierárquica
- ✓ Gera tabelas desnormalizadas.
 - ✓ Geralmente é necessária a introdução de uma coluna que identifica o tipo do objeto.
 - ✓ Alguns relacionamentos do modelo OO podem não fazer sentido no modelo relacional.

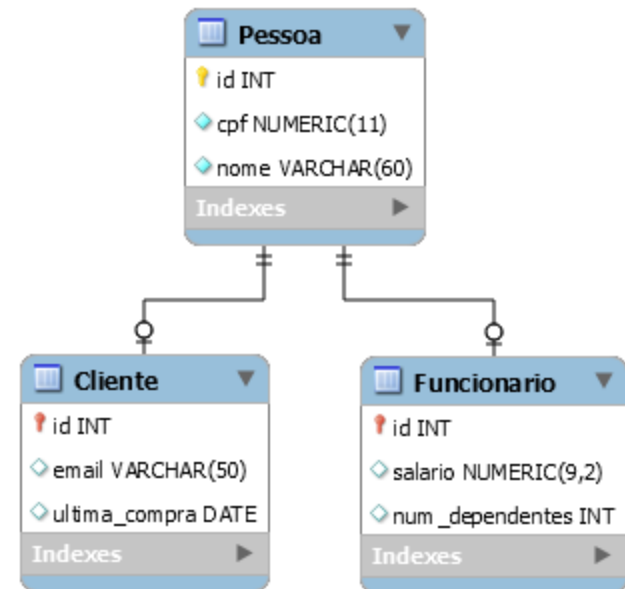
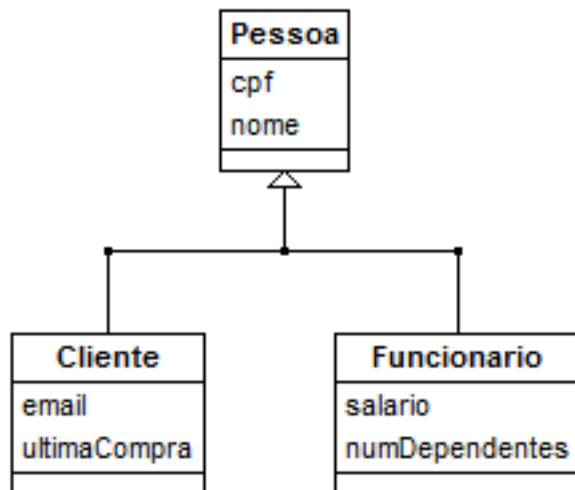


b) Uma tabela para cada classe concreta da hierarquia

- ✓ Deve ser montada uma estratégia para garantir que os IDs das tabelas não serão idênticos.
- ✓ Manutenção das colunas comuns pode ser problemática, principalmente se existirem muitas classes na base da hierarquia (replicação de esquema).
- ✓ Implementação de relacionamentos da classe mãe é muito problemático no modelo relacional.



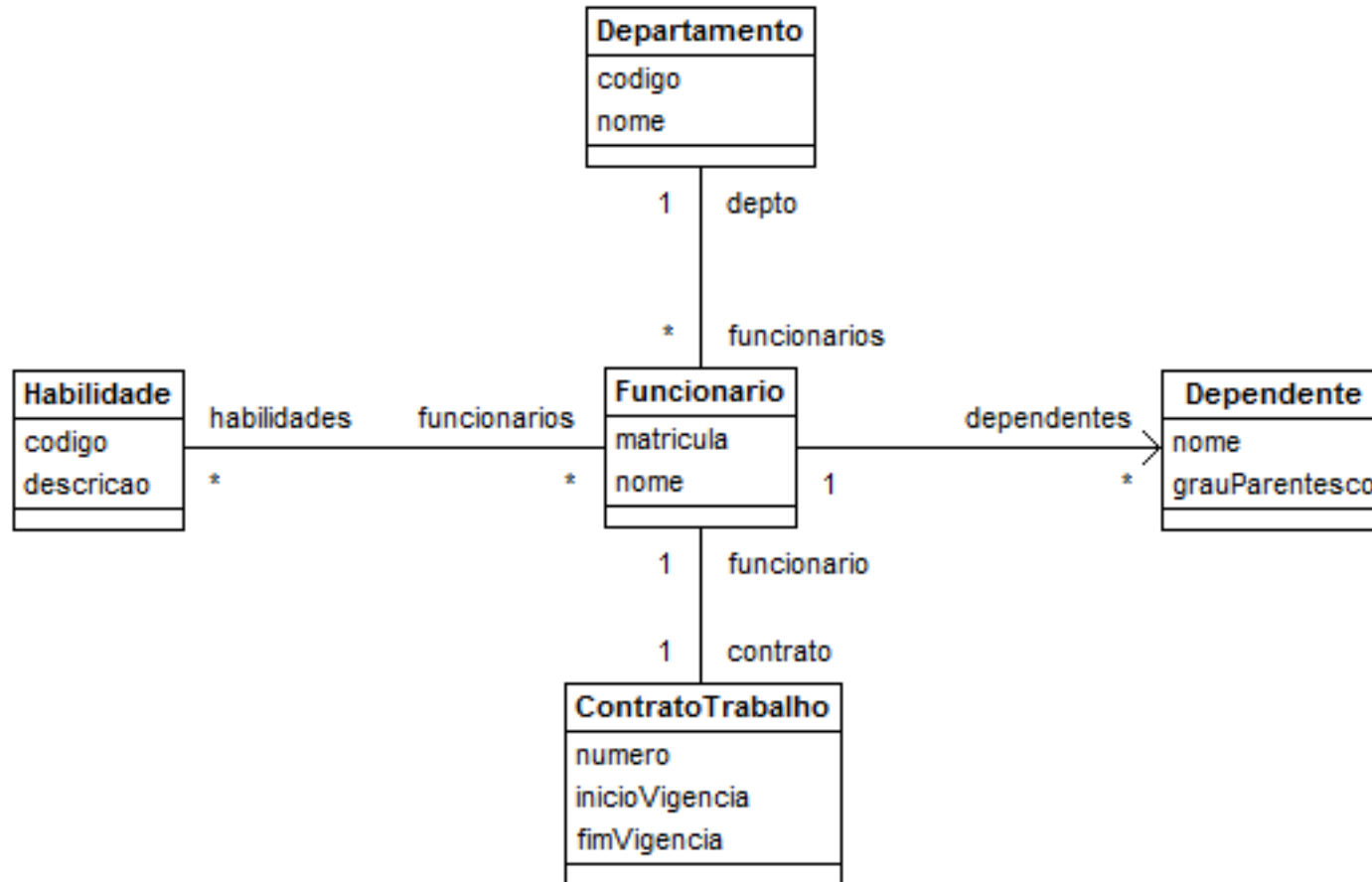
- c) Uma tabela para cada classe da hierarquia (classes com atributos)
- ✓ Devem ser criadas *constraints* no BD para garantir que somente **UMA** tabela filha estará relacionada com a tabela mãe.
 - ✓ A inclusão ou alteração de um objeto do modelo OO requer múltiplos comandos INSERT/UPDATE com controle de transação.
 - ✓ A recuperação de um objeto do modelo OO requer o *join* de pelo menos duas tabelas (em alguns SGBDs é possível trabalhar com o conceito de VIEW).
 - ✓ A exclusão de um objeto do modelo OO pode ser implementada usando DELETE CASCADE nos relacionamentos.



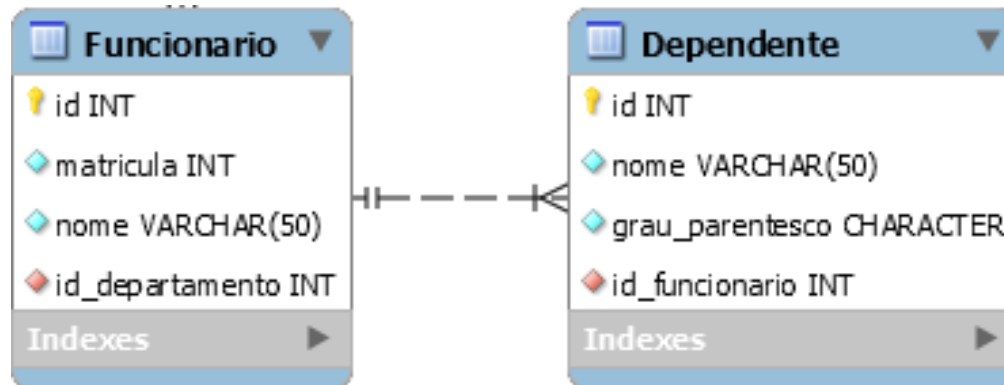
3. Mapeamento de Associações

- ✓ Existem **duas formas** de classificar relacionamentos em modelos OO:
 - a) Baseado na multiplicidade: 1:1, 1:N ou M:N
 - b) Baseado na direção: unidirecional ou bidirecional
- ✓ Nos modelos relacionais temos **apenas a classificação baseada na multiplicidade**, pois esses modelos só trabalham com relacionamentos **bidirecionais**.
- ✓ Uma característica dos relacionamentos do modelo relacional que não existe no modelo OO é:
 - A entidade origem do relacionamento **identifica** ou **não identifica** a entidade destino do relacionamento, ou seja, a chave primária da origem faz parte ou não da chave primária do destino.

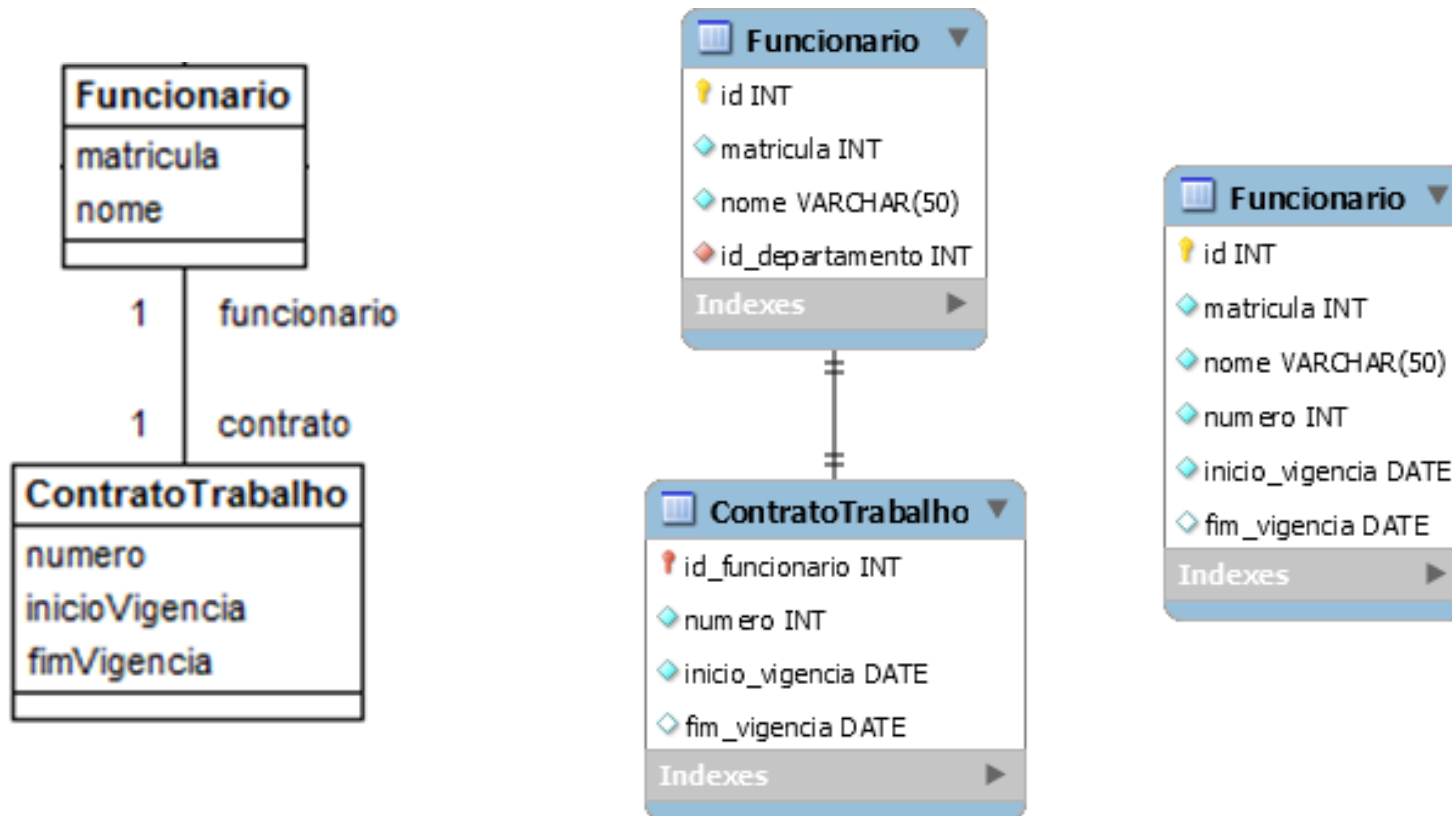
- Nesse modelo de classes temos vários tipos de Associações:



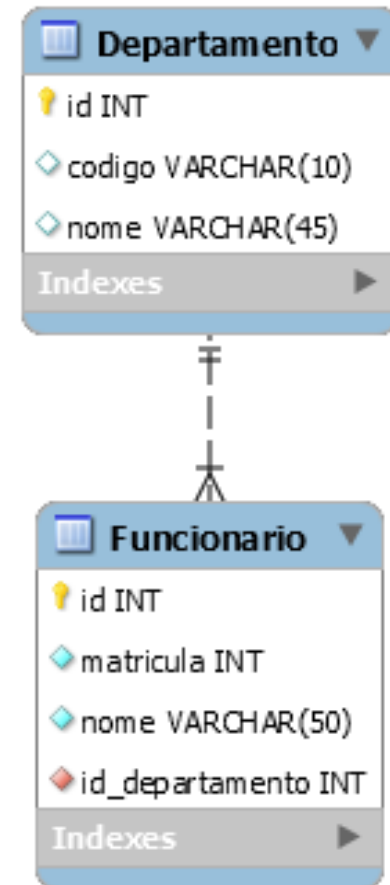
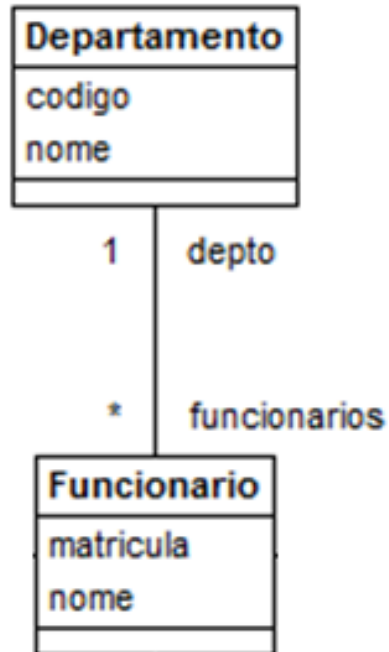
- Funcionário → Dependente (unidirecional):
 - ✓ Relacionamento 1:N
 - ✓ Funcionário não identifica Dependente: identificador do Funcionário se transforma em chave estrangeira (FK – Foreign Key) no Dependente.



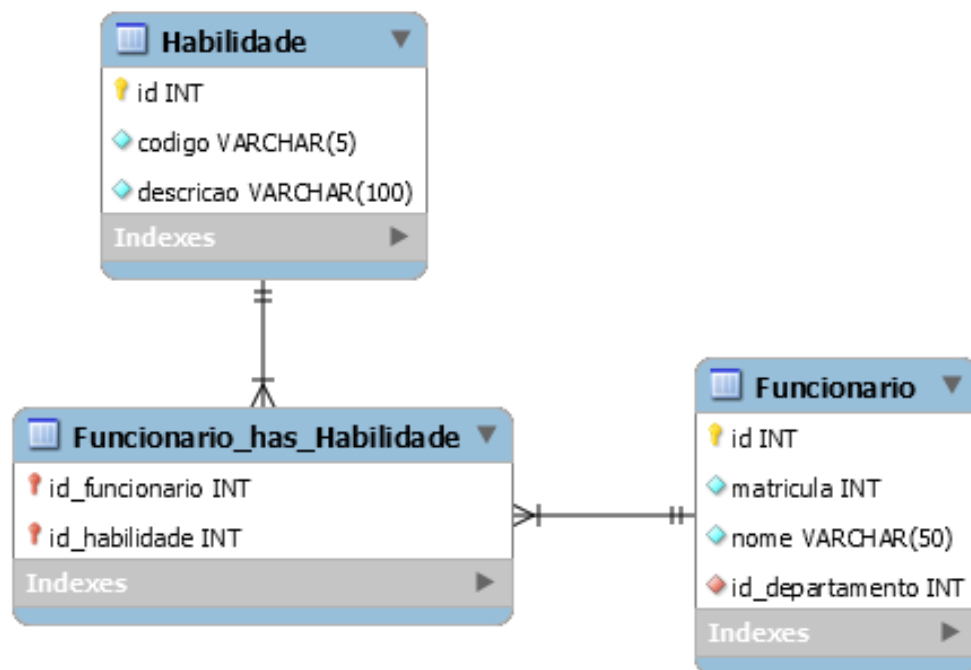
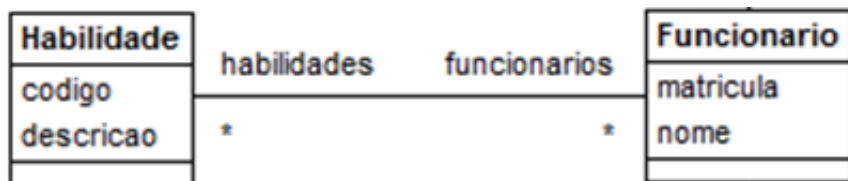
- Funcionário ↔ ContratoTrabalho (bidirecional):
 - ✓ Relacionamento 1:1
 - ✓ Funcionário identifica ContratoTrabalho: identificador do Funcionário faz parte da chave primária (PK – Primary Key) no ContratoTrabalho.



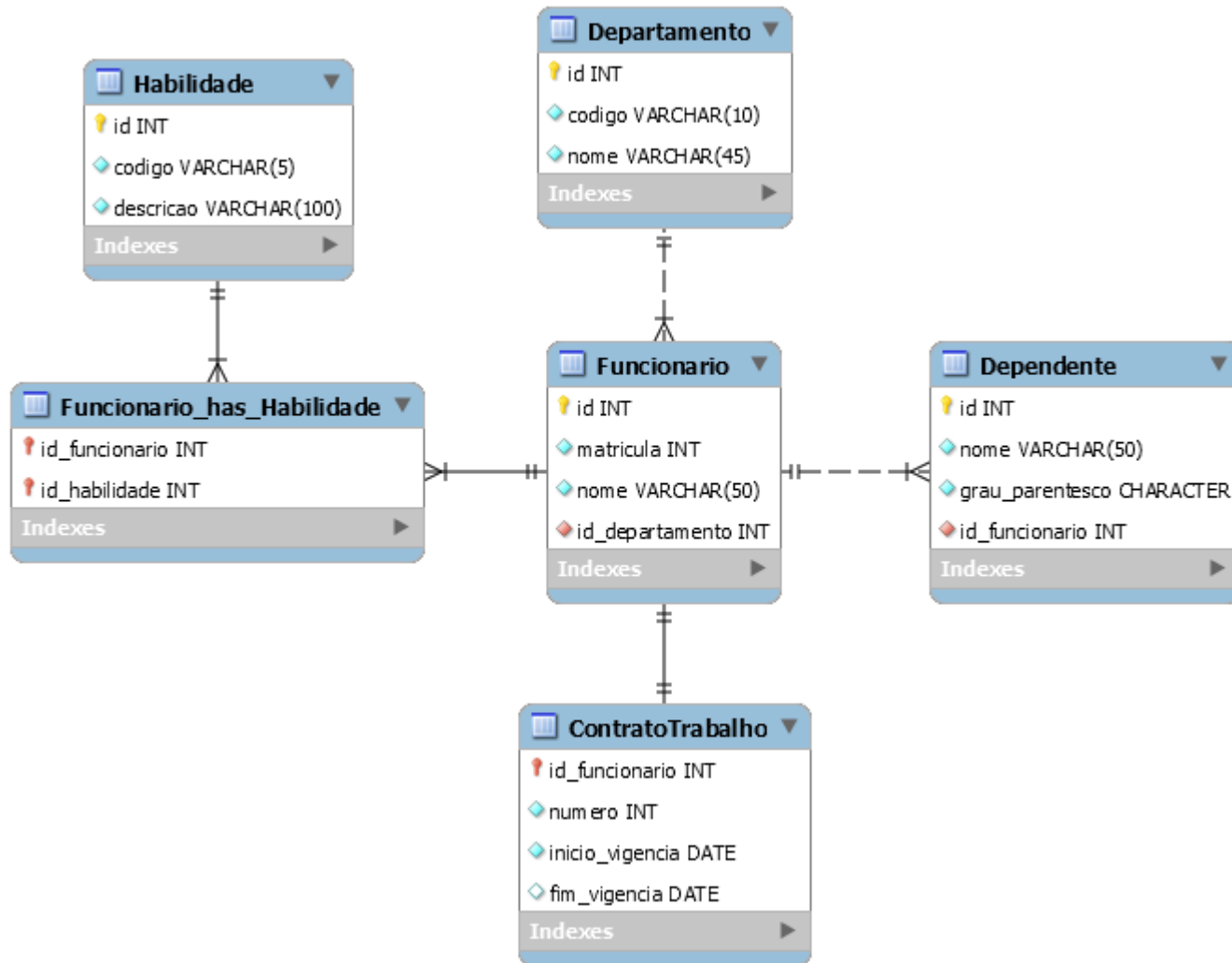
- Departamento ↔ Funcionário (bidirecional):
 - ✓ Relacionamento 1:N
 - ✓ Departamento não identifica Funcionário



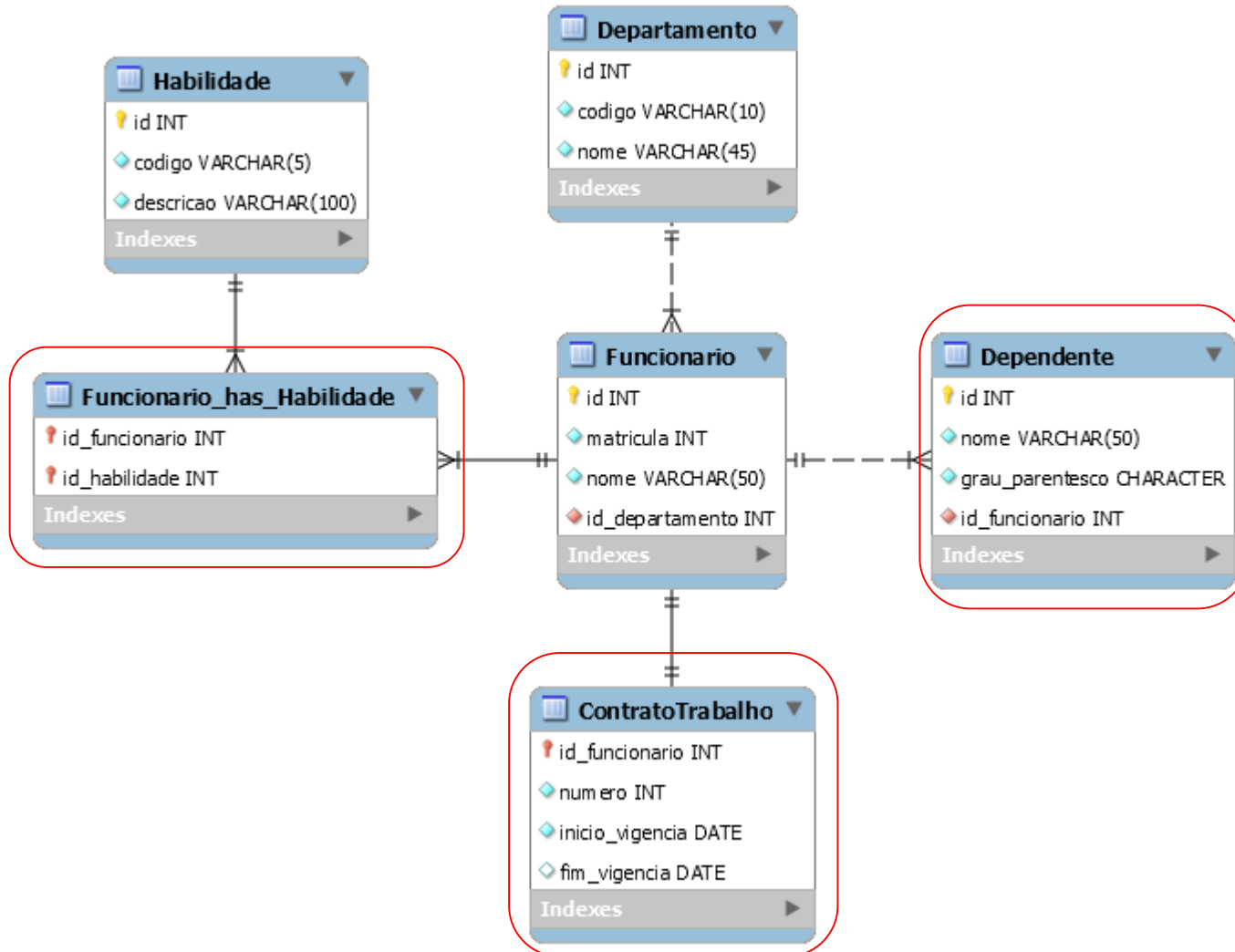
- Funcionário ↔ Habilidade (bidirecional):
 - ✓ Relacionamento N:M
 - ✓ Criação de tabela associativa com as chaves primárias de Funcionário e Habilidade.



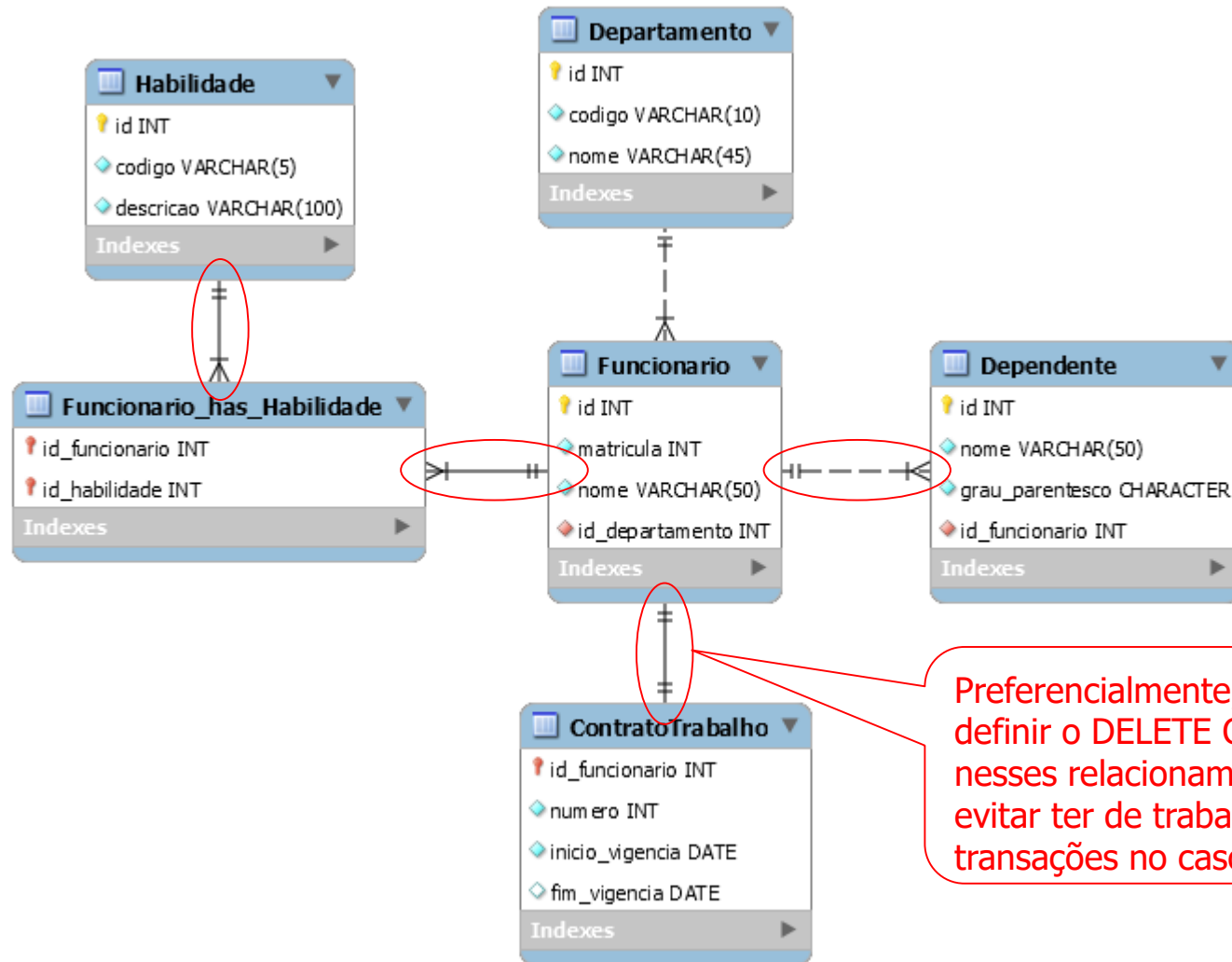
➤ Modelo ER:



- É importante, nesse modelo, analisar as **entidades fracas**:



- É importante, nesse modelo, analisar as **entidades fracas**:



4. Mapeamento de Agregações

- Segue os mesmos princípios do mapeamento das associações.

5. Mapeamento de Composições

- Segue os mesmos princípios do mapeamento das associações.
- O relacionamento entre a **entidade todo** e a **entidade parte** deve ter um **DELETE CASCADE**, pois como a classe parte não tem "vida própria", caso a classe todo seja excluída, as partes também devem ser excluídas.

6. Mapeamento de Atributos Estáticos

➤ Duas estratégias possíveis:

- a) Uma tabela para **cada classe** com **todos** os atributos estáticos da classe.
- b) Uma tabela **única** com **todos** os atributos estáticos de **todas** as classes do modelo OO. Essa é a estratégia mais usada.